SCALA 3 · OPENCV 4.13

scalacv

A typed, resource-safe OpenCV for the JVM — from a first pipeline to a production vision service



The scalacv authors

Covers scalacv 0.1.0 · OpenCV 4.13.0 · Scala 3.3 LTS

scalacv: Computer Vision in Scala 3

Copyright © 2026 the scalacv authors. All rights reserved.

scalacv is free software, published under the Apache License, Version 2.0. You may obtain a copy of the licence at apache.org/licenses/LICENSE-2.0. This book is documentation for that library and is distributed on the same terms, without warranties or conditions of any kind, either express or implied.

Written for scalacv 0.1.0, OpenCV 4.13.0, Scala 3.3 LTS, JDK 17+. Because the library is pre-1.0, a minor version bump may break compatibility; every code listing here was written against the version named above.

OpenCV is licensed under the Apache License 2.0 by OpenCV.org. The native binaries this library loads are the Bytedeco JavaCPP presets for OpenCV, also Apache-2.0. Neither project endorses this one. Scala is a trademark of the École Polytechnique Fédérale de Lausanne.

The cover mark is a six-bladed aperture — the camera diaphragm that decides, mechanically and irreversibly, how much light reaches the sensor. It was chosen for the same reason this book keeps returning to lifetimes: the interesting part of an imaging system is not what it computes but what it lets go of.

Set in Libertinus Serif, with Lato for display and DejaVu Sans Mono for code. Typeset with Typst.

For everyone who has watched a JVM process die at 6 GB resident
while the heap graph sat flat at 40 MB.

Table of Contents

Preface ..... **xx**
Who this book is for xx
Why this book exists xxi
How this book is organised xxi
Conventions used in this book xxii
Using the code examples xxiii
What you need installed xxiii
Acknowledgements xxiv
1 Why This Library Exists **5**
What the Java bindings actually are 5
The collector cannot see what it does not own 6
The types you cannot free at all 7
Why finalizers and cleaners do not save you 7
Four commitments follow from this 9
Typed everything 9
Explicit resource ownership 9
Genuinely headless 10
Errors as values where they belong, exceptions where they belong 10
What this library is not 11
The shape of it 11
Where this goes next 12
2 Setting Up **13**
The four artifacts 13
Why there are two dependency lines 14
The fat option, and what it costs 16
The build files 16
What `OpenCv.load()` actually does 18
The extraction cache 19
Hello, it works 21
Verifying the install 22
When it goes wrong anyway 23
Where this leaves you 23
3 Your First Pipeline **25**
A synchronous scoped entry point 26
What each operation does to the pixels 27
The boundary where failure becomes a value 28
A pipeline with no file on disk 30
Detect, then annotate 31
The mistake everybody makes once 32

Running it 33
Next: the rule underneath the chain 35
4 The Image Type 37
Three kinds of member 38
Move semantics, and what happens when you get them wrong 39
Pointing the error at the real mistake 40
Branching: `copy`, and the borrowed `mat` 41
Getting an image in the first place 42
The two escape hatches 44
Why `faces` is not a member 44
How an `Image` fails 45
Putting it together 47
5 Lifetimes: Managed, Releasable, and Scope 49
The failure this replaces 50
`Managed[A]` 50
Why a `Class`, and not the object 52
`take`, and why it is not yours to call 52
Several handles at once: `Managed.scope` 52
`adopt`, for values that arrive already wrapped 54
`Releasable[A]`: how a thing gets freed 54
What `nativeHandle` actually does 57
The finalizer, and the double free it would cause 58
When reflection is refused 59
Finding the line that spent a handle 60
Which level to reach for 61
What getting it wrong looks like from outside 62
Next 62
6 The Error Model 65
The dividing line 65
Six cases 66
NativesMissing 67
DecodeFailed 67
LoadFailed 67
EncodeFailed 68
CalibrationFailed 68
NativeCall 68
Cv.attempt, and the guard that makes the policy real 69
The failures that throw 70
IllegalArgumentException: what the library can see coming 70
IllegalStateException: the handle you already spent 70
Composing 71
Error messages as a feature 73
Three things not to do 75

Next 75
7 Reading, Writing, and Interop 79
Four functions, symmetric in pairs 80
Why a missing file is a `Left` and not an empty `Mat` 80
Who frees the Mat 81
`read`, `reading`, and the leak between them 82
Deciding what comes out of the decoder 83
Formats, and what the extension decides 84
The round trip that never touches a disk 85
Crossing into AWT 86
BGR, and the picture that comes out wrong 88
Paths, and what a path is not 89
What comes next 89
8 Typed Constants and Colour Spaces 91
The mechanism 91
The families 92
Where the enum is the wrong shape 93
Threshold is a bitmask 94
ImreadFlags only looks like a bitmask 94
BorderType is one type over two domains 94
Colour, and the space you reason in 96
HSV separates the question 96
Hue is 0 to 179, not 0 to 359 97
The other spaces 98
When the enum you need is missing 98
The value types 99
Where this goes next 100
9 Filtering and Morphology 101
A kernel, slid over the image 101
Blurring, and choosing which blur 102
Median, and why the scan needs it 103
Bilateral, the expensive one 104
The scan, first attempt 104
Edges: derivatives, and the two-threshold trick 106
Canny, and what the two thresholds actually do 107
Sharpening 107
Morphology: the operators for a binary mask 108
What happens at the edge of the image 110
What filtering costs 111
Where this leads 112
10 Geometric Transforms 113
Resize: two modes, and why they are two methods 113

Choosing an interpolation 115
Crop, and the check OpenCV does not perform 116
The exact turns 117
Arbitrary rotation, and the canvas that has to grow 117
Border modes: what fills the corners 119
Reading a 2×3 affine 120
Three points or four: affine versus perspective 121
Flattening the docket 122
The ordering trap: detect small, draw large 124
Where this leaves the docket 126
11 Thresholding and Colour Segmentation 127
One number for the whole image 127
The Threshold model 128
Letting the histogram choose: Otsu 129
Giving up on a single level: adaptive thresholding 130
Thresholding the right axis: colour in HSV 131
Red, and the seam in the hue wheel 132
Cleaning the mask 133
The tracker, end to end 134
Choosing the bounds 136
What the mask is for 137
12 Contours and Shape Analysis 139
Getting a binary image to trace 140
Finding the contours 140
Retrieval: which outlines you get 141
Approximation: how the outline is stored 142
What a Contour carries 142
Measuring: four numbers and four ratios 144
Classifying by vertex count 145
Filtering: the highest-value line in the pipeline 146
Fitting a shape that is not upright 147
Drawing the result back 148
The whole pass 148
Where this goes next 150
13 Finding Lines and Circles 151
Voting instead of fitting 151
What the transforms will accept 152
Segments, lines, and votes 153
Polar form, and how to draw it 154
Ranking by votes 155
Tuning, as a procedure 156
Circles, and the escape hatch 157

Worked example: the skew of a page 158
When Hough is the wrong tool 160
Where the angle goes next 160
14 Drawing and Annotation 161
Drawing mutates, and says so 162
The primitives 163
Colour is a Scalar, and a Scalar is BGR 164
Text, and the plate that makes it readable 165
Two tiers, and who owns the Mat 167
Keeping the original 168
Overlays that know what they are drawing 169
Where the primitives stop 169
Next 170
15 Photographic and Stylisation Operations 171
Tone and colour 172
Colour maps, and why the default matters 173
Detail and texture 174
Repair and composition 175
What these cost 177
A look you can repeat 177
Where this goes next 179
16 The Picture Scene Graph 183
A module of its own 183
The frame you cannot annotate twice 184
A picture is a value 185
The primitives 186
Styling, and the rule that surprises people 187
Dashed strokes 188
Transforms and layout 189
Colour: RGBA here, BGR underneath 191
Compositing onto a frame 192
Testing a picture without rendering it 193
When to stay with the drawing verbs 194
Next 195
17 Charts 197
Six factories, one return type 197
Counting faces across a clip 199
A chart is a picture, so it composites like one 200
What Chart does not draw 202
The magnitude fold 203
Design that survives the medium 203

Where this goes next 205
18 Animation and GIF 207
A drawing that takes the frame number 207
Holding every frame, and not holding it 210
What the GIF format charges you 211
Frame rate, and why the useful band is narrow 212
From processed video to a GIF 213
When the encode fails 214
Next 215
19 Video I/O 219
Opening a source 219
What the file claims about itself 221
Walking the frames 221
The borrowing contract 222
Seeking, frame index, and wall-clock time 225
Writing: the Recorder 225
FourCC, containers, and the zero-byte file 227
The worked example: an annotated re-encode 228
Performance: decode, resize, skip 230
Where this goes next 231
20 Cameras and Recording 233
Opening a source 233
Everything the source tells you is a guess 234
Warm-up, and why your first frame is black 235
Asking for a resolution 235
The frame loop 236
Batches 237
Recording, and the size you do not know yet 237
Codecs, and the container that comes with them 238
write borrows, and at 30 fps that is the whole game 238
The frame you read is not the frame that is happening 239
When the camera goes away 240
Network cameras 241
The whole loop 243
What the frames are worth looking at for 244
21 Motion Detection 245
What "moved" means in pixels 245
The detector 246
Frame difference and its knobs 248
A background model instead of a memory of one frame 248
Tuning as a procedure 250

The failure modes you will actually hit ... 251
Motion as a trigger ... 252
The watcher ... 253
Where this goes next ... 255
22 Streaming and Backpressure ... 257
The arithmetic that decides everything ... 258
Measure the lag before you tune anything ... 258
Why the mean is the wrong number to watch ... 260
Drop the oldest, not the newest ... 261
Who frees the frame that was dropped ... 262
Three shapes that work ... 263
The single-consumer loop ... 263
A fixed worker pool ... 264
Two stages: a cheap gate before an expensive detector ... 265
Stopping ... 266
What comes next ... 267
23 Models: Fetching, Caching, Verifying ... 271
Three payloads, and only one is a download ... 272
A model is a spec, not a URL ... 272
What `fetch` actually does ... 273
The two specs that ship ... 274
What the library knows how to decode ... 275
Everything fails as `LoadFailed` ... 276
`file://` is not a special case ... 277
Pin the model, or pin nothing at all ... 278
Licences travel with weights, not with the library ... 279
The triage service, at boot ... 279
What this bought ... 280
24 Face Detection ... 281
What Viola-Jones actually does ... 281
Loading a cascade without lying to yourself ... 282
Running the cascade ... 284
YuNet: the detector to actually use ... 285
The Face record ... 286
faces and markFaces ... 287
Choosing between them ... 288
Build once, detect many ... 289
What actually breaks these detectors ... 291
Where this goes next ... 291
25 Face Recognition ... 293
The embedding, and what it is made of ... 293

Two models, and neither ships with the library 294
From a Face to a vector 295
Why you pass the frame and not a crop 295
Comparing two embeddings 297
Thresholds, and the trade you are actually making 297
The Gallery 298
A gallery is an index, not your store of record 299
Enrolment: several shots, and what to do with them 299
The worked example 300
What a score is not 302
Where this leaves you 303
26 Deep Networks and ONNX 305
Inference only, and only ONNX 306
Three functions and one owner 306
Loading the model 307
The blob, and the four numbers 308
The checklist 310
The forward pass 311
Backends and targets 312
Decoding: from a tensor to an answer 313
A classifier: one row of scores 313
A detector: many rows, most of them duplicates 314
More than one output 315
Batching, and why it rarely pays here 315
The feeder, end to end 316
Next 317
27 Object Detection 319
The words on the model card 319
What the library owns, and what it does not 321
Choosing a model, and being honest about the cost 322
Letterbox, and the two coordinate systems that are not the same 323
Decode once, and filter while you decode 325
Two thresholds, chosen from frames rather than from feel 327
The whole loop 328
From boxes to tracks 330
28 QR Codes, ArUco, and Augmented Reality 333
Two printed markers, two different jobs 334
Reading the sticker 334
How large does the sticker have to be? 335
ArUco: identity you can measure 336
Printing the tags 337
From four corners to a pose 338

What `rvec` actually is 339
The camera model is the part you have to earn 340
Standing a cube on the tray 340
Projecting your own geometry 341
When the cube wobbles 342
The bench loop 342
Where the numbers come from 343
29 Pose Estimation and Gestures 345
A skeleton is a graph with names 345
The two topologies that ship 347
Two output layouts, one decode 347
Choosing between them 349
From frame to skeleton 349
Drawing it 350
Head pose, and six matrices at once 351
What the call actually owns 351
Yaw, pitch and roll, and the sign that flips 353
Gestures: geometry, not weights 353
A gesture is a trajectory; this layer sees a frame 354
Making it steady enough to act on 354
The session monitor 356
Where the numbers come from 357
30 Tracking 359
The Kalman filter, without the matrices 360
ObjectTracker: the tracking-by-detection loop 362
Tuning, and where it goes wrong 363
Counting across a line 364
Following one object without a detector 366
Detector, tracker, overlay 368
Where this goes next 369
31 Camera Calibration 371
The camera as a function 371
What the lens does to a straight line 372
The `Intrinsics` type 373
The distortion vector has legal lengths 374
Describing the board 375
Finding the board in a frame 376
Shooting a set worth solving 377
Solving 378
Reading the reprojection error 379
When it fails 380
Straightening the frame 380

All the pixels, or only the valid ones ... 381
Where the numbers live afterwards ... 381
What the bench rig gets ... 382
32 Visual Navigation ... 385
The line between front end and back end ... 385
Optical flow: following what you already found ... 386
The mistake: trusting a lost point ... 387
The forward--backward check ... 387
Features: recognising a place you have left ... 388
Matching, and what is thrown away ... 389
Motion between two frames ... 390
The odometry loop ... 391
Depth from a stereo pair ... 391
From a depth map to obstacles, and from obstacles to a decision ... 392
Absolute pose: where am I, really ... 393
Mapping: the grid and the loop ... 394
The occupancy grid ... 394
Loop closure ... 395
The front end, assembled ... 397
A trajectory from a video ... 398
What is honestly here ... 399
Next ... 400
33 OCR Preprocessing ... 401
The seam ... 401
What `forOcr` does, one step at a time ... 403
Deskew, and what two degrees costs ... 405
Cropping to the text ... 407
Resolution, and the order of operations ... 408
Tables, columns, and text that is not level ... 408
Bringing your own engine ... 410
The receipt, end to end ... 411
Where this goes next ... 412
34 Screen Analysis and Video Conferencing ... 413
Part one: what is on the screen ... 414
Template matching is correlation, and only correlation ... 415
Every occurrence, and the hole each one leaves behind ... 416
Scale and rotation: the limit you must design around ... 416
Anti-aliasing, subpixel text, and where minScore should sit ... 417
What changed since the last capture ... 418
Part two: the person and everything else ... 420
Inside the blend, because it explains both knobs ... 420
From tensor to mask: Segmenter ... 421
The four things that look wrong ... 422

The latency budget ... 423
A camera to a recorder, correctly scoped ... 424
What these two hand to Part VI ... 425
35 Performance ... 429
Two budgets, two instruments ... 429
Measure before you change anything ... 430
What the benchmarks actually measured ... 431
Where the time goes ... 433
Resolution, which is quadratic ... 433
Colour conversion, kernels, and copies ... 433
Resize early ... 434
Hoist the natives out of the loop ... 435
What the JVM contributes, and what it cannot fix ... 436
The first hundred frames lie ... 436
`-Xmx` does not bound your footprint ... 437
GC tuning does not help native pressure ... 437
Native threading fights your thread pool ... 437
A profiling recipe ... 438
Next ... 438
36 Concurrency and Thread Safety ... 439
The rule in one line ... 439
What `Managed` actually promises ... 441
A spent `Image` is spent everywhere ... 442
Detectors, nets and matchers: one per worker ... 443
OpenCV is already using your cores ... 446
Virtual threads change the scheduler, not the boundary ... 447
The mistakes, collected ... 448
Where this leaves you ... 448
37 ZIO Integration ... 451
The module ... 452
Scope, which is `Managed.scope` with a cancellation story ... 453
The error channel: `CvError` typed, programmer errors as defects ... 454
Frames as a stream, with Chapter 19's borrowing contract intact ... 455
Backpressure you can actually have ... 457
A detector per fiber is not free ... 458
A complete application ... 459
Lifting the rest yourself ... 461
Where this goes next ... 462
38 Observability ... 463
The signals that are worth a panel ... 463
Watching native memory, which is the whole ball game ... 465
The one diagnostic flag the library actually has ... 468

Logging: almost nothing, once per frame 468
Sampling: one frame in N, and the only debugger you get 469
Tracing one frame from sensor to answer 470
Alert on the derivative, not on the level 471
Where this goes next 472
39 Degradation and Error Budgets 473
Sort failures by what will catch them 473
The ladder, rung by rung 475
A ladder you can read 476
Budgets: the numbers that make a rung acceptable 479
Timeouts, and the thing you cannot have 480
Circuit-breaking a detector 481
Bulkheads 482
When the process dies anyway 483
What comes next 484
40 Testing 485
Run the module that owns the suite 485
Draw the fixture, do not ship it 486
Compare with a tolerance, never byte for byte 487
Golden images, and the trap in them 488
Properties, where the law is the point 489
Testing lifetimes 490
The leaks module 491
Isolation: one load, one JVM, one scope 493
What this project's CI actually runs 493
Snippets that cannot rot 495
Skip what the runner cannot provide 495
Where this leaves you 496
41 Deploying to Production 497
Package for one platform, not for all of them 497
The cache that is not in your image 499
A Dockerfile, line by line 500
Budgeting memory when the heap is not the footprint 501
Models: in the image, on a volume, or nowhere at all 503
Fail at boot, not on the first request 504
Kubernetes: requests, limits, and the pool you did not configure 505
Rolling upgrades, and models that do not collide 506
When it still goes wrong 507
42 Troubleshooting 509
Start-up and the build 510
The natives are not there 510

The build resolves but nothing runs ... 511
Memory and lifetimes ... 511
RSS climbs, heap does not ... 511
A spent handle ... 512
A hard crash ... 513
The module system said no ... 514
Wrong output ... 515
The image will not read ... 515
The recording is empty ... 515
The colours are swapped ... 516
The detector finds nothing ... 516
Results differ between machines ... 517
Performance and hardware ... 517
The first frames are slow ... 517
Everything is slow on a big image ... 518
The camera opens but delivers nothing ... 518
Calibration will not converge ... 519
Filing a bug that can be fixed ... 519
Where the reference material lives ... 520
43 Dropping to OpenCV Java ... 523
Three altitudes ... 523
Going down: `mat` borrows, `managed` hands over ... 526
The mid level, and the contract it writes down ... 527
Coming back up ... 529
The full recipe for an unwrapped call ... 529
Reading the C++ documentation, landing in Java ... 532
Mat basics, for when you must ... 533
Where the Java bindings are wrong, and what the library does about it ... 534
When to stay high ... 536
44 Operations Reference ... 537
What each group needs on the classpath ... 538
How to read a row ... 538
Constructing and destroying an image ... 539
Queries ... 540
Colour and channels ... 541
Filtering and smoothing ... 542
Edges and gradients ... 543
Morphology ... 543
Thresholding and masks ... 544
Geometry: resize, crop, rotate, warp ... 545
Photographic and stylisation ... 546
Drawing ... 547
Contours and shape measurement ... 548

Hough ... 549
I/O and encoding ... 550
Interop ... 551
Where to go from a row ... 552
45 Enums and Types Reference ... 553
The core enums ... 553
Colour conversion ... 553
Resizing, borders and mirroring ... 554
Morphology ... 556
Thresholding ... 556
Contours ... 557
Drawing ... 558
False colour ... 558
Reading an image ... 559
Enums that live beside their operations ... 559
The geometry value types ... 561
Result types ... 562
When the case you need is missing ... 564
Next ... 565
46 Notebooks and Sample Data ... 567
Almond: scalacv in a Jupyter cell ... 567
Flags to start the kernel with ... 569
Why the display method is toBufferedImage ... 569
The leak a notebook makes easy ... 570
scala-cli, when a notebook is more machinery than you need ... 571
Sample data you draw rather than download ... 572
A scene with countable objects ... 572
A scene composed rather than chained ... 573
A chessboard for calibration ... 573
An ArUco marker sheet ... 574
A document for OCR ... 574
A moving object for tracking and motion detection ... 575
A gradient and a noise field ... 576
The inputs you have to supply yourself ... 577
Real datasets, and the licence attached to each of them ... 577
What good output looks like ... 578
Where this leaves you ... 578
47 Glossary ... 579
A ... 580
B ... 580
C ... 581
D ... 582
E ... 582

F 583
G 583
H 583
I 584
J 584
K 585
L 585
M 585
N 586
O 586
P 586
Q 587
R 587
S 588
T 588
V 588
Y 589
Z 589
Where to look next 589

Preface

Computer vision on the JVM starts out easy, and then quite suddenly is not. Reading a JPEG, converting it to grey, running an edge detector and writing the result back to disk is a dozen lines against OpenCV's Java API, and it works the first time. The trouble arrives somewhere around the two-thousandth image, when a service that has sat quietly inside a modest heap for a week is killed by the kernel for holding six gigabytes of resident memory — and the heap dump, the first place anyone looks, shows nothing wrong at all.

The reason is that OpenCV's central type does not live on the heap. A `Mat` is roughly forty bytes of Java object standing in front of a pixel buffer the native library allocated somewhere the garbage collector cannot see. The collector schedules its work by heap pressure, and heap pressure is the wrong signal: forty bytes per megabyte will not provoke a collection, so a frame loop can exhaust the machine while the collector, correct by its own lights, never runs. Measured on this project's test machine: 2000 allocations of `Mat(1000, 1000, CV_8UC3)`, references dropped, no explicit `System.gc()`, finished at 5,865 MB resident. The same program calling `release()` finished at 144 MB. Forty-one times, on JDK 21 and again on JDK 25.

The OpenCV documentation will not help you, because outside the JVM the problem does not exist: C++ destroys a `Mat` when it leaves scope, and Python's bindings reference-count. It is a JVM problem, and it has two siblings the upstream docs are equally silent about — an API where every option is an untyped `int` constant, so a transposed digit compiles and quietly produces the wrong picture, and a native distribution whose obvious load path initialises a GTK2-linked `highgui` and, on a machine with no desktop, takes `objdetect` down with it.

`scalacv` takes those three problems as its subject: a Scala 3 API over the OpenCV 4.13 Java bindings, typed where the bindings are untyped, scoped where they leak, headless by construction. This is also a book about computer vision, because a wrapper is only worth reading about if it takes you somewhere — so by the end you will have built edge detectors, contour counters, face detectors, a motion alarm, a calibrated camera, and a service that runs all of it without falling over.

Who this book is for

You are a Scala 3 developer who needs to do something with images or video and would rather not spend a fortnight learning the C++ conventions of a library accreting since 2000. Or you are a JVM engineer who has already been burned by native memory — a `Mat`, a `ByteBuffer`, a JNI handle, something — and you want to know exactly who frees what before any of it goes near production.

Either way, this book assumes you read Scala 3 without effort: a case class, an extension method, a `for` comprehension over `Either`, a `given`. Nothing needs macros or type-level programming, though there is a ZIO layer for people who already run one. No OpenCV knowledge is assumed — kernels, colour spaces, homographies and camera intrinsics arrive

at the point of use, with the least theory that makes the parameters mean something, and the arithmetic appears where a formula is the difference between choosing a threshold and guessing one.

Two things the book does not do. It does not teach you to train neural networks — Part V runs an ONNX model through OpenCV's DNN module, it does not produce one. And it does not present the library as a Java API: `scalacv` returns `Seq`, `Option` and `Either` through extension methods brought in by `import scalacv.*`, and is not designed to be called from Java.

Why this book exists

OpenCV's Java bindings are generated — a faithful, mechanical translation of a C++ header set, and that faithfulness is what makes them awkward from Scala. The names keep C++ capitalisation, so `Imgproc.GaussianBlur`, `Imgproc.Canny`, `Imgproc.Sobel` and `Imgproc.Laplacian` sit in otherwise lower-camel code. `Imgcodecs.imread` reports a missing file by returning an empty `Mat` rather than throwing, so the failure surfaces three calls later. `Imgproc.HoughLinesP` hands back `CV_32SC4` integers where you expect floats. Each is defensible as a translation and indefensible as an API you have to remember.

The lifetime story is worse, and it is why the library exists at all. Of the 188 `org.opencv.*` types that own native memory, exactly three expose a public `release()`. `CascadeClassifier`, `Net`, `QRCodeDetector`, `ArucoDetector` and 181 others give you no way to free them from Java — there is no method to forget to call, which is a worse position than having one. Measured the same way as the `Mat` loop: 4000 leaked `KalmanFilter`s reached 54 GB of resident memory, against 86 MB when `scalacv` freed them.

> NATIVE MEMORY
> The book's signature callout. It marks the places where native lifetime is at stake: who owns a handle, when it is freed, and what happens if you touch it afterwards. Getting that wrong costs gigabytes of resident memory, or a SIGSEGV with no Scala stack trace. It is never a matter of style.

None of this is in the OpenCV documentation, which is written for C++ and adapted for Python. The answers are scattered across mailing lists, bug reports and the bindings' own source; this book collects them.

How this book is organised

Seven parts, in the order you would meet the material building something real.

- **Part I, Foundations** — why the bindings need wrapping, the dependency block and why the natives arrive as lines of their own, a first pipeline, the `Image` type, the ownership model, and the error policy that decides what returns `Either[CvError, A]` and what throws.

- **Part II, Working with Pixels** — reading and writing, the typed enums that replace the `int` constants, filters and morphology, geometric transforms, colour segmentation, contours, Hough, drawing, photo transforms.
- **Part III, The Graphics Layer** — the `Picture` scene graph in `scalacv-graphs`: primitives, layout, affine transforms, dashed strokes, the `Color` palette, `Chart`, and `Animation` with its GIF encoder, composited onto an image by `draw`.
- **Part IV, Video and Cameras** — the borrowing `Video.frames` path and the copy-or-borrow decision that separates a convenient frame loop from a fast one, `Camera` and `Recorder` on a source that never ends, motion detection, and what to do when the pipeline falls behind.
- **Part V, Vision Applications** — where models come from, faces and recognition, ONNX inference, object detection, ArUco markers, pose, tracking, calibration, visual navigation, OCR preprocessing, screen capture and conferencing.
- **Part VI, Into Production** — where the time and the memory go, concurrency, the ZIO layer, observability, degradation, testing in CI with no display server, deployment, troubleshooting.
- **Part VII, Appendices** — the escape hatch to raw OpenCV, the operation and enum tables, notebook display, and the glossary.

Read Parts I and II in order; after that the parts stand alone, and a chapter that leans on an earlier one says so in its first paragraph.

Conventions used in this book

Every type, method, parameter, enum case, flag and path is set in code, so `Image.reading` and `ColorConversion.BgrToGray` stay unambiguous mid-sentence. Listings the prose refers back to are numbered per chapter — `Example 4-2` is the second listing in Chapter 4 — and so are tables, as `Table 4-1`. Five callouts appear, and they mean different things. A sixth device, the boxed sidebar, carries a digression that would derail a paragraph — there is one below.

Table P-1. The callout vocabulary, in ascending order of how much attention it wants.

Callout	What it means
Note	An aside. Skip it and lose nothing but context.
Tip	A shortcut, an idiom, or a better default than the obvious one.
Warning	Something that will bite: wrong results, a build that will not finish.
Caution	Something that costs data or money if you get it wrong.
Native memory	A native lifetime hazard — ownership, release, use-after-release.

The prose spells British, as the library's own documentation does: **colour**, **grey**, **behaviour**. The API does not, because OpenCV does not — the text speaks of a colour conversion and then calls `ColorConversion.BgrToGray`, of a greyscale image whose method is `gray`. An identifier is always spelled the way the compiler spells it.

Using the code examples

The listings are Apache-2.0 licensed, like the library. Use them in your own programs freely; attribution is welcome and not required. They are trimmed for the page: unless a chapter says otherwise, assume that `import scalacv.*` heads the file and one `OpenCv.load()` has already run — both shown in full in Chapter 2, implicit thereafter.

The canonical versions live in the repository, and they are canonical because they are compiled. Every snippet under `docs/mdoc/` tagged `mdoc` is compiled against the real library — natives on the classpath, not a stub — and most are executed as well, so a drifted snippet fails the build instead of misleading a reader; the programs in `examples/` are compiled by Mill on every change, and `./mill examples.runMain scalacv.smoke` proves the natives loaded on your machine. Where the book and the repository disagree, the repository is right.

What you need installed

A JDK, 17 or newer. A build tool: this project uses Mill 1.1.7, which fetches its own launcher and provisions its own JDK, but the library is an ordinary published artifact, so sbt works as well — as does any Coursier-backed tool, `scala-cli` included, that can express a dependency classifier. Nothing else: no system OpenCV, no `apt-get` step, no display server, no GUI toolkit. `OpenCv.load()` is headless, and so is every test in the library's own suite.

You add two kinds of dependency line: the library, and the natives for your platform.

Example P-1. The minimum, in Mill: the core library plus natives for one platform.

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
  mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
)
```

Both native lines are needed: `libopencv_core` links against `libopenblas`, so dropping the second leaves you a half-loaded library rather than an honest failure. Chapter 2 has the classifier for every platform, the sbt spelling, the optional `scalacv-vision`, `scalacv-graphs` and `scalacv-zio` modules, and what lands on disk the first time you run: roughly 196 MB on Linux, extracted once into `~/.javacpp` and reused forever after, relocatable with `-Dorg.bytedeco.javacpp.cachedir`.

Why the natives are a separate line

It looks like an oversight, and it is not. The `scalacv` artifact depends on the OpenCV **Java API** jar, which contains no native code; the machine code ships in per-platform classifier jars, and no build tool can put a classifier into a published POM. A library that picked one for you would be picking wrong for everyone on a different machine.

There is an escape hatch: `org.bytedeco:opencv-platform:4.13.0-1.5.13` bundles every platform and works anywhere, for about 408 MB instead of the 36–80 MB one platform costs — a fine trade on a laptop, a poor one in a container image.

Get it wrong and `OpenCv.load()` does not hand you a bare `UnsatisfiedLinkError`: it throws `CvError.NativesMissing`, whose message is both coordinates, copy-pasteable, with the classifier for the platform you are actually on already filled in.

Acknowledgements

This book, and the library it documents, rest on other people's work. The OpenCV project first: a quarter-century of algorithms, maintained by Intel, Willow Garage and a very large community, Apache-2.0 licensed since 4.5.0. Everything in Parts II through V is OpenCV doing the arithmetic; `scalacv` only decides who owns the result. Bytedeco's JavaCPP Presets, by Samuel Audet and contributors, are how those natives reach a JVM at all: the classifier jars, the extraction into `~/.javacpp`, and the loading machinery behind a headless `OpenCv.load()`. Under that sits OpenBLAS, doing the linear algebra.

The name `scalacv` and the original spark come from `mcallisto/scalacv` by Mario Càllisto, a Scala 2.11 wrapper over the OpenCV 3.0 Java API begun in 2015; two example ideas trace further back, to `rladstaetter/isight-java` and `chimpler/blog-scala-javacv`. None of those repositories carried a license, so this library is a clean-room rewrite sharing no code with any of them: the effect model, the module layout and the underlying Java API all changed, and every call site was rewritten. The credit is for the inspiration, and it is recorded here and in the repository's `NOTICE`.

Chapter 1 picks up where this preface stops: with the forty bytes standing in front of the six gigabytes, and what a typed, scoped API has to do about them.

PART I

Foundations

What OpenCV's Java bindings are, what they get wrong on the JVM, and the five ideas — typed constants, owned handles, move semantics, scoped lifetimes and errors-as-values — that the rest of the library is built out of.

CHAPTER 1

Why This Library Exists

How forty bytes of Java object can cost you six gigabytes of machine.

A vision service is one of the few kinds of program that can die without ever misbehaving. The heap graph is flat. There are no long pauses, no promotion storms, no allocation rate worth naming. Every dashboard you have is green, and the container is killed anyway, at three in the morning, with a kernel OOM message and no Java stack trace anywhere in the logs. Restart it and the clock begins again: forty minutes of perfect health, then the same death.

The reason is that almost nothing your JVM monitoring can see is where the memory went. OpenCV stores pixels in native memory, allocated by C++ through `cv::fastMalloc`, and hands the JVM a small Java object holding a `long` address. A 1920 × 1080 three-channel frame is about six megabytes of pixels behind roughly forty bytes of Java. Decode thirty of those a second and you are moving 180 MB/s of native memory behind 1.2 KB/s of heap. The garbage collector runs on heap pressure. It sees the 1.2 KB/s. It has no reason to run at all, and so it does not, and the six megabytes stay where they are.

This is not a bug in OpenCV, and it is not a bug in the JVM. It is what happens when a memory model that reclaims by tracing meets a memory model that reclaims by explicit free, and nobody writes down which one owns what. The Java bindings will happily let you not write it down. They will let you drop a `Mat` on the floor, and the program will keep working — for a while, on your laptop, on the test image, at the scale you developed against.

scalacv exists because that failure is worth engineering against directly rather than discovering repeatedly. The typed API, the enums, the `Either`s, the fluent pipeline — all of that is pleasant, and none of it is the reason. The reason is that in this library, every native object has exactly one owner and a defined moment of death, and misusing one is a Scala exception at the offending line rather than a segfault an hour later.

What the Java bindings actually are

A great deal of confusion here comes from imagining the bindings are something more than they are.

OpenCV is a C++ library. Its Java API is generated: a code generator walks the C++ headers and emits, for each exported class, a Java class holding a single `long nativeObj` field and a set of `private static native` methods that take that address as their first argument. `org.opencv.core.Mat`, `org.opencv.objdetect.CascadeClassifier`, `org.opencv.dnn.Net` — all of

them are this shape. There is no Java-side data structure, no buffer, no `byte[]`. There is an address, and a set of JNI calls that dereference it.

That shape has three consequences, and they recur throughout this book.

The first is that the Java object's size tells you nothing about what it costs. A `cv::Mat` header is around a hundred bytes in C++, and the Java object in front of it is smaller still; the pixel buffer hanging off that header is the megabytes. An empty `Mat` and a `Mat(1000, 1000, CV_8UC3)` are indistinguishable in a heap dump, and one of them holds three megabytes of the machine. Every profiler you already trust is measuring the wrong side of the pointer.

The second is that every method call is a JNI transition. The JIT compiler cannot see across it, cannot inline it, and cannot optimise it away — which is why a per-pixel accessor on a `Mat` is slow, not because OpenCV is slow but because it is a border crossing per pixel.

The third is the dangerous one. A method call on a freed address is not an exception. It is a read of memory the allocator has handed to someone else, which is a `SIGSEGV` when you are lucky and a plausible-looking wrong answer when you are not. The JVM does not catch this. There is no stack trace, because there is no Java frame to unwind — the process is gone, and the `hs_err` file points at C++ you did not write.

> **NATIVE MEMORY**
> Freeing a native object twice is worse than either leaking it or using it after free. A double `delete` corrupts the C++ allocator's own bookkeeping, and the crash it eventually produces happens somewhere else entirely, at an unpredictable later moment, in code that is completely innocent. Every design decision in this chapter that looks over-cautious is aimed at that failure.

The collector cannot see what it does not own

The gap between heap pressure and native pressure is measurable in a few lines, and this project measures it. The test allocates 2000 `Mat(1000, 1000, CV_8UC3)` — three megabytes of pixels each — drops the reference to every one of them immediately, and never calls `System.gc()`. Nothing about that workload is pathological; it is what a batch job over a directory of photographs does. Then it reads process resident-set size and compares it against the same loop with one line added.

Table 1-1. 2000 unreleased Mats against the same 2000 released, measured as process RSS.

Workload	Final RSS
references dropped, no `release()`	5,865 MB
`release()` on each	144 MB

Forty-one times the memory, for one omitted call, in a program that at no point holds more than one live reference. The same ratio reproduces on JDK 21 and on JDK 25, so it is not an artefact of one collector's tuning. And note what the second row proves: reclamation was

always possible. The memory was never unreachable-but-unfreeable. Nothing in the runtime had any motive to make it happen, and no deadline by which to try.

That last point is the one people argue with, so it is worth stating flatly. The JVM is not failing here; it is doing exactly what it promises, which is to manage the heap. Forty bytes per frame is not heap pressure at any frame rate you can achieve, so nothing ever runs that would call `release()` on your behalf — and by the time the cgroup limit is hit, the killer that arrives is the kernel's, not the JVM's.

The types you cannot free at all

If the story ended at `Mat`, the fix would be discipline, and this library would be a thin convenience. It does not end there.

Of the 188 `org.opencv.*` types that own native memory, exactly **three** expose a public `release()`: `Mat`, `VideoCapture` and `VideoWriter`. The other 185 do not. `CascadeClassifier` does not. `Net` does not. `QRCodeDetector`, `ArucoDetector`, `KalmanFilter` — none of them. Each one carries a `private static native void delete(long)` that the generator emitted, and a `finalize()` that calls it, and nothing else. From Java, with the public API alone, there is no supported way to free a detector before the collector decides to.

The cost of that scales with the object, and a detector is not a small object. A `KalmanFilter` holds several matrices; a loaded `Net` holds the entire model's weights. Measured the same way: 4000 leaked `KalmanFilter`s reached 54 GB of resident memory, against 86 MB when released.

scalacv frees them anyway. That decision — to reach past the public API of the bindings rather than accept a 54 GB leak — is the most consequential piece of engineering in the library, and worth understanding before you rely on it.

Why finalizers and cleaners do not save you

Three mechanisms are usually proposed at this point. Each is genuinely appealing, and each is worth walking to its end, because each genuinely does not work.

Finalizers

The bindings already have them. It is a common myth that `finalize()` no longer runs; it runs. The problem is *when*. A finalizer runs once the collector has determined the object is unreachable, which requires the collector to run, which requires heap pressure — and forty bytes per frame produces none, as the table above measures. Finalization is also unordered, single-threaded through one queue, and, since JDK 18, deprecated for removal, which means the mechanism this library would be depending on is one the platform has announced it intends to delete.

A Cleaner

`java.lang.ref.Cleaner` fixes the deprecation and the ordering, and fixes nothing else. A cleaner action still fires on the same trigger: the referent becoming unreachable, observed by

the collector, under heap pressure. Registering a cleaner around a `Mat` gives you a better-engineered version of the same latency, which is unbounded. You cannot cite a `Cleaner` in an argument about a fixed memory ceiling, because it makes no promise about time.

A phantom-reference budget

You can go further and count bytes yourself, tripping a manual drain when a native budget is exceeded — which is roughly what JavaCPP does with `Pointer.totalBytes()` and `-Dorg.bytedeco.javacpp.maxBytes`. This is a real technique. It is also blind to the memory in question, for a reason covered in the sidebar below, and it still leaves the 185 releaseless types with nothing to call.

So scalacv does the unglamorous thing instead: it frees them itself, at a moment you determine, and it fails loudly if it cannot. `Releasable[A]` is the type class that says how a native type dies, and it has exactly two regimes. Which one applies is not a style choice; it is dictated by what the generated binding exposes.

Table 1-2. The two release regimes, and what dictates which one applies.

Native type	How it frees
`Mat`, `VideoCapture`, `VideoWriter`	the public `release()`. For `Mat` this drops the pixel *buffer* immediately — the part that costs megabytes.
the other 185 (`CascadeClassifier`, `Net`, `ArucoDetector`, `KalmanFilter`, …)	a cached `MethodHandle` onto the binding's private `delete(long)`, after *disarming* the binding's finalizer.

The second row is where the care lives: freeing one of the 185 is a three-step sequence whose order is not negotiable, and getting it wrong is the double free described at the top of this chapter. The bridge reaches for private API, so when it cannot — a class loaded from a named module, a finalizer field it is not permitted to zero — it throws rather than quietly falling back to the collector, because a leak is recoverable and a corrupted heap is not. Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, sets the sequence and its refusals out in full.

Why the obvious leak detector reports nothing

The first instinct on suspecting a native leak is to reach for JavaCPP's accounting: `Pointer.totalBytes()`, and the `-Dorg.bytedeco.javacpp.maxBytes` budget derived from it. This project's memory audit tried exactly that, and found it completely blind: a deliberate leak of roughly 1.4 GB of `org.opencv.core.Mat` moved `totalBytes()` by zero bytes.

The explanation is that those buffers are never allocated through a JavaCPP `Pointer` allocator at all. OpenCV's own JNI calls `cv::fastMalloc`, on the far side of a boundary JavaCPP has no visibility into. Its counter is accurate about the memory it allocated and silent about the memory that actually costs you the machine.

So scalacv's leak suite measures process RSS instead — `/proc/self/statm` field two on Linux, falling back to `Pointer.physicalBytes()` elsewhere — runs a workload three hundred times after a warm-up, and asserts the growth stays under a bounded tolerance. The bound is deliberately not zero, because allocator arenas and the code cache mean RSS never returns exactly to baseline, while a per-iteration leak of even a modest `Mat` clears it easily over that many rounds. The suite owns its own JVM, because RSS is process-global and a concurrently running suite would contaminate it. That is a clumsier instrument than a counter, and the only one that sees the thing being measured.

Four commitments follow from this

Everything else in the library is downstream of the argument above, and falls into four commitments.

Typed everything

OpenCV's Java API is a C++ API in translation, which means its parameters are `int`s carrying enumerated meanings. `Imgproc.cvtColor(src, dst, 6)` is a legal call; the 6 is `COLOR_BGR2GRAY`. So is `Imgproc.cvtColor(src, dst, 7)`, and so is every other integer you could type there, and the compiler has no opinion about which one you meant. Nor do the families of constants occupy separate spaces: the `int` that `cvtColor` reads as a colour conversion, the one `resize` reads as an interpolation and the one `threshold` reads as a mode are the same type, so a constant lifted from one family passes silently into another. scalacv replaces every one of those with a typed enum: `ColorConversion.BgrToGray`, whose single job is to carry `Imgproc.COLOR_BGR2GRAY` in its `cvValue`, so that you never type the number and never hand one family's constant to another.

Explicit resource ownership

`Managed[A]` is the ownership primitive, and it makes two guarantees, both of which exist because getting them wrong is a JVM crash rather than an exception. Release is a compare-and-set, so a second release is a no-op and never a double free. Access after release throws an ordinary `IllegalStateException` on the Scala side, before anything crosses JNI.

Example 1-1. A released handle fails in Scala, not in C++.

```
import org.opencv.core.{CvType, Mat}

val m = Managed(Mat(4, 4, CvType.CV_8UC1))
m.close()
m.close()        // no-op: release happens exactly once
m.use(identity)  // throws IllegalStateException, not SIGSEGV
```

On top of that, the high-level `Image` adds move semantics: a transform *consumes* the image it was called on and returns a new one, so a chain of any length holds exactly one live `Mat`

at a time. The mistake this catches is the one people actually make — branching off a source and then reusing it:

Example 1-2. The wrong version: the source was already spent.

```
val img  = Image.blank(8, 8)
val gray = img.gray   // .gray moved the Mat out of `img`
img.width             // throws: `img` was spent by .gray
```

Example 1-3. The right version: branch off a copy, which is independent.

```
val src        = Image.blank(120, 80, Scalar.White)
val edgeBytes  = src.copy.gray.canny(50, 150).bytes(".png")  // works on a clone
val thumbBytes = src.resize(30, 20).bytes(".png")            // consumes `src`
```

Note what the type system does and does not do here. It does not reject the wrong version at compile time; both listings compile. The guarantee is narrower and still worth having: the failure is a Scala-side `IllegalStateException` naming the type and telling you to call `.copy`, thrown before any address reaches JNI, rather than undefined behaviour in C++. The exception fires at the reuse, which is rarely the interesting line. Start the JVM with `-Dscalacv.trackOwnership=true` and it carries, as its cause, the stack of the transform that actually spent the handle. It is off by default because it allocates a `Throwable` on every consume; the read happens only on the already-failing path, so a correct program pays nothing for it.

Genuinely headless

The obvious way to bring OpenCV's natives up — `Loader.load(classOf[opencv_java])` — eagerly initialises the whole preset graph, and `opencv_highgui` is GTK2-linked on Linux. On a machine without GTK it throws, and takes `objdetect`, `calib3d`, `features2d` and `video` down with it. That is precisely the set this library needs most. `OpenCv.load()` instead comes up through a GUI-free preset, extracts the platform payload, and resolves the JNI shim's dependencies on demand rather than loading them speculatively — which turns out to matter for correctness and not only for tidiness, as Chapter 2 shows. It needs no display server and no `apt-get` on any runner, and the whole test suite runs that way.

Errors as values where they belong, exceptions where they belong

The library draws a deliberate line. Failures that are *data-dependent and expected* — a missing file, undecodable bytes, a model that will not load, a calibration that will not converge — return `Either[CvError, A]`, with `CvError` a typed hierarchy you can match on: `DecodeFailed`, `EncodeFailed`, `LoadFailed`, `CalibrationFailed` and `NativeCall`. `NativesMissing` sits outside that list: natives that will not load, or a release bridge that will not open, are always thrown rather than returned, because there is no program state in which continuing is the right move. Failures that are *programmer errors* — an even Gaussian kernel, a negative radius, reusing a consumed handle — throw too, because they are bugs to fix rather than conditions to branch

on. Modelling a use-after-move as a `Left` would invite you to recover from it, and there is nothing to recover: the handle is gone.

What this library is not

Three disclaimers, because each one saves you a wrong expectation.

It is **not a reimplementation of OpenCV**. There is no pixel loop in this repository that could have been an OpenCV call. Every operation bottoms out in the same `Imgproc`, `Core` or `Calib3d` function you would have called yourself, with the same numerics and the same results.

It is **not a wall in front of OpenCV**. The escape hatches are documented API, not accidents: `image.mat` borrows the underlying handle, `image.managed` hands the whole ownership over, `Image.wrap` takes it back, and a mid-level layer of extension methods on `Mat` gives you every operation one tier down while still returning a `Managed[Mat]` you own — and never writing to its receiver, so it can be applied to a borrowed frame with no ceremony. When the high-level pipeline does not cover your case, you drop a tier; you do not leave the library.

It is **not designed to be called from Java**. The API returns `Seq`, `Option` and `Either`, reaches you through extension methods activated by `import scalacv.*`, and uses Scala 3 features throughout. It wraps a Java library; it is not one.

The shape of it

Here is the whole argument as code. It draws its own input, so it runs with no image file and no display server — which is how it is tested.

Example 1-4. One chain, from a blank canvas to encoded bytes.

```
import scalacv.*

OpenCv.load()

val edges: Either[CvError, Array[Byte]] =
  Image
    .blank(160, 120, Scalar.White)
    .drawRect(Rect(30, 30, 90, 60), Scalar.Black)
    .gray
    .canny(50, 150)
    .bytes(".png")
```

Count the `release()` calls: there are none, and none is missing. Every stage consumes the image it was given and returns the next one, so by the time `canny` runs there is nothing left holding the grey `Mat`, and `bytes` — a terminal — releases the last handle as it produces the array. The only thing that survives the chain is plain data: a `Left` carrying a typed `CvError`, or a `byte[]` you can keep forever.

When the input comes from a file rather than a constructor, `Image.reading` scopes it to a block and releases it on success, on failure, and on exception:

```
Image.reading("photo.jpg") { img => img.gray.blur(2).canny(80,
160).write("edges.png") }
```

Chapter 4, *The Image Type*, unpacks the ownership model that makes the chain safe: which calls borrow and which consume, what `copy` actually costs, and how to step down to the `Mat` tier when `Image` does not reach far enough. Chapter 5, *Lifetimes*, goes underneath it to `Managed`, `Releasable`, and `Managed.scope`, which owns the several native objects an operation such as `solvePnP` needs at once.

Where this goes next

The argument is made; what remains is a working process. Chapter 2, *Setting Up*, takes you from an empty build file to a JVM that has loaded the natives — the classifier for every supported platform, the Mill, `sbt` and `scala-cli` spellings of the same lines, the optional `scalacv-vision`, `scalacv-graphs` and `scalacv-zio` modules, what `OpenCv.load()` writes to disk the first time it runs, and how to prove on your own machine that it worked.

CHAPTER 2

Setting Up

Four artifacts, two dependency lines, one load call, and the cache that pays for all of it.

Most Scala libraries install by adding a line to a build file. You add the coordinate, the resolver walks the POM graph, and the code you wrote against the API on the website runs. That machinery was designed around one assumption: that the thing being resolved is portable bytecode, identical on every machine that will ever run it.

An OpenCV binding breaks that assumption in the first minute. Almost nothing you are installing is Scala, and almost nothing is Java. OpenCV is a C++ library, and what your program actually calls is several hundred megabytes of machine code compiled separately for Linux on x86-64, Linux on ARM64, macOS on Apple silicon, macOS on Intel and Windows on x86-64. Those builds are not interchangeable, they are not small, and Maven's vocabulary for "the same artifact, compiled for a different machine" is the classifier — a field that a published POM cannot carry through to a downstream consumer. So the dependency resolution that works perfectly everywhere else stops one step short of a program that runs.

The result is the most common way to have a bad first day with a JVM computer-vision library. The Scala compiles. The types resolve. `org.opencv.core.Mat` is right there in the autocomplete. Then the first call that crosses into native code dies with an `UnsatisfiedLinkError` naming a shared object you have never heard of, and says nothing about which of the five platform jars you were supposed to add. scalacv treats that as a design problem rather than a documentation problem: `OpenCv.load()` catches the link failure and rewrites it into a message naming the platform you are actually on, with the exact two lines to paste. Better still not to arrive there.

The four artifacts

> **WARNING**
> The installation coordinates in this book name source tag `v0.4.2`, not a verified Maven Central deployment. Central upload and attestation are disabled; tag workflows create draft releases. Build a clean checkout of that tag with `./mill __.publishLocal` and enable local Ivy resolution (`ivy2Local` in Coursier; sbt includes it by default). Unreleased fixes described in this book need the current checkout's actual `./mill show core.publishVersion` instead. Keep all four artifact versions aligned. Website deployment is not registry publication.

scalacv defines publication coordinates under the group id `com.worxbend`, as four separate artifacts. Only the first is required. The other three are opt-in, and the split is real rather than

cosmetic: `vision` and `graphs` depend on `core` and on nothing else, so a service that reads and filters images never pulls a SLAM loop-closure detector into its jar.

Table 2-1. The published artifacts, and what lives in each.

Artifact	What it holds
`scalacv`	The core: `Image`, `Managed`, `Camera`, `Video`, `Recorder`, contours, thresholding, the filter catalogue and the draw verbs
`scalacv-vision`	`Cascades`, `FaceRecognizer`, `Dnn`, `Ocr`, `Ar`, `MotionDetector`, `Features`, `OpticalFlow`, plus pose, tracking, calibration and the navigation front end
`scalacv-graphs`	The `Picture` scene graph, `Color`, `Chart` and animated GIF output
`scalacv-zio`	The ZIO integration, in package `scalacv.zio`

Those names are the `artifactName` values in the project's own `build.mill`, not the directory names — the module directories are `core`, `vision`, `graphs` and `zio`, and without the explicit override Mill would have published `com.worxbend:core_3`.

One consequence of the split is worth learning before it bites. Each module owns a distinct package: `scalacv`, `scalacv.vision`, `scalacv.graphs`, or `scalacv.zio`. Add the dependency and the wildcard import for every layer you use:

Example 2-1. Core plus optional layers: keep only the imports whose modules you use.

```
import scalacv.*
import scalacv.vision.*
import scalacv.graphs.*
import scalacv.zio.*
```

A missing module or import reports a missing symbol or member rather than a missing dependency. `Not found: Cascades` is at least recognisable. `value faces is not a member of scalacv.Image` is the confusing one, because `import scalacv.*` brings `Image` into scope but cannot activate an extension in `scalacv.vision`. Both the vision jar and `import scalacv.vision.*` are required.

> **NOTE**
> If a scalacv method the documentation describes will not compile, check its dependency and wildcard import before you check your spelling. A core import does not import the optional layers.

Why there are two dependency lines

`scalacv`'s own POM declares exactly one OpenCV coordinate: `org.bytedeco:opencv`, without a classifier. That artifact is the OpenCV **Java API** — the `org.opencv.*` classes, the JNI method declarations, and no machine code whatsoever. It is enough to compile against and useless to run.

The machine code ships in sibling artifacts distinguished only by a classifier: same group, same name, same version, different payload. A build tool can express that locally. It cannot express it in a POM it publishes — Mill 1.1.7 cannot write a classifier into a POM at all, so anything classified in `core`'s dependency list would be silently stripped and consumers would resolve nothing. And if the library did pick a platform on your behalf, it would be the wrong one for everybody who does not share it. So the second line is yours.

Table 2-2. The classifier for each supported target.

Where the code runs	classifier
Linux x86-64	`linux-x86_64`
Linux ARM64	`linux-arm64`
macOS Apple silicon	`macosx-arm64`
macOS Intel	`macosx-x86_64`
Windows x86-64	`windows-x86_64`

Pick for the machine that will **run** the code, not the one that builds it: a `linux-x86_64` jar in a container image built on an Apple-silicon laptop is a straight failure at startup. bytedeco also publishes `-gpu` variants of `linux-x86_64`, `linux-arm64` and `windows-x86_64` for CUDA work, and publishes no OpenCV natives at all for `windows-arm64` — scalacv's own build refuses to derive a classifier for that platform and says so in those words rather than resolving something that cannot exist.

The second half of the trap is that one classifier line is not enough. Here is the version people write first:

Example 2-2. Wrong: one native line. `OpenCv.load()` will not get through this.

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64"
)
```

`libopencv_core` has a `NEEDED` entry on `libopenblas.so.0`, which ships only in openblas's own classifier jar. Omit it and you get a half-loaded library: `load()` fails with an `UnsatisfiedLinkError` that scalacv converts into a `CvError.NativesMissing` naming exactly what is absent. Both lines, always, and both with the same classifier:

Example 2-3. Right: the API arrives through scalacv, the natives through two classifier jars.

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
  mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
)
```

Two lines for you, three coordinates in total. The classifier-less API jar is the third, and it arrives transitively through `scalacv`. That distinction matters if you ever vendor the natives into a build of your own: a classifier dependency **replaces** the classifier-less artifact in resolution rather than adding to it, which is why scalacv's own `build.mill` declares all three explicitly in its `opencvNatives` block. Declare only the classifier and you get a classpath with zero `org/opencv/` classes on it.

The fat option, and what it costs

If you would rather not choose — a workshop, a demo repository, a laptop that dual-boots — there is one coordinate that works everywhere:

```
mvn"org.bytedeco:opencv-platform:4.13.0-1.5.13"
```

It bundles every platform's natives, including openblas, so it replaces both classifier lines — at about **408 MB** against the 36–80 MB a single platform pair weighs (for `linux-x86_64`, about 31 MB of `opencv` and 20 MB of `openblas`). A good trade on a developer machine, a poor one in a container you will pull a thousand times a day.

The build files

Mill is the project's own tool, so its syntax is the one every scalacv source file and error message is written against. Note the quirk of `mvn"…"`: the classifier goes after a semicolon, inside the interpolated string, not as a separate argument.

Example 2-4. A complete consumer `build.mill`.

```
//| mill-version: 1.1.7
package build

import mill.*, scalalib.*

object app extends ScalaModule {
  def scalaVersion = "3.3.8"

  def mvnDeps = Seq(
    mvn"com.worxbend::scalacv:0.4.2",         // core: images, video, contours,
drawing, filters
    mvn"com.worxbend::scalacv-vision:0.4.2",  // detectors, DNN, pose, tracking,
OCR, calibration
    mvn"com.worxbend::scalacv-graphs:0.4.2",  // the Picture scene graph, charts,
animated GIFs
    mvn"com.worxbend::scalacv-zio:0.4.2",     // only if you use ZIO

    mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
    mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
  )
```

```
  def forkArgs = Seq("-Djava.awt.headless=true")
}
```

Delete the scalacv lines you do not need; keep both bytedeco ones. In sbt the same set uses `%%` for the Scala artifacts and `%` for the bytedeco jars, which are ordinary Java-world coordinates with no Scala suffix:

Example 2-5. The sbt equivalent.

```
libraryDependencies ++= Seq(
  "com.worxbend" %% "scalacv"        % "0.4.2",
  "com.worxbend" %% "scalacv-vision" % "0.4.2",
  "com.worxbend" %% "scalacv-graphs" % "0.4.2",
  "com.worxbend" %% "scalacv-zio"    % "0.4.2",
  "org.bytedeco" %  "opencv"         % "4.13.0-1.5.13" classifier "linux-x86_64",
  "org.bytedeco" %  "openblas"       % "0.3.31-1.5.13" classifier "linux-x86_64"
)
```

scala-cli is the fastest way to try the library at all, because a single file carries its own dependencies. Its `using dep` directives take coursier's dependency syntax, where the classifier is a comma-separated attribute:

Example 2-6. A self-contained scala-cli header.

```
//> using scala 3.3.8
//> using dep com.worxbend::scalacv:0.4.2
//> using dep org.bytedeco:opencv:4.13.0-1.5.13,classifier=linux-x86_64
//> using dep org.bytedeco:openblas:0.3.31-1.5.13,classifier=linux-x86_64
//> using javaOpt -Djava.awt.headless=true
```

scalacv is compiled with Scala 3.3.8 — the LTS line, deliberately, because TASTy is not forward-compatible and a library published from the Next line cannot be read by anyone on 3.3.x — and targets JDK 17 and later. On JDK 24 and newer, `System.load` emits a JEP 472 warning and Scala 3.3.8's `LazyVals$` provokes four lines of `sun.misc.Unsafe` deprecation noise. Neither is a problem; both are noise, and the project silences them with two flags that **do not exist** on 17 or 21, where passing them is not a warning but `Unrecognized option` and a JVM that does not start.

Example 2-7. Runtime flags, but only on JDK 24 and newer.

```
java --enable-native-access=ALL-UNNAMED \
     --sun-misc-unsafe-memory-access=allow \
     -Djava.awt.headless=true \
     -cp app.jar hello
```

What `OpenCv.load()` actually does

Call it once, at the top of your program, before anything else in the library. It is idempotent and thread-safe — a `\@volatile` flag guarding a double-checked block — so putting it at the top of every entry point costs nothing after the first, and you should never gate it behind a flag of your own. `OpenCv.isLoaded` reports whether the first call has completed.

What it does not need is as important as what it does. It needs no GUI toolkit, no X server, no `DISPLAY`, and no `apt-get install libgtk2.0-0t64` on a CI runner. That is not a happy accident of the bundled build; it is the reason the loader does not use the one-line form every OpenCV-on-the-JVM snippet reaches for.

Why not `Loader.load(classOf[opencv_java])`?

That one-liner initialises the **whole** preset graph. One member of that graph is `opencv_highgui`, which on Linux is linked against GTK2. On a headless box it throws — and takes `objdetect`, `calib3d`, `features2d` and `video` down with it, which is to say precisely the modules a vision library exists to call.

`libopencv_java` itself links no GUI toolkit at all. So `OpenCv.load()` brings javacpp up through a GUI-free preset (`org.bytedeco.opencv.global.opencv_core`), extracts the platform payload with `Loader.cacheResources`, and then loads only the JNI shim plus whatever that shim actually asks for. On Linux the shim never asks for highgui; on macOS it does. The same code is correct on both without a platform conditional. Do not "simplify" it back.

The loading itself is demand-driven, and the reason is a crash rather than an efficiency argument. The bundled `libopencv_highgui.so` carries **unversioned** `NEEDED` entries — `libopencv_core.so`, not `libopencv_core.so.413`. Speculatively `dlopen`-ing every module library with `RTLD_GLOBAL` makes the dynamic linker search the system path for those names, and on a machine that happens to have a distribution OpenCV installed it does not fail. It succeeds, mapping six `libopencv_*.so.5.0.0` system libraries into the global namespace, where they interpose on the 4.13.0 symbols the rest of the process is using.

> **NATIVE MEMORY**
> Two OpenCV ABIs in one address space is not a link error — it is a `Mat` allocated by one version and freed by the other. The process dies inside `cv::Mat::release()` with no Java stack trace, at whatever point the two ABIs first meet, which may be minutes after startup and nowhere near the code that caused it. This was reproduced exactly that way on a developer machine with a system OpenCV 5 installed, and it is why the loader never loads a library it was not asked for.

So on Linux and macOS the sequence is: try `System.load` on the JNI shim; read the missing soname out of the `UnsatisfiedLinkError`; load **that one library**, by absolute path, from the extracted payload; retry. A dependency the payload cannot satisfy becomes a real

`CvError.NativesMissing` rather than something the linker quietly resolves against whatever the host has lying around, and asking twice for the same soname is treated as a failure rather than an infinite retry.

Windows is the exception. Its link error reads `Can't find dependent libraries` and names nothing, so there is no soname to act on and a bulk retry-load is used instead — safe there for the same reason it is unsafe elsewhere, since Windows DLL names embed the version (`opencv_core4130.dll`, not `opencv_core.dll`), and `opencv_highgui` there links only the always-present USER32 and GDI32.

The gate on that fallback is the platform string, not the shape of the error, and the distinction is load-bearing. "The message named no library" is not a Windows-only condition: a cross-classloader conflict, an undefined symbol, a `noexec` cache mount and an architecture mismatch all produce messages the soname parser cannot read, on every platform. Deciding to bulk-load from the error alone would sweep the payload with `RTLD_GLOBAL` on exactly the Linux machines the previous paragraph is about. So off Windows an unparseable error is rethrown untouched, and the gate fails closed — an absent or unrecognised platform means "not safe", because the cost of wrongly declining is a clear error and the cost of wrongly allowing is a segfault with no Java stack trace.

One more piece of defence explains a library that appears to be in the payload and is ignored anyway. javacpp materialises the unversioned aliases as symlinks, and on a machine with OpenCV already installed one can point straight out of the cache — where this was found, `libopencv_highgui.so` resolved to `/usr/lib/libopencv_highgui.so.5.0.0`, whose own dependencies name OpenCV 5.x. Any extracted entry whose canonical path escapes the directory javacpp extracted it into never enters the payload map at all.

> **WARNING**
> `Image.blank`, `Image.read`, `Camera.open` and every other constructor cross into native code. Calling one before `OpenCv.load()` is the single most common first-run mistake. Put the call at the top of your `main`, or in a test fixture, so it always runs first.

The extraction cache

The natives are not on your classpath as loadable code. They are payloads **inside** the bytedeco jars, and before the dynamic linker can touch them they have to exist as real files on a real filesystem. That is what the first `OpenCv.load()` in a fresh environment does: it unpacks the platform's libraries out of the jars into a cache directory, once. On Linux that is about **196 MB** under `~/.javacpp`, and every later run reuses it. Extraction is content-addressed and idempotent, so re-running never re-extracts and two processes can share one directory safely.

That once-per-environment cost is invisible on a laptop and very visible on a cold container start. Point javacpp elsewhere when `~/.javacpp` does not suit — a read-only home, a thin image layer, a cache shared across CI builds:

Example 2-8. Relocating the native cache, two ways.

```
java -Dorg.bytedeco.javacpp.cachedir=/var/cache/javacpp -jar your-app.jar

# Or, for containers where you do not control the java invocation:
export JAVA_TOOL_OPTIONS="-Dorg.bytedeco.javacpp.cachedir=/var/cache/javacpp"
```

The directory must be writable on first use; after that it can be mounted read-only. Because extraction is idempotent, running any main that calls `OpenCv.load()` at image-build time bakes the unpacked libraries into a layer and makes the first `load()` in every container instant. The opposite choice — mounting the cache on a shared volume — keeps the image thin and pays the warm-up once per volume. Pick per workload; there is no default that is right for both.

Two smaller stores sit beside it. Haar cascade XML is a classpath resource, and OpenCV only knows filesystem paths, so `Cascades.resolve` extracts the one you name out of the classifier jar's `share/opencv4/haarcascades/` directory into the same javacpp cache — on demand, cached after the first call, and needing no native load of its own, which is why it works before `OpenCv.load()`. A cascade-based detector therefore ships nothing extra. The exception is `windows-x86_64`, whose jar carries an empty `share/` directory and no cascades at all; there `resolve` returns a `CvError.LoadFailed` naming the platform rather than a null. Downloaded DNN and face-recognition models are the other store, and they are yours to place: `Models.fetch` writes them where you point it, checking each file against the size and the SHA-256 its `ModelSpec` pins — on download **and** on every cache hit, so a truncated file is caught on the run after the one that fetched it.

> **NATIVE MEMORY**
> Set a ceiling in production, and set the right one. `-Dorg.bytedeco.javacpp.maxPhysicalBytes` reads process RSS, so it sees OpenCV's `cv::fastMalloc` buffers and will stop a leaking `Mat` loop before the kernel does. `-Dorg.bytedeco.javacpp.maxBytes` tracks only javacpp's own `Pointer` accounting, which is blind to `org.opencv.core.Mat` entirely — gate on it and a leak runs RSS to the moon while the number you are watching sits flat.

Five settings cover everything a scalacv process wants told at startup. None is required to get a first run working; all are worth knowing before a run happens somewhere you cannot attach a debugger.

Table 2-3. The properties a scalacv process reads at startup.

Property	Effect
`org.bytedeco.javacpp.cachedir`	Where the natives are unpacked. Defaults to `~/.javacpp`; must be writable the first time, and may be read-only afterwards
`org.bytedeco.javacpp.maxPhysicalBytes`	The ceiling to set. Reads process RSS, so it counts OpenCV's own `cv::fastMalloc` buffers

`org.bytedeco.javacpp.maxBytes`	javacpp's `Pointer` accounting only, which never sees an `org.opencv.core.Mat`. Not a leak guard
`java.awt.headless`	Set it to `true` as an assertion: a GUI-toolkit regression then fails loudly instead of opening a window on somebody's laptop
`scalacv.trackOwnership`	Records where each native handle was spent, so a use-after-move exception carries the consuming call as its cause. Off by default — it allocates a `Throwable` per spend

`JAVA_TOOL_OPTIONS` carries any of them into a JVM whose command line you do not control — in practice, a container entrypoint written by someone else. Reach for `scalacv.trackOwnership` the first time a pipeline throws `IllegalStateException: this Mat has already been released or consumed` and the stack trace points at the reuse rather than at the call that spent the handle.

Hello, it works

The smallest honest proof that the install is complete is a program that allocates a real image, runs a real OpenCV kernel over it, and produces bytes — with no file on disk to go missing and no window to open. `Image.blank` gives you a canvas, `bytes` encodes and releases in one step, and the whole thing is four calls.

Example 2-9. `hello.scala` --- a headless smoke test with no inputs.

```
import scalacv.*

@main def hello(): Unit =
  OpenCv.load()

  val png: Either[CvError, Array[Byte]] =
    Image
      .blank(160, 120, Scalar.White)
      .drawRect(Rect(30, 30, 90, 60), Scalar.Black)
      .gray
      .canny(50, 150)
      .bytes(".png")

  png match
    case Right(bytes) => println(s"OK: ${bytes.length} bytes of PNG")
    case Left(err)    => println(s"failed: ${err.getMessage}")
```

A white canvas, a black rectangle drawn on it, converted to grey, run through Canny edge detection, encoded as a PNG in memory. Every step consumes the image it was called on and hands back a fresh one, so exactly one native buffer is alive at any moment and `bytes` releases the last of them — which is the ownership rule Chapter 5 is entirely about. For now the only thing to read out of it is the printed line.

Run it with whichever build you set up:

```
./mill app.runMain hello       # Mill
sbt "runMain hello"            # sbt
scala-cli run hello.scala      # scala-cli
```

The first run pauses for the extraction described above and then prints a number of bytes. Every run after that starts immediately. If instead you get a stack trace, read the next section before you change anything.

Verifying the install

`Core.VERSION` is the first thing everyone prints, and it proves nothing. It is a plain static `String` resolved from constants at class-initialisation time: a program with **zero** natives on the classpath prints `4.13.0` and exits successfully, having crossed no JNI boundary at all. scalacv's own smoke-test example allocates a `Mat` instead, precisely because allocation is the thing under test.

What you can trust is `scalacv.Build`, which reports how the artifact on your classpath was actually built. Every value in it is generated from the `Deps` block in `build.mill` rather than typed out a second time, so it cannot drift from the versions the build resolved — which is the whole point of a number you are going to quote in a bug report.

Table 2-4. What `scalacv.Build` exposes.

Value	What it reports
`Build.scalaVersion`	The Scala version scalacv was compiled with
`Build.openCvVersion`	The OpenCV release, in OpenCV's own numbering: `4.13.0`
`Build.openCvArtifactVersion`	The `org.bytedeco:opencv` coordinate version, OpenCV paired with javacpp: `4.13.0-1.5.13`. This is the string that goes in a build file
`Build.openBlasArtifactVersion`	The `org.bytedeco:openblas` version the natives need

The distinction between the last two is the one that costs people an afternoon. The natives for your platform must resolve at **exactly** the artifact version, because the Java API and the JNI shim it calls have to come from the same javacpp build; `4.13.0` on its own is not a coordinate. Extend the hello program to print all of it, and you have a report worth pasting into an issue:

Example 2-10. A version report you can paste into a bug report.

```
import org.bytedeco.javacpp.Loader

println(s"scalacv  built with Scala ${Build.scalaVersion}")
println(s"opencv   org.bytedeco:opencv:${Build.openCvArtifactVersion}")
println(s"openblas org.bytedeco:openblas:${Build.openBlasArtifactVersion}")
println(s"platform ${Loader.getPlatform}")
println(s"loaded   ${OpenCv.isLoaded}")
```

```
println(s"runtime  ${org.opencv.core.Core.VERSION} (expected
${Build.openCvVersion})")
println(s"headless ${java.awt.GraphicsEnvironment.isHeadless}")
```

`Loader.getPlatform` is worth printing on its own: it is the exact string the loader will look for inside the jars, and if it disagrees with the classifier you added, that disagreement **is** your bug. The project's own examples module carries two mains in this spirit — `jvmReport`, which prints `java.version`, `java.vendor` and `java.vm.name` so a three-JDK CI matrix cannot silently run the same JDK three times, and `smoke`, which allocates an 8×8 `Mat` across JNI and constructs one of each detector to prove its module linked, releasing every one.

When it goes wrong anyway

If the natives are missing or wrong, `OpenCv.load()` does not hand you the linker's error. It throws `CvError.NativesMissing` carrying this, with the version numbers taken from the build and the platform taken from `Loader.getPlatform` at the moment of failure:

```
OpenCV natives are not on the classpath (<the underlying message>).

scalacv depends on the classifier-less OpenCV Java API only, because a build tool
cannot
express a per-platform classifier in a published POM. Add the natives for your
platform:

  "org.bytedeco" % "opencv"   % "4.13.0-1.5.13" classifier "linux-x86_64"
  "org.bytedeco" % "openblas" % "0.3.31-1.5.13" classifier "linux-x86_64"

Both lines are needed: libopencv_core links libopenblas. If you would rather not
pick a
platform, "org.bytedeco" % "opencv-platform" % "4.13.0-1.5.13" bundles every
one, at a cost of about 408 MB.
```

One failure wears that exception's clothes without being it, and gets its own message: scalacv loaded from two classloaders at once. The loader detects that case and prints the real fix rather than asking for dependencies you demonstrably already have. Chapter 42, *Troubleshooting*, has the message and the remedy in full.

> **TIP**
> Before you debug a native problem, check the cheap things in order: both bytedeco lines present, both with the **same** classifier, that classifier matching `Loader.getPlatform` on the machine that runs the code, and `OpenCv.load()` called before the first constructor. Four checks resolve almost every first-day failure.

Where this leaves you

You now have a project that resolves, a JVM that loads roughly 196 MB of C++ without a window manager anywhere in sight, a cache that makes every subsequent start immediate,

and a four-call program that proves all of it. What you do not yet have is any feel for what those four calls cost, or why each one hands back a new `Image` instead of mutating the one you gave it.

Chapter 3, *Your First Pipeline*, answers the first half. It takes the skeleton nearly every vision program has — get pixels, simplify them, measure something, draw the answer, write it out — and builds it as one chain that frees itself at every link, with `Image.reading` scoping a file to a block and `Either` carrying the failures that are data rather than defects. Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, then goes underneath the chain to `Managed`, `Releasable` and the move semantics that make it leak-free: who owns a native handle, when it is freed, and what the `IllegalStateException: this Mat has already been released or consumed` means when you reach for one that has already been spent. From here on, every listing in this book assumes the `import scalacv.*` and the single `OpenCv.load()` written above, and repeats them only where the listing is a complete program.

CHAPTER 3

Your First Pipeline

Read, transform, detect, annotate, write — as one chain that frees itself.

Almost every computer-vision program has the same five-part skeleton. Get some pixels. Simplify them until the thing you care about is the only thing left. Ask a question of what remains. Draw the answer back onto the picture, so that a person can check it. Hand the result on — to a file, to an HTTP response, to the next frame. The interesting part is usually one line somewhere in the middle. The other four parts are plumbing, and the plumbing is where programs written directly against OpenCV's Java API go wrong.

They go wrong for one specific reason. A `Mat` — OpenCV's image buffer — is about forty bytes on the JVM heap standing in front of megabytes of native memory the garbage collector cannot see and will not free. Heap pressure is the only thing that triggers a collection, and heap pressure is uncorrelated with how much native memory you are holding. Measured on this project's own test machine: two thousand `Mat(1000, 1000, CV_8UC3)` allocations with the references dropped and no explicit `System.gc()` finish at 5,865 MB resident. The same two thousand with `release()` called finish at 144 MB. Forty-one times over, and nothing in between reports a fault — the process grows quietly until the kernel takes an interest.

The Java shape of a pipeline makes forgetting the path of least resistance. Each operation writes into a destination `Mat` you allocated beforehand, so a five-step pipeline names five intermediates, and every one of them needs releasing on the success path and on each of the paths where something throws. scalacv's high-level answer is to make the intermediates unnameable. Every operation on an `Image` returns a new `Image` and **spends** the one it was called on, so the chain holds exactly one live buffer however long it grows, and the last step in the chain releases that.

This chapter builds one program from end to end with that chain. The subject is a photograph of a noticeboard: first an edge map of it, then a version that counts the QR codes stuck to it, boxes the faces in the photographs pinned beside them, and writes an annotated copy. Along the way the program picks up the failure policy the library actually has, learns to run with no file on disk at all, and makes — deliberately — the mistake every newcomer makes with move semantics.

Two lines come first, and they come before anything else in the process:

```
import scalacv.*

OpenCv.load()
```

The import brings in the whole surface: `Image`, `Scalar`, `Rect`, the enums, and every extension method from whichever modules are on the classpath. You write it once per file, not once per feature. `OpenCv.load()` links the native libraries; it is idempotent and thread-safe, so calling it again from anywhere costs nothing, but calling anything else before it costs you a link error.

A synchronous scoped entry point

There are two ways to get an `Image` from a file, and they are not equally forgiving. Start with the forgiving one.

Example 3-1. The whole pipeline, in one expression.

```
Image.reading("noticeboard.jpg") { img =>
  img.gray.blur(2).canny(80, 160).write("edges.png")
}
```

`Image.reading` opens the path, hands the resulting `Image` to your block, and closes it when the block returns — on success, on a `Left`, and on an exception thrown from anywhere inside. It is the preferred entry point for synchronous work. Do not let raw handles or lazy effects escape its callback; manually owned resources still need explicit cleanup.

Two details support this cleanup policy. Release is idempotent, so `reading` closing an image the block already consumed — and the chain above does consume it, at `write` — is a no-op rather than a double free. And the whole body runs inside `Cv.attempt`, so a `CvError.NativeCall` thrown by `gray` or `canny` partway down the chain comes back as a `Left` instead of escaping past a signature that promised an `Either`.

> **NATIVE MEMORY**
> `reading` closes the image it opened, not every image you derive from it. Its guarantee covers the source and the chain the source flows into; an `Image` you create separately inside the block — from `copy`, say — is yours to close. And nothing stops you returning the `Image` out of the block, which is precisely the thing not to do: it is closed the moment the block returns, so what escapes is a spent handle that throws on first use.

The type of that expression is worth reading carefully, because it surprises people once. `reading` returns `Either[CvError, A]` where `A` is whatever the block returned — and the block returned `write`'s own `Either[CvError, Unit]`. So the full type is `Either[CvError, Either[CvError, Unit]]`: one layer for the read, one for the write. Flatten it with `flatMap(identity)` when you care about the outcome:

```
val done: Either[CvError, Unit] =
  Image.reading("noticeboard.jpg") { img =>
    img.gray.blur(2).canny(80, 160).write("edges.png")
  }.flatMap(identity)
```

What each operation does to the pixels

Read the chain left to right and it is four verbs, each one narrowing what the image contains.

gray

`gray` converts a three-channel BGR image to one channel, by the standard luminance weighting — green counts for most, blue for least, because that is roughly how a human eye responds. It is `convert(ColorConversion.BgrToGray)` under a shorter name, and it is the first move in most pipelines for two reasons: colour is not what edge detection is looking at, and one channel is a third of the work.

Everything downstream assumes it. `canny` produces a `CV_8UC1` result whatever you feed it, and `equalizeHist` and `adaptiveThreshold` accept nothing else — both are documented as `CV_8UC1` only, and a three-channel argument throws a `CvError.NativeCall` naming the operation that rejected it — a `Left` inside `reading`, which folds those throws for you — rather than being silently reinterpreted as one channel the library picked for you.

blur

`blur(radius)` is a Gaussian blur expressed as a radius rather than a kernel: `radius` 2 is a 5×5 kernel, `radius` 3 a 7×7, and `radius` 0 is the identity. A Gaussian replaces each pixel with a weighted average of its neighbourhood, the weights falling off with distance, so the fine, high- frequency variation — sensor noise, JPEG ringing, paper grain — averages out while real structure survives.

That matters more than it looks, because the next step measures gradients, and a gradient is a difference between neighbouring pixels. Skip the blur and every speck of noise is a small, sharp gradient of its own; Canny dutifully reports each one and the output is a snowstorm. A radius of 1 or 2 is the usual dose. If you want the kernel and sigmas spelled out instead, `gaussianBlur(kernel, sigmaX, sigmaY)` takes them directly; if the noise is salt-and-pepper rather than grain, `medianBlur` is the sharper tool — it takes a radius on the same scale, where 1 is a 3×3 neighbourhood, and it replaces each pixel with the median of that neighbourhood rather than a weighted mean, so a lone bright speck is discarded outright instead of being smeared over its neighbours.

> **NATIVE MEMORY**
> Three operations, one buffer. `gray` allocates a single-channel `Mat` and releases the colour one it read from; `blur` allocates its output and releases the grey input; `canny` does the same again; `write` encodes and releases the last. At no point in that chain are two images live, and there is no name in scope you could have used to keep one alive by accident.

canny

`canny(threshold1, threshold2)` is the edge detector, and the two thresholds are the part worth understanding, because they are not a range and they are not interchangeable.

Canny computes the gradient magnitude at every pixel, thins the result so that only the ridge line of each edge survives, and then applies **hysteresis**: any pixel above `threshold2` is

accepted outright as a strong edge; any pixel below `threshold1` is discarded; a pixel between the two is accepted only if it is connected to a chain that reaches a strong pixel. That middle band is the whole design. A single threshold either breaks long faint edges into dashes or floods the image with noise; a strong-and-weak pair lets a faint stretch of a real contour survive on the strength of the part of the contour that is unambiguous.

So `threshold1` is the weak, linking threshold and `threshold2` the strong one, and a ratio of about 1:2 or 1:3 between them is the conventional starting point — 80 and 160 in the listing above, 50 and 150 in a great deal of published example code. Both parameters are `Double` and silently swappable, which is why the scaladoc tells you to name them at the call site when the values are not obviously ordered: `canny(threshold1 = 80, threshold2 = 160)`.

The two remaining parameters are rarely touched. `apertureSize` (default `3`) is the size of the Sobel kernel used for the gradient, and `l2Gradient` (default `false`) switches the magnitude from the cheap `|Gx| + |Gy|` approximation to a true Euclidean norm.

Table 3-1. The chain, operation by operation.

Step	Channels	What it does to the pixels
`gray`	3 ⟶ 1	luminance-weighted greyscale; colour is discarded, not averaged
`blur(2)`	1 ⟶ 1	5×5 Gaussian; suppresses noise so gradients mean something
`canny(80, 160)`	1 ⟶ 1	gradient, thinning, then hysteresis between the two thresholds
`write("edges.png")`	—	encodes by file extension, then releases

The boundary where failure becomes a value

Four methods on `Image` turn failure into a value, and they are exactly the four that touch the outside world: `Image.read`, `Image.decode`, `write` and `bytes`, each returning `Either[CvError, A]`. `Image.reading` is a fifth only in the sense that it is built on `read`. Everything between them — the transforms — returns a bare `Image`.

That split is a policy, not an accident. A missing file, a JPEG truncated by a failed upload, a path whose parent directory does not exist: those are conditions of the data, they will happen in production, and a caller has something sensible to do about each. A negative blur radius or a reused image is a bug in the program, and wrapping it in an `Either` only lets it travel further from where it was written. The former are values; the latter throw.

`Image.read` gives you the `Either` directly, and `flatMap` threads it:

Example 3-2. The same pipeline, threading the Either by hand.

```
val edges: Either[CvError, Unit] =
  Image.read("noticeboard.jpg")
    .flatMap(_.gray.blur(2).canny(80, 160).write("edges.png"))
```

This form exists because `reading`'s block does not always fit: when the image is one input among several, when you are already inside a `for` comprehension over other `Either`s, when the result of the read is meant to be passed onwards rather than consumed on the spot. It is equally leak-free **provided the chain reaches a terminal**, because `write`, `bytes` and `close` all release. What it does not give you is a guarantee. If the chain is abandoned halfway — an early `return`, a branch that produces a `Left` before the terminal, an exception from a call you did not expect to throw — the `Image` is still holding its `Mat` and nothing will take it back. That is the whole difference between the two forms, and it is why `reading` is the default.

`Image.decode` is `read` for bytes you already have: an HTTP request body, a database BLOB, a test fixture. `bytes(format)` is the mirror image of `write` — it encodes to an in-memory image file and releases, so an HTTP handler never has to touch the filesystem to answer with a PNG.

All three read entry points — `Image.read`, `Image.decode`, `Image.reading` — carry a second, defaulted parameter, `flags: ImreadFlags = ImreadFlags.Color`, which decides what the decoder produces before a pixel reaches you. `ImreadFlags.Grayscale` decodes straight to one channel and makes the later `gray` call unnecessary; `ImreadFlags.Unchanged` keeps an alpha channel a colour read would drop. The type is a case class of `(color, scale, ignoreOrientation)` rather than an OR-able bitmask, because OpenCV's `IMREAD_*` constants are not orthogonal bits — each reduced-size value already bakes in its own colour bit, and `IMREAD_UNCHANGED` is `-1`, whose bits swamp anything OR-ed with it. So `ImreadFlags(ImreadColor.Grayscale, ImreadScale.Half)` is a single named constant, not a combination, and the codec skips the discarded detail instead of decoding it and throwing it away.

The failures are specific enough to act on. `CvError.DecodeFailed` carries the path and a `details` string that distinguishes "there is no file at this path", "this path is a directory, not a file", "the file is empty" and bytes no registered decoder recognises. `CvError.EncodeFailed` does the same for the write side, down to "no encoder is registered for this extension".

Why scalacv never calls imread

The decisive reason is the path, not the error: `imread` narrows the filename through JNI and Windows then reads the result in the process's ANSI code page, so a path with a non-ASCII character resolves to a name that is not there. Chapter 7, *Reading, Writing, and Interop*, has the detail.

So `Images.read` does not call `imread` at all. The JVM resolves the path, checks that it exists, is not a directory and is not empty, reads the bytes itself, and hands only the decoding to OpenCV through `imdecode`. `Images.write` is the same trade in reverse: the pixels are encoded in memory first and the bytes are written by the JVM, so a failed encode can no longer leave a half-written file on disk.

Distinguishable errors are the side effect, and they are the reason the four `details` strings above can exist: `imread` signals all four with the same empty `Mat`, plus a `findDecoder` warning printed to stderr that no API lets you suppress. That last warning still appears for genuinely undecodable bytes, because it comes from `imdecode`.

It is not free. The encoded image passes through a JVM byte array, so peak heap grows by the size of that array, and a file above 2 GB is out of reach because `Files.readAllBytes` cannot return an array that long. `path` also means a filesystem path resolved by the JVM — not a classpath resource, not a URL, not a glob. The same narrowing hazard applies to every other `String`-path native call in the library — `VideoCapture`, `VideoWriter`, `CascadeClassifier.load`, `Dnn.readNet` — and none of those has an in-memory equivalent to reroute through, so they stay ASCII-path-only on Windows.

A pipeline with no file on disk

You do not need a photograph to run any of this, and it is worth knowing the headless idiom early because it is how the library tests itself — this repository ships no image fixtures at all, and its examples draw their own input.

`Image.blank(width, height, color, channels)` gives you a canvas. The `draw*` methods paint on it; coordinates are pixels with the origin at the top-left, `x` growing right and `y` growing **down**. Colours are `Scalar` values in BGR order, so `Scalar.Red` is `Scalar(0, 0, 255)` — use the named constants and the order stops mattering.

Example 3-3. The same four verbs, on a scene the program draws for itself.

```
val edges: Either[CvError, Array[Byte]] =
  Image
    .blank(160, 120, Scalar.White)
    .drawRect(Rect(30, 30, 90, 60), Scalar.Black)
    .gray
    .canny(50, 150)
    .bytes(".png")
```

That runs in a unit test on a build agent with no display server, no `apt-get` and no fixture directory, and it exercises the identical code path as the version that reads a file. The only substitutions are `blank` plus `drawRect` for `read`, and `bytes` for `write`.

Note that `drawRect` sits in the chain like any other step. Drawing mutates the `Mat` in place — the image owns it, so there is nothing to copy — but it still spends the receiver and hands back a fresh `Image`, so the rule stays uniform: whatever you called it on is gone afterwards.

Detect, then annotate

Now the middle of the program, where it asks a question. Detection is a **query**: it reads the image and leaves it alive, so you can run several detectors on one image and then draw all their answers.

Example 3-4. Count the codes, box the faces, label the result.

```
val marked: Either[CvError, Unit] =
  FaceDetect.create(FaceDetect.ModelFileName, Size(320, 320)).flatMap { detector
=>
    detector.use { yunet =>
      Image.reading("noticeboard.jpg") { img =>
        val codes: Seq[QrCode] = img.qrCodes      // query: borrows
        val found: Seq[Face] = img.faces(yunet)   // query: borrows

        img.markFaces(found)                      // transform: consumes
          .drawText(s"${codes.size} codes, ${found.size} faces", Point(10, 30),
Scalar.Red)
          .write("annotated.png")
      }.flatMap(identity)
    }
  }
```

`qrCodes` returns `Seq[QrCode]`, each one a `text` and a `corners: Seq[Point]`. The text is empty when OpenCV located a symbol but could not decode it — a blurred or partly covered code — and those entries are deliberately kept rather than filtered out, because the corners are still good enough to draw an overlay or to re-crop and try again.

`faces(detector)` returns `Seq[Face]`, each with a `box`, a `score` in `[0, 1]`, and exactly five `landmarks` in a fixed order: right eye, left eye, nose tip, right mouth corner, left mouth corner. "Right" means the subject's right, so it appears on the left of the image. The `box` is not clipped to the frame — YuNet regresses boxes from anchors, so a face at the edge legitimately yields a negative `x`, and `crop` rejects such a rectangle outright. `Face.clippedBox(width, height)` is what to crop by: the intersection with the frame, as an `Option[Rect]` that is `None` when the box falls entirely outside it.

The detector comes from `FaceDetect.create(modelPath, inputSize, scoreThreshold, nmsThreshold)`, which returns `Either[CvError, Managed[FaceDetectorYN]]`. The last two default to `0.9f` and `0.3f`: the first is the confidence below which a detection is not reported, the second the overlap above which two boxes are treated as the same face and the weaker one dropped. The model is a download rather than a shipped asset — `FaceDetect.ModelFileName` names the file every OpenCV Zoo mirror uses, and `FaceDetect.downloadModel(dir)` takes a `java.nio.file.Path`, fetches it, verifies its pinned SHA-256 and answers `Either[CvError, Path]`. `faces` also has an overload taking the `Managed[FaceDetectorYN]` directly, which is the better call when the handle is in scope: the spent-handle guard travels with the argument instead of being thrown away by a bare `.get`.

> **NATIVE MEMORY**
> Look at what crosses back from a detector: `QrCode`, `Face`, `Point`, `Rect` — immutable Scala case classes, copied out of OpenCV's result matrices. Nothing you receive owns native memory, so there is nothing for you to free.
>
> That is not a stylistic preference. `QRCodeDetector` and `ArucoDetector` are two of the 185 `org.opencv.*` types that own native memory and expose only a private `delete(long)` — no public `release()` at all. `qrCodes` therefore builds its detector, uses it and frees it inside the one call, and hands you data instead of a handle. There is no version of that API where a caller could have done the freeing.

`markFaces(faces, color)` draws a box per face and a dot per landmark; `drawText(text, at, color, scale)` writes a label. Both are transforms, so they consume and return, and the chain ends at `write`. One thing about `drawText`: `at` is not the top-left corner. OpenCV anchors text on its baseline, so a `y` of 0 puts almost the whole string above the top of the image and draws nothing visible. `Draw.textSize` measures the string for you when placement has to be exact: it answers a `TextMetrics` carrying the text's `size` and, separately, the `baseline` depth beneath it, and a background box has to be `size.height + baseline` tall or every `g` and `y` is clipped. And only the Hershey vector fonts exist — OpenCV cannot render a system font, and anything non-ASCII comes out as `?`.

The mistake everybody makes once

Here is a version of the annotation step that looks obviously correct and is not.

```
Image.reading("noticeboard.jpg") { img =>
  val thumb = img.resize(320, 240)   // transform: consumes `img`
  val codes = img.qrCodes            // throws: `img` is spent
  thumb.drawText(s"${codes.size} codes", Point(10, 30)).write("thumb.png")
}
```

`resize` is a transform. It took the `Mat` out of `img`, resized it into a fresh one, and left `img` holding nothing. The next line asks the spent handle for its buffer, and instead of reading freed memory it throws:

```
java.lang.IllegalStateException: this Mat has already been released or consumed —
using it now would crash the JVM from native code. A high-level Image is spent by
any
transform (gray/blur/…) or terminal (write/bytes/close); call `.copy` before the
first
use if you need it twice. Run with -Dscalacv.trackOwnership=true to record where
it
was consumed.
```

That exception is the point of the design. In the Java API the same mistake reaches JNI and the process dies with a segmentation fault and no Scala stack trace — there is nothing to

attach a debugger to and nothing to log. Here it is an ordinary JVM exception, thrown on the Scala side before anything crosses the boundary, and the hint at the end is real: start the JVM with `-Dscalacv.trackOwnership=true` and the exception carries, as its cause, a stack trace of the line that consumed the handle. The throw fires at the reuse, which is rarely the interesting line; the cause points at the transform that spent it.

The fix is to branch explicitly with `copy`, which is a deep copy and therefore an independent image:

```
Image.reading("noticeboard.jpg") { img =>
  val codes = img.qrCodes                       // query: `img` is still alive
  val label = s"${codes.size} codes"
  for
    _ <- img.copy.resize(320, 240).write("thumb.png")   // the copy is spent, not
`img`
    _ <- img.drawText(label, Point(10, 30)).write("annotated.png")
  yield ()
}
```

The ordering rule that falls out of this is short: run every query first, take a `copy` for every branch you need, and let the last branch consume the original.

Table 3-2. Three kinds of method, three effects on the receiver.

Kind	Examples	Effect on the receiver
Query	`width`, `channels`, `contours()`, `qrCodes`, `faces`	borrows — call as often as you like
Transform	`gray`, `blur`, `canny`, `crop`, `drawRect`, `markFaces`	consumes — returns a new `Image`
Terminal	`write`, `bytes`, `close`	consumes and releases the native memory

`managed` is the one member that fits none of the three: it spends the `Image` without freeing anything, handing the underlying `Managed[Mat]` — and the obligation to release it — to the caller. `mat` is its opposite number, borrowing the raw `org.opencv.core.Mat` for any `org.opencv.*` call the fluent surface does not wrap while ownership stays where it was. Between them they are the escape hatch that keeps the low-level API one method away.

Running it

Two lines of dependency, as ever: scalacv itself, and the OpenCV natives for the platform you are actually on. This project keeps POMs platform-neutral, so the native coordinates are yours to pick.

Example 3-5. A Mill module for the program in this chapter.

```
package build

import mill.*, scalalib.*

object scan extends ScalaModule {
  def scalaVersion = "3.3.8"
  def mvnDeps = Seq(
    mvn"com.worxbend::scalacv:0.4.2",
    mvn"com.worxbend::scalacv-vision:0.4.2",   // faces, QR, markers
    mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
    mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
  )
}
```

With `@main def scan(path: String)` in package `demo`, that runs as:

```
./mill scan.runMain demo.scan noticeboard.jpg
```

Both bytedeco lines are needed: `libopencv_core` links `libopenblas`, so dropping the second gives you a half-loaded library rather than a clean failure. Swap `linux-x86_64` for the classifier that matches where the code will run — Chapter 2 has the full list, along with `org.bytedeco:opencv-platform`, which bundles every platform and works anywhere for about 408 MB instead of 36–80 MB.

For a single file, scala-cli needs no build definition at all — the directives are the build:

Example 3-6. The same program as one scala-cli script.

```
//> using scala 3.3.8
//> using dep com.worxbend::scalacv:0.4.2
//> using dep com.worxbend::scalacv-vision:0.4.2
//> using dep org.bytedeco:opencv:4.13.0-1.5.13,classifier=linux-x86_64
//> using dep org.bytedeco:openblas:0.3.31-1.5.13,classifier=linux-x86_64

import scalacv.*

@main def scan(path: String): Unit =
  OpenCv.load()
  Image.reading(path)(_.gray.blur(2).canny(80, 160).write("edges.png"))
    .flatMap(identity)
    .fold(e => println(s"failed: ${e.getMessage}"), _ => println("wrote
edges.png"))
```

```
scala-cli run scan.scala -- noticeboard.jpg
```

> **NOTE**
> On JDK 24 and newer, noise from two unrelated sources precedes every run. `--enable-native-access=ALL-UNNAMED` silences the JEP 472 warning that `System.load` emits when

> a native library is loaded from an unnamed module. `--sun-misc-unsafe-memory-access=allow` silences the four lines that scala3-library 3.3.8's `LazyVals$` provokes, which has nothing to do with OpenCV. This repository's own build passes both; add them as `forkArgs` in Mill or `//> using javaOpt` in scala-cli if the warnings bother you. Neither flag exists before JDK 24, and passing either to a 17 or 21 JVM is not a warning but `Unrecognized option` — the process does not start.

The first `OpenCv.load()` on a machine extracts the native libraries out of those jars into `~/.javacpp` — around 196 MB on Linux — and every later run reuses that cache; a read-only home or a thin container layer wants `-Dorg.bytedeco.javacpp.cachedir=…` instead.

Next: the rule underneath the chain

Everything in this chapter rests on one sentence: a transform consumes the image it was called on. It bought a leak-free pipeline with no scopes, no `try` / `finally`, and no name in scope for an intermediate you could forget — and it cost one `IllegalStateException` that you have now met.

Chapter 4, *The Image Type*, takes the type apart member by member: the full catalogue of queries, transforms and terminals, what `mat` and `managed` are for when the fluent surface runs out, and why `crop` returns an independent copy rather than a view onto the pixels it was cut from. Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, goes underneath it — what `Managed[A]` actually is, why `release` may be called twice and `get` may not, how `Releasable` reaches the 185 OpenCV types with no public `release()`, and what `Managed.scope` does when one computation needs several native objects alive at once.

CHAPTER 4

The Image Type

One owned Mat, three kinds of member, and a rule that makes a pipeline free itself.

Every OpenCV program has the same shape. Pixels come in from somewhere — a file, a socket, a camera — go through four or five operations, and something goes back out: a smaller file, a byte array, a count. Written against the Java bindings, that shape costs more than it looks like it should. Each `Imgproc` call wants a destination `Mat` you allocated yourself and writes into it, so a five-step pipeline names six matrices, five of which are garbage the instant the next step returns. None of them frees itself. `Mat` is about forty bytes on the heap in front of megabytes off it, so the collector sees nothing worth collecting and never runs; the process grows until the kernel notices. Measured on this project's own test machine, two thousand `Mat(1000, 1000, CV_8UC3)` with their references dropped and no `System.gc()` finish at 5,865 MB resident. The same two thousand with `release()` called finish at 144 MB.

So the accounting has to be done by hand, and it has to survive an exception thrown from the middle of the chain, which means a `try`/`finally` per intermediate, or one `finally` that releases six things in reverse order and has to cope with the ones that were never successfully allocated. This is not hard. It is merely relentless, and it is wrong the first time somebody adds a step in the middle without touching the cleanup at the bottom.

Most wrappers over OpenCV fix the pleasant half of this. They name the integer constants, return the destination instead of taking it as an out-parameter, and give the operations verbs rather than `Imgproc.` prefixes. The result reads better and leaks exactly as much as it did before, now with an extra layer between you and the `release()` you still have to remember. The unpleasant half — who owns each buffer, and when it dies — is the half that keeps the process alive.

`Image` takes one. It owns exactly one native `Mat` and enforces a rule about who is allowed to hand that `Mat` on. The rule costs something real, and the cost is stated up front rather than buried: you cannot use one `Image` twice. What it buys is a pipeline with no scope, no `try`, and no `release` call anywhere in it, that still holds exactly one live `Mat` no matter how many steps long it is.

Example 4-1. The pipeline the rest of the chapter takes apart.

```
import scalacv.*

OpenCv.load()
```

```
Image.reading("page.jpg") { img =>
  img.gray.blur(2).canny(80, 160).write("edges.png")
}
```

That line allocates four matrices — the one `Image.reading` opened, then one each for `gray`, `blur` and `canny` — and holds exactly one of them at any moment. All four are freed, including on the path where `canny` throws part-way. Understanding why that is true, and what it stops you from writing, is the whole of this chapter.

Three kinds of member

Everything on `Image` is one of three shapes. Which shape a member has tells you what happens to the image you called it on, and there is nothing else to know about ownership afterwards.

A *query* borrows. It reads the underlying `Mat` and gives back plain immutable Scala data — an `Int`, a `Size`, a `Seq[Contour]` of value types — and the image is still yours when it returns. `width`, `height`, `size`, `channels` and `isEmpty` are queries; so is `contours`, which runs `findContours` on a borrowed `Mat` and hands back Scala values. Because what a query returns owns no native memory, the result outlives the image it came from without any care on your part. You can close the image and keep the contours.

A *transform* consumes. `gray`, `blur`, `canny`, `resize`, `crop`, the whole `draw*` family, the photo verbs like `sepia` and `stylize` — each returns a **new** `Image` and spends the one it was called on. The receiver's handle is dead when the call returns: not dangling, not stale, dead in a way the library can detect. That is what makes the chain in Example 4-1 leak-free without a scope. Each step either frees the previous `Mat` or moves it into the result, so at no point do two of them exist.

A *terminal* consumes and releases. `write` encodes to a path and frees; `bytes` encodes into memory and frees; `close` frees and nothing else. After any of them the handle is spent exactly as after a transform, and the native memory is gone. There is one terminal that consumes without freeing — `managed`, which hands the whole `Managed[Mat]` to you instead — and it is the only member of `Image` that can leave live native memory behind. It gets its own section later.

Table 4-1. Representative members of each kind. `copy` and `toBufferedImage` borrow, yet build something new.

Query — borrows	Transform — consumes	Terminal — consumes
`width`, `height`	`gray`, `convert`	`write(path)`
`size`, `channels`	`blur`, `gaussianBlur`	`bytes(format)`
`isEmpty`	`canny`, `threshold`	`close()`
`contours(...)`	`resize`, `scale`, `crop`	`managed` (no free)

`mat`	`drawRect` , `drawText`
`toBufferedImage`	`sepia` , `stylize` , `filter`
`copy`	`inRange` , `applyMask` , `blend`

`copy` and `toBufferedImage` sit off the grid in the same direction: both borrow the `Mat` and build a fresh buffer from it, so the receiver stays alive and you hold a second, independent thing. Everything else that returns an `Image` spends the one it was given.

> **NATIVE MEMORY**
> An `Image` that never reaches a terminal leaks. Dropping the reference is not enough, for exactly the reason the chapter opened with: the forty bytes the collector can see are not the megabytes the process is holding. If the tail of your work is a query rather than a `write` or a `bytes` , put the work inside `Image.reading` , or `close()` it yourself.

Move semantics, and what happens when you get them wrong

The rule is worth stating precisely, because "consumes" is doing real work in those sentences. A transform does not copy your image and leave the original alone, and it does not mutate your image in place and hand it back. It moves the `Mat` out from behind your handle. Here is the private helper every transform on `Image` goes through, verbatim:

Example 4-2. The three lines that make the whole type work.

```
private def transform(op: Mat => Managed[Mat]): Image =
  try Image(op(handle.get))
  finally handle.release()
```

`op` borrows the source `Mat` , allocates a destination, runs the native call and returns a `Managed` around the result; the `finally` frees the source whether or not that succeeded. Two properties fall out of those three lines. The first is the one you use every day: after `img.gray` , the `Mat` that `img` was holding is freed, so a twenty-step chain allocates twenty matrices and holds one. The second matters on the day something goes wrong: the release is in a `finally` , so an operation that throws part-way still frees its input, and the mid-level layer underneath frees the half-built destination before the exception propagates. A failing pipeline leaks nothing either.

The in-place operations — the `draw*` family — go through a sibling helper, `paint` , which takes the `Mat` out of the handle rather than releasing it, mutates it, and wraps it in a new `Image` . Same outcome for the caller, no pixel copy at all: drawing five rectangles and a caption on a 4K frame touches the pixels under the shapes and nothing else.

What all this rules out is using one handle twice.

Example 4-3. The mistake everybody makes once.

```
val img = Image.read("page.jpg").getOrElse(sys.error("no page"))

val edges = img.gray.canny(80, 160)   // consumes img
println(img.width)                    // img was spent two steps ago
```

The second line does not read freed memory. It throws:

```
java.lang.IllegalStateException: this Mat has already been released or consumed —
using it now would crash the JVM from native code. A high-level Image is spent by
any
transform (gray/blur/…) or terminal (write/bytes/close); call `.copy` before the
first
use if you need it twice. Run with -Dscalacv.trackOwnership=true to record where
it
was consumed.
```

That is `Managed`, the layer under `Image`, doing the one thing that cannot be done anywhere else. Calling a method on a freed OpenCV object segfaults the JVM from native code: no stack trace, no `catch`, no test report, only a dead process and an hs_err file. `Managed` holds its object in an `AtomicReference` that is nulled by whichever of `release` or `take` gets there first, so the check happens in Scala, before anything crosses JNI, and a second release is a no-op rather than a double free. An `IllegalStateException` on the line that made the mistake is the difference between a five-second fix and an afternoon.

Pointing the error at the real mistake

The exception fires at the **reuse**, and the reuse is rarely the interesting line. In Example 4-3 the two lines are adjacent and there is nothing to diagnose; in real code the image is spent inside a helper forty lines away and the failure lands on a `width` call that is entirely innocent.

The flag named in that error message is the fix. Start the JVM with `-Dscalacv.trackOwnership=true` and every handle records a `Throwable` at the moment it is spent; when a spent handle is later touched, that `Throwable` is attached as the **cause** of the `IllegalStateException`, so the stack trace you get names the transform or terminal that actually consumed the image as well as the line that tripped over it.

```
java -Dscalacv.trackOwnership=true -cp ... com.example.Batch
```

It is off by default. Turn it on to debug a use-after-move and leave it off in production; Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, has the per-spend cost and the reason the flag has to go on the command line rather than into a `System.setProperty` call.

The verb that could not be called close

OpenCV's morphology has five compound operations, and one of them is `MORPH_CLOSE` — a dilation followed by an erosion, the standard way to seal small holes in a mask. A fluent image type also needs a `close()`, because `AutoCloseable` is how the JVM spells "release this". Two meanings, one short, obvious, universally understood name.

`Image` gave the name to the resource. `close()` frees the native memory, and morphological closing is reached through `morphology(MorphOp.Close)` alongside its four siblings — `Open`, `Gradient`, `TopHat`, `BlackHat`. The scaladoc says so in one dry parenthesis: there is no bare `close` method because `close()` already releases the image.

It is the right way round. Getting the wrong `close` would have been a lifetime bug in a type whose entire purpose is lifetimes, and lifetime bugs in native memory are the expensive kind. Erosion and dilation, which collide with nothing, keep their own top-level verbs (`erode`, `dilate`) because you reach for them constantly. Everything else in the family goes through `morphology`, which is one word longer and never ambiguous.

Branching: `copy`, and the borrowed `mat`

Move semantics forbid feeding one image into two chains, and sooner or later you want to. There are two ways out, and they are not interchangeable — one costs a buffer, the other costs nothing.

When both branches genuinely need to **transform** the pixels, take a `copy`. It borrows the receiver and clones the `Mat`, so afterwards you hold two independent images with two independent lifetimes, and each chain spends its own.

Example 4-4. Two chains, two lifetimes, one deliberate copy.

```
val page      = Image.read("page.jpg").getOrElse(sys.error("no page"))
val forHuman  = page.copy                      // borrows: `page` is still alive

val preview   = forHuman.scale(0.25).bytes(".jpg")   // consumes forHuman
val forEngine = page.gray.adaptiveThreshold(blockSize = 15, c = 10).bytes(".png")
```

> **TIP**
> `copy` is the one place in the API where a pixel copy is deliberate. Every other step threads a single buffer through the chain. If you find yourself calling `copy` inside a per-frame loop, that is a signal to restructure the loop rather than to accept the allocation — one full-frame clone per frame at 30 fps is a lot of memory bandwidth spent on a branch you may not need.

Often you do not need it. If one of the two branches only **reads** — a mean, a detector, a measurement, a draw straight onto the image you already own — drop to the borrowed `Mat` instead. `mat` is a query: it hands out the raw `org.opencv.core.Mat` and the `Image` keeps ownership of it. The mid-level extension operations and the whole typed `org.opencv.*` surface are available on it, and none of that consumes the image.

Example 4-5. A read-only branch that costs no buffer.

```
val page = Image.read("page.jpg").getOrElse(sys.error("no page"))

val meanGrey = org.opencv.core.Core.mean(page.mat).`val`(0)  // a raw
org.opencv.* call
val codes    = page.qrCodes                                  // an extension
query

val out = page                          // still alive, still ours
  .drawText(s"${codes.size} codes, mean ${meanGrey.toInt}", Point(10, 30))
  .write("annotated.png")               // now spent, and freed
```

The rule for telling them apart is short. If the second use needs its own pixels, `copy`. If it only needs to look at these ones, borrow the `Mat`.

> NATIVE MEMORY
> Borrowing is asymmetric on the members that take another image as an **argument**. `applyMask`, `blend`, `inpaint` and `seamlessCloneInto` consume the receiver and merely borrow what you pass in — so the mask or the background you handed over is still live when the call returns, and freeing it is your job. A segmentation that builds a mask, applies it, and forgets to `close()` the mask leaks one full-size `Mat` per call.

Getting an image in the first place

There are five ways to construct one, plus the scoped entry point, and they differ mainly in where the pixels come from and whether the outside world can refuse.

Table 4-2. The constructors on the `Image` companion.

Constructor	Returns	For
`Image.read(path, flags)`	`Either[CvError, Image]`	a file on disk
`Image.decode(bytes, flags)`	`Either[CvError, Image]`	an image file already in memory
`Image.blank(width, height, color, channels)`	`Image`	a fresh canvas to draw on
`Image.fromBufferedImage(image)`	`Image`	a frame from AWT, Swing or `ImageIO`
`Image.wrap(handle)`	`Image`	adopting a `Managed[Mat]` you hold
`Image.reading(path, flags)(use)`	`Either[CvError, A]`	read, work, close — all scoped

`read` and `decode` return `Either` because they are the boundary: a path that is not there, a path that is a directory, an empty file, bytes that are not an image. `imread` reports all four the

same unhelpful way, by handing back an empty `Mat`, so `read` does not call it at all — the JVM resolves the path and reads the bytes, and only the decode is left to OpenCV. A failure comes back as a `CvError.DecodeFailed` carrying the path, with which of the four it was in the error's `details`: "there is no file at this path", "this path is a directory, not a file", "the file is empty", or the decoder's own complaint about the bytes. `blank`, `fromBufferedImage` and `wrap` cannot fail against the outside world and do not pretend to: `blank` rejects a non-positive size, or a channel count other than 1, 3 or 4, with `IllegalArgumentException` up front, because that is a bug in your code rather than a fact about the world.

`Image.reading` is the scoped entry point to reach for unless you have a reason not to. It reads the file, runs your block, and closes the image afterwards — on success, on failure, and on an exception thrown from the middle of your chain. It is harmless if the block already consumed the image, because release is idempotent.

Example 4-6. The scoped form, where the body ends in a query.

```
val inkBlocks: Either[CvError, Int] =
  Image.reading("page.jpg") { img =>
    val binary = img.gray.threshold(128)
    try binary.contours().size
    finally binary.close()
  }
```

`reading` closes the image it opened, and nothing else. `gray` and `threshold` spend their receivers, so `img` needs no help; the binary image at the end of that chain is a different object, and `contours` is a query, so nothing has consumed it when the block returns. That is what the `try` / `finally` is for. A body whose last verb is `write` or `bytes` needs neither line — but the moment a chain ends in a measurement rather than an output, its tail is yours to close.

There is a second guarantee inside `reading` that is easy to miss. The whole block runs inside `Cv.attempt`, so a `CvError.NativeCall` thrown by a transform in your chain comes back as a `Left` rather than escaping past the `Either` you are already holding. The return type is honest about the whole body, not only about the read.

Because `Image` is `AutoCloseable`, `scala.util.Using` manages one too, which is the form to use when the image did not come from a path:

```
Using(Image.fromBufferedImage(frame))(_.gray.canny(80, 160).bytes(".png"))
```

Both spellings end at the same place. `reading` is tidier when the pixels come from a path; `Using` covers everything else, and the moments when an image shares a scope with a detector or a video writer.

The two escape hatches

`Image` is a convenience, never a wall, and the two members that prove it are worth keeping straight because they differ on exactly one point: who frees the `Mat` afterwards.

`mat` *borrows*. The `Image` still owns the buffer, still frees it at its terminal, and you must not call `release()` on what you were handed. Use it for any `org.opencv.*` call this type does not wrap, and for the mid-level extension operations, which take a borrowed `Mat` and return an owned `Managed[Mat]`.

`managed` *hands ownership over*. It spends the `Image` — the handle is dead afterwards, exactly as after `write` — but it does not free anything. What you get back is a live `Managed[Mat]` that is now yours to `use`, to `release()`, or to hand to `Image.wrap` to build a new `Image` around.

Example 4-7. Down to the mid-level tier and back.

```
val page = Image.read("page.jpg").getOrElse(sys.error("no page"))

// Borrowed: the Image still owns the Mat and will free it.
val edgeBytes = page.mat
  .cvtColor(ColorConversion.BgrToGray)
  .pipe(_.canny(80, 160))
  .use(Images.encode(_, ".png"))

page.close()                                        // still ours, still to be
closed

// Handed over: nothing frees this but you.
val canvas: Managed[Mat] = Image.blank(640, 480).managed
val png = canvas.use(Images.encode(_, ".png"))   // released at the end of `use`
```

The mid-level chain has the same one-live-buffer property, spelled with different words: `cvtColor` returns an owned `Managed[Mat]`, `pipe` releases its input once the next operation has produced its own, and `use` releases the last one after `Images.encode` returns. What none of them touches is `page`'s buffer — `mat` lends it out and takes it back — which is why the `close()` is not optional.

> **NATIVE MEMORY**
> `managed` is the only exit from `Image` that leaves live native memory behind. Drop the returned `Managed[Mat]` on the floor and it leaks precisely as any stray `Managed` would — the `Image` that used to be responsible for it is spent and will never run its terminal. Give the result to `use`, release it in a `finally`, or wrap it straight back into an `Image`.

Why `faces` is not a member

`Image` has more than sixty members and none of them mentions a face, a QR code or a skeleton. That is not an omission. Detection and its overlays arrive as **extension methods**,

brought in by `import scalacv.vision.*` beside core's `import scalacv.*`, and they read at the call site exactly as members would:

Example 4-8. Extension methods, indistinguishable at the call site.

```
import scalacv.*

// `detector` is a FaceDetectorYN loaded from a model file --- see Chapter 24.
Image.reading("crowd.jpg") { img =>
  val found = img.faces(detector)          // extension, in FaceDetect.scala
  img.markFaces(found)                     // extension, in FaceDetect.scala
     .drawText(s"${found.size} faces", Point(10, 30))   // member
     .write("marked.png")                  // member
}
```

The reason for the split is what the class is **about**. `Image` is about an image: its size, its pixels, the operations that turn one image into another. Face detection is about faces — it needs a `FaceDetectorYN` you loaded from a model file, it returns a `Face` with landmarks, and it belongs with the rest of the face code. Putting it on the class would mean the class grows a member for every vision application anyone ever adds, and the core module would have to know about ONNX networks, ArUco dictionaries and OCR engines to compile at all. So `qrCodes` and `arucoMarkers` live in `Detectors.scala`, `faces` and `markFaces` in `FaceDetect.scala`, `detectHaar` in `Cascades.scala`, `arMarkers` and `drawMarkerAxes` in `Ar.scala`, `forOcr` in `Ocr.scala`, `drawSkeleton` in `Pose.scala` — every one of them in the separate `scalacv-vision` module, which depends on the core and not the other way round.

The discipline survives the move. An extension method is built out of the same members you have, so it is a query or a transform for the same reasons they are: `qrCodes` and `detectHaar` read `img.mat` and are therefore queries, while `forOcr` — which is `gray`, then `medianBlur`, then `adaptiveThreshold`, then `deskew` — is a transform, and spends the image like any other chain of four. You do not have to learn a second set of rules for the extension surface, and `contours` stayed a member precisely because it is not a domain: it is core image processing that any binary image wants.

> **NOTE**
> The internal helper `paint`, which the library's own overlay verbs are built on, is `private[scalacv]` — visible to `Pose.scala` and friends, not to you. That visibility exists so the library can put its domain verbs in their own files, not as a public extension point. To write your own overlay, draw through the borrowed `img.mat` and return the image unchanged; the mid-level draw operations mutate in place, so the caller's next verb consumes it exactly as it would after a built-in draw.

How an `Image` fails

Four things go wrong on this type, and they are deliberately not made to look alike.

Table 4-3. What each kind of failure does, and what you write to handle it.

What happened	How it surfaces	What you do
A boundary operation failed — `read`, `decode`, `write`, `bytes`	`Left(CvError)`	handle the value
OpenCV rejected the pixels at runtime	throws `CvError.NativeCall`	wrap in `Cv.attempt` if you want a value
An argument the library can check is wrong	throws `IllegalArgumentException`	fix the call
A spent handle was used again	throws `IllegalStateException`, `copy` or restructure the chain	

Transforms do not return `Either`, and that is a decision rather than an oversight. A chain of twenty `Either`-returning steps is unreadable, and the failures in question are rare and data-dependent: a `canny` on an image with the wrong depth, an `equalizeHist` on three channels when it wants one. When OpenCV throws for one of those, the library wraps it in `CvError.NativeCall`, which names the operation and keeps OpenCV's own message verbatim. It is unchecked, so it is invisible at the call site — which is the honest trade for a chain that reads like a sentence.

Argument mistakes that the library can see without touching a pixel are rejected before the native call, with `IllegalArgumentException` and a message containing the value you passed: `blur(-1)`, `scale(0.0)`, `medianBlur(0)`, a `blend` weight outside `[0, 1]`, a `crop` rectangle that does not fit. Those are bugs in your code and are not meant to be pattern-matched.

When you do want a transform's throw as a value, `Cv.attempt` folds it into one:

Example 4-9. Folding an unchecked throw back into an Either.

```
val edgeCount: Either[CvError, Int] =
  Cv.attempt("edge count"):
    val edges = page.gray.canny(80, 160)
    try edges.contours().size finally edges.close()
```

Note what the `try` / `finally` is for: `contours` is a query, so nothing in that block releases the image, and the block has to. `Cv.attempt` catches OpenCV's exceptions and the library's own `CvError`s; it deliberately lets `IllegalArgumentException` and `IllegalStateException` through, because a programmer error should not come back as a `Left` you might quietly log. Chapter 6 works through the full `CvError` hierarchy and the reasoning behind that line.

Putting it together

The running pipeline, assembled: read a photograph of a page, keep a reduced-size preview for a human and a binarised version for an OCR engine, count the ink blocks, and free everything on every path.

Example 4-10. One read, two deliberate copies, two terminals, nothing leaked.

```
import scalacv.*

def process(path: String): Either[CvError, Int] =
  Image.reading(path) { page =>
    val binary = page.copy.gray.threshold(128)                 // a copy, spent
by the chain
    val blocks = try binary.contours() finally binary.close()  // a query, so
this tail is ours

    for
      _ <- page.copy.scale(0.25).write("preview.jpg")          // copy borrows,
scale consumes it
      _ <- page.forOcr().write("ocr-input.png")                // consumes `page`
itself
    yield blocks.size                                          // plain data,
safe once all is freed
  }.flatten
```

Read it against the three kinds. `copy` borrows, so `page` survives both branches and is spent only by the last of them. `scale`, `gray`, `threshold` and `forOcr` each consume what they were called on, so no intermediate outlives its successor. `contours` borrows and returns value types, which is why `blocks.size` is legal on the `yield` line, long after every `Mat` in the function has been freed — and, because it borrows, why `binary` is the one image here that has to be closed by name. The two `write` calls are terminals and free the images they were called on.

The interesting path is the failing one. If the preview write fails — an unwritable directory, an extension OpenCV has no encoder for — the comprehension short-circuits and `page` is never consumed at all. Nothing in the body frees it, and nothing has to: `Image.reading` closes

it on the way out, which is a no-op on the happy path and a rescue on this one. The `flatten` is there because the block returns an `Either` of its own and `reading` wraps it in a second; two failure channels, one shape at the call site.

What it does **not** have is a `release`, a `Mat`, or a cleanup block per step. One `finally`, on the one value a query left alive, is the whole of the accounting. That is the trade the type makes: one rule to hold in your head — a transform spends its receiver — in exchange for never doing the rest of the bookkeeping yourself.

Chapter 5 goes down one tier, to the layer this one kept behind a curtain: what `Managed[A]` is and why it can release exactly once, how `Releasable` gives a `release()` to the 185 `org.opencv.*` types that were never given one, and how `Managed.scope` owns the six handles a single `solvePnP` needs without six levels of indentation. Chapter 6 takes up the error model: what each `CvError` case actually means, where the line between a returned failure and a thrown one falls, and how to build a pipeline that reports the difference to whoever is on call.

CHAPTER 5

Lifetimes: Managed, Releasable, and Scope

How a library frees native memory the JVM cannot see, and refuses to free it twice.

A service decodes frames from a camera, runs a detector over each one, writes a JPEG, and repeats. It runs for six minutes and gets OOM-killed. You take a heap dump and find nothing: the heap is flat at 300 MB, the collector has run happily throughout, and the largest retained set is a logging buffer. Meanwhile the resident set size of the process, which is the number the kernel actually killed you over, went to six gigabytes.

Nothing is wrong with your Java. The problem is that an OpenCV `Mat` holds its pixels off-heap and keeps about forty bytes of Java object on-heap to point at them. A 1000 × 1000 three-channel image is three megabytes of `cv::fastMalloc` behind forty bytes the collector can see. The collector runs when the heap comes under pressure, and heap pressure has nothing to do with native pressure — so the loop exhausts the machine while the JVM stays relaxed. Measured on this project's own test machine: 2000 × `Mat(1000, 1000, CV_8UC3)`, references dropped, no explicit `System.gc()`, and the process finishes at **5 865 MB** of RSS. The identical loop calling `release()` finishes at **144 MB**. That is 41×, and it reproduces on JDK 21 and JDK 25 alike.

The mechanism that is supposed to save you here is `finalize()`, and the generated OpenCV bindings do carry one. It is not disabled — that is a common myth — but it only runs when the collector runs, and the loop above is exactly the case where it will not. Waiting for a finalizer to reclaim native memory is waiting for an event whose trigger is uncorrelated with the thing you need reclaimed. It is also deprecated for removal.

There is a second problem behind the first, and it is worse. Of the 188 `org.opencv.*` types that own native memory, exactly **three** — `Mat`, `VideoCapture` and `VideoWriter` — expose a public `release()`. `CascadeClassifier`, `Net`, `QRCodeDetector`, `ArucoDetector`, `KalmanFilter` and 180 others do not. There is no method to call. So a library that wants to free them has to answer two separate questions: *when* does a native object get freed, and *how*. `Managed[A]` answers the first. `Releasable[A]` answers the second. `Managed.scope` answers the case where one operation needs six live handles at once and none of them may outlive it. This chapter is those three types, and it is the chapter the rest of the book leans on.

The failure this replaces

Start with the mistake, because the shape of the mistake explains every design decision that follows.

Written the way OpenCV is normally written from Java, a frame loop looks like this, and it leaks:

Example 5-1. The version that gets your container OOM-killed.

```
// WRONG — nothing here is ever freed, and nothing will free it for you.
while running do
  val frame = Mat()
  capture.read(frame)
  val gray = Mat()
  Imgproc.cvtColor(frame, gray, Imgproc.COLOR_BGR2GRAY)
  val faces = MatOfRect()
  detector.detectMultiScale(gray, faces)
```

That is a leak, which is at least a failure you can watch happen. The interesting thing is what happens when you notice and start freeing by hand: the failure changes character entirely. Calling a method on an OpenCV object whose native memory you have already freed segfaults from native code. There is no stack trace, no exception to catch, no test report — the JVM writes an `hs_err` file and the process is gone. Freeing the same pointer twice is undefined behaviour that merely *often* happens to survive, which is worse, because it means your test suite passes and production does not. Both of these were reproduced on this project's machine before the guards existed; the double free was found in an unrelated test suite, where its only symptom was a worker process that died without writing a result.

That is the standard against which `Managed` should be judged. It is not competing with a neat exception; it is competing with a signal.

> **NATIVE MEMORY**
> A native handle has two ways to hurt you and they pull in opposite directions. Free it too late — or never — and you leak megabytes per iteration behind forty visible bytes. Free it twice, or touch it after freeing, and you corrupt the heap or crash the process with no Java-side evidence at all. Any lifetime scheme has to close both, and closing one carelessly opens the other.

`Managed[A]`

`Managed[A]` owns one native object and releases it exactly once. It is a `final class` implementing `AutoCloseable`, and its entire state is an `AtomicReference[A | Null]` holding the object, the `Releasable[A]` it was constructed with, a `Class[?]` captured at construction, and a nullable `Throwable` used only for diagnostics. The constructor's `initial` parameter is deliberately not among them; the subsection below says why.

Two guarantees fall out of that reference, and both exist because getting them wrong is a JVM crash rather than an exception. First, release is a compare-and-set: `getAndSet(null)` admits exactly one caller, so a second `release()` is a no-op instead of a double free, and sixty-four threads racing to release the same handle produce exactly one free. Second, access after release throws `IllegalStateException` on the Scala side, before anything crosses JNI.

Example 5-2. The whole contract, in five lines.

```
val m = Managed(Mat(64, 64, CvType.CV_8UC3))
m.get.rows              // 64 — the underlying object, borrowed
m.release()             // frees the pixel buffer
m.release()             // no-op; the CAS already took the reference
m.get                   // throws IllegalStateException, not SIGSEGV
```

The API is small enough to put in one table.

Table 5-1. The `Managed[A]` surface.

Member	What it does
`get: A`	The underlying object, borrowed. Throws `IllegalStateException` if the handle has been released or consumed — deliberately eager, because the alternative is a segfault.
`use[B](f: A => B): B`	Runs `f` on the object and releases afterwards, in a `finally`, so the release happens on an exception too. Prefer this over holding a handle.
`release(): Unit`	Frees the native memory through the `Releasable` supplied at construction. Idempotent.
`close(): Unit`	`release()`, under the name `AutoCloseable` wants. This is what makes the whole of `scala.util.Using` accept a `Managed`.
`isReleased: Boolean`	Whether the reference has been taken — by a release or by a transfer.
`toString: String`	`Managed(<released>)` or `Managed(…)`, from a single snapshot of the reference so a concurrent release cannot make it print `Managed(null)`. Diagnostics only.
`Managed(a)`	Wraps `a`, requiring a `Releasable[A]` in scope.
`Managed.use(a)(f)`	Wrap and scope in one call — the scaladoc's "form to reach for by default". `a` is by-name, so it is not evaluated until the wrap happens.
`Managed.scope(body)`	Owns several objects for the duration of `body`. Its own section below.

Because `Managed` is `AutoCloseable`, `scala.util.Using` and `Using.Manager` already accept it through `Using.Releasable.AutoCloseableIsReleasable`. The library deliberately does not define its own `given` for that, which sounds like a missed opportunity and is not: defining one made every `use(Managed(...))` call ambiguous between the two instances.

Why a `Class`, and not the object

One field in `Managed` looks redundant and is not. The class captures `initial.getClass` into a private `ofType` at construction so that the error message can name the type — "this Mat has already been released" — without the `Managed` holding a reference to the object itself.

Keeping the constructor parameter alive would defeat the point of nulling the reference in `release()`: the released object would stay strongly reachable for as long as the handle lived, and the small `cv::Mat` header it owns — the part the collector reclaims rather than `release()` — could never be collected. A `Class` costs one field and its classloader already keeps it alive.

`take`, and why it is not yours to call

There is one more method on `Managed`, and it is `private[scalacv]`:

```
private[scalacv] def take(): A
```

`take()` hands the object out and leaves the handle spent *without freeing it.* Ownership transfers to the caller. It exists for the high-level `Image`, which threads one live `Mat` through a chain of handles: an in-place step takes the `Mat` out of the current handle, mutates it, and rewraps it in a new one. The predecessor handle is now spent — `get` throws — yet nothing has been freed and nothing has been copied. `Image.paint` is the internal that does this; the drawing verbs built on it, and `Image.managed`, are what you see of it from outside.

`take` is deliberately not `release()`, which would free the very object being transferred, and it is deliberately not public. A public `take()` would be a hole straight through the ownership model: a caller could extract a live `Mat` from a handle, drop the handle, and have no owner left to free it. If you want a second independent object, `Image.copy` clones the pixel buffer and gives you one.

A pure transform such as `img.gray` spends its receiver differently: `Image.transform` runs the mid-level op on the borrowed `Mat`, wraps the fresh `Mat` it produced, and releases the source in a `finally`. Either way the receiver ends up spent, which is the move semantics `Image` presents; only the in-place path transfers a live pointer, and only that path needs `take`.

Several handles at once: `Managed.scope`

`Managed.use` scopes exactly one object. Real OpenCV calls frequently want more than one. `solvePnP` takes object points, image points, a camera matrix, distortion coefficients and two output vectors — six native objects, all of which someone has to free, none of which the caller wants back.

The obvious way to write that in Scala is the one with a hole in it:

Example 5-3. The `try`/`finally` shape, which is not as safe as it looks.

```
// WRONG — every allocation before the `try` is unguarded.
val objectPoints = MatOfPoint3f(model*)
```

```
val imagePoints  = MatOfPoint2f(points*)
val camera       = buildCameraMatrix()   // if this throws, the two above are
stranded
val rvec         = Mat()
val tvec         = Mat()
try
  Calib3d.solvePnP(objectPoints, imagePoints, camera, dist, rvec, tvec)
finally
  objectPoints.release(); imagePoints.release(); camera.release()
```

The `finally` only protects what was allocated before control reached the `try`. A constructor that throws part-way — a degenerate landmark set, a bad calibration file — strands everything already built. Nesting six `Managed.use` blocks closes that hole and buries the two interesting lines under six levels of indentation.

`Managed.scope` is both, correctly. It hands your body a `Scope`, conventionally named `own`, which registers an object and returns it, so a scoped acquisition reads as an ordinary binding.

Example 5-4. Head-pose estimation, as the library actually writes it.

```
Managed.scope: own =>
  val objectPoints = own(MatOfPoint3f(model*))
  val imagePoints  = own(MatOfPoint2f(face.landmarks.map(_.toCv)*))
  val camera       = own(intrinsics.cameraMatrix)
  val distortion   = own(intrinsics.distCoeffs)
  val rvec         = own(Mat())
  val tvec         = own(Mat())
  Cv.attempt("solvePnP") {
    val ok = Calib3d.solvePnP(
      objectPoints, imagePoints, camera, distortion, rvec, tvec,
      false, Calib3d.SOLVEPNP_EPNP
    )
    if !ok then None
    else
      val rotation = own(Mat())
      Calib3d.Rodrigues(rvec, rotation)
      val euler = Calib3d.RQDecomp3x3(rotation, own(Mat()), own(Mat()))
      Some(HeadPose(yaw = euler(1), pitch = euler(0), roll = euler(2)))
  }.getOrElse(None)
```

That is `HeadPose.estimate` from `vision`, with the `org.opencv.calib3d` prefixes folded into `Calib3d.` for width. Copy the shape, not the snippet: `Intrinsics.cameraMatrix`, `distCoeffs` and the `_.toCv` on a landmark are all `private[scalacv]`. Everything structural is public — `Managed.scope`, `own`, `Cv.attempt`.

The important word is *registered*. Each object joins the scope the moment it is created, not at the end of a declaration block, so a throw anywhere — in a later constructor, in the native call, in the decode afterwards — releases everything acquired so far, in reverse acquisition order. Note the three `own(Mat())` calls that appear part-way down, inside the `else` branch: they are registered no less reliably than the six at the top, because registration is a function

call rather than a syntactic position. Underneath, `scope` is `Using.Manager`, so the exception a failing body throws propagates unchanged and a failure raised by a *release* is attached to it as a suppressed exception rather than replacing it. The original cause is never lost.

`own` hands back the object itself rather than a `Managed`, which is a deliberate asymmetry with the rest of the library. A scoped handle has no second owner to be protected from: the scope releases it exactly once, at the end, so there is nothing for a wrapper to guard against and every `.get` it would force you to write would be noise.

`adopt`, for values that arrive already wrapped

Every mid-level `Ops` transform hands back a `Managed[Mat]`. Passing one to `own` would wrap a handle in a second handle. `adopt` takes over the existing one instead, so the `Mat` is still released exactly once:

Example 5-5. Mixing raw constructions and mid-level results in one scope.

```
val edgePixels = Managed.scope: own =>
  val src   = own(Mat.zeros(64, 64, CvType.CV_8UC3))
  val gray  = own.adopt(src.cvtColor(ColorConversion.BgrToGray))
  val edges = own.adopt(gray.canny(80, 160))
  Core.countNonZero(edges)
```

`adopt` is a separate name rather than an overload of `apply` for a reason worth knowing: `Managed[A]` is itself an `A` as far as overload resolution is concerned, so the compiler would have to choose between the two candidates before searching for the `Releasable` that distinguishes them. A separate name sidesteps a resolution puzzle that would have had a silently wrong answer.

> **WARNING**
> Nothing acquired inside a `scope` may escape it. The value the body returns must be plain data — a number, a `Seq[Double]`, a case class — or an object owned somewhere else. Returning a scoped `Mat` gives you a handle to freed memory, and the next method call on it is a segfault. This is the same rule `use` already carries, applied to a group; in the listing above, `countNonZero` returns an `Int`, and in the head-pose listing the body returns a `HeadPose` of three doubles.

`Releasable[A]`: how a thing gets freed

`Managed` decides *when*. `Releasable` decides *how*, and it is a one-method typeclass:

```
trait Releasable[-A]:
  def release(a: A): Unit
```

It is contravariant in `A`, so an instance written for a supertype serves every subtype. Which instance applies to a given OpenCV type is not a style choice — it is dictated by what the generated Java binding exposes — and there are three shapes.

Table 5-2. The three ways a `Releasable` gets built.

Shape	Applies to	How it frees
The three built-in `given`s	`Mat`, `VideoCapture`, `VideoWriter`	Calls the type's own public `release()`. Nothing reflective, nothing to configure.
`Releasable.handle(getNativeAddr)`	A binding whose address is not a `long` field called `nativeObj`	Reads the address through the function you supply, then goes through the shared free sequence — which still dis-

`Releasable.nativeHandle`	The other 185 `org.opencv.*` types	Reads `nativeObj` reflectively, then goes through the same sequence. No accessor to supply, so none to get wrong.

The three built-in instances need no import ceremony — they live in the `Releasable` companion, so they are found by the implicit search automatically. `Managed(Mat(...))` compiles with nothing but `import scalacv.*`.

For the other 185, you declare an instance where you need one. The library does this at fourteen sites across `vision` and `graphs`, and more again in its tests and examples; they all look alike:

```
private given Releasable[KalmanFilter] = Releasable.nativeHandle
```

Some are public, so you can borrow them: `Cascades` exposes `given Releasable[CascadeClassifier]`, `Dnn` exposes `given Releasable[Net]`, `FaceDetect` exposes `given Releasable[FaceDetectorYN]`. They live in those objects rather than in the `Releasable` companion, so `import scalacv.*` does not find them — name them, with `import Cascades.given` or `import Dnn.given`.

What `nativeHandle` actually does

`nativeHandle` takes no accessor. It reads the address out of the binding's own `nativeObj` field reflectively, walking up the class hierarchy for a field of that name and type `long`, caching the opened `java.lang.reflect.Field` per class in a `ConcurrentHashMap`.

Reading a field rather than calling `getNativeObjAddr` sounds like the weaker choice and is not. The disarm step described in the next section has to reflect on that exact field on this exact code path anyway, so the `Field` is already open and cached by the time the address is wanted — reading it costs nothing beyond a lookup that was going to happen regardless. It also fails in the same place, with the same message, as the disarm would, rather than succeeding at reading an address the disarm then refuses to neutralise.

The reason it exists at all is duplication. Every one of the 185 types spells its accessor identically, so the `handle` call naming it was written out the same way at every site — and every one of those was a chance to hand one type another type's accessor, which compiles cleanly and frees the wrong pointer. `nativeHandle` cannot be given the wrong one. `handle` stays for a binding whose address lives somewhere other than a `nativeObj` field; it lifts no other requirement, because the free sequence it shares still disarms `nativeObj` and still calls the class's own `delete(long)`.

Why `Mat` is not freed the way everything else is

It would be tidier if all 188 types went through one mechanism, and the library deliberately does not do that.

`Mat` exposes no accessible `delete(long)` — only its superclass's private `CleanableMat.n_delete` — so the bridge the other 185 use is not available to it in the first place. But even if it were, it would not be worth taking. `Mat.release()` drops the reference to the pixel *buffer* at once, and the buffer is the multi-megabyte allocation that actually costs you memory. What `release()` leaves behind is the `cv::Mat` *header*, roughly a hundred bytes, reclaimed by the inherited `CleanableMat.finalize()` whenever the collector next gets round to it.

A hundred bytes on a collector's schedule is a real cost and a very small one. Paying reflection, a cached `MethodHandle` and a finalizer disarm to reclaim it a little sooner is not a trade worth making. The part that matters is already gone the instant `release()` returns.

The finalizer, and the double free it would cause

Here is the part that makes freeing the 185 types genuinely difficult rather than merely fiddly.

Every one of those generated handle classes carries this, verbatim:

```
protected void finalize() throws Throwable { delete(this.nativeObj); }
```

It is unconditional. It does not check whether the pointer is still live, because from the binding's point of view there was never a way for it not to be. So if the library reads the address, calls `delete(addr)`, and then drops the Java object, `delete` runs *twice* on the same address: once from the library, and once from the finalizer thread whenever the collector next runs. That is heap corruption. It does not fail at the second delete — it surfaces as a `double free or corruption` message or a SIGSEGV somewhere else entirely, at an unpredictable later moment, which is exactly how it was originally found.

`Managed`'s compare-and-set is no defence here. It makes the library's *own* release idempotent; it knows nothing about a finalizer running on another thread against an address it has never seen.

The fix is to zero the field first. `NativeFinalizer.disarm` writes `0L` into `nativeObj`, so the finalizer's eventual `delete(this.nativeObj)` becomes `delete(0)`, which C++ defines as a no-op on a null pointer. The order in the shared free sequence is therefore fixed, and commented as such in the source:

Example 5-6. The release sequence both non-`Mat` shapes share.

```
private def free[A <: AnyRef](a: A, addr: Long): Unit =
  if addr != 0L then
    // Disarm BEFORE deleting, never after: between the two there is a window in
which the
    // finalizer could run against a pointer we have already freed.
    NativeFinalizer.disarm(a)
    NativeDelete.of(a.getClass).invokeExact(addr): Unit
```

Read the guard first: an address of zero means the object was already freed, or never held one, so the whole sequence is skipped. Then note the order. Reversing those two lines would leave a window — short, but real — in which the collector could run the finalizer against a pointer already handed to `delete`. The library's own test asserts the mechanism rather than inferring it from survival: allocate a `CascadeClassifier`, check `getNativeObjAddr` is non-zero, release it through a `Managed`, and assert `getNativeObjAddr` is now exactly zero.

> **NATIVE MEMORY**
> This is why you must not free one of the 185 handle types by reaching for its private `delete` bridge yourself, even if you find it. Freeing without disarming is a double free with a delay fuse on it, and the delay is however long it takes the collector to notice a forty-byte object. Put the

> handle in a `Managed` with a `Releasable.nativeHandle` instance and let the ordering be someone else's problem.

When reflection is refused

Both reflective steps can fail, and neither degrades quietly. `delete(long)` is private API with no compatibility promise, and `setAccessible` stops working the moment OpenCV's classes are loaded from a named module rather than the classpath — something a consumer controls through `--add-opens` and the library cannot.

Every one of those failures throws `CvError.NativesMissing` rather than falling back to the collector, on the grounds that an unbounded leak which looks like success is a worse outcome than a loud failure. There are five distinct refusals, and the message tells them apart.

Table 5-3. Every way the reflective release refuses, and what it says.

What could not be done	What the message reports
No `long nativeObj` anywhere in the class hierarchy	The type has no field to disarm, so it cannot be freed safely. This is the one case no accessor argument fixes — `handle` shares the same disarm.
`setAccessible` on `nativeObj` refused	The field cannot be made writable, with the computed `--add-opens` remedy. `nativeHandle` also fails here when it goes to read the address, because it reads through the same cached `Field`.
Writing `0L` into `nativeObj` refused	Refusing to free it, "because the binding's finalizer would then free it a second time".
The class declares no `delete(long)`	These bindings are a build the library does not know how to free; the message asks for the bytedeco version.
`setAccessible` on `delete(long)` refused	The bridge cannot be opened, with the same remedy and a line saying the library fails here rather than falling back to the collector, "because that fallback does not reclaim native memory in any useful timeframe".

The second and third rows are the ones that matter most. If the address can be read but `nativeObj` cannot be written, the library *refuses to delete at all*: a leak is recoverable where a corrupted heap is not.

The remedy is computed from the class's own module and package rather than guessed at:

```
cannot make org.opencv.objdetect.CascadeClassifier.nativeObj writable
(InaccessibleObjectException).
```

```
scalacv must zero this field before freeing the object, because the binding's
finalizer calls delete(nativeObj) unconditionally and would otherwise free the
same pointer twice.

  --add-opens <module>/org.opencv.objdetect=ALL-UNNAMED

scalacv refuses to free the object rather than risk corrupting the heap.
```

`<module>` there stands for whatever `cls.getModule.getName` returns, because `addOpensRemedy` computes the line at the moment it fails rather than hard-coding a guess. If the class is in the unnamed module — OpenCV on the classpath, which is the normal case — there is no module to open and therefore no flag that would help. The library says so in those words, and asks for a bug report, instead of printing an `--add-opens` line with a `null` module name in it.

Finding the line that spent a handle

The commonest lifetime error in practice is not a leak and not a double free. It is reuse: an `Image` touched after a transform already consumed it. The `IllegalStateException` fires at the reuse, which is almost never the line you need to look at.

Example 5-7. The error arrives one line too late to be useful.

```
val img  = Image.blank(16, 16)
val gray = img.gray     // this line spent `img`
gray.close()
img.width               // …and this line throws
```

The default message does what it can with that: it names the type, states the fix, and points at the flag that would locate the cause.

```
this Mat has already been released or consumed — using it now would crash the
JVM from native code. A high-level Image is spent by any transform (gray/blur/…)
or terminal (write/bytes/close); call `.copy` before the first use if you need it
twice. Run with -Dscalacv.trackOwnership=true to record where it was consumed.
```

It names `Mat`, not `Image`, and that is not a slip: an `Image` wraps a `Managed[Mat]`, and the captured `Class` is the class of the thing actually owned. The rest of the sentence is written for the case that produces it — an `Image` used after a move — which is why a `Managed[Mat]` talks about transforms, terminals and `.copy`.

Start the JVM with `-Dscalacv.trackOwnership=true` and every handle that is spent — by a `release()` or by an internal `take()` — records a `Throwable` at the moment it is spent. When a spent handle is later touched, that `Throwable` is attached as the *cause* of the `IllegalStateException`, so the stack trace you get names the transform or terminal that

actually consumed the handle rather than only the line that tripped over the result. The trailing sentence about the flag disappears from the message, because it is no longer needed.

> **TIP**
> The flag is read once at class load through `java.lang.Boolean.getBoolean("scalacv.trackOwnership")`, so it must be set on the command line — setting the property from inside `main` is too late. Leave it off in production: it allocates a `Throwable` every time a handle is spent, which on a frame loop is every frame. The read that consumes it lives only on the already-failing path, so a program that never misuses a handle pays nothing for having the feature, only for having it switched on.

Which level to reach for

Four tiers, and the right answer is nearly always the first one.

Table 5-4. Choosing an ownership tool.

Reach for	When
`Image.reading(path) { … }`	Almost always. The block's image is released when it returns — on success, on failure, and on exception — and every intermediate in a transform chain frees itself as the next stage consumes it. Keep borrows and lazy work inside the callback; explicit ownership escapes still need cleanup.
`Managed.use(a) { … }` or `.use`	One native object, one scope, and the result is not a native object. The mid-level `Ops` layer hands you `Managed[Mat]` values that fit this directly.
`Managed.scope { own => … }`	Two or more native objects live at the same time, and the answer is plain data. Anything that calls into `Calib3d`, or builds a detector plus its inputs plus its outputs.
A bare `Managed` you hold	A handle whose lifetime genuinely outlives any lexical scope — a detector loaded once at startup and used for the life of the service. Close it in your shutdown path.

In the current implementation, consuming transforms, copies and `.managed` / `pipe` keep the callback scope responsible for their live successors, even after the original wrapper is spent. Use `img.detach` to transfer one branch out explicitly, and close the returned image yourself; other branches remain scoped. The custom release strategy moves with its owner. These fixes are newer than the 0.4.1 source tag. Raw Mat allocations and arbitrary concurrent use are not covered.

The one thing to avoid is a scope whose result is the thing the scope owns. It is easy to write by accident, because `use` returns whatever the body returns and the type checker is perfectly happy:

Example 5-8. A leak the compiler will not catch.

```
// WRONG — `use` frees the blur output, then returns the canny Mat, which now
// outlives its own Managed and has no owner left to free it.
src.gaussianBlur(Size(5, 5)).use(_.canny(50, 150))

// RIGHT — `pipe` feeds the intermediate forward and frees it, handing back an
// owned Managed. Reach for `use` only at the terminal stage that produces a
// non-Mat: a count, a Seq[Rect], some bytes.
src.gaussianBlur(Size(5, 5)).pipe(_.canny(50, 150))
```

What getting it wrong looks like from outside

You will not see a leak in a heap dump, and you will not see it in JavaCPP's own accounting either. This project's memory audit established that `Pointer.totalBytes()` is **blind** to these allocations: a deliberate 1.4 GB `org.opencv.core.Mat` leak moved it by zero bytes, because those buffers come from OpenCV's own JNI through `cv::fastMalloc` rather than through JavaCPP's tracked allocators. The `-Dorg.bytedeco.javacpp.maxBytes` budget derived from `totalBytes()` cannot gate a leak it cannot see.

The signal that does see them is process RSS. The library's own leak suite reads it from `/proc/self/statm` on Linux, falling back to `Pointer.physicalBytes()` elsewhere, and runs each workload 40 times to warm up and then 300 times measured, asserting that RSS grows by no more than 48 MB across the measured run. The tolerance is bounded rather than zero on purpose — allocator arenas and the JIT code cache mean RSS never returns exactly to baseline — but a per-iteration leak of even a modest `Mat` clears 48 MB long before iteration 300, while a correct workload stays flat. That suite gets a JVM to itself, because RSS is process-global and any other suite running concurrently would contaminate it.

The failure mode is dramatic when it is a handle type rather than a `Mat`. Four thousand leaked `KalmanFilter`s measured **54 GB**. The same four thousand, released, measured **86 MB**.

> **TIP**
> To catch a leak in your own workload before RSS catches it for you, cap JavaCPP's physical budget and run: `java -Dorg.bytedeco.javacpp.maxPhysicalBytes=512M -jar your-app.jar`. That ceiling is RSS-derived, so unlike `maxBytes` it does see these buffers. A leak then fails fast and loud instead of slowly eating the machine. Run the same workload under both a tight and a generous cap, so you can tell a genuine leak from a ceiling that is too small for your working set. Note what the cap throws when it fires: a `java.lang.OutOfMemoryError`, not a `CvError`, so no `Either` in this library catches it and no `Cv.attempt` turns it into a `Left`. In a container, set it comfortably below the container's own limit, so the JVM produces that error with a stack trace instead of the kernel producing an OOM kill with nothing.

Next

Every guard in this chapter throws. `Managed.get` on a spent handle throws `IllegalStateException`; `Releasable.nativeHandle` throws `CvError.NativesMissing` rather than leaking or corrupting the heap. That is not the library's answer to every failure, and it is not

meant to be — a missing file, an unreadable JPEG and a `cvtColor` given the wrong channel count are not bugs in your program, and they do not deserve an exception. Chapter 6, *The Error Model*, draws that line: which failures come back as `Either[CvError, A]` values you are expected to handle, which ones throw because they mean your code is wrong, and why the library refuses to blur the two. You have already met one of its cases in this chapter — `CvError.NativesMissing`, the one the reflective release path raises when the module system will not let it do its job.

CHAPTER 6

The Error Model

Which failures are values, which are bugs, and why the library refuses to blur the two.

Ask OpenCV's Java API to read a file that is not there and it does not complain. `Imgcodecs.imread` hands back a `Mat` — an ordinary object, non-null, ready to be passed around — whose `empty()` is `true`. Your program carries on. Ten lines later a `cvtColor` is asked to convert that emptiness from BGR to grey, and *that* is where the failure surfaces: an `org.opencv.core.CvException` whose message names a C++ assertion in a file you have never opened, several call frames from the mistake you actually made.

That is one of three shapes failure takes in the bindings. `imwrite` returns a bare `false`, with no indication whether the directory was missing, the extension unsupported, or the disk full. `imencode` returns `false` for one class of problem and throws for another. A `CascadeClassifier` constructed on a wrong path throws nothing at all: you get a classifier that loads no cascade and finds nothing, forever, quietly. Three incompatible conventions, and no rule connecting any of them to the kind of mistake being reported.

There are two tempting ways to flatten that, and both are wrong. Wrap every call in `scala.util.Try` and every failure becomes a value — including the ones that are bugs, which then travel through your `recover` blocks and get logged at `WARN` instead of stopping the program at the line that was wrong. Go the other way and make the whole API total, returning `Either` everywhere, and the type is a lie: `CvException` escapes from ordinary in-memory operations, on Mats the library never touched, and no wrapper can prevent it.

So scalacv asks one question of every failure, and answers it once, in the type. *Could a correct program, given different data, hit this?* A file the user chose is not there; a download 404s; a photograph is too blurred for the corner finder. Those are outcomes, and they come back as `Either[CvError, A]`. A negative blur radius, an image reused after a transform consumed it — no change of input makes those correct. They are bugs, and they throw.

The running example for this chapter is small enough to hold in your head and touches every case: a service that takes an uploaded photograph, finds the faces in it, draws a box round each one, and writes a JPEG. It has a data-dependent failure at each end, a resource to load, a native call in the middle, and at least two ways to hold it wrong.

The dividing line

Eight kinds of thing can go wrong, and the library places each one in a regime deliberately.

Table 6-1. Where each failure is delivered, and what you are expected to do about it.

Failure	Delivered as	What you do
Missing or undecodable image	`Left(DecodeFailed)`	branch on it
Missing model, cascade, video source	`Left(LoadFailed)`	branch on it
Unwritable destination, no encoder	`Left(EncodeFailed)`	branch on it
Ill-posed calibration	`Left(CalibrationFailed)`	capture more views
Unforeseen native rejection	`NativeCall` — `Left` from `Cv.attempt`, otherwise thrown	usually a bug; sometimes handled
Natives absent, or the release bridge shut	thrown `NativesMissing`	fix the build or the JVM flags
Bad argument	thrown `IllegalArgumentException`	fix the call
Reusing a spent handle	thrown `IllegalStateException`	fix the code

The first four rows are the `Either`. They are the failures a correct program meets in the course of doing its job, and they are the ones the type system should be talking about. The last three throw, and the fifth is the interesting hybrid: a native rejection is usually a bug, but not always, so it is available both ways.

Why the ADT is an exception hierarchy

`CvError` is declared `sealed abstract class CvError(message: String, cause: Throwable | Null) extends RuntimeException(message, cause)`. An error ADT that extends `RuntimeException` looks like a compromise, and it is one — a considered one.

The core cannot be made total. `CvException` comes back across JNI from operations this library did not initiate and cannot foresee, so somewhere a `Throwable` has to be caught and carried. Because `CvError` *is* a `Throwable`, it carries: it can hold the original as its `cause` without a lossy conversion, it can be rethrown by `Cv.orThrow`, and it can cross a `try` / `catch` written by somebody who has never heard of this library.

What you give up is nothing much: the hierarchy is `sealed`, so `match` is still exhaustive. What you gain is a type that behaves correctly at the one boundary where the language cannot help you.

Six cases

`CvError` has exactly six shapes, and each one is named for the moment you meet it.

NativesMissing

The JVM cannot reach native OpenCV. The case carries `details: String` and `cause: Throwable | Null`, defaulted to `null` — the union type is there because most of these are constructed from a situation rather than from an exception, and the ones that *are* wrapping something (an `InaccessibleObjectException`, a `NoSuchMethodException`) keep it, since its message names the module or package that could not be opened.

`details` is not a description of the problem. It is the remedy, written to be copied out of your terminal and pasted into a build file or a launch script. That is why this case is thrown rather than returned: there is no sensible way to carry on, and nothing to branch on.

Despite the name, only one of its two causes happens at start-up. `OpenCv.load()` throws it when the per-platform classifier jars are absent. `Releasable`'s reflection bridge — the machinery that frees the 185 `org.opencv.*` types with no public `release()` — throws it when a native handle is freed, which is typically in the middle of a pipeline that has been working for twenty minutes. Both messages are shown later in this chapter.

DecodeFailed

`final case class DecodeFailed(path: String, details: String)`. Image bytes did not become an image. This is the case that exists because `imdecode` does not throw on rubbish — it returns an empty `Mat`, and the check has to be explicit. scalacv makes it for you and turns the result into a `Left`, so the failure surfaces at the read rather than at whichever later operation happened to be handed the emptiness.

Reading from a file adds the filesystem cases. `Images.read` does the byte-fetching with the JVM's own file I/O and leaves only the decoding to OpenCV, so `details` says which of them happened instead of collapsing them into one message: `there is no file at this path`, `this path is a directory, not a file`, `the file is empty`, `this is not a usable filesystem path` followed by the reason `Path.of` gave, or `the bytes are not an image in a format OpenCV can decode`.

> **NATIVE MEMORY**
> The empty `Mat` that a failed decode produces still owns a native handle, even with no pixel buffer behind it. `Images` releases it on the failure path before constructing the `Left`. It matters more than it sounds: one leaked handle per failed read is nothing, and one leaked handle per failed read inside a retry loop pointed at a URL that has been 404ing for a week is a leak like any other.

LoadFailed

`final case class LoadFailed(resource: String, details: String)`. A *non-image* resource could not be resolved, loaded, or verified: a Haar cascade, an ONNX network, a downloaded model, a video source, a `Recorder` whose codec the build does not have. It is kept apart from `DecodeFailed` deliberately — an HTTP 404 on a model download or a checksum mismatch is not an image-decode failure and should not read like one.

The running example meets this first, because the cascade has to be loaded before there is anything to detect with.

Example 6-1. The first fallible step: the classifier.

```
import scalacv.*
import org.opencv.objdetect.CascadeClassifier

val cascade: Either[CvError, Managed[CascadeClassifier]] =
  Cascades.load(CascadeName.FrontalFaceAlt)
```

`Cascades.load` extracts the XML from the bytedeco jar and loads it, and it reports the silent case as well as the loud one: OpenCV builds an empty classifier for a path it cannot read, so scalacv checks `empty()` and returns `Left(LoadFailed)` with a message saying so, rather than handing back a detector that finds nothing.

EncodeFailed

`final case class EncodeFailed(path: String, details: String)`. The other end of the pipeline. Two different failures share the one case so that a single `case EncodeFailed(...)` catches both: an extension with no registered encoder — checked with `haveImageWriter` up front, rather than letting `imencode` throw — and a destination the JVM cannot write to, which `details` separates into `the parent directory does not exist` and `the destination is not writable`.

`Images.write` encodes fully into a byte array and only then writes, which costs you the encoded size in heap and buys something worth having: a failed encode can no longer leave a truncated file where a valid one used to be.

CalibrationFailed

`final case class CalibrationFailed(details: String)`. `Calibration.fromChessboard` could not produce intrinsics: fewer than `minViews` of the images showed the whole board, or `calibrateCamera` did not converge. This is the purest data-dependent failure in the library — it turns entirely on what the capture actually saw — so it is returned, and the `details` tell you what to do about it, naming both numbers: how many views showed the grid, out of how many you supplied. The fix is another twenty minutes with the board, not an edit to the call.

NativeCall

`final case class NativeCall(operation: String, cause: Throwable)`. The residual case: a `CvException` from an ordinary in-memory operation. A channel-count violation, a size mismatch, a depth the operation does not accept. Note that `cause` here is `Throwable`, not `Throwable | Null` — this case never exists without something to wrap.

`operation` is the string the call site supplied, and it is the whole value the wrapper adds. OpenCV's own message is preserved verbatim and deliberately not parsed: that text is not

a stable interface, and turning it into error codes would be inventing structure upstream never promised.

Cv.attempt, and the guard that makes the policy real

Everything the library wraps already returns an `Either`. `Cv.attempt` is for when you leave the paved road — a raw `org.opencv.*` call scalacv does not cover — and want it to obey the same policy.

Example 6-2. Lifting a raw OpenCV call into the error model.

```
import org.opencv.core.{Core, Mat}

def meanBrightness(m: Mat): Either[CvError, Double] =
  Cv.attempt("Core.mean")(Core.mean(m).`val`(0))
```

The implementation is five lines and three `catch` clauses, and it is worth knowing them exactly. A `CvException` becomes `Left(NativeCall(operation, e))`. A `CvError` the block produced itself is passed through unchanged, so lifting an already-lifted call does not double-wrap it. And a *bare* `java.lang.Exception` — matched by exact class, `e.getClass == classOf[Exception]` — also becomes a `NativeCall`, because OpenCV's `throwJavaException` degrades to a plain `Exception` for anything that is not a `cv::Exception`: `std::bad_alloc`, `std::out_of_range`, and whatever it does not recognise.

That exact-class test is the load-bearing part of the whole error model. Because it demands the class be `Exception` itself, every subclass keeps travelling.

Table 6-2. What crosses `Cv.attempt`, and what does not.

Thrown inside the block	Comes out as
`org.opencv.core.CvException`	`Left(NativeCall)`
A `CvError` the block produced	`Left`, unchanged
`java.lang.Exception`, exact class	`Left(NativeCall)`
`IllegalArgumentException`	propagates
`IllegalStateException`	propagates
`InvalidPathException`	propagates
Any `Error` — `OutOfMemoryError`, `UnsatisfiedLinkError`	propagates

Read the bottom half of that table as a design statement. The split between values and bugs is not a convention the library asks you to remember; it is enforced by a class comparison, and a programmer error physically cannot become a `Left`. `Images` relies on this in the other direction: `Path.of` throws `InvalidPathException`, a plain `RuntimeException` that `attempt` will not catch, so `read` and `write` guard the path resolution explicitly rather than let a throw escape a function whose contract is to return an `Either`.

When a native failure genuinely is a bug at your call site, `Cv.orThrow(operation)(block)` is `attempt` that rethrows instead of returning — literally `attempt(operation)(a).fold(throw _, identity)`

The failures that throw

Two exception types are part of the policy rather than gaps in it.

IllegalArgumentException: what the library can see coming

Where a bad argument is visible before anything crosses JNI, a `require` rejects it, with a message that quotes the value you passed.

```
Image.blank(8, 8).blur(-1)
// java.lang.IllegalArgumentException: requirement failed:
//   blur radius cannot be negative, got -1
```

Do not try to catch that as a value. The following looks like defensive programming and is not:

Example 6-3. Wrong. The require throws straight through attempt.

```
// `radius` came from a query parameter, so this looks careful. It is not:
// IllegalArgumentException is not an Exception by exact class, so it
// propagates out of `attempt` and this Either is never a Left.
def thumbnail(img: Image, radius: Int): Either[CvError, Array[Byte]] =
  Cv.attempt("blur")(img.blur(radius).bytes(".jpg")).flatten
```

It is worse than a `Right` that should have been a `Left`. The `require` fires inside `blur`, before the `bytes` terminal that would have released the image, so every request carrying a negative radius throws *and* strands a Mat. The mistake underneath is a category error: an untrusted number has been carried all the way to a precondition instead of being checked where it entered the system. Validate at the boundary, and by the time the value reaches `blur` it is a programmer error again — which is exactly what the `require` is for.

Example 6-4. Right. Untrusted input is validated where it arrives.

```
def thumbnail(img: Image, radius: Int): Either[String, Array[Byte]] =
  if radius < 0 || radius > 20 then
    img.close()
    Left(s"blur radius must be between 0 and 20, got $radius")
  else img.blur(radius).bytes(".jpg").left.map(_.getMessage)
```

IllegalStateException: the handle you already spent

`Managed` releases exactly once, and reading from one that has been released or transferred throws `IllegalStateException` on the Scala side — before anything reaches JNI, where the same mistake is a SIGSEGV with no stack trace and no test report. `Image` inherits that guard,

and because every transform *moves* the image, the commonest way to trip it is to use a value twice.

Example 6-5. Wrong. Every transform spends its receiver.

```
val img = Image.blank(8, 8)
val a = img.gray     // consumes img
val b = img.blur(2)  // IllegalStateException: this Mat has already been
                     // released or consumed
```

Example 6-6. Right. Copy before the first use when you need two.

```
val original = Image.blank(8, 8)
val edges = original.copy.gray.canny(80, 160) // works on a copy
val small = original.resize(4, 4)             // original still live here
```

This one is deliberately not an `Either`, and the reason is worth stating plainly: threading it through a `flatMap` would let somebody *handle* it. There is no handling to be done. The value is gone, the pipeline downstream of the mistake is meaningless, and the only correct response is to change the source. An `Either` would offer a recovery that cannot exist.

> **NATIVE MEMORY**
> The exception fires at the *reuse*, which is rarely the line you need to see. Start the JVM with `-Dscalacv.trackOwnership=true` and the `IllegalStateException` carries, as its cause, the stack of the transform or terminal that actually spent the handle. It is off by default because it allocates a `Throwable` every time a handle is spent; the code that reads it lives only on the already-failing path, so a program that never misuses a handle pays nothing for the option.

Composing

Because every fallible step is an `Either[CvError, A]`, they thread with `flatMap`, and the first failure short-circuits the rest — the encode never runs when the decode already failed. The running example is small enough to write as one comprehension, and its type then says precisely what can go wrong: nothing but a `CvError`.

Example 6-7. Read, detect, annotate, write --- and leak the cascade.

```
def annotate(path: String, out: String): Either[CvError, Int] =
  for
    cascade <- Cascades.load(CascadeName.FrontalFaceAlt)  // LoadFailed
    result  <- Image.reading(path) { img =>               // DecodeFailed
                 val boxes = cascade.use(c => img.detectHaar(c, minNeighbors =
5))
                 img.drawRects(boxes).write(out).map(_ => boxes.size)
               }
```

```
    count   <- result                                    // EncodeFailed
  yield count
```

Two things in that listing are doing real work. A third is quietly wrong.

`Image.reading` is the scoped entry point: it closes the image when the block returns, on success, on failure, and on an exception, and closing is idempotent, so it is still correct when the block already consumed the image with a terminal. It also runs the whole body inside `Cv.attempt`, which is why a `NativeCall` thrown by a transform halfway down the chain comes back as a `Left` instead of escaping past a signature that promised an `Either`.

The `count <- result` line is the price of that scoping. `reading` returns `Either[CvError, A]`, and here `A` is itself `Either[CvError, Int]`, because the body ends in `write`. Binding it on its own line flattens the nesting and keeps the comprehension readable; `.flatten` on the outer `Either` does the same job in one call when you would rather not name it.

What is wrong is the first generator. `Cascades.load` hands back a `Managed[CascadeClassifier]` that is caller-owned, and the only thing that releases it is the `cascade.use` in the second generator's body. When `Image.reading` returns a `Left` — the file is not there, the bytes are not an image — that body never runs, the comprehension short-circuits, and the classifier stays allocated. The signature is honest, the types check, and the process loses a native handle on every bad upload.

> **NATIVE MEMORY**
> This is the sharp edge of composing `Either` over native resources: a generator that acquires and a later generator that releases are joined by nothing the compiler enforces, and every `Left` between them skips the release. The same shape with an image rather than a cascade:
>
> ```
> for
>   img   <- Image.decode(upload)   // a live Mat now exists
>   boxes <- classify(img)          // Left here, and img is never closed
>   _     <- img.drawRects(boxes).write(out)
> yield boxes.size
> ```
>
> Nothing warns you; the heap stays flat and RSS climbs. Either put the acquisition inside a scope that closes it — `Image.reading` for a path, `Managed.use` for a handle, `Using.resource` for a decoded upload, since `Image` is `AutoCloseable` — or arrange the comprehension so that nothing between the acquisition and its terminal can produce a `Left`.

The fix is not a `try` / `finally` bolted on, and not a rule to remember. It is to stop letting a scope be implied by the order of the generators and make it a block that encloses everything that can short-circuit. `Managed.use` releases in a `finally`, so a `Left` inside it, an exception, and a clean return all reach the same release.

Example 6-8. The same pipeline, with the classifier's lifetime made explicit.

```
def annotate(path: String, out: String): Either[CvError, Int] =
  Cascades.load(CascadeName.FrontalFaceAlt).flatMap { cascade =>
    cascade.use { c =>
      Image
        .reading(path) { img =>
          val boxes = img.detectHaar(c, minNeighbors = 5)
          img.drawRects(boxes).write(out).map(_ => boxes.size)
        }
        .flatten
    }
  }
```

That is the idiom `Cascades` documents — `Cascades.load(name).map(_.use(c => image.detectHaar(c)))` — with a `flatMap` in place of the `map` because the body is itself fallible. `detectHaar` borrows the classifier and returns a `Seq[Rect]` of plain data, so nothing native escapes the block.

Fallbacks compose the same way. `orElse` gives you a second attempt at the whole pipeline, and the `Left` from the first is discarded only where you say so.

Example 6-9. A fallback that is explicit about what it swallows.

```
def avatar(path: String, out: String): Either[CvError, Int] =
  annotate(path, out).orElse {
    Image.reading("assets/placeholder.png") { img =>
      img.resize(256, 256).write(out).map(_ => 0)
    }.flatten
  }
```

Error messages as a feature

Two of this library's messages are not diagnostics so much as instructions, and both are worth seeing before you meet them at three in the morning.

The first is the one every new user hits. scalacv depends on the classifier-less OpenCV Java API, because this project keeps POMs platform-neutral and its pinned Mill publisher cannot encode dependency classifiers — so the natives are a line you add yourself, and forgetting it is the expected state, not an exotic misconfiguration. `OpenCv.load()` therefore does not fail with a link error. It detects the platform you are actually on and prints the lines for it, with the versions read from `Build.openCvArtifactVersion` and `Build.openBlasArtifactVersion` — regenerated on every compile from `build.mill`'s `Deps` block, so the numbers in the message cannot drift away from the ones the library was built against.

Example 6-10. What a missing native payload tells you, on Linux x86-64.

```
OpenCV natives are not on the classpath (...).
```

```
scalacv depends on the classifier-less OpenCV Java API only, because a build tool
cannot express a per-platform classifier in a published POM. Add the natives for
your platform:

  "org.bytedeco" % "opencv"   % "4.13.0-1.5.13" classifier "linux-x86_64"
  "org.bytedeco" % "openblas" % "0.3.31-1.5.13" classifier "linux-x86_64"

Both lines are needed: libopencv_core links libopenblas. If you would rather not
pick a platform, "org.bytedeco" % "opencv-platform" % "4.13.0-1.5.13" bundles
every one, at a cost of about 408 MB.
```

There is a second text behind the same case, for the failure that wears its clothes without being it: a JVM maps a given native library file into exactly one classloader, so a second classloader loading scalacv gets `already loaded in another classloader` even though the jars are demonstrably present. Telling that user to add dependencies they already have sends them down a dead end, so `OpenCv` detects the phrase and prints different advice — load scalacv from a classloader both sides share.

The second message comes from the release path, and it is the more surprising one.

Example 6-11. The release bridge, refusing to guess.

```
cannot open org.opencv.objdetect.CascadeClassifier.delete(long)
(InaccessibleObjectException).

  --add-opens <module>/org.opencv.objdetect=ALL-UNNAMED

scalacv fails here rather than falling back to the garbage collector, because
that fallback does not reclaim native memory in any useful timeframe.
```

The flag is computed from the class's own module and package rather than guessed, which is why a class in the unnamed module — OpenCV on the classpath, the normal case — produces no flag at all. `addOpensRemedy` says so in words instead, and asks for a bug report, because a reflection failure there is not a module-access problem and a `--add-opens` line naming a `null` module would be worse than useless.

> **NATIVE MEMORY**
> Both `NativeDelete` and `NativeFinalizer` throw rather than degrade, and the reason differs in each case. If the private `delete(long)` cannot be opened, the fallback would be to leave the object to the collector — an unbounded native leak that looks exactly like success. If the `nativeObj` field cannot be zeroed, scalacv refuses to free the object *at all*: the binding's `finalize` calls `delete(this.nativeObj)` unconditionally, so freeing without first disarming it means the same pointer is deleted twice, and a corrupted heap surfaces somewhere else entirely, much later. A leak is recoverable. A double free is not. So the library takes the leak, loudly, and hands you the flag that fixes it.

Three things not to do

Do not catch Throwable around a native call.
It is the reflex, and here it is precisely wrong. `Throwable` swallows the programmer errors this whole model exists to keep visible — the `IllegalArgumentException` from a `require`, the `IllegalStateException` from a spent handle — and turns a bug you would have fixed in five minutes into a warning line and a wrong picture. If you are shedding load under memory pressure, catch `OutOfMemoryError` specifically, not its supertype.

Do not swallow a NativeCall.
`case Left(_: CvError.NativeCall) => None` compiles, runs, and destroys the only information anybody was ever going to get. OpenCV's message is not parsed by the library precisely because it is not a stable interface — which means it is *all* there is. The `operation` string and that message together are the whole account of what happened inside the native call. Log both before you decide the frame is not worth having.

Do not reach for Try where the library gives you Either.
Wrapping `Images.read` in a `Try` puts every failure back into one undifferentiated bucket: `DecodeFailed` and a spent-handle bug arrive as the same `Failure`, and the exhaustive `match` over the sealed hierarchy — the thing that tells you at compile time when you have forgotten a case — is gone. The library has already sorted the failures for you. Taking the sorted result and re-mixing it is work that buys nothing.

Next

The error model is easiest to see at the one boundary where all three of OpenCV's failure conventions meet at once: turning files into pixels and back. Chapter 7, *Reading, Writing, and Interop*, takes `Images.read`, `decode`, `encode` and `write` apart in detail — what each one actually calls, why `imread` and `imwrite` are not used at all, and what that choice costs in heap.

PART II

Working with Pixels

Classical image processing, in the order you actually apply it: get the pixels in, describe them in typed terms, filter them, move them, reduce them to a mask, measure what the mask contains, and draw the answer back on top.

CHAPTER 7

Reading, Writing, and Interop

The boundary where the outside world becomes pixels, and pixels become bytes again.

Every vision program is a sandwich. The filling is the interesting part — the blurs, the contours, the detectors — but it is held together by two slices that nobody writes home about: getting pixels in, and getting them back out. A file on disk. An upload body. A BLOB in Postgres. A `BufferedImage` that came from `ImageIO` because the rest of the service was written before anyone said the word OpenCV. The filling is where the algorithms live; the bread is where the bugs live.

They live there because this is the one edge of OpenCV where the error reporting is genuinely inconsistent, and the half you reach for first fails without a sound. `imread` on a path that does not exist returns a `Mat`. `imread` on a directory returns a `Mat`. `imread` on a text file you renamed to `.png` returns a `Mat` — and prints a `findDecoder` warning to stderr that nothing in your process can silence. All three of those `Mat`s have `empty() == true` and none of them throws. Miss the check and you do not fail at the read; you fail four calls later, inside a `gaussianBlur` that had nothing to do with your mistake, with a `CvException` whose message is about kernel sizes. Meanwhile `imencode` with an extension nobody registered an encoder for does throw — so the two halves of the same subsystem disagree about what failure even is.

There is a second problem underneath the first, and it is the one this book keeps returning to. A decoded image is native memory. Whoever creates it has to decide who frees it, and OpenCV's Java bindings decide nothing: `imdecode` hands you a `Mat` and forgets you existed. In a request handler that decodes one upload per request, "forgets you existed" is a leak measured in megabytes per request, on a heap that never grows enough to trigger a collection.

scalacv answers both in the same object. Every function on the I/O boundary returns `Either[CvError, ?]` — the silent failures and the loud one flattened into a single value you handle where you made the call — and every function that produces an image says, in its return type, who owns it. Those functions, the flags that steer the decoder, the formats OpenCV will and will not write, and the bridge to `java.awt.image.BufferedImage` are what follows.

One example runs through it: a thumbnail endpoint that takes an uploaded photo as a byte array, produces a small edge-map preview, and returns it as PNG bytes without ever touching the filesystem — because a service that writes temp files to make a thumbnail is a service that fills a disk at three in the morning. It ends up four lines long, every one of them load-bearing.

Four functions, symmetric in pairs

`Images` is the object at the boundary. It has four public functions, and their shape is the whole design: file or memory, in or out.

Table 7-1. The `Images` surface: two axes, four functions.

	Read (in)	Write (out)
File	`read(path, flags)`	`write(path, mat)`
Memory	`decode(bytes, flags)`	`encode(mat, ext)`

Spelled out, with the defaults the source actually declares:

Example 7-1. The whole I/O surface. Four signatures, one error type.

```
object Images:
  def read(path: String, flags: ImreadFlags = ImreadFlags.Color): Either[CvError,
Managed[Mat]]
  def decode(bytes: Array[Byte], flags: ImreadFlags = ImreadFlags.Color):
Either[CvError, Managed[Mat]]
  def write(path: String, mat: Mat): Either[CvError, Unit]
  def encode(mat: Mat, ext: String = ".png"): Either[CvError, Array[Byte]]
```

The two inbound functions return `Managed[Mat]`, so ownership is stated rather than assumed. The two outbound ones take a bare `Mat`, borrow it, and leave it exactly as they found it — neither `write` nor `encode` releases the image you handed it. A function that produces native memory must say who frees it; a function that only reads pixels must not surprise you by freeing anything.

The failures collapse into two `CvError` cases. Anything that stops an image from becoming pixels is `CvError.DecodeFailed(path, details)`; anything that stops pixels from becoming an image file is `CvError.EncodeFailed(path, details)`. Both are case classes, and both extend `RuntimeException` through the sealed `CvError` hierarchy — so you can match on them as data, or throw them at the edge of a service, whichever the surrounding code already does.

Why a missing file is a `Left` and not an empty `Mat`

`Images.read` does not call `imread`. It reads the file itself, with `java.nio.file.Files`, and hands only the codec work to OpenCV through `decode`. That one decision buys three things.

The first is that the filesystem questions get answered by the filesystem. `imread` reports “no such file”, “that is a directory”, “the file is zero bytes” and “those bytes are not an image” with the same empty `Mat`; `read` distinguishes all four, and the distinction lands in the error’s `details` field:

Example 7-2. Four causes, four messages --- and every one of them names the file.

```
Images.read("/does/not/exist.png").left.map(_.getMessage)
// could not decode an image from '/does/not/exist.png': there is no file at this
path

Images.read("/tmp").left.map(_.getMessage)
// could not decode an image from '/tmp': this path is a directory, not a file

Images.read("empty.png").left.map(_.getMessage)
// ... : the file is empty

Images.read("notes.txt").left.map(_.getMessage)
// ... : the bytes are not an image in a format OpenCV can decode
```

An error message that hedges across four causes is wrong three times out of four, and the library's own test suite pins that these stay distinct. Note that `read` puts the path **back** into the error on the way out: `decode` never saw a filename, so on its own it names its source `<N bytes>`.

The second is a Windows bug that stops existing. The JNI narrows a Java `String` with `GetStringUTFChars`, and OpenCV's imgcodecs hands the resulting modified UTF-8 straight to `fopen`, which on Windows reads it in the process's ANSI code page — so any non-ASCII character in a path resolves to a different, nonexistent name. `φοτο.png` is not missing; it is misreported as missing, and `imwrite` on the same path returns a bare `false`. Upstream has not fixed it (`opencv#4292` is still open against 4.13) and offers no wide-character entry point instead. Doing the file I/O on the JVM removes the narrowing altogether, which is why the library's test suite round-trips a file whose name mixes Cyrillic, a diaeresis and CJK characters.

The third is that `write` encodes fully before it touches the destination, so a failed encode can no longer leave a half-written file behind.

The price is worth stating: the encoded file passes through a JVM byte array, so peak heap grows by the file's size, and a file above 2 GB is out of reach because `Files.readAllBytes` cannot return an array that long.

> **WARNING**
> The rerouting covers `Images` only. Every other `String`-path native call in the library — `VideoCapture`, `VideoWriter`, `CascadeClassifier.load`, `Dnn.readNet` — still goes through the same narrowing, because none of them has an in-memory equivalent to reroute through. On Windows, keep those paths ASCII.

Who frees the Mat

A `Managed[Mat]` coming out of `read` or `decode` is caller-owned: nothing else holds a reference to it and nothing else will free it. The staging buffers used inside — the `MatOfByte` that `encode` writes into, the one `decode` reads from, and the empty `Mat` a failed decode hands back —

are all released before the function returns, so exactly one native object is left alive on the success path and none at all on the failure path.

> **NATIVE MEMORY**
> `Images.read` and `Images.decode` hand you native memory with your name on it. Prefer `Images.read(p).map(_.use(...))` — which releases when the block returns, on success, on failure and on exception — over storing the `Managed[Mat]` in a `val` and hoping a later line frees it. The failure path leaks nothing either, which matters more than it sounds: that is the path a retry loop takes thousands of times.

Example 7-3. Consume in place; the Mat is gone by the time the expression has a value.

```
Images.read("photo.jpg").map(_.use { mat =>
  mat.rows * mat.cols          // work here; released when `use` returns
})
```

`read`, `reading`, and the leak between them

Most code should not be naming `Mat` at all. `Image` is the fluent layer: it wraps one `Managed[Mat]` and gives you move semantics, where each transform consumes the image it was called on and returns a fresh one. `Image.read` and `Image.decode` are `Images.read` and `Images.decode` with an `Image` around the result, and `write` and `bytes` are terminals — they encode **and** release, in one step.

That leaves one trap, and it is worth showing wrong first, because it is the mistake people actually make. `width` is a query. Queries borrow; they do not consume:

Example 7-4. Wrong. Nothing here ever releases the image.

```
val w: Either[CvError, Int] = Image.read("photo.jpg").map(_.width)
```

The `Either` carries an `Int`, the `Image` is unreachable, and its `Mat` is still resident: the chain never reached a terminal, so nothing freed it. `Image.reading` exists so that this cannot happen. It reads the path, runs your body, and closes the image afterwards — even when the body already consumed it, because release is idempotent, and even when the body throws.

Example 7-5. Right. The image is closed when the block returns, whatever the block did.

```
val w: Either[CvError, Int] = Image.reading("photo.jpg")(_.width)
```

Table 7-2. The two entry points differ in exactly one thing: who closes.

Entry point	Returns	Closing

`Image.read(path, flags)`	`Either[CvError, Image]`	yours — a terminal (`write`, `bytes`, `close`) must be reached
`Image.reading(path, flags)(use)`	`Either[CvError, A]`	automatic, on every exit path
`Image.decode(bytes, flags)`	`Either[CvError, Image]`	yours, as with `read`

There is a second, quieter reason to prefer `reading`. Its whole `use` body runs inside `Cv.attempt`, so a `CvError.NativeCall` thrown by a transform in the middle of the chain comes back as a `Left` rather than escaping past the `Either`. With `read`, only the read itself is covered by the `Either`; a `canny` that OpenCV rejects still throws.

> **TIP**
> Do not let the `Image`, or anything derived from it, escape the `use` block. It is closed when the block returns, so a chain that ends in `img` rather than in bytes hands back a corpse. Return pixels (`bytes`), a measurement, or a `BufferedImage` — never the `Image`.

Deciding what comes out of the decoder

Both `read` and `decode` take an `ImreadFlags`, defaulting to `ImreadFlags.Color`. It is a case class with three fields — `color`, `scale` and `ignoreOrientation` — and three named constants cover most calls: `ImreadFlags.Color` forces 3-channel BGR, `ImreadFlags.Grayscale` forces one channel, and `ImreadFlags.Unchanged` gives you the pixels exactly as stored, alpha channel and all.

It looks like a bitmask and is not, which is the reason it is a type at all. Each `IMREAD_REDUCED_*` constant already bakes in its colour bit, and `IMREAD_UNCHANGED` is `-1`, whose bits swamp anything you OR onto it. So `ImreadFlags` maps the `(color, scale)` pair **totally** onto exactly one named constant, exposed as `cvValue`, and only `ignoreOrientation` — which skips the EXIF rotation — is genuinely independent and OR-ed on top.

Table 7-3. `ImreadColor`, and what each case asks the decoder for.

`ImreadColor`	Effect
`Grayscale`	one channel
`Color`	3-channel BGR — the default
`ColorRgb`	3-channel RGB, with no `BgrToRgb` step afterwards
`Unchanged`	exactly as stored, alpha and all
`AnyDepth`	keep 16- or 32-bit depth instead of downcasting to 8-bit

`ImreadScale` is the one that pays for itself. `Full`, `Half`, `Quarter` and `Eighth` decode at 1/1, 1/2, 1/4 and 1/8 of full resolution, and each is strictly cheaper than reading full and resizing afterwards, because the codec skips the detail it is going to discard rather than producing it first. A 4000×3000 source at `Quarter` decodes to 1000×750 and never materialises the full image at all.

Combinations OpenCV has no constant for are rejected at construction rather than quietly OR-ed into something else — reduced-size decode exists only for `Grayscale` and `Color`, and `Unchanged` can carry no extra bit:

```
ImreadFlags(ImreadColor.AnyDepth, ImreadScale.Half)   // require fails
ImreadFlags(ImreadColor.Unchanged, ImreadScale.Half)  // require fails
```

That is an `IllegalArgumentException`, not a `Left`, and correctly so: it is a programmer error, not a data-dependent failure. It is also the first line of the thumbnail endpoint, since a preview has no use for the other fifteen-sixteenths of the pixels:

Example 7-6. The thumbnail endpoint, first draft: decode small, and never at full size.

```
val preview = ImreadFlags(ImreadColor.Grayscale, ImreadScale.Quarter)

def thumbnail(upload: Array[Byte]): Either[CvError, Image] =
  Image.decode(upload, preview)
```

Formats, and what the extension decides

On the way out, the format is chosen by the filename extension, and by nothing else. `Images.write` splits on the last `.` — the same thing OpenCV's own `findEncoder` does with `strrchr`, so the format chosen cannot drift from the format that was checked. `Images.encode` takes that extension directly: `".png"`, `".jpg"`, `".webp"`. A leading period is added for you if you leave it off, because `imencode` fails silently without one and the mistake is easy to make. `encode(m, "png")` and `encode(m, ".png")` produce byte-identical output.

An extension with no registered encoder is the failure that OpenCV throws for rather than reports. `Images` heads it off by asking `Imgcodecs.haveImageWriter` first, so it comes back as `CvError.EncodeFailed` — the same case as an unwritable destination. That one case covers every way the **destination** can fail: an unknown extension, a path this filesystem cannot represent, a missing parent directory, a directory you may not write to.

What it does not cover is a codec that rejects the **pixels**. `imencode` throws a `CvException` for those, and `Cv.attempt` turns the throw into `CvError.NativeCall("imencode('.jpg')", cause)` rather than into an `EncodeFailed`, because it is a data-dependent native failure like any other. Handing a 4-channel image to a format with no fourth channel is the usual way to meet it, which makes the next table more load-bearing than it looks.

Table 7-4. The extensions worth knowing, and what each trades.

`ext`	Lossy?	Alpha	Use it for
`".png"`	no	yes	screenshots, masks, anything you re-process
`".jpg"` / `".jpeg"`	yes	no	photos, where a smaller file wins
`".webp"`	yes, here	yes	modern web delivery

`".bmp"`	no	no	a raw, decoder-free dump

Alpha is the trap in that table. Decode a transparent PNG with `ImreadFlags.Unchanged` and you are holding a 4-channel BGRA image — `img.channels` says `4` — and the formats without a fourth channel will not take it. Flatten it deliberately, with `img.convert(ColorConversion.BgraToBgr)`, so the loss is a line in your code rather than a `NativeCall` from the encoder.

The other thing the table does not offer is a dial. `Images.encode` takes `(mat, ext)` and nothing else: no JPEG quality, no PNG compression level, no WebP lossless switch. Every file comes out with whatever OpenCV's default is for that format, and `".webp"` is lossy for the same reason — the lossless mode lives behind an encoder parameter this surface does not expose. When one of those knobs genuinely matters, the escape hatch is the one the library uses everywhere: borrow the `Mat` and call OpenCV directly.

Example 7-7. Encoder parameters, on the layer below. `img.mat` borrows; the Image stays yours to close.

```
import org.opencv.core.{MatOfByte, MatOfInt}
import org.opencv.imgcodecs.Imgcodecs

def jpegAt(img: Image, quality: Int): Either[CvError, Array[Byte]] =
  Managed.use(MatOfByte()): buffer =>
    Managed.use(MatOfInt(Imgcodecs.IMWRITE_JPEG_QUALITY, quality)): params =>
      if Imgcodecs.imencode(".jpg", img.mat, buffer, params) then
Right(buffer.toArray)
      else Left(CvError.EncodeFailed(".jpg", "imencode returned false"))
```

Both staging objects are `Mat` subclasses, so `Managed.use` frees them on the way out and the borrowed `img` is untouched — exactly the arrangement `Images.encode` has inside it, with one argument more.

> **WARNING**
> `write` does not create parent directories. A path into a folder that does not exist is a `Left` whose details read "the parent directory does not exist" — told apart from "the destination is not writable", which `imwrite` signals with the same bare `false` — but it is still a failure. `flatMap` the result; do not assume success.

The round trip that never touches a disk

`encode` and `decode` compose into a full in-memory round trip, and that is the shape of every HTTP image service: bytes arrive, pixels happen, bytes leave. At the `Image` layer the round trip never names a `Mat` and never names a `release`, because `bytes` is a terminal — it encodes and releases in the same call.

Example 7-8. The thumbnail endpoint, finished. No file is created; nothing is left to free.

```
val preview = ImreadFlags(ImreadColor.Grayscale, ImreadScale.Quarter)

def thumbnail(upload: Array[Byte]): Either[CvError, Array[Byte]] =
  Image
    .decode(upload, preview)                    // bytes -> Image, at 1/4
resolution
    .flatMap(_.blur(2).canny(80, 160).bytes(".png"))
```

Four lines, and the interesting property is what is **not** in them. No temp file, so nothing to clean up and no disk to fill. No `Mat`, so nothing to release. No `try` / `finally`, because `bytes` releases whether the encode succeeded or not. And no exception: a truncated upload, a PDF someone renamed to `.jpg`, an empty body — each comes back as `Left(DecodeFailed(...))` naming which it was. The empty array is rejected before it ever reaches OpenCV, so "no bytes arrived" is never confused with "the bytes were not an image".

> **NATIVE MEMORY**
> `bytes` and `write` consume the `Image`. After `img.bytes(".png")` the `img` handle is spent, and touching it throws `IllegalStateException` from Scala rather than reading freed memory from JNI. That is the point of move semantics: a long chain holds exactly one live `Mat` at a time, never a pile of intermediates. Run the JVM with `-Dscalacv.trackOwnership=true` and the exception will point at the call that consumed it.

Crossing into AWT

The rest of the JVM does not speak `Mat`. It speaks `java.awt.image.BufferedImage` — `ImageIO` reads and writes one, Swing paints one, `Graphics2D` draws into one, and Almond, the Scala Jupyter kernel, renders one inline when it is the value of a cell. Two methods span that boundary, and both **copy**, so ownership is never shared and never ambiguous.

Table 7-5. The AWT bridge. Both directions copy.

Direction	Method	Behaviour
scalacv ⟶ AWT	`img.toBufferedImage`	borrows; `img` stays alive. 1 channel ⟶ `TYPE_BYTE_GRAY`, 3 ⟶ `TYPE_3BYTE_BGR`, 4 ⟶ flattened to BGR
AWT ⟶ scalacv	`Image.fromBufferedImage(bi)`	returns a new owned 3-channel BGR `Image`; the source stays yours

`toBufferedImage` requires depth `CV_8U`. That is a `require`, so a 16-bit or float image throws `IllegalArgumentException` with a message telling you what to do about it — and the restriction is not fussiness. A disparity map, a distance transform, a raw Sobel response: all float, and silently truncating one to bytes produces a picture that looks like noise and says nothing about why. Bring it down deliberately instead: `normalize()`, whose `depth` already defaults

to `OutputDepth.Unsigned8`, rescales into 0–255 grey, and `colorMap(Colormap.Viridis)` renders it as legible false colour.

A 4-channel image is not rejected; it is converted with `ColorConversion.BgraToBgr` on the way across, because AWT's `TYPE_3BYTE_BGR` has nowhere to put the fourth channel. Transparency is therefore lost at the AWT boundary — worth knowing before you go looking for it in the rendered PNG.

Coming the other way, `fromBufferedImage` accepts every `BufferedImage` type there is. A `TYPE_3BYTE_BGR` source already stores bytes in the interleaving `CV_8UC3` wants, so its backing array copies straight in; anything else — ARGB, indexed, a custom raster, a translated sub-raster from `getSubimage` — is normalised by drawing it once through a known BGR layout. The result is always 3-channel, whatever went in.

Example 7-9. Into AWT and back. Note which side owns which object.

```
Image.reading("photo.jpg") { img =>
  val grey = img.copy.gray                  // a copy, so `img` survives the
branch
  val awt =
    try grey.toBufferedImage                // borrows: `grey` is still alive
afterwards
    finally grey.close()                    // ... so `grey` is yours to close
  val back = Image.fromBufferedImage(awt)   // a fresh, independently owned Image
  try back.width
  finally back.close()
}
```

Three objects, three owners, and only one of them is `reading`'s to clean up: the block closes `img`; `grey` and `back` are closed by the code that made them. That is the price of branching.

> **NATIVE MEMORY**
> `toBufferedImage` borrows: the `Image` it was called on is still alive and still yours to close. `fromBufferedImage` produces a new owned `Image`, and it will not be closed by anything else. In a notebook you can be casual — the cell ends, the JVM reclaims — but the same two lines in a request handler need the `close`, and `Image.reading` is the way to stop needing to remember.

In an Almond cell, this is why the bridge exists at all. Return a `BufferedImage` as the value of the cell and the picture appears:

```
Image.reading("photo.jpg")(img => img.gray.canny(80, 160).toBufferedImage)
```

Because transforms move the image, displaying an intermediate step means displaying a `.copy`, as in the listing above: `img.blur(5).toBufferedImage` consumes `img`, and the next line of the cell throws `IllegalStateException`. `img.copy.blur(5).toBufferedImage` gives the cell a throwaway to render and leaves the pipeline intact.

BGR, and the picture that comes out wrong

If a scalacv image ever renders with an orange sky and blue skin, stop and check channel order. OpenCV stores colour as **blue, green, red**. Almost everything else on the JVM — AWT's packed `int` pixels, `ImageIO`'s PNG writer, every hex colour you have ever typed — is RGB. The two are the same three bytes in the opposite order, which is why the bug is so hard to see: nothing crashes, nothing is empty, the image is the right size, and the colours are wrong.

The library states the convention rather than hiding it, in the scaladoc and in the constants:

Example 7-10. Wrong, then right. The first line is blue.

```
val red = Scalar(255, 0, 0)   // wrong: that is 255 in the BLUE channel
val ok  = Scalar(0, 0, 255)   // right --- and this is exactly Scalar.Red
```

Use `Scalar.Red`, `Scalar.Green` and `Scalar.Blue` and the question never arises. Where you do need the other order, ask for it at the boundary instead of shuffling channels by hand: decode with `ImreadFlags(ImreadColor.ColorRgb)` and the decoder gives you RGB directly, with no conversion step; convert an image you already hold with `img.convert(ColorConversion.BgrToRgb)`.

> **WARNING**
> The AWT bridge does **not** need a conversion. `toBufferedImage` writes into `TYPE_3BYTE_BGR`, which stores bytes in the same B, G, R order a `CV_8UC3` `Mat` does, and `fromBufferedImage` normalises back through the same layout. A round trip through AWT preserves your pixels exactly — the library's tests assert it to within one greylevel. Inserting a `BgrToRgb` "to be safe" is how you **create** the swapped-colour bug, not how you avoid it.

Why BGR, of all orders?

The usual explanation is that OpenCV inherited it from the hardware and file formats of the late 1990s: Windows BMP stores its pixels blue-first, and so did much of the frame-grabber software of the era, so the cheapest thing an imaging library could do was leave the bytes where the capture card had put them. By the time RGB had won everywhere else, changing it would have broken every program ever written against the library.

A defensible decision, then, and one that has cost the world an enormous number of debugging hours since. What a wrapper can do is make the convention **visible** — a `Scalar.Red` that is literally `Scalar(0, 0, 255)`, a `ColorConversion.BgrToRgb` you have to name, an `ImreadColor.ColorRgb` you can ask for at the decoder — rather than leaving it as folklore you are expected to have absorbed.

Paths, and what a path is not

`Images.read` resolves a filesystem path with `java.nio.file.Path.of`, and that is all it does. It is not a classpath resource, not a URL, and not a glob. A relative path resolves against the JVM's working directory, which in a `mill` or `sbt` run is the module or project root and in a container is whatever the image's `WORKDIR` says — so a relative path that works in your tests and fails in production has usually not moved at all; the process has.

A path the filesystem cannot represent is a `Left` rather than a throw, which takes a little care to arrange: `Path.of` raises `InvalidPathException` for an embedded NUL byte, and for the characters NTFS forbids on Windows. That is a plain `RuntimeException`, which `Cv.attempt` deliberately does not catch, so `Images` catches it explicitly — a function whose whole contract is to return an `Either` does not get to throw.

For a resource that lives inside a jar — a test fixture, a bundled watermark, a template you ship with the application — there is no `Images` function to call, and there does not need to be. Read the bytes with the JDK and decode them:

Example 7-11. A classpath resource is bytes, and bytes already have an entry point.

```
def fixture(name: String): Either[CvError, Image] =
  Option(getClass.getResourceAsStream(name)) match
    case Some(in) => try Image.decode(in.readAllBytes()) finally in.close()
    case None => Left(CvError.DecodeFailed(name, "no such classpath resource"))
```

That is the general lesson of this chapter, in four lines. `decode` is the real entry point and `read` is a convenience on top of it; anything you can turn into an `Array[Byte]` — a jar entry, an S3 object, a `bytea` column, a WebSocket frame — is already an image source.

What comes next

Pixels are in memory now, owned by something that will free them, in a channel order you can name. `ImreadFlags` was the first of this library's answers to OpenCV's integer constants, and it will not be the last: Chapter 8, *Typed Constants and Colour Spaces*, is about the rest of them — `ColorConversion`, `Interpolation`, `BorderType`, `Threshold` — and about why a total mapping onto named cases catches, at compile time, the class of mistake that `ImreadFlags` catches at construction. Everything the following chapters do to pixels is written in those enums.

CHAPTER 8

Typed Constants and Colour Spaces

Six is greyscale. Seven is also greyscale. Only one of them is the one you meant.

OpenCV expresses almost every option as an `int`. Which colour space to convert into, how to extrapolate pixels past the edge of an image, which contours to report, how to interpolate during a resize, which Hershey font to draw with — all of it arrives at the native layer as a bare integer, and the Java binding faithfully passes it through. `Imgproc.cvtColor(src, dst, 6)` is a complete, compiling, working call. There are more than a hundred colour-conversion codes alone, and the binding gives every one of them the same type.

That design is defensible in C++, where the constants come from an `enum` and the compiler still knows something about them. In the Java API they are `public static final int` fields, and the type information is gone. `Imgproc.COLOR_BGR2GRAY` is 6. `Imgproc.COLOR_RGB2GRAY` is 7. Both are legal third arguments to `cvtColor`, both produce a single-channel image of exactly the expected dimensions, and both compile. The difference is that greyscale luminance is a weighted sum — roughly 30% red, 59% green, 11% blue — and the two codes disagree about which stored channel is which. Hand a BGR image to code 7 and the blue channel is weighted as though it were red. Nothing throws. You get a grey image that is subtly, consistently wrong, and you find out three stages later when a contour count drifts or a threshold that was tuned on Tuesday stops holding on Wednesday.

Larger mistakes are louder. Pass a `BORDER_` constant where a `RETR_` was expected and you are still passing an `int`; the compiler has no opinion. OpenCV usually notices — an assertion fires natively and the binding rethrows it as a `CvException` — but "usually" is carrying weight there: some codes are in range for the wrong parameter and mean something else there.

The remedy is not clever, and scalacv does not pretend otherwise: give each family of constants a Scala 3 `enum`, and let the compiler do the one thing it is good at. What is worth reading closely is where that mapping is *not* one-to-one — two of OpenCV's constant families are not enumerations at all, and a third is one enum standing over two domains that accept different values. And one family, colour conversion, deserves more than a lookup table, because choosing the right colour space is the difference between a segmentation that survives a cloud passing over the window and one that does not.

The mechanism

Every typed constant in scalacv is a Scala 3 `enum` with a single parameter, `cvValue`, carrying the OpenCV integer. `Enums.scala` is the file, and `ColorConversion` is the pattern.

Example 8-1. The enum declaration, in full for the first two cases.

```
enum ColorConversion(val cvValue: Int):
  case BgrToGray extends ColorConversion(Imgproc.COLOR_BGR2GRAY)
  case GrayToBgr extends ColorConversion(Imgproc.COLOR_GRAY2BGR)
  // ... BgrToRgb, RgbToBgr, BgrToHsv, HsvToBgr,
  //     BgrToLab, LabToBgr, BgrToBgra, BgraToBgr
```

Three things follow from that one line, and they are the whole contract.

The case names are OpenCV's own names transliterated, not reinvented. `COLOR_BGR2GRAY` becomes `BgrToGray`: the family prefix goes, because the enum type already carries it; the screaming-snake spelling becomes `UpperCamelCase`; and the `2` is spelled `To`, because `Bgr2Gray` reads like a version number. A reader who knows the OpenCV constant can guess the scalacv case, and the reverse.

The integer is not hidden. `cvValue` is a public `val`, so `ColorConversion.BgrToGray.cvValue` is 6 and you can print it, log it, or hand it to a raw call. The wrapper's job is to stop you passing the wrong constant, not to stop you seeing the right one.

And the whole set is enumerable: `ColorConversion.values` is an `Array` of every case, which is what makes autocompletion useful the moment you type `ColorConversion.` and what makes the table below something you consult twice and then stop needing.

The families

Thirteen typed-constant families stand on their own in `Enums.scala`, and one more — `OutputDepth`, the destination depth for the derivative operators — lives in `Ops.scala`, beside the operations that take it. Four further enums in that same file exist to serve the two structured types described in the next section rather than to be passed on their own: `Threshold.Mode` and `Threshold.Auto`, `ImreadColor` and `ImreadScale`.

Outside core the pattern travels, but not unchanged. `CaptureBackend` in `Video.scala` and `ArucoDictionary` in `Detectors.scala` carry a `cvValue` exactly as above. Two others deliberately do not: `Codec` in `Codec.scala` carries `fourcc: Int`, because a codec identifier is four packed characters rather than an OpenCV flag, and `CascadeName` in `Cascades.scala` carries `fileName: String`, because what it names is a bundled resource. `TrackerKind` in `Tracker.scala` carries nothing at all — there is no integer behind `Csrt`, `Kcf` and `Mil`, only three different Java factory classes that `Tracker.create` matches on. The accessor is named for what it holds, so seeing `cvValue` on a case is itself the signal that a raw OpenCV constant is underneath.

Table 8-1. The typed constant families, and what each one steers.

Enum	Steers	Cases
`ColorConversion`	`convert`, `cvtColor`	`BgrToGray`, `GrayToBgr`, `BgrToRgb`, `RgbToBgr`, `BgrToHsv`, `HsvToBgr`,

		`BgrToLab`, `LabToBgr`, `BgrToBgra`, `BgraToBgr`
`Interpolation`	`resizeTo`, `scale`, `rotate(degrees)`	`Nearest`, `Linear`, `Cubic`, `Area`, `Lanczos4`
`BorderType`	pixels past the edge	`Constant`, `Replicate`, `Reflect`, `Reflect101`, `Wrap`
`Flip`	`flip`	`Horizontal`, `Vertical`, `Both`
`Rotation`	lossless quarter-turns	`Clockwise`, `CounterClockwise`, `Half`
`MorphShape`	the structuring element	`Rect`, `Ellipse`, `Cross`
`MorphOp`	`morphology`	`Open`, `Close`, `Gradient`, `TopHat`, `BlackHat`
`AdaptiveMethod`	`adaptiveThreshold`	`Mean`, `Gaussian`
`ContourRetrieval`	`findContours`	`External`, `List`, `CComp`, `Tree`
`ContourApproximation`	`findContours`	`None`, `Simple`, `Tc89L1`, `Tc89Kcos`
`Colormap`	`colorMap`	`Autumn`, `Bone`, `Jet`, `Ocean`, `Hot`, `Magma`, `Inferno`, `Plasma`, `Viridis`, `Turbo`
`Font`	mid-level `drawText`	`Simplex`, `Plain`, `Duplex`, `Complex`, `Triplex`, `Script`
`LineType`	rasterisation	`Connected4`, `Connected8`, `AntiAliased`
`OutputDepth`	`sobel`, `laplacian`, `normalize`	`SameAsSource`, `Unsigned8`, `Signed16`, `Float32`, `Float64`

Two case names collide with things already in scope, deliberately. `ContourApproximation.None` and `ContourRetrieval.List` are named after `CHAIN_APPROX_NONE` and `RETR_LIST`; renaming them to dodge `scala.None` and `scala.List` would break the one property that makes the scheme worth having, and qualified there is no ambiguity. `Enums.scala` writes the cost back out where a reader will meet it: `Threshold`'s `auto` parameter defaults to `scala.None`, spelled in full, so nobody scanning a file that also defines a case called `None` has to stop and work out which one is meant.

Where the enum is the wrong shape

An `enum` says: a value is exactly one of these cases, and every case is as good as any other wherever the type is accepted. Two of OpenCV's constant families break the first half of that, and a third breaks the second. All three would have produced an API that could not express the common call, or one that quietly accepted a value the native layer refuses.

Threshold is a bitmask

`Imgproc.threshold` takes a mode — binary, inverted binary, truncate, to-zero, inverted to-zero — optionally OR-ed with an automatic method, `THRESH_OTSU` or `THRESH_TRIANGLE`. Asking for Otsu's method is not choosing a sixth mode; it is modifying one of the five. So `Threshold` is a case class carrying both parts, and `cvValue` does the OR:

Example 8-2. A mode, plus at most one automatic modifier.

```
final case class Threshold(mode: Threshold.Mode, auto: Option[Threshold.Auto] =
scala.None):
  def cvValue: Int = mode.cvValue | auto.fold(0)(_.cvValue)
  def computesThreshold: Boolean = auto.isDefined
```

The two nested enums are real enumerations — `Mode` has `Binary`, `BinaryInv`, `Truncate`, `ToZero`, `ToZeroInv`; `Auto` has `Otsu` and `Triangle` — and the `Option` encodes the fact that the two automatic methods are mutually exclusive with each other but not with any mode. Three companion shortcuts cover the calls people actually write: the `Threshold.Binary` value for a fixed cutoff, and the `Threshold.otsu(mode)` and `Threshold.triangle(mode)` methods for the automatic ones, both of which default `mode` to `Mode.Binary` so that `Threshold.otsu()` is the whole call.

`computesThreshold` exists because of what `threshold` returns. OpenCV's function hands back a `double`, which most wrappers discard; for Otsu and Triangle that number is the cutoff OpenCV chose, and it is frequently the reason you called the function at all. scalacv keeps it, as `ThresholdResult(value: Double)`, and `computesThreshold` tells you whether the number means anything.

ImreadFlags only looks like a bitmask

The `IMREAD_*` constants have the shape of OR-able bits and are not. Each `IMREAD_REDUCED_*` value already bakes its colour choice in, so OR-ing a colour flag onto a reduction quietly decodes something other than what you asked for, and `IMREAD_UNCHANGED` is `-1`, whose bits swallow every other flag entirely.

So `ImreadFlags` models the truth: a `(color, scale)` pair that maps *totally* onto exactly one named constant, plus `ignoreOrientation` as the only genuinely independent flag. Combinations OpenCV has no constant for are rejected by `require` rather than OR-ed into something plausible-looking — there is no reduced-size decode for `ColorRgb`, `Unchanged` or `AnyDepth`, and `Unchanged` cannot carry a reduction or an orientation bit at all.

BorderType is one type over two domains

`BorderType` is the most interesting of the three, because the value is legal in one half of the library and fatal in the other. OpenCV packs two different sets of accepted values into the same `int`. `copyMakeBorder` — behind `pad` and `border` — and `warpAffine` — behind `rotated` — honour all five modes including `Wrap`. The `imgproc` filter family does

not: `gaussianBlur`, `boxBlur`, `sobel` and `laplacian` all reach `cv::FilterEngine::init`, which asserts `columnBorderType != BORDER_WRAP` and aborts.

Left to native code, that is not even a consistent failure. Verified against OpenCV 4.13, `GaussianBlur` with `BORDER_WRAP` on `CV_8U` input *silently ignores* the mode — the SIMD path never reaches the assertion — and aborts on `CV_32F`. So this is the bug that ships:

```
// Raw OpenCV: fine all through development on 8-bit frames,
// aborts the first time it meets a float image.
Imgproc.GaussianBlur(src, dst, org.opencv.core.Size(5, 5), 0, 0,
Core.BORDER_WRAP)
```

scalacv rejects it at the Scala boundary instead. `gaussianBlur`, `boxBlur`, `sobel` and `laplacian` each open with `BorderType.requireFilterSupport(op, border)`, a `require` in `BorderType`'s companion that takes the scalacv operation name and the caller's mode. Both depths then fail identically, before any native call, with an `IllegalArgumentException` that names the operation, the assertion it would have hit, and where the mode *is* legal:

```
requirement failed: gaussianBlur does not support BorderType.Wrap: OpenCV's
imgproc
filters assert columnBorderType != BORDER_WRAP. Wrap is valid only for pad/border
(copyMakeBorder) and rotated (warpAffine).
```

The library is explicit that this is a stopgap. Splitting the type — a `FilterBorder` without `Wrap`, widening into a `TransformBorder` with it — would make the mistake unrepresentable rather than merely detected, and that is the stated end state; it is a breaking change to four public signatures, so it waits. Two related values stay out of the enum entirely: `BORDER_ISOLATED`, a modifier that means nothing for any call scalacv exposes, and `BORDER_TRANSPARENT`, which `copyMakeBorder` throws on and which leaves `warpAffine`'s freshly-allocated destination uninitialised.

There is also no single border default across the library, and the split is on purpose.

Table 8-2. The border default depends on what you are doing.

Operation	Its default
`pad`, `border`, mid-level `rotated`	`Constant`, filled with `Scalar.Black`
`gaussianBlur`, `boxBlur`, `sobel`, `laplacian`	`Reflect101`

Mirroring is right for a filter, because a constant black edge bleeds inward and darkens the border of a blurred image; a constant colour is right for padding, because a visible margin is usually the entire point. The parameter names differ with the domain, too: `pad` and `border` take `borderType`, while the filters and `rotated` take `border` — and `rotated` splits the mode from the colour, taking `border` alongside a separate `color`.

Colour, and the space you reason in

Everything above is machinery. Colour conversion is where the typed constants stop being a type-safety story and start being a modelling decision, so it is worth working one problem all the way through.

The problem: a photograph of a shelf, and you want the bottles with green labels — how many, and where.

The instinct is to threshold on the pixel values directly. OpenCV Mats are BGR by default, not RGB — a trap worth internalising, and the reason `Scalar.Red` is `Scalar(0, 0, 255)` — so "green" is a box around high channel-1 values with low channel 0 and 2. It works on the photograph you tuned it against and collapses on the next one, taken half an hour later with the blind half-drawn, because BGR encodes brightness and colour together in all three channels at once. A green label in bright light is `(40, 200, 60)`; the same label in shadow is `(14, 70, 21)`. Same colour, same object, and no fixed box in BGR contains both without also containing half the shelf.

HSV separates the question

HSV splits a pixel into hue — which colour it is, as an angle around a wheel — saturation — how vivid, from grey to pure — and value — how bright. The green label's hue barely moves between sunlight and shadow; it is value that collapses, and saturation that softens. So "green, at any brightness" becomes a wide range on one channel and a generous floor on the other two, which is a condition that survives the blind being drawn.

That is why `toHsv` gets a name of its own on `Image`, as only `gray` otherwise does: eight of the ten conversions are spelled `convert(ColorConversion.X)`, and these two are reached for often enough that the long form was noise.

Example 8-3. BGR in, a binary mask out.

```
val written: Either[CvError, Unit] =
  Image.reading("shelf.jpg") { shelf =>
    val mask = shelf.copy.toHsv.inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
    try shelf.applyMask(mask).write("green-labels.png")
    finally mask.close() // applyMask only borrowed it
  }.flatten
```

`inRange` yields a single-channel `CV_8UC1` mask — 255 where every channel falls inside the bounds, 0 elsewhere — whatever the source's channel count was. `applyMask` then keeps the original pixels only where the mask is white. Both `reading` and `write` return an `Either[CvError, ?]`, so the block produces a nested pair that `flatten` collapses into the single `Left` a caller can act on — the decode failing and the encode failing are both failures to produce the file.

NATIVE MEMORY

> `applyMask` and `blend` take a second image that is **borrowed**, not consumed: the mask stays alive after the call and you close it yourself. Every other verb in that chain spends its receiver, which is why the mask is built from `shelf.copy` — without the copy, `toHsv` would consume the only `Image` you have and `applyMask` would be called on a spent handle. Inside `Image.reading` the original is released for you on the way out, on success and on exception alike; the mask is not, because `reading` never owned it.

Hue is 0 to 179, not 0 to 359

The bounds in that listing are the part that catches everyone. A hue wheel has 360 degrees. An 8-bit channel holds 0 to 255. OpenCV's answer, for 8-bit images, is to halve the angle: **hue runs 0–179**, while saturation and value run the full 0–255. Green sits near 120° on the wheel, which is 60 here.

So the mistake readers actually make is to write the degrees they know:

```
// Wrong: 100--140 on a 0--179 scale is blue. This mask comes back almost
entirely black.
hsv.inRange(Scalar(100, 80, 80), Scalar(140, 255, 255))
```

and get an empty result with no error, because 100 to 140 is a perfectly valid range that happens to select blue. The halving applies only to 8-bit images; convert to `CV_32F` first and OpenCV keeps the full 0–360. Nothing in the type system distinguishes the two, so the rule is: on the 8-bit images you will spend almost all of your time with, halve the degrees.

Table 8-3. Hue centres on OpenCV's 8-bit 0--179 scale. Widen the saturation and value floors to admit washed-out or shadowed pixels.

Colour	Hue	Note
Red	0–10 *and* 170–179	Wraps the seam — needs two ranges
Orange	≈10–20	
Yellow	≈25–35	
Green	≈35–85	The labels above
Cyan	≈85–95	
Blue	≈100–130	
Magenta	≈140–160	

Red is the awkward one, and not because of the scale. Hue is circular, and red sits exactly on the 0/179 seam, so no single `inRange` captures it: you build two masks, one at each end, and OR them together. `inRange` is wrapped and a raw bitwise OR is not, so that one step drops to `org.opencv.core.Core.bitwise_or` — which is exactly what the escape hatch below is for.

The other spaces

Greyscale is the input format for most of the library: edges, thresholds, contours and `colorMap` all expect a single channel, and `adaptiveThreshold` and `equalizeHist` accept `CV_8UC1` and nothing else. `gray` is the shortcut, and it is exactly `convert(ColorConversion.BgrToGray)`.

Lab is the one to reach for when you need perceptual colour *distance* — how different two colours look to a person. Euclidean distance in BGR is close to meaningless for that; in Lab it is roughly proportional to perceived difference, which is what makes it the space for white balance and for matching a swatch against a sample.

`BgrToRgb` and `RgbToBgr` only reorder channels, for the boundary with libraries that assume RGB; `BgrToBgra` and `BgraToBgr` add and drop alpha. Those two and the two greyscale cases are the only four that touch the channel count — the rest change what each channel *means* and leave the count alone, so a 3-channel BGR image becomes a 3-channel HSV one and `channel(0)` on the result is the hue plane rather than the blue one.

Why blue comes first

Nothing about BGR is principled; it is a fossil. When OpenCV's first releases appeared around the turn of the century, the ordering that Windows device-independent bitmaps and the frame-grabber cards of the day used for 24-bit colour was blue-green-red, and reading a frame straight into a buffer without permuting it was free. Every later decision inherited that: `imread` returns BGR, `imwrite` expects BGR, the drawing functions take BGR scalars, and a quarter-century of tutorials assume it.

The cost lands downstream. A frame handed to a library that assumes RGB — a neural-network preprocessor, an image viewer, a plotting library — comes back with the reds and blues swapped, which is the most recognisable bug in computer vision: a person with blue skin against an orange sky. scalacv does not try to fix this: reordering channels silently would put the library permanently out of step with the OpenCV documentation its users read. It names the ordering instead — in `Scalar`'s scaladoc, in the constants, and in `BgrToRgb` being spelled out rather than called `toRgb`.

When the enum you need is missing

`ColorConversion` has ten cases. OpenCV has more than a hundred conversion codes. That gap is not an oversight to be apologised for; it is the point. The ten are the ones the library uses, tests and documents, and adding `COLOR_BGR2YCrCb` to the enum on the grounds that it exists would trade a set you can read for a set you have to search.

When you need one of the others, you drop a level and pass the raw `int`. This is a supported move with a documented shape, not a workaround:

Example 8-4. A conversion the enum does not name, done at the raw level and lifted straight back up.

```
import org.opencv.core.Mat
import org.opencv.imgproc.Imgproc

val luma: Either[CvError, Unit] =
  Image.reading("shelf.jpg") { shelf =>
    val raw = Mat()
    Imgproc.cvtColor(shelf.mat, raw, Imgproc.COLOR_BGR2YCrCb) // borrowed in,
fresh Mat out
    Image.wrap(Managed(raw)).channel(0).write("luma.png")     // adopted;
released on write()
  }.flatten
```

Three doorways make that work, and knowing which is which is the whole skill. `img.mat` **borrows** the underlying `org.opencv.core.Mat`: the `Image` still owns it, so a raw call may read from it and write into a destination of its own, but must never release it. `img.managed` **hands ownership over**, leaving the `Image` spent and the returned `Managed[Mat]` yours to release. `Image.wrap(managed)` hands ownership back the other way.

> **NATIVE MEMORY**
> `Imgproc.cvtColor` fills a bare `Mat` that has no owner: if the wrapping line is never reached, it leaks a full-size pixel buffer that the collector will not reclaim in any useful timeframe. Wrap a raw result in `Managed` — or adopt it as an `Image` — in the block that created it, so something is on the hook for the release whatever happens next. And never call `release()` on `img.mat`: that frees a pointer the `Image` still believes it owns, which is a double free rather than an exception.

The mid level is the same move with the ownership already sorted out: every pixel-producing extension op on a `Mat` — `cvtColor`, `gaussianBlur`, `inRange` — hands back an owned `Managed[Mat]` allocated by `Mats.produce`, which releases the half-built destination itself if the native call throws. `pipe` threads one op into the next and releases each intermediate in a `finally`; `Mats.chain` is the same fold written as a list of stages. A typed `cvtColor` and a raw one compose in the same chain, and the only difference is who wraps the output.

The value types

Constants are half of the vocabulary. The other half is geometry, and `Geometry.scala` defines five value types: `Point`, `Point3`, `Size`, `Rect` and `Scalar`.

Table 8-4. The geometry value types. All are immutable Scala case classes, copied across the native boundary.

Type	Fields	Element	Invariant
`Point`	`x`, `y`	`Double`	none
`Point3`	`x`, `y`, `z`	`Double`	none
`Size`	`width`, `height`	`Double`	neither side negative
`Rect`	`x`, `y`, `width`, `height`	`Int`	non-negative extent

`Scalar`	`v0` – `v3`	`Double`	none

They are copies, not wrappers, which is the design decision worth understanding. `org.opencv.core.Rect` is a mutable Java object with public fields, and a `Seq` of them handed back by a detector is a set of live handles whose contents can change underneath you. Copying four ints at the boundary is cheap, and it turns detector output into ordinary immutable Scala data — something you can pattern-match on, key a `Map` by, or send to another thread.

> **NATIVE MEMORY**
> Because they are copies, geometry survives the image. A `Seq[Rect]` from a detector is still valid long after the `Mat` it was measured from has been released — which is exactly what lets `Image.reading` free the image at the end of the block while you keep the boxes. Nothing in these five types points at native memory, so nothing in them can dangle.

The invariants are enforced where they are cheap. `Size` and `Rect` both `require` a non-negative extent, though `Rect`'s origin may be negative, since a region of interest can legitimately extend past the top-left of an image. `Rect.area` returns a `Long`, because `width * height` overflows a signed `Int` past roughly a 46340-pixel side. `Point.distanceTo` uses `math.hypot`, which will not overflow while squaring.

`Scalar` is the pixel value: up to four channel components, in whatever order the `Mat` uses, unset channels defaulting to 0. Its constants are ordered for BGR, which is the last time this chapter will say so: `Scalar.Red` is `Scalar(0, 0, 255)`, `Scalar.Blue` is `Scalar(255, 0, 0)`, and `Scalar.Black` and `Scalar.White` sidestep the question entirely.

Where this goes next

The mask cut from the shelf photograph is not finished work. A freshly thresholded mask is grainy at the edges and speckled in the middle: stray pixels that passed the hue test, small holes where a highlight blew the saturation out, and counting contours on it now would count the speckle. Cleaning that up is morphology — `MorphOp.Open` to erase the specks, `MorphOp.Close` to fill the holes, both shaped by a `MorphShape` — and it is the subject of the next chapter, *Filtering and Morphology*, which picks the mask up where this one put it down.

CHAPTER 9

Filtering and Morphology

Smoothing, sharpening, edges, and the operators that clean up a mask.

Put a scanned invoice on your desk and look at it the way a program has to. The paper is not white: it is a field of values between 190 and 245, drifting darker toward the corner where the scanner lid did not quite close. The ink is not black either — the thin strokes of a 9-point font land on the sensor as a smear of greys. Scattered across the page are isolated specks, single pixels at 30 and single pixels at 250, dust and sensor noise in equal measure.

Now write the rule that separates ink from paper. Any rule that looks at one pixel at a time is already lost. A pixel at 140 is ink in the bright half of the page and paper in the shadowed corner; a pixel at 30 is either the middle of a stroke or a speck of dust, and nothing about that pixel distinguishes the two. The information you need is not in the pixel. It is in the pixel's neighbourhood — what is around it, and how much like its surroundings it is.

That is the single idea behind every operation in this chapter. A filter looks at a small window around each pixel, computes one number from that window, and writes it to the same position in a fresh destination image. Change what the window computes and you get a different filter: an average smooths, a difference finds edges, a median throws out the specks, a maximum grows bright regions. The destination is always separate from the source, which is why scalacv offers no in-place variant of any of them.

The practical work, then, is the sequence: which filter, in what order, with what window size. A wrong order costs you the thin strokes. A window one size too large closes the counters in the letter "e". By the end of this chapter that scan will be a clean binary mask of the ink, and every step of getting it there will have a reason.

A kernel, slid over the image

A *kernel* is a small grid of numbers — 3×3, 5×5, 7×7. To filter an image with it, centre the grid on a pixel, multiply each kernel number by the pixel underneath it, add the products together, and write that sum to the destination. Then move one pixel right and do it again. That is the whole operation, and everything interesting comes from choosing the numbers.

Three rules let you predict the result without doing any of the arithmetic:

- If the numbers are all positive and *sum to one*, the output is a weighted average of the neighbourhood. Overall brightness is preserved and detail is smeared. That is a blur.
- If the numbers *sum to zero*, flat regions produce zero and only places where the image changes produce anything at all. That is a derivative — an edge detector.

- If the centre is a large positive number, its neighbours are negative, and the whole thing still sums to one, you get the original image plus its own difference from its surroundings. That is a sharpen.

scalacv does not expose a general convolution: there is no `filter2D` on the public surface, and the named operations carry their own kernels. `emboss` is the only one that writes a convolution kernel out by hand — three rows of `CV_32F` behind the library's single `filter2D` call — and it is worth reading precisely because it is directional and therefore easy to predict:

Example 9-1. The one kernel scalacv writes out by hand.

```
-2  -1   0
-1   1   1
 0   1   2
```

The numbers sum to one, so mid-grey stays mid-grey. They are negative up-and-left and positive down-and-right, so a boundary running across that diagonal produces a bright side and a dark side — relief lighting, from the top left. Rotate the sign pattern and the light moves. That is all `emboss` is, and every other operation in this chapter is the same trick with different numbers.

Blurring, and choosing which blur

The reason to blur before almost anything else is that noise is high-frequency and structure is not. An averaging window pulls a lone speck back toward its neighbours and leaves a broad stroke roughly where it was — but it does the same to real detail, so the question is never whether to blur but how much, and with which family. scalacv gives you four, and they differ in what they preserve rather than in how blurry the result looks.

Table 9-1. The blur family: what each one computes, and what survives it.

Operation	What the window computes	Edges	Reach for it when
`boxBlur`	plain average of the window	smeared	you want the cheapest possible smoothing
`gaussianBlur`	bell-weighted average, centre heaviest	smeared	general denoise, and before every edge detector
`medianBlur`	the median value in the window	mostly kept	salt-and-pepper specks
`bilateralFilter`	weighted average, but only over pixels of similar value	kept	you must smooth flat regions and keep boundaries sharp

The high-level entry point is `blur(radius)`, which is a Gaussian expressed in the unit you actually think in. A radius of 2 is a 5×5 kernel — the side is `radius * 2 + 1`, so radius 1 is 3×3, radius 3 is 7×7. A negative radius is rejected with `IllegalArgumentException`; a radius of 0 is the identity.

Example 9-2. Radius is half the kernel side, minus the centre pixel.

```
Image.reading("scan.jpg") { img =>
  img.gray.blur(2).write("soft.png")   // 5x5 Gaussian
}
```

That identity case is not free, and it is the first place native lifetime intrudes on a chapter that looks like pure arithmetic.

> **NATIVE MEMORY**
> `blur(0)` copies no pixels, but it still *spends* the image it was called on, exactly like every other transform. The implementation moves the `Mat` out of the receiver's handle into a fresh `Image` rather than handing the same handle back, so the source is left spent instead of aliased alive. Anything else would let you hold two `Image` values over one native buffer, and the second `close()` would be a double free. The rule is uniform: after `img.blur(n)`, for any `n`, `img` is gone.

When you need to name the kernel and the standard deviation yourself, `gaussianBlur` takes them directly. Both extents must be odd and positive, or `Size(0, 0)` to let OpenCV derive the kernel from the sigmas — and with `Size(0, 0)` you must give a positive `sigmaX`, because otherwise there is nothing to derive from. A `sigmaY` of 0 means "same as `sigmaX`", OpenCV's own convention and not a degenerate value. `Size` holds two `Double`s and they are truncated toward zero on the way into native code, so the check runs on the truncated integers: `Size(4.9, 4.9)` is rejected as an even 4×4 kernel rather than accepted as 5×5. `boxBlur` runs that same check with the zero kernel disallowed, because it has no sigma to derive a size from.

Example 9-3. An explicit kernel and sigma, high level and mid level.

```
img.gaussianBlur(Size(5, 5), sigmaX = 1.5)              // Image
mat.gaussianBlur(Size(0, 0), sigmaX = 2.0)              // Managed[Mat]
```

> **NOTE**
> The mid-level operation is called `boxBlur`, not `blur`, on purpose. `Image.blur` is a radius-based *Gaussian* a `Mat` method sharing that name would silently change filter family — and output — the moment a reader dropped from `image.blur(2)` to `image.mat.blur(...)`. Two algorithms, two names.

Median, and why the scan needs it

A Gaussian averages, and an average is dragged by an outlier. A single black speck in a white field does not disappear under a 5×5 Gaussian; it spreads into a grey smudge 5 pixels across, which is worse, because now it survives a threshold as a blob instead of a dot.

A median has no such weakness. Sort the values in the window, take the middle one, and a lone speck — by definition a minority in its own neighbourhood — is discarded outright

rather than averaged in. Edges survive too: at a boundary, more than half the window is on one side, so the median lands on that side rather than halfway between. That is the whole case for `medianBlur` on a scanned page, where the noise is exactly salt-and-pepper. The high-level form takes a radius like `blur` does, so radius 1 is the 3×3 window, and unlike `blur` there is no identity: radius must be at least 1. The mid-level form takes the kernel side directly and requires it odd and at least 3.

Example 9-4. Two spellings of the same 3x3 median.

```
img.medianBlur(1)      // Image — radius 1, so a 3x3 window
mat.medianBlur(3)      // Managed[Mat] — the side, odd and >= 3
```

Bilateral, the expensive one

`bilateralFilter` asks a second question. A Gaussian weights a neighbour by how far away it is; a bilateral filter weights it by how far away it is *and* by how different its value is. A neighbour across a strong boundary is discounted almost to nothing, so the average never crosses the edge: flat regions go smooth and the boundary stays as crisp as it started.

You pay for it. There is no separable shortcut and no small fixed kernel: the cost per pixel scales with the diameter of the neighbourhood, so assume this one is an order of magnitude dearer than a Gaussian until you have measured otherwise.

Example 9-5. Bilateral: the high-level defaults, and the mid-level call that has none.

```
img.bilateralFilter()                                    // 9, 75, 75
img.bilateralFilter(diameter = 9, sigmaColor = 75, sigmaSpace = 75)
mat.bilateralFilter(9, 75.0, 75.0)                       // no defaults
here
```

A `diameter` of 0 or less lets OpenCV derive the neighbourhood size from `sigmaSpace`. `sigmaColor` is the one to tune first: raise it and more distant values count as "similar", so more edges get smoothed away; lower it and the filter becomes conservative to the point of doing nothing.

The scan, first attempt

With one blur chosen, here is the obvious pipeline for the invoice — and it is wrong in two ways that are worth seeing before the fix.

Example 9-6. The version that loses the shadowed corner.

```
Image.reading("scan.jpg") { img =>
  img.gray
     .blur(2)
     .threshold(127)
```

```
      .write("ink-bad.png")
}
```

The first fault is the blur. A 5×5 Gaussian on 9-point text at 300 dpi is wider than the strokes it is smoothing; the thin verticals come out grey, and the threshold then decides they are paper. The second fault is `threshold(127)`. One cut for the whole page cannot be right for both the bright half and the corner the lid missed: high enough to catch the ink in the shadow and the bright half fills with paper texture; low enough to keep the bright half clean and the corner goes solid black.

Both faults have the same shape — a fixed global decision cannot cope with something that varies across the image. That is what `adaptiveThreshold` is for: it computes a cut per neighbourhood rather than once for the page, which is why it is the standard preparation step before OCR.

Example 9-7. The same page, with a filter that does not smear and a cut that follows the light.

```
Image.reading("scan.jpg") { img =>
  img.gray
     .medianBlur(1)                                  // 3x3: specks go, strokes
stay
     .adaptiveThreshold(blockSize = 25, c = 8, mode = Threshold.Mode.BinaryInv)
     .write("ink.png")
}
```

`blockSize` is the odd neighbourhood side the local threshold is computed over. It wants to be comfortably larger than a stroke and smaller than the lighting variation — 25 pixels is a reasonable start for body text on a 300 dpi scan. `c` is subtracted from the local mean, so raising it keeps less: turn it up when background texture starts coming through. `mode = Threshold.Mode.BinaryInv` puts the ink at 255 and the paper at 0, the convention every operation in the rest of this chapter expects. Like `equalizeHist`, the operation is `CV_8UC1` only — the other reason `gray` comes first — and `blockSize` must be odd and at least 3.

Name those arguments. The `Image` verb leads with the two you tune, `adaptiveThreshold(blockSize, c,`; the mid-level `Ops` version mirrors OpenCV's own `method, inverse)`
`(maxValue, method, blockSize, c,`. The divergence cannot bite silently — the leading para-`inverse)`
meters have different types across the tiers, `blockSize: Int` against `maxValue: Double`, so a positional call written for one tier does not compile against the other — and named arguments make the question moot anyway.

The result is close, and it is not clean. Scattered through it are one- and two-pixel white specks the median did not catch, and some strokes have hairline gaps where the paper showed through. That is a job for a different family of operator entirely.

Edges: derivatives, and the two-threshold trick

Before the morphology, the other reason to filter: finding where the image changes. An edge is a place where intensity moves quickly, so an edge detector is a derivative. `sobel` is the workhorse — the derivative in x, in y, or a mixed higher-order one, over a kernel that also does a little smoothing so noise does not register as a hundred tiny edges. It lives at the `Mat` tier only, as does `laplacian`: there is no `Image.sobel`, because a raw derivative is rarely the last step. Computed correctly it comes back 16-bit signed, and something has to bring it down to 8 bits before it is an image you can write. So the worked example drops a tier.

Example 9-8. The horizontal derivative --- and the depth that makes it correct.

```
Mats.chain(frame)(
  _.cvtColor(ColorConversion.BgrToGray),
  _.sobel(dx = 1, dy = 0, depth = OutputDepth.Signed16),
  _.convertScaleAbs()
)
```

That `depth` argument is the trap the `OutputDepth` type exists to warn you about. Its default, `OutputDepth.SameAsSource`, means "give the destination the source's depth" — and on the commonest input, an 8-bit unsigned image, that clips every negative derivative to zero. A derivative is negative wherever the image gets darker left-to-right, which is one side of every edge in the frame, so half of each edge silently disappears and nothing tells you. Compute into `OutputDepth.Signed16`, which keeps the negative lobe, and bring it back down with `convertScaleAbs`, which scales, takes the absolute value, and saturating-casts to 8-bit. `laplacian` — the second derivative, and therefore isotropic rather than directional — has the identical trap and the identical remedy.

> **NATIVE MEMORY**
> Every mid-level operation allocates a fresh destination `Mat` and hands you something you own, so a three-stage chain written naively strands two full-size buffers. `Mats.chain` exists for this shape: it releases each intermediate the moment the next stage has produced its own output, in a `finally`, so a stage that throws does not leak its input either. The source you pass in is borrowed and never released — it belongs to whoever created it. `chain` is a fold over `pipe`, the `Managed[Mat]` extension that feeds one stage and releases its input; `pipe` is what to reach for when there is only one stage to add.

There is no separate `scharr` method, because OpenCV's Scharr operator *is* Sobel with a particular 3×3 kernel, and the library exposes it the way OpenCV does: pass `kernelSize = -1`. The check on that parameter says as much in its own words — `sobel kernelSize must be odd and positive, or -1 for the 3x3 Scharr kernel` — so the one value that looks illegal is the documented one. Scharr is the more rotationally accurate of the two at that size, so prefer it when you will use the gradient's *direction* and not only its magnitude.

Example 9-9. Scharr is a kernel size, not a method.

```
mat.sobel(dx = 1, dy = 0, kernelSize = -1, depth = OutputDepth.Signed16)
```

Canny, and what the two thresholds actually do

`canny` is not a kernel. It is a four-step algorithm that runs a Sobel internally, thins the response down to one-pixel ridges, and then decides which of those ridges to keep — and the deciding is where its two thresholds come in. They are widely misread as "low detail" and "high detail" knobs, which they are not.

The rule is *hysteresis*. A candidate whose gradient is above `threshold2` is accepted unconditionally: those are the strong edges. A candidate below `threshold1` is discarded outright. Everything in between — the weak edges — is kept *only if it is connected, through other in-between pixels, to something already accepted.*

That solves a specific problem. A single threshold on a real image is always wrong: high enough to reject texture, and long edges break into dashes wherever they dim; low enough to keep them continuous, and the frame fills with noise. Hysteresis lets you set the strong threshold aggressively — high enough that essentially nothing spurious clears it — and then rescue the dim continuations of the edges it did find, because a genuine edge that fades is still attached to the part of itself that did not. So: `threshold2` is the level above which you are confident you have an edge, and `threshold1` is between a half and a third of it.

Table 9-2. Starting values for canny, for a normally-exposed 8-bit image.

`threshold1`	`threshold2`	Character
50	150	permissive — fine detail and some texture; the usual first try
80	160	the pairing this library's own examples use throughout; balanced
100	200	conservative — strong boundaries only, texture rejected
30	90	low-contrast material: faded scans, fog, underexposure

> **WARNING**
> `canny(threshold1, threshold2)` takes two `Double`s in an order nothing enforces, and swapping them does not fail — it quietly changes which edges survive. Name them at the call site (`canny(threshold1 = 80, threshold2 = 160)`) whenever the pair is not obviously ordered. The result is always `CV_8UC1` regardless of the input type, and `apertureSize` — the internal Sobel aperture — accepts only 3, 5 or 7. Anything else is rejected in Scala with `IllegalArgumentException`, because OpenCV's own response is to abort in native code.

Sharpening

Sharpening is the inverse of blurring, quite literally. Take the image, take a blurred copy of it, and the difference between them is precisely the detail the blur removed; add that difference

back on top of the original and the detail is exaggerated. The technique is *unsharp masking*, named after the darkroom practice of using a deliberately *unsharp* negative as the mask.

`sharpen(amount)` is that in one call — `image + amount * (image - blur(image))`, implemented as a Gaussian with `Size(0, 0)` and a sigma fixed at 3.0, followed by an `addWeighted` of `1 + amount` against `-amount`. An `amount` of 0 is a no-op, around 1 is a firm sharpen, and a negative amount is rejected.

Example 9-10. Unsharp masking, and the point at which it stops helping.

```
img.sharpen()          // amount = 1.0
img.sharpen(0.4)       // gentle
img.sharpen(3.0)       // haloes
```

Learn the failure mode by sight. Because the added detail is a difference across an edge, a large `amount` puts a bright fringe on the bright side and a dark fringe on the dark side — a *halo*. Once you can see haloes, the number is too high, and no further sharpening will recover detail that was never in the file.

Morphology: the operators for a binary mask

The scan's remaining problems — isolated specks and hairline gaps — are no longer noise in the grey-level sense. The image is binary now: ink at 255, paper at 0. What is left is a question about *shape*, and morphology is the family that answers shape questions.

Morphology also slides a window, but instead of averaging it takes an extremum. *Erosion* writes the minimum of the neighbourhood, so a white region survives only where the whole window fits inside it: bright regions shrink from the boundary inward, and anything smaller than the window disappears. *Dilation* writes the maximum, so a white region grows outward by the window's radius and any dark gap narrower than the window is filled. Neither is much use alone — both resize everything — but their compositions are.

Table 9-3. The compound operators, and what each is for on a binary mask.

`MorphOp`	Definition	Use it to
`Open`	erode, then dilate	delete specks smaller than the kernel, leaving everything else the size it was
`Close`	dilate, then erode	fill holes and join gaps narrower than the kernel, leaving everything else the size it was
`Gradient`	dilation minus erosion	outline every region — a one-pass boundary of a mask
`TopHat`	source minus its opening	isolate the bright detail smaller than the kernel
`BlackHat`	closing minus the source	isolate the dark detail smaller than the kernel

`Open` and `Close` are the two you will use constantly because each removes something without resizing what it leaves. Erosion alone shrinks the specks *and* the strokes; erosion followed by dilation deletes the specks — once eroded away there is nothing left for the dilation to grow back — and restores the strokes to roughly their original width. `Close` is the same argument reversed, which is why it fills gaps without fattening the glyphs.

Example 9-11. The scan, finished.

```
Image.reading("scan.jpg") { img =>
  img.gray
     .medianBlur(1)                                    // sensor specks
     .adaptiveThreshold(blockSize = 25, c = 8, mode = Threshold.Mode.BinaryInv)
     .morphology(MorphOp.Open, radius = 1)             // survivors of the median
     .morphology(MorphOp.Close, radius = 1)            // rejoin broken strokes
     .write("ink.png")
}
```

Two knobs decide what those calls do, and there are only two. The first is `radius`, which works much as it does for `blur`: the structuring element is `radius * 2 + 1` pixels on a side, so radius 1 is 3×3 — and, as with `medianBlur` and unlike `blur`, it must be at least 1. Radius is the size threshold in the most direct sense available — `Open` at radius 1 deletes anything a 3×3 square cannot sit inside, radius 2 anything a 5×5 cannot. On text, radius 1 is usually where you stop; radius 2 starts closing the counters in “e” and “a”.

The second is `shape`, a `MorphShape` with three cases.

Table 9-4. Structuring-element shapes, and when the choice matters.

`MorphShape`	The element	Choose it when
`Rect`	a filled square — the default	you have no reason to prefer another; it is the fastest
`Ellipse`	a filled disc inscribed in the square	the shapes are round or organic and a square element would leave square corners on them
`Cross`	the centre row and centre column only	you want to affect horizontal and vertical runs and leave diagonals alone — table rules, scan lines

At radius 1 the three are nearly indistinguishable — a 3×3 disc and a 3×3 square differ by four corner pixels. The choice starts to matter from radius 2 upward, and it matters most on `Close`: closing a round blob with a `Rect` element leaves visible square corners on it, which is exactly the artefact `Ellipse` avoids.

> **NATIVE MEMORY**
> Morphology is the one family here that needs a second native allocation — the structuring element itself, from `getStructuringElement`. `Ops` builds it inside `withStructuringElement`, which frees it on every path including the throwing one, so a morphology call still leaves you owning

exactly one buffer: the destination. The kernel never reaches your code and there is nothing about it to release.

Two asymmetries to know. The mid-level `erode`, `dilate` and `morphology` all take an `iterations` parameter (at least 1, applying the operator repeatedly) and the `Image` verbs do not, so on `Image` you chain the calls or drop a tier. And there is deliberately no bare `close` method on `Image` for `MorphOp.Close` — `close()` already means "release the native memory", and two meanings for that name would be an excellent way to free an image you meant to filter.

What happens at the edge of the image

Centre a 5×5 kernel on the pixel at `(0, 0)` and two of its rows and two of its columns hang off the image. Every filter in this chapter has to invent those pixels, and `BorderType` is how you say what to invent.

Table 9-5. Border modes, shown on the pixels `a b c d | ...` at the left edge.

`BorderType`	Invents	Notes
`Constant`	`0 0 0 0 | a b c d`	a fixed colour, black by default; the default for `pad`, `border` and `rotated`
`Replicate`	`a a a a | a b c d`	the edge pixel, repeated
`Reflect`	`d c b a | a b c d`	mirrored, with the edge pixel duplicated
`Reflect101`	`e d c b | a b c d`	mirrored *about* the edge pixel, so it is not duplicated — the default for every filter that takes a border
`Wrap`	`w x y z | a b c d`	the opposite edge, as if the image tiled — *not valid for filters*

`Reflect101` is the filter default for a good reason: it introduces no artificial gradient at the boundary. `Constant` puts a hard black step just outside the image, and an edge detector will happily report a bright edge around your whole frame that is not in your data. `Replicate` is the alternative when reflection would be wrong — a gradient image, where the mirrored continuation runs backwards.

Four filters take a `border` parameter: `gaussianBlur`, `boxBlur`, `sobel` and `laplacian`, each defaulting to `Reflect101`. `medianBlur`, `bilateralFilter` and `canny` do not expose one, and neither does any high-level filter — `Image.pad` and `Image.border` take a `borderType`, defaulting to `Constant` filled with `Scalar.Black`, but `Image.blur` and `Image.gaussianBlur` do not, so changing a filter's border means dropping that stage to the `Mat` tier.

WARNING

`BorderType.Wrap` is valid for `pad`, `border` and `rotated`, and invalid for `gaussianBlur`, `boxBlur`, `sobel` and `laplacian`. Passing it to one of the four filters throws `IllegalArgumentException`

from `BorderType.requireFilterSupport` before any native call, naming the operation you asked for.

One int, two domains

That last restriction is not scalacv being fussy. OpenCV packs two different sets of accepted values into one `int`, and `BorderType` is the union of them: `copyMakeBorder` (behind `pad` and `border`) and `warpAffine` (behind `rotated`) honour all five modes, while the `imgproc` filter family does not. Every filter routes through `cv::FilterEngine::init`, which asserts `columnBorderType != BORDER_WRAP` and aborts.

Leaving that to native code would be tolerable if it were consistent. It is not. Verified against OpenCV 4.13: `GaussianBlur` with `BORDER_WRAP` *silently ignores* the mode on `CV_8U` input — the SIMD path never reaches the assertion — and aborts on `CV_32F`. So the same call succeeds against the 8-bit frames it was developed on and crashes the first time it meets a float image, in native code, with no Scala stack frame in sight. Checking at the Scala boundary makes both depths fail identically and names the parameter you actually passed.

Two types — a filter border without `Wrap`, widening into a transform border with it — would make the mistake unrepresentable, but that is a breaking change to four public signatures, so for now the check is a runtime one.

What filtering costs

Cost scales with the kernel, and not linearly. A naive $k \times k$ convolution does k^2 multiply-adds per pixel, so going from 3×3 to 9×9 is nine times the arithmetic, not three.

The saving grace is *separability*. A Gaussian kernel factors into a horizontal pass and a vertical pass over one-dimensional kernels, giving the same result for $2k$ multiply-adds per pixel instead of k^2. OpenCV does this for you — `GaussianBlur`, `blur` and `Sobel` all run separably — so a Gaussian's cost grows roughly linearly with the kernel side. `medianBlur` and `bilateralFilter` do not factor that way, which is the other half of why bilateral filtering is the expensive one.

Real numbers, from this project's harness on one developer machine: the canonical `gray` ⟶ `blur` ⟶ `canny` chain measured 145.3 µs at 640×480, 574.2 µs at 1920×1080, and 7556 µs at 3840×2160. That last figure is more than 13× the 1080p one for 4× the pixels — above some size the working set stops fitting in cache and the curve bends. One machine's microseconds, not yours: the ratios reproduce, the absolutes do not.

TIP
Before you tune a filter chain for speed, check what OpenCV is already doing on your machine. `ConfigProbeBench` prints `useOptimized`, `getNumThreads`, `getNumberOfCPUs` and the parallel, IPP and OpenCL lines of the build information, then measures how a bilateral filter on a 1280×720

> scene scales as `Core.setNumThreads` is stepped through 1, 2, 4 and your core count. The answer is a property of your machine, so run it and read your own.

The project's own rule, worth adopting: no optimisation without a measured delta *and* a bit-identical output hash. A change that is faster because it quietly computes something different is not an optimisation, and a timing number alone cannot tell the two apart. Chapter 35 is where the harness, the reading rule for a confidence interval, and the optimisation that was measured and then deliberately *not* built all live.

Where this leads

The scan is now a binary mask: ink at 255, paper at 0, specks gone, strokes joined. Every step was in service of that — the median to survive the sensor, the local threshold to survive the lighting, the opening and closing to survive both.

Chapter 10, *Geometric Transforms*, takes the other half of the imgproc surface — the operations that move pixels rather than recombine them, and the interpolation choice that decides what a resized image loses.

This chapter also leaned on `adaptiveThreshold` ahead of its proper treatment. Chapter 11, *Thresholding and Colour Segmentation*, gives the cut itself the attention it deserves: the `Threshold.Mode` cases, Otsu and Triangle and the value they compute for you, how to choose between a global cut, an automatic one and the per-neighbourhood variant used here — and then builds masks a second way entirely, from colour in HSV, handing them straight back to the `Open` and `Close` you have just met.

CHAPTER 10

Geometric Transforms

Every pixel that moves is a pixel you invented, and a buffer you now own.

A phone photograph of a delivery note arrives at 4032×3024, held at arm's length, tilted about four degrees off upright, and shaped like a trapezium because the camera was not parallel to the desk. Nothing downstream wants that. The OCR engine wants a flat, upright, roughly 1200×1600 greyscale rectangle with the page filling the frame. Between the two lies this chapter: resize, crop, flip, rotate, warp. It is the least glamorous part of a vision pipeline and the part that most often decides whether the clever part works.

Moving pixels is not free the way copying bytes is free. A geometric transform asks, for every pixel in the destination, where that pixel came from in the source — and the answer is almost never a whole pixel. It is 41.7% of the way between two columns and 0.3 of the way down two rows. Everything in this chapter except the quarter-turns and the axis-aligned crop therefore *invents* values that were never in the input. The interpolation you choose is a choice about what it invents, and the wrong choice is not a crash; it is a slow, invisible loss of the detail your detector was going to key on.

The second cost is the one this book keeps returning to. A transform does not modify a `Mat`; it allocates a second one and fills it. Rotating a 12-megapixel BGR frame allocates a destination of at least 36 MB, and the source's 36 MB is still resident until somebody frees it. A six-stage chain written the obvious way holds six buffers at once and offers, per buffer, about forty bytes of on-heap header as the only evidence to the collector that any of them exist.

scalacv answers that with move semantics. Every transform on `Image` spends the image it was called on and hands back a new one, so a chain of any length holds exactly one live `Mat`; the mid-level `Managed[Mat]` extension ops make the same guarantee explicit with `pipe` and `Mats.chain`. Both levels appear in this chapter, because a document-flattening pipeline needs one operation — a perspective warp — that `Image` does not wrap, and reaching under the high-level API without leaking is a skill this chapter is a good place to learn.

Resize: two modes, and why they are two methods

The docket has to get smaller before anything else happens, both because 12 megapixels is more page than any detector needs and because every subsequent stage costs proportionally less on a smaller frame. There are two ways to say how much smaller, and scalacv keeps them apart.

`resize(width, height)` and `resizeTo(size, interpolation)` name an absolute destination in pixels. `scale(factor, interpolation)` multiplies both sides by one number. At the mid level

the same split is `resize(size, interpolation)` and `scaled(fx, fy, interpolation)`, where `scaled` takes independent factors for the two axes.

Note which of those five takes no filter. `Image.resize(width, height)` is a two-integer convenience that forwards to `resizeTo` with the default `Interpolation.Linear`, and there is no overload that adds a filter to the integer pair. When the filter matters — and the next subsection argues that on a downscale it always does — the call to reach for is `resizeTo(Size(w, h), Interpolation.Area)` rather than `resize(w, h)`.

Example 10-1. Absolute size, uniform factor, and independent factors.

```
import scalacv.*

OpenCv.load()

Image.reading("docket.jpg") { full =>
  full.copy.resize(1200, 1600).close()                              // absolute
  full.copy.resizeTo(Size(1200, 1600), Interpolation.Area).close()  // absolute,
chosen filter
  full.copy.scale(0.25, Interpolation.Area).close()                 // uniform
factor
  full.mat.scaled(fx = 0.25, fy = 0.5).use(_.rows)                  // mid-level,
per-axis
}
```

They are two methods rather than one overload because OpenCV distinguishes the modes by passing `Size(0, 0)` as a sentinel, and a sentinel size has no business in a typed API. Keeping them apart also lets each one validate what it actually receives, and the two need different validation — which turns out to be more interesting than it sounds.

Two roundings, on purpose

`resize` truncates. A `Size` carries two `Double` extents and OpenCV truncates them toward zero on the way into native code, so `Size(1.9, 1.9)` asks for a 1×1 image. `scaled` rounds — and rounds half to *even*, because OpenCV derives the destination as `cvRound(cols * fx)` × `cvRound(rows * fy)` and `cvRound` is a banker's rounding.

The library validates each one the way its own native path behaves: `resize` checks the truncated integers, `scaled` uses `math.rint` against the receiver's extent. That asymmetry is deliberate and documented in the source. A single shared rule would be wrong for one of them: on a 100-pixel-wide source, `fx = 0.006` legitimately yields a 1-pixel result that truncation would reject, and `fx = 0.025` yields 2 where `math.round` insists on 3.

The payoff is the error you get when a computed target collapses. `Size(width * 0.004, height * 0.004)` on a small sprite is positive as a `Double` and empty as a `cv::Size`; without the check, OpenCV aborts with `CV_Assert(inv_scale_x > 0)` and hands you

a `CvError.NativeCall` quoting a C++ expression. With it you get an `IllegalArgumentException` naming your own argument.

Choosing an interpolation

`Interpolation` is a true enumeration with five cases, and its `.cvValue` is the raw `Imgproc.INTER_*` constant if you ever need to hand it to a call scalacv does not wrap. The default everywhere in the library is `Interpolation.Linear`, which is the right default and the wrong choice at both ends of the range.

Table 10-1. When each interpolation is the right one.

`Interpolation`	Reach for it when	What it actually does
`Nearest`	Label maps, binary masks, index images	Copies the closest source pixel. Blocky, but exact values — 0 and 255, a class id — survive instead of being averaged into something that is neither
`Linear`	General-purpose, either direction	The default. A weighted average of four neighbours; fast, and good enough that most pipelines never change it
`Cubic`	Enlarging	Sixteen neighbours and a smoother reconstruction. Keeps edges crisp where `Linear` softens them. Slower
`Area`	Shrinking	Averages every source pixel that falls into the destination pixel. This is the one that matters: it is the only filter that looks at all the data it is throwing away
`Lanczos4`	Enlarging, when quality beats throughput	An eight-tap windowed sinc. The sharpest option and the slowest

The rule worth memorising is the one about `Area`. Shrink with `Linear` and each destination pixel is built from four source pixels, but at a factor of 0.25 each destination pixel stands for a 4×4 block of the source — so twelve of every sixteen source pixels are never read at all. The result is aliasing: fine repeating texture — the ruling on a form, a halftone screen, the weave of a fabric — folds into moiré patterns that were not in the scene and that no downstream stage can tell from real structure. `Area` averages the whole footprint instead, so the detail is discarded as an average rather than as a coin flip.

> **TIP**
> Shrink with `Interpolation.Area`, enlarge with `Interpolation.Cubic` or `Interpolation.Lanczos4`, and use `Interpolation.Nearest` for anything whose pixel values are labels rather than light.

So the first line of the docket pipeline is a downscale with `Area`, not the default:

Example 10-2. The first stage: a working copy at a quarter size.

```
val working = full.copy.scale(0.25, Interpolation.Area)   // 4032x3024 ->
1008x756
```

> **NATIVE MEMORY**
> `full.copy` allocates a second 36 MB buffer, and `scale` allocates a third — 2.3 MB, since each side is quartered. The copy is spent by `scale` and freed immediately, so two buffers survive the line: `full`, which `Image.reading` will close, and `working`, which is now yours. Without the `.copy` you would not have needed the second allocation at all — but you would also have spent `full`, and the whole point of the pipeline below is that the full-resolution original is still there at the end.

Crop, and the check OpenCV does not perform

`crop(rect)` cuts an axis-aligned region out of the image. The `Rect` is `Rect(x, y, width, height)` — corner first, then extent, which is a different shape from `Point` plus `Size` and trips everybody up exactly once.

Two things happen before OpenCV sees anything. First, the rectangle is checked against the image:

```
require(
  rect.x >= 0 && rect.y >= 0 && rect.x + rect.width <= width && rect.y +
rect.height <= height,
  s"crop $rect does not fit inside ${width}x$height"
)
```

This matters more than it looks. A `Rect` that runs off the edge is not a rare accident; it is the normal output of padding a detector's bounding box by twenty pixels for context, and OpenCV's `submat` responds to it by throwing from native code with a message about a region that does not fit inside a matrix, naming neither of the values you passed. The `require` fails in Scala, before JNI, quoting your rectangle and the image it did not fit.

Second, the crop is copied. `submat` gives an aliasing view onto the parent's pixel data — a new `Mat` header pointing into the same buffer — and scalacv clones it, releases the view, and hands back an independent image.

> **NATIVE MEMORY**
> An aliasing `submat` view outlives nothing. Release the parent and the view is a live handle into freed memory, which is a SIGSEGV with no Scala frame in it. `crop` clones so the result is a real image you can keep, put in a `Map`, or send to another thread after the source is long gone. That is the same reasoning behind `Rect` and `Point` being ordinary case classes: everything that crosses the boundary out of native memory is copied, so nothing you hold is secretly a pointer.

The exact turns

Three operations move pixels without inventing any. `flip(how)` mirrors, and `Flip` is named for the visible effect rather than for OpenCV's flip code: `Flip.Horizontal` mirrors left-to-right, `Flip.Vertical` top-to-bottom, `Flip.Both` does both, which is a 180° point reflection. `rotate` with a `Rotation` does a quarter or half turn as a pure pixel shuffle.

Table 10-2. The lossless rotations.

`Rotation`	Angle	Effect on the frame
`Clockwise`	90° clockwise	Width and height swap
`CounterClockwise`	90° anticlockwise	Width and height swap
`Half`	180°	Unchanged

Nothing is resampled, so these are exact and they compose: four `Clockwise` turns return the original bytes. They are also the cheapest fix for a camera mounted sideways. Use them whenever the angle is a multiple of 90, and reach for the arbitrary rotation below only when it is not.

Arbitrary rotation, and the canvas that has to grow

The docket is tilted four degrees. Correcting that is an arbitrary-angle rotation, which OpenCV expresses as a 2×3 affine matrix fed to `warpAffine`. Written directly, the obvious version is wrong:

Example 10-3. Wrong: the corners fall off the edge.

```
import org.opencv.core as cv
import org.opencv.imgproc.Imgproc
import scalacv.*

/** Rotates about the centre, into a destination the same size as the source. */
def clipped(src: Image, degrees: Double): Image =
  val w = src.width
  val h = src.height
  Managed.scope: own =>
    val m = own(Imgproc.getRotationMatrix2D(cv.Point(w / 2.0, h / 2.0), degrees,
1.0))
    val dst = Managed(cv.Mat())
    Imgproc.warpAffine(src.mat, dst.get, m, cv.Size(w.toDouble, h.toDouble))
    Image.wrap(dst)
```

Rotate a rectangle inside a frame of its own size and the four corners leave the frame. At 30° on a 160×120 image the rotated rectangle keeps about 83% of its area inside the old frame, so a sixth of the picture is gone — and it is gone from the corners, which is exactly where the page edges you were about to detect live. The fix is arithmetic, not a flag: the destination

has to be the axis-aligned bounding box of the rotated rectangle, and the matrix's translation column has to be shifted so the rotated image lands inside it.

For a source `w` wide and `h` high, rotated by an angle whose matrix carries `cos` and `sin` in its first row, the enlarged canvas is

```
newW = round(h * abs(sin) + w * abs(cos))
newH = round(h * abs(cos) + w * abs(sin))
```

and the third column of each row gains `(newW - w) / 2` and `(newH - h) / 2` respectively — a recentring, so the rotation about the old centre ends up about the new one. For the 160×120 frame at 30° that is a 199×184 destination — nearly twice the pixel count of the 160×120 source, which is the price of not losing the corners.

You do not have to write that. `Image.rotate(degrees, scale)` and the mid-level `rotated` do it for you, which is why the high-level rotation never clips:

Example 10-4. Right: the canvas grows to fit.

```
val tilted = Image.blank(160, 120).rotate(30)                // 199x184, nothing
clipped
val zoomed = Image.blank(160, 120).rotate(30, scale = 0.5)   // turn and halve in
one warp

// Mid-level, on a borrowed Mat: a Managed[Mat] with the knobs Image does not
surface.
val spun = frame.mat.rotated(
  degrees = 15,
  scale = 1.0,
  interpolation = Interpolation.Cubic,
  border = BorderType.Constant,
  color = Scalar.White
)
```

`Image.rotate` surfaces two of `rotated`'s five parameters; the other three — `interpolation`, `border` and `color` — are why the mid-level call still earns its place. Note the default rather than the name: the fill defaults to `BorderType.Constant` here and on `pad` / `border` (a geometric transform exposes never-seen pixels, where reflecting image content would smear it into the border), while the **filters** (`gaussianBlur`, `sobel`, …) default to `Reflect101`, which keeps the kernel's support inside real image content and avoids edge ringing.

The `scale` argument is not a convenience wrapper around a separate resize — it goes into the same matrix, so a rotate-and-shrink resamples once instead of twice. One resampling is always better than two.

> **WARNING**
> `Image.rotate(degrees)` measures the angle *counter-clockwise*, which is OpenCV's `getRotationMatrix2D` convention, so `rotate(90.0)` and `rotate(Rotation.CounterClockwise)`

> agree. `Picture.rotate(degrees, about)` in the `scalacv-graphs` scene graph measures the same angle *clockwise*, because it works in screen coordinates where y points down and it turns vector geometry rather than pixels. The two are documented and both are correct for what they transform; they are not interchangeable.

Border modes: what fills the corners

A rotated rectangle inside its bounding box leaves four triangular regions that correspond to no source pixel at all, and something has to go in them. That something is a `BorderType`, and the same enum controls what `pad` and `border` put in the margin they add. It has five cases; OpenCV's sixth constant, `BORDER_ISOLATED`, is deliberately absent, because it is a modifier that only changes the behaviour of region-of-interest calls scalacv does not yet expose, and an enum case that is silently ignored is worse than one that is missing.

Table 10-3. Border modes, and where each one belongs.

`BorderType`	Fills with	Use it for
`Constant`	The `color` you pass; `Scalar.Black` by default	A visible frame, a letterbox, or a white page background under a deskew
`Replicate`	The nearest edge pixel, repeated outward	Padding before a filter, so the filter does not see a fake edge
`Reflect`	A mirror of the edge, including the edge pixel	Seamless tiling
`Reflect101`	A mirror excluding the edge pixel	OpenCV's own default for filter borders
`Wrap`	Pixels from the opposite side	Genuinely periodic images — and only via `pad`, `border` or `rotated`

Example 10-5. Padding a page with white instead of black.

```
Image.reading("docket.jpg") { img =>
  img.pad(24, color = Scalar.White).write("matted.png")          // uniform
margin
}

Image.reading("docket.jpg") { img =>
  img.border(top = 4, bottom = 4, left = 20, right = 20,
             borderType = BorderType.Replicate).write("framed.png")
}
```

`Wrap` is the one value in the enum that is not universally accepted. `copyMakeBorder` — behind `pad` and `border` — honours it, and so does `warpAffine` behind `rotated`. OpenCV's `imgproc` filter family does not: `gaussianBlur`, `boxBlur`, `sobel` and `laplacian` all reach `cv::FilterEngine::init`, which asserts that the border type is not `BORDER_WRAP` and aborts.

> **WARNING**
> That abort is inconsistent across depths — the same call can pass silently on one and kill the process on another — which is why scalacv runs `BorderType.requireFilterSupport` on every filter's `border` argument rather than letting native code decide; Chapter 9, *Filtering and Morphology*, has the measurement. `BORDER_TRANSPARENT` is absent from the enum entirely for a related reason: `copyMakeBorder` throws on it and `warpAffine` with it leaves the freshly-allocated destination uninitialised, so it can only ever produce a crash or garbage pixels.

Reading a 2×3 affine

A rotation is one member of a family. Every affine transform is a 2×3 matrix that maps a source point to a destination point by

```
x' = a*x + b*y + c
y' = d*x + e*y + f
```

which is a 2×2 linear part — rotation, scale, shear, and any composition of them — plus a translation in the third column. The scene graph in `scalacv-graphs` carries exactly these six numbers internally and exposes them as `Picture.translate(dx, dy)`, `Picture.rotate(degrees, about)` and `Picture.scale(factor, about)`; at the pixel level, you build the matrix yourself as a `CV_64F` `Mat` and hand it to `warpAffine`.

Table 10-4. The affine building blocks, as 2×3 matrices.

Transform	Matrix	Note
Identity	`[1 0 0 ; 0 1 0]`	Copies
Translate by `(dx, dy)`	`[1 0 dx ; 0 1 dy]`	Pure third column; sub-pixel `dx` resamples
Scale by `(sx, sy)`	`[sx 0 0 ; 0 sy 0]`	About the origin, not the centre
Rotate by θ	`[cos θ  −sin θ 0 ; sin θ  cos θ 0]`	About the origin. With y pointing down this turns clockwise on screen
Shear x by `k`	`[1 k 0 ; 0 1 0]`	Slants verticals; leaves horizontals alone

Anything about a point other than the origin is a translate, then the transform, then the inverse translate — which is precisely what `getRotationMatrix2D(center, angle, scale)` composes for you, and why its third column is not zero.

Example 10-6. A hand-built shear, warped by hand.

```
import org.opencv.core as cv
import org.opencv.core.CvType
```

```
import org.opencv.imgproc.Imgproc
import scalacv.*

/** Slants `src` by `k` (x shifted by k*y), into a canvas wide enough for the
lean. */
def sheared(src: Image, k: Double): Image =
  val w = src.width
  val h = src.height
  Managed.scope: own =>
    val m = own(cv.Mat(2, 3, CvType.CV_64F))
    m.put(0, 0, 1.0, k, if k < 0 then -k * h else 0.0)
    m.put(1, 0, 0.0, 1.0, 0.0)
    // Allocated last and wrapped immediately: this is the result, so the scope
must not own it.
    val dst = Managed(cv.Mat())
    Imgproc.warpAffine(
      src.mat, dst.get, m,
      cv.Size(w + math.abs(k) * h, h.toDouble),
      Interpolation.Linear.cvValue,
      BorderType.Constant.cvValue,
      cv.Scalar(255, 255, 255)
    )
    Image.wrap(dst)
```

That listing is the pattern for every unwrapped OpenCV call in this chapter, so it is worth reading twice. `Managed.scope` owns the working matrices and releases them in reverse order even if `warpAffine` throws part-way. The destination is created *outside* the scope's ownership, because it is the one object that has to survive the block, and `Image.wrap` transfers that ownership to the returned `Image`. `.cvValue` gives the raw int for the typed enums, so you keep the typed vocabulary right up to the JNI boundary.

> **NATIVE MEMORY**
> `getRotationMatrix2D` and `getPerspectiveTransform` both *return* a fresh `Mat`. It is small — six or nine doubles — but it is native, it is yours, and nothing in OpenCV's Java API suggests as much. A per-frame warp that forgets it leaks a `Mat` header and its buffer thirty times a second. Wrapping the return value in `own(...)` the moment it appears is the habit that makes the mistake impossible.

Three points or four: affine versus perspective

An affine transform has six degrees of freedom, which is exactly three point correspondences. It can translate, rotate, scale, shear, and it preserves one thing absolutely: parallel lines stay parallel. A rectangle becomes a parallelogram, never a trapezium.

The docket is a trapezium. Its top edge is shorter than its bottom edge because the top of the page was further from the lens, and no affine matrix can undo that. What you need is a *projective* transform — a 3×3 homography with eight degrees of freedom, which is four point correspondences — applied with `warpPerspective`.

Table 10-5. Which warp the job needs.

	Correspondences	Matrix	Preserves
`warpAffine`	3 point pairs	2×3	Parallel lines, midpoints, ratios along a line
`warpPerspective`	4 point pairs	3×3	Straight lines only

The distinction is not academic. If the tilt is a rotation in the image plane — a page that was straight on the desk but scanned crooked — an affine rotation is correct and cheaper, and `Image.deskew(maxAngle)` will even find the angle for you. It thresholds with an inverted Otsu so the ink becomes the non-zero pixels, fits a minimum-area rectangle to them, folds that rectangle's angle into the equivalent tilt in the range −45° to 45°, and rotates by it, filling the exposed corners white. It returns a plain copy in two cases: a tilt under 0.1°, which is not worth a resample, and a tilt beyond `maxAngle` (default `45.0`), which is treated as a misread on the grounds that a page of large graphics can fool the estimate.

One difference from `rotated` is easy to miss and matters here: `deskew` keeps the frame size. It warps into a destination of the source's own `cols` × `rows` rather than into a grown bounding box, because a deskew is a correction of a degree or two and growing the canvas for it would leave every downstream stage handling a slightly different frame size per page. The corners it exposes are inside the original frame, which is why filling them white is enough. If the tilt is out of the plane rather than in it, no amount of affine rotation helps and only a homography will flatten it.

Flattening the docket

scalacv wraps `warpAffine` behind `rotated` and `deskew`, but it does not wrap `warpPerspective`. The low-level surface is one step away, as always, and the four-corner warp is a good demonstration that the escape hatch is a door rather than a hole. The work is in three parts: find the quadrilateral, order its corners, and warp them onto a rectangle.

Example 10-7. Finding the page: the largest four-sided contour.

```
import scala.util.Using
import scalacv.*

/** The page outline, if the largest contour simplifies to a quadrilateral. */
def pageQuad(img: Image): Option[Seq[Point]] =
  Using.resource(img.copy.gray.blur(2).canny(60, 180)) { edges =>
    edges.contours()
      .filter(_.area > 0.2 * img.width * img.height)
      .maxByOption(_.area)
      .map(c => c.approx(0.02 * c.perimeter))
      .map(_.points)
      .filter(_.size == 4)
  }
```

`Contour.approx` is Ramer–Douglas–Peucker simplification: no vertex more than `epsilon` pixels off the original outline, with `epsilon` conventionally a small fraction of the perimeter. Two percent turns a noisy scan of a rectangle back into four corners. The area filter drops the small contours — a logo, a stamp, the shadow of a pen — before the sort, so a `maxByOption` cannot be fooled by a large but non-rectangular blob. `Using.resource` closes the edge image whether the body returns a quad or `None`; `contours` is a query, so it borrows and leaves the image alive, which means something still has to close it.

`getPerspectiveTransform` cares about the order of the four points, and `findContours` gives them in traversal order starting from wherever the scan happened to begin. Fix that with plain arithmetic on plain data — one of the quieter benefits of `Point` being a case class:

Example 10-8. Ordering four corners clockwise from the top-left.

```
def clockwiseFromTopLeft(quad: Seq[Point]): Seq[Point] =
  val cx = quad.map(_.x).sum / quad.size
  val cy = quad.map(_.y).sum / quad.size
  // atan2 increases clockwise on screen, because y points down.
  val ring = quad.sortBy(p => math.atan2(p.y - cy, p.x - cx))
  val start = ring.indices.minBy(i => ring(i).x + ring(i).y)   // top-left
minimises x + y
  ring.drop(start) ++ ring.take(start)
```

And then the warp itself, in the same shape as the shear above:

Example 10-9. Four corners onto a flat page.

```
import org.opencv.core as cv
import org.opencv.core.MatOfPoint2f
import org.opencv.imgproc.Imgproc
import scalacv.*

/** Warps the quadrilateral `corners` (clockwise from top-left) onto a flat
`width`x`height` page.
  * `src` is borrowed: it is neither consumed nor released.
  */
def flatten(src: Image, corners: Seq[Point], width: Int, height: Int): Image =
  require(corners.size == 4, s"a perspective warp needs four corners, got
${corners.size}")
  require(width > 0 && height > 0, s"flatten needs a positive page, got ${width}
x$height")
  Managed.scope: own =>
    val from = own(MatOfPoint2f(corners.map(p => cv.Point(p.x, p.y))*))
    val to = own(MatOfPoint2f(
      cv.Point(0, 0),
      cv.Point(width - 1, 0),
      cv.Point(width - 1, height - 1),
      cv.Point(0, height - 1)
    ))
    val h = own(Imgproc.getPerspectiveTransform(from, to))
    val dst = Managed(cv.Mat())
```

```
    Imgproc.warpPerspective(
      src.mat, dst.get, h,
      cv.Size(width.toDouble, height.toDouble),
      Interpolation.Linear.cvValue,
      BorderType.Constant.cvValue,
      cv.Scalar(255, 255, 255)
    )
    Image.wrap(dst)
```

Three native objects are handed to `own` inside that block — the two point matrices and the homography `getPerspectiveTransform` returns — and the scope frees all three in reverse order, on the way out or on the way through an exception. The destination is the one allocation the scope does not own, and `Image.wrap` is where its ownership transfers to the returned `Image`.

The destination corners are `width - 1` and `height - 1`, not `width` and `height`, because they are pixel centres rather than an extent: the last column of a `width`-wide image is at index `width - 1`. Getting that wrong scales the output by a fraction of a pixel, which is invisible on a photograph and measurable the moment somebody reads coordinates back off the flattened page.

The ordering trap: detect small, draw large

The pipeline now has all its pieces, and assembling them exposes the mistake this chapter exists to prevent. Finding the page on a 12-megapixel frame is wasteful; the page outline is a few thousand pixels of edge and it is as findable at a quarter of the size. So you detect on `working` and warp the original — and the coordinates do not line up.

Example 10-10. Wrong: coordinates from one image, pixels from another.

```
Image.reading("docket.jpg") { full =>
  val working = full.copy.scale(0.25, Interpolation.Area)
  val quad = pageQuad(working)                  // corners in 1008x756 space
  working.close()
  quad.map(q => flatten(full, clockwiseFromTopLeft(q), 1200, 1600))
  // The corners are a quarter of the way in. The warp flattens the top-left
  // sixteenth of the page onto the output and calls it a document.
}
```

Every coordinate a detector, a contour finder or a template matcher returns is expressed in the pixel space of the image you handed it. Scale the image and you have scaled that space. Mapping a result back means dividing by the factor you multiplied by — and getting the factor from the images, not from the argument you passed, because `scaled` rounds the destination with `cvRound` and the achieved factor is only approximately the one you asked for.

Example 10-11. Right: derive the factor from the sizes, and map back through it.

```
/** The mapping between a downscaled working image and the full-resolution
source. */
final case class Downscale(fullWidth: Int, fullHeight: Int, workWidth: Int,
workHeight: Int):
  val fx: Double = fullWidth.toDouble / workWidth
  val fy: Double = fullHeight.toDouble / workHeight

  def point(p: Point): Point = Point(p.x * fx, p.y * fy)

  def rect(r: Rect): Rect =
    Rect(
      math.round(r.x * fx).toInt,
      math.round(r.y * fy).toInt,
      math.round(r.width * fx).toInt,
      math.round(r.height * fy).toInt
    )

def flatDocket(path: String, page: Size): Either[CvError, Option[Image]] =
  Image.reading(path) { full =>
    val working = full.copy.scale(0.25, Interpolation.Area)
    val up = Downscale(full.width, full.height, working.width, working.height)
    val quad =
      try pageQuad(working)
      finally working.close()
    quad.map: q =>
      flatten(full, clockwiseFromTopLeft(q).map(up.point), page.width.toInt,
page.height.toInt)
  }
```

Three details in that listing pay for themselves. `Downscale` is built from `full.width` and `working.width` rather than from the literal `0.25`, so it carries the factor that was achieved rather than the one that was requested — and `cvRound`'s half-to-even rule means those two are not always the same number. The `try` / `finally` closes `working` even when `pageQuad` throws, which it can: `canny` on a pathological image raises `CvError.NativeCall`, an unchecked throw that no `Either` in the signature would have caught. And `up.point` is applied to the ordered corners rather than to the raw contour, so the ordering logic works entirely in one coordinate space and only the four survivors are converted.

> **NATIVE MEMORY**
> `flatDocket` returns an `Image` that has never reached a terminal, which means the caller owns it. That is the honest signature for a factory — but it is also the one shape in the high-level API that can leak, because nothing closes for you once a value escapes `reading`. Give it to `Using.resource`, or end the chain in `write` or `bytes`, both of which release. The same warning applies to `clipped`, `sheared` and `flatten` above: every one of them hands back an `Image` nobody else will close.

The same arithmetic runs on rectangles. A `Rect` from a Haar cascade, or the `location` of a `TemplateMatch` from `Screen.findAll` in `scalacv-vision`, maps back with `Downscale.rect`, which is what lets you detect on a small frame and draw on a large one:

Example 10-12. Detect at a quarter size, annotate at full size.

```
// `template` must already be at the working scale: a matcher compares pixels,
not proportions,
// so a full-resolution template will not be found in a quarter-resolution frame.
Image.reading("desk.png") { full =>
  val working = full.copy.scale(0.25, Interpolation.Area)
  val up = Downscale(full.width, full.height, working.width, working.height)
  val hits =
    try Screen.findAll(working, template, minScore = 0.85)
    finally working.close()
  full.drawRects(hits.map(h => up.rect(h.location)),
Scalar.Green).write("annotated.png")
}
```

The rounding in `Downscale.rect` is `math.round` rather than the `cvRound` the resize itself used, and that is deliberate: this is a box to draw, not a destination to allocate, so half-to-even buys nothing and half-up is the one a reader can predict. A box that lands a pixel wide of the object is a cosmetic error; a destination size that rounds differently from OpenCV's is a native assertion.

It composes, as long as you apply the inverses in reverse order: a pipeline that downscales, then crops, then rotates produces coordinates that must be un-rotated, then offset by the crop origin, then divided by the scale factor. That is why keeping each transform as a value — a `Downscale`, a `Rect`, an angle — rather than as loose `Double`s scattered across a method is worth the ceremony.

Where this leaves the docket

The page arrives at 4032×3024 and leaves as a flat 1200×1600 rectangle, having been resampled exactly twice: once by `Area` on the way down to a working size, and once by the perspective warp onto the final page. Everything in between — the contour search, the corner ordering, the coordinate mapping — moved numbers rather than pixels, which is the shape every geometric pipeline should converge on. Resample as few times as you can, choose the filter to match the direction, and keep the transform itself as a value you can invert.

That flat page is now a mask waiting to happen. This chapter leaned on `canny` and on `Contour.approx` to find the page without ever explaining how a pixel becomes a one or a zero. The next chapter, *Thresholding and Colour Segmentation*, is where that gets its own treatment: the `Threshold` bitmask and its automatic modes, `adaptiveThreshold` under uneven lighting, and masks built from colour in HSV rather than from intensity. The chapter after it takes the masks and turns them into measurable geometry — areas, perimeters, hulls, and the bounding boxes that the `Downscale` mapping above exists to move between resolutions.

CHAPTER 11

Thresholding and Colour Segmentation

Every pixel gets one bit, and the whole rest of the pipeline depends on which one.

A classical vision pipeline spends most of its length working with images that have exactly two values in them. Contour finding wants a binary image. Morphology is defined on one. Connected components, blob measurement, the moments that give you a centroid — all of them are stated over a set of foreground pixels, and none of them knows or cares what colour that set used to be. The mask is the currency: you buy every downstream measurement with one, and the quality of the measurement is capped by the quality of the mask you paid with.

Which puts an uncomfortable amount of weight on one decision. Somewhere near the front of the pipeline a function has to look at a pixel and answer yes or no, and every ambiguity in the scene — a shadow across the object, a highlight on its shoulder, a background the same brightness as the thing you want — gets collapsed into that answer and is gone. You cannot recover it later. A contour drawn around a mask that leaked into the shadow is not a contour of the object; it is a contour of the object plus the shadow, and it will report a centroid that drifts every time a cloud passes.

So this chapter is about the several ways scalacv lets you make that decision, in rough order of how much they know about the image. A fixed threshold knows nothing and is right only when you control the lighting. Otsu's method reads the histogram and picks the level for you. Adaptive thresholding gives up on a single level entirely and computes one per neighbourhood. And colour segmentation stops thresholding brightness altogether, moves into a space where "green" is a range on one axis, and thresholds that instead.

The running example is the one from the library's own tutorial: a green ball on a workbench, filmed by a webcam, to be followed around the frame. It is small enough to fit on a page and awkward enough to make every failure mode show up.

One number for the whole image

The simplest possible mask is a comparison against a constant. `Image.threshold` is that, and it takes the value, the value written into the foreground, and the flavour of comparison:

```
def threshold(value: Double, maxValue: Double = 255, kind: Threshold =
Threshold.Binary): Image
```

It runs on the pixel values it is given, so in practice it follows `gray` — and Otsu, below, requires a single channel outright. Point it at the bench and it does exactly what you asked:

Example 11-1. A global threshold on a greyscale frame.

```
Image.reading("bench.png") { img =>
  img.gray.threshold(127).write("mask.png")
}
```

And the mask is useless. The ball is a mid-green whose luminance is close to the varnished bench it is sitting on, so at 127 the two are on the same side of the line: either the ball is swallowed by the background or half the bench comes along with it. Drop the level to 90 and the ball separates on the lit side of the frame while the shaded corner turns solid white. There is no value of `value` that works, because the scene does not have one: brightness is not the property that distinguishes a green ball from brown wood.

That is the honest failure of a global threshold, and it is worth stating plainly because the fix people reach for first — nudging the number until this photograph looks right — produces a program that works on this photograph. The next frame has a different exposure and the number is wrong again.

Where a fixed threshold does earn its place is downstream of something that has already normalised the signal for you: a frame difference, where the foreground is "changed" and the background is "unchanged"; a distance map; a confidence image out of a detector. In all of those the meaning of the value is fixed by construction and a constant is the right tool.

The Threshold model

OpenCV's `THRESH_*` constants are a bitmask, not an enumeration: a comparison mode OR-ed with at most one automatic-selection modifier. scalacv models that shape rather than flattening it, so `Threshold` is a case class carrying a `Threshold.Mode` and an `Option[Threshold.Auto]`, and `cvValue` is the OR of the two.

Table 11-1. The five comparison modes of `Threshold.Mode`, for a pixel value `v` and a threshold `t`.

Mode	Result	What it is for
`Binary`	`v > t` ⟶ `maxValue`, else 0	The mask you almost always want
`BinaryInv`	`v > t` ⟶ 0, else `maxValue`	Dark objects on a light ground — printed text
`Truncate`	`v > t` ⟶ `t`, else `v`	Clipping highlights; the result is not binary
`ToZero`	`v > t` ⟶ `v`, else 0	Keeping the bright pixels' actual values
`ToZeroInv`	`v > t` ⟶ 0, else `v`	Keeping the dark ones

`Threshold.Binary` — the value, not the mode — is the plain fixed binary case and the default for both tiers of the API. Any other fixed mode is the case class applied directly, with `auto` taking its `None` default: `Threshold(Threshold.Mode.BinaryInv)` binarises printed text with the ink white. The two automatic modes are reached through the

constructors `Threshold.otsu(mode)` and `Threshold.triangle(mode)`, each of which defaults its `mode` to `Mode.Binary`, so `Threshold.otsu()` is Otsu plus a plain binary comparison and `Threshold.otsu(Threshold.Mode.BinaryInv)` is Otsu plus an inverted one. `computesThreshold` reports whether OpenCV is going to pick the level itself, which is the same question as whether the returned `ThresholdResult` means anything.

Letting the histogram choose: Otsu

Otsu's method looks at the greyscale histogram and searches every possible threshold for the one that minimises the variance within the two resulting groups — equivalently, that maximises the variance between them. It is asking: of all the ways to cut this histogram in two, which cut produces the two tightest clusters?

The assumption buried in that question is that there are two clusters. Otsu works beautifully on a bimodal histogram — printed text on paper, a backlit object, a fluorescent bead on a dark ground — because the two modes are exactly what it is looking for. On a histogram with one broad hump it still returns a number, cheerfully, and that number is somewhere in the middle of the hump, splitting a single population in half along a line that means nothing.

The number is often the reason you called. The mid-level `Ops.threshold` is the tier that hands it back, returning a pair of the mask and a `ThresholdResult`:

```
def threshold(value: Double, maxValue: Double = 255, kind: Threshold =
Threshold.Binary)
    : (Managed[Mat], ThresholdResult)
```

With an automatic mode the `value` argument is ignored — pass `0` — and `ThresholdResult.value` carries the level OpenCV settled on.

Example 11-2. Otsu on the bench frame, reporting the level it chose.

```
val chosen: Either[CvError, Double] =
  Image.reading("bench.png") { img =>
    img.mat.cvtColor(ColorConversion.BgrToGray).use { grey =>
      val (mask, level) = grey.threshold(0, 255, Threshold.otsu())
      mask.release()   // only the number was wanted here
      level.value      // e.g. 138.0
    }
  }
```

The high-level `Image.threshold` accepts the same `kind` and drops the computed value, because the common case is "binarise this" rather than "tell me the level". When you want both, drop a tier.

> **NATIVE MEMORY**
> The pair that `Ops.threshold` returns has an owned `Managed[Mat]` in the first slot, and `Managed` frees nothing on its own. Writing `grey.threshold(0, 255, Threshold.otsu())._2.value` to grab

only the number leaks a full-size mask per call. The README puts a measured number on what that costs: 2000 unreleased 1000×1000 three-channel Mats finish at 5 865 MB of RSS, against 144 MB when the same 2000 are released — and a frame loop reaches 2000 in about a minute. Either `release()` it as above, or consume it with `use` / `pipe` — `_1` exists so a pipeline can thread the mask onward, not so the mask can be discarded silently.

Otsu, Triangle, and the shape of your histogram

Nobuyuki Otsu published his method in 1979, and it has outlived a great deal of more sophisticated work because it is exhaustive over a 256-bin search space — there is nothing to tune and nothing to get wrong.

`Threshold.Auto` carries a second option, `Triangle`, which fits a line from the histogram's peak to its far end and takes the bin furthest from that line. Where Otsu wants two modes, Triangle wants one dominant mode and a long tail — fluorescence images, sparse bright features on a big uniform background. If Otsu is giving you a level that sits inside your one big peak, Triangle is the next thing to try, and it costs one identifier to find out.

The library uses Otsu on itself: `Ops.deskew` binarises with `Threshold.otsu(Threshold.Mode.BinaryInv)` before measuring the text angle, because a page of print is the bimodal histogram Otsu was designed for and the ink needs to be the foreground.

Giving up on a single level: adaptive thresholding

Photograph a receipt on a desk under one lamp and the top of the page is nearly white while the bottom is a dim grey — darker, quite possibly, than the ink at the top. No global threshold can separate ink from paper in that image, Otsu included, because the two populations overlap when you pool the whole frame. But they do not overlap *locally*: within any small window, the ink is darker than the paper immediately around it.

That is what adaptive thresholding computes. For each pixel it takes the mean (or a Gaussian-weighted mean) over a `blockSize` × `blockSize` neighbourhood, subtracts a constant `c`, and thresholds against that. The lighting gradient cancels, because it is present in both the pixel and its neighbourhood.

Example 11-3. The document scan --- the case where nothing else works.

```
Image.reading("receipt.jpg") { img =>
  img.gray.adaptiveThreshold(blockSize = 31, c = 7).write("receipt-bw.png")
}
```

Two knobs, and they pull in different directions. `blockSize` must be odd and at least 3; it sets how big a region counts as "local", and it wants to be comfortably larger than the features you are keeping but smaller than the scale on which the lighting changes. Too small and the

interior of a thick stroke starts comparing itself to its own ink and hollows out. Too large and you are back to a global threshold. `c` is subtracted from the local average, so raising it demands that a pixel be more clearly darker than its surroundings before it is kept — it is the noise knob. On flat paper with a faint gradient, `c` near 2 is fine; on a phone photo with sensor noise, 7 to 10 cleans up a speckled result far better than blurring first.

`AdaptiveMethod` chooses the weighting: `Mean` gives every pixel in the block the same vote, `Gaussian` weights the centre more heavily. `Gaussian` is the default and is slightly gentler at edges; `Mean` is cheaper and is often indistinguishable on text.

> **WARNING**
> The two tiers spell `adaptiveThreshold`'s parameters in a different order on purpose. `Image` leads with the two you actually tune; `Ops` mirrors OpenCV's own order. The divergence cannot bite silently — the leading parameters have different types across the tiers, so a positional call meant for one does not compile against the other — but it is still a reason to name your arguments and stop thinking about it.

Table 11-2. `adaptiveThreshold` across the two tiers.

Tier	Signature
`Image`	`adaptiveThreshold(blockSize: Int = 11, c: Double = 2.0, method: AdaptiveMethod = AdaptiveMethod.Gaussian, inverse: Boolean = false): Image`
`Ops` (on `Mat`)	`adaptiveThreshold(maxValue: Double = 255, method: AdaptiveMethod = AdaptiveMethod.Gaussian, blockSize: Int = 11, c: Double = 2.0, inverse: Boolean = false): Managed[Mat]`

There is no free `Threshold.Mode` here: adaptive thresholding is binary or nothing, and `mode` picks which side. For scanned text you want `mode = Threshold.Mode.BinaryInv`, so the ink comes out white and the contours you find afterwards are the letters rather than the space between them.

Thresholding the right axis: colour in HSV

Back to the green ball. The reason no brightness threshold works is that brightness is the wrong axis. Nor does thresholding BGR help: a single real-world colour smears across all three channels as the light changes, so no fixed box in BGR captures "green" for longer than one exposure.

HSV separates the question. Hue is the colour itself, saturation is how far it is from grey, value is how bright it is. Under changing light the value moves a lot, the saturation moves some, and the hue barely moves at all — which makes "green, at any brightness" a range on a single channel. `toHsv` is the move, and `convert(ColorConversion.BgrToHsv)` is the general form it delegates to.

One thing to internalise before writing a single bound: in OpenCV's 8-bit HSV, **hue runs 0–179**, not 0–359, because the degrees are halved to fit in a byte. Saturation and value use the

full 0–255. A `Scalar` handed to `inRange` is therefore `Scalar(hue, sat, val)` on exactly that scale, and a bound of 120 means blue, not green.

`inRange` produces the mask: a `CV_8UC1` image, 255 where every channel of the source falls inside `[lo, hi]` and 0 everywhere else, regardless of how many channels the source had.

Example 11-4. The colour mask, in the two lines it takes.

```
val mask = frame.copy.toHsv.inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
mask.channels // 1 — a binary mask, whatever the source had
```

The `copy` is doing real work. Every `Image` transform consumes its receiver, so `frame.toHsv` would spend the frame and leave you holding a mask with no pixels to apply it to. Branching off a copy is the idiom whenever one path builds the mask and another supplies the content.

The other half of the pair is `applyMask`, which keeps this image where the mask is non-zero and blacks out the rest:

Example 11-5. Segment the scene: mask on one branch, pixels on the other.

```
val scene = Image.blank(320, 240, Scalar(30, 30, 30))
  .drawCircle(Point(210, 120), 28, Scalar.Green, Thickness.Filled)

val mask = scene.copy.toHsv.inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
val justGreen: Either[CvError, Array[Byte]] = scene.applyMask(mask).bytes(".png")
mask.close() // applyMask borrowed it — close it yourself
```

> **NATIVE MEMORY**
> `applyMask` *borrows* its argument. The receiver is consumed as usual and the returned image carries the result forward, but the mask you passed in is still alive when the call returns and nothing will close it for you. The same is true of `blend`'s `other`. The rule across the whole library is that the receiver is spent and a second image argument is borrowed — so every `applyMask` in your code should have a `close()` on the mask somewhere below it, and in a per-frame loop that `close()` belongs in a `finally`.

Red, and the seam in the hue wheel

Hue is circular and red sits on the seam, at both ends of the 0–179 range at once. A single `inRange` is a box, and no box captures both ends. This is the trap everyone hits, usually while wondering why the same code that found the green ball finds nothing at all on the red one.

Table 11-3. Hue centres on OpenCV's 0--179 scale. Widen the saturation and value floors to admit washed-out and shadowed pixels.

Colour	Hue	Note
Red	0–10 *and* 170–179	Wraps the seam — needs two ranges
Orange	10–20	

Yellow	25–35	
Green	35–85	The ball
Cyan	85–95	
Blue	100–130	
Magenta	140–160	

The fix is two masks, OR-ed together. `inRange` and `applyMask` are wrapped, but a raw bitwise OR is not, so this is one of the places you reach past the wrapped API into `org.opencv.core.Core`:

Example 11-6. Both ends of the wheel, and the unmanaged Mat it leaves you holding.

```
import org.opencv.core.{Core, Mat}

val hsv  = redScene().toHsv     // a frame of its own; toHsv spends it
val low  = hsv.copy.inRange(Scalar(0, 80, 80), Scalar(10, 255, 255))
val high = hsv.copy.inRange(Scalar(170, 80, 80), Scalar(179, 255, 255))

val redMask = Mat()
Core.bitwise_or(low.mat, high.mat, redMask)
val red = Image.wrap(Managed(redMask))   // adopted before anything else can
throw

low.close(); high.close(); hsv.close()
```

> **NATIVE MEMORY**
> `Core.bitwise_or` fills a `Mat` you allocated, and that `Mat` is outside the library's ownership model entirely — nothing tracks it, nothing releases it, and a `finally` you did not write is a leak that runs at frame rate. `Image.wrap(Managed(redMask))` adopts it on the spot and puts it back under the normal rules. Do the adoption in the same expression that created the raw `Mat`, not three lines later where an early return can slip between them.

The alternative you will see suggested is rotating the hue channel by 90 so red lands in the middle of the range and one box suffices. It is a real technique, but it wants modular arithmetic: the rotation has to wrap 179 back round to 0. `adjust` cannot express that — it saturates at 255 rather than wrapping — so implementing it means going to `Core` anyway. Two ranges is the shorter road, and it is what the library's own documentation recommends.

Cleaning the mask

A freshly cut mask is never clean. Sensor noise puts isolated white pixels across the background; a specular highlight on the ball desaturates a patch of it out of range and punches a hole. Both are handled by morphology, which Chapter 9 covers in full — here is the part that matters at the end of a segmentation.

Opening is an erosion followed by a dilation: it removes bright detail smaller than the kernel and leaves everything larger essentially untouched. Closing is the reverse pairing, and it fills dark holes smaller than the kernel. The order is always open then close, because you want the specks gone before you start filling things in — close first and you will have grown the specks into blobs that opening can no longer remove.

Both live on `MorphOp`, alongside `Gradient`, `TopHat` and `BlackHat`, and all five are applied through the same method. `Image.morphology(op, radius, shape)` is the whole of the high-level signature; the mid-level `Ops.morphology` adds an `iterations` count, for the case where you want the same small kernel applied repeatedly rather than one large one. Note that there is no bare `close` verb on `Image` for `MorphOp.Close` — `close()` already means "release this image" — which is why the op goes through `morphology` and not through a name of its own:

Example 11-7. Threshold, then open, then close --- the standard clean-up.

```
val clean = frame.copy.toHsv
  .inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
  .morphology(MorphOp.Open, radius = 2)   // drop the speckle
  .morphology(MorphOp.Close, radius = 3)  // fill the highlight's hole
                                          // `clean` is yours: close it or feed
it a terminal
```

The `radius` is not the kernel size: the structuring element's side is `radius * 2 + 1`, so `radius = 2` is a 5×5 kernel. Choose it from the size of the noise, not by taste. Look at the raw mask, find the largest speck you want gone, and pick the smallest radius that exceeds it — then stop, because every extra pixel of radius also rounds off the object you are keeping. For a 640×480 webcam frame, `radius = 1` or `2` covers ordinary sensor noise. If you need 5 to clean the mask, the bounds are wrong and morphology is papering over it. There is no identity radius: a `require` rejects anything below 1, unlike `blur(0)`, which is deliberately a no-op — a zero-radius structuring element is not a thing OpenCV can build.

`MorphShape` defaults to `Rect`, which is the right default for cleaning noise. `Ellipse` is worth switching to when the object is round and you can see the corners of a square kernel in the result.

`medianBlur` is the other tool for this job and is sometimes better: on a binary mask it erases isolated pixels without eroding the edges at all. `Image.medianBlur` takes a radius, so `medianBlur(1)` is a 3×3 window; the mid-level op on `Mat` takes the odd kernel size directly and rejects anything even or below 3.

The tracker, end to end

Everything above assembles into the tracker. Convert, threshold the colour, clean, take the largest contour, read its centroid.

Example 11-8. One frame of the colour tracker.

```
def locate(frame: Image): Option[Point] =
  val mask = frame.copy.toHsv
    .inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
    .morphology(MorphOp.Open, radius = 2)
    .morphology(MorphOp.Close, radius = 3)
  try
    mask.contours()
      .filter(_.area > 200)      // a speck is never the target
      .maxByOption(_.area)
      .flatMap(_.centroid)
  finally mask.close()
```

`contours()` borrows the mask and returns `Seq[Contour]` — plain immutable Scala data, copied out of native memory before the call returns, so the contours stay valid after the mask is released. `Contour.area` is the shoelace area (`Imgproc.contourArea`), which is measured between the *centres* of the boundary pixels and so runs a little under the pixel count: a filled 100×50 rectangle reports 99 × 49 = 4851, not 5000. That matters only if you are comparing it against a count you derived some other way; as a relative size for picking the largest blob it is exactly right. `centroid` is derived from image moments and is an `Option` because a degenerate zero-area blob has no centre of mass. The `filter` before `maxByOption` is what stops a single surviving speck from being promoted to "the target" on a frame where the ball is out of view; without it, `maxByOption` returns `Some` of whatever noise is left and your tracker reports a position with total confidence.

Wrap that in a camera loop and it is a live tracker:

Example 11-9. The same logic over a video, with nothing accumulating.

```
Camera.usingFile("bench.mp4") { cam =>
  cam.foreach() { frame =>
    locate(frame) match
      case Some(p) => println(s"target at (${p.x.toInt}, ${p.y.toInt})")
      case None    => println("target lost")
  }
}
```

> **NATIVE MEMORY**
>
> `Camera.foreach` owns each frame and closes it when your block returns, which is why `locate` takes a `copy` rather than consuming the frame it was handed — consuming it would leave `foreach` closing an already-spent handle, which is harmless (release is idempotent) but would also mean you could not draw on the frame afterwards. Inside `locate` , exactly one mask is allocated and exactly one is freed, in a `finally` , so a thousand-frame video allocates a thousand masks and holds one. That is the shape every per-frame pipeline should have: one owner, one `finally` , nothing crossing the loop boundary.

To annotate rather than print, `drawCircle` draws into the frame's own Mat — no copy is made — but it is still a transform in the ownership sense: it takes the handle, mutates the pixels,

and hands back a *new* `Image` around the same buffer. The result is what owns the frame afterwards, so it has to be kept and closed:

Example 11-10. Marking the frame, and keeping the handle the drawing hands back.

```
Camera.usingFile("bench.mp4") { cam =>
  cam.foreach() { frame =>
    val marked = locate(frame) match
      case Some(p) => frame.drawCircle(p, 8, Scalar.Red, Thickness.Stroke(3))
      case None    => frame
    marked.close()
  }
}
```

> **NATIVE MEMORY**
> Discarding a drawing verb's result — `frame.drawCircle(...)` as a bare statement — leaks the frame, and it leaks it in the one shape that looks safest. `drawCircle` goes through the internal `paint`, which *takes* the handle out of `frame` and rewraps the same Mat in the returned `Image`. The `frame` you still hold is now spent, so `Camera.foreach`'s own `close()` on it releases nothing, and the only handle that could have freed the buffer was the value you threw away. Bind the result. The `None` branch above rebinds the untouched frame instead, so exactly one name owns the buffer on both paths and one `close()` covers both.

Choosing the bounds

Which leaves the one number nobody can give you: the bounds. `Scalar(35, 80, 80)` to `Scalar(85, 255, 255)` is a starting point for green, not an answer, and the gap between a tracker that works in the room you wrote it in and one that works in the next room is decided here rather than anywhere above.

Start by measuring rather than guessing. Crop a small patch of the object out of a representative frame, convert it to HSV, and take its mean. That last step is one of the few places in a segmentation where the wrapped API runs out: `Image` exposes size, channel count and the borrowed Mat, but no pixel accessor, and `Scalar.from` — the conversion that would hand OpenCV's four doubles back as a `Scalar` — is `private[scalacv]`. For one number, reach through `Image.mat` to `Core.mean` and read its `val` array directly:

Example 11-11. Sampling a patch to find where the object actually sits in HSV.

```
import org.opencv.core.Core

val sample: Either[CvError, Array[Double]] =
  Image.reading("ball.png") { img =>
    val patch = img.crop(Rect(204, 114, 12, 12)).toHsv
    try Core.mean(patch.mat).`val`   // (hue, sat, val, 0)
    finally patch.close()
  }
```

> **NATIVE MEMORY**
> `crop` and `toHsv` each consume their receiver, so after that first line the only live `Image` in the block is `patch` — `img` is already spent, and `reading`'s own `close()` is a no-op on it. Nothing frees `patch` for you, because it never passed through a terminal. The `finally` is not decoration.

Then widen the box, asymmetrically. Hue is the channel you trust, so keep it tight — ±15 either side of the measured value is generous for a distinctly coloured object, and going wider is how green starts catching yellow tape and cyan mugs. Saturation and value are the channels the lighting moves, so open them right up: a floor around 80 with a ceiling of 255 admits the object in shade and under a highlight, while the floor still rejects the near-grey pixels whose hue is numerically green but visually meaningless. It is that saturation floor, not the hue range, that keeps a grey bench out of a green mask — a nearly-unsaturated pixel has a hue, but it is noise.

Finally, measure on more than one photograph. Bounds tuned on a single image are fitted to that image's white balance, and every camera's automatic white balance moves hue by a few units between a sunlit frame and a tungsten one. Sample the object in the worst light you expect and the best, take the union, and widen a little more. If the union is so wide that it starts admitting the background, the answer is not a cleverer threshold — it is a different coloured ball, or a different cue altogether.

What the mask is for

A mask on its own is a picture of a decision. What makes it useful is what comes next: turning those white regions into shapes with area, perimeter, bounding boxes and centroids that you can count, sort and track. `contours()` appeared once in this chapter, called with its defaults and with no explanation of the `ContourRetrieval` and `ContourApproximation` knobs it takes, of what a hierarchy of nested contours means, or of why `Contour` is plain Scala data rather than a native handle. Chapter 12, *Contours and Shape Analysis*, is where that gets paid off.

CHAPTER 12

Contours and Shape Analysis

From a binary mask to measured objects you can count, sort and reject.

A threshold answers one question, once per pixel: is this pixel foreground? That is a useful answer and it is never the answer anyone asked for. The requirement that reached you was phrased about things — how many parts are on the tray, is any of them short of tolerance, which one is lying skew, is the label centred on the box. Between the per-pixel answer and the per-object question sits a step that most of classical vision turns on, and it has a name: tracing the boundary of every connected region and keeping the boundary as data.

That boundary is a contour. Walk the edge of one white blob, one pixel at a time, all the way round until you are back where you started, and write down where you went. What you get is an ordered, closed ring of coordinates, and it changes what you are allowed to ask. A blob of pixels has no area; a closed polygon does. A blob has no centre, no corner count, no orientation and no best-fit box; a polygon has all four, and each of them is arithmetic rather than image processing. Once the mask becomes a `Seq[Contour]`, the rest of the pipeline is Scala collections work.

The step is also where OpenCV's Java bindings are at their most treacherous. `Imgproc.findContours` does not return anything; it fills a `java.util.List[MatOfPoint]` you pass in, plus a hierarchy `Mat`. Every element of that list is a live native handle the caller is expected to free individually, and nothing about the signature says so. The values behave impeccably — you read their points, you measure them, you print them — while the process's resident set climbs by a frame's worth of boundary chains every iteration of the loop. The library's own source calls it the single most reliable leak in the OpenCV Java API, and that is the right description: it is the one where doing the obvious thing leaks every time.

So `findContours` in this library copies the points across the native boundary, frees every `MatOfPoint` and the hierarchy `Mat` before it returns, and hands back ordinary immutable Scala data. The consequence is that this chapter's API is small: one call at each of the two levels, two enums, and one case class with eight members. Everything else is arithmetic on plain values, and none of it can crash your process.

The running example is a top-down camera over machined parts on a dark conveyor, lit so the metal reads bright against the belt. By the end of the chapter it produces a `Seq[Part]` — one entry per object, with a position, a size and a shape verdict — and an annotated frame to look at when the numbers disagree with your eyes.

Getting a binary image to trace

`findContours` accepts a single-channel 8-bit image (`CV_8UC1`, or `CV_32SC1`) and nothing else. Non-zero is foreground, zero is background. Hand it a three-channel frame and OpenCV raises, which this library reports as a `CvError.NativeCall` naming the operation `findContours` rather than as a bare `CvException`; hand it a `Mat` with no data at all and you get an `IllegalArgumentException` before any native call happens, because an empty input is a bug in your code, not a data-dependent failure.

Two routes normally get you there, and they answer different questions. Thresholding gives you the outline of a filled region: one contour per solid blob, with a meaningful area and centroid. Canny gives you the outline of edges, so a thick stroke yields the outside of the stroke and the inside of it — two contours around what your eye calls one line. For counting and measuring solid objects, threshold; for tracing thin structure, Canny.

Example 12-1. The mask the rest of the chapter measures. `copy` keeps the source frame alive for annotation later.

```
import scalacv.*

val mask: Image =
  frame.copy                                  // branch: `frame` stays usable
       .gray                                  // CV_8UC1, which findContours
requires
       .threshold(200)                        // bright metal on a dark belt
       .morphology(MorphOp.Open, radius = 2) // erode then dilate: specks vanish
```

The opening is not decoration. A single stray bright pixel is a legitimate connected region, and `findContours` will faithfully hand you a contour for it. `MorphOp.Open` erodes and then dilates, so anything thinner than the structuring element disappears and everything else comes back to roughly its original size — two pixels of radius removes sensor speckle and JPEG ringing without touching a part. Deleting noise here, in one pass over the mask, is cheaper than filtering a thousand one-point contours afterwards.

Finding the contours

At the high level, `contours` is a method on `Image`, and it is a query rather than a transform: it borrows the image, returns plain data, and leaves the image alive for the next call. That is the opposite of `gray`, `threshold` and every `draw*`, which consume the receiver.

At the mid level, `findContours` is an extension method on a raw `Mat` with the same two parameters and the same defaults. `Image.contours` delegates straight to it.

Example 12-2. Two levels, one behaviour. The defaults are `External` and `Simple` in both.

```
// high level: a query on an Image, which stays alive afterwards
val cs: Seq[Contour] = mask.contours()
```

```
// mid level: an extension on the borrowed Mat, for when you are already there
val cs2: Seq[Contour] = mask.mat.findContours(
  retrieval = ContourRetrieval.External,
  approximation = ContourApproximation.Simple
)
```

Both return `Seq[Contour]` in OpenCV's own order, which is a scan-order artefact rather than a promise — sort by something you care about before you rely on the sequence. An image with no foreground at all yields an empty `Seq`, not an error.

> **NATIVE MEMORY**
> The `Seq[Contour]` outlives the `Mat` it came from. `findContours` releases every `MatOfPoint` it allocated in a `finally` block — so a failure partway through frees what was already allocated, and there is no handle left in the result to forget. Close the mask, close the image, return the contours, cache them, hand them to another thread, measure them an hour later.

Retrieval: which outlines you get

Picture a bright square with a square hole punched out of its middle, and inside that hole a smaller bright square sitting free. Three boundaries exist in the mask: the outside of the big square, the edge of the hole, and the outside of the small square. `retrieval` decides which of them come back to you and how they are related.

Table 12-1. The four retrieval modes. Nesting is computed for `CComp` and `Tree`, but not exposed.

Mode	Returns	Nesting
`External`	the outermost outline of each region only — the big square, and nothing inside it (**default**)	ignored
`List`	every boundary, flat: outer, hole, and inner square, in one undifferentiated `Seq`	flattened
`CComp`	every boundary, organised two-level: outer boundaries and their holes	two-level
`Tree`	every boundary, in a full parent/child nesting tree	full tree

`External` is the default because it is what a caller who never mentions the hierarchy almost always means: on a conveyor, one part is one contour. A hole in a washer is not a second part, and a reflection inside the part is not a third — switching to `List` on that tray silently doubles the count.

> **NOTE**
> The hierarchy arrives from OpenCV as an `Nx1 CV_32SC4 Mat` of indices — an untyped, unmanaged structure of exactly the shape this library exists to remove — so handing it back would undo the point of copying the contours out. Until a typed nesting API exists, choose between `External` and `List`, and use `Tree` only when the count itself is what you want.

> **WARNING**
> `area` measures the outline it is given, and an outline knows nothing about what is inside it. The outer contour of a washer reports the area of a solid disc; the hole is not subtracted. If your parts have holes and the area matters, retrieve with `List` and subtract the inner areas yourself, or measure something the hole does not affect, such as the bounding box. `List` gives you outer boundaries and holes in one flat `Seq` with no marker saying which is which, so pairing them up means testing containment yourself — workable for one hole per part, guesswork for a lace pattern.

Approximation: how the outline is stored

The second parameter decides how much of the traced boundary is kept. A 40×30 filled rectangle has $2 \times (40 + 30) - 4 = 136$ pixels on its boundary, counting each corner once. `Simple`, the default, collapses straight runs to their endpoints and stores that rectangle as four points. `None` keeps all 136.

Table 12-2. The four approximation modes.

Mode	Effect	Reach for it when
`Simple`	collapses straight runs to their endpoints (**default**)	corner counting, polygon work, drawing
`None`	keeps every pixel on the boundary	curvature, arc-length precision, sub-shape analysis
`Tc89L1`	Teh–Chin chain approximation, L1 metric	you want a smoother polygon than `Simple` gives
`Tc89Kcos`	Teh–Chin chain approximation, k-cosine metric	as above, with a different corner metric

The compression is not only about memory, though on a frame full of blobs the difference between four points and several hundred per object is real. It changes what a vertex count means: `Simple` has already done most of a shape classifier's work on axis-aligned figures — which is exactly why `approx` exists for the ones it has not.

What a Contour carries

`Contour` is a `final case class` with a single field — `points: Seq[Point]` — and seven derived members. The five measurements are `lazy val`s, computed on first read and then cached, so `filter(_.area > 400).sortBy(_.area)` measures each contour once rather than twice, and a contour you discard on a cheaper test never pays for a native call at all. Only `isEmpty`, which reads the points and nothing else, and `approx`, which takes an argument, are plain `def`s.

Table 12-3. The whole of `Contour`. Nothing here holds a native handle.

Member	Type	What it gives you

`points`	`Seq[Point]`	the outline vertices, in order; whole numbers widened to `Double`
`isEmpty`	`Boolean`	true only for a hand-built empty contour; `findContours` never emits one
`boundingRect`	`Rect`	the upright bounding box, from `Imgproc.boundingRect`
`area`	`Double`	enclosed area by the shoelace formula; always non-negative
`perimeter`	`Double`	closed arc length, from `Imgproc.arcLength` with `closed = true`
`centroid`	`Option[Point]`	centre of mass from image moments; `None` where undefined
`convexHull`	`Contour`	the tightest convex outline enclosing these points
`approx(epsilon, closed)`	`Contour`	Ramer–Douglas–Peucker simplification

`points` comes back as whole numbers even though `Point` carries `Double` fields: `findContours` produces `CV_32SC2`, so every coordinate is an integer that happens to have been widened. The `Double` is there because the same `Point` type serves feature and pose work, where sub-pixel positions matter.

`centroid` is an `Option` for a reason worth stating, because it is the member people most often `get` without thinking. It divides the first-order moments by `m00`, the zeroth moment, which is the enclosed area. A contour of three collinear points, or of a single pixel, encloses nothing; `m00` is zero and the centroid is genuinely undefined rather than merely awkward. The `Option` is that fact, not defensive padding.

`convexHull` returns a new `Contour` whose vertices are **exactly** points of the original — `Imgproc.convexHull` computes indices, which the library maps back through `points` rather than re-deriving coordinates. For an L-shaped outline the hull drops the one reflex vertex that pokes inward and fills the notch, so the hull's area is always at least the contour's own; the ratio of the two is a usable measure of how dented a shape is.

Why a 40 × 30 rectangle has an area of 1131

Fill a 40 × 30 block of pixels, trace it, and `area` reports 1131, not 1200. This looks like an off-by-one and is not. `Imgproc.contourArea` applies the shoelace formula to the polygon whose vertices are the **centres** of the boundary pixels, and the centres of a 40-wide block span 39 pixels, not 40. So the polygon is 39 × 29 = 1131, and the perimeter is $2 \times (39 + 29) = 136$ — the same 136 the `None` chain reports as a point count, and that is an identity rather than a coincidence: the chain holds $2 \times (w + h) - 4$ pixels, the centre polygon measures $2 \times ((w - 1) + (h - 1))$, and those are the same expression rearranged. Area is the measurement that shrinks under the centres convention; perimeter comes out unchanged.

`boundingRect` uses the other convention: OpenCV's box is inclusive of the extreme pixels, so the same shape reports `width == 40`. That is why the library delegates to `Imgproc.boundingRect` rather than taking a minimum and maximum over the points in Scala — re-deriving the convention by hand is how two definitions drift apart over a release or two.

The practical rule: `area` is a measurement, `boundingRect.area` is an envelope, and they will never be equal even for a perfect rectangle. Compare like with like, and never use one as a sanity check on the other.

Measuring: four numbers and four ratios

The four direct measurements — area, perimeter, bounding box and centroid — are the raw material, and they are where `Contour` stops. What survives contact with a real production line is usually a ratio of two of them, because a ratio is dimensionless: it does not change when the camera moves 50 mm closer to the belt, and it does not need recalibrating when you swap a lens. The library wraps no such ratio, and it does not need to — each is two or three lines over members it already has, and an `extension` block puts them on `Contour` as though they had shipped with it.

Example 12-3. The four ratios. None of these is library API; this block is the whole of their definition.

```
extension (c: Contour)

  /** Bounding-box width over height. 1.0 for a square or a circle,
    * large for a bar lying along x, small for the same bar upright. */
  def aspectRatio: Double =
    val box = c.boundingRect
    if box.height == 0 then 0.0 else box.width.toDouble / box.height

  /** How much of its own bounding box the shape fills. A rectangle
    * approaches 1.0, a circle about 0.785, a diagonal bar under 0.5. */
  def extent: Double =
    val box = c.boundingRect.area
    if box == 0L then 0.0 else c.area / box.toDouble

  /** Area over convex-hull area. 1.0 for a convex shape; low means
    * deep concavities — a gear's teeth, a hand's fingers, two parts
    * that have been welded into one blob by the threshold. */
  def solidity: Double =
    val hull = c.convexHull.area
    if hull == 0.0 then 0.0 else c.area / hull

  /** 4·pi·area / perimeter². 1.0 for a perfect circle, about 0.785
    * for a square, and falling fast as the outline gets ragged. */
  def circularity: Double =
    if c.perimeter == 0.0 then 0.0 else 4 * math.Pi * c.area / (c.perimeter *
c.perimeter)
```

Each earns its keep somewhere different. `aspectRatio` separates a bar from a disc with no shape fitting at all. `extent` catches shapes that lie diagonally, which the upright box cannot express — the cheapest possible "is this thing skew?" test. `solidity` is the standard detector for two objects touching: two discs that overlap in the mask make one contour whose hull spans both, and the ratio drops through the floor while area and bounding box both still look plausible. `circularity` answers "round or not" without counting vertices, and degrades gracefully as an outline gets ragged instead of failing outright.

> **TIP**
> `centroid` and `boundingRect` disagree for any shape that is not symmetric, and the disagreement is information. The centre of the bounding box is where the object **fits**; the centroid is where its mass **is**. For a pick-and-place target you want the centroid. For a crop rectangle you want the box.

Classifying by vertex count

Ask how many sides a shape has and the naive move is to count `points`. On a `Simple` contour of a clean axis-aligned rectangle that even works, which is unfortunate, because it means the technique survives the first test and fails on the first rotated part. A rotated rectangle's edges are staircases of pixels, and `Simple` cannot collapse a staircase: it reports dozens of vertices for a four-sided shape.

`approx` is the fix. It runs Ramer–Douglas–Peucker simplification: drop as many vertices as possible, subject to no point of the original outline lying more than `epsilon` pixels away from the simplified one. Set `epsilon` well and a staircase collapses back to the four corners it was always meant to be.

The mistake to avoid is a fixed `epsilon`:

```
c.approx(3.0).points.size   // wrong: 3 px is loose on a 40 px part, tight on a
400 px one
```

Three pixels of tolerance erases real corners on a small part and leaves every staircase step on a large one. Since `epsilon` is a distance that must scale with the shape, take it as a fraction of the perimeter:

Example 12-4. Vertex counting that works at any scale. 0.02 is the usual starting point.

```
def sides(c: Contour): Int = c.approx(0.02 * c.perimeter).points.size

def classify(c: Contour): String = sides(c) match
  case 3 => "triangle"
  case 4 => if c.aspectRatio > 0.95 && c.aspectRatio < 1.05 then "square" else
"rectangle"
  case 5 => "pentagon"
  case 6 => "hexagon"
```

```
  case n if n > 6 && c.circularity > 0.85 => "circle"
  case _ => "unknown"
```

Two per cent of the perimeter is the rule of thumb, and it is only that. Raise it towards 0.04 when the mask is ragged and you keep getting spurious corners; drop it towards 0.01 when genuine shallow corners are being smoothed away. `approx` rejects a negative `epsilon` with an `IllegalArgumentException` rather than passing it to OpenCV, and `closed` defaults to `true`, which is right for anything `findContours` produced — those are always closed curves, and asking for the open length of one silently drops the final edge.

The circle case is worth its own note. A circle has no vertices to count; what it has is a vertex count that stays high as you shrink `epsilon`, because there is no straight run anywhere to collapse. Pairing that with high `circularity` is more robust than either test alone.

Filtering: the highest-value line in the pipeline

Here is the whole detection, written the way it is written the first time:

```
val parts = mask.contours()   // and every dust mote on the belt is now a "part"
```

A three-pixel speck is a connected region. So is the corner of a reflection, so is a scratch on the belt, so is one hot pixel. The count is wrong, the statistics are wrong, and the failure is silent because a `Seq` of the wrong length looks exactly like a `Seq` of the right one.

One clause fixes it:

Example 12-5. Reject by area first. Everything downstream gets cheaper and more honest.

```
val parts = mask.contours().filter(_.area >= 400)
```

Pick the threshold from the smallest object you must not miss, then halve it. A part 30 × 30 pixels across at the working distance encloses roughly 900 px², so 400 leaves margin for a partly occluded one while still deleting anything the optics could not have resolved. Area is the right axis for the test because it falls as the square of the linear size: a speck one tenth the diameter of a part is one hundredth of its area, so the two populations sit either side of a wide, uncrowded gap.

Because the result is a plain `Seq`, the rest is the standard library: `sortBy(_.boundingRect.x)` orders objects left to right along the belt, `maxBy(_.area)` grabs the largest, `flatMap(_.centroid)` collects the positions that exist, `partition` splits pass from fail. None of it touches native memory, so none of it can be got wrong in a way that costs you a process.

Fitting a shape that is not upright

`boundingRect` is axis-aligned, which is a poor description of a part lying at 30°: the box is far larger than the object and its width and height say more about the angle than about the part. When orientation matters, fit a shape instead of enclosing one. OpenCV has three fits for this, and scalacv does not wrap them — so this is a good place to use the escape hatch the library keeps open on purpose.

Table 12-4. The three fits, straight from `Imgproc`. All three take a `MatOfPoint2f`.

Call	Returns	Use it when
`minAreaRect`	`RotatedRect`	the object is rectangular and rotated: gives centre, size and angle
`minEnclosingCircle`	centre and radius, via out-parameters	you need a bounding radius, or a rotation-invariant size
`fitEllipse`	`RotatedRect`	the object is round-ish and you want its axes and tilt (needs at least 5 points)

`RotatedRect` is a plain Java object with `center`, `size` and `angle` fields and no native handle, so once you have one you can keep it. Getting there means building a `MatOfPoint2f`, and that **is** native memory.

Example 12-6. Fitting a rotated box. `Managed` owns the point buffer; `Cv.orThrow` names the call if OpenCV raises.

```
import org.opencv.core.MatOfPoint2f
import org.opencv.imgproc.Imgproc

def minAreaBox(c: Contour): org.opencv.core.RotatedRect =
  val pts = c.points.map(p => org.opencv.core.Point(p.x, p.y))
  Managed(MatOfPoint2f(pts*)).use: m =>
    Cv.orThrow("minAreaRect")(Imgproc.minAreaRect(m))

// the tilt of a part, in degrees, from the fitted box
def tilt(c: Contour): Double = minAreaBox(c).angle
```

> **NATIVE MEMORY**
> `MatOfPoint2f` is a `Mat`. Constructing one allocates off-heap, and OpenCV's fitting functions take ownership of nothing — the buffer is still yours when the call returns. `Managed(...).use` frees it on the way out, including when the native call throws, which is exactly the path a hand-written `release()` at the end of the method misses. It is the pattern `Contour`'s own members use internally: each materialises the points as a native Mat — a `MatOfPoint` for `boundingRect`, `area` and `centroid`, a `MatOfPoint2f` for `perimeter` and `approx`, because `arcLength` and `approxPolyDP` accept nothing else — measures it, and frees it before returning plain Scala data.

The library reaches for `minAreaRect` itself in `deskew`, with one difference worth copying. It does not fit the box to a contour: it binarises the page with an inverted Otsu threshold, collects every ink pixel with `Core.findNonZero`, and fits the rectangle to that cloud, because a page of text is hundreds of separate blobs with no single outline to trace. The fitted `angle` is the tilt, and OpenCV's convention for it is not intuitive — the angle lives in a quarter-turn range, so a rectangle at 5° and the same rectangle at 95° can report the same number. `deskew` folds it into a tilt of at most 45° either way before rotating, and treats anything past its `maxAngle` (45° by default) as a misread. Orientation logic built on `minAreaRect` needs both.

Drawing the result back

Numbers agree with each other far more readily than they agree with reality. The fastest way to find out whether your contours are the ones you think they are is to paint them onto the original frame and look.

`drawContours` exists at both levels. On a `Mat` it mutates the receiver and returns `Unit`; on an `Image` it is a transform, so it consumes the image and hands back a new one. `Thickness.Default` is a one-pixel outline, `Thickness.Stroke(2)` is wider, and `Thickness.Filled` fills them — which is the usual way to turn a set of contours back into a mask after you have filtered it.

Example 12-7. Annotating the original frame. `drawRects` and `drawText` are transforms too, so this chains.

```
val kept = mask.contours().filter(_.area >= 400)

// each call consumes the image and returns the next one; `write` releases
frame.drawContours(kept, Scalar.Green, Thickness.Stroke(2))
     .drawRects(kept.map(_.boundingRect), Scalar.Red)
     .drawText(s"${kept.size} parts", Point(10, 30), Scalar.White, scale = 0.8)
     .write("tray-annotated.png")
```

Note the colours. OpenCV `Mat`s are BGR, so `Scalar.Red` is `Scalar(0, 0, 255)`. The library's constants are ordered correctly, but the moment you write a literal by hand you are in BGR, and the mistake looks like a rendering bug rather than a channel-order one.

> **NATIVE MEMORY**
> `drawContours` is not leak-free: the generated Java binding strands one `Mat` per polygon on the way in, upstream of anything the library can reach. It is bounded per call and unbounded across a video loop, so pass the whole `Seq` to one call rather than looping and calling it once per contour — fifty contours in one call, not fifty calls. Chapter 14, *Drawing and Annotation*, has the full accounting.

The whole pass

Everything above, as one function. It takes a frame, returns measured objects, and leaves the caller holding nothing that needs freeing.

Example 12-8. Mask to measured parts. The `Seq[Part]` outlives every native object in the function.

```
import scalacv.*

final case class Part(
    id: Int,
    centre: Point,
    box: Rect,
    areaPx: Double,
    tiltDegrees: Double,
    shape: String
)

def inspect(frame: Image, minArea: Double = 400.0): Seq[Part] =
  val mask = frame.copy.gray.threshold(200).morphology(MorphOp.Open, radius = 2)
  try
    mask.contours()                          // External + Simple
        .filter(_.area >= minArea)           // drop specks before measuring
anything
        .sortBy(_.boundingRect.x)            // left to right along the belt
        .zipWithIndex
        .flatMap { (c, i) =>
          c.centroid.map { centre =>         // no centroid, no part: it enclosed
nothing
            Part(
              id = i,
              centre = centre,
              box = c.boundingRect,
              areaPx = c.area,
              tiltDegrees = minAreaBox(c).angle,
              shape = classify(c)
            )
          }
        }
  finally mask.close()                       // the query borrowed it; closing is
yours
```

The end of that function is where the ownership rules land. `contours` borrows the mask and leaves it alive — unlike `gray` and `threshold` above it, which consumed their receivers and are already gone — so closing is the caller's job, and `close()` is idempotent, which makes the `finally` safe even on paths that never reached the trace. `frame` is untouched, because `copy` branched at the top so the caller can annotate the original afterwards.

The reverse of that reasoning is the failure mode. Drop the `copy` and `gray` consumes `frame` itself; the annotation call then throws `IllegalStateException` from `Managed` — in Scala, before anything reaches JNI, with a message that tells you the image was spent by a transform and suggests `.copy`. Run with `-Dscalacv.trackOwnership=true` and it will also point at the line that consumed it.

Wrap the whole thing in `Image.reading` and the frame is closed for you on every path, including the exceptional ones:

```
Image.reading("tray-0417.png")(frame => inspect(frame))
```

Where this goes next

Contours find things that are solid. They are the right tool when your subject is a region — a part, a blob, a cell, a coloured patch — and the wrong one when it is a line that was never filled in: a lane marking, a wire, the edge of a page against a desk. A line has no interior to trace, and thresholding one produces a ribbon whose contour describes the ribbon rather than the line. The Hough transform is the classical answer to that, and it is what the next chapter takes up: `houghLines` and its `PolarLine` results, `houghLinesP` returning drawable `Segment` values, and `houghLinesWithAccumulator` for when the vote count itself is the evidence you need.

The drawing calls used here to check the work — `drawContours`, `drawRects`, `drawText`, the `Thickness` type and the BGR ordering that catches everyone once — get their full treatment in Chapter 14, along with text measurement and why the whole of it lives in `core` rather than in a GUI module.

CHAPTER 13

Finding Lines and Circles

Recovering straight structure from an edge map that is full of holes.

A photograph of a page on a desk looks like a straightforward problem right up to the moment you write code for it. What you want is the four edges of the sheet, or the baselines of the text, so that you can work out how far the page is rotated. What you have, after `canny`, is a `CV_8UC1` bitmap in which some pixels are set and the rest are not. Nothing in that bitmap says which set pixels belong together. The edge of the page and the rim of the shadow under a coffee cup are the same white, and both are only pixels.

Fitting a straight line to a set of points is a solved problem — least squares, four lines of Scala. The difficulty is that fitting presupposes the grouping, and the grouping is the actual question. It gets worse: real edges arrive in pieces. A page edge crosses a shadow and drops out for thirty pixels; a ruled line is interrupted by every descender crossing it; an anti-aliased boundary flickers on and off as the gradient wanders either side of the hysteresis threshold. Any method that needs a contiguous run of edge pixels reports the pieces rather than the line, and then you are writing the code that stitches pieces back together — the same problem again, with more state.

The Hough transform turns the question round. Instead of asking a candidate line which pixels lie on it, it asks each edge pixel which lines could pass through it, and lets that pixel vote for all of them. One pixel is consistent with an infinite family of lines, so one pixel tells you nothing. But a line that is really in the image collects a vote from every pixel along it — and it makes no difference whether those pixels are contiguous, because votes are added, not linked. Thirty missing pixels cost thirty votes out of eight hundred. Where the votes pile up, there is a line.

That is the whole idea, and the rest of this chapter is its cost. The vote count is the parameter you tune, and a vote count is neither a distance nor a fraction: a line spanning a 4000-pixel photograph collects ten times the votes of the same line in a 400-pixel thumbnail, so a threshold that works on one is silently wrong on the other. And every vote comes from the edge map underneath, so a Hough call is never tuned on its own — you are tuning `canny` and the transform together.

Voting instead of fitting

Picture a second image, one you never display. Its axes are not x and y but the two numbers that name a line, and each cell is a counter. For every set pixel in the edge map, the transform walks the family of lines through that pixel and adds one to each corresponding counter. A

cell holding 640 then means: 640 edge pixels are consistent with this particular line. That second image is the accumulator, and the transform's output is its peaks.

The obvious way to name a line, $y = mx + c$, is unusable here: a vertical line has infinite slope, so half of the accumulator would be unreachable and the sampling would be wildly uneven. The parameterisation everyone uses instead — introduced for exactly this purpose by Duda and Hart in 1972 — drops a perpendicular from the origin to the line and names the line by that perpendicular: `rho`, its length in pixels, and `theta`, its angle in radians. Every line in the plane gets one finite, bounded pair, verticals included.

The single most common misreading of a Hough result follows directly. `theta` is the angle of the **normal**, not of the line. A horizontal line has a vertical normal, so it comes back at `theta = Pi/2`; a vertical line comes back at `theta = 0`. `PolarLine`'s own scaladoc says so, and it is worth reading the result out in degrees the first few times.

The accumulator's cell size is the other pair of knobs, and both are parameters of every call: `rho`, the distance resolution in pixels (default `1.0`), and `theta`, the angle resolution in radians (default `math.Pi / 180`, one degree). Finer cells separate nearly coincident lines, at the price of splitting one real line's votes across neighbouring cells, which lowers its peak and can push it under your threshold. That trade is why a Hough call finding nothing sometimes wants a **coarser** accumulator rather than a lower threshold.

What the transforms will accept

All three transforms are extension methods on `org.opencv.core.Mat`, brought in by `import scalacv.*`, and all three demand a non-empty, 8-bit, single-channel image. Handing one a colour photograph is the first mistake everybody makes, because at the call site it reads perfectly well.

Example 13-1. The version that does not run.

```
Image.reading("page.jpg") { img =>
  img.mat.houghLinesP(threshold = 80) // IllegalArgumentException
}
```

The check happens in Scala, before anything crosses JNI, and names the offending Mat: `houghLinesP needs a
but got
3024x4032 of type CV`

Without that check the same call fails on the far side of JNI, with a message that names neither the call nor the reason.

The precondition catches the type, and only the type. The second mistake is quieter:

> **WARNING**
>
> `img.gray.mat.houghLinesP(threshold = 80)` compiles, passes the precondition, and returns rubbish. A greyscale photograph is `CV_8UC1`, but the transform treats every non-zero pixel as an edge pixel entitled to vote — and in a photograph that is nearly every pixel. You get a long list

> of plausible-looking segments that correspond to nothing. Run `canny` first, and when a result makes no sense, write the edge map out and look at it before touching any Hough parameter.

Example 13-2. Edges first, then the transform.

```
import scalacv.*

OpenCv.load()

val segments = Image.reading("page.jpg") { img =>
  val edges = img.gray.canny(threshold1 = 80, threshold2 = 160) // Image, CV_8UC1
  val found = edges.mat.houghLinesP(threshold = 80, minLineLength = 120,
maxLineGap = 10)
  edges.close()
  found
}
```

`Image.reading` closes `img` for you, on success, on failure and on exception. It does not close `edges`, because `edges` is a different `Image` that `img.gray` produced by consuming `img` — so that one `close()` is yours, and the block returns `Either[CvError, Seq[Segment]]`.

> **NATIVE MEMORY**
> `edges.mat` **borrows** the Mat; the `Image` still owns it and will release it. Do not call `release()` on what `mat` hands you. In the other direction, what comes back from a transform is ordinary immutable Scala data — `Segment` is four `Int`s and `PolarLine` two `Float`s, with no pointer behind either — so `segments` stays valid long after every Mat in this block is gone. The intermediate `Nx1` Mat that OpenCV writes its answer into never escapes at all: the private `Hough.decoding` helper allocates it, runs the transform, decodes it, and releases it in a `finally`. There is no handle here for you to leak.

Segments, lines, and votes

Table 13-1. The three entry points, all extension methods on `Mat`.

Method	Returns	Reach for it when
`houghLines`	`Seq[PolarLine]`	you want an orientation or a position and endpoints are irrelevant — a dominant angle, a vanishing point, the tilt of a page
`houghLinesP`	`Seq[Segment]`	you want to draw or measure the actual runs of edge: table sides, bar-code bars, the border of a form
`houghLinesWithAccumulator`	`Seq[PolarLineWithVotes]`	you want to rank or threshold the polar lines yourself, because you need the vote counts

`houghLinesP` — the probabilistic transform — is the one most code wants, and it takes two parameters the standard transform has no use for. `minLineLength` discards segments shorter than that many pixels (`0.0`, the default, keeps everything), and `maxLineGap` is the largest break, in pixels, still bridged into a single segment (`0.0` bridges nothing). Together they answer fragmentation: `maxLineGap` re-joins the run a shadow broke, `minLineLength` drops the specks that survived.

`Segment` carries `x1`, `y1`, `x2`, `y2` as `Int`, and this is not a rounding decision — the Mat underneath really is int32. It also gives you `start` and `end` as `Point`s and a `length`, which makes the near-universal filter a one-liner:

```
segments.filter(_.length > 200)
```

Segments are invisible until you draw them. `drawSegments` is their renderer, it lives on `Mat`, and like every `draw*` operation it mutates its receiver:

Example 13-3. Annotating the borrowed Mat of an image you still own.

```
Image.reading("page.jpg") { img =>
  val edges = img.copy.gray.canny(threshold1 = 80, threshold2 = 160)
  val found = edges.mat.houghLinesP(threshold = 80, minLineLength = 120,
maxLineGap = 10)
  edges.close()
  img.mat.drawSegments(found, Scalar.Red, Thickness.Stroke(2))
  img.write("lines.png")
}
```

Note the `copy`. Every `Image` transform consumes its receiver, so without it `img.gray` spends `img` and the later `img.mat` throws `IllegalStateException` at the reuse rather than at the move. Run with `-Dscalacv.trackOwnership=true` and that exception carries the consuming call as its cause.

Polar form, and how to draw it

A `PolarLine` is infinite; it has no endpoints to draw. To render one you pick two points far enough apart to leave the frame. Drop the perpendicular from the origin — that foot is at `(rho * cos(theta), rho * sin(theta))` — then walk both ways along the line's own direction, which is the normal turned through a right angle: `(-sin(theta), cos(theta))`.

Example 13-4. Turning an infinite line into something drawable.

```
def toSegment(line: PolarLine, reach: Double): Segment =
  val a = math.cos(line.theta.toDouble)
  val b = math.sin(line.theta.toDouble)
  val x0 = a * line.rho
  val y0 = b * line.rho
  Segment(
    math.round(x0 - reach * b).toInt,
```

```
    math.round(y0 + reach * a).toInt,
    math.round(x0 + reach * b).toInt,
    math.round(y0 - reach * a).toInt
  )

// `reach` only has to exceed the image diagonal for the segment to cross the
whole frame.
val reach = math.hypot(edges.width.toDouble, edges.height.toDouble)
edges.mat.houghLines(threshold = 150).map(toSegment(_, reach))
```

The rounding here is yours, not OpenCV's: these `Segment`s are drawing coordinates you invented, not the int32 endpoints `houghLinesP` measured. `drawSegments` cannot tell the two apart — so do not go on to treat one as a measurement.

Table 13-2. Reading a `PolarLine` back, for a 400x300 image.

`theta`	`rho`	The line
`0`	`130`	vertical, at `x = 130`
`Pi/2` (90°)	`50`	horizontal, at `y = 50`
`3Pi/4` (135°)	`0`	the main diagonal: through the top-left corner, descending to the right at 45°
a hair under `Pi` (179°)	about `-127`	near-vertical around `x = 130`, tilted one degree the other way from `theta = 0`

Ranking by votes

`houghLines` returns its lines sorted strongest-first but throws the magnitudes away, so there is no way to ask how much stronger the winner was. `houghLinesWithAccumulator` keeps them, in a `PolarLineWithVotes(rho, theta, votes)` whose first two fields are the `PolarLine`'s and whose third is an `Int` — call `.line` on one to drop the votes and get the plain `PolarLine` back. The votes are the raw accumulator counts, so `lines.map(_.votes)` says at a glance whether you have one dominant structure or six equal candidates.

Three results, three element types

OpenCV returns all three answers as an anonymous `Nx1` multi-channel Mat whose channels have no names and whose element type differs per transform: `HoughLines` gives `CV_32FC2` holding `(rho, theta)`, `HoughLinesWithAccumulator` gives `CV_32FC3` holding `(rho, theta, votes)`, and `HoughLinesP` gives `CV_32SC4` holding `(x1, y1, x2, y2)` — int32, not float. Nothing in the Java signature tells you any of this. Read the probabilistic result with a float accessor and it throws; read a float result with the wrong channel count and you silently reinterpret bit patterns into coordinates that look almost plausible.

`Hough` decodes all three through `Mat.get(row, 0): Array[Double]`, the one accessor that is depth-generic on the Java side and so correct for all three shapes. The library's tests assert on the recovered geometry rather than on "some lines came back", because a wrong decoder passes the weaker test.

Parameter order is the other deliberate departure. OpenCV's C++ signature puts `rho` and `theta` before `threshold`; scalacv puts `threshold` first, because it is the only argument without a sensible default, which makes `edges.houghLinesP(50)` the short form for an edge `Mat` named `edges`. The cost is that positional calls do not transfer between the two APIs. Name your arguments and it stops mattering.

Tuning, as a procedure

Hough tuning has a reputation for being mystical, which it earns only when people change parameters in an arbitrary order. There is a sequence, and it works because each step has a symptom you can see.

1. **Look at the edge map.** Write it to a file. If the structure is not visible to you as a human, no threshold will find it, and you are tuning `canny`, not Hough. `canny(80, 160)` is a reasonable start for a well-lit photograph; halve both for a flat one.
2. **Start deliberately permissive.** `edges.mat.houghLinesP(threshold = 50, minLineLength = edges.width / 4.0, maxLineGap = 5)`. You should get too much — a useful state, where nothing at all tells you only that something, somewhere, is wrong.
3. **Raise `threshold` until the noise stops.** Move in steps of about 20 and watch the count, not the pictures. Noise segments die off in a rush; the structure you want holds. If everything dies at once, the real lines were never much stronger than the noise, and the answer is a better edge map.
4. **Now relax `maxLineGap`.** With the noise gone, one real edge reported as four pieces is a fragmentation problem, and `maxLineGap` is its only cure: 5 to 10 for a clean scan, 15 to 25 for a photograph with shadows. Raise it too far and two collinear edges either side of a real gap — the dashes of a lane marking, the perforations on a sheet — merge into one segment.
5. **Trim with `minLineLength` last,** once you know how long the real segments come out.

> **TIP**
> For `houghLines`, `minTheta` and `maxTheta` bound the reported angle in radians (defaults `0.0` and `math.Pi`). If you only care about near-horizontal lines, bracket `math.Pi / 2` instead of filtering afterwards: the accumulator never scores the other orientations, which is both faster and much less noisy than discarding them later. `srn` and `stn` divide `rho` and `theta` for a coarse-to-fine search and default to `0.0`, which selects the classic transform — leave them there unless you have measured a reason not to.

Circles, and the escape hatch

The same voting idea extends to circles: an edge pixel votes for every circle that could pass through it, and the accumulator gains a third dimension for the radius. OpenCV implements that as `Imgproc.HoughCircles`, and scalacv does **not** wrap it — the line transforms are the ones the library commits to. The circle result has a different shape again (one row of `N` three-channel entries, read by column rather than by row), and its parameters are markedly more brittle than the line transforms'. What the library gives you instead is a clean way down to the raw call.

Example 13-5. HoughCircles through the low-level API.

```
import scalacv.*
import org.opencv.core.Mat
import org.opencv.imgproc.Imgproc

final case class Circle(x: Float, y: Float, radius: Float)

// `gray` is borrowed: single-channel, 8-bit, and blurred — not an edge map.
def circles(gray: Mat, minDist: Double): Seq[Circle] =
  Managed.use(Mat()) { out =>
    Imgproc.HoughCircles(
      gray, out, Imgproc.HOUGH_GRADIENT,
      1.0,      // dp
      minDist,
      100.0,    // param1
      30.0,     // param2
      0, 0      // minRadius, maxRadius: 0 means "unbounded"
    )
    Vector.tabulate(out.cols()) { i =>
      val v = out.get(0, i)
      Circle(v(0).toFloat, v(1).toFloat, v(2).toFloat)
    }
  }
```

> **NATIVE MEMORY**
> This is the one place in the chapter where a Mat is yours. `Imgproc.HoughCircles` writes into the output Mat you hand it and returns `void`; nothing in the binding frees that Mat, and nothing in the signature says you are now the one who has to. `Managed.use` releases it when the block returns, exception or not, and the `Vector` it returns is plain data. Once per frame in a video loop without that release, this is the textbook leak the library exists to prevent — forty bytes on the heap, megabytes off it, and no collection triggered by either.

Table 13-3. What each `HoughCircles` parameter actually controls.

Parameter	Meaning

`dp`	inverse ratio of accumulator resolution to image resolution. `1.0` is the same grid; `2.0` is half the resolution and roughly a quarter of the memory and time. Start at `1.0`, and raise it before you lower `param2`.
`minDist`	minimum distance between the centres of two reported circles. Too small and one circle is reported several times over; too large and genuinely adjacent circles are suppressed. A sensible start is the smallest radius you expect to see.
`param1`	the **upper** Canny threshold used internally — the lower one is half of it. This is where the edge map comes from, which is why the input is a greyscale image, not an edge image.
`param2`	the accumulator threshold for centres. This is the noise knob: lower it and you get false circles everywhere, raise it and real ones vanish. Tune it last, and by halving or doubling.
`minRadius`, `maxRadius`	the radius range in pixels; `0` for either leaves it unbounded. Bounding both is the cheapest accuracy improvement available, because it shrinks the search directly.

The standing advice to blur first is not superstition. `HoughCircles` estimates a gradient direction at every edge pixel and votes along it, so pixel-level noise does not merely add spurious edges — it **misaims** the votes the real edges cast. A median blur of radius 2 (a 5×5 neighbourhood) before the call is the usual fix:

```
val detected: Either[CvError, Seq[Circle]] =
  Image.reading("coins.jpg") { img =>
    val prepared = img.gray.medianBlur(radius = 2) // Image, CV_8UC1
    val found = circles(prepared.mat, minDist = 20)
    prepared.close()
    found
  }
```

If your round things are solid and well separated, contours are usually the better tool anyway: take `contours()` and score each one's circularity as `4 * math.Pi * c.area / (c.perimeter * c.perimeter)`, which is `1.0` for a perfect circle and falls away as the outline gets less round. That gives you area, centroid and bounding box for free, and never invents a circle where there is only an arc.

Worked example: the skew of a page

Back to the page on the desk. The measurement is a single number — how many degrees the page is rotated — and the segments already contain it. The long, near-horizontal ones are the text baselines and the top and bottom edges of the sheet; each carries the same tilt, plus noise.

Example 13-6. Skew, in degrees, from the segments of a document photograph.

```
/** The dominant tilt of `edges` in degrees, positive when the structure runs
downhill to
  * the right --- a clockwise tilt on screen. None when nothing long and near-
horizontal
  * was found.
  */
def skewDegrees(edges: Mat, maxTilt: Double = 20.0): Option[Double] =
  val minLength = edges.cols / 4.0
  val tilts = edges
    .houghLinesP(threshold = 80, minLineLength = minLength, maxLineGap = 20)
    .map(s => math.toDegrees(math.atan2((s.y2 - s.y1).toDouble, (s.x2 -
s.x1).toDouble)))
    .map(a => if a > 90 then a - 180 else if a <= -90 then a + 180 else a) //
fold to (-90, 90]
    .filter(a => math.abs(a) <= maxTilt)
    .sorted
  if tilts.isEmpty then None else Some(tilts(tilts.size / 2))
```

Three decisions there are worth naming. The fold to `(-90, 90]` exists because a segment's endpoints come back in no particular order, so one physical line yields either 3° or −177° depending on which end OpenCV listed first — average those and you get nonsense. The `maxTilt` filter discards the vertical structure, the page's own left and right edges, which are no less real and would otherwise dominate. And the estimate is the **median**, not the mean: one surviving diagonal — a pen lying across the page — moves a mean by degrees and a median not at all.

Example 13-7. The measurement, end to end.

```
val skew: Either[CvError, Option[Double]] =
  Image.reading("page.jpg") { img =>
    val edges = img.gray.canny(threshold1 = 60, threshold2 = 180)
    val angle = skewDegrees(edges.mat)
    edges.close()
    angle
  }
```

Straightening from there is `image.rotate(skew)` on a freshly read `Image`, once the number is out of the `Option`. The absent minus sign is the point: `rotate` turns counter-clockwise about the centre, `skewDegrees` measures a clockwise tilt as positive, and the two conventions cancel. The rotation expands the canvas as it goes, so no corner is clipped — which is where the fill matters. `Image.rotate` leaves the exposed corners in `Scalar.Black`, and black corners on a scan look exactly like ink to whatever reads the page next.

The library's own `deskew()` fills those corners white, and it does not use Hough at all: it binarises with an inverted Otsu threshold so the text becomes the foreground, collects the ink pixels with `findNonZero`, fits a `minAreaRect` to the lot, and rotates by that rectangle's angle folded into `(-45, 45]`. It keeps the original frame size rather than expanding it, which is what you want for a page that was already the right shape. Two angles are refused rather

than applied: anything beyond `maxAngle` (default `45.0`), read as a misdetection, and anything under a tenth of a degree, not worth an interpolation pass. Both leave you a plain copy.

The two estimators fail differently, which is the useful thing to know about them. `minAreaRect` over the ink measures the text block, so it wants a page that is mostly text and is confused by a large figure. The Hough estimate measures long straight runs, so it wants ruled lines, table borders or a clean page edge, and finds nothing to work with on a sparse page of prose. Where a document has both, comparing the two angles is a confidence check neither gives you alone.

When Hough is the wrong tool

Hough finds straight runs of edge, and only that. Three families of problem look like they are asking for lines and are not.

Closed shapes. If what you want is a region — its area, its centroid, whether it is a rectangle — then `contours()` gives you the outline as connected data, and `Contour.approx` collapses a noisy quadrilateral to four corners in one call. Hough hands you four unrelated segments plus the job of pairing them into corners, which is harder than it sounds when one side is missing.

Structure across two images. Matching a scene between frames, or finding a known object in a photograph, is a correspondence problem, not a line-detection one. `Features.detect` and `Features.matches` do that with ORB keypoints, which survive rotation and scale changes that move every Hough line in the frame.

Anything semantic. Hough cannot tell a lane marking from a crack in the tarmac, or a shelf edge from its own shadow: both are equally straight and vote identically. Where the distinction lives in what the thing **is** rather than in its geometry, that is a learned detector's job — `Dnn.fromOnnx` and an ONNX model — and the honest use of Hough is as a cheap prefilter that proposes candidates for something else to judge.

Where the angle goes next

The skew number computed here is not the end of anything; it is an input. Chapter 33 takes the OCR pipeline up in full — `forOcr`, the grayscale, denoise, adaptive-threshold and deskew chain that decides whether the text comes out clean, and the `OcrEngine` you plug in behind it. The nearer stop is Chapter 14, which turns `drawSegments` and the rest of the annotation surface — `Thickness`, `LineType`, the BGR ordering of `Scalar` — from the one call used here into the full instrument.

CHAPTER 14

Drawing and Annotation

Putting marks on an image, on a machine that has no screen.

A vision pipeline does not throw when it is wrong. A detector hands back four boxes; three of them sit on faces and the fourth sits on a door handle, and nothing in the return type distinguishes them. A contour filter keeps the blobs above 400 pixels of area and quietly discards the one you cared about because a shadow split it in two. A threshold that was tuned indoors leaks half the background outdoors, and the only symptom is that a centroid drifts. None of this produces a stack trace, a log line, or a failed assertion. It produces plausible numbers.

The way you find out is to draw the answer back onto the image and look. That is not a debugging convenience layered on top of computer vision — it is the primary instrument. A green rectangle in the wrong place tells you in one glance what an hour of reading coordinates will not, and the same rectangle burned into a saved frame is the artefact you attach to a bug report, hand to a reviewer, or diff against last week's build.

Which is awkward, because the machine that runs the pipeline is a container on a build agent with no display, no X server and no window manager. OpenCV's own answer to "look at the image" is `imshow`, which lives in the highgui module and wants a GUI toolkit present at build and run time; on a headless runner it is either absent or it opens nothing. scalacv does not have a display window at all, and does not want one. Annotation here is ordinary raster work: you paint into the pixels, then `write` the result or hand the encoded bytes to whatever is going to serve them.

That is why the drawing primitives live in `core` rather than in a module of their own. `Draw.scala` references nothing from highgui and nothing from JavaFX. The same handful of functions that produce a review JPEG also burn a timestamp into a recorded frame and rasterise a polygon into a mask, and splitting `core` in two to separate them would draw a line that does not exist in the work.

The running example for this chapter is a batch job of the kind that annotation exists for: a gate camera drops stills into a directory, a motion stage has already reduced each one to a handful of `Rect` regions, and the job's task is to write a version of the still that a human on a review queue can judge in two seconds. It starts as one line and ends up as something that stays readable over a sunlit driveway.

Drawing mutates, and says so

Everywhere else in scalacv an operation leaves its input alone and returns a fresh, caller-owned result. Drawing is the deliberate exception. Every primitive mutates the image it is handed and returns `Unit`:

```
def drawRect(
    rect: Rect,
    color: Scalar = Scalar.White,
    thickness: Thickness = Thickness.Default,
    lineType: LineType = LineType.Connected8
): Unit
```

The reason is that OpenCV's drawing functions have no out-of-place form. `cv::rectangle` rasterises straight into the buffer you give it; there is no variant that returns a new image. Wrapping that in a pure-looking signature would mean cloning a full frame for every annotation, and a per-frame overlay — five boxes, five labels, a HUD line — cannot afford five clones of a 1080p frame at thirty frames a second. So the library keeps the mutation and makes it impossible to miss: every operation is named `draw…` or `fill…`, and every one returns `Unit`, so no call site can mistake one for a transform.

The primitives are extension methods on `org.opencv.core.Mat`, brought in by `import scalacv.*`. They are the mid-level tier: you allocated the `Mat`, you draw into it, you release it. `Mat` is not a scalacv type and the library does not re-export it, so a file that works at this tier imports it from OpenCV alongside the library. The shape is then always make, draw, encode, release, and `Managed.use` does the release:

Example 14-1. The mid-level shape: a Mat you own, drawn into and encoded.

```
import scalacv.*
import org.opencv.core.{CvType, Mat}

val bytes: Either[CvError, Array[Byte]] =
  Managed.use(Mat.zeros(120, 200, CvType.CV_8UC3)) { canvas =>
    canvas.drawLine(Point(10, 10), Point(190, 110), Scalar.Red,
Thickness.Stroke(2))
    Images.encode(canvas, ".png")
  }
```

A `Mat` with no allocated data has nothing to draw into. Passing one to native code aborts with a message that names neither the call nor the reason, so every operation checks first and throws `IllegalArgumentException` — `drawLine needs an image with data; this Mat is empty`. That is a programmer error, not a data-dependent failure, so it throws rather than returning a `CvError`.

The primitives

There are nine of them on `Mat`, and they divide by what they can be filled with. `Thickness` is a sealed trait with two cases: `Thickness.Stroke(pixels)`, an outline at least one pixel wide, and `Thickness.Filled`, a solid shape. `Thickness.Default` is `Stroke(1)` and is what every operation uses when you say nothing.

The split exists because OpenCV encodes "filled" as a thickness of `-1`. That sentinel is a value ordinary arithmetic on a thickness will happily produce by accident, and it is meaningful only for closed shapes: `cv::line` asserts `0 < thickness`, so handing the fill sentinel to a line or to text aborts in native code. Giving lines and text a parameter typed `Thickness.Stroke`, and shapes one typed `Thickness`, turns that crash into a compile error. `Thickness.Stroke(0)` throws too, which leaves `Filled` as the only way to say "solid".

Table 14-1. The drawing primitives on `Mat`. Every one also takes `color: Scalar` and `lineType: LineType`.

Operation	Shape arguments	Thickness	On `Image`
`drawLine`	`from: Point`, `to: Point`	`Stroke`	no
`drawArrow`	`from`, `to`, `tipLength: Double = 0.1`	`Stroke`	no
`drawRect`	`rect: Rect`	`Thickness`	yes
`drawCircle`	`center: Point`, `radius: Int`	`Thickness`	yes
`drawText`	`text: String`, `at`, `font`, `scale`	`Stroke`	partly
`drawPolyline`	`points: Seq[Point]`, `closed: Boolean = true`	`Stroke`	no
`fillPolygon`	`points: Seq[Point]`	always filled	no
`drawContours`	`contours: Seq[Contour]`	`Thickness`	yes
`drawSegments`	`segments: Seq[Segment]`	`Stroke`	no

Two of those exist to make detector output visible. `drawContours` renders what `findContours` returns, and with `Thickness.Filled` it is the usual way to turn a set of contours back into a mask. `drawSegments` renders the `Seq[Segment]` that `houghLinesP` produces, which is otherwise a list of four-integer tuples nobody can read.

Coordinates outside the image are clipped rather than rejected — OpenCV's behaviour, and what makes drawing a detection that runs off the edge of a frame safe. A face box regressed from an anchor at the frame boundary legitimately has a negative `x`, and the box still draws. A negative radius on `drawCircle` is a different matter: that is a mistake, not an edge case, so it is rejected up front. And an empty point list is a no-op rather than a failure, because a polyline is frequently the output of a filter and filtering everything away is a legitimate result.

`drawArrow`'s `tipLength` is a fraction of the whole line's length rather than a pixel count, which is OpenCV's convention and the right one: a flow field whose arrows vary from three pixels to three hundred keeps proportionate heads without any work at the call site. It sits last in

the signature, after `lineType` rather than beside the geometry, because it reaches an eight-argument `Imgproc.arrowedLine` overload the shorter forms cannot; pass it by name.

> **NATIVE MEMORY**
> `drawPolyline`, `fillPolygon` and `drawContours` are not leak-free, and it would be dishonest to imply otherwise. scalacv allocates one `MatOfPoint` per polygon and releases every one of them in a `finally`. But the generated Java binding for these three then runs the list through `Converters.vector_vector_Point_to_Mat`, which allocates one `Mat` per polygon plus one for the outer vector and releases none of them. That residue is upstream in the official Java API and cannot be fixed from here without reimplementing the converter. It is bounded per call and unbounded across a video loop, so the number of calls is the variable you control: batch every polygon you have into as few `drawContours`, `drawPolyline` and `fillPolygon` calls as you can — one call for fifty contours strands fifty-one headers, fifty calls strand a hundred — and watch RSS rather than heap, because none of this residue is on the Java side. `drawRect`, `drawCircle`, `drawLine` and `drawText` are unaffected — they take plain values, not vectors.

Two shapes are absent that people go looking for: there is no `drawEllipse` and no marker glyph. Both exist one layer up, in the `Picture` scene graph of the `scalacv-graphs` module, as `Picture.ellipse` and `Picture.arc`; the last section of this chapter says when crossing that line is the right move.

Colour is a Scalar, and a Scalar is BGR

Colour is not a type in this library. It is a `Scalar` — up to four channel components, in whatever channel order the `Mat` uses — and OpenCV's default order is blue, green, red.

> **WARNING**
> `Scalar(255, 0, 0)` is blue. Every time a rectangle comes out the wrong colour, this is why. The `Scalar` is a raw pixel value, so it inherits the channel order of the image it is written into, and for the 3-channel images OpenCV decodes by default that order is BGR.

The named constants encode the order so you do not have to think about it, and they are the whole palette — five of them, on the `Scalar` companion.

Table 14-2. The named colours, and what they are in channel order.

Constant	Value	Note
`Scalar.Black`	`Scalar(0, 0, 0)`	the default fill of `Image.blank`
`Scalar.White`	`Scalar(255, 255, 255)`	the default `color` of all nine `Mat` primitives
`Scalar.Red`	`Scalar(0, 0, 255)`	not `(255, 0, 0)`
`Scalar.Green`	`Scalar(0, 255, 0)`	the default of `drawRects`, `markFaces`, `drawTracks`, `drawSkeleton`
`Scalar.Blue`	`Scalar(255, 0, 0)`	not `(0, 0, 255)`

Anything else you write by hand, in channel order: `Scalar(0, 191, 255)` is amber. Unset channels default to `0`, so `Scalar(255)` on a single-channel mask is white and needs no padding.

Text, and the plate that makes it readable

Here is the first version of the gate-camera annotator, and it is wrong in a way that costs everybody an afternoon at least once:

Example 14-2. Text at the wrong anchor. Nothing appears.

```
Image.reading("gate-0417.jpg") { img =>
  img.drawRects(regions, Scalar.Green, Thickness.Stroke(2))
     .drawText(s"${regions.size} regions", Point(12, 0))
     .write("review/gate-0417.jpg")
}
```

The boxes appear. The label does not. `at` is not the top-left corner of the text: OpenCV anchors a string at the left end of its baseline, so a `y` of `0` puts every ascender above the top of the image and leaves only the descenders of a `g` or a `y` — often nothing at all — inside the frame. Move the anchor down past the cap height and the label appears:

```
.drawText(s"${regions.size} regions", Point(12, 24))
```

That is the fix for a label at a fixed corner. It is not a fix for a label that has to sit above a detection box whose position you do not know, or for a label drawn over a photograph rather than a blank canvas, where white text on a sunlit driveway is invisible. Both of those need the string's dimensions, which is what `Draw.textSize` answers:

```
def textSize(
    text: String,
    font: Font = Font.Simplex,
    scale: Double = 1.0,
    thickness: Thickness.Stroke = Thickness.Default
): TextMetrics
```

It is the one part of drawing that asks a question instead of changing an image, and its parameters are deliberately the same ones `drawText` takes — if they do not match the call you are about to make, the answer does not describe it. What comes back is a `TextMetrics(size: Size, baseline: Int)`, and the `baseline` is separate for a reason: it is how far the descenders reach below the anchor, so a box that encloses the whole string has to be `size.height + baseline` tall. Leave it out and every `g`, `y` and `p` is clipped by the bottom edge of the plate you drew to make them readable.

With that, the label helper the annotator actually needs. It measures, draws a filled plate with a little padding, and puts anti-aliased text on top:

Example 14-3. A label that stays readable over any background.

```
def label(
    mat: Mat,
    text: String,
    at: Point,
    scale: Double = 0.6,
    fg: Scalar = Scalar.White,
    bg: Scalar = Scalar.Black,
    padding: Int = 3
): Unit =
  val m = Draw.textSize(text, scale = scale)
  val plate = Rect(
    (at.x - padding).toInt,
    (at.y - m.size.height - padding).toInt,
    (m.size.width + 2 * padding).toInt,
    (m.size.height + m.baseline + 2 * padding).toInt
  )
  mat.drawRect(plate, bg, Thickness.Filled)
  mat.drawText(text, at, fg, scale = scale, lineType = LineType.AntiAliased)
```

`LineType.AntiAliased` is worth the cost on text and on any diagonal a human will look at closely; `Connected8` is the default and is fine for boxes. `Connected4` is the third case and produces slightly thinner diagonals.

Now the annotator can put a label on every region, anchored immediately above each box:

Example 14-4. Every region boxed and labelled, in one pass over the borrowed Mat.

```
Image.reading("gate-0417.jpg") { img =>
  regions.zipWithIndex.foreach: (r, i) =>
    img.mat.drawRect(r, Scalar.Green, Thickness.Stroke(2))
    label(img.mat, s"region ${i + 1}", Point(r.x.toDouble, (r.y - 6).toDouble))

  label(img.mat, s"${regions.size} regions · gate · 04:17", Point(12, 26), scale
= 0.7)
  img.write("review/gate-0417.jpg")
}
```

Why the fonts look like a plotter drew them

OpenCV cannot render a system font. The only faces it has are the Hershey fonts — vector glyph sets digitised by Allen V. Hershey at the US Naval Weapons Laboratory in the 1960s, designed to be drawn by a pen plotter as sequences of straight strokes rather than filled outlines. That is why they scale smoothly at any `scale` and why they thicken by stroke width instead of by weight.

scalacv exposes six of those faces, as the `Font` enumeration: `Simplex` (plain sans, the default), `Plain` (small and thin), `Duplex` (double-stroke, heavier), `Complex` (serif), `Triplex` (triple-

stroke serif, heaviest) and `Script` (handwriting-style). The enumeration has no bold flag and no italic case — `Duplex` and `Triplex` are what "bolder" means here, and beyond them the only lever is `thickness`.

The other consequence is ASCII. A non-ASCII character is drawn as `?`, silently. If your labels carry names, currency symbols or anything accented, OpenCV text is not the renderer for them; draw the overlay geometry here and composite real typography elsewhere.

Two tiers, and who owns the Mat

Everything so far has been the mid-level tier, reached through `img.mat`. There is a higher one: `Image` exposes an everyday subset of the same operations as chainable transforms. `drawRect`, `drawCircle`, `drawText`, `drawContours` and `drawRects` are members of `Image`, and each one consumes the image it is called on and returns a fresh one, so annotation reads as a pipeline that ends in a terminal:

Example 14-5. The high-level tier: annotation as a chain that ends in a terminal.

```
val annotated: Either[CvError, Array[Byte]] =
  Image.blank(width = 200, height = 120)
    .drawRect(Rect(20, 20, 70, 60), Scalar.Green)
    .drawCircle(Point(150, 60), 30, Scalar.Red, Thickness.Filled)
    .drawText("scene", Point(16, 100), Scalar.White)
    .bytes(".png")
```

Nothing is copied by either tier. Internally an `Image` draw takes the `Mat` out of its handle, mutates it in place, and rewraps it in the returned `Image` — the move that makes the old handle throw `IllegalStateException` rather than alias live pixels. The difference between the tiers is not performance, it is who tracks ownership: you, or the `Image`.

What the high-level tier costs you is knobs. `lineType` does not survive the trip up to `Image` on any verb, and `Image.drawText` is `drawText(text, at, color, scale)` — no `font`, no `thickness`. `drawLine`, `drawArrow`, `drawPolyline`, `fillPolygon` and `drawSegments` have no `Image` form at all. Going the other way, `drawRects` exists only on `Image`: it paints a whole `Seq[Rect]` in one pass, which is the shape detector output actually arrives in, and it is the one place the high-level tier is the more convenient of the two.

You do not have to choose per pipeline, only per line. `img.mat` is a query — it borrows the handle and leaves the `Image` alive — so a chain can drop to the mid-level tier for one bold, anti-aliased heading and pick up again on the next line, all writing into the same pixels.

Example 14-6. Mixing the tiers: the same Mat, two signatures.

```
val img = Image.blank(200, 120)
img.mat.drawArrow(Point(10, 60), Point(120, 60), Scalar.Green)
```

```
img.mat.drawText(
  "bold",
  Point(10, 30),
  Scalar.White,
  font = Font.Duplex,
  scale = 0.8,
  thickness = Thickness.Stroke(2),
  lineType = LineType.AntiAliased
)
img.drawText("go", Point(130, 66), Scalar.White).bytes(".png")
```

NATIVE MEMORY

`img.mat` hands you the underlying `Mat` without transferring ownership. Draw on it, pass it to a detector, read from it — but do not call `release()` on it. The `Image` still believes it owns that buffer, and the next transform in the chain will read freed memory. Let a terminal (`write`, `bytes`) or `close()` do the releasing, or take ownership explicitly with `managed`, which spends the `Image` and hands the `Managed[Mat]` over.

Keeping the original

A transform consumes its receiver, and a draw is a transform, so a single `Image` cannot be both annotated and kept. When you need both — the clean frame for the next pipeline stage, the marked-up one for the review queue — take a `copy` first. `Image.copy` is an independent deep copy, which is to say a second full frame of native memory.

That is also the mechanism for a translucent overlay, which OpenCV has no direct support for: draw the solid shapes onto a copy, then blend the copy back over the original. `Image.blend(other, weight)` computes `this * weight + other * (1 - weight)`, borrowing `other` and consuming `this`:

Example 14-7. A translucent shade over a region, by drawing on a copy and blending back.

```
Image.reading("gate-0417.jpg") { img =>
  val shade = img.copy
  regions.foreach(r => shade.mat.drawRect(r, Scalar.Black, Thickness.Filled))
  try img.blend(shade, 0.65).write("review/gate-0417.jpg")
  finally shade.close()
}
```

Read the arithmetic where it matters. Outside the regions the copy is pixel-for-pixel the original, so `0.65 × p + 0.35 × p` is `p` and the blend changes nothing. Inside them the copy is black, so what survives is sixty-five per cent of the original and the region comes back dimmed by the remaining thirty-five. Swap `Scalar.Black` for a colour and the same shape tints instead of dims; invert which image is drawn on and it greys out everything the stage is **not** looking at. The weight is the only knob, and `blend` rejects one outside `[0, 1]` with an

`IllegalArgumentException` rather than letting `addWeighted` clip its way to a plausible-looking wrong answer.

> **NATIVE MEMORY**
> `blend` borrows `other`; it does not consume it. The `copy` is yours, and nothing else will free it. In the listing above the `finally` is doing real work: it releases the second frame whether the blend succeeds, fails, or throws a `CvError.NativeCall`. In a per-frame loop the same shape without the `finally` costs one whole frame per iteration, which is exactly the failure mode Chapter 5 measured — a heap that stays small while the resident set climbs.

Overlays that know what they are drawing

The nine primitives are general-purpose. `scalacv-vision` builds one-call overlays on top of them, each turning a typed result straight into pixels, and each living beside the type it renders rather than in `Image`. They are extension methods on `Image` in `scalacv.vision`, so add the vision dependency and `import scalacv.vision.*` alongside `import scalacv.*`. Every one is built on the same internal `paint` that the `Image` draw verbs use — so each consumes its receiver and hands back a fresh `Image`, exactly like `drawRect`.

`markFaces(faces, color = Scalar.Green)` draws a box per `Face` and a filled dot per landmark — the one-call "show me what YuNet found". `drawSkeleton(pose, minScore = 0.3f, color, jointColor)` draws a line per bone and a dot per confident keypoint. `drawTracks(tracks, color)` draws a box per `ObjectTrack` with its id above it. `drawMarkerAxes` and `drawMarkerCube` project a 3-D coordinate frame or a wireframe cube onto every ArUco marker in the image, given the camera `Intrinsics` and the marker's real side length; the axes are red, green and blue for X, Y and Z, which is the classic "is my pose right?" overlay.

None of these is doing anything you could not write yourself from this chapter. `markFaces` is a `drawRect` on the face box and a filled `drawCircle` of radius 2 on each landmark, and that is the entire body. They earn their place by being one call at the point where you need them, and their sources are worth reading as worked examples of the primitives at their intended scale. Each is covered alongside the detector that produces its input: faces in the face-detection chapter, skeletons in pose estimation, tracks in tracking, and both marker overlays in the marker-AR chapter.

Where the primitives stop

Every recipe in this chapter has been a small pile of primitive calls held together by a helper function, and that is the honest signal. When the helper starts taking a style, when two call sites want the same overlay at different positions, when you need a dashed outline, a rotated shape, or an overlay composed once and drawn onto many frames — you have started writing a scene graph by hand, and there is one in the box.

`Picture`, in the `scalacv-graphs` module, is a composable value: shapes, dashed strokes, text, transforms and transparency, built up with combinators and rendered in one call by `image.draw(picture)`. It renders through exactly the primitives in this chapter, so it costs nothing in fidelity; what it costs is a second dependency, and what it buys is that an overlay becomes a value you can name, transform, reuse and test rather than a sequence of side effects.

The line is worth stating plainly rather than leaving to taste. Use the primitives directly when the annotation is a handful of marks whose coordinates you already have — a box round a detection, a timestamp in a corner, a contour rasterised into a mask. Reach for `Picture` when the overlay has structure: repeated elements, a dash pattern, a rotation, an opacity, an ellipse, or a shape that gets positioned by something other than the pixel coordinates it will end up at.

Next

Chapter 15, on the photographic and stylisation operations, closes out the pixel-processing half of the book: the verbs — denoising, sharpening, tone work — whose results are judged by eye rather than by an assertion, which is to say the verbs that need everything this chapter said about getting a look at the answer.

Chapter 16 then takes the second half of the split above and builds it out: `Picture` as an algebra, the combinators that compose one overlay out of others, and the styling — dashes, transparency, affine transforms — that the raw OpenCV primitives cannot express at all. Everything it draws still ends up going through `drawLine`, `drawPolyline` and `drawText`, so the pixels are the ones you already know how to reason about.

CHAPTER 15

Photographic and Stylisation Operations

The verbs judged by eye rather than by assertion, and what they cost you.

A local-history archive hands you four thousand scanned prints and a flatbed that will not be operated by a human again. The scans come off it faded, warm where the dye has shifted over sixty years, and speckled with dust that the glass picked up somewhere around print six hundred. The archive wants two things out of each file. It wants a picture a visitor will look at on a screen, and it wants a record a program can search — who is in it, what is written on the back, which of the four thousand are the same building from different years.

Those two wants pull in opposite directions, and that tension is the whole subject of this chapter. Every operation the book has covered so far was in service of measurement. A blur exists to make a threshold behave; a threshold exists to make contours findable; contours exist to produce numbers. You can write an assertion about each of them: this many components, that bounding box, this angle. The operations in this chapter have no such test. `sepia` is correct when it looks like sepia. `stylize` is correct when a person says the picture looks painted. The only honest acceptance criterion is somebody's eye, and the library says so by giving them defaults that look reasonable and leaving the tuning to you.

That does not make them decorative — `inpaint`, `blend` and `seamlessCloneInto` repair and composite pixels for reasons unrelated to taste, and `colorMap` is the only way to make a single-channel result visible at all. But it does mean the group needs a different kind of care. Several of these calls are among the most expensive things you can ask scalacv to do, and they are exactly the calls somebody tries once on a still, likes, and drops into a thirty-frames-a-second loop. Several destroy, on purpose, the structure a detector downstream was relying on. Neither failure announces itself.

Everything here is a transform in the sense Chapter 4, *The Image Type*, set out: the call consumes the `Image` it was made on, hands back a new one, and releases the old `Mat` on the way through. A chain of eight grades holds one live buffer, not eight. Where a call takes a second image — a mask, a background, the other side of a blend — that argument is *borrowed*, and closing it stays your job.

> **NOTE**
> The identifiers follow OpenCV's American spelling while the prose follows the library's own documentation, so the page says "colour map" and the code says `colorMap`. The same split catches `gray`, `stylize`, `posterize` and `Colormap`.

Tone and colour

The tone grades are the cheap half of the chapter. Most of them are a single pass over the pixel data with no neighbourhood at all, which is why you can afford them per frame when you cannot afford anything in the next section.

`gamma(g)` and `posterize(levels)` are both a 256-entry lookup table, built in Scala and applied to every channel with one `Core.LUT` call. `gamma` fills the table with `pow(i / 255.0, 1.0 / g) * 255`, so `g` below 1 darkens the mid-tones and above 1 lifts them, while pure black and pure white stay where they were — the curve is pinned at both ends. That is the difference between it and `adjust(brightness, contrast)`, which multiplies and adds and therefore clips as soon as it runs out of range. Useful values sit between about 0.7 and 1.4; the method `require`s only that `g` is positive, so `gamma(12)` runs and gives you a white rectangle.

`posterize(levels)` quantises each channel to `levels` evenly-spaced tones, with a step of `255.0 / (levels - 1)`. It `require`s `levels` in `[2, 256]`, and 256 is the identity. Four to eight reads as a deliberate screen-printed look; below four it reads as a bug.

`saturate(factor)` is not an HSV round trip. It converts to grey, converts that back to three channels, and then takes a weighted sum of the original against the grey: `factor` at the source, `1 - factor` at the grey version. So `saturate(0)` is a three-channel greyscale — still `CV_8UC3`, which matters more than it sounds — `saturate(1.0)` is the identity, and anything above 1 is an extrapolation *away* from grey rather than an interpolation towards it. Extrapolation clips, so 1.2 to 1.5 is vivid and 3.0 is a poster of six colours. The `require` is only that `factor` is non-negative.

`temperature(shift)` is a fixed 3×3 colour matrix run through `Core.transform`, and the matrix is smaller than the name suggests: it scales blue by `1 - 0.3 * shift`, scales red by `1 + 0.3 * shift`, and leaves green alone. `shift` is `require`d to be in `[-1, 1]`, so the extreme setting is a thirty-per-cent swing on two channels. Positive is warm, negative is cool, and the honest description is a white-balance nudge, not a correction: it has no idea where your white point is.

`sepia` is the same mechanism with a matrix nobody tunes — the classic `0.131, 0.534, 0.272 / 0.168, 0.686, 0.349 / 0.189, 0.769, 0.393`, in BGR order — and takes no argument at all. `emboss` is a 3×3 `filter2D` with the directional kernel `-2, -1, 0 / -1, 1, 1 / 0, 1, 2`, at `OutputDepth.SameAsSource`. That kernel sums to 1, which is a deliberate choice with a visible consequence: a flat region keeps its own brightness instead of collapsing to mid-grey, so scalacv's emboss is a relief cut into the original tones rather than the grey plate you may be expecting from other tools.

Table 15-1. The tone verbs, their arguments, and the range that is actually useful.

Verb	Rejects	Useful range	Reach for it when

`gamma(g)`	`g <= 0`	0.7 – 1.4	shadows are blocked up and the highlights are fine
`posterize(levels)`	outside `[2, 256]`	4 – 8	you want flat banding as a look
`saturate(factor)`	`factor < 0`	0 – 1.5	colour has faded, or you want grey that still chains
`temperature(shift)`	outside `[-1, 1]`	−0.5 – 0.5	the cast is warm or cool and you know which
`adjust(brightness, contrast)`	nothing	±30, 0.8 – 1.4	you want a linear level change, clipping and all
`sepia`	—	—	a period wash, once
`emboss`	—	—	relief, or a cheap directional derivative

Colour maps, and why the default matters

`colorMap(map: Colormap)` is the odd one here, because it is not a look. It takes a single-channel image — a disparity map, a motion field, a distance transform, any scalar field — and false-colours it so a person can see it. Without it, the output of `StereoDepth.disparity` or a `MotionDetector` is a grey rectangle that conveys almost nothing at a glance.

`Colormap` has exactly ten cases, matching OpenCV's `COLORMAP_*` set, and the choice between them is not cosmetic. Five — `Viridis`, `Magma`, `Inferno`, `Plasma` and `Turbo` — are perceptually uniform: equal steps in the value produce equal-looking steps in colour, and they degrade sensibly to greyscale. `Jet` does not. Its rainbow has bright bands at cyan and yellow that the eye reads as boundaries, so a smooth ramp acquires edges that are not in the data, and it is close to unreadable for a colour-blind viewer or on a monochrome printout. The enum's own scaladoc says it plainly: the perceptually-uniform ones are the honest choice for data, `Jet` is the classic-but-misleading rainbow.

Table 15-2. The ten colour maps and what each is for.

Case	Family	Use it for
`Viridis`	perceptually uniform	the default honest choice for data
`Magma`	perceptually uniform	dark ground, high drama
`Inferno`	perceptually uniform	heat and energy fields — what `Filter.heatmap` uses
`Plasma`	perceptually uniform	bright, high-contrast data
`Turbo`	perceptually uniform	a rainbow when you must have one, corrected
`Jet`	legacy rainbow	matching an older tool's screenshot, and nothing else
`Hot`	sequential	thermal imagery
`Bone`	sequential grey-blue	the X-ray look
`Ocean`, `Autumn`	sequential	stylistic

There is one mistake here that everybody makes once, and it is not a colour choice. `applyColorMap` accepts `CV_8UC1` and `CV_8UC3` only. A float response — a Sobel derivative taken at `OutputDepth.Float32`, a distance transform — is `CV_32F`, and handing it straight to `colorMap` aborts inside native code:

```
// Wrong: the response is CV_32F, and applyColorMap does not take it.
response.colorMap(Colormap.Viridis)
```

The fix is one call, and it is the reason `normalize` defaults to `OutputDepth.Unsigned8` rather than to OpenCV's own `dtype = -1`:

Example 15-1. Normalising is what makes a float result viewable, not merely rescaled.

```
response.normalize(0, 255).colorMap(Colormap.Viridis).write("response.png")
```

With `-1` the rescale would hand back a `CV_32F` holding the values 0 to 255, which looks like it worked and is not viewable by anything: `toBufferedImage` rejects it and `colorMap` still aborts. For an already-8-bit image the default is the same conversion either way, so a plain contrast stretch is unaffected by it.

Detail and texture

`edgePreserving`, `enhance`, `stylize` and `sketch` are a different kind of animal. They do not map pixels; they run an edge-aware solver across the whole image, and every one of them wants an 8-bit three-channel input. Give one a greyscale `Mat` and OpenCV asserts, which reaches you as `CvError.NativeCall("stylization", …)` — the operation name comes from `Mats.produce`, so the error tells you which call it was even in the middle of a chain.

```
// Wrong: gray leaves one channel, and all four of these need three.
scan.gray.stylize()
```

If what you wanted was a monochrome stylisation, desaturate without dropping channels — `scan.saturate(0).stylize()` — which is exactly why `Filter.grayscale` is defined as `saturate(0)` rather than as `gray`.

`edgePreserving(strength: Float = 60, detail: Float = 0.4f)` is the foundation of the other three and the most useful of them on its own. It smooths flat regions while leaving edges intact, which makes it a gentle denoise you can run before a threshold or a contour pass when you want to kill texture without softening boundaries. scalacv fixes OpenCV's filter choice at `Photo.RECURS_FILTER`; the alternative, `Photo.NORMCONV_FILTER`, is one `Photo.edgePreservingFilter` call away on a borrowed `mat` if you need it.

`enhance(strength: Float = 10, detail: Float = 0.15f)` — `detailEnhance` at the mid level — runs the same smoothing and adds the difference back, boosting local contrast. It is the "clarity" slider, and what `Filter.vivid` and `Filter.dramatic` are built from.

`stylize(strength: Float = 60, detail: Float = 0.45f)` pushes the smoothing much harder and saturates the result. Raising `strength` towards 80 and dropping `detail` towards 0.3 makes it more abstract, not more detailed — the opposite of what the parameter name suggests the first time.

`sketch(strength: Float = 60, detail: Float = 0.07f, shade: Float = 0.02f)` is the pencil rendering. Note how much smaller its `detail` default is than the others: 0.07 against 0.45. These are not the same scale across the four operations, and the only way to find your value is to render a few.

The sketch OpenCV throws away

`Photo.pencilSketch` is the one call in this family with two outputs. It fills a greyscale sketch *and* a colour one, in a single pass, from the same intermediate. scalacv's `sketch` returns only the colour version — and the mid-level `pencilSketch` does the same — so the grey plate is allocated, written, and released, every call.

The implementation is honest about it: `Managed.use(Mat())` owns the discarded output for exactly the duration of the native call and frees it whether the call succeeds or throws. The cost is one extra buffer for the length of one operation, and the alternative — returning a tuple that almost every caller destructures and discards half of — would have made the common case worse to read. If you want the grey sketch, `Photo.pencilSketch` on a borrowed `mat` gives you both, and the two `Mat`s are then yours to release.

Table 15-3. The photo-module operations and their defaults.

`Image` verb	Mid-level `Mat` op	Signature
`edgePreserving`	`edgePreserving`	`(strength: Float = 60, detail: Float = 0.4f)`
`enhance`	`detailEnhance`	`(strength: Float = 10, detail: Float = 0.15f)`
`stylize`	`stylize`	`(strength: Float = 60, detail: Float = 0.45f)`
`sketch`	`pencilSketch`	`(strength: Float = 60, detail: Float = 0.07f, shade: Float = 0.02f)`

Repair and composition

`inpaint(mask: Image, radius: Double = 3.0)` reconstructs whatever lies under a mask from the pixels around it. The mask is `CV_8UC1`, and non-zero marks the pixels to *throw away and reconstruct*. `applyMask` reads non-zero as the pixels to *keep* — the same convention aimed at the opposite outcome, and worth saying out loud, because the two sit next to each other in the API and neither will complain about the other's mask. Hand `inpaint` one built for `applyMask` and it dutifully reconstructs everything except the damage. `radius` is how far out from each damaged pixel OpenCV looks for material; 3 to 5 covers dust and thin scratches, and much higher smears rather than repairs, because the algorithm has no model of what should be there, only of what is nearby.

scalacv fixes the algorithm at `Photo.INPAINT_TELEA`, the fast marching method. OpenCV's other choice, `Photo.INPAINT_NS`, solves a Navier–Stokes formulation instead and tends to carry long-range structure — a continuing straight edge — a little further across a wide gap, at more cost. It is not exposed as a parameter; if you want it, call `Photo.inpaint` yourself on a borrowed `mat`, which is a two-line drop out of the high-level API and back.

For the archive scans, the mask is the interesting part. Dust on a print is bright detail smaller than any real feature, which is precisely what a top-hat morphology isolates.

Example 15-2. A dust mask from the scan itself: bright specks, thresholded, grown to cover their edges.

```
def dustMask(scan: Image): Image =
  scan.gray
    .morphology(MorphOp.TopHat, radius = 3)
    .threshold(40)
    .dilate(radius = 2)
```

`morphology(MorphOp.TopHat, radius = 3)` is the source minus its own opening, so anything brighter than its surroundings and smaller than the 7×7 kernel survives and everything else goes to zero. `threshold(40)` turns that into the binary mask `inpaint` requires, and the `dilate` grows each speck by two pixels so the repair covers its soft halo rather than leaving a ring. The argument is a copy, because `dustMask` consumes what it is given.

> **NATIVE MEMORY**
> `inpaint`, `applyMask`, `blend` and `seamlessCloneInto` all follow one rule, and it is not the rule the rest of the chapter follows. The receiver is *consumed* and becomes the result; every `Image` you pass as an argument is *borrowed* and is still yours to close. Nothing in the type signature says so, and a leaked mask is a full-size native buffer that the collector has no reason to reclaim — exactly the shape of leak Chapter 1 measured at 5,865 MB. Wrap the mask in `scala.util.Using`, or close it on the line after the call. Do not let it fall out of scope.

`blend(other: Image, weight: Double = 0.5)` is the ordinary weighted average: `this * weight + other * (1 - weight)`, with `weight` `require`d to be in `[0, 1]` and `other` borrowed. Both images must match in size and type, which is the constraint that bites — a three-channel scan and a one-channel mask are not blendable, and the failure arrives from native code rather than from a `require`. It is a cross-fade, a watermark, a two-exposure merge; nothing clever, and cheap.

`seamlessCloneInto(background: Image, mask: Image, center: Point)` is the clever one. Pasting an object into a photograph fails on the seam: however good the cut-out, the boundary carries the lighting of the image the object came from, and the eye finds that line immediately. Poisson cloning solves the problem by discarding the object's absolute colours and keeping only its gradients, then solving for the pixel values that reproduce those gradients while matching the background exactly along the boundary. There is no seam because the boundary values are, by construction, the background's own.

The receiver is the foreground object, `mask` marks which of its pixels are the object, `center` is where in the background the object's centre lands, and the result is background-sized. scalacv fixes the mode at `Photo.NORMAL_CLONE`; OpenCV also offers `Photo.MIXED_CLONE`, which takes the stronger gradient of foreground and background at each pixel and so lets background texture show through a thin object, and `Photo.MONOCHROME_TRANSFER`. Both are reachable through `Photo.seamlessClone` on borrowed `mat`s.

What these cost

The two halves of this chapter are not in the same league, and the difference is large enough to decide your architecture rather than your style.

The tone grades — `gamma`, `posterize`, `saturate`, `temperature`, `sepia`, `emboss`, `adjust`, `colorMap` — are one or two passes over the pixels with no iteration: a table lookup, a per-pixel matrix, a 3×3 convolution, or in `saturate`'s case two colour conversions and a weighted sum. Any of them can go on a video path without much thought.

The four edge-aware operations — `edgePreserving`, `enhance`, `stylize`, `sketch` — run a solver over the entire frame and are heavier by a wide margin. `inpaint` and `seamlessCloneInto` are heavier again and, worse, data-dependent: `inpaint` costs what the masked area costs, and Poisson cloning solves a linear system over the pasted region. A larger hole is not a little slower; it is a different amount of work.

> **WARNING**
> Do not take a number from this page, or from any other page, for those six. The library publishes measured figures in `docs/mdoc/benchmark-results.md` for what it has actually benchmarked, and the stylisation operations are not among them — so the honest statement is the ordering, not a millisecond count. They are also among the operations OpenCV parallelises internally, which means their cost on your machine depends on `getNumThreads` and on whether the build has IPP.

Chapter 35 settles this properly: the benchmark harness, how to read a confidence interval, and `ConfigProbeBench`, which prints your runtime's `useOptimized`, `getNumThreads` and build information and then measures how a heavy internally-parallel operation scales as `Core.setNumThreads` is stepped through 1, 2, 4 and your core count. That answer is a property of your hardware. The project's rule applies here more than anywhere: no optimisation without a measured delta *and* a bit-identical output hash, because a change that is faster by computing something slightly different is not an optimisation, and a timing number alone cannot tell the two apart.

A look you can repeat

Back to the four thousand scans. The grade is three tone operations, in this order: cool the warm cast the dye shift introduced, restore the colour that faded with it, and pull the

mid-tones back down where the print has gone flat and light. Each is cheap, each has one argument, and together they are a house look — which is exactly what `Filter` exists to name.

Example 15-3. Three verbs, one name, applied four thousand times.

```
val archive = Filter("archive")(_.temperature(-0.2).saturate(1.3).gamma(0.9))

Image.reading("scans/0417.tif") { scan =>
  Using.resource(dustMask(scan.copy)) { mask =>
    scan.inpaint(mask, radius = 4.0).filter(archive).write("graded/0417.jpg")
  }
}
```

`Filter` is nothing but a `String` and an `Image => Image`, so `archive` is a value you can log, show in a picker, and compose: `archive.andThen(Filter.sharpen)` has the `name` `"archive+sharpen"`. The seventeen built-in looks in `Filter.all` are built the same way from the verbs in this chapter — `noir` is `saturate(0).adjust(contrast = 1.3).gamma(0.9)`, `vintage` is `sepia.saturate(0.85).gamma(0.9)` — and reading that catalogue teaches the grades faster than any table.

> **NATIVE MEMORY**
> A contact sheet is the natural next thing to build, and it is where a filter catalogue turns into a leak. `Filter.all.map(f => image.filter(f))` throws on the second filter, because the first consumed `image`. The fix that people reach for — `Filter.all.map(f => image.copy.filter(f))` — runs, and strands seventeen full-size buffers unless every result reaches a terminal. `bytes(".png")` or `write(…)` on each one closes it; anything else does not.

Now the other half of the archive's request: the searchable records. Run the grade first and feed the graded file to the face detector, and you will get a worse index than you would have got from the raw scan — possibly a much worse one, with no error anywhere to tell you.

> **CAUTION**
> Photographic operations are subtractive with respect to features. `edgePreserving` and `stylize` exist to remove texture, and texture is precisely what an ORB keypoint is: `Features.detect` will find fewer corners on a stylised frame and match them worse. `posterize` replaces smooth gradients with flat plateaus separated by hard steps, so `canny` finds the banding boundaries instead of the object. `emboss` is a directional derivative, so everything downstream is looking at a gradient field that merely resembles a picture. `gamma` and `saturate` move the intensity distribution a detector was trained on.
>
> Grade a copy for the human. Run the detector on the original. When both must happen, `copy` before the first transform is the cheapest correct answer, and it costs one buffer.

One exception runs the other way: `edgePreserving` before a threshold or a contour pass often *helps*, because there the texture it removes is noise and the edges it keeps are the boundaries you were about to look for. The rule is not "never before analysis". It is that a photographic

operation is a decision about what to discard, and the only safe version of that decision is one you made deliberately, knowing what comes next.

Where this goes next

The grade is now a value with a name, applied to a still and written to disk. That closes out the pixel-processing half of the book. Every verb from Chapter 7 onward has had the same shape: it takes an `Image`, does something to the pixels, and hands back an `Image`, and the interesting question was always which pixels and at what cost.

Chapter 16 keeps the pixels and changes what you build on top of them. The annotation over a graded frame — the box, the label, the legend — was a sequence of mutating `drawRect` and `drawText` calls in Chapter 14; `scalacv-graphs` makes it a `Picture`, an immutable value you can compose, transform and test before a single pixel moves. Nothing in this chapter changes there. The grade is still applied to the pixels underneath, and the overlay is composited on top of the result.

The cost ordering above comes due later, in Chapter 19 and Chapter 20, when the still becomes a frame arriving thirty times a second from `Video.frames` or `Camera.taking` rather than once from a flatbed, and `Recorder.write` borrows that frame's `Mat` so a recorded pipeline copies nothing per frame. Every look you have built here can run on that path. Most of them should not.

PART III

The Graphics Layer

`scalacv-graphs`: an immutable scene graph that turns an overlay from a sequence of mutating calls into a value you can build, transform, test and composite — plus the charts and animations that fall out of it for free.

CHAPTER 16

The Picture Scene Graph

An overlay you can build, transform and test before a single pixel moves.

Chapter 14 gave you a set of drawing verbs, and they are the right tool for exactly as long as an overlay stays small. Two rectangles and a caption on a debug frame: `drawRect`, `drawText`, write it out, look at it. Nobody needs a scene graph for that, and this chapter is not going to pretend otherwise.

The trouble starts at about the fifth element. An annotated frame in a real system is a box per detection, in a colour that encodes the class; a readable tag over each box, with the score to two decimal places; a connector between the boxes the tracker believes belong to the same object; and a legend in one corner mapping colour to class name. Written as drawing verbs, that is thirty or forty imperative calls against a `Mat`, interleaved with the arithmetic that decides where each one goes, all of it executing for its side effect and none of it producing a value you can hold.

Everything you would normally want to do with that overlay is then out of reach. You cannot render it twice — once at full resolution for the archive, once at a quarter size for the operator's preview — without running the whole thing again and hoping the two agree. You cannot move it, because "it" is not a thing; it is a sequence of calls with pixel coordinates baked into each one. You cannot check whether a legend entry landed six pixels too low without rendering it and looking, because the only artefact it leaves behind is pixels. And you cannot reuse the legend in the report generator, because the legend is not a value either.

`scalacv-graphs` answers this with one idea, borrowed from Doodle and adapted to image space: a drawing is a value. `Picture` is an immutable description of what to draw — shapes, styles, transforms, groups — built out of ordinary Scala data and owning no native memory. You assemble it, transform it, measure it, compare it, cache it and pass it around like any other case class. Rasterisation happens once, at the end, when you hand the finished value to an image. Up to that moment nothing has touched a pixel.

A module of its own

`Picture` does not live in the core artifact. It is published separately as `scalacv-graphs`, whose whole dependency list is `core` and the same OpenCV Java bindings `core` already pulls in — not `scalacv-vision`, not any GUI toolkit. If you want it, add the second line:

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"com.worxbend::scalacv-graphs:0.4.2",
```

```
  // plus the two native lines for your platform, as in Chapter 2
)
```

The split is not ceremony: a service that decodes frames, thresholds them and writes a mask has no use for a scene graph. What the extra dependency buys is `Picture` and its supporting types `Color`, `Dash` and `Bounds`, plus `Chart`, `Animation`, and the `image.draw(picture)` extension that bridges back to `Image` — in `scalacv.graphs`, brought into scope with `import scalacv.graphs.*` beside the core `import scalacv.*`. Vision similarly needs `import scalacv.vision.*`.

The frame you cannot annotate twice

The running example for this chapter is an inspection camera over a conveyor. A detector upstream hands you a sequence of parts, each with a box, a class name and a confidence:

```
final case class Part(box: Rect, name: String, score: Double)
```

The obvious first attempt does not survive contact with Chapter 4's move semantics.

Example 16-1. Wrong. `drawRect` consumes the frame, so the second part throws.

```
Image.reading("belt.jpg") { frame =>
  parts.foreach(p => frame.drawRect(p.box, Scalar(0, 255, 0),
Thickness.Stroke(2)))
  frame.write("belt-annotated.png")
}
```

`drawRect` is a transform: it spends the receiver and hands back the annotated image. The first iteration takes the handle, and the second gets an `IllegalStateException` from a `Managed` that has already been moved. The fix is to thread the image through, which is what a fold is for.

Example 16-2. Correct, and still nothing but a sequence of side effects.

```
Image.reading("belt.jpg") { frame =>
  parts
    .foldLeft(frame) { (img, p) =>
      img
        .drawRect(p.box, Scalar(0, 255, 0), Thickness.Stroke(2))
        .drawText(p.name, Point(p.box.x.toDouble, p.box.y - 6.0), Scalar(0, 255,
0), scale = 0.5)
    }
    .write("belt-annotated.png")
}
```

This is correct code and it will ship. It is also the last version of the overlay that fits in one listing: add per-class colours, tags with backgrounds, connectors and a legend, and the fold's

accumulator becomes a hundred lines of arithmetic whose only output is a file you have to open in a viewer to review.

A picture is a value

Here is the same annotation as a `Picture`. Note what is absent: no `Image`, no `Mat`, no `Scalar`, no ownership question, and nothing that has to run in a particular order.

Example 16-3. The per-part annotation, as data.

```
def annotate(part: Part, color: Color): Picture =
  Picture
    .roundedRectangle(part.box, radius = 6)
    .stroke(color, 2)
    .dashed
    .on(tag(f"${part.name} ${part.score}%.2f", part.box.topLeft, color))
```

`roundedRectangle(rect, radius)` is one of the companion's constructors, and it clamps the radius to half the shorter side so an over-large corner degrades into a stadium rather than into nonsense; `stroke(color, width)` sets the outline colour and width in one call; `dashed` asks for the `Dash(10, 8)` preset; and `a.on(b)` draws `a` over `b`. The result is a small tree — an overlay of the styled box and the tag — and it has drawn nothing.

The tag itself is worth building by hand, because it shows the one combinator that makes layout possible — measurement:

Example 16-4. A tag: text over a background box, sized from the text's own extent.

```
def tag(text: String, at: Point, color: Color): Picture =
  val glyphs = Picture.text(text, at).strokeColor(Color.Black).fontScale(0.45)
  glyphs.bounds match
    case Some(b) =>
      val box = Rect(
        (b.minX - 4).round.toInt,
        (b.minY - 3).round.toInt,
        (b.width + 8).round.toInt,
        (b.height + 6).round.toInt
      )
      glyphs.on(Picture.rectangle(box).fillColor(color).noStroke)
    case None => glyphs
```

`bounds` returns `Option[Bounds]` — `None` for a picture that draws nothing, a `Bounds` with `minX`, `minY`, `maxX`, `maxY`, `width`, `height`, `centerX` and `centerY` otherwise. For text it is not a guess: the measurement runs the string through `Draw.textSize` with the style's own font and scale, so the descenders on a `g` or a `y` are inside the box rather than clipped by it.

There is a ready-made version of the whole tag on the companion, and it is worth reading its signature closely before you reach for it:

```
Picture.label(
  text: String,
  at: Point,
  textColor: Color = Color.White,
  background: Color = Color.Black,
  padding: Int = 4,
  fontScale: Double = 0.5,
  font: Font = Font.Simplex
): Picture
```

`at` is the box's top-left, not the text's baseline, and the box comes out `size.width + 2·padding` by `size.height + baseline + 2·padding` — that `baseline` term being the descender room a `y` or a `g` needs, measured rather than guessed.

Painter order matters: the intended label is `glyphs.on(box)`, with the background drawn before its text. The current implementation fixes the reversed order in the 0.4.1 source release. When using that older tag, construct `glyphs.on(box)` yourself; the unreleased fix is covered by visible-glyph pixel assertions, not merely nonempty bounds.

The primitives

Every constructor on the `Picture` companion returns a `Picture`, so every one of them styles, transforms, measures and composes the same way. There is no separate "shape" type and no builder.

Table 16-1. Every `Picture` constructor, with the parameters that have defaults spelled out.

Constructor	Signature
`circle`	`circle(center: Point, radius: Double)`
`rectangle`	`rectangle(rect: Rect)`
`roundedRectangle`	`roundedRectangle(rect: Rect, radius: Double)`
`line`	`line(from: Point, to: Point)`
`polyline`	`polyline(points: Seq[Point], closed: Boolean = false)`
`polygon`	`polygon(points: Seq[Point])`
`text`	`text(text: String, at: Point)`
`ellipse`	`ellipse(center, rx, ry, rotation = 0, segments = 64)`
`arc`	`arc(center, rx, ry, startDegrees, endDegrees, rotation = 0, segments = 48)`
`sector`	`sector(center, rx, ry, startDegrees, endDegrees, rotation = 0, segments = 48)`
`curve`	`curve(p0, c0, c1, p1, segments = 32)`
`quadraticCurve`	`quadraticCurve(p0, control, p1, segments = 24)`
`regularPolygon`	`regularPolygon(center, sides, radius, rotation = 0)`
`star`	`star(center, points, outer, inner, rotation = 0)`
`dot`	`dot(at: Point, radius: Double = 3)`

`marker`	`marker(at: Point, color: Color, radius: Double = 3)`
`cross`	`cross(at: Point, size: Double = 5)`
`arrow`	`arrow(from, to, headLength = 12, headAngle = 28)`
`label`	`label(text, at, textColor = White, background = Black, padding = 4, fontScale = 0.5, font = Simplex)`
`empty`	`val empty: Picture`
`all`	`all(pictures: Seq[Picture])`
`grid`	`grid(pictures: Seq[Picture], columns: Int, gap: Double = 8)`

Underneath there are exactly four drawing primitives — a circle, a quad, a path and a run of text — and everything else in that table is built from them. `ellipse`, `arc`, `sector`, `roundedRectangle`, `curve`, `quadraticCurve`, `regularPolygon` and `star` all reduce to a path with enough points to look smooth, which is why they fill, dash and transform with exactly the same rules as a hand-written `polygon`. `segments` is your control over that trade: drop it for a hundred small markers per frame, raise it for a full-screen arc.

`regularPolygon` requires at least three sides, `star` at least two points and `grid` at least one column; each of those is a `require`, so a nonsensical value throws where you wrote it rather than producing a degenerate shape three layers down.

> **NOTE**
> Coordinates are image pixels: the origin is the top-left corner and `y` increases downward, the same convention as every `Rect` a detector hands you. A detection box therefore needs no conversion — `Picture.rectangle(part.box)` is already in the right space. The cost of that choice is that positive rotation angles turn clockwise, which is the opposite of the mathematical convention and the same as everything else in OpenCV.

Styling, and the rule that surprises people

Styling methods return a new picture wrapping the old one, carrying an immutable `StylePatch` that adjusts the style in effect. An unstyled picture is drawn with a white one-pixel outline, no fill, `Font.Simplex` at a font scale of `0.5`, and antialiasing on.

Table 16-2. The styling combinators. Each returns a new `Picture`; none mutates.

Method	Effect
`strokeColor(c)`	Outline colour
`strokeWidth(w)`	Outline width, clamped to at least 1
`stroke(c, w)`	Both, in one call (`w` defaults to 1)
`noStroke`	No outline at all
`fillColor(c)`	Fill colour — closed shapes only
`noFill`	No fill

`strokeDash(d)`	A dash pattern
`dashed` / `dotted`	`Dash(10, 8)` / `Dash(1, 6)`
`solidStroke`	Clears an inherited dash
`font(f)` / `fontScale(s)`	Text font and size
`smooth(on)`	Antialiasing, on by default

A style set on a group is a default that its members inherit unless they set their own. That is the behaviour you want — `Picture.all(everything).strokeColor(Color.Green)` colours a whole overlay, and any member that already asked for red keeps it — and it follows from one rule about how the tree is walked: styles are applied from the outside in, so **the setting nearest the primitive wins**.

The consequence catches everyone once. Chained styling calls are not last-write-wins:

```
val a = Picture.circle(c, 10).strokeColor(Color.Red).strokeColor(Color.Blue)  //
draws RED
```

`.strokeColor(Color.Red)` produced the inner node and `.strokeColor(Color.Blue)` wrapped it, so red is applied last and blue never survives. The same rule explains why `Picture.dot` — which is defined as a circle with a white fill and no stroke — ignores a fill you set afterwards:

Example 16-5. Wrong, then right: restyling a picture that already styled itself.

```
Picture.dot(p).fillColor(Color.Red)          // still white: dot's own fill is
nearer the leaf
Picture.marker(p, Color.Red)                 // right --- marker takes the colour
as an argument
Picture.circle(p, 3).fillColor(Color.Red).noStroke   // or style a bare primitive
yourself
```

Read `solidStroke` the same way: it clears a dash inherited from an enclosing group, not one already set closer to the shape.

Dashed strokes

OpenCV has no dashed line. `Imgproc.line` draws solid, and that is the whole of it. `Picture` supplies dashes by walking each segment of a path and emitting one short `line` call per dash, so `Dash(on, off)` alternates run lengths in pixels along the path — both must be positive, which `Dash` enforces with a `require`.

Table 16-3. The three dash presets.

Preset	Pattern	Reads as
`Dash.dashed`	`Dash(10, 8)`	Long dashes
`Dash.dense`	`Dash(4, 4)`	Tight dashes

`Dash.dotted`	`Dash(1, 6)`	Dots

A dashed circle is the one place the representation shows through: with no dash a circle goes to `Imgproc.circle`, and with a dash it is redrawn as a 48-sided polygon so the dashes can be spaced along it. At any radius you would put on a frame the difference is under a pixel.

Transforms and layout

`translate(dx, dy)`, `at(point)`, `rotate(degrees, about = Point(0, 0))` and `scale(factor, about = Point(0, 0))` each wrap a picture in an affine transform. Unlike styles, transforms chain in reading order: `p.rotate(30, c).translate(10, 0)` rotates and then translates, because composition accumulates as the tree is walked and the outermost transform ends up applied last.

The transform reaches everything a shape is made of. A rotated rectangle is genuinely turned, because its four corners are transformed as points. `scale` also scales circle radii and text, through a uniform scale factor derived from the transform's determinant — which is what makes it possible to author an overlay once at full-frame coordinates and render it into a preview:

Example 16-6. One overlay, two resolutions, from the same value.

```
Image.reading("belt.jpg") { frame =>
  val small = frame.copy.scale(0.5).draw(overlay.scale(0.5)).write("belt-
preview.png")
  val full = frame.draw(overlay).write("belt-full.png")
  small.flatMap(_ => full)
}
```

Read the ownership as carefully as the geometry. `frame.copy` clones the pixels so the original survives; `scale`, `draw` and `write` then consume that clone in turn, and `write` releases it. Only after the preview is on disk does `frame` itself get spent. Running both writes to completion before combining the two `Either`s is deliberate: a `for` comprehension would short-circuit on the first `Left` and leave the second canvas unwritten and unreleased.

> **WARNING**
> A transform moves text but does not turn it. OpenCV's `putText` has no rotation parameter, so a rotated `Picture.text` is placed at the rotated position and drawn upright. Scale is honoured — the renderer multiplies the style's `fontScale` by the transform's scale factor — and rotation is not. `bounds`, meanwhile, transforms the measured box like any other geometry, so under a rotation it reports a turned rectangle the renderer will never draw, and laying out rotated text with `beside` or `above` will not line up. If you need turned lettering, you are drawing it as paths.

On top of transforms sit three layout combinators that place pictures relative to one another by measuring them, so you never compute an offset by hand.

Table 16-4. Composition and layout.

Combinator	Meaning
`a.on(b)` / `a.under(b)`	Overlay `a` over / under `b`
`Picture.all(seq)`	Group a sequence, the first element at the bottom
`a.beside(b, gap = 8)`	Place `b` to the right of `a`, vertical centres aligned
`a.above(b, gap = 8)`	Place `b` below `a`, horizontal centres aligned
`Picture.grid(seq, columns, gap = 8)`	Row-major grid, every cell sized to the largest picture
`a.bounds`	The axis-aligned extent, as `Option[Bounds]`

`beside` and `above` measure both boxes and place the second relative to the first. If either picture measures nothing they degrade rather than throw: laying an empty path beside a square gives back the square. That matters more than it sounds, because an empty path is the ordinary result of filtering a set of detections down to none. `grid` differs deliberately: it places by index, so a measureless picture keeps its cell and the rest of the grid stays aligned.

`Picture.empty` is the identity for `on`, and it measures as `None`, so every layout combinator passes it straight through. That makes it the seed a fold wants:

Example 16-7. The legend: a swatch beside a name, stacked, with no offsets computed by hand.

```
val palette: Seq[Color] = Color.wheel(classes.size)

def swatch(name: String, color: Color): Picture =
  Picture
    .rectangle(Rect(0, 0, 14, 14))
    .fillColor(color)
    .noStroke
    .beside(Picture.text(name, Point(0,
11)).strokeColor(Color.White).fontScale(0.45), gap = 6)

val legend =
  classes
    .zip(palette)
    .map((name, color) => swatch(name, color))
    .foldLeft(Picture.empty)(_.above(_, gap = 6))
    .translate(12, 12)
```

The whole overlay is then one expression, and the connectors that link consecutive parts are three lines of it:

Example 16-8. The finished overlay.

```
def centre(r: Rect): Point = Point(r.x + r.width / 2.0, r.y + r.height / 2.0)

val connectors = Picture.all(
  parts.sliding(2).collect { case Seq(a, b) =>
    Picture.arrow(centre(a.box), centre(b.box)).stroke(Color.White.fadeOut(0.4),
1)
  }.toSeq
)

val overlay =
  legend
    .on(Picture.all(parts.zip(palette).map((p, c) => annotate(p, c))))
    .on(connectors)
```

Doodle, adapted rather than copied

The picture-as-a-value idea, the `on` composition, `strokeColor` / `fillColor` / `strokeDash`, and the `beside` / `above` layout all come from Doodle, the Creative Scala functional-graphics library. What changed is everything that touches the domain. Doodle's plane is centred and `y`-up; this one is image pixels, top-left origin, `y` down, so a detector's `Rect` drops straight in. Doodle renders through Java2D or a canvas; this renders through `Imgproc` onto a `Mat` you already own. And three things are here that a general graphics backend gives you for free and OpenCV does not: dashed strokes, per-shape alpha compositing, and text measured by its real font metrics — most of what annotating computer-vision output actually needs.

Colour: RGBA here, BGR underneath

`Color` is the graphics layer's palette: a case class of `red`, `green`, `blue` and `alpha`, each an `Int` in `[0, 255]`, with `alpha` defaulting to `255` and a `require` that rejects anything outside the range. It is a different type from core's `Scalar` on purpose, and the two differ in both channel order and meaning.

Table 16-5. The two colour types, and which layer uses which.

	`Scalar` (core)	`Color` (graphs)
Channel order	BGR — `Scalar.Red` is `Scalar(0, 0, 255)`	RGB — `Color.Red` is `Color(220, 40, 40)`
Alpha	A fourth channel the drawing verbs ignore	Honoured: real per-shape blending
Range	`Double`, unchecked	`Int` in `[0, 255]`, checked
Used by	`drawRect`, `drawText`, `Image.blank`, `org.opencv.*`	`Picture` styling, `Chart`, `Animation`
Bridge	`scalar.toColor`	`color.toBgrScalar`

Construction goes through `Color.rgb(r, g, b)`, `Color.rgba(r, g, b, a)`, `Color.gray(v)`, the case class directly, or `Color.hsl(hue, saturation, lightness, alpha = 255)` — hue in degrees, wrapped into a turn, saturation and lightness in `[0, 1]`, clamped. The alpha is not clamped: it goes straight into the case class, so it still has to be a legal channel value. Fifteen named constants cover the usual overlay work, from `Color.Transparent` through `Color.Black`, `Color.White`, three greys and the nine hues.

`lighten(a)` and `darken(a)` blend toward white and black; `blend(other, a)` mixes; `withAlpha(a)` sets alpha outright and `fadeOut(a)` reduces it by a fraction; and `spin(degrees)`, `complement`, `saturate(a)` and `desaturate(a)` round-trip through `hsl`, which is also readable directly as a `(hue, saturation, lightness)` tuple. `desaturate(1)` is grey.

Two generators produce palettes, and choosing between them is a statement about your data:

- `Color.wheel(n, saturation = 0.65, lightness = 0.55)` spaces `n` hues evenly around the wheel. Categorical: use it for classes, tracks, series — things with no order.
- `Color.ramp(from, to, n)` blends `n` steps between two colours. Sequential: use it for a heat scale, a confidence gradient, anything ordered.

`Color.categorical` is `wheel(8)`, kept around as a sensible default. Both generators accept `n = 0` and refuse a negative one.

The bridge is lossy in exactly one way, and it is documented rather than hidden: `color.toBgrScalar` reorders to BGR and **drops the alpha**, because a `Scalar`'s fourth channel is not an alpha the OpenCV drawing verbs honour. If you want translucency baked into a colour you are handing to `drawRect`, pre-blend it with `blend` against the background you expect. Going the other way, `scalar.toColor` rounds and clamps each channel into `[0, 255]` and returns a fully opaque colour, so `Scalar(-10.0, 127.6, 300.0).toColor` is `Color(255, 128, 0)`.

Compositing onto a frame

Three calls turn a `Picture` into pixels, and they differ only in where the pixels come from.

Table 16-6. The three ways to rasterise, and what each one does with ownership.

Call	Canvas	Ownership
`picture.render(w, h, background = Color.Black)`	A fresh `w` × `h` image	Returns an `Image` you own and must close
`picture.renderOn(image)`	An existing image	Consumes `image`, returns the annotated one
`image.draw(picture)`	The same thing, from the image side	Consumes `image`, returns the annotated one

`image.draw(picture)` is the one you will write. It is an extension method on `Image`, defined in the graphs module so that `Image` itself carries no dependency on the graphics layer, and it does no copying: it takes the image's handle, renders the picture into the `Mat` in place, and

rewraps the same buffer as a new `Image`. That is why it consumes its receiver — exactly like `blur`, `resize` or any other transform in Chapter 4, and for the same reason.

> **NATIVE MEMORY**
> A `Picture` owns no native memory. It is JVM data all the way down: case classes, a `Seq[Point]` or two, and some functions. Hold one for the life of the process, share it across threads, hand it to a hundred frames — there is nothing in it to leak and nothing in it to close. What it is not is load-free: `bounds` measures text through `Draw.textSize`, and `Picture.label` calls that at construction time, so any picture that involves a string still wants `OpenCv.load()` to have run. Shapes on their own do not. The native object in this chapter is the `Image`, and it obeys the rules Chapters 4 and 5 set out — `draw` and `renderOn` spend it, `render` hands you a new one that nothing will close for you, and the terminals (`write`, `bytes`, `close`) release. `frame.copy` before `draw` is how you keep the original, and it costs a full clone of the pixels.

Alpha is the one place the renderer needs a copy of your pixels. OpenCV's drawing functions are opaque — they overwrite what they touch and have no notion of blending — so to honour a translucent `Color`, `Picture` computes a conservative bounding box for the shape (stroke half-width, round joins and antialiasing spread included, clipped to the image), saves that region, paints the shape opaquely, and blends the region back toward what it covered. The arithmetic is identical to cloning and blending the whole frame, but a small badge on a 4K frame costs a badge-sized clone instead of a frame-sized one. Both temporaries go through `Managed.use`, so they are released even if the draw throws.

> **TIP**
> Antialiasing is on by default and is what makes an overlay look drawn rather than stamped. Turn it off with `smooth(false)` when you need exact pixel values — a mask you are going to threshold, or a test that asserts a specific channel value at a specific coordinate.

Testing a picture without rendering it

Because a picture is data, most of what you want to assert about an overlay needs no image, no canvas and no rendered pixel. The layout is the part that actually breaks, and the layout is what `bounds` reports. Keep one honest caveat in view: measuring text is a call into `getTextSize`, so a suite that lays out a label still opens with `OpenCv.load()` in `beforeAll`, exactly as the library's own graphics suites do. What it does not need is a canvas, an encoder, or a fixture image on disk.

Example 16-9. Layout assertions that never touch a pixel.

```
test("the legend stacks one row per class"):
  val legend = buildLegend(Seq("bolt", "nut", "washer"))
  val b = legend.bounds.get
  assert(b.height > 3 * 14, "three 14-pixel swatches plus the gaps")
```

```
test("beside places the second shape to the right of the first"):
  val a = Picture.rectangle(Rect(0, 0, 20, 20))
  val b = Picture.rectangle(Rect(0, 0, 20, 20))
  assertEquals(a.beside(b, gap = 10).bounds.get.width, 50.0)   // 20 + 10 + 20

test("a frame with no detections annotates to nothing"):
  assertEquals(buildOverlay(Seq.empty).bounds, None)
```

That last one is the assertion an imperative overlay cannot express at all. “Draws nothing” is a property of a value; it is not a property of a sequence of calls that happened not to execute.

Styling is immutable data (`StylePatch`), not an opaque closure. Identically constructed shapes with identical style patches can be structurally equal. Structural equality is still not visual equivalence: different scene trees can paint the same pixels. Assert on bounds for geometry, recorded commands for painter order, and rendered pixels for colour and glyph visibility.

When the question genuinely is “what colour is that pixel”, render a small canvas and read it back. `smooth(false)` makes the answer exact, and the image is yours to close:

```
val img = Picture.circle(Point(50, 50),
20).fillColor(Color.Red).noStroke.render(100, 100, Color.Black)
try assert(img.mat.get(50, 50)(2) > 180, "the centre should be red")
finally img.close()
```

Two conventions bite in that one line. `Mat.get` takes `(row, column)` — `y` first, then `x` , which is why the library’s own graphics suites define a `px(img, x, y) = img.mat.get(y, x)` helper rather than calling `get` directly. And the array it returns is in the `Mat` ’s BGR order, so red is index `2` while `Color.Red` names it first. That is the `Color` / `Scalar` distinction showing up in a test.

When to stay with the drawing verbs

None of this deprecates Chapter 14. `Picture` bottoms out in the same `Imgproc` calls those verbs make, and it costs a tree walk, some `Option` handling and a second artifact on the classpath. When the overlay is two rectangles and a caption on a frame you are about to look at once, the verbs are shorter:

```
Image.reading("debug.png")(_.drawRect(roi, Scalar.Green).drawText("roi",
roi.topLeft).write("out.png"))
```

Reach for a `Picture` when the overlay has structure — when something is composed from parts, laid out relative to something else, rendered at more than one size, reused in more than one place, or worth a test. Reach for a drawing verb when you want to see one thing, now. Both arrive through the same `import` , and mixing them within one program is normal.

Next

The `Picture` algebra was designed for annotation, but nothing in it knows what it is drawing. A bar chart is a stack of rectangles laid out with the same combinators; an animation is a picture valued by frame number. Chapter 17, *Charts*, takes the value you have been building here and points it at data: `Chart.bars`, `line`, `area`, `scatter`, `pie` and `histogram`, six functions that each hand back a `Picture` sized to a `width` × `height` box, so a plot drops into the corner of a frame with `chart.at(Point(x, y))` and needs no new vocabulary at all. Chapter 18, *Animation and GIF*, points it at time instead — `Animation.record` to a video, `Animation.gif` to the format nobody has to install anything to watch, `Animation.frames` for a `Seq[Image]` whose lifetimes are yours to get right, and `Animation.foreach` for the streaming form that closes each canvas for you.

CHAPTER 17

Charts

Turning a column of numbers into pixels, on a machine with no screen.

A vision service produces two kinds of output. The first is an image — a frame with boxes on it, a mask, a crop — and the library has spent sixteen chapters on that one. The second is a number that changes over time: faces per frame, motion energy per second, the length of a queue, the confidence the model had before it stopped being sure. That output has nowhere to go. It is usually the more interesting of the two, and it usually ends its life as a column of doubles in a log that nobody opens.

The obvious fix is to plot it, and the obvious fix is where the trouble starts. The machine doing the work is a container: no windowing system, no X socket, no fonts beyond whatever the base image happened to ship. A plotting library that reaches for AWT wants a headless flag and a font path and will still fail differently on the build agent than on your laptop. One that renders in a browser wants a browser. Adding either to a deployment whose whole selling point is that `OpenCv.load()` needs no `apt-get` is a poor trade for six bars and a line.

The process is already holding a renderer. It draws antialiased polylines, filled polygons and text onto a buffer of pixels, it needs no display to do it, and Chapter 16 wrapped it in a compositional value type. A bar chart is a sequence of filled rectangles. A line chart is a polyline. Once `Picture` exists, charts are not a new capability — they are an arrangement of one you have.

`Chart` is that observation, written down. Six functions, each taking numbers and a box and returning a `Picture`. Nothing else: no window, no canvas object, no plotting session to configure. This chapter uses one running job to show the shape of it — count the faces in every frame of a thirty-second clip, plot the count against time, and burn the plot into a corner of the last frame so the whole answer travels as a single PNG in a chat message.

Six factories, one return type

`Chart` lives in the `scalacv-graphs` module alongside `Picture`, `Color` and `Animation`; the same `import scalacv.*` Chapter 16 already relied on brings it in. The running example needs a second optional artifact — `com.worxbend::scalacv-vision:0.4.2`, for the face detector — while the video reader itself is core. Neither has had its own chapter yet: Chapter 19 covers `Video` properly and Chapter 24 the detector, and what is used here is the minimum of each.

Every factory validates its box the same way — a non-positive `width` or `height` throws `IllegalArgumentException` with the message `a chart needs a positive box, got 0x80`, be-

cause a zero-width box does not produce an empty chart, it produces degenerate `Rect`s and a division by a zero span.

Table 17-1. The whole `Chart` API. Every one returns a `Picture` sized to its `width`×`height` box, origin top-left.

Factory	Signature
`bars`	`bars(values: Seq[Double], width: Int, height: Int, color: Color = Color.Blue, gap: Int = 4)`
`line`	`line(values: Seq[Double], width: Int, height: Int, color: Color = Color.Green, strokeWidth: Int = 2)`
`area`	`area(values: Seq[Double], width: Int, height: Int, color: Color = Color.Blue, strokeWidth: Int = 2)`
`scatter`	`scatter(points: Seq[(Double, Double)], width: Int, height: Int, color: Color = Color.Red, radius: Double = 3)`
`pie`	`pie(values: Seq[Double], width: Int, height: Int, palette: Seq[Color] = Color.categorical)`
`histogram`	`histogram(data: Seq[Double], bins: Int, width: Int, height: Int, color: Color = Color.Purple)`

Degenerate **data** is not an error, and this is the line worth remembering: a bad box throws, bad numbers draw nothing. `bars` and `scatter` return `Picture.empty` for an empty sequence; `line` and `area` return it for anything shorter than two points, because the divisor that spreads points across the width is `values.size - 1`; `pie` returns it when the magnitudes sum to zero or the palette is empty; `histogram` returns it for empty `data`, and additionally **throws** when `bins` is less than one — a check made before the box is checked. `Picture.empty` composites onto a frame as nothing at all, which is the right behaviour for a chart driven by a stream that has not produced any data yet.

The scaling is worth knowing exactly, because it is the source of every surprise in this chapter. `bars`, `line` and `area` divide by `values.map(math.abs).max.max(1e-9)`, so the largest magnitude in the series lands at the top of the box and everything else is a fraction of it. That floor of `1e-9` is the reason an all-zero series is a flat line along the baseline rather than a `NaN` polyline that OpenCV would refuse. There is a two-pixel inset — the tallest point sits at `y = 2`, not `y = 0` — so a stroke at the peak stays inside the box instead of being clipped by its own edge. `scatter` is the only one that maps a genuine range: it takes the minimum and maximum of both axes and fits them into the box, inset by `radius` so the extreme markers are whole; an axis with no spread at all — every `x` identical — is centred rather than divided by zero. `pie` fits a circle of radius `math.min(width, height) / 2 - 2` into the box, starts its first slice at `-90` degrees — twelve o'clock — and cycles the palette with `i % palette.size` when there are more slices than colours. `histogram` bins `data` into `bins` equal-width buckets across `[min, max]` and then hands the counts to `bars` with `gap = 1`.

Counting faces across a clip

The measurement comes first, and it has nothing to do with charting. Open the video, run the detector on every frame, keep the count, and keep the last frame. `Video.framesCopied` — Chapter 5's `Managed`, applied to video — is the right traversal here: it clones each frame into a caller-owned `Managed[Mat]` as you pull it, and it clones as you pull rather than up front, which is exactly what "I want to keep the last one" requires. The plain `Video.frames` hands you one Mat reused for every frame; keeping a reference to it would keep nothing.

Example 17-1. One pass over the clip: a count per frame, and the final frame kept.

```
import scalacv.*
import org.opencv.core.Mat
import org.opencv.objdetect.FaceDetectorYN

def timeline(path: String, detector: FaceDetectorYN): Either[CvError,
(Seq[Double], Option[Image])] =
  Video.open(path).map { capture =>
    capture.use { cap =>
      Video.framesCopied(cap) { frames =>
        var last: Option[Managed[Mat]] = None
        val counts = frames.map { owned =>
          last.foreach(_.release())      // the previous frame is no longer the
last one
          last = Some(owned)
          FaceDetect.detect(detector, owned.get).size.toDouble
        }.toVector                       // forces the traversal, in order, once
        (counts, last.map(Image.wrap))
      }
    }
  }
```

> **NATIVE MEMORY**
> Three lifetimes are in play in that loop and only one of them is automatic. `capture.use` releases the `VideoCapture`. Each frame `framesCopied` hands you is yours: the loop releases the previous one before adopting a new one, so exactly one clone is alive at a time regardless of how long the clip is, and the survivor becomes an `Image` the caller must close. The detector is the third. `FaceDetectorYN` is one of the 185 `org.opencv.*` types with no public `release()` at all; scalacv frees it anyway through `Releasable.nativeHandle`, which is why it reaches you from `FaceDetect.create` as a `Managed[FaceDetectorYN]` and why letting the raw handle escape that `Managed` costs you the model's weights for the life of the process. `FaceDetect.detect` also **mutates** the detector — it sets the input size to each frame's size — so one detector belongs to one thread.

Thirty seconds at thirty frames per second gives 900 counts. The first instinct is a bar per frame.

```
Chart.bars(counts, 240, 56)     // 900 bars in a 240-pixel box
```

What renders is the first forty-eight bars and then nothing. `bars` computes `barWidth` as `math.max(1, (width - gap * (n + 1)) / n)`, and at 900 values that expression is negative, so the clamp fires and every bar is one pixel wide. The layout then places bar `i` at `gap + i * (barWidth + gap)` — `4 + 5i` here — which for the forty-ninth bar is `x = 244`, already past the right edge. OpenCV clips it, silently, and you are looking at the first second and a half of the clip. The rule is arithmetic: a box holds about `width / (gap + 1)` bars — 48 in a 240-pixel box at the default gap of 4 — and a bar chart is the wrong instrument above that.

Two right answers. Downsample into as many buckets as the box can hold, or use the series charts, whose x-coordinate is `i / (n - 1) * width` and therefore accommodates any length by putting several samples on the same pixel column.

Example 17-2. A second-by-second bar chart, and a per-frame trace, from the same counts.

```
val perSecond = counts.grouped(30).map(g => g.sum / g.size).toVector  // 30
buckets
val bars      = Chart.bars(perSecond, 240, 56, Color.Cyan)

val trace = Chart.line(counts, 240, 56, Color.Cyan)
  .on(Chart.area(counts, 240, 56, Color.Cyan))   // the fill under its own
outline
```

Thirty buckets fit: `(240 - 4 × 31) / 30` is a bar three pixels wide, and the last of them ends at `x = 210`, comfortably inside the box. `area` is `line` with the polyline closed down to the baseline and filled at `color.fadeOut(0.7)`, drawn over its own outline. The two agree on every vertex by construction, because both are built from one private `seriesPoints` definition rather than from two that could drift apart. Overlaying a fresh `line` on top, as above, restores the outline to full strength through the faded fill, which reads better over a photograph than either chart alone.

A chart is a picture, so it composites like one

Nothing above has touched a pixel. `Chart.area(counts, 240, 56, Color.Cyan)` is a tree of case classes; it has the same `at`, `on`, `translate`, `scale` and `bounds` as any other `Picture`, and it turns into pixels only where Chapter 16 said it would — at `render`, which allocates a fresh canvas, or at `image.draw`, which consumes an existing image and returns the annotated one.

Example 17-3. The plot, a backing panel, and a title tag, burned into the last frame.

```
// detector: Managed[FaceDetectorYN], from FaceDetect.create(modelPath, Size(320,
320))
timeline("lobby.mp4", detector.get).foreach { (counts, lastFrame) =>
  lastFrame.foreach { frame =>
    val panel = Rect(12, frame.height - 108, 264, 96)  // 12 px of margin around
the 240x56 plot
    val plot  = Chart.line(counts, 240, 56, Color.Cyan)
      .on(Chart.area(counts, 240, 56, Color.Cyan))
```

```
      .at(Point(panel.x + 12, panel.y + 32))

    val overlay = Picture.all(Seq(
      Picture.roundedRectangle(panel, radius =
6).fillColor(Color.Black.withAlpha(150)).noStroke,
      plot,
      Picture.text(f"faces/frame, peak ${counts.max}%.0f", Point(panel.x + 12,
panel.y + 22))
        .strokeColor(Color.Black)
        .fontScale(0.45)
        .on(Picture.rectangle(Rect(panel.x + 8, panel.y + 6, 200, 22))
              .fillColor(Color.Cyan)
              .noStroke)
    ))

    frame.draw(overlay).write("lobby-summary.png")   // draw consumes; write
releases
  }
}
```

`Picture.all` groups a sequence with the first element at the bottom, so the panel paints, the plot paints over it, and the title tag paints over both. That tag is text composed **onto** its own filled box with `.on`, rather than the ready-made `Picture.label`, for the reason Chapter 16's warning gives: `label` composes with `under`, which puts the fill on top, so an opaque background paints out the text it was meant to sit behind. The panel is the piece that is easy to skip and expensive to skip: a chart drawn straight onto a frame has whatever contrast the frame happens to give it, which for a cyan trace over a sunlit floor is none. One rounded rectangle at `Color.Black.withAlpha(150)` buys a known background — and because the `Picture` layer honours alpha per shape (OpenCV's own drawing verbs do not), the frame still shows through it.

> **NATIVE MEMORY**
> The chart layer allocates no native memory. A `Picture` is heap data; build a hundred of them per frame and OpenCV never hears about it. Native cost appears at exactly two boundaries. `picture.render(w, h, background)` allocates a fresh `Mat` and returns an `Image` you own — the call for a chart in a report, where there is no frame to draw onto. `image.draw(picture)` consumes its receiver and returns the annotated image. Above, `write` is the terminal that releases it.

For the report case — a chart as a file in its own right, no video involved — the same picture goes through `render` instead:

```
val latencies: Seq[Double] = ???            // one detector call, in
milliseconds, per frame

Chart.histogram(latencies, bins = 24, width = 480, height = 160)
```

```
  .render(480, 160, Color.White)
  .write("latency.png")
```

What Chart does not draw

This is a chart layer for overlays and reports, sized to that job, and it is worth being blunt about the boundary rather than discovering it at 2 a.m. There are no axes. No tick marks, no tick values, no gridlines, no title, no legend, no units. There is no way to pin a range: every series is normalised to its own largest magnitude, always, and there is no `min` / `max` parameter to say otherwise. There is no log scale, no stacking, no grouped bars, and no multi-series call.

Table 17-2. The missing pieces, and where each one actually comes from.

Not in `Chart`	Build it with
Axis lines	`Picture.line` along the two edges of the box
Tick values, titles, units	`Picture.text`, or a text-over-box tag built with `.on` (Chapter 16) for a readable panel behind it
Gridlines	`Picture.line` per gridline, `.dotted`, at `Color.LightGray.fadeOut(...)`
A fixed y-range	Nothing — normalise the data yourself and draw the polyline directly
Two series on one scale	`Picture.polyline`, with a peak you choose (below)
Values below a baseline	`scatter`, or shift the data positive and draw your own zero line

The multi-series limit is the one that misleads. Two `Chart.line` calls over the same box compose perfectly — `a.on(b)` — and each has been scaled to its **own** peak, so a series that never exceeds 3 and one that reaches 300 are drawn to the identical height. The chart looks like a comparison and is not one, and dividing the data first does not help: the normalisation is recomputed from whatever you pass in. When two series must share a scale, drop one level and write the mapping yourself — four lines of the same `Picture` vocabulary, which is all `Chart` does internally:

Example 17-4. A series on a scale you control, rather than on its own.

```
def series(values: Seq[Double], peak: Double, width: Int, height: Int): Picture =
  Picture.polyline(values.zipWithIndex.map { (v, i) =>
    Point(i.toDouble / (values.size - 1) * width, height - v / peak * (height -
2))
  })

val faces, bodies: Seq[Double] = ???             // two counts from the same pass
over the clip
val peak = (faces ++ bodies).max                 // one peak for both
val both = series(faces,  peak, 240, 56).strokeColor(Color.Cyan).strokeWidth(2)
  .on(series(bodies, peak, 240, 56).strokeColor(Color.Orange).strokeWidth(2))
```

The `height - 2` is the same two-pixel inset the built-in charts use; keeping it means your hand-rolled series lines up with a `Chart.area` fill drawn beside it.

The magnitude fold

One more consequence of that normalisation deserves its own warning, because it produces a chart that is entirely plausible and entirely wrong. `bars`, `line` and `area` plot `math.abs(v)`. Face counts are never negative, so the running example never notices — but the moment you chart the **change** in the count, the data is signed:

```
val delta = counts.zip(counts.drop(1)).map((a, b) => b - a)
Chart.line(delta, 240, 56)     // draws |b - a|: every departure looks like an
arrival
```

Nothing throws. Nothing is clipped. The series is folded upwards about zero, and a frame where two people left renders identically to one where two arrived. `pie` rectifies the same way, treating a negative value as a positive share of the total. Shift signed data into the positive range yourself and draw the baseline where zero landed:

Example 17-5. Signed data, plotted honestly.

```
val lo    = math.min(0.0, delta.min)           // the shift floor, never above
zero
val above = delta.map(_ - lo)                  // now every value is >= 0
val peak  = above.max.max(1e-9)                // the floor Chart applies
internally, applied here
val zeroY = 56 - (-lo) / peak * (56 - 2)       // where the original zero ends up

val plot = Chart.line(above, 240, 56, Color.Yellow)
  .on(Picture.line(Point(0, zeroY), Point(240,
zeroY)).strokeColor(Color.LightGray).dotted)
```

`scatter` is the exception and the escape hatch: it maps the true `(x, y)` range into the box, so negatives land where they belong. `histogram` is unaffected — it bins the raw values over their own `[min, max]` range, so negative **data** falls in the right bucket, and the counts it then charts are never negative.

Design that survives the medium

A chart in a report is read at arm's length on a bright screen. A chart burned into the corner of a 1080p frame is read after an encoder has been through it, on a phone, at half size, in a chat client. The second case is the unforgiving one.

Measure the type, do not guess it. Text is drawn with OpenCV's Hershey vector fonts at `fontScale`, whose default is `0.5`. What that means in pixels depends on the font and the string, and `Draw.textSize` will tell you rather than leaving it to a trial render:

```
Draw.textSize("faces/frame", Font.Simplex, 0.5).size.height   // the glyph
height, in pixels
```

`Draw.textSize`'s own `scale` defaults to `1.0`, not to the `Picture` style's `0.5`, so pass the scale you are actually drawing at — the two defaults differ, and a measurement taken at the wrong one is off by a factor of two. The `baseline` field beside `size` is the descender depth; a box drawn `size.height` tall clips every `g` and `y`, which is the arithmetic a `Picture`'s own `bounds` already does for you — it measures text as `size.height` above the baseline and `baseline` below.

Two consequences follow. Hershey glyphs are strokes, not filled outlines, so a small `fontScale` at the default `strokeWidth` of 1 produces one-pixel-wide letters — and a one-pixel line is the first thing a lossy encoder discards. There is no thinner setting to blame: `strokeWidth` clamps its argument with `math.max(1, width)`, so 1 is the floor, and the only remedies are a heavier stroke or a larger scale. If you must go small, thicken before you shrink. And the Hershey fonts are ASCII: a non-ASCII character is drawn as `?`, so no degree sign, no micro sign, no en dash in an axis label. Write `deg` and `us` and be understood.

Label directly; do not spend a legend. A legend costs a swatch, a gap and a word per series. In a 200-pixel-wide overlay that is a third of the chart given over to a lookup table, and it puts the name somewhere the eye has to travel to and back. Put the name at the end of its own line instead, in its own colour:

```
val last   = Point(240, 56 - faces.last / peak * (56 - 2))   // the series'
right-hand end
val glyphs = Picture.text("faces", last).strokeColor(Color.Black).fontScale(0.4)
val tag = glyphs.bounds.fold(glyphs) { b =>
  glyphs.on(
    Picture
      .rectangle(Rect((b.minX - 2).round.toInt, (b.minY - 2).round.toInt,
                      (b.width + 4).round.toInt, (b.height + 4).round.toInt))
      .fillColor(Color.Cyan)
      .noStroke
  )
}
```

`bounds` measures the string with the style's own font and scale, descenders included, and sizing a filled box from that measurement is what makes a tag readable over a frame where bare `Picture.text` would land white-on-white half the time. Build it as `glyphs.on(box)` rather than reaching for `Picture.label`: as Chapter 16 warns, `label` composes the box with `under`, so an opaque background is drawn last and hides the text. `Picture.text` anchors on the glyph baseline, so a tag anchored at `x = width` hangs off the right-hand end of the plot: leave it that room in the panel, or subtract the measured width and put the tag inside.

What a codec does to a thin red line

Video compression does not treat colour the way it treats brightness. Every codec you are likely to write through — MJPEG, H.264, VP9 — keeps luminance at full resolution and subsamples chrominance, usually to half in each direction, then quantises the colour planes harder than the luma plane. A thin line whose only separation from its background is **hue** is therefore attacking the exact channel the encoder has decided matters least: red on green at matched brightness smears into a brown smudge at any bitrate you would actually ship.

The fix is to separate by lightness, not by hue alone. `Color` gives you the number: `color.hsl` returns `(hue, saturation, lightness)`, and the third component is the one that survives. `Color.Yellow` against `Color.Black.withAlpha(150)` is a lightness contrast that reads at any bitrate; `Color.Red` against `Color.Purple` is a hue contrast that reads on your monitor and nowhere else. `Color.wheel(n)` spaces hues evenly and holds lightness fixed at `0.55` — excellent for categorical labels in a report, indifferent for lines that have to survive an encoder. For that case, pick from the named colours by brightness, or build a `Color.ramp` from a dark colour to a light one and let the scale carry the meaning.

Where this goes next

Six functions, a `Picture` out of each, and the rest is the composition vocabulary you already had. That is the whole of the chart layer, and its limits are real: no axes, no ranges, one scale per series, magnitudes only. What it buys is a plot rendered by the same engine that rendered the frame, in the same process, with no display server, no dependency beyond the graphics module Chapter 16 already added, and no native memory of its own.

The counts chart in the corner of one still frame is a summary. The obvious next question is what it looks like as the clip plays — a trace that grows a point per frame, written back out as a video or a shareable GIF. Chapter 18, *Animation and GIF*, is a `Picture` valued by frame number, and it is the last piece of the graphics module.

CHAPTER 18

Animation and GIF

Frames over time, and the one output format nobody has to install anything to watch.

A still image answers a question about a moment. Most of the questions worth asking in this library are about a stretch of time instead: does the tracker keep its identity when two people cross, does the mask flicker when a cloud goes over, does the threshold you tuned on frame 40 still hold at frame 400. None of those can be settled by a PNG, and none of them can be settled by a number either — a reviewer who reads "mean IoU 0.81" learns less in ten seconds than one who watches the box jump off the object and come back.

So you want to hand someone a moving picture. The obstacles are dull and real. A video needs a codec the recipient's player links, and Chapter 2's honesty about native payloads applies to the encoding side too: the `org.bytedeco` `linux-x86_64` payload this project builds against ships no FFmpeg plugin, so `Codec.Mp4v` fails to open a writer there and `Codec.Mjpg` in an `.avi` is what you actually get. An `.avi` full of motion JPEG is not something you drop into a pull-request comment. GIF is: every code-review tool, chat client and issue tracker written in the last two decades plays one inline, on a loop, with no plugin and no click.

The second obstacle is the one this book keeps returning to. A frame is a `Mat`, a `Mat` is megabytes of off-heap pixels behind about forty bytes of on-heap handle, and an animation is defined by having a lot of frames. Chapter 1 measured what happens when you hold them: 2000 `Mat(1000, 1000, CV_8UC3)` allocations with their references dropped finished at 5,865 MB of resident memory, against 144 MB for the same 2000 with `release()` called. An animation API that materialises every frame before writing anything is that measurement, with a friendly name on it.

This chapter is about `Animation`, in the `scalacv-graphs` module alongside the `Picture` of Chapter 16 and the `Chart` of Chapter 17. It has four entry points, two of which write a file and two of which hand you rendered images; the difference between them is entirely a difference of how many canvases are alive at once, and picking the wrong one is the failure mode above.

A drawing that takes the frame number

An animation here is not a timeline, a tween or a scene you mutate. It is a function `Int => Picture`: given a frame index, produce the immutable drawing for that frame. Everything else — rendering onto a canvas, closing the canvas, encoding — belongs to the four combinators, and they differ only in what they do with the rendered `Image`.

Table 18-1. The four `Animation` entry points. Every one takes the frame function last, so it reads as a block.

Entry point	Returns	Canvases alive at once
`record(path, frames, width, height, fps = 30, background = Color.Black, codec = Codec.Mjpg)`	`Either[CvError, Long]`	One — written straight through a `Recorder`
`gif(path, frames, width, height, fps = 15, background = Color.Black, loop = true)`	`Either[CvError, Long]`	All of `frames` of them, by necessity
`foreach(count, width, height, background = Color.Black)(frame)(f)`	`Unit`	One — closed for you before the next is drawn
`frames(count, width, height, background = Color.Black)`	`Seq[Image]`	All `count`, and every one is yours to close

The `Long` that `record` and `gif` return is the number of frames written. Neither accepts a negative `frames`, and neither accepts an `fps` of zero or less — `gif` checks it itself, because it divides by it to get a delay, and `record` gets the same check one level down from

`Recorder.open`. Both are `require`s, so they throw where a bad encode would have returned a `Left`; the reasoning is at the end of this chapter. A `gif` call with `frames = 0` short-circuits to `Right(0L)` without touching the filesystem.

The running example for this chapter is the aperture on the cover: six blades that close and open again on a loop. It is a good test case because it is entirely synthetic — no camera, no file, no model download — and because flat blades of one colour each are exactly the kind of drawing GIF is good at, for reasons the next section makes concrete.

Example 18-1. One blade, as a function of how far the aperture is open.

```
import scalacv.*

val centre = Point(160, 120)

// A wedge pointing inward, its tip `r` pixels from the centre, turned into
place.
def blade(index: Int, r: Double): Picture =
  Picture
    .polygon(Seq(
      Point(centre.x + r, centre.y),
      Point(centre.x + 105, centre.y - 62),
      Point(centre.x + 105, centre.y + 62),
    ))
    .fillColor(Color.hsl(18 + index * 4, 0.55, 0.42))
    .noStroke
    .rotate(index * 60, about = centre)

def aperture(i: Int): Picture =
  val r = 26 + 54 * (1 - math.cos(2 * math.Pi * i / 60.0)) / 2
  Picture
    .circle(centre, 104)
    .strokeColor(Color.gray(90))
    .strokeWidth(3)
    .on(Picture.all((0 until 6).map(b => blade(b, r))))
```

Nothing there is animated. `aperture` is a pure function returning a value, and the value knows nothing about time; the only thing that makes it an animation is that `i` reaches it. Writing the GIF is one call:

Example 18-2. Sixty frames, a seamless loop, and a file.

```
OpenCv.load()

Animation.gif("aperture.gif", frames = 60, width = 320, height = 240, fps = 12.5)
(aperture)
// Right(60L)
```

The frame count and the period agree deliberately. `aperture` has period 60, so frames 0 through 59 cover one full cycle and frame 60 would be a duplicate of frame 0 — rendering

exactly 60 gives a loop with no stutter at the seam. A frame count that does not divide the period is why a generated GIF hiccups once a cycle.

Swap `gif` for `record` and the same function becomes a video, at the cost of the container contract: `record` defaults to `Codec.Mjpg`, which opens only in an `.avi`, so the path's extension is part of the call and `spin.mp4` fails to open a writer even where the codec is present. That is Chapter 19's territory; the point here is that the frame function is untouched.

What is not untouched is the arithmetic inside it. `aperture` puts its centre at `Point(160, 120)`, which is the middle of a 320×240 canvas and the top-left quarter of anything larger, so a bigger canvas wants either a bigger `centre` or a `.scale(...)` around the result — the drawing has no idea what it will be rendered onto. The listings below render it at 1280×720 anyway, because the memory arithmetic is the point they are making.

Holding every frame, and not holding it

`Animation.frames` looks like the friendly one. It hands back a `Seq[Image]`, which is the shape every other Scala API in the world would give you, and it is the shape that will hurt you:

```
// Wrong: 300 live native canvases before the first byte leaves the process.
val shots = Animation.frames(count = 300, width = 1280, height = 720)(aperture)
shots.foreach(img => img.bytes(".png").foreach(upload))
```

Three hundred 720p canvases are 2.6 MB of pixels each, so that line reserves about 790 MB of native memory before the first upload. The loop does hand most of it back — `bytes` is a terminal: it encodes and then releases the image it was called on — but only for the frames it actually reaches. If `upload` throws on the fifth, the remaining 295 are unreachable and unreleased, and the forty bytes of handle each one leaves on the heap is not enough to make a collector care. The scaladoc on `frames` says so in as many words: **each image is yours to close**.

The right version depends on whether the frames need to outlive the loop. Almost always they do not, and then `foreach` is the answer: it renders one canvas, hands it to you, and closes it in a `finally` before drawing the next.

Example 18-3. The streaming shape: one canvas alive, however long the animation.

```
val recorded: Either[CvError, Unit] =
  Recorder.using("aperture.avi", Size(1280, 720), fps = 25) { rec =>
    Animation.foreach(count = 900, width = 1280, height = 720)(aperture) { canvas
=>
      rec.write(canvas).fold(e => throw e, _ => ())
    }
  }
```

Nine hundred frames at 720p is 2.3 GB of pixels if you hold them and 2.6 MB if you do not. The `Image` handed to the block is a real owned image — transform it, detect on it, draw on

it — and whatever you do with it, the canvas that was rendered for you is released when the block returns, on success and on exception alike. If you genuinely need the frames later (compositing a contact sheet, feeding an encoder that wants random access) `frames` is there, and then the closing is your job: a `try` / `finally` around the whole traversal, not a `close` inside it.

> **NATIVE MEMORY**
> `Animation.gif` is the one entry point that cannot stream. `Imgcodecs.imwriteanimation` takes an `org.opencv.imgcodecs.Animation` holding a `java.util.List[Mat]` of every frame at once, so `gif` renders all `frames` canvases and keeps them alive until the encode returns. That is not a leak — the `finally` closes every one, including the ones already rendered when a frame function throws partway — but it is a peak, and the peak is `frames × width × height × 3` bytes. A 60-frame GIF at 320×240 costs about 13 MB. The same 60 frames at 1080p cost 356 MB, and a 150-frame one costs 890 MB. If you find yourself wanting a long, large GIF, the size of the file will have talked you out of it before the memory does.

Table 18-2. What each entry point holds, for a 60-frame animation. Three bytes per pixel, BGR.

Canvas	One frame	`gif` (60 frames)	`foreach` (any count)
320×240	0.22 MB	13 MB	0.22 MB
640×480	0.9 MB	53 MB	0.9 MB
1280×720	2.6 MB	158 MB	2.6 MB
1920×1080	5.9 MB	356 MB	5.9 MB

What the GIF format charges you

`gif` has no third-party dependency behind it. It builds OpenCV's own `Animation` structure — `set_loop_count(0)` for an endless loop or `1` for a single pass, `set_frames` with the list of canvases, `set_durations` with a `MatOfInt` of per-frame delays — and calls `Imgcodecs.imwriteanimation`. Everything on that path is already in the OpenCV natives you loaded in Chapter 2, which is why the `scalacv-graphs` jar can offer GIF output without dragging an image encoder, an AWT `ImageWriter` or a headless-display assumption in with it. The 0.1.0 split of the published artifact into `scalacv`, `scalacv-vision` and `scalacv-graphs` was made so that a consumer who only wants `Image.read(…).gray.canny(…)` pulls in neither a SLAM detector nor a GIF encoder; that split only pays off if the GIF path stays this thin.

What you are charged for instead is the format. GIF stores a table of colour indices, at most 256 of them per frame, compressed with LZW over the index stream. Two consequences follow directly, and both of them are about what your drawings should look like.

The first is the palette. Your `Picture` was rendered into a 24-bit BGR canvas with 16.7 million colours available, and the encoder has to reduce that to 256 per frame; OpenCV dithers to fit, which is to say it scatters the quantisation error into a pattern of neighbouring indices so that the average is right even though no single pixel is. A photograph survives that treatment

as something recognisable but visibly speckled. A drawing made of flat fills does not need the treatment at all, because it never had more than a handful of colours to begin with.

The second is the compression. LZW earns its keep on runs — long stretches of the same index, and repeated short sequences it can put in its dictionary. A flat fill is the best case it has. A dithered gradient is close to the worst: the dithering that saved the appearance of your gradient also destroyed every run in it, so a region that looked like one colour to you can cost more bytes than the detailed part of the frame.

The practical rule falls out of the two together. Draw with few, flat, well-separated colours — `Color.wheel(n)` gives exactly that: `n` hues spaced evenly round the circle at one saturation and one lightness (`0.65` and `0.55` unless you pass your own), and `Color.categorical` is the ready-made `wheel(8)`. Keep the background one solid colour. Use `Color.ramp(from, to, n)` sparingly and with a small `n` — it interpolates, and every intermediate step it produces is a colour the encoder has to find room for in the same 256. Alpha is not a problem to draw with, since `Picture` composites transparency onto the canvas before the encoder ever sees it; a half-transparent overlay becomes one more flat colour.

Frame rate, and why the useful band is narrow

`gif` computes each frame's delay as `max(1, round(1000.0 / fps))` milliseconds and stores it for every frame. The format itself is coarser than that: a GIF's per-frame delay is recorded in hundredths of a second, so a request the millisecond value cannot express in whole centiseconds is rounded on the way into the file. Rates that divide cleanly — 10 fps at 100 ms, 12.5 at 80, 20 at 50, 25 at 40 — come back out exactly. The default of 15 does not: `1000.0 / 15` is 66.67 ms, which the rounding stores as 67, and 67 ms is 6.7 centiseconds, which a player reads as 7. A GIF asked for 15 fps therefore plays at 14.3, and a 150-frame loop meant to run ten seconds runs ten and a half. For a looping demo nobody notices; for a clip that has to stay in step with a soundtrack or a timestamp overlay, pick a rate from the clean list.

Below about 8 fps the eye stops reading motion and starts reading a slideshow. Above about 15, each extra frame is a full palette-compressed image added to a file that has no inter-frame motion compensation to spread the cost over — GIF's only concession is that unchanged pixels can be left out of a frame, and a moving camera has almost none. Between those, 10 fps is the reliable choice for a review artefact and 12.5 the one to reach for when the motion is fast. The way to get smooth motion in a GIF is not more frames per second; it is fewer pixels.

A GIF a reviewer will actually watch

Three numbers decide whether your animation is looked at or scrolled past. Width: 480 to 640 pixels is legible inline in a pull request and is a quarter of the bytes of 1280. Duration: six to ten seconds, because a loop shorter than that reads as a glitch and one longer than that gets

scrolled past before it makes its point. Frame count: at 10 fps those two give you 60 to 100 frames, which is also the range where the file stays in the low hundreds of kilobytes.

If what you want to show runs longer than ten seconds, do not slow the frame rate to fit it in. Decimate the source — keep every third frame of a 30 fps capture and play the result at 10 — so the motion runs at real speed and only the sampling is coarser. The second example below does exactly that.

From processed video to a GIF

The obvious way to turn a processed video into an animation is to keep the processed frames and encode them. That is the failure this chapter opened with, and it is worse here than for a synthetic animation, because the frames arrive from a decoder that would happily hand you tens of thousands of them.

The shape that works is two passes with a plain Scala value in between. Pass one walks the video, measures what you care about, and lets every frame go; what survives into pass two is a `Vector` of case classes, which the JVM heap handles without difficulty because it is what the JVM heap is for. Pass two is an ordinary `Animation.gif` over that vector.

Example 18-4. Pass one: measure every frame, keep no frame.

```
final case class Sample(centre: Point, area: Double)

val track = scala.collection.mutable.ArrayBuffer.empty[Sample]

val walked: Either[CvError, Unit] =
  Camera.usingFile("clip.mp4") { cam =>
    cam.foreach() { frame =>
      // `frame` is closed for us; the mask this chain produces is not, so it is
scoped by hand.
      val mask = frame.gray.blur(2).threshold(90)
      try
        mask.contours().maxByOption(_.area).foreach { c =>
          c.centroid.foreach(p => track += Sample(p, c.area))
        }
      finally mask.close()
    }
  }
```

`Camera.foreach` releases the `Image` it hands you when your block returns, including when your block consumed it — release is idempotent, so a chain that spends the frame is fine. What it does not know about is the image at the **end** of your chain. `mask` is a live native handle that the `contours` query borrows and nothing else owns, so it gets the `try` / `finally`. Miss that line and the loop leaks one mask per frame: on a two-minute 720p clip, 3,600 single-channel masks and a little over 3 GB.

Peak native memory for that pass is three `Mat`s, however long the video runs: the buffer the decoder reuses, the copy `foreach` hands you, and the destination the current step of the chain is writing into — `gray`, `blur` and `threshold` each allocate their output before releasing their input, so two links of the chain are alive for the length of one call and never more. `track` grows by one small case class per frame, on the heap, where growth is the collector's problem and not yours.

Example 18-5. Pass two: animate the measurements, at a tenth of the frames.

```
// 30 fps in, every third sample out: a 10 fps GIF that runs at real speed.
val samples = track.toVector.zipWithIndex.collect { case (s, i) if i % 3 == 0 =>
s }.take(100)

val encoded: Either[CvError, Long] =
  Animation.gif("track.gif", frames = samples.size, width = 320, height = 240,
fps = 10) { i =>
    val trail = Picture.polyline(samples.take(i +
1).map(_.centre)).strokeColor(Color.Cyan)
    val dot = Picture.marker(samples(i).centre, Color.Yellow, radius = 8)
    val bars = Chart.bars(samples.take(i + 1).map(_.area), width = 300, height =
40, color = Color.Green)
    dot.on(trail).scale(0.5).on(bars.at(Point(10, 190)))
  }
```

The `scale(0.5)` is the whole coordinate mapping. The trail and the marker are built in the source video's coordinates — the clip here is 640×480 — and scaling about the origin, which is what `Picture.scale` does by default, puts them on the 320×240 canvas. Match the factor to your own clip: the samples came out of the decoder at its resolution, and nothing downstream of `contours` remembers what that was. The chart is composed afterwards, in canvas coordinates, so it is not scaled with them.

Two honest wrinkles. `Chart.bars` scales to the tallest value in the series it is handed, so a growing prefix re-scales whenever a new maximum arrives; if that distracts, hand every frame the full series and animate a marker along it instead. And frame 0 has a one-point trail, which draws nothing at all — `Picture` strokes a path only from two points on, so the first frame is the marker alone rather than an error.

A hundred frames at 320×240 costs about 22 MB of peak native memory during the encode, and lands as a file in the low hundreds of kilobytes: an artefact you can drop into a review comment, which loops forever because `loop` defaults to `true`, and which shows in eight seconds what a table of per-frame centroids would not show at all.

When the encode fails

Both `gif` and `record` return `Left` rather than throwing when the write fails, and they fail in different places. A GIF the encoder refuses comes back as `CvError.EncodeFailed(path, "imwriteanimation returned false")`. A `record` whose writer never opened comes back as

`CvError.LoadFailed`, whose message names the codec and points at `Codec.Mjpg` in an `.avi`; a `record` that opened and then failed on a frame comes back as the `Left` that `Recorder.write` produced. Whenever a file may already have been started, both delete it before returning the `Left`. That deletion matters more than it looks. A truncated GIF still opens in a viewer and still shows its first few frames, so a partial file reads as success to everybody downstream; removing it makes a failure look like a failure. An `IllegalArgumentException` is a different matter and stays a throw, on both sides of the call: a `Picture.star` with one point, a `Chart` with a zero-width box, a negative `frames`, an `fps` of zero. Those are your bug, not a condition of the output device, and a `Left` would only invite you to log them and carry on.

`gif` writes to a path, not to a buffer. When what you need is bytes — attaching to an HTTP request, posting through an API — write to a temporary file and read it back; the encode has already happened by the time `gif` returns.

Next

Everything in this chapter drew its own pixels. The frame function is pure, the canvas is blank until a `Picture` touches it, and the only native memory in play is the one the renderer allocated. Chapter 19 turns to video — `Video.frames`, `CaptureOptions`, and the `Recorder` that `Animation.record` writes through — where the frames come off a decoder rather than a renderer, the `Mat` you are handed on each iteration is borrowed and reused rather than owned, and end-of-stream and failure look identical until you ask the right way.

PART IV

Video and Cameras

Frames arrive whether or not you are ready for them. Reading and writing video, driving a live camera, detecting motion cheaply, and deciding what happens when the pipeline falls behind the source.

CHAPTER 19

Video I/O

One reused buffer, a writer that fails loudly, and the arithmetic between frame index and wall-clock time.

A still image is a resource you open, use, and drop. A video is the same resource arriving thirty times a second for as long as you keep asking, and every one of those arrivals is a `Mat` — around six megabytes of off-heap pixels at 1080p, behind about forty bytes on the heap. Video is where Chapter 1's cautionary tale becomes arithmetic: a minute of untouched 1080p frames is over ten gigabytes of native memory that the heap graph will not mention.

That arithmetic rules out the abstraction most Scala programmers reach for first. "The frames of a video" reads like a lazy sequence, and a `LazyList` is exactly wrong here for a reason that has nothing to do with taste: it memoises. Once a cell is evaluated it holds its head forever so that a second traversal is cheap, which means every frame the list ever produced stays reachable. Either nothing is ever released — an unbounded native leak — or frames are released as they are consumed and the list becomes a field of dangling handles that the next traversal hands back as empty Mats. There is no version of that API where memoisation and per-frame release are both correct.

The second thing a video is, and a still image is not, is a stateful cursor. A capture has a position; reading advances it; the read blocks in native code with no timeout of its own; and the backend reports "the file ended" through the identical failure it uses for "the camera was unplugged". None of that is visible in a type, so it has to be visible in the API's shape instead.

Output fails more quietly still. Ask OpenCV for a writer in a codec the build does not ship and its usual answer is not an exception: it hands back a `VideoWriter` left quietly closed. Every subsequent `write` then does nothing — and reports nothing, because `VideoWriter.write` returns `void`. You get a file that plays for zero seconds, out of a program in which every line reported success. That failure, and how scalacv turns it into a value you cannot ignore, is the second half of this chapter.

Opening a source

Everything in this chapter ships in the core artifact, `com.worxbend::scalacv:0.4.2` — the same single dependency Part II ran on. Video needs no extra module: `Video`, `Camera`, `Recorder` and `Codec` all live in the `scalacv` package, and the only other import any listing here needs is `org.opencv.videoio`, for the raw `VideoCapture` type and the `CAP_PROP_*` constants.

`Video.open` has four overloads across two shapes: a device index, or a source string, each with and without a `CaptureOptions`. Both return `Either[CvError, Managed[VideoCapture]]`.

Example 19-1. Every open is an `Either`, and the handle inside it is owned.

```
import scalacv.*
import scala.concurrent.duration.*
import org.opencv.videoio.VideoCapture

val fromFile: Either[CvError, Managed[VideoCapture]]    = Video.open("clip.mp4")
val fromCamera: Either[CvError, Managed[VideoCapture]]  = Video.open(0)
val fromNetwork: Either[CvError, Managed[VideoCapture]] =
  Video.open("rtsp://camera.local/stream", CaptureOptions.withTimeout(5.seconds))
```

The `source` string is whatever the backend understands: a filesystem path, an `rtsp://` or `http://` URL, a `frame_%04d.png` image-sequence pattern, a GStreamer pipeline. OpenCV resolves it itself and knows nothing about classpath resources.

Whether a source opens is data-dependent — a missing file, an unreadable container, a protocol no backend on this classpath can drive — so it is a `Left` carrying a `CvError`, never a throw. What is a programming error stays a throw: an empty source string or a negative device index fails the `require` at the top of `open`, before OpenCV is involved.

The guarantee that matters is what `open` checks before handing the capture back. It sets `setExceptionMode(true)` for the duration of the open, so a missing file becomes a `CvError` quoting OpenCV's own message and naming the path, and it verifies `isOpened` afterwards. You never receive a `Right` holding a capture that cannot deliver frames — and the test suite pins both ends of that: a nonexistent path is a `Left`, and so is an existing `.avi` holding 512 bytes of the letter `A`. Neither becomes a video with no frames in it, the failure mode that costs you an afternoon because an empty result looks exactly like a correct one.

> **NATIVE MEMORY**
> `VideoCapture` is one of exactly three `org.opencv.*` types with a real public `release()` — the others are `Mat` and `VideoWriter`. The other 185 native-owning types expose only a private `delete(long)` and have to be freed through the handle bridge. A capture needs none of that machinery, and `Video.open` still wraps it in a `Managed[VideoCapture]`: release becomes an atomic get-and-set that frees exactly once, and touching a released capture throws `IllegalStateException` on the Scala side rather than segfaulting from JNI. Every failure path inside `open` releases the half-built capture before returning the `Left`, so a failed open leaks nothing.

The handle is caller-owned. Prefer `.use`, which releases on every exit path:

```
Video.open("clip.mp4").map(_.use(c => Video.frames(c)(_.size)))
```

If the capture has to outlive a single block, release it yourself in a `finally`; `release()` is idempotent, so belt and braces costs nothing.

What the file claims about itself

`Video.info(capture)` returns a `CaptureInfo`. It `require`s an open capture — against a closed one every field would be a query through a dead handle — and every field it returns is a `CAP_PROP_*` query, which is to say what the backend **claims**, not what will actually decode.

Example 19-2. Metadata, which is advisory in every field.

```
Video.open("clip.mp4").map { capture =>
  capture.use { c =>
    val meta: CaptureInfo = Video.info(c)
    (meta.size, meta.fps, meta.frameCount, meta.backendName)
  }
}
```

Table 19-1. `CaptureInfo`, and where each field is unreliable.

Field	Type	When it lies
`width` / `height`	`Int`	a camera may report `0` before it has delivered a frame; `size` bundles the pair as a `Size`
`fps`	`Double`	`0` for a camera that has not warmed up; a nominal, not measured, rate for a file
`frameCount`	`Long`	`0` for a live source (the question is meaningless); off by a frame or two on some containers
`backendName`	`String`	this one does not — it names the backend that opened the source

`frameCount` is clamped at zero on the way out, because some backends answer `-1`.

> **WARNING**
> Never make `frameCount` a loop bound. `for i <- 0 until Video.info(c).frameCount.toInt` is a bug in two directions at once: against a live source it reports `0` and you process nothing, and against a container that over-reports you read past the end. The only frame count that is true is the one the frame loop actually delivers. Use `info` to size a writer or draw a progress bar, and let the iterator decide when the video is over.

Walking the frames

Example 19-3. The frame loop, and its full signature.

```
def frames[A](capture: VideoCapture, attemptsPerFrame: Int = 1)(f:
Iterator[BorrowedMat] => A): A
```

`Video.frames` runs your function over an `Iterator[BorrowedMat]` created when the block begins and retired when it returns. The iterator owns **exactly one** `Mat` and decodes every frame into it, in place. That is the whole design: the native footprint of a frame loop is one

frame, whether the video runs five seconds or five hours, and there is no per-frame allocation to pay for.

Three behaviours of that iterator are worth knowing before you write against it. `hasNext` is idempotent — it holds a decoded frame pending rather than decoding again — which matters because `for f <- frames` desugars into exactly the `hasNext; hasNext; next()` shape that a naive implementation would turn into a silent frame-dropper. `next()` past the end throws `NoSuchElementException`, rather than handing back an empty `Mat` that looks like a legitimate black frame. And the capture is only borrowed: `frames` neither releases nor rewinds it, so a second call resumes exactly where the first stopped.

Example 19-4. Two traversals of one capture read consecutive spans.

```
Video.open("clip.mp4").map { capture =>
  capture.use { c =>
    val firstTen = Video.frames(c)(_.take(10).size)  // frames 0..9
    val nextTen  = Video.frames(c)(_.take(10).size)  // frames 10..19 — resumes,
does not rewind
    (firstTen, nextTen)
  }
}
```

`attemptsPerFrame` is how many consecutive failed reads end the stream. The default of `1` is right for a file, where the first `false` from `read` **is** end-of-file; a live camera can drop a frame without the stream being over, and a small value — 2 to 5 — rides that out. It is a bound and not a retry-forever, because `read` blocks in native code with no timeout of its own and an unbounded loop would turn a dead source into a thread that both hangs and spins.

Inside the loop, `frames` turns OpenCV's exception mode **off** and restores it afterwards. OpenCV signals end-of-file through the same `CvException` it uses for a broken stream — measured on a clean five-frame file, the end of the video arrives as `cap.cpp:533 error: (-2:Unspecified error) in function 'grab'`. With exception mode on, the loop could not tell a finished video from an unplugged camera. Off, a `read` returning `false` ends the stream cleanly and a genuine decode failure still surfaces as `CvError.NativeCall`.

The borrowing contract

This is the one place in scalacv where a `Mat` you are handed is not yours — the exact inverse of the contract everywhere else in the library. The frame is **borrowed**: valid from the `next()` that returned it until you next ask the iterator for anything, at which point the same buffer is overwritten with the following frame, and released for good when the `frames` block returns, on the exception path too.

So the mistake is the one that looks most like ordinary Scala:

Example 19-5. Wrong. Not ten owned frames --- ten views that are spent when the block ends.

```
Video.open("clip.mp4").map { capture =>
  capture.use { c =>
    Video.frames(c)(_.toVector)   // collects views that throw after the
traversal
  }
}
```

The collection itself succeeds, but accessing any retained `BorrowedMat` throws `IllegalStateException` after the traversal. The view checks liveness before JNI; extracting and retaining the raw `frame.mat` bypasses that protection and remains unsafe. `toList`, `sliding` and `buffered` retain views too, so reduce each frame or take an owned copy before combining.

The right version reduces each frame to something owned **before** pulling the next one:

Example 19-6. Right. Each frame becomes a number inside the loop; the vector holds Ints.

```
val contourCensus: Either[CvError, Vector[Int]] =
  Video.open("clip.mp4").map { capture =>
    capture.use { c =>
      Video.frames(c) { frames =>
        frames.map { frame =>
          frame.mat.cvtColor(ColorConversion.BgrToGray)
            .pipe(_.canny(80, 160))
            .use(_.findContours().size)
        }.toVector
      }
    }
  }
```

`toVector` is fine here and unsafe two listings ago, and the difference is the element type. Every mid-level `Ops` call — `cvtColor`, `canny`, `resize`, `gaussianBlur` — allocates its own destination and hands back a `Managed[Mat]` you own, aliasing nothing. So the whole of Part II runs correctly over a borrowed frame, and what escapes the `map` is an `Int`.

> **NATIVE MEMORY**
> The frame's buffer belongs to the iterator, and the iterator is retired before its `Mat` is released when the block exits. That ordering is load-bearing: `VideoCapture.read` on a released `Mat` would quietly reallocate it and hand back a real frame in memory nothing owns and nothing will free. Retirement is what makes an iterator you accidentally let escape **inert** — it reports no more frames, and `next()` throws a message naming the cause — rather than a slow leak.

Table 19-2. What a borrowed frame supports, and what the liveness guard rejects.

Operation	Safe?	Why
Read pixels, query `empty`, `size`, `findContours`	yes	consumed before the next pull

Any `Ops` call: `cvtColor`, `canny`, `resize`	yes	allocates its own owned destination
`rec.write(frame.mat)`, encoding to bytes	yes	the work happens before the next pull
`toList` / `toVector` / `sliding` / `buffered`	no	spent views; access throws
Stashing the `Mat` in a `var`, field or collection	no	the view is spent after the next pull or at block exit
`frame.clone()`, keeping and releasing the clone	yes	a raw `Mat` you now own; `framesCopied` wraps it for you

When you genuinely need frames that outlive the loop — comparing three, picking the sharpest, compositing — reach for `Video.framesCopied`. It has the same signature as `frames`, `attemptsPerFrame` and its default of `1` included, but over an `Iterator[Managed[Mat]]`: each frame is cloned into a caller-owned handle with its own pixel buffer. The clone happens as you pull, so frames you never reach are never copied, and everything you do pull is yours to release.

Example 19-7. Frames you can keep, at one full-frame copy each.

```
Video.open("clip.mp4").map { capture =>
  capture.use { c =>
    val firstThree: Vector[Managed[Mat]] = Video.framesCopied(c)
(_.take(3).toVector)
    try firstThree.foreach(m => analyse(m.get))
    finally firstThree.foreach(_.release())
  }
}
```

The price is one allocation and one full-frame copy per frame, which is why it is not the default. `Camera`, which hands you an owned `Image` per frame, is `framesCopied` with the copy wrapped and the lifetime handled — so before writing the listing above, check whether `Camera` already does what you want.

Why the frame source is an Iterator and nothing friendlier

`Iterator` is not an elegant type. It was chosen because every more elegant candidate retains, and retention is what a single reused buffer cannot survive. `LazyList` and `Stream` memoise by definition; `Seq` is strict; a reactive stream buffers to give you back-pressure.

Even within `Iterator` the combinators split cleanly. `map`, `filter`, `zipWithIndex`, `take`, `foreach` and `foldLeft` pull one element and forget it, and are safe; `sliding`, `grouped`, `buffered`, `duplicate` and every `to*` conversion hold elements, and are wrong on a borrowed frame. The rule that survives contact with the API: a combinator is safe on a borrowed frame exactly when it never has two frames in hand at once. When you need two — a frame difference, motion detection, temporal smoothing — `framesCopied` is not a nicety, it is the requirement.

Seeking, frame index, and wall-clock time

`Video` has no `seek`. That is deliberate: seeking is a property of the capture, not of a traversal, and only file-backed sources support it at all. The capture is borrowed rather than hidden, so you set the position on it directly with the videoio constants.

Example 19-8. Jump to frame 100, then read from there.

```
import org.opencv.videoio.Videoio

Video.open("clip.mp4").map { capture =>
  capture.use { c =>
    val accepted = c.set(Videoio.CAP_PROP_POS_FRAMES, 100.0)  // file sources
only
    val landedOn = c.get(Videoio.CAP_PROP_POS_FRAMES)         // where it
actually put you
    (accepted, landedOn, Video.frames(c)(_.take(30).size))    // 30 frames from
there
  }
}
```

Frame index and wall-clock time are not the same quantity, and the gap between them is where timestamps go wrong. The index is an exact integer the decoder counts. The time is `index / fps`, and `fps` came out of `CaptureInfo`, where it is the container's **nominal** rate. For constant-frame-rate material the two agree; for anything variable — a phone recording that dropped frames under thermal load, a screen capture, an RTSP stream — they drift, and the drift accumulates over the clip. If the timestamp matters, ask the backend for its own answer with `Videoio.CAP_PROP_POS_MSEC` rather than multiplying an index by a nominal rate. If only ordering matters, the index is exact and free.

Seeking is approximate in a second way, which is why the listing above reads the position back rather than trusting it. A decoder can usually only start at a keyframe, so a backend may land you on the nearest one rather than on the frame you named, and `set` returning `true` says the property was accepted, not that the cursor is where you asked. Seek, then ask.

> **NOTE**
> `frames` resuming rather than rewinding composes with seeking: set a position, take a span, set another, take another. The capture carries the state; the iterator is one traversal of it.

Writing: the Recorder

Output is a `Recorder`: a `VideoWriter` fixed at open time to one frame size, one frame rate, one codec, and one channel count.

Example 19-9. Opening a writer. Everything but the path and the size has a default.

```
def open(
    path: String,
    size: Size,
    fps: Double = 30.0,
    codec: Codec = Codec.Mjpg,
    color: Boolean = true
): Either[CvError, Recorder]
```

`fps` must be positive and both extents of `size` must be positive; both are `require`d rather than returned as a `Left`, because a zero in either is a caller who passed `CaptureInfo` fields straight through without reading the warning on them, not a property of the data.

`Recorder.using` takes the same five parameters plus a `use: Recorder => A` block, closes the writer on every exit path, and returns `Either[CvError, A]` — so a `using` nested inside another `Either`-returning scope is one `.flatten` away from the flat result you want. `write` comes in two overloads — `write(image: Image)` and `write(frame: Mat)` — returning `Either[CvError, Unit]`, and both **borrow**: writing neither consumes nor releases the frame. The `Mat` overload exists so that a borrowed frame from `Video.frames` can go straight into the encoder without the clone an `Image` would force.

Example 19-10. A frame-for-frame re-encode with no copy anywhere in it.

```
val reencoded: Either[CvError, Long] =
  Video.open("clip.mp4").flatMap { capture =>
    capture.use { c =>
      val meta   = Video.info(c)
      val outFps = if meta.fps > 0 then meta.fps else 30.0
      Recorder.using("copy.avi", meta.size, outFps, Codec.Mjpg) { rec =>
        Video.frames(c) { frames =>
          frames.foldLeft(0L)((n, mat) => n + rec.write(mat).fold(throw _, _ =>
1L))
        }
      }
    }
  }
```

Three details in that listing are load-bearing. `meta.fps` goes through a guard because `Recorder.open` requires a positive rate while `CaptureInfo` is advisory in every field — passing a source's `0` through unexamined turns a data-dependent failure into an `IllegalArgumentException` out of a call whose type promised a `Left`. The `foldLeft` counts as it goes, so "how many frames were written" is a value rather than a comment. And `fold(throw _, …)` is what stops a rejected write from being silently discarded: `write` returns an `Either`, and an `Either` dropped in statement position is the same class of bug as ignoring the result of `open`. That throw leaves the surrounding `Either` rather than joining it; wrapping the block in `Cv.attempt` folds it back into a `Left`, which is exactly what `Camera.recordTo` does internally.

> **NATIVE MEMORY**
> `VideoWriter` is the third of the three types with a public `release()`, and a `Recorder` wraps it in a `Managed[VideoWriter]` exactly as a capture is wrapped. `close()` is idempotent and is what finalises the container — a writer that is never closed can leave the file's index unwritten, so the clip is not merely leaked, it is unplayable. `Recorder.using` closes it for you. If `open` fails, the half-built `VideoWriter` is released before the `Left` is returned, in the same discipline `Video.open` applies to a capture.

Two things `write` rejects with an `IllegalArgumentException` rather than a `Left`, because both are programming errors and not data-dependent failures: a frame whose dimensions do not match the size the recorder was opened at, and a frame that is not 8-bit. The second check earns its place. `VideoWriter.write` returns `void` and the encoder never inspects the depth, so a `CV_32F` frame — a float-depth Sobel, a distance transform, a raw disparity map — used to be accepted, its bytes reinterpreted as 8-bit pixels, and a perfectly playable file of noise produced with every call reporting success. Bring a float frame down with `normalize`, which produces 8-bit by default, or with `convertScaleAbs` — the two the rejection message itself names.

FourCC, containers, and the zero-byte file

A `Codec` is a four-character code — a FourCC — packed into an `Int` in pure Scala, using the same bit layout as OpenCV's `CV_FOURCC`. Packing it in Scala rather than calling `VideoWriter.fourcc` means the enum can be referenced before `OpenCv.load()` has run.

Table 19-3. The four codecs, and what each one costs you in portability.

Codec	`fourcc`	Container	Availability
`Codec.Mjpg`	`MJPG`	`.avi`	**The default.** Motion-JPEG, served by videoio's built-in writer — no FFmpeg, no GStreamer, no system codec. Large files.
`Codec.Mp4v`	`mp4v`	`.mp4`	MPEG-4 Part 2. Needs videoio linked against FFmpeg or a platform encoder.
`Codec.Avc1`	`avc1`	`.mp4`	H.264. Best compression, only if the build ships an H.264 encoder.
`Codec.Xvid`	`XVID`	`.avi`	Xvid MPEG-4.

The container is half of the bargain, in both directions: MJPG opens only inside an `.avi`, `Mp4v` only inside an `.mp4`. Codec and extension always move together, and the commonest form of the mistake is changing one and forgetting the other — `Codec.Mjpg` with an `out.mp4` path fails to open even though the codec itself is present on every build.

`Mjpg` is the default for a concrete reason rather than a conservative one: the `org.bytedeco` `linux-x86_64` and `windows-x86_64` payloads this library pins ship **no FFmpeg plugin at all**, so `Mp4v` and `Avc1` do not open there.

Now the failure this section is named for. At the raw OpenCV level an unavailable codec gives you nothing to check: `VideoWriter.open` usually leaves `isOpened` false rather than throwing — it can throw at the codec boundary, which is why `Recorder.open` guards against both — and from then on `write` returns `void` and does nothing. What is left on disk is an empty file or a bare container header, produced by a program in which every line reported success.

`Recorder.open` converts that into a value you cannot accidentally ignore. It checks `isOpened` and returns `Left(CvError.LoadFailed)` naming the path, the codec, and the fallback to try. So the way to detect a missing codec is to stop discarding the `Either`:

Example 19-11. Prefer the compact codec, fall back to the one that always opens.

```
def openRecorder(base: String, size: Size, fps: Double): Either[CvError,
Recorder] =
  Recorder.open(s"$base.mp4", size, fps, Codec.Mp4v)
    .orElse(Recorder.open(s"$base.avi", size, fps, Codec.Mjpg))
```

A second signal is worth wiring into any batch job: the frame count. `Camera.recordTo` returns `Either[CvError, Long]`, the number of frames actually written, and a `Right(0)` means the source yielded nothing and no file was created. Asserting a positive count catches the empty-source case that a `Right` alone does not distinguish.

The worked example: an annotated re-encode

Put the pieces together: read a clip, count the external contours in each frame, burn that count into the frame as text, write the result out as a video. The interesting part is that the annotation **consumes** the frame.

`Camera.recordTo` is the one-call form of the whole loop: `recordTo(path, fps = 0, codec = Codec.Mjpg, attemptsPerFrame = 3)(transform)`. It reads every frame, applies your `Image => Image` transform, and writes the results, sizing the recorder from the **first transformed frame** rather than from `CaptureInfo` — which is what makes it correct against a source that answers `0 × 0` until it has delivered something. The `fps` default of `0` means **derive it**: the source's reported rate if it has one, `30.0` if it does not — the same guard the previous listing had to write out by hand.

Example 19-12. Read, annotate, write. `recordTo` owns every lifetime in the loop.

```
import scalacv.*

val written: Either[CvError, Long] =
  Camera.usingFile("clip.mp4") { cam =>
    cam.recordTo("annotated.avi", codec = Codec.Mjpg, attemptsPerFrame = 1)
{ frame =>
      val edges = frame.copy.gray.canny(80, 160)
      val n = try edges.contours().size finally edges.close()
      frame.drawText(s"$n contours", Point(12, 28), Scalar.Green)
```

```
    }
  }.flatten
```

Two details carry the listing. `frame.copy` is there because `gray` is a transform and transforms consume: without the copy, the frame would be spent before `drawText` saw it. And the last expression consumes `frame` and returns a new `Image`, which `recordTo` closes after writing — the original handle is already spent by then, and `close()` on a spent handle is a no-op rather than a double free. `attemptsPerFrame = 1` is the file setting; the default of `3` suits a camera and costs two extra blocking reads at end-of-file here.

The transform must hand back an 8-bit, three-channel frame, because the recorder is opened `color = true` — which is why an edge video's transform has to end `.convert(ColorConversion.GrayToBgr)`. Resizing inside the transform is allowed, as long as **every** frame is resized the same way; a size that changes part-way through is reported as a `Left`, not thrown.

`recordTo` writes every frame it reads. When that is not what you want — writing only the frames that pass a test, writing two outputs, interleaving something else — open the `Recorder` yourself and drive the loop:

Example 19-13. The explicit form: a recorder, a frame loop, and a filter between them.

```
val busy: Either[CvError, Long] =
  Camera.usingFile("clip.mp4") { cam =>
    val outFps = if cam.fps > 0 then cam.fps else 30.0
    Recorder.using("busy.avi", cam.size, outFps, Codec.Mjpg) { rec =>
      var kept = 0L
      cam.foreach(1) { frame =>
        val edges = frame.copy.gray.canny(80, 160)
        val n = try edges.contours().size finally edges.close()
        if n > 20 then
          val marked = frame.drawText(s"$n contours", Point(12, 28),
Scalar.Green)
          try rec.write(marked).fold(throw _, _ => kept += 1)
          finally marked.close()
      }
      kept
    }
  }.flatten
```

`cam.foreach` hands you an owned `Image` and closes it when your function returns, so the frames you skip need nothing from you. The ones you annotate produce a second `Image` that `foreach` knows nothing about: `marked` is yours, `write` only borrows it, and the `finally` closes it on the throwing path too. The `.flatten` is the nested-`Either` tax named earlier — `usingFile` and `using` each contribute one. Sizing the recorder from `cam.size` is safe only because the source is a file that already reports its geometry; against a camera, take the size from the first frame as `recordTo` does, and keep the `fps` guard either way.

Performance: decode, resize, skip

This book has no frames-per-second number to give you, and neither does the benchmark suite: every measured figure in `benchmark-results.md` is one call to one operation over a synthetic scene, and decode, colour conversion, your own logic and encode all interact. Measure your pipeline, on your material.

What the measurements **do** settle is where the wins are not. Reusing destination `Mat`s across frames — the "arena" that received wisdom calls the largest single win in an OpenCV loop — was benchmarked against the plain allocating pipeline over the `gray → blur → canny` chain, at three frame sizes. Both variants produce bit-identical output, verified by hash, so the delta is the per-frame allocate-and-free cost and nothing else.

Table 19-4. `ArenaReuseBench`: reusing destination Mats across frames, per chain.

Frame	Fresh allocation	Reused destinations	Delta
640×480	145.3 µs	139.1 µs	−4%
1920×1080	574.2 µs	576.3 µs	+0.4% — reuse was **slower**
3840×2160	7556 µs	7493 µs	−0.8%

At 1080p and above the delta is under one percent and changes sign, which is the shape of noise rather than of a win — and this harness runs both variants inside one un-forked JVM, so a sub-one-percent gap between them is precisely the kind of number its own methodology says not to trust. The 4% at 640×480 is the largest honest reading in the table, and it buys 6 µs off a chain that costs 145. An opt-in arena for that meant a large new API surface and the reversal of the no-in-place ownership contract the whole library rests on, so it was not built.

The wins that are real are the same win: stop copying frames.

- **Do not copy what you only read.** A 1080p BGR frame is about six megabytes, so copying every one at 30 fps is roughly 180 MB/s of allocation — an order of magnitude more traffic than the arena above was ever going to save. `Video.frames` allocates nothing per frame; `Camera` costs one full-frame clone per frame for the convenience of an owned `Image`. Stay on `Camera` until a profiler says that clone is your bottleneck, then drop to `Video.frames` on `cam.capture`, which is the same capture, borrowed.
- **Write borrowed frames straight through.** `Recorder.write`'s `Mat` overload exists so that a re-encode costs no clone at all.
- **Resize at the head of the pipeline, not in the middle of it.** Halving each side quarters the pixel count, and everything downstream — blur, threshold, contours — is linear or worse in pixels. Detection on a 640-wide frame followed by scaling the coordinates back up is usually a better trade than detection on a 1080p one.

Skipping frames deserves a caveat, because the obvious spelling does not save what you think. A `filter` or a `zipWithIndex` over the iterator is safe — neither retains — but the iterator has already decoded every frame it offers, so filtering saves your processing and none of the decode, which is frequently the largest single cost in the loop.

Example 19-14. Process every fifth frame --- but the other four were still decoded.

```
Video.frames(c) { frames =>
  frames.zipWithIndex.collect { case (frame, i) if i % 5 == 0 =>
    frame.mat.cvtColor(ColorConversion.BgrToGray)
      .pipe(_.canny(80, 160))
      .use(_.findContours().size)
  }.toVector
}
```

To skip the decode as well, advance the capture yourself. `VideoCapture.grab()` moves the source forward without decoding the frame or copying it into a `Mat` — it is exactly what `Video.open` uses to discard warm-up frames on a camera. Pull one frame through the iterator, grab past the ones you do not want, then pull again; `frames` resuming rather than rewinding is what makes that composition work. On a file, seeking is cheaper still where the container supports it.

Where this goes next

The capture is opened, walked, seeked and closed; the frame is borrowed and then reduced or copied on purpose; the writer fails as a `Left` at open rather than as a zero-byte file at three in the morning. Every guarantee here rested on the source being finite and cooperative: `frameCount` meant something, `attemptsPerFrame = 1` was correct, and end-of-stream arrived exactly once.

Chapter 20, *Cameras and Recording*, removes all three. A camera reports no frame count, delivers an under-exposed first frame while its auto-exposure loop converges, drops a frame without the stream being over, and never ends at all — and the `CaptureOptions` fields this chapter passed over, `backend`, `warmupFrames` and the two best-effort timeouts, are the levers for every one of those.

CHAPTER 20

Cameras and Recording

Live capture, where the source has no end, no honesty about itself, and no patience.

A video file is a well-behaved thing. It has a beginning, an end, a fixed frame size and a frame rate written into its container, and reading it twice gives you the same frames in the same order. Every chapter up to this one has assumed that shape, or something like it: an input that sits still while you work on it.

A camera is none of that. It is hardware with a driver in front of it, and the driver has opinions. It will tell you the frame rate is zero, because no frame has come out yet and it does not care to guess. It will accept a request for 1280 × 720 and give you 640 × 480 without mentioning the substitution. It will hand you a black first frame, because the sensor's auto-exposure loop has not converged, and report it as a complete success. It queues frames on your behalf, so the image you receive describes the recent past. And at some point — a jostled cable, a switch reboot, a power-saving mode you did not know about — it will stop delivering, in a way bit-for-bit identical to a file reaching its last frame.

None of that is fixable at the library layer. What a library can do is refuse to paper over it: return the failures as values, say which numbers are advisory, and free the six megabytes of pixels behind every frame whatever else goes wrong. That is what `Camera` and `Recorder` do. They are thin, deliberately, and the interesting part of this chapter is not their surface but the four operational problems underneath: warm-up, latency, reconnection, and the per-frame copy. The running example — watch a camera, find faces, draw boxes, record the result, stop cleanly on Ctrl-C — accounts for none of them at first and all four by the end.

One piece of setup. The face-detection lines below — `Cascades`, `CascadeName` and the `detect` / `detectHaar` verbs — live in `scalacv-vision`, not in the core artifact, so add `com.worxbend::scalacv-vision:0.4.2` beside the core dependency of Chapter 2, and add `import scalacv.vision.*` beside `import scalacv.*`. The rest of the chapter is core only.

Opening a source

`Camera.open` takes a device index; `Camera.openFile` takes anything the videoio backend understands — a filesystem path, an `rtsp://` or `http://` URL, a `frame_%04d.png` numbered sequence. Both hand back an `Either[CvError, Camera]`, because whether a source opens is data-dependent: the device may not exist, may be held by another process, or may speak a protocol no compiled-in backend can read. None of those is a programming error, so none of them throws.

Example 20-1. The four ways in, and the two you should use.

```
import scalacv.*

val byIndex: Either[CvError, Camera] = Camera.open(0)
val byUrl: Either[CvError, Camera]   = Camera.openFile("rtsp://camera.local/
stream")

// The scoped forms close the camera on every exit path, including an exception.
Camera.using(0) { cam => cam.snapshot().flatMap(_.write("shot.png")) }
Camera.usingFile("clip.mp4") { cam => cam.recordTo("blurred.avi")(_.blur(4)) }
```

`Camera` is `AutoCloseable` and caller-owned. `using` and `usingFile` wrap your block's result in the `Right` and surface a failed open as the `Left`, so the outer type of that first `using` is `Either[CvError, Either[CvError, Unit]]` — the inner `snapshot` has an error case of its own. `flatten` it.

> **NATIVE MEMORY**
> A `VideoCapture` is one of exactly three `org.opencv.*` types with a real public `release()`, the others being `Mat` and `VideoWriter`. That does not make the handle optional: an unreleased capture holds the device open, and on Linux the next process to ask for `/dev/video0` gets a `Left` saying the device is in use. Prefer the scoped form; if you hold a `Camera` across calls, close it in a `finally`.

Everything the source tells you is a guess

`cam.info` returns a `CaptureInfo`; `cam.size` and `cam.fps` are shortcuts onto it. Every field is a `CAP_PROP_*` query answered by the backend, and the backend is permitted to answer badly.

Table 20-1. What CaptureInfo reports, and the conditions under which it is wrong.

Field	Type	When it lies to you
`width` / `height`	`Int`	`0` on a camera that has not yet delivered a frame
`fps`	`Double`	`0` for a camera that has not delivered a frame
`frameCount`	`Long`	`0` for any live source, where the question is meaningless; off by a frame or two in some containers
`backendName`	`String`	— the one field that is true

Hence a rule with no exceptions: never write a loop bound from `frameCount`. `for i <- 0 until cam.info.frameCount.toInt` processes nothing against a webcam, and reads past the end or stops short against a file whose container rounds. The only true frame count is the one the loop delivers, and the loop already knows when to stop.

Warm-up, and why your first frame is black

A webcam is not ready when `open` returns. Auto-exposure, auto-white-balance and auto-gain are closed loops running on the device, converging over a handful of real frames, and there is no property to poll for "converged" — so the only fix is to pull frames and throw them away.

`CaptureOptions.warmupFrames` is how many to discard. Its default is `None`, meaning "let the source decide": a device index discards five, a file or URL discards none. That split is the whole point. A file has no exposure loop, its first frame is as correct as its hundredth, and discarding frames there would skip real content rather than junk.

Example 20-2. Overriding the warm-up in either direction.

```
val patient = CaptureOptions(warmupFrames = Some(20))   // a dim room, a slow
sensor
val exact   = CaptureOptions(warmupFrames = Some(0))    // no discards: I want
frame zero
```

Asking for a resolution

`CaptureOptions` carries a backend, two timeouts and the warm-up count, and nothing else — no `resolution` field and no `fps` field, because scalacv does not wrap a setting it cannot honour. Both are set on the raw capture through the borrowed `cam.capture`, and every one of those calls is a request rather than an instruction.

Example 20-3. Ask for a mode; then find out what you were given.

```
import org.opencv.videoio.Videoio

/** Requests a capture mode and returns what the driver actually settled on. */
def request(cam: Camera, width: Int, height: Int, fps: Double): CaptureInfo =
  val cap = cam.capture                       // borrowed; still owned by the
Camera
  cap.set(Videoio.CAP_PROP_FRAME_WIDTH, width.toDouble)
  cap.set(Videoio.CAP_PROP_FRAME_HEIGHT, height.toDouble)
  cap.set(Videoio.CAP_PROP_FPS, fps)
  cam.info                                    // read it back — this is the only
answer that counts
```

`set` returns a `Boolean`, and `true` means the backend accepted the property, not that the hardware honoured the value. A camera offering 640 × 480, 1280 × 720 and 1920 × 1080 will snap a request for 800 × 600 to the nearest mode it has and report success. Read `cam.info` back, size your recorder from what it says, and treat a disagreement as normal rather than an error.

> **WARNING**

> Changing the mode after `open` invalidates the warm-up `open` already paid for: the sensor re-negotiates and the exposure loop starts over. A `cam.taking(5)(_ => ())` straight after the `set` calls costs five frame periods — a sixth of a second at 30 fps — and saves you a dark first frame.

The frame loop

`foreach` runs your function over every frame, each one a fresh, owned `Image` on exactly the terms Chapter 4 set out, and closes that `Image` when your function returns — on success, on failure, and on exception.

Example 20-4. The signature, which has two parameter lists.

```
def foreach(attemptsPerFrame: Int = 3)(f: Image => Unit): Unit
```

The empty parentheses are load-bearing: `cam.foreach() { frame => … }`, or `cam.foreach(1) { … }` for a file, where the first failed read genuinely is end-of-stream. The default of `3` is a bound, not a retry policy — a live camera drops frames without being dead, and three consecutive empty reads is where the library stops guessing. It cannot be unbounded, because `VideoCapture.read` blocks in native code with no timeout of its own, so infinite retry against a dead device is a hung thread that also spins a core.

`foreach` closes the `Image` it gave you, so it is tempting to conclude nothing inside the loop can leak. Here is the loop that follows from that conclusion:

Example 20-5. Wrong: one full frame leaked per iteration, forever.

```
cam.foreach() { frame =>
  val boxes = frame.detectHaar(classifier)
  rec.write(frame.drawRects(boxes))          // the drawn Image is never closed
}
```

`drawRects` is a transform, so Chapter 4's move semantics apply: it **consumes** `frame` and returns a **new** `Image` wrapping the same `Mat`. The handle `foreach` holds has been spent, so the `close()` in its `finally` is a no-op, and the new `Image` — the one that owns the pixels — never reaches a terminal, because `rec.write` borrows. Every iteration strands six megabytes at 1080p, 180 MB per second at 30 fps, none of it visible to the collector: Chapter 1's failure, reproduced exactly.

Example 20-6. Right: the annotated image is named, and closed.

```
cam.foreach() { frame =>
  val boxes = frame.detectHaar(classifier)
  val marked = frame.drawRects(boxes, Scalar.Green, Thickness.Stroke(2))
  try rec.write(marked).fold(throw _, identity)
```

```
  finally marked.close()
}
```

The rule, once: inside `foreach`, the library owns the frame it handed you, and you own everything a transform hands back.

Batches

When you need several frames at once — to compare them, composite them, or pick the sharpest — `take(count, attemptsPerFrame = 3)` returns them as a `Seq[Image]`, every element a live resource in a bare collection no type can warn you about. `taking` is the scoped counterpart, and the one to reach for:

Example 20-7. A scoped batch closes every frame, including on an exception.

```
def taking[A](count: Int, attemptsPerFrame: Int = 3)(use: Seq[Image] => A): A

cam.taking(5) { frames =>
  frames.map(f => (f.width, f.height))       // plain Ints escape; the five
Images must not
}
```

Frames beyond the end of the stream are absent rather than an error, so a batch may come back shorter than you asked for.

Recording, and the size you do not know yet

A `Recorder` is fixed at open time to one frame size, one frame rate and one codec, and every frame written must match that size and be 8-bit. Both are `require`s, because a mismatched pipeline is a programming error — and for depth the check is the only signal that can exist: `VideoWriter.write` never inspects the depth, so a `CV_32F` frame would encode as a playable file of noise with every call reporting success.

That fixed geometry collides with everything the last section said about a camera's self-reporting. The obvious code is wrong:

Example 20-8. Wrong: on a live camera this throws out of a method that promises an Either.

```
Camera.using(0) { cam =>
  Recorder.using("out.avi", cam.size, cam.fps) { rec => … }
}
```

`cam.fps` is `0` for a camera that has not delivered a frame, and `Recorder.open` opens with `require(fps > 0, …)` — so this raises an `IllegalArgumentException` from inside a call whose type says it reports failure as a `Left`. `cam.size` can be `0 × 0` for the same reason, and trips the positive-size `require` one line further down.

Take the geometry from a decoded frame instead, which cannot misreport its own dimensions, and supply a frame rate rather than asking for one. `Camera.recordTo` does exactly that: it opens the recorder from the **first transformed frame**, and its `fps` parameter defaults to `0`, meaning "derive it from the source, falling back to 30 when the source reports none".

Example 20-9. Right: read, transform, write, with the recorder sized from real pixels.

```
val written: Either[CvError, Long] =
  Camera.usingFile("clip.mp4") { cam =>
    cam.recordTo("edges.avi")(_.gray.canny(80,
160).convert(ColorConversion.GrayToBgr))
  }.flatten
```

`transform` is therefore free to resize, as long as it resizes every frame the same way; a size that changes part-way through is a `Left` rather than a throw, because a source that renegotiates mid-stream (adaptive RTSP, a Media Foundation format change) is not the transform's mistake. A source that yields no frames returns `Right(0)` and creates no file. The trailing `convert(ColorConversion.GrayToBgr)` is not decoration: `canny` produces a single-channel edge map, and the recorder was opened `color = true`.

Codecs, and the container that comes with them

Chapter 19's codec table applies here unchanged, and so does its rule: the codec and the file extension travel together. What is worth repeating in a camera chapter is why the default is the ugly one. `Codec.Mjpg` in an `.avi` is served by videoio's built-in MJPEG writer — no FFmpeg, no GStreamer, no system codec — and the `org.bytedeco` `linux-x86_64` and `windows-x86_64` payloads this project builds against ship no FFmpeg plugin at all, so `Mp4v` and `Avc1` do not open there. A default of `Mp4v` would have returned a `Left` on the very natives the library pins. An unavailable codec is a `Left` and never a silent black file: OpenCV leaves `isOpened` false, and `Recorder.open` turns that into a `CvError.LoadFailed` naming the fallback to try.

write borrows, and at 30 fps that is the whole game

`Recorder`'s two `write` overloads both borrow. `write(image: Image)` does not consume, which is why the corrected loop still had to close `marked`. `write(frame: Mat)` exists for one reason: `Camera` sits on `Video.framesCopied`, which clones every frame so the `Image` is safe to keep, while `Video.frames` (Chapter 19's borrowing exception) decodes into one reused `Mat` and allocates nothing. Annotate a frame and write it straight out, and that clone buys nothing.

Example 20-10. A zero-copy annotate-and-record loop: one Mat for the whole stream.

```
Video.frames(cam.capture, attemptsPerFrame = 3) { frames =>
  frames.foreach { frame =>                                  // borrowed, reused,
never copied
    val boxes = frame.detect(classifier)                     // mid-level:
```

```
Seq[Rect], no handles
    boxes.foreach(b => frame.drawRect(b, Scalar.Green, Thickness.Stroke(2)))
    rec.write(frame).fold(throw _, identity)                // borrows the same
Mat
  }
}
```

The mid-level `drawRect` and `drawText` mutate their receiver and return `Unit`, so the annotations land in the decode buffer itself and do not accumulate: the next `read` overwrites it. The library's own benchmark notes put the priority plainly — stop copying frames before you start reusing buffers. Per-frame destination allocation measured inside the noise; the copies were worth an order of magnitude more.

> **NATIVE MEMORY**
> Chapter 19's borrowing contract is in force throughout that loop. The `Mat` is valid from the `next()` that produced it until you next touch the iterator, and released when the `frames` block returns. `frames.toList` compiles and gives you N references to one buffer holding the last frame. Reduce inside the loop, or use `Camera` and pay for the clone.

The frame you read is not the frame that is happening

Between the sensor and your `read` there is a queue. The driver fills it as frames arrive; you drain it as fast as your processing allows. If your loop takes longer than a frame period the queue grows and never shrinks, and the frame you are handed ages with it. Nothing reports this: the frames keep arriving, keep decoding, keep looking correct. Whether that matters depends on what they are for.

Table 20-2. Latency: when the backlog is a bug and when it is free.

Job	What a growing backlog does
Recording, re-encoding	Nothing. Every frame is footage; late is fine, missing is not
Motion detection, frame differencing	Changes the measurement — two frames 200 ms apart are not two frames 33 ms apart
Interaction, gesture, a live endpoint	Breaks it. The answer describes a moment that has passed
Optical flow, tracking, odometry	Breaks it differently — these assume small motion between frames

Two levers exist, both on the borrowed capture, because neither behaves the same way on two backends. The first is the driver's queue depth, exposed as a capture property:

Example 20-11. A diagnostic, not a fix.

```
val accepted = cam.capture.set(Videoio.CAP_PROP_BUFFERSIZE, 1.0)
val reported = cam.capture.get(Videoio.CAP_PROP_BUFFERSIZE)
```

`accepted` being `true` does not mean the driver honoured it; `reported` may be the value you asked for, the value in force, `0` or `-1`; and nothing in `CaptureInfo` can confirm it either way. Try it, then measure — if the backlog does not change, you did not get it.

The second lever is the split between `grab()` and `retrieve(mat)`, which is what `read` does together: `grab` fetches the next frame from the device, `retrieve` decodes and colour-converts it. The decode is the expensive half, so discarding a queued frame with a bare `grab()` is far cheaper than reading it — a way to catch up to the head of the queue. But `grab` blocks exactly as `read` does when the queue is empty: ask for two extra grabs when only one frame is queued and the second waits on the sensor, putting you a frame period behind a source you were keeping up with. Reasonable on a source you have measured yourself to be behind on; a bad permanent setting.

> **TIP**
> For a recorder, ignore all of this. A recording has no latency requirement, only a completeness requirement — which dropping frames violates. Make the consumer fast enough and let the queue do its job.

When the camera goes away

`Camera.foreach` and `Camera.snapshot` end the stream after `attemptsPerFrame` consecutive empty reads, and OpenCV cannot say which reason applied: the file ended, or the device is gone. For a camera only one of those is possible, and a `Camera` whose device has disappeared does not heal.

So the supervision boundary is the `Camera` itself: close it and go back through `open`. Backoff so a dead device does not spin a core, a cap on the wait so a device that returns is picked up within a bounded time, and a bounded count so a permanently dead device becomes an error somebody can act on rather than a process that looks healthy and does nothing.

Example 20-12. A supervised capture loop with a reopen budget.

```
import scala.annotation.tailrec
import java.util.concurrent.atomic.AtomicBoolean

/** Runs `onFrame` over a source, reopening it when the stream ends. Gives up
after
  * `maxReopens` consecutive attempts that produce no frames, returning the last
failure.
  */
def supervised(
    open: () => Either[CvError, Camera],
    maxReopens: Int,
```

```
    running: AtomicBoolean
)(onFrame: Image => Unit): Either[CvError, Long] =

  // The shift is capped: an uncapped one overflows to a negative sleep, which
throws.
  def backoff(failures: Int): Unit = Thread.sleep(math.min(250L <<
math.min(failures, 8), 30_000L))

  @tailrec
  def loop(seen: Long, failures: Int, last: Option[CvError]): Either[CvError,
Long] =
    if !running.get then Right(seen)
    else if failures > maxReopens then
      Left(last.getOrElse(CvError.LoadFailed("camera", s"no frames after
$maxReopens reopens")))
    else
      open() match
        case Left(err) =>
          backoff(failures)
          loop(seen, failures + 1, Some(err))
        case Right(cam) =>
          var got = 0L
          try cam.foreach(3) { frame => got += 1; onFrame(frame) }
          finally cam.close()
          // A reopen that delivered frames earns the budget back; one that
delivered
          // nothing spends another line of it.
          if got > 0 then loop(seen + got, 0, None)
          else
            backoff(failures)
            loop(seen, failures + 1, last)

  loop(0L, 0, None)
```

Two things around that loop are easy to get wrong. A reopened camera is cold again — the exposure loop restarts, so nothing derived from overall brightness should be seeded from the first frame after a reopen. And reopens are a signal, not an implementation detail: count them. Chapter 39 treats "reopens per camera-hour" as an error-budget line with a target of under one, on the grounds that a camera reopening several times an hour is telling you something about the cable, the switch or the contended device that no retry logic will fix.

Network cameras

Two protocols cover almost all of them, and they fail differently.

An **MJPEG** camera serves a `multipart/x-mixed-replace` HTTP stream of independently compressed JPEG frames. Every frame is a keyframe, so there is no inter-frame state to lose: drop one, reconnect mid-stream, and the next frame still decodes on its own. The cost is bandwidth — no temporal compression at all. Open one with `Camera.openFile("http://…")`.

RTSP is what almost every IP security camera speaks, carrying H.264 or H.265, so a dropped packet corrupts frames until the next keyframe. It also needs a backend that can decode it —

FFmpeg or GStreamer — and if your OpenCV build ships neither, `open` returns a `Left` that retrying will not change.

For both, set the timeouts. `CaptureOptions.withTimeout` sets OpenCV's `CAP_PROP_OPEN_TIMEOUT_MSEC` and `CAP_PROP_READ_TIMEOUT_MSEC`, and network sources are where they earn their keep:

Example 20-13. Timeouts, and the backend that honours them.

```
import scala.concurrent.duration.*

val opts = CaptureOptions.withTimeout(5.seconds, CaptureBackend.FFmpeg)
val stream: Either[CvError, Camera] = Camera.openFile("rtsp://camera.local/
stream", opts)
```

Be precise about what that buys. FFmpeg and GStreamer honour the timeouts for network sources; V4L2, AVFoundation and the built-in MJPEG reader ignore them entirely, and nothing in the API says which backend you got — so a wedged USB webcam blocks forever whatever you set. They apply only at open time, which is why they travel through `CaptureOptions` rather than `cam.capture.set`. And a backend that does not understand them rejects the open outright: a local `.avi` opened with the parameters attached reports `isOpened == false` where the same file opens fine without them, which is why `Video.open` retries without them.

Naming a backend deserves the same caution. `CaptureBackend.Any` takes the first registered backend that works, and that is what you want in production; one that is not compiled into the build on your classpath cannot open anything, so naming it turns a working `open` into a failing one. If you must pin, put the named backend above `Any` in a ladder so a build without it still opens.

The one place a window appears

Nothing in this chapter needs a display. Capture, detection, annotation and recording are computation over pixels, and the core carries no GUI dependency — OpenCV's `imshow` needs a toolkit that resolves differently on every host, which is exactly the dependency a headless library must not have.

To watch frames arrive, the `examples-gui` module carries `scalacv.CamFaceDetect`: a JavaFX application that pulls frames through `Video.frames`, runs a Haar cascade, draws the boxes, and paints the result into a window by encoding each frame to PNG bytes for a JavaFX `Image` — no AWT, no `SwingFXUtils`. Run it with `./mill examples-gui.runMain scalacv.CamFaceDetect`. It is an example, not a library: `examples-gui` is never built in CI and never published, because OpenJFX resolves per host. Read it for the `setOnCloseRequest` that releases the cascade and the capture; do not depend on it. And note what a display costs — `Mat`-to-`BufferedImage` mea-

sures 976 µs at 1920 × 1080 with three channels, 3% of a 30 fps budget spent on looking at the thing.

The whole loop

Everything above, assembled: open a camera, load a bundled Haar cascade, learn the true frame geometry from a decoded frame rather than from `info`, record annotated frames through the borrowing `write`, and stop on Ctrl-C via a shutdown hook that flips a flag — so the loop finishes the frame it is on and the `using` blocks close the recorder and the camera in order rather than the process dying with a half-written file.

Example 20-14. Capture, detect, annotate, record, and shut down cleanly.

```
import java.util.concurrent.atomic.AtomicBoolean
import org.opencv.objdetect.CascadeClassifier
import scalacv.*

@main def watch(index: Int, out: String): Unit =
  OpenCv.load()

  val running = AtomicBoolean(true)
  Runtime.getRuntime.addShutdownHook(Thread(() => running.set(false), "scalacv-
stop"))

  val result = Camera.using(index) { cam => record(cam, out, running) }.flatten
  result.fold(e => println(s"capture failed: ${e.getMessage}"), n =>
println(s"wrote $n frames"))

def record(cam: Camera, out: String, running: AtomicBoolean): Either[CvError,
Long] =
  for
    cascade  <- Cascades.load(CascadeName.FrontalFaceAlt)
    // A decoded frame cannot misreport its own size; `cam.size` can, and does.
    geometry <- cam.snapshot().map(first => try first.size finally first.close())
    written  <- cascade.use { classifier =>
                  Recorder.using(out, geometry, fps = 15) { rec =>
                    loop(cam, rec, classifier, running)
                  }
                }
  yield written

/** The hot loop: one borrowed Mat for the whole stream, annotated in place,
written through. */
def loop(cam: Camera, rec: Recorder, classifier: CascadeClassifier, running:
AtomicBoolean): Long =
  var written = 0L
  Video.frames(cam.capture, attemptsPerFrame = 3) { frames =>
    while running.get && frames.hasNext do
      val frame = frames.next()
      val boxes = frame.detect(classifier, minNeighbors = 5)
```

```
      boxes.foreach(b => frame.drawRect(b, Scalar.Green, Thickness.Stroke(2)))
      frame.drawText(s"${boxes.size} faces", Point(10, 24), Scalar.White, scale =
0.7)
      rec.write(frame).fold(throw _, _ => written += 1)
  }
  written
```

Count the native objects and where each dies. The capture belongs to the `Camera`, released by `Camera.using`. The cascade — one of the 185 types with no public `release()`, freed through Chapter 5's handle bridge — is released by `cascade.use`. The `VideoWriter` is released by `Recorder.using`, which is what finalises the file. The snapshot `Image` is closed in its own `finally`, one line after its size is read. The decode buffer is the single `Mat` `Video.frames` owns and releases when the block returns. Five native lifetimes, five scopes, and no `release()` written by hand anywhere.

The `while running.get && frames.hasNext` ordering matters too: with the flag checked first, no further read starts once Ctrl-C has been pressed, so shutdown latency is bounded by the frame being processed rather than by a blocking read against a device that may already have gone quiet.

What the frames are worth looking at for

This chapter got frames into your program and back out to a file with their lifetimes accounted for, and said almost nothing about what to compute from them. The cheapest useful answer is the one most specific to a fixed camera: has anything in this scene changed? Frame differencing costs a subtraction and a threshold, runs at full frame rate on hardware that could not load a network, and turns a continuous stream into a sparse list of events worth recording. It is also, as the latency table hinted, the algorithm most sensitive to everything this chapter warned about. Chapter 21, *Motion Detection*, takes `MotionDetector` and its `Motion` result apart on those terms.

CHAPTER 21

Motion Detection

Finding what changed, so an expensive detector only runs when there is something to find.

A camera bolted above a loading bay looks at the same scene for weeks. The concrete does not move. The roller door does not move. For hours at a stretch the only thing in the frame that changes is sensor noise and the shadow of the fence creeping across the tarmac. Then a van arrives, something happens for ninety seconds, and nothing happens again until morning.

If the job is "tell me when a van arrives", a person-and-vehicle detector run on every frame spends essentially its whole budget confirming that concrete is still concrete — which is why the camera needs a GPU it should not need, and why the service falls behind on the one minute of the day that mattered. Recognition is also fragile: it wants a model file, a resolution, a lighting condition, and a confidence threshold you tuned on somebody else's dataset.

Noticing that something changed is none of those things: arithmetic over pixel differences, with no weights and no colour, working on a grainy infrared feed at three in the morning as well as at noon. It cannot tell a van from a fox, and does not need to — its job is to decide which frames are worth thinking about. Everything here refines one idea: keep a picture of the scene when nothing is happening, compare each new frame against it, report the pixels that disagree.

One assumption underneath all of it is load-bearing: the camera does not move. A fixed camera — a doorway cam, a trail cam, an ESP32-CAM dribbling low-frame-rate MJPEG over Wi-Fi — is what `MotionDetector` is built for. Point it at a handheld phone or a panning PTZ head and every pixel changes on every frame, which is a true answer to the question asked and a useless one. That case belongs to optical flow — `OpticalFlow`, Chapter 32 — not to this chapter.

One piece of setup first. `MotionDetector` and `Motion` live in `scalacv-vision`, not in the core artifact — this is the first chapter in this part of the book to need the second module. Add `scalacv-vision` at the same version as the core dependency of Chapter 2, then import both `scalacv.*` and `scalacv.vision.*`. The motion types and extensions belong to `scalacv.vision`; core's wildcard import does not activate them.

What "moved" means in pixels

Frame differencing is five stages, and each one deletes a specific class of false positive. Every tuning parameter you will meet is a knob on one of these stages, and knowing which stage a knob belongs to is most of knowing which knob to turn.

The frame arrives as BGR. Colour says nothing about whether something moved — a red car and a blue car change the same pixels — so the first step converts to a single channel with `ColorConversion.BgrToGray`, a two-thirds reduction in the work every later stage does, for no loss.

Next, a Gaussian blur. A CMOS sensor returns a different value for the same photon count on every exposure, and at any useful sensitivity that speckle sits comfortably above the difference threshold you want for real objects. Blurring spreads each pixel over its neighbours, so isolated noise averages away while a moving object — hundreds of contiguous pixels — survives intact, which is what lets the threshold stay low without every speck firing.

Then the difference itself: `absdiff` against the retained baseline, giving one channel where each value is how much that pixel changed. `threshold` turns that into a binary mask — 0 or 255 — and a `dilate` closes the pinholes a hard threshold punches through the middle of a real blob, so a person does not arrive as forty separate specks.

Finally the mask is measured twice. `Core.countNonZero` divided by the pixel count gives the fraction of the frame that changed. `findContours` over the same mask (outermost contours only, as in Chapter 12) gives one outline per blob, and each outline's `boundingRect` gives a box; boxes below a minimum area are dropped and the survivors sorted largest first.

Example 21-1. The five stages, in the order the detector applies them.

```
BGR frame ──▶ grey ──▶ blur ──▶ absdiff(baseline) ──▶ threshold ──▶ dilate ──▶
mask
                                                                          │
                                          countNonZero / pixels ──▶ ratio ┘
                                          findContours ──▶ boundingRect ──▶ regions
```

Both measurements come out of that one mask, but they are gated by different parameters — the ratio against `motionRatio`, the boxes by `minArea` — so a frame can report `moving = true` with zero regions: a great deal of the frame changed, no single blob survived the gate. Read `moving` for "should I care?" and `regions` for "where, exactly?".

The detector

`MotionDetector` is a trait with three operations, reached through one of two factories. `detect` feeds the next frame and reports what moved; `reset()` forgets accumulated state; `close()` releases native memory, which is what its `AutoCloseable` gets you. `detect` is overloaded — the second takes encoded bytes and is the MJPEG entry point, below. Both factories return that same trait, so switching strategy is one word and nothing else in your loop moves.

The result is a `Motion` — a plain case class, immutable, and still valid long after the frame it came from has been freed.

Table 21-1. Everything a `Motion` carries.

Member	Type	Meaning
`moving`	`Boolean`	`true` once `ratio` crosses the detector's `motionRatio`.
`ratio`	`Double`	the fraction of the frame that changed, in `[0, 1]`.
`regions`	`Seq[Rect]`	bounding boxes of the moving blobs, largest first, already filtered by `minArea`.
`regionCount`	`Int`	how many boxes survived the `minArea` gate.
`largest`	`Option[Rect]`	the biggest region, or `None`.

`Motion.still` is the constant for "nothing moved" — `moving = false`, `ratio = 0.0`, no regions — and it is what the very first frame returns, because there is nothing yet to compare it against. That frame becomes the baseline.

The running example for this chapter is that loading-bay camera: an ESP32-CAM streaming MJPEG at about five frames a second, to be watched, gated, and eventually recorded from. Here is the mistake to get out of the way first, because it is the one everyone writes.

Example 21-2. Wrong. A detector built inside the loop can never detect anything.

```
Camera.usingFile("http://bay-cam.local:81/stream") { cam =>
  cam.foreach() { frame =>
    val detector = MotionDetector.frameDifference()   // fresh state, every frame
    if detector.detect(frame).moving then alert()     // always Motion.still
    detector.close()
  }
}
```

Every frame is a first frame, so every frame is a baseline, so `moving` is never `true` and the alert never fires — silent rather than wrong, the worst failure mode a watchdog can have. A detector is **stateful** by design, and that state is the entire mechanism. It is also not thread-safe: feed frames in order, and give each thread its own.

Example 21-3. Right. One detector, outside the loop, closed when the loop ends.

```
Camera.usingFile("http://bay-cam.local:81/stream") { cam =>
  val detector = MotionDetector.frameDifference(minArea = 1500)
  try
    cam.foreach() { frame =>
      val motion = detector.detect(frame)
      if motion.moving then
        println(f"motion: ${motion.regionCount} region(s), ${motion.ratio *
100}%.1f%% of frame")
    }
  finally detector.close()
}
```

`Camera.usingFile` returns an `Either[CvError, A]`, so a stream that will not open is a value you handle rather than an exception — the error model of Chapter 6, unchanged. `foreach` hands you an owned `Image` per frame and closes it when your block returns, which is why there is no `close` on `frame` here.

> **NATIVE MEMORY**
> `detect` **borrows** the frame. It reads the pixels and does not release them — the opposite of a transform like `gray` or `blur`, which consumes its receiver (Chapter 4). Inside `Camera.foreach` that is invisible, because the loop owns the frame. Outside it, you close every frame you build.
>
> The detector itself owns native memory nothing else will free. `frameDifference` retains one full-size single-channel `Mat` — the baseline — from its first frame until `reset()` or `close()`. `backgroundSubtraction` holds a `BackgroundSubtractorMOG2`, one of the 185 `org.opencv.*` types with no public `release()` at all; scalacv frees it through `Releasable.nativeHandle`, the private-`delete` bridge of Chapter 5. Build one per frame, as in the wrong listing above, and you leak a background model per frame with nothing in the JVM motivated to notice.

Frame difference and its knobs

`MotionDetector.frameDifference` takes four parameters, all defaulted, and each one belongs to exactly one stage of the pipeline above.

Table 21-2. `frameDifference` parameters, their defaults, and the stage each one controls.

Parameter	Default	What it controls
`threshold`	`25`	per-pixel intensity delta, 0–255, that counts as changed. Lower is more sensitive.
`minArea`	`500`	moving blobs smaller than this many pixels are dropped — the noise gate on `regions`.
`blurRadius`	`2`	pre-blur applied before differencing, to suppress sensor noise; `0` disables it.
`motionRatio`	`0.002`	fraction of the frame that must change for `moving` to be `true`.

Each is validated on construction — `threshold` in `[0, 255]`, `motionRatio` in `[0, 1]`, `minArea` and `blurRadius` non-negative — so a value outside those ranges is an `IllegalArgumentException` at the call, not a detector that quietly does nothing.

The defaults are deliberately loose. `motionRatio = 0.002` means two-tenths of one per cent of the frame raises the flag: on a 640 × 480 feed, roughly 600 pixels, a person at the far end of a yard. You will almost always end up raising it.

A background model instead of a memory of one frame

Comparing each frame to the one immediately before it has two consequences. It absorbs a change instantly: a car that parks is motion for the frames it takes to stop, and from the next frame on it is part of the scene, because the baseline already contains it. And it lights

up wherever pixels **changed**, so a moving object appears twice — a hole where it was, a blob where it is. Fine for "did anything move?", poor input for anything that wants to follow an object.

`MotionDetector.backgroundSubtraction` takes the other approach: OpenCV's MOG2 maintains a statistical model of what each pixel normally looks like and flags the pixels that depart from it. MOG2 marks background as 0, foreground as 255 and cast shadow as 127, and the detector keeps only strong foreground by thresholding at 200 — so shadows are detected and then discarded rather than reported as objects. The clean-up is a morphological **open** (Chapter 9) rather than a plain dilate: erode then dilate, which removes isolated foreground speckle instead of growing it.

Table 21-3. `backgroundSubtraction` parameters. `minArea` and `motionRatio` mean what they mean above.

Parameter	Default	What it controls
`history`	`200`	how many recent frames the background model blends over.
`varThreshold`	`16`	Mahalanobis distance a pixel must exceed to count as foreground. Higher is stricter.
`detectShadows`	`true`	detect cast shadows and drop them. Costs a little.
`minArea`	`500`	the noise gate on `regions`.
`motionRatio`	`0.002`	frame-fraction threshold for `moving`.
`learningRate`	`-1`	how fast the model adapts; `-1` lets OpenCV choose from the frame count and `history`.

MOG2 needs a warm-up: with no history it has no idea what "normal" is, so the first frames over-report. Let it watch the empty scene for a second or two — the repository's own example warms up over 40 frames — before trusting its output.

Table 21-4. Choosing between the two strategies. The choice is about the scene, not the API.

	`frameDifference`	`backgroundSubtraction`
Compares against	the previous frame	an adaptive model (MOG2)
Cost	cheap, immediate	heavier, needs a warm-up
Slow lighting drift	no	yes
Marks the object	where it was **and** where it is	only where it is now
Absorbs a stopped object in	one frame	on the order of `history` frames
`reset()`	drops the baseline	a no-op — the model relearns on its own

The last two rows decide most real deployments. A car that parks in front of a frame-difference detector stops being motion immediately. The same car in front of MOG2 keeps raising foreground until the model blends it in, on the order of `history` frames — at the

default of 200, seven seconds of a 30 fps feed, forty seconds of the 5 fps ESP32 stream — and when it drives away, the space it left is foreground for `history` frames more: a ghost, until the model relearns the tarmac. Neither is a bug; both are the price of the respective memories.

The MJPEG shape

An ESP32-CAM's `/stream` endpoint is not a video in any container sense. It is a `multipart` HTTP response whose parts are independent JPEG files, one per frame, and that is why a motion pipeline built for it has a byte-shaped entry point:

```
val motion: Either[CvError, Motion] = detector.detect(jpegBytes)
```

This overload decodes and detects in one call, and closes the decoded frame for you in a `finally`. Doing it by hand as `Image.decode(bytes).map(detector.detect)` compiles, detects the same motion, and leaks a frame on every iteration — the `Image` that `decode` returns is yours, and nothing in that expression closes it.

The `Left` case is narrow: those bytes were not a decodable image. A truncated frame in the middle of a Wi-Fi stream becomes a value you log and skip, rather than an exception that tears down a loop which has been running since Tuesday.

Tuning as a procedure

Adjusting whichever number you can remember the name of produces a detector that works on the afternoon you tuned it. There is an order that converges, and it follows the pipeline. Record ten minutes of the actual camera first — a real event and a stretch of nothing — so every change is measured against the same footage. Then:

1. **`minArea` first.** By a wide margin the most effective knob, because it is the only one that distinguishes objects by **size** rather than by intensity. Sensor speckle and a swaying leaf are a handful of pixels; a person at the range you care about is hundreds or thousands. Measure — on a still, do not guess — how many pixels the smallest thing you want to catch covers at the far end of the scene, and gate at about half of that.
2. **Then `threshold` and `blurRadius`, together.** They trade against each other. Raising `blurRadius` removes single-pixel noise before the comparison, which lets you **lower** `threshold` and catch low-contrast movement — a dark coat against dark tarmac — without the speckle firing.
3. **`motionRatio` last,** because it only decides the boolean. Log `ratio` for a day and look at the distribution: the gap between the noise floor and a real event is usually a factor of ten, and any value in that gap works.

TIP

> `ratio` is the cheapest telemetry a vision service can emit. It is one `Double` per frame, it needs no decoding to interpret, and a week of it plotted against time will show you the daily lighting cycle, the moment the auto-exposure hunts, and the exact hour a spider built a web across the lens.

The failure modes you will actually hit

None of these are hypothetical. A fixed outdoor camera meets all of them within a month.

Table 21-5. What goes wrong outdoors, and what to do about it.

Failure	What you see	What to do
Slow lighting drift	`ratio` creeps up over an hour; nothing has moved	`backgroundSubtraction` — this is precisely what the adaptive model is for
Auto-exposure hunting	the whole frame flips brightness for one or two frames	raise `motionRatio`; require motion on two consecutive frames before acting
Camera shake in wind	thin regions along every high-contrast edge in the scene	raise `minArea` and `blurRadius`; mount the camera better — this is a hardware fault
Rain, snow, foliage	many small regions, scattered, every frame	`minArea` first; then `backgroundSubtraction`, which learns a repetitive leaf
Day/night IR switch	one frame of total change, then a grey feed	`reset()` on the transition if you can detect it; otherwise ride it out with a consecutive-frames rule

Two of those deserve more than a table row. The consecutive-frames rule is worth building in generally: hold the alarm until `moving` has been true on two or three frames in a row, and every single-frame artefact — an exposure step, a compression glitch, a moth crossing the lens — stops firing, for one or two frames of latency you will not notice.

And the IR switch has a sharper edge: some cameras change resolution when they change mode. A frame that is not the size of the retained baseline cannot be differenced, so OpenCV's `absdiff` throws and scalacv surfaces it at the boundary as `CvError.NativeCall`. What matters is what happens next: the detector neither leaks that frame nor strands a half-swapped baseline, so a correctly-sized frame detects normally afterwards. One exception at a real scene change, not a wedged detector.

Example 21-4. A mid-stream size change is a value, not the end of the loop.

```
cam.foreach() { frame =>
  val motion =
    try Right(detector.detect(frame))
    catch case e: CvError => Left(e)
```

```
  motion match
    case Right(m) if m.moving => alert(m.regions)
    case Right(_)             => ()
    case Left(err) =>
      println(s"frame rejected: ${err.getMessage}")
      detector.reset()   // the stream changed shape; take a fresh baseline
}
```

`reset()` is the right response to any **deliberate** scene change — the camera was repositioned, the floodlights came on, the stream renegotiated — because it treats the next frame as a fresh baseline instead of reporting the change itself as an event.

Motion as a trigger

Now the payoff. The reason to compute all of this is to **not** compute something else: put the detector in front of the expensive stage and run that stage only on the frames that pass. That is three lines more than the ungated version, and it changes the cost profile of the whole service.

Example 21-5. The gate: a face detector that runs on the frames where something happened.

```
Camera.usingFile("http://bay-cam.local:81/stream") { cam =>
  val gate = MotionDetector.frameDifference(minArea = 1500, motionRatio = 0.004)
  FaceDetect.create("models/face_detection_yunet_2023mar.onnx", Size(320,
320)).foreach { yunet =>
    var openFor = 0
    try
      cam.foreach() { frame =>
        if gate.detect(frame).moving then openFor = 15   // hold the gate open
        else if openFor > 0 then openFor -= 1

        if openFor > 0 then
          val faces = frame.faces(yunet)
          if faces.nonEmpty then
            frame.copy.markFaces(faces).write(s"hits/
${System.currentTimeMillis}.png")
      }
    finally
      yunet.release()
      gate.close()
  }
}
```

The `openFor` counter is not decoration, and leaving it out is the mistake that makes gated pipelines look unreliable. A motion detector reports **change**, and a person who walks into frame and then stands still stops producing change while remaining entirely present: gating strictly on `moving` would run the face detector for the two seconds of walking and then blind you for as long as they stand there. Holding the gate open for a fixed number of frames —

15 here, three seconds of a 5 fps stream — converts "something changed" into "something is probably still here".

The arithmetic is worth doing explicitly. Ungated, every frame pays the detector's cost; gated, every frame pays the gate's cost and only the passing fraction pays the detector's on top. Gating wins whenever the gate cost plus the pass rate times the detector cost is less than the detector cost alone — at a two per cent pass rate, whenever the detector costs more than about 1.02 times the gate. It pays as soon as the expensive stage is more expensive than the cheap one, which is the whole reason you called it expensive.

And the gate is genuinely cheap. `GrayBlurCloneBench` measures its front end directly — the default `blurRadius = 2`, so a 5 × 5 Gaussian, over an already-grey frame — at 126.8 ± 1.7 µs for 1920 × 1080; around it sit the greyscale conversion and single passes over one 8-bit channel. A DNN forward pass is measured in milliseconds. At a two per cent pass rate the expensive stage's contribution to the per-frame cost falls to a fiftieth, and the gate does not show up against what it saved.

> **WARNING**
> Those microseconds came from one developer machine, and `docs/mdoc/benchmark-results.md`, the page that records them, says plainly that the deltas reproduce and the absolutes do not. Measure both costs on your own hardware before you size a service on them. The **shape** of the argument survives any machine; the fiftyfold does not, because the pass rate is a property of your scene, not of the library.

The watcher

Everything so far assembles into the thing people actually want: a camera that writes a clip when something happens and stops when it is over. The interesting part is not the recording — `Recorder`, from Chapter 19, handles that — but the cooldown, because motion is bursty. A person crossing the yard produces motion, a gap while they pass behind a pillar, and motion again; without a cooldown that is three clips of four seconds instead of one clip of twelve.

Example 21-6. A clip watcher: open on the first motion, close after a run of quiet frames.

```
import java.nio.file.Path
import java.time.Instant

final class ClipWatcher(dir: Path, fps: Double, cooldownFrames: Int) extends
AutoCloseable:

  private var recorder: Option[Recorder] = None
  private var quiet = 0

  def onFrame(frame: Image, motion: Motion): Unit =
    if motion.moving then
      quiet = 0
      if recorder.isEmpty then start(frame)
```

```
    else quiet += 1

    recorder.foreach { rec =>
      rec.write(frame).fold(e => println(s"dropped a frame: ${e.getMessage}"),
identity)
      if quiet >= cooldownFrames then stop()
    }

  private def start(frame: Image): Unit =
    val path = dir.resolve(s"clip-${Instant.now().toEpochMilli}.avi").toString
    Recorder.open(path, frame.size, fps) match
      case Right(rec) => recorder = Some(rec)
      case Left(err)  => println(s"cannot record: ${err.getMessage}")

  private def stop(): Unit =
    recorder.foreach(_.close())
    recorder = None
    quiet = 0

  def close(): Unit = stop()
```

`Recorder.open` defaults to `Codec.Mjpg`, the one codec videoio can always write, which is why the path ends in `.avi` — MJPG does not open in an `.mp4` container. The recorder is sized from the frame that triggered it, so it inherits what the stream is actually delivering rather than what `info` claims — and that size is then fixed. `write` returns `Left` when OpenCV rejects the encode, but a frame of the wrong size is a programmer error and **throws**, which is the day/night resolution switch arriving in the writer instead of the detector. Where a camera does that, close the clip on the size change and open a new recorder. Driving the watcher is the loop you already have:

Example 21-7. The whole watcher: a stream, a gate, and clips on disk.

```
import scala.concurrent.duration.*

Camera.usingFile(
  "http://bay-cam.local:81/stream",
  CaptureOptions.withTimeout(5.seconds)
) { cam =>
  val detector = MotionDetector.frameDifference(minArea = 1500, motionRatio =
0.004)
  val watcher = ClipWatcher(Path.of("clips"), fps = 5.0, cooldownFrames = 25)
  try cam.foreach()(frame => watcher.onFrame(frame, detector.detect(frame)))
  finally
    watcher.close()
    detector.close()
}
```

`cooldownFrames = 25` at five frames a second is five seconds of quiet before a clip is closed — long enough to bridge a pillar, short enough that an empty yard is not written to disk. `CaptureOptions.withTimeout` sets the open and the read deadline together (Chapter 20), and

it matters for a Wi-Fi camera specifically: without it, a stream that stops delivering can block the read indefinitely instead of ending the loop. The deadline is best-effort — it is a hint to whichever backend opened the source — but the failure it prevents is the common one.

> **NATIVE MEMORY**
> Three ownership rules meet in `onFrame`, and they are all different. `detect` **borrows** the frame. `Recorder.write` also **borrows** it — it reads `image.mat` and does not release — which is what makes it safe to call inside `Camera.foreach`, whose own `finally` closes the frame afterwards.
>
> Drawing does not borrow. `drawRects` consumes its receiver and hands back a new `Image` owning the same `Mat`, so writing annotated clips means taking a copy and closing it yourself:
>
> ```
> val marked = frame.copy.drawRects(motion.regions, Scalar.Red)
> try rec.write(marked) finally marked.close()
> ```
>
> Skip the `.copy` and the frame you spend is the one `foreach` is holding; skip the `close` and you leak a full frame every time something moves — which is to say, every time the system is doing its job.

Where this goes next

A `Motion` gives you boxes per frame and nothing across frames. It cannot tell you that the box on the left of this frame is the same van as the box in the middle of the last one, so it cannot count vans, measure how long one stayed, or tell one leaving from another arriving. `ObjectTracker`, Chapter 30, is the consumer that can: it never looks at the image, only at the boxes. Background subtraction is the better source of those boxes, because it marks only where an object is; frame differencing would hand the tracker two blobs per object and invite it to invent an identity for the hole.

Example 21-8. Motion boxes in, stable identities out --- the counting pipeline in miniature.

```
val watcher = MotionDetector.backgroundSubtraction(minArea = 1500)
val tracks = ObjectTracker.create(iouThreshold = 0.3, maxAge = 5)
try
  cam.foreach() { frame =>
    val live = tracks.update(watcher.detect(frame).regions)
    live.foreach(t => println(s"#${t.id} at ${t.box} for ${t.age} frames"))
  }
  println(s"${tracks.count} distinct objects crossed the bay")
finally
  tracks.close()
  watcher.close()
```

`update` returns a `Seq[ObjectTrack]` — a persistent `id`, the current `box`, and the `hits` and `age` counts that say how much to trust it — while `count` is the running total of distinct objects stitched together. Tracker and detector share a contract: stateful, holding native memory, not safe to share across threads.

Before that comes the problem this chapter has been assuming away: that the loop keeps up. A detector falling behind a live camera neither throws nor slows the camera down — it measures older and older frames, and for frame differencing a skipped frame corrupts the **measurement** rather than delaying it, because the baseline and the current frame are no longer adjacent. Chapter 22, *Streaming and Backpressure*, comes next: the three things you can do when frames arrive faster than you can process them, and how to drop the ones you skip without leaking them.

CHAPTER 22

Streaming and Backpressure

What to do when frames arrive faster than you can process them.

A camera has no interest in your pipeline. The sensor exposes on its own clock, the driver decodes what the sensor gives it, and neither of them will wait while your detector finishes the previous frame. At 30 frames per second a new one appears every 33.3 milliseconds, and it appears whether or not you are ready for it. This is the one place in the book where the data source is a physical device with its own opinion about time, and it changes what "handling load" means.

Backpressure — a slow consumer telling a producer to wait — is the usual answer, and it is the one answer you cannot have here. A queue library gives it to you for free because both ends are programs. A camera is not a cooperating producer; it is a device. There is no protocol by which your face detector informs a CMOS sensor that it needs another twenty milliseconds. The vocabulary of streaming applies to the half of the system you wrote and not at all to the half that generates the data.

What makes this genuinely dangerous rather than merely annoying is that falling behind produces no error. There is no exception, no log line, no dropped-frame counter. The reads keep succeeding, the frames keep decoding, and every one of them is real. They are just increasingly **old**. A detector running twenty seconds behind the world is not a slow detector, it is a wrong one, and it will report its wrong answers with total confidence for as long as you leave it running.

There are exactly three responses to a producer that outruns a consumer: drop work, buffer it, or slow the producer down. This chapter is about which of the three you can actually have, what buffering costs when every queued item is a six-megabyte native allocation, and how to write the drop discipline so that the frames you throw away are also freed. The repository has a page on this — `docs/mdoc/streaming-and-backpressure.md` — and the patterns below are its patterns, because this is not an area to improvise in.

Every listing here assumes `import scalacv.*`, plus `org.opencv.core.Mat` wherever a raw frame is named and `java.util.concurrent.atomic.{AtomicBoolean, AtomicReference}` for the threaded ones. Two listings reach into `scalacv-vision` — `MotionDetector` and `Dnn`, with `org.opencv.dnn.Net` — so that artifact has to be on the classpath alongside the core `scalacv` one; the rest of the chapter is core only.

The arithmetic that decides everything

Suppose the source delivers 30 frames per second and your loop body takes 50 milliseconds. Each frame you process costs 50 ms of wall clock but advances you only 33.3 ms through the stream. You lose 16.7 ms per frame, permanently, and the loss compounds because nothing ever gives it back.

Table 22-1. A 50 ms loop body against a 33.3 ms frame budget. Nothing is dropped; everything gets older.

After	Produced	Consumed	Backlog	Frame age
1 second	30	20	10 frames	0.3 s
10 seconds	300	200	100 frames	3.3 s
1 minute	1800	1200	600 frames	20 s

The backlog is real memory sitting somewhere, and if you did not allocate it, the driver did. There **is** a buffer between the sensor and your code — V4L2 on Linux, AVFoundation on macOS, Media Foundation or DirectShow on Windows, FFmpeg's receive buffer for an RTSP stream. You did not create it, you cannot see its depth, and `CaptureInfo` does not report it. When you fall behind, that is where the frames pile up until it fills, at which point the driver starts discarding them itself, silently, with a policy you did not choose and cannot query.

Now take the three responses in turn.

Slow the source works perfectly — for a file. Every frame loop in scalacv is a pull loop: `Camera.foreach`, `Camera.recordTo`, `Video.frames` and `Video.framesCopied` all bottom out in a `while` loop calling `VideoCapture.read`, which blocks until a frame is available. Against a file on disk that blocking **is** backpressure. A slow consumer makes the whole job take longer and loses nothing. If your source is a file, this chapter is not about you.

Buffer is what you get by default from a live source, and it is a way of converting a latency problem into a memory problem. The next section prices it.

Drop is the only lever a live source leaves you, and the only question worth arguing about is which frames to lose.

> **WARNING**
> `attemptsPerFrame` is not a rate limit. `Camera.foreach(attemptsPerFrame = 3)` and `Video.frames(cap, attemptsPerFrame = 3)` control exactly one thing: how many consecutive **empty** reads it takes before the traversal decides the source is finished. `Video.frames` defaults it to `1`, which is right for a file where the first empty read is end-of-file; the `Camera` helpers default it to `3`, so a webcam that drops one frame keeps going. Neither value skips frames, bounds latency, or changes the read rate.

Measure the lag before you tune anything

You cannot ask a capture how far behind you are. But a pull loop hands you the measurement anyway, because the interval between two frame arrivals tells you which side of the budget

you are on. If your body is faster than the source, `read` blocks on the sensor and the interval settles at the source's real frame period. If your body is slower, `read` returns immediately — something is always queued — so the interval between arrivals **is** your body's duration.

So timestamp each frame at the top of the loop, divide consecutive deltas by the frame budget, and watch the ratio. scalacv ships no metrics facade of any kind, so the meter is yours to write; it is about thirty lines and you write it once.

Example 22-1. A lag meter: budgets used per frame, where 1.0 is break-even.

```
final case class LagReport(frames: Long, meanRatio: Double, worstRatio: Double,
slowFrames: Long)

/** Single-threaded on purpose: it lives inside one frame loop, and the loop is
one thread. */
final class LagMeter(nominalFps: Double):
  private val budgetNs: Double = if nominalFps > 0 then 1e9 / nominalFps else 0.0
  private var previousNs: Long = 0L
  private var seenOne: Boolean = false
  private var frames: Long = 0L
  private var slow: Long = 0L
  private var worst: Double = 0.0
  private var total: Double = 0.0

  def tick(nowNs: Long): Unit =
    if budgetNs > 0 && seenOne then
      val ratio = (nowNs - previousNs) / budgetNs
      frames += 1
      total += ratio
      if ratio > worst then worst = ratio
      if ratio > 1.0 then slow += 1
    previousNs = nowNs
    seenOne = true

  def report: LagReport =
    LagReport(frames, if frames == 0 then 0.0 else total / frames, worst, slow)
```

Wiring it into a real loop is three lines, and the only subtlety is where the budget comes from.

Example 22-2. The meter inside a Camera loop. Never divide by a reported zero.

```
def analyseFrame(frame: Image): Unit = ??? // the detector, the network, the real
work

def measuredLoop(source: String): Either[CvError, LagReport] =
  Camera.usingFile(source) { cam =>
    val lag = LagMeter(if cam.fps > 0 then cam.fps else 30.0)
    cam.foreach(3) { frame =>
      lag.tick(System.nanoTime())
      analyseFrame(frame)
    }
```

```
    lag.report
  }
```

Every field of `CaptureInfo` is a `CAP_PROP_*` query answered by the backend, and `fps` is a claim rather than a measurement: a live camera commonly reports `0` until it has delivered a frame, and some report a nominal rate they never achieve. That is why the example falls back rather than trusting `cam.fps`. If the ratio is the number you alert on, derive the budget from the rate you **configured the device for**, or from a calibration run against the real hardware.

Why the mean is the wrong number to watch

`LagReport` carries four fields and not one, which is the whole point. A loop that averages 0.85 of its budget looks healthy and can still be broken: if one frame in twenty takes three budgets — a garbage collection, a detector meeting a crowded scene, a network stream that stalls — then 5 per cent of your frames are late, the driver's buffer absorbs them, and the age of what you are analysing ratchets upward in steps that the mean averages away completely.

The number that shows it is a high percentile. `worstRatio` is a maximum, which is a p100 and therefore noisy — one unlucky sample defines it forever — but paired with `slowFrames` it tells you whether the worst case is an event or a habit. Twelve slow frames in a hundred thousand is a GC pause. Twelve hundred is a design problem. For a service that matters, keep the samples and emit a histogram, so that "p99 above the frame budget" becomes something you can alert on; the repository's observability guidance names exactly that alert.

There is one reading of the meter that looks like success and is not. A ratio sustained **below** 1.0 means reads are returning instantly because frames are already queued — you are draining a backlog, and the frames you are looking at are old. It ends when the queue empties, and it should be read as a warning.

Table 22-2. Reading a sustained lag ratio.

Ratio	What it means	What to do
≈ 1.0	Keeping up. `read` blocks on the sensor, which is where the wait belongs.	Nothing.
> 1.0	You are the bottleneck. Frame age grows by (ratio − 1) × budget per frame.	Do less per frame, then drop on purpose.
< 1.0 sustained	You are draining a backlog. The frames you see are stale.	Treat as a warning, not a win.
≈ 1.0 with spikes	A slow frame the driver's buffer absorbed.	Watch the slow-frame count, not the mean.

A ratio of 1.02 needs no architecture. Measure first; the rest of this chapter is for the loops that are genuinely over budget.

Drop the oldest, not the newest

Once you accept that frames must be lost, the policy question is which ones. For live vision — monitoring, detection, tracking-by-detection, an inference endpoint answering "what is in front of the camera **now**" — the answer is nearly always **latest-frame-wins**: keep the newest frame and discard the backlog behind it.

This is where the obvious tool does the wrong thing. A bounded `java.util.concurrent` queue drops by refusing the **incoming** item when it is full, which means it preserves the oldest frames and rejects the freshest. That is drop-newest, exactly backwards for this workload.

Example 22-3. Wrong twice: it keeps the stale frames, and it leaks the fresh ones.

```
// WRONG. `offer` refuses the NEW frame when the queue is full, so the worker
keeps
// chewing through history — and the refused clone is dropped without being
released.
val queue = java.util.concurrent.ArrayBlockingQueue[Managed[Mat]](8)

def fillQueue(cam: Camera): Unit =
  Video.framesCopied(cam.capture, attemptsPerFrame = 3) { frames =>
    frames.foreach(frame => queue.offer(frame))  // `false` when full, and nobody
looks
  }
```

Two defects, and the second one is the one this book is about. `offer` returns `false` when the queue is full and the caller ignores it, so that `Managed[Mat]` — a real clone, with its own pixel buffer — goes out of scope with nothing holding a reference and nothing having called `release()`. The garbage collector will reclaim the forty-byte Java object and leave the six megabytes behind it exactly where they are. At 30 frames per second, that is a leak that runs at the frame rate of your camera.

> **NATIVE MEMORY**
> A queue of frames is a queue of native buffers. A 1920 × 1080 three-channel frame is 1920 × 1080 × 3 = 6,220,800 bytes — call it 6.2 MB. A queue bounded at 8 holds up to 50 MB. A queue bounded at 1000 holds up to **6.2 GB**, which will kill the container long before it delays a frame. And an unbounded queue is not "a queue with no limit"; it is a native-memory leak with a scheduler attached, because the only thing that ever bounds it is the OOM killer. Whatever depth you choose, choose it in megabytes and then divide.

The right shape is not a deeper queue. It is a queue of depth one, where a newly arriving frame **displaces** the frame already waiting and frees it on the spot.

Example 22-4. A one-frame mailbox. The newest frame wins and the loser is released immediately.

```
/** Ownership moves with the frame: `offer` takes it from the caller, `take`
gives it to the
```

```
  * caller, and whatever is displaced is released here, because at that instant
this slot is
  * the only thing that still knows about it.
  */
final class LatestFrame:
  private val slot = AtomicReference[Managed[Mat]](null)

  /** Publishes `frame`, releasing whatever it displaces. The caller must not
touch it again. */
  def offer(frame: Managed[Mat]): Unit =
    Option(slot.getAndSet(frame)).foreach(_.release())

  /** Takes the newest frame, if any. The caller now owns it and must release it.
*/
  def take(): Option[Managed[Mat]] = Option(slot.getAndSet(null))

  /** Releases anything still in the slot. Call it once nothing can `offer`
again. */
  def drain(): Unit = take().foreach(_.release())
```

`getAndSet` is a single atomic operation, and that is what makes this correct without a lock: exactly one caller ever sees a given frame, so exactly one caller is responsible for releasing it. A second `take()` against an empty slot gets `None`, not a second reference to something already freed. Note also why this must be `Video.framesCopied` rather than `Video.frames` — a frame that crosses a thread boundary has to have its own pixel buffer, and the borrowed reused `Mat` from `frames` is valid only until the next pull.

Who frees the frame that was dropped

This is the part to get exactly right, because every row below is a transfer of ownership and a missing release at any of them is an unbounded leak.

Table 22-3. Ownership of a cloned frame as it crosses the slot.

Moment	Owner	Who releases it
`frames.next()` returns a clone	the capture thread	the capture thread, unless it offers
`offer(frame)` returns	the slot	—
a later `offer` displaces it	the displacing writer	`offer`, before it returns
`take()` returns `Some(frame)`	the thread that took it	that thread, via `use`
the writer has stopped and been joined	whoever calls `drain()`	`drain()`

Three ways to get it wrong, all of which compile and none of which fails a test on your laptop:

- **Filtering with `takeWhile`.** `frames.takeWhile(_ => running.get)` pulls a frame, evaluates the predicate on it, and discards the element when the predicate is false. `framesCopied` cloned it before the predicate ran, so the final clone is dropped without being released. Use a `while` loop that checks the flag **before** pulling.

- **Draining too early.** Calling `drain()` while the capture thread can still `offer` frees a frame the writer may be about to displace, and the frame offered after the drain is never freed at all. Stop the writer, join it, then drain.
- **Losing a taken frame.** A worker that takes a frame and throws before releasing it leaks that frame. `Managed.use` releases in a `finally`; use it rather than a bare `get`.

Example 22-5. The two halves. The capture thread never waits; the worker always gets the newest frame.

```
def process(frame: Mat): Unit = ??? // the detector, the network, the tracker

// Checking `running` before `hasNext` is why this is a while loop and not
`takeWhile`:
// no clone is ever created that the slot does not receive.
def captureInto(source: String, newest: LatestFrame, running: AtomicBoolean):
Either[CvError, Unit] =
  Camera.usingFile(source) { cam =>
    Video.framesCopied(cam.capture, attemptsPerFrame = 3) { frames =>
      while running.get && frames.hasNext do newest.offer(frames.next())
    }
  }

def workerLoop(newest: LatestFrame, running: AtomicBoolean): Unit =
  while running.get do
    newest.take() match
      case Some(frame) => frame.use(process)  // `use` releases even if `process`
throws
      case None        => Thread.sleep(5)     // nothing new yet; back off rather
than spin
```

If the worker took 200 ms and six frames arrived meanwhile, five were dropped — deliberately, at a known point in the code, in favour of the freshest. That is the entire difference between this and the driver silently discarding frames on your behalf.

> **CAUTION**
> Latest-frame-wins is wrong whenever frames are only meaningful as a sequence. Recording and re-encoding lose footage. Frame differencing and background subtraction compare frames whose spacing they cannot observe, so a 200 ms gap silently becomes a different measurement. Optical flow, odometry and trackers assume small motion between frames, and a dropped frame is precisely the large jump they handle worst. If you must drop **and** the sequence matters, record an arrival timestamp with each frame and make the algorithm take the elapsed time as a parameter.

Three shapes that work

The single-consumer loop

One thread reads and processes, which is what `Camera.foreach` already gives you. No queue, no handoff, no ownership question — the `Image` is closed for you when the body returns.

Reach for this until the lag meter says you cannot, and when it does, spend the first effort on making the body cheaper rather than on architecture. Detection cost scales with pixels, so halving each side quarters the pixel count; the cost is missing the smallest objects, which now occupy a quarter of the pixels they did.

```
cam.foreach(3) { frame =>
  val small = frame.scale(0.5)  // consumes `frame`; a quarter of the pixels to
detect on
  try detect(small) finally small.close()
}
```

The current callback scope follows this consuming successor, so early `close` is optional and idempotent. The older 0.4.1 tag closed only the original wrapper; there the explicit successor close is necessary. To retain a branch beyond the current callback, explicitly `detach` it and accept the responsibility to close it later.

A fixed worker pool

When the expensive stage parallelises — independent per-frame inference, several cameras at once — put a fixed number of workers behind the slot and give each one its own native resources. Not a shared one. A `Net`, a `FaceRecognizer`, a `Tracker` and every stateful detector in the library is a native object with mutable internal state, and a second thread entering an in-flight native call corrupts it. The failure is a segmentation fault in C++, not an exception you can catch.

Example 22-6. One net per worker thread, built on first use and never shared.

```
val perThreadNet = ThreadLocal.withInitial[Either[CvError, Managed[Net]]] { () =>
  Dnn.fromOnnx("model.onnx")
}
```

> **NATIVE MEMORY**
> A pooled detector is native memory with no owner but you. Of the 188 `org.opencv.*` types that own native memory, exactly three expose a public `release()` — the three that `Releasable` frees with a plain `_.release()`: `Mat`, `VideoCapture` and `VideoWriter`. `Net` is not among them, and neither is `CascadeClassifier`, `QRCodeDetector` or `ArucoDetector`. The `Managed[Net]` is what frees it, which means a `ThreadLocal` whose threads retire without closing their entry leaks one loaded model per thread. Size the pool, keep it fixed, and close each net when its thread retires — a pool that grows with load is a memory profile that grows with load.

Cap the inner thread pools when you fan out. OpenCV and OpenBLAS each run one, and if your outer parallelism already saturates the cores, the inner pools fight it. Set `OPENBLAS_NUM_THREADS=1` and `OMP_NUM_THREADS=1` in the environment, or call `org.opencv.core.Core.setNumThreads(1)` from code before you spread work. For a single

sequential pipeline, leave OpenCV's threading on — it parallelises the heavy kernels for you. Measure both.

Two stages: a cheap gate before an expensive detector

The pattern that usually removes the problem rather than managing it is the staged one from Chapter 21, *Motion Detection*: run something cheap on every frame, and the expensive thing only on the frames that survive. A motion gate on a downscaled frame costs a small fraction of a network forward pass, and on a static camera it rejects the overwhelming majority of frames without ever cloning them.

Example 22-7. Stage one on the capture thread, on a borrowed frame; only survivors are cloned.

```
def gatedCapture(cam: Camera, gate: MotionDetector, newest: LatestFrame, running:
AtomicBoolean): Unit =
  Video.frames(cam.capture, attemptsPerFrame = 3) { frames =>
    while running.get && frames.hasNext do
      val borrowed = frames.next()                              // valid until the
next pull
      val small = Image.wrap(borrowed.resize(Size(320, 240)))  // resize
allocates its own output
      val moving = try gate.detect(small).moving finally small.close()
      if moving then newest.offer(Managed(borrowed.clone()))   // the clone the
worker will own
  }
```

Three things make this correct. `Video.frames` borrows one reused `Mat`, so the loop pays no per-frame clone, and the gate then runs on a 320 × 240 copy rather than a 1080p one; `resize` allocates its own destination and never aliases its receiver, so running it over a borrowed frame yields an `Image` that you own and must close; and the full-frame clone happens only on the branch where a worker is actually going to want it. The borrowed frame never leaves the loop body — retaining it would give you a reference to a buffer that the next `read` overwrites.

The queue you did not build

Two levers exist on the driver's buffer, and both are on the raw `VideoCapture` that `Camera` lends you through `cam.capture`. scalacv wraps neither, because neither behaves the same way on two backends.

`cam.capture.set(Videoio.CAP_PROP_BUFFERSIZE, 1.0)` asks the backend to keep only the newest frame, moving the drop policy into the driver. Some backends implement it, some ignore it, some reject it; `set` returning `true` does not mean it was honoured, and `get` may report the value you asked for, the value in force, `0`, or `-1`. Nothing in `CaptureInfo` reports the depth, so the only confirmation is behavioural — the sustained-below-1.0 backlog goes away.

`VideoCapture` also splits a read in two: `grab()` fetches the next frame from the device and `retrieve(mat)` decodes and colour-converts it. Decoding is the expensive half, so discarding a

queued frame with a bare `grab()` is cheaper than reading it. The catch is that `grab` blocks exactly like `read` when the queue is empty, so a fixed skip count turns a source you were keeping up with into one you are behind on by that many frame periods. It is a fair trick for a source you **know** you are behind on, and a bad permanent setting.

Stopping

A `read` blocked in native code cannot be interrupted from the JVM. `Thread.interrupt()` sets a flag that a thread parked inside OpenCV's C++ never looks at, and there is no cancel and no timeout parameter on `read`. So after you set `running` to false, the capture thread is still inside `read` and stays there until the source delivers a frame, the connection breaks, or a backend-honoured read timeout fires. `CaptureOptions.withTimeout(5.seconds)` sets OpenCV's `CAP_PROP_OPEN_TIMEOUT_MSEC` and `CAP_PROP_READ_TIMEOUT_MSEC`, which FFmpeg and GStreamer honour for network sources and which V4L2, AVFoundation and the built-in MJPEG reader ignore entirely — so set it for `rtsp://` and `http://` and leave it off for files and local cameras.

That leaves a daemon capture thread as the reliable bound: a thread stuck in `read` cannot then stop the JVM from exiting, and native memory it still holds is reclaimed by the operating system when the process dies, which is the one moment not releasing is acceptable. So the thread is created daemon, and the shutdown stops the writer, `join`s it with a bound, and drains the slot **only** if the thread actually came back.

Example 22-8. Start it as a daemon; drain only after the writer has provably stopped.

```
def start(source: String, newest: LatestFrame, running: AtomicBoolean): Thread =
  val body: Runnable = () =>
    captureInto(source, newest, running).fold(e => println(e.getMessage),
identity)
  val reader = Thread(body, "scalacv-capture")
  reader.setDaemon(true)  // a thread parked in a native read must not hold the
JVM open
  reader.start()
  reader

def shutdown(reader: Thread, running: AtomicBoolean, newest: LatestFrame): Unit =
  running.set(false)  // the loop stops pulling once the current read returns
  reader.join(5000)   // may time out; a read blocked in native code cannot be
interrupted
  // Draining while the writer still lives is the early-drain leak from the
ownership table.
  // If the thread never came back, leave the slot alone and let process exit
reclaim it.
  if !reader.isAlive then newest.drain()
```

CAUTION

> Do not close the capture from another thread to unblock the read. A `VideoCapture` is a native object with one owner (Chapter 36, *Concurrency and Thread Safety*), and releasing it while a decode is in flight is a data race in C++ — a segmentation fault, not an exception: no stack trace, no catch, a dead process. The capture must be closed by the thread that reads it, after the read has returned, which is what `Camera.usingFile`'s `finally` already does.

What comes next

Everything in this chapter is a primitive you had to build: a mailbox, a lag meter, an ownership table enforced by discipline. Chapter 37, *ZIO Integration*, shows the same loops expressed with an effect system, where `Scope` releases the capture and the buffer on success, failure **and** interruption, and blocking reads sit on a blocking pool instead of starving the fibers doing real work. Be clear about what that does not buy you: `ZIO.attemptBlockingInterrupt` delivers a JVM interrupt, and a thread inside OpenCV's native read never observes one. Nothing anywhere cancels a blocked `read`. Bound it at the source, or plan for a thread that may not come back.

The lag ratio, the slow-frame count and the native-memory gauge are numbers you now know you need and have nowhere to send. Chapter 38, *Observability*, is about where they go: the four numbers a vision service is judged on — process resident set size, the ratio of frames processed to frames the source produced, per-stage latency at the tail, and the distribution of detection scores — the memory measurement that actually sees a `Mat` leak, and why the library emits no logs, no metrics and no traces of its own. The chapters between here and there change subject rather than direction: the frame source has been fully described, and what runs **on** a frame — models, faces, networks, markers, poses — is the next part of the book. Every detector in it is a native object with the ownership rules this chapter has just spent a whole slot enforcing.

PART V

Vision Applications

`scalacv-vision`: faces, deep networks, markers, poses, tracks, calibration and the visual-navigation front end. Every one of them owns native memory that the Java bindings will not free for you.

CHAPTER 23

Models: Fetching, Caching, Verifying

A detector is code plus weights, and only one of those two things fits in a jar.

Everything in the book so far has been self-contained. A blur is arithmetic over a neighbourhood, a contour is a walk along a boundary, a Hough transform is a vote in a parameter space. You added a dependency, called a function, and the answer came back — there was never a third thing that had to be true about the machine.

A learned detector breaks that. The code that runs YuNet is in the OpenCV native library you already have; the 232,589 bytes that make it a **face** detector rather than an untrained graph are not. They live in a file somebody else trained, under somebody else's licence, on a host neither you nor this library controls. Between "add the dependency" and "detect a face" there is now a step that fails for reasons unrelated to your program: the network is down, the mirror moved, the proxy rewrote the response, the container has no outbound route at all.

The tempting fix is to commit the file — 232 kB, working on every machine forever. scalacv does not, and the reason is not size: SFace is about 37 MB and even that would fit in a jar. It is that the moment a copy of YuNet lands in this repository, scalacv takes on the obligation to reproduce YuNet's MIT notice and hands it to every downstream redistributor of the jar. `THIRD-PARTY.md` records the model as fetched rather than vendored, and keeps OpenCV's attribution and YuNet's as separate lines. A runtime fetch leaves the obligation with whoever chose to redistribute the weights.

So the file arrives at run time, and this chapter is plumbing rather than algorithm: how to name a model, how to prove the bytes you got are the bytes you meant, where to put them, and what to do when none of that works. One example runs through it: the start-up path of a photo-triage service that detects faces on upload and matches them against an enrolled gallery. It needs two models before it can serve a request, and must refuse its health check until it has them.

> **NOTE**
> This part of the book opens the `scalacv-vision` module, and the split matters here. `Models` and `ModelSpec` are in the **core** `scalacv` artifact: fetching a file and proving it intact needs nothing else on the classpath. Everything the fetch is **for** — `FaceDetect`, `FaceRecognizer`, `Dnn`, `PoseEstimator`, `Segmenter`, `Cascades` — lives in `scalacv-vision`. Add `mvn"com.worxbend::scalacv-vision:0.4.2"` beside the core line and your platform's natives; Chapter 2 has the sbt and scala-cli spellings.

Three payloads, and only one is a download

Three kinds of binary payload get conflated, and they arrive by three different routes. Only one of them is missing.

Table 23-1. What ships in the jars, and what does not.

Payload	In the jars?	How you get it
OpenCV natives (`.so`, `.dylib`, `.dll`)	Yes	extracted on the first `OpenCv.load()`, into `~/.javacpp`
Haar and LBP cascade XML	Yes	extracted from the bytedeco jars on the first `Cascades.load(CascadeName.FrontalFaceAlt` — no network
ONNX models (YuNet, SFace, pose, segmentation, yours)	No	this chapter

The cascades are the only detector family that needs no provisioning step at all — a real property, even though they are markedly less accurate than YuNet. The natives are a cache, not a download: Chapter 2 covered the 196 MB `OpenCv.load()` unpacks once on Linux. Only the third row involves a URL.

A model is a spec, not a URL

The naive version of this is a string.

```
val url = "https://media.githubusercontent.com/media/opencv/opencv_zoo/main/" +
  "models/face_detection_yunet/face_detection_yunet_2023mar.onnx"
```

That string is four decisions collapsed into one, three of them invisible: where the file comes from, what it is called once it lands, what happens when that host is unreachable, and whether the bytes that arrived are the bytes anybody intended. `ModelSpec` separates them.

Example 23-1. The whole `ModelSpec` surface. A private constructor and two named builders.

```
final case class ModelSpec private (
    fileName: String,
    urls: Seq[String],
    sha256: Option[String],
    sizeBytes: Option[Long]
)

object ModelSpec:
  def apply(
      fileName: String,
      urls: Seq[String],
      sha256: String,
      sizeBytes: Option[Long] = None
```

```
  ): ModelSpec

  def unverified(fileName: String, urls: Seq[String]): ModelSpec
```

The constructor is `private`, so those two builders are the only ways in, and the asymmetry between them is the design. `ModelSpec(...)` takes `sha256` as a required `String`: the verifying form is what you get by default and by accident. `ModelSpec.unverified(...)` is the opt-out, and it says the word at the call site, in the diff, and in the review. Both reject an empty `fileName` and empty `urls` with an `IllegalArgumentException`, and `sizeBytes` must be positive if given — a spec with no source or no name is a programmer error, not a runtime condition. `urls` is tried first to last; `fileName` is fixed rather than derived from the URL, which is what makes caching possible, because without a name agreed in advance "is this already here?" has no answer.

`sizeBytes` earns its place by changing the message rather than the outcome. A wrong-size file fails the checksum anyway — but **"expected 232589 bytes for `face_detection_yunet_2023mar.onnx` but got 131"** sends the reader to the download, while **"SHA-256 mismatch"** invites them to suspect tampering. It is checked first, so it costs one `stat` rather than a full digest over a file that was never the model.

What `fetch` actually does

One method does the work.

```
def fetch(spec: ModelSpec, into: Path): Either[CvError, Path]
```

`into` is a **directory**, created if absent; the file is written to `into.resolve(spec.fileName)` and that path comes back. The sequence inside is worth stating in full, because every step is a defence.

1. **Is it already here?** A regular file at the target that still passes verification — the pinned size, then the pinned hash — is returned untouched and no socket opens. This is what makes it safe to call on every start-up.
2. **Create the directory.** A failure here is `Left(CvError.LoadFailed(into.toString, "could not create th` Only this step sits under that message; a wider `try` would report a refused connection as a directory problem.
3. **Try each URL in turn.** The first that downloads **and** verifies wins, and the fold stops there — the remaining mirrors are not contacted. Every failure is accumulated in order, so if all of them fail you get one `Left` listing what each one did.
4. **Download to a sibling temp file**, named `.model-<random>.part` in the target's own directory, so the final move stays on one filesystem.
5. **Verify the temp file**, before anything else looks at those bytes — with the message re-attributed to the URL it came from rather than the temp path, because the mirror is what you have to act on.

6. **Move it onto the target**, and only then. Then **delete the temp file** in a `finally`, whatever happened.

The last four steps are one property: an interrupted run, a full disk, a `SIGKILL` mid-transfer or a mirror that dies halfway can never leave a truncated model at the target path. Either the file at `into/<fileName>` is a verified model or it does not exist.

> **WARNING**
> An unreadable cache entry is a **miss**, not a failure. Hashing it opens the file, which can throw for reasons unrelated to the model — deleted between the `isRegularFile` check and the read, permissions lost, volume gone — and `fetch`'s signature promises to report failure as a `Left`. Re-downloading answers all of those, and a destination that really is unusable produces a `Left` naming the real problem.

Each mirror gets a 15-second connect budget and a 60-second request budget, and the request budget covers the whole exchange rather than idle time. That matters for the 37 MB model on a slow link: every mirror times out and you get a `Left` that reads like an outage but is a bandwidth problem. Redirects are followed — the client is built with `Redirect.NORMAL`. Do not wrap `fetch` in a retry loop; the retry across mirrors is the loop, and a second one on top multiplies the time to a clear failure.

The two specs that ship

Two models are pinned in the library itself, because two of its APIs are inert without them.

Table 23-2. The built-in specs, with the values the source declares.

	YuNet	SFace
Spec	`FaceDetect.modelSpec`	`FaceRecognizer.modelSpec`
File name	`face_detection_yunet_2023mar.onnx`	`face_recognition_sface_2021dec.onnx`
Size pinned	`FaceDetect.ModelSizeBytes` = 232589	none
Digest	`FaceDetect.ModelSha256`	`0ba9fbfa…4e79`
Mirrors	2, both on `media.githubusercontent.com`	1, a `github.com/…/raw/main/…` URL
Fetch	`FaceDetect.downloadModel(dir)`	`Models.fetch(FaceRecognizer.modelSpec, dir)`
Load	`FaceDetect.create(path, inputSize)`	`FaceRecognizer.load(path)`
Produces	boxes plus 5 landmarks per face	a 128-value `FaceEmbedding`

`FaceDetect.downloadModel(into)` is one line — `Models.fetch(modelSpec, into)` — and exists only as the discoverable name beside the detector that needs it. The two were separate implementations of the same download-verify-move dance until they were merged, which is how one of them ended up carrying a bug the other had already fixed. Once you fetch more than one model, use `Models.fetch` and a list of specs. YuNet's first mirror pins the exact commit that last touched the file, so those bytes cannot change underneath you; the second

follows `main`, so a repository reorganisation degrades to a fallback instead of an outage. Neither is trusted — the checksum decides, on both.

The 131-byte model

Large files in a Git repository are usually stored with Git LFS: the repository holds a small text **pointer**, and the real bytes live on a separate media host. The OpenCV Zoo keeps its `.onnx` files this way.

Fetch one from a `raw.githubusercontent.com` URL and you get HTTP 200 and the pointer. Nothing errors. You now have a 131-byte file called `face_detection_yunet_2023mar.onnx`, and the first thing to complain is OpenCV's ONNX importer, with a message about the graph rather than the download.

Both defences are in the source. `FaceDetect.ModelUrls` uses `media.githubusercontent.com/media/…`, the host that serves the object rather than the pointer — and that pointer at least carries an `oid sha256:` line, which is where `ModelSha256` was cross-checked from. And `FaceDetect.modelSpec` pins `sizeBytes`, so a pointer is reported as the wrong byte count rather than an opaque digest mismatch. SFace takes the other route: its single `github.com/…/raw/…` URL is followed through whatever redirect that host answers with, and the checksum decides what arrived. Expect this failure first in your own specs: a three-digit byte count is a pointer or an error page, not a corrupt model.

What the library knows how to decode

Two specs ship, but three further capabilities have a decoder in the library waiting for a model you supply. A tensor does not say what its numbers mean — `[1, 1, 17, 3]` is seventeen keypoints, not seventeen **named** keypoints — so scalacv splits the job: you bring the ONNX, and the library names the layout and turns the tensor into typed data.

Table 23-3. The model families with a decoder in the library.

Capability	Family	Input	Decoder, and the shape it wants
Face detection	YuNet	`Size(320, 320)`	`image.faces(detector)` — 15-column rows
Face recognition	SFace	fixed by the model	`rec.embed(image, face)` — 128 floats
Pose, regression	MoveNet-style	`Size(192, 192)`	`PoseEstimator.decode(out, size, KeypointLayout.Regression, PoseTopology.CocoBody17)` — `[1, 1, K, 3]`
Pose, heatmap	OpenPose-style	as documented	the same call with `KeypointLayout.Heatmap` — `[1, K, H, W]`

Hand landmarks	MediaPipe hand, as ONNX	`Size(224, 224)`	`PoseEstimator.decode(…, PoseTopology.Hand21)` then `GestureRecognizer`
Selfie segmentation	MODNet, MediaPipe selfie	`Size(256, 256)`	`Segmenter.decodeMask(out, size)` — `[1, 1, H, W]` or `[1, 2, H, W]`
Anything else	any ONNX export	as documented	none — `Dnn.fromOnnx`, and read the tensor yourself

A topology is not in the file, which is why `PoseTopology` is an argument: its `size` must match the model's keypoint count or `decode` fails by name rather than throwing OpenCV's total-size exception. The parameter that goes wrong silently is `swapRB`. Almost every published model was trained on RGB while OpenCV decodes to BGR, so `Image.estimatePose` and `Image.segment` default it to `true` while `Dnn.blobFromImage` keeps OpenCV's own `false`. Keypoints that are plausible but consistently off are this, first.

Everything fails as `LoadFailed`

One error case covers all of it. `CvError.LoadFailed(resource, details)` is a named resource that could not be resolved, loaded, or verified — a model file, a cascade, an ONNX network, a download. It is deliberately not `DecodeFailed`, which is about image **bytes**: an HTTP 404 for a model is not an image-decode failure and should not read like one. Chapter 6 laid out the hierarchy; this chapter uses one branch of it. The `resource` field tells you which stage failed, because it is set to a different thing at each one.

Table 23-4. Reading a `LoadFailed` from the download path.

`resource` is	Then `details` says
the directory you passed	`could not create the download directory: …` — `into` is not writable, or a file already sits at that path
`spec.fileName`	`could not be downloaded from any source.` followed by one indented line per mirror
a URL	what that one mirror did: `HTTP 404 from …`, a JDK timeout message, a size mismatch, or a digest mismatch
the model path	`there is no readable file at this path.` — `FaceDetect.create`'s own pre-check, before OpenCV sees it

Learn the two verification messages so you recognise them at three in the morning. The size mismatch is the one quoted above; the digest mismatch reads **"SHA-256 mismatch for `face_detection_yunet_2023mar.onnx`: expected …, got …. Refusing to load an unverified model."** Roughly the right size with the wrong digest means the publisher moved the file under the same name and your pin is stale — precisely the situation the pin exists to surface.

The edges are worth stating. `Models.fetch` sends a bare `GET` with no headers, so a model behind a token or a private bucket is out of reach. There is no proxy knob, no resume, no progress, and no size limit — a mirror that serves ten gigabytes writes ten gigabytes to disk and **then** fails the checksum. Nor is there locking between processes. All of that has the same workaround, and it is the next section.

`file://` is not a special case

The scheme dispatch inside `download` has two branches. `http` and `https` go through the timeout-bounded `HttpClient`; everything else falls back to `uri.toURL.openStream()`, because the JDK HTTP client does not serve `file://`. That one fallback is what makes offline deployment fall out of the design rather than being bolted onto it.

Example 23-2. The same code path on a connected machine and an air-gapped one.

```
val yunet = ModelSpec(
  fileName = "face_detection_yunet_2023mar.onnx",
  urls = Seq(
    "file:///opt/models/face_detection_yunet_2023mar.onnx",  // the baked-in
copy, if there is one…
    "https://media.githubusercontent.com/media/opencv/opencv_zoo/main/" +
      "models/face_detection_yunet/face_detection_yunet_2023mar.onnx"  // …
otherwise the network
  ),
  sha256 = FaceDetect.ModelSha256,
  sizeBytes = Some(FaceDetect.ModelSizeBytes)
)
```

A machine that has the file never opens a socket; a machine that does not falls through to the mirror. Both run the same verification, so a container whose `/opt/models` layer is corrupt, or whose read-only mount points at last quarter's file, fails at start-up with a named error instead of misbehaving at inference time. The pin is not only a network defence: it states which bytes this build was tested against.

It is also the answer to every limitation above. A model behind an artifact registry, a 37 MB file on a link too slow for the request budget, a proxy that needs headers — fetch it with a tool that can do those things at image-build time and point a `file://` spec at the result. You keep the integrity gate and lose nothing. The container shape then mirrors the native cache, which Chapter 41 covers in full: bake the models into an image layer for a self-contained cold start, or mount them from a shared volume for a thin image.

> **NATIVE MEMORY**
> Nothing in `Models` owns native memory. Everything it **unlocks** does. `FaceDetect.create` hands back `Either[CvError, Managed[FaceDetectorYN]]` and `Dnn.fromOnnx` hands back `Either[CvError, Managed[Net]]`; `FaceRecognizer.load` hands back an `Either[CvError, FaceRecognizer]` whose `FaceRecognizer` is an `AutoCloseable` wrapping a

`Managed[FaceRecognizerSF]`. Only `Mat`, `VideoCapture` and `VideoWriter` expose a public `release()`; the other 185 generated types — these three among them — do not, so `Managed` frees them through `Releasable.nativeHandle`, which reads the address and disarms the binding's unconditional `finalize()` **before** freeing the pointer. Without that disarm a released `FaceDetectorYN` is a live double-free, and because a DNN allocates enough to make the collector run mid-suite, this library's own face tests are where that SIGSEGV first appeared, on the `Finalizer` thread, inside `Java_org_opencv_objdetect_FaceDetectorYN_delete`. Build one detector at boot and keep it; a service that builds one per request allocates a network per request.

Pin the model, or pin nothing at all

`Models.fetch` keys its cache on `into.resolve(spec.fileName)` and nothing else, and the test for "is the file on disk still good?" is "does it match what the spec pins". Follow both halves of that sentence through the `unverified` case and the consequence is larger than a lost integrity check. With a hash pinned, a model upgrade rolls itself out: same name, new hash, the stale file fails verification and is replaced. With `ModelSpec.unverified`, the cache test is vacuously true for **anything** already at that path. Ship new weights under the old file name and every replica that has ever run keeps answering with the old ones, with no error anywhere and no request to the URL.

> **CAUTION**
> A silently stale model is worse than a broken one. A broken model fails at boot, on one replica, with a message naming the file — you find out in minutes and nothing downstream believed anything false. A stale model serves plausible answers indefinitely: accuracy drifts, your recognition threshold stops meaning what it meant, and the first evidence is a ticket about a result nobody can reproduce, because the replica that produced it has since been rescheduled onto a node with a different cache.

The habit that fixes it: **put the version in the file name.** `yolov8n-v3.onnx`, not `yolov8n.onnx`. Two versions then coexist on disk, a rollback is a configuration change rather than a cache purge, and a rolling deploy never has two replicas fighting over one path. Pin the hash as well, because the digest is the only thing that survives a model being republished under a name you already trusted. Then record what ran: a `ModelSpec` is a value with a `fileName` and an `Option[String]` digest, both cheap to log at start-up and stamp onto whatever your service emits beside a result. A stored `FaceEmbedding` is meaningless without knowing which SFace produced it — a gallery enrolled under one version and queried under another degrades quietly rather than failing.

> **TIP**
> A converted model's digest is **yours**. `tf2onnx` is not byte-reproducible across tool versions, so if you converted a TFLite export — which is what MediaPipe publishes, and OpenCV's ONNX importer will not read — pin the artefact you actually tested and give it a versioned name. Conversion is a build-time step; do not attempt it at run time.

Licences travel with weights, not with the library

Say this plainly, because it is the part discovered during a legal review rather than during development. scalacv is Apache-2.0. The models are not scalacv. YuNet is MIT (Shiqi Yu), and `THIRD-PARTY.md` carries that attribution separately from OpenCV's own. SFace comes from the same OpenCV Zoo under the terms published there, which scalacv does not restate for you; a model you found on a hub carries whatever its author chose — sometimes Apache-2.0, sometimes MIT, sometimes non-commercial or research-only with no exception for what you are building. Bake a model into a container image and you are redistributing it, terms and all. Read them before the file goes into the image, not after the image goes to a customer. The download is the point at which that decision becomes yours — which is why it is a download.

The triage service, at boot

Everything above assembles into one function. The triage service needs both models on disk and both handles built before it answers a health check, and it must say which stage failed if it cannot.

Example 23-3. Fetch, then load, then gate readiness on the result.

```
import java.nio.file.Path
import scalacv.*
import org.opencv.objdetect.FaceDetectorYN

final case class Triage(detector: Managed[FaceDetectorYN], recognizer:
FaceRecognizer)

/** Everything that must exist before this process serves a request. */
def warmUp(modelDir: Path, expected: Size): Either[CvError, Triage] =
  OpenCv.load()                                    // idempotent, thread-safe,
extracts on first call
  for
    yunet      <- Models.fetch(FaceDetect.modelSpec, modelDir)
    sface      <- Models.fetch(FaceRecognizer.modelSpec, modelDir)
    detector   <- FaceDetect.create(yunet.toString, expected)
    recognizer <- FaceRecognizer.load(sface.toString)
  yield Triage(detector, recognizer)
```

The comprehension is over `Either`, so the first `Left` short-circuits carrying its own `resource` and `details` — you never guess whether the download or the load failed. Both fetches are idempotent, so every boot after the first costs two digests and no sockets. Nothing here reads an `Image`, so there is no consumed handle to trip over, and both native owners leave in the `Triage` value — the only place anything can release them later.

Example 23-4. A `LoadFailed` at boot is a refusal to start, not a warning to log.

```
warmUp(Path.of("/var/lib/triage/models"), Size(320, 320)) match
  case Right(ready) =>
```

```
    server.run(ready)                                  // your service, holding both
handles for its life

  case Left(CvError.LoadFailed(resource, details)) =>
    System.err.println(s"triage cannot start: '$resource' --- $details")
    sys.exit(78)                                       // EX_CONFIG: this will not
fix itself on retry

  case Left(other) =>
    throw other
```

The mistake this replaces is the one that looks more robust: catch the failure, log it, start anyway, resolve the model lazily on the first request that needs a face. That process passes its health check, joins the load balancer, and then fails every request with an error about a missing file — on every replica at once, minutes after the deploy that caused it. A process that refuses to start fails one replica at a time, in front of the deploy that broke it, with the file name in the message. The final `case Left(other) => throw other` matters too: `CvError` is a `RuntimeException` hierarchy, so anything that is not a `LoadFailed` — `NativesMissing`, say — reaches the top of `main` with its own message rather than one exit code.

`Size(320, 320)` is the frame size you expect, not a constraint. `FaceDetect.create` requires an `inputSize` because `FaceDetectorYN`'s constructor does, but `FaceDetect.detect` re-sets it on every frame, which is what lets differently sized images work at all. It still matters — YuNet's anchors are laid out for it — so pass the size you expect and off-size detections stay well calibrated. The corollary from the same mechanism: a detector is mutated by every `detect` call and is **not** safe to share across threads. Give each worker its own, built from the same verified file.

What this bought

Three properties, from one small object. The model that loads is the model you pinned, on every machine and every boot, or the process names the stage that failed and stops. The offline case is the online case with a different first URL. And the licence stayed with whoever chose to ship the weights.

What none of it tells you is whether the bytes are a network OpenCV can import: `Models.fetch` moves bytes and checks a digest, while whether the graph loads is discovered by `FaceDetect.create`, `FaceRecognizer.load` or `Dnn.fromOnnx`, each with its own `Left`. Chapter 24, *Face Detection*, picks up the file this chapter put on disk — YuNet against the Haar cascades it replaces, the fifteen columns a detection row carries, and why a detector is something you build exactly once.

CHAPTER 24

Face Detection

Two detectors, twenty years apart, and why you build either of them exactly once.

A face is the first thing anyone tries to detect, and it is a fortunate choice. Faces are roughly rigid, roughly the same shape, roughly the same arrangement of dark and light in every human being alive, and there are enormous public datasets of them. That made face detection the first object detection problem to be considered solved for practical purposes — and it was solved twice: once with hand-designed features and a clever early-exit search, and again with a convolutional network small enough to fit in a quarter of a megabyte. scalacv exposes both, and they are not interchangeable.

The interesting difficulty is not which of them finds more faces — that question has a boring answer, and this chapter gives it. It is that both detectors are native objects with a lifetime, and both are the kind of native object people build inside a loop. A `Mat` that leaks is six megabytes; a detector that leaks is a parsed model, and building one per frame also pays to parse the model per frame, so the leak and the performance bug arrive together and disguise each other. Chapter 5 gave you `Managed`, and Chapter 23 gave you the model file.

There is a second, quieter difficulty: neither detector reports failure the way you would expect. `CascadeClassifier` does not throw for a path it cannot read — it constructs an empty classifier that detects nothing for the rest of the process, which looks exactly like an empty room. `FaceDetectorYN` does throw for a bad model, but its `detect` returns a status flag rather than a face count, and reading it as a count reports one face on every successful call. Both mistakes produce plausible output. scalacv catches them at the boundary, and it is worth knowing what it is catching.

Both detectors live in `scalacv-vision`, not in the core artifact — `Cascades.scala` and `FaceDetect.scala` are both in that module — so every listing in this chapter needs `com.worxbend::scalacv-vision:0.4.2` on the classpath beside the core dependency of Chapter 2. The vision package is `scalacv.vision`: add `import scalacv.vision.*` for `detectHaar`, `faces` and `markFaces`, alongside `import scalacv.*` for core.

Start with the older detector, because its failure modes are the ones you will be talking a colleague out of.

What Viola-Jones actually does

The 2001 Viola-Jones detector is built from three ideas that fit together so well it is still worth reading about, even though you will rarely deploy it now.

The first is the *integral image*. Precompute, for every pixel, the sum of every pixel above and to the left of it; the sum of any axis-aligned rectangle is then four table lookups and three additions, regardless of how large the rectangle is. The detector's features are differences of adjacent rectangle sums — a dark band above a light band, which is what an eye socket over a cheekbone looks like — so every feature costs a constant handful of memory reads at any scale.

The second is that one such feature is a terrible classifier, barely better than a coin flip. Boosting weights hundreds of these weak classifiers into one strong classifier, choosing at each round the feature that best corrects the mistakes of the rounds before it.

The third idea made it real time on hardware of the period. Instead of running one enormous strong classifier over every window, arrange the strong classifiers in a *cascade*: a very cheap stage first, then a slightly more expensive one, and so on. A window that fails any stage is rejected immediately and never sees another. The overwhelming majority of windows in any photograph are not faces and are thrown out by the first two stages, which between them evaluate a handful of features, so the average cost per window is close to the cost of the cheapest stage. That is the whole trick, and it is why the file is called a cascade.

Repeat the scan at a pyramid of scales and you have a face detector that ran at video rates on a 2001 desktop CPU, with no GPU and no model download. In 2026 it remains useful for two reasons: it needs no model file you have to ship, and it is understandable end to end. It is the fallback, not the default. It wants upright, roughly frontal, evenly lit faces, and it produces false positives at a rate that will surprise you the first time you point it at foliage.

Loading a cascade without lying to yourself

The cascades are XML files inside the bytedeco per-platform classifier jar, under `share/opencv4/haarcascades/`. They are a classpath resource, and OpenCV is a C++ library that only knows filesystem paths, so they have to be extracted before they can be loaded. `Cascades` does that through javacpp's resource cache, and needs no native library to do it — `Cascades.resolve` works before `OpenCv.load()` has been called.

scalacv names the bundled cascades with a type rather than a string because of the silent failure above: a mistyped path produces a classifier whose `empty()` is `true` and whose every `detectMultiScale` returns nothing. `CascadeName` removes the typo, and `Cascades.load` checks `empty()` afterwards anyway and returns a `Left` — releasing the real-but-useless handle first, because a classifier that loaded no model still allocated one.

Example 24-1. A cascade is loaded by name, scoped, and used inside its scope.

```
import scalacv.*

OpenCv.load()

Cascades.load(CascadeName.FrontalFaceAlt).foreach { detector =>
```

```
  detector.use { classifier =>
    Image.reading("group.jpg") { photo =>
      val faces: Seq[Rect] = photo.detectHaar(classifier)
      photo.copy.drawRects(faces, Scalar.Green).write("group-haar.png")
    }
  }
}
```

`Cascades.load` hands back an `Either[CvError, Managed[CascadeClassifier]]`, and `use` releases the classifier on every exit path. Keep the detection inside that scope: `detectHaar` takes the raw `CascadeClassifier` rather than the `Managed` — a defaulted overload for both is not expressible — so the spent-handle guard only protects you while the `Managed` is in scope. The `.copy` is there because drawing consumes an `Image` (Chapter 4) and `photo` belongs to the `reading` scope; the copy is consumed by `drawRects` and released by `write`.

`CascadeName` covers the faces — `FrontalFaceAlt` (the usual first choice, with markedly fewer false positives), `FrontalFaceAlt2`, `FrontalFaceDefault` (the original Viola-Jones cascade, fast and permissive) and `ProfileFace` — the features (`Eye`, `EyeTreeEyeglasses`, `LeftEye2Splits`, `RightEye2Splits`, `Smile`), the bodies (`FullBody`, `UpperBody`, `LowerBody`) and `RussianPlateNumber`. The LBP cascades and the two cat detectors are deliberately absent, and both remain reachable through `Cascades.loadFrom`, which takes a filesystem path and applies the same `empty()` check.

WARNING

The `windows-x86_64` bytedeco jar ships an empty `share/` directory and no cascade XML at all, unlike every other platform. `Cascades.resolve` and `Cascades.load` can only return a `Left(CvError.LoadFailed(...))` there, and the message says so in those words rather than leaving you to guess. If you target Windows, ship `haarcascade_frontalface_alt.xml` with your application and swap in `Cascades.loadFrom("cascades/haarcascade_frontalface_alt.xml")` — it returns the same type, so nothing else in your code changes.

NATIVE MEMORY

`CascadeClassifier` is one of the 185 generated OpenCV binding types with no public `release()`. Its `delete(long)` is private, so scalacv reaches it through `Releasable.nativeHandle`: read the address, disarm the binding's unconditional `finalize()`, *then* free the pointer. Without that disarm a released classifier is a live double free, and the crash arrives from the JVM's own Finalizer thread with no frame of yours on the stack.

A leaked classifier costs the parsed cascade — every stage, every weak classifier, every feature rectangle — held in native memory behind a Java object small enough that the collector will never feel it. One is a rounding error. One per frame is a service that dies overnight with a flat heap graph, the failure Chapter 1 opened with. `Cascades.load(...).map(_.use(...))` is the shape that releases its registered handles; `Managed.scope` is the shape for holding several detectors at once.

Running the cascade

`image.detectHaar(classifier)` returns `Seq[Rect]` — plain immutable Scala data, copied out of the `MatOfRect` OpenCV filled in and released before the method returns, so the rectangles stay valid after everything involved has been freed. The receiver is borrowed, not consumed. The mid-level form, `mat.detect(classifier, ...)`, is the same method on a raw `Mat` with the same defaults.

Three parameters control the search, and their defaults are conservative.

Table 24-1. The three knobs on Haar detection, and what each one trades.

Parameter	Default	What it trades
`scaleFactor`	`1.1`	How much the detection window grows per pyramid level. Just above 1 — say `1.05` — means more levels, so more windows examined, so more faces found and a slower scan. A value at or below 1 would never terminate, and is rejected by a `require`.
`minNeighbors`	`3`	How many overlapping detections a candidate needs before it is reported. Raise it to 5 or 6 to kill false positives; lower it when real faces are being missed. Cannot be negative.
`minSize`	`None`	Objects smaller than this are never examined. Setting it is the cheapest speed-up available, because it removes whole pyramid levels from the search rather than filtering results afterwards.

There is no `maxSize`. OpenCV's underlying call has one and scalacv does not expose it, because an upper bound on face size is a common source of detectors that mysteriously stop working when someone walks towards the camera. Filter large boxes in Scala instead; `Rect.area` is right there.

Cascades want a single-channel, contrast-normalised image. A colour `Mat` works and is slower; a backlit one works badly. The preparation is two transforms, and because both consume the image you have to own the result:

Example 24-2. Preparing the frame the way the cascade wants it.

```
Image.reading("group.jpg") { photo =>
  val prepared = photo.copy.gray.equalizeHist
  try prepared.detectHaar(classifier, scaleFactor = 1.05, minNeighbors = 6,
minSize = Some(Size(40, 40)))
  finally prepared.close()
}
```

`prepared` is an owned `Image` that nothing else will release — `gray` and `equalizeHist` each consume their receiver and hand back a new one, and `detectHaar` only borrows. Forgetting

the `finally` leaks one full-size greyscale frame per call: the leaked detector in different clothes.

YuNet: the detector to actually use

YuNet is a small convolutional network — 232,589 bytes of ONNX — that `FaceDetect`'s own documentation calls *far more accurate and far faster* than the Haar cascades. Take the speed half of that as the library's stated position rather than a figure you can quote: scalacv publishes no benchmark that times the two detectors against each other, so this book will not invent one. The accuracy half you will see for yourself inside a dozen frames. YuNet tolerates pose, expression and lighting that send a cascade home empty, reports a calibrated confidence per detection, and returns five facial landmarks that Chapter 25 will need. It is the detector to reach for; the cascades remain for heritage and for environments where no model file can be shipped. The model is not vendored in this repository, and that is a licensing decision rather than a size one.

Why the model is a download and not a jar

YuNet is MIT licensed (Shiqi Yu). Committing a copy here, or shipping one inside a published jar, would oblige scalacv to reproduce that notice everywhere the jar goes; a runtime download keeps the obligation with whoever redistributes it. It is recorded in `THIRD-PARTY.md`, and it is not an accident to be tidied away by committing the file.

A model is executable content fetched from a host nobody here controls, so `FaceDetect.downloadModel(into)` — an alias for `Models.fetch(FaceDetect.modelSpec, into)` — tries `FaceDetect.ModelUrls` in order, checks the size against `ModelSizeBytes` and then the SHA-256 against `ModelSha256`, and moves the file into place from a temp file beside it only once both pass. An interrupted run cannot leave a truncated model behind, and a mismatch is a `Left` carrying both digests. `into` is a directory, created if absent; the file name is fixed at `FaceDetect.ModelFileName` — `face_detection_yunet_2023mar.onnx` — and that fixity is what makes the already-downloaded check possible at all. The check is a `stat` and a digest of a file already on disk, so calling `downloadModel` on every start-up is the intended use.

Both mirrors are `media.githubusercontent.com` rather than `raw.githubusercontent.com`, and that is load-bearing: the OpenCV Zoo keeps its `.onnx` files in Git LFS, and the `raw` host serves the 131-byte LFS *pointer* for them — with HTTP 200, so nothing looks wrong until the network fails to load. The pinned size check turns that into "expected 232589 bytes, got 131" instead of a hash mismatch that invites you to suspect tampering.

`FaceDetect.create` builds the detector:

```
def create(
    modelPath: String,
    inputSize: Size,
```

```
    scoreThreshold: Float = 0.9f,
    nmsThreshold: Float = 0.3f
): Either[CvError, Managed[FaceDetectorYN]]
```

`inputSize` does not constrain what you may later detect on — see below — but it still matters: YuNet's anchors are laid out for it, so passing the size of the frames you actually expect keeps detections on off-size images well calibrated. `scoreThreshold` is the minimum confidence reported, inheriting OpenCV's own default of 0.9; lowering it catches small and profile faces at the cost of false positives. `nmsThreshold` is the IoU above which two overlapping boxes are treated as the same face and the weaker one dropped. A zero side or a threshold outside `[0, 1]` is an `IllegalArgumentException` — those are typos, not runtime conditions. A missing, unreadable or unimportable model is a `Left`: unlike `CascadeClassifier`, `FaceDetectorYN.create` genuinely throws for a bad model, and an unhandled `CvException` out of a constructor is not a useful failure for someone who mistyped a path.

Four things about the underlying `FaceDetectorYN` are easy to get wrong, and `FaceDetect` handles each of them rather than leaving it to you.

- **The input size is fixed at construction and enforced at detect time.** OpenCV's `detect` checks the frame size for equality and throws a `CvException` if a later frame differs by a single pixel — which is what happens the first time you feed it a resized frame, or a webcam that renegotiated its resolution. `FaceDetect.detect` calls `setInputSize` for every frame, unconditionally, so any `Mat` works. The cost is that a detector is stateful and **not safe to share across threads**. Give each thread its own.
- **`detect` returns a status flag, not a face count** — `1` when the network ran, `0` when the input was empty. Reading it as a count silently reports one face on every successful call. The count is `faces.rows()`.
- **No faces means a 0×0 result Mat**, not an N×15 Mat with zero rows. A decode loop that trusts `cols()` without checking `empty()` first reads column 14 of a Mat with no columns.
- **A detection row has 15 columns**, all `CV_32F`. `FaceDetect.ResultColumns` is that 15, and any other width raises `CvError.NativeCall` naming the mismatch rather than decoding garbage.

`FaceDetect.detect` also refuses an image that is not 8-bit three-channel BGR, with an `IllegalArgumentException` naming the actual type. YuNet's blob step needs `CV_8UC3`, and without the check the failure surfaces from inside the DNN module as a message about layer shapes that says nothing about the real mistake. Convert a greyscale frame back with `image.convert(ColorConversion.GrayToBgr)` first.

The Face record

A detection comes back as a `Face`: plain immutable Scala data copied out of OpenCV's result Mat, which is released before `detect` returns. There is no native handle to own, which is why the result is a `Seq[Face]` and not a `Managed`.

`box` is a `Rect`, `score` is the model's confidence in `[0, 1]`, and `landmarks` is exactly five `Point`s, always in this order: `rightEye`, `leftEye`, `noseTip`, `rightMouthCorner`, `leftMouthCorner`. Each has a named accessor. "Right" is the *subject's* right, so the right eye appears on the left of the image — a distinction that costs an afternoon exactly once.

The box is **not** clipped to the frame. YuNet regresses boxes from anchors, so a face at the edge of the image legitimately yields a negative `x` or `y`, or a box running past `width` / `height`. `Image.crop` and `Mat.submat` both reject such a rectangle, so the naive crop fails on the one frame where somebody walks in from the side:

```
val thumbs = faces.map(f => frame.copy.crop(f.box))   // throws on any face at an
edge
```

`Face.clippedBox` is the fix, and it is a method rather than a footnote because every caller would otherwise write the four-way `min` / `max` by hand and put an off-by-one in it. It returns the intersection with the frame as an `Option[Rect]`, `None` when the box lies entirely outside. The intersection is half-open on both axes, matching what `submat` expects, so a box that merely touches an edge answers `None` rather than a zero-extent `Rect` that `submat` would throw on.

Example 24-3. Cropping every detected face safely.

```
val boxes  = frame.faces(detector).flatMap(_.clippedBox(frame))
val thumbs = boxes.map(r => frame.copy.crop(r))
```

The `clippedBox(image)` overload is `clippedBox(image.width, image.height)`; the image is only queried for its size, so it stays alive and owned by you.

faces and markFaces

Two extension methods on `Image` carry the whole high-level story, and `import scalacv.*` brings them in. `image.faces(detector)` runs the detection, overloaded on both the raw `FaceDetectorYN` and the `Managed[FaceDetectorYN]` the loaders hand back; prefer the `Managed` form, since the spent-handle guard then travels with the argument instead of being discarded by a bare `.get`. `image.markFaces(faces, color)` draws a box per face and a dot per landmark, defaulting to `Scalar.Green`.

`faces` borrows. It reads the image and returns data; it does not consume the receiver and does not release the detector. That matters because the obvious defensive move is wrong:

```
photo.markFaces(photo.copy.faces(detector)).write("marked.png")   // the copy is
never released
```

The `.copy` allocates a full frame, hands it to a method that only reads it, and nothing ever frees it — a leak of one frame per call, hidden inside a line that looks careful. Because `faces` borrows, there is nothing there to defend against:

Example 24-4. Detect on the borrowed image, then consume it to draw.

```
def annotate(in: String, out: String, detector: FaceDetectorYN): Either[CvError,
Int] =
  Image
    .reading(in) { photo =>
      val faces = photo.faces(detector)
      photo.copy.markFaces(faces).write(out).map(_ => faces.size)
    }
    .flatMap(identity)
```

Here the `.copy` *is* earned: `markFaces` consumes its receiver, `photo` is owned by the `reading` scope, and the annotated image `markFaces` returns is released by `write`. The `flatMap(identity)` flattens `reading`'s `Either` around the block's own.

Wiring it up needs the model, the detector and one scope:

Example 24-5. A still photograph, end to end.

```
import java.nio.file.Path
import org.opencv.objdetect.FaceDetectorYN
import scalacv.*

OpenCv.load()

val cache = Path.of(sys.props("user.home"), ".cache", "scalacv-models")

val marked: Either[CvError, Int] =
  for
    model    <- FaceDetect.downloadModel(cache)
    detector <- FaceDetect.create(model.toString, Size(1280, 720))
    count    <- detector.use(d => annotate("group.jpg", "group-yunet.png", d))
  yield count
```

Every stage that can fail is a `Left`: no network, a corrupt download, a model OpenCV will not load, an unreadable photograph, an unwritable output. A machine with no network degrades to a message.

Choosing between them

Table 24-2. The two detectors, side by side.

	Haar cascade	YuNet
API	`Cascades.load`, `image.detectHaar`	`FaceDetect.create`, `image.faces`

Result per face	`Rect`, nothing else	`Face`: box, `score`, five landmarks
Model source	bundled in the bytedeco classifier jar; absent on `windows-x86_64`	232,589-byte ONNX, downloaded and SHA-256 verified at runtime
Speed	a pyramid scan whose cost rises sharply as `scaleFactor` falls	one fixed-cost forward pass, whatever the scene holds; documented as far faster, not measured here
Accuracy	more false positives; needs `minNeighbors` tuning per scene	markedly higher, with a calibrated confidence you can threshold
Pose tolerance	upright and roughly frontal; a separate `ProfileFace` cascade for the rest	tolerant of moderate yaw, roll and expression
Preprocessing	wants greyscale and `equalizeHist`	wants 8-bit BGR `CV_8UC3`, nothing else
Threading	one classifier per thread	one detector per thread — `detect` mutates its input size
Failure when unavailable	`Left` from `Cascades.load`, naming the platform	`Left` from `downloadModel` or `create`, naming the stage

The rule of thumb is short: if you can fetch a 232 kB file, use YuNet. Use a cascade when you cannot, or when you are detecting one of the non-face objects `CascadeName` covers and have nothing better trained.

Build once, detect many

The most common performance bug in this area is not a slow algorithm but a detector constructed inside the frame loop. It is easy to write, it produces correct output, and it costs more than the detection itself by a wide margin:

```
camera.foreach() { frame =>
  Cascades.load(CascadeName.FrontalFaceAlt).foreach {          // wrong: per
frame
    _.use(c => log(frame.detectHaar(c).size))
  }
}
```

That parses an XML cascade tree, allocates the native classifier and frees it again — thirty times a second, for a scan that costs a fraction of the parse. Only the resource extraction is spared: javacpp caches the file it wrote, so `Cascades.resolve` after the first call is a lookup. The parse is not cached, and it is the expensive half. The YuNet version of the same mistake is worse: `downloadModel` inside the loop re-digests 232,589 bytes on every frame before `create` re-inflates the ONNX graph. The model fetch and the detector construction belong at start-up; the loop should contain nothing but detection and drawing.

Example 24-6. Faces in a video, with the detector hoisted and the results thresholded twice.

```
val recorded: Either[CvError, Long] =
  for
    model    <- FaceDetect.downloadModel(cache)
    detector <- FaceDetect.create(model.toString, Size(1280, 720), scoreThreshold
= 0.6f)
    written  <- detector.use { d =>
                  Camera
                    .usingFile("interview.mp4") { camera =>
                      camera.recordTo("interview-faces.avi") { frame =>
                        val faces = frame.faces(d).filter(_.score >= 0.9f)
                        frame.markFaces(faces, Scalar.Green)
                      }
                    }
                    .flatMap(identity)
                }
  yield written
```

Two thresholds, deliberately. The detector is built with `scoreThreshold = 0.6f` so marginal detections reach your code at all, and the drawing filter keeps only those at or above `0.9f`. Adopt that split as a habit: the network's threshold decides what you are allowed to see, and a plain Scala `filter` decides what you act on. Raising the network's threshold throws information away inside a native call you cannot inspect; filtering a `Seq[Face]` is a decision you can log, tune, or make per-region without rebuilding anything.

Everything in that loop has an owner. `Camera.usingFile` closes the capture on every path. `recordTo` reads every frame as an owned `Image`, hands it to the transform, writes what comes back and closes it. `faces` borrows the frame, `markFaces` consumes it and returns the annotated image that `recordTo` releases — so the frame's Mat is written and freed exactly once, with no copy anywhere in the loop. `detector.use` frees the detector when the video ends, by exception included.

NATIVE MEMORY

`FaceDetectorYN` is also one of the 185 types with no public `release()`, and it is where the double-free hazard was first found: a DNN allocates enough to make the collector run mid-suite, so a released detector whose `finalize()` had not been disarmed crashed the test JVM from the Finalizer thread, inside `Java_org_opencv_objdetect_FaceDetectorYN_delete`. That is why `Releasable.nativeHandle` disarms before it frees.

The 232,589-byte file is only the serialised weights. A live detector also holds the inflated graph and the per-layer blobs, sized for whatever input size was last set, and none of that appears on the Java heap either. The `given Releasable[FaceDetectorYN]` in `FaceDetect` and the `given Releasable[CascadeClassifier]` in `Cascades` are both public, so a detector you build yourself — from a `MatOfByte` buffer, or with a `topK` other than OpenCV's default of 5000 — is managed on the same terms with `import FaceDetect.given`.

What actually breaks these detectors

Both detectors fail in the same directions. YuNet fails later along each of them, and it tells you so through the score rather than by returning nothing.

Profile. A face turned past roughly forty-five degrees stops looking like the thing either detector was trained on. For a cascade the answer is a second pass with `CascadeName.ProfileFace`, run over the frame and its horizontal mirror, since that cascade is trained on one side only. For YuNet, lower `scoreThreshold` and filter on `score` and box geometry afterwards.

Occlusion. A hand across the mouth is survivable; sunglasses are worse than they look, because the eye region carries most of a cascade's early-stage discrimination and is where YuNet's first two landmarks sit. A mask removes the mouth corners and shifts the box, so downstream alignment that trusts `noseTip` and both mouth corners degrades before the detection itself does.

Small faces. This is the one you control, and the trade is direct: doubling the frame's long side quadruples the pixels examined. On a 4K stream where the people you care about are a few metres from the camera, downscale to 720p and set a `minSize` — the same faces, several times faster. For a crowd shot, do the opposite: keep the resolution, leave `minSize` at `None`, and lower `scaleFactor` towards `1.05`.

Backlight. A subject against a window is close to a silhouette, and a cascade reads silhouettes as absence. `equalizeHist` recovers a surprising amount of it for almost nothing, which is why it is the standard preparation. YuNet is more tolerant but not immune, and when backlit frames specifically lose faces the fix is exposure upstream, not a detector parameter.

Rotation in the image plane. Neither detector handles a head tilted past about thirty degrees, and neither has a parameter for it. For a camera mounted at an angle, rotate the frame before detection (Chapter 10) and rotate the boxes back afterwards.

The detector is rarely the thing to tune first. Frame size, `minSize` and lighting move the numbers further than any threshold does, in directions you can reason about.

Where this goes next

You now have a box, a confidence and five landmarks per face, all of it plain Scala data that outlives every native object involved. The landmarks are not decoration: they are the input to alignment, and alignment is what turns "there is a face here" into "this is the same face as that one."

Chapter 25, *Face Recognition*, picks up there. `FaceRecognizer.load` returns a recognizer holding the SFace network — a second model download, and at roughly 37 MB a far larger one — and its `embed(image, face)` takes the same `Face` this chapter produced, aligns and crops it by those five landmarks, and hands back a `FaceEmbedding` wrapping a `Vector[Float]` with no native memory in it at all. `Gallery.empty.enroll(name, embedding)` accumulates the

people you know, and `identify(embedding)` answers the question detection cannot: not "is there a face here" but "whose".

CHAPTER 25

Face Recognition

From "there is a face" to "this is the same face".

The detector in Chapter 24 answers a question about geometry. It hands back a `Face` — a box, five landmarks, a score — and every one of those numbers is a statement about pixels. Nothing in it knows who the person is, and nothing in it can be made to know: YuNet was trained to find faces in general, and it finds yours exactly as readily as it finds a stranger's.

The obvious way to close that gap is to train a classifier with one output per person. It is also the wrong shape for almost every system anyone actually builds. A classifier has to be retrained when somebody joins, retrained when somebody leaves, and it needs enough labelled images of each person to learn a decision boundary — which is precisely what you do not have on the day a new employee arrives with one photograph from their pass. Worse, a fixed set of outputs cannot say "none of these"; softmax always sums to one, so the stranger who walks past the camera is assigned, with confidence, to whichever of your staff he most resembles.

Recognition is done the other way round. A network is trained once, on faces it will never see again, with an objective that has nothing to do with your particular people: map a face image to a fixed-length vector such that two pictures of one person land close together and pictures of different people land far apart. The identities in the training set are scaffolding; what is kept is the mapping. Feed it a face it has never encountered and it still places that face somewhere sensible, because it learned a geometry rather than a list.

That turns identification into a lookup. Store one vector per enrolled person, embed the face in front of you, take the nearest stored vector, and accept it only if it is near enough. Adding a person is appending a row. Removing one is deleting a row. And "none of these" falls out of the threshold rather than having to be modelled — which, as this chapter will keep insisting, is the case people forget to write and the case that decides whether the system is usable at all.

The embedding, and what it is made of

scalacv wraps OpenCV's SFace recogniser, `org.opencv.objdetect.FaceRecognizerSF`, behind `FaceRecognizer`. Like the detector of Chapter 24 it lives in `scalacv-vision`, so everything in this chapter needs that module on the classpath as well as core. It produces a `FaceEmbedding`:

```
final case class FaceEmbedding(values: Vector[Float])
```

That is the whole type. A hundred and twenty-eight `Float`s in an immutable `Vector`, with a `require` that rejects an empty one. There is no native handle in it, no `Mat`, nothing to release.

This matters more than it looks: an embedding outlives every native object that produced it, so you can free the recogniser, free the frame, shut the camera down, and the vector still compares. It serialises like any other `Vector[Float]`, and it goes into a database column without ceremony.

The three-step model is worth holding in your head, because each step lives in a different part of the library and the middle one is the only one that needs a neural network:

```
detect a face   ->   embed it into a 128-vector   ->   compare or look it up
 YuNet, Face          SFace, FaceEmbedding           cosineSimilarity, Gallery
```

Two models, and neither ships with the library

Recognition needs both networks. The YuNet detector from Chapter 24 finds the face and, critically, supplies the five landmarks SFace uses to align it. SFace itself is a separate ONNX file, `face_recognition_sface_2021dec.onnx`, about 37 MB, from the OpenCV Zoo. Neither model is committed to this repository: `THIRD-PARTY.md` records YuNet as downloaded rather than vendored precisely because redistributing it would oblige scalacv to reproduce its notice, and no model file ships in the published jars at all.

`FaceRecognizer.modelSpec` is the registry entry for SFace: a `ModelSpec` like any other, carrying a file name, a list of URLs, and a pinned SHA-256 that `Models.fetch` checks before OpenCV is ever shown the bytes. It is leaner than `FaceDetect.modelSpec`, which has two mirrors and a `sizeBytes` of `232589` against SFace's one Zoo URL and no size. `sizeBytes` is optional and only ever improves the failure message: a mirror that answers a model request with an HTML error page reports *expected 232589 bytes, got 131* for YuNet, and a bare SHA-256 mismatch for SFace. Both downloads are equally safe; one of them explains itself better.

Example 25-1. Both models, fetched and checksum-verified, into one directory.

```
import java.nio.file.Path
import scalacv.*

OpenCv.load()
val models = Path.of("models")

val yunet: Either[CvError, Path] = FaceDetect.downloadModel(models)
val sface: Either[CvError, Path] = Models.fetch(FaceRecognizer.modelSpec, models)
```

`FaceRecognizer.load` takes the path and returns an `Either`, on the same terms as every other model loader in the book. A path with no file at it comes back as `CvError.LoadFailed` carrying the message *no such file — supply the SFace ONNX model path* a file that is not an SFace network fails inside OpenCV's ONNX importer and is caught into a `Left` as well. Neither is an exception, because neither is a bug — mistyping a path is a thing users do.

> **NATIVE MEMORY**
> A `FaceRecognizer` owns a native `FaceRecognizerSF`, and behind that ONNX handle sit the model's weights in full — the 37 MB the file carries, parsed into native memory, against a Scala object the heap measures in tens of bytes. `FaceRecognizerSF` is one of the 185 generated binding types with no public `release()`, so the class holds a `Managed[FaceRecognizerSF]` built with `Releasable.nativeHandle` — the same bridge `FaceDetect` uses for `FaceDetectorYN`, and for the same reason: it reads the native address and disarms the binding's unconditional `finalize()` before freeing the pointer, without which a released handle is a live double-free.
> `FaceRecognizer` is `AutoCloseable` and its `close()` calls `handle.release()`. Close it — `Using.resource`, or explicitly in a `finally`. Leaking one in a loop that rebuilds the recogniser per request is the fastest way to reproduce Chapter 1′s measurement: flat heap, rising resident set, an OOM kill with no Java stack trace.

From a Face to a vector

`embed` is the entire inference surface:

```
def embed(image: Image, face: Face): FaceEmbedding
```

Two arguments, and the relationship between them is the thing to get right. `image` must be the BGR frame the `Face` was detected in, because a `Face` is nothing but coordinates into that exact image. `embed` borrows both — it consumes neither, so you can detect once and embed every face in the frame inside a single `Image.reading` block with no `.copy` anywhere.

Internally it rebuilds the 1×15 detection row SFace's `alignCrop` expects (box, five landmark pairs, score — YuNet's own output format, reconstructed from the decoded `Face`), calls `alignCrop` to produce a canonical crop, then `feature` to produce the vector, and copies the numbers out of OpenCV's result Mat. All of it happens inside a `Managed.scope`, so the three intermediate native objects are released whether the call returns or throws.

Example 25-2. Detect and embed inside one borrow of the frame.

```
Image.reading("ada.jpg") { img =>
  img.faces(detector).headOption.map(face => rec.embed(img, face))
}
```

Why you pass the frame and not a crop

The first thing a reader reaches for is to cut the face out and hand the recogniser a tidy thumbnail. `Face.clippedBox` even exists to make that crop safe. It is the wrong move here, and it fails quietly.

Example 25-3. Wrong: the landmarks index the frame, not the thumbnail.

```
import scala.util.Using

Image.reading("group.jpg") { frame =>
  val face = frame.faces(detector).head
  Using.resource(frame.copy.crop(face.clippedBox(frame).get)): thumb =>
    rec.embed(thumb, face) // the five points now land somewhere else entirely
}
```

The lifetimes in that listing are all correct — `copy` gives an owned `Image`, `crop` consumes it and hands back another, and `Using.resource` closes the survivor — which is part of why the bug is hard to see. The only thing wrong with it is the argument to `embed`.

A face 600 pixels into a group photograph has landmarks with x-coordinates near 600. Crop the box out and the same eye is now 20 pixels from the left edge of a 90-pixel thumbnail, but the `Face` you passed still says 600. If you are lucky, `alignCrop` rejects the geometry and you get a `CvError.NativeCall`. If you are not, it aligns on whatever happens to be at those coordinates and returns a perfectly well-formed 128-vector describing nothing — which then goes into your gallery, scores badly against the person it belongs to, and occasionally scores well against someone else.

The right version has nothing removed from it but the crop:

Example 25-4. Right: the frame the detector saw, and the face as detected.

```
Image.reading("group.jpg") { frame =>
  val face = frame.faces(detector).head
  rec.embed(frame, face)
}
```

Alignment is not a formality that a bigger network would let you skip. SFace was trained on faces warped to a canonical position using exactly these five points — subject's right eye, left eye, nose tip, right mouth corner, left mouth corner, in that order, as Chapter 24 sets out. A head tilted fifteen degrees is, to a raw crop, a different picture; after `alignCrop` it is the same picture. That warp is where most of the pose robustness in the system comes from, and it is free, because the detector already produced the landmarks it needs.

> **WARNING**
> A `FaceRecognizer` is one per thread, exactly like the detector. `alignCrop` and `feature` both run against a single native object, and `FaceRecognizerSF.feature()` writes into an internal buffer that `embed` copies its numbers out of. If a second thread calls `embed` between that write and that copy, the first thread copies out the other face's vector — no exception, no warning, an embedding belonging to somebody else. Build one recogniser per worker, or guard a shared one with a lock. `Gallery` and `FaceEmbedding` are immutable and genuinely safe to share.

Comparing two embeddings

`FaceEmbedding` carries both of SFace's metrics. They point in opposite directions, which is the one thing to keep straight.

Table 25-1. SFace's two metrics, their direction, and their published same-person cutoffs.

Metric	Direction	Same person	Call
Cosine similarity	higher is more alike, in `[-1, 1]`	0.363 or above	`a.cosineSimilarity(b)`
L2 distance	lower is more alike, from 0	about 1.13 or below	`a.l2Distance(b)`

Both return a `Double`, and both `require` that the two embeddings have the same length — a check that only bites when you mix a real 128-dimensional vector with a hand-written stand-in, which is worth doing in tests and worth never doing anywhere else.

Pick one metric and stay with it. They are not interchangeable at the call site: a 0.4 that means "same person" under cosine means "obviously not" under L2, and a comparison written against the wrong direction inverts your whole system without ever throwing. `Gallery` uses cosine similarity, and `Gallery.CosineThreshold` is the only cutoff the library exposes as a constant; the 1.13 figure lives in `l2Distance`'s scaladoc and nowhere else, so a lookup keyed on L2 distance is one you write yourself. That asymmetry is a reason to stay on cosine unless you have a specific need not to.

Why the metrics do their arithmetic in Double

Both loops widen each `Float` to `Double` before multiplying or subtracting, rather than accumulating in `Float` and widening the sum. It is a deliberate correction, not idiom. `a * b` on two `Float`s is `Float` multiplication, so every term of the dot product was being rounded to 24 bits of mantissa while the two norms, which did carry a `.toDouble`, were not — three accumulators in one expression disagreeing about their own arithmetic. The comparison downstream is against a fixed cutoff, so a systematic rounding bias is precisely the thing that can nudge a borderline face across the line. Neither loop allocates an intermediate collection either: `identify` runs one of them once per enrolled face, and a 128-element `Vector` per comparison is a per-lookup cost with nothing to show for it.

Thresholds, and the trade you are actually making

`Gallery.CosineThreshold` is `0.363`. Its scaladoc says what it is and no more: SFace's recommended same-person cutoff for cosine similarity. It is not a number this library tuned, it is not a number measured on your data, and it knows nothing about your camera, your lighting, or your population.

Treat it as the place to start, not the place to stop. The threshold is a dial with a cost at each end and no setting that has neither:

- Lower it and you get *false accepts*: strangers matched to enrolled people. In an access control system that is the failure that matters, and it is silent — the door opens, nobody files a ticket.
- Raise it and you get *false rejects*: enrolled people reported as unknown. This failure is loud, annoying, and self-reporting, which is why it is the safer direction to err in for anything that gates access.

Which end costs more is a property of your application, not of the model, and it is the one design decision here that a library cannot make for you. `identify` therefore takes the threshold as an argument with the recommended value as its default:

```
def identify(embedding: FaceEmbedding, threshold: Double =
Gallery.CosineThreshold): Option[FaceMatch]
```

Do not tune it by intuition on the three photographs you happen to have. Collect pairs you know are the same person and pairs you know are not, score them all, and look at where the two distributions overlap. If they barely overlap, the exact cutoff hardly matters. If they overlap heavily, no cutoff will save you and the problem is upstream — bad crops, missed detections, or too few enrolments.

The Gallery

`Gallery` is the lookup, and it is a value.

Example 25-5. The gallery's whole surface.

```
val gallery = Gallery.empty
  .enroll("ada", adaFrontal)
  .enroll("ada", adaProfile) // the same name, a second pose
  .enroll("grace", graceFrontal)

gallery.size      // 3
gallery.names     // Seq("ada", "ada", "grace") — duplicates included
gallery.isEmpty   // false

gallery.identify(probe) match
  case Some(FaceMatch(name, similarity)) => f"$name ($similarity%.3f)"
  case None                              => "unknown"
```

`enroll` returns a new gallery rather than mutating the one you had, so a gallery is safe to share across threads, to snapshot, and to hold as a `val` while a background job builds the next version. `identify` scores the probe against every entry, discards everything below the threshold, and returns the highest of what is left as a `FaceMatch(name, similarity)` — or `None`.

That `None` is the case to write first. A recogniser deployed anywhere real sees strangers far more often than it sees enrolled people, and a pipeline whose only branch is `Some` will either crash on the first passer-by or, worse, be written with a `.getOrElse("unknown")` that quietly discards the score you needed for the audit log. Keep the similarity: it is the only evidence you have about how close the call was.

A gallery is an index, not your store of record

`Gallery` keeps its entries in a `private val`, and its public surface is five members: `enroll`, `identify`, `names`, `size` and `isEmpty`. `names` gives you the names — with duplicates, in enrolment order — and nothing gives you the embeddings back. That is deliberate, and it decides where persistence has to live: you cannot save a `Gallery`, because you cannot read one.

So keep the enrolments yourself and treat the gallery as something you build from them. A `FaceEmbedding` is a `Vector[Float]` in a wrapper, so the pair is already the storable form:

Example 25-6. Enrolments are the durable thing; the gallery is derived from them.

```
final case class Enrolment(name: String, embedding: FaceEmbedding)

def galleryOf(enrolments: Seq[Enrolment]): Gallery =
  enrolments.foldLeft(Gallery.empty)((g, e) => g.enroll(e.name, e.embedding))
```

Rebuilding is cheap — `enroll` appends to a `Vector` — so a service that reloads its enrolments on a schedule can build a fresh gallery and swap the reference, with readers on the old one unaffected because neither value ever changes.

Enrolment: several shots, and what to do with them

One reference photograph per person works in a demo and disappoints in a corridor. The variation that breaks recognition is not identity, it is pose, illumination and expression — a face lit from the side, turned twenty degrees, or mid-sentence, embeds noticeably further from its own frontal portrait than you would like.

The fix is more enrolments per person, deliberately varied: frontal and turned each way, with and without glasses, under whatever lighting the deployment actually has. `Gallery` supports this directly — the same name may be enrolled any number of times, and `identify` takes the best-scoring of a person's entries. Five references cost five dot products per lookup, which for a gallery of any plausible size is nothing.

The alternative is to average a person's embeddings into one vector and enrol that. It halves the lookup work and smooths out a single unlucky shot, and it costs you the ability to match a pose that sits far from the mean — an average of a frontal and two profiles resembles none of them especially well. Because an embedding is plain data, you can build the mean yourself:

Example 25-7. Averaging several embeddings into one enrolment.

```
def mean(embeddings: Seq[FaceEmbedding]): FaceEmbedding =
  require(embeddings.nonEmpty, "no embeddings to average")
  val n = embeddings.head.values.size
  val sums = Array.ofDim[Double](n)
  for e <- embeddings; i <- 0 until n do sums(i) += e.values(i).toDouble
  FaceEmbedding(sums.map(s => (s / embeddings.size).toFloat).toVector)
```

The division by `embeddings.size` changes no cosine score whatsoever — `cosineSimilarity` divides through by both norms, so it is blind to scale — but keep it anyway, or `l2Distance` against that entry becomes meaningless.

The default advice is to keep them all. Store the individual vectors even if you enrol an average as well, because you can always recompute the average from the vectors and you can never recover the vectors from the average, and the day you want to re-tune your threshold you will want the raw distribution.

The worked example

Everything above, assembled: fetch both models, enrol a handful of named people from labelled photographs, then identify every face in a new frame, reporting a score or an honest `unknown`.

Example 25-8. Enrol a gallery from labelled photographs.

```
import org.opencv.objdetect.FaceDetectorYN

def enrol(
    detector: Managed[FaceDetectorYN],
    rec: FaceRecognizer,
    people: Seq[(String, Seq[String])]
): Gallery =
  people.foldLeft(Gallery.empty) { case (g0, (name, paths)) =>
    paths.foldLeft(g0) { (g, path) =>
      Image
        .reading(path) { img =>
          img.faces(detector).headOption match
            case Some(face) => g.enroll(name, rec.embed(img, face))
            case None       => g // no face in the reference shot; nothing to
enrol
        }
        .getOrElse(g) // an unreadable file leaves the gallery as it was
    }
  }
```

Two failures are handled there and neither is an exception. A reference photograph with no detectable face contributes nothing, and a file that will not read leaves the gallery unchanged — `reading` hands back a `Left` and `getOrElse` absorbs it. In a real enrolment tool you would report both rather than swallow them; the shape is what matters here.

Example 25-9. Identify every face in a frame, unknowns included.

```
final case class Sighting(box: Rect, name: Option[String], similarity: Double)

def identifyAll(
    gallery: Gallery,
    detector: Managed[FaceDetectorYN],
    rec: FaceRecognizer,
    path: String
): Either[CvError, Seq[Sighting]] =
  Image.reading(path) { frame =>
    frame.faces(detector).map { face =>
      val embedding = rec.embed(frame, face)
      // -1.0 is the least a cosine similarity can be and the filter is `>=`, so
this always
      // returns the nearest enrolment; the accept/reject decision is made here,
with the score.
      gallery.identify(embedding, threshold = -1.0) match
        case Some(FaceMatch(n, s)) if s >= Gallery.CosineThreshold =>
Sighting(face.box, Some(n), s)
        case Some(FaceMatch(_, s))                                 =>
Sighting(face.box, None, s)
        case None                                                  =>
Sighting(face.box, None, 0.0)
    }
  }
```

`Sighting` keeps the box, so the result can be drawn with the tools from Chapter 14, and keeps the name as an `Option` rather than the string `"unknown"` — an unknown face is a different kind of answer from a named one, and the type should say so before a downstream `match` has to guess.

The threshold of `-1.0` is the trick worth stealing. `identify` at its default returns `None` for anything below the cutoff, and `None` carries no number, so the score you most want in the log — the one for the face that *nearly* matched — is the one the default throws away. Cosine similarity is bounded below by `-1` and `identify` filters on `>=`, so a threshold of `-1.0` admits every entry and hands back the best one whatever it scored. Compare against `Gallery.CosineThreshold` yourself and you get the same decision plus the evidence for it. The last case, `None`, is then reachable only for an empty gallery, which is exactly the condition it should be reporting.

Example 25-10. The whole pipeline, with both native owners released exactly once.

```
val people = Seq(
  "ada" -> Seq("ada-frontal.jpg", "ada-left.jpg", "ada-lamp.jpg"),
  "grace" -> Seq("grace-frontal.jpg", "grace-smiling.jpg")
)

val result =
  for
```

```
    yunetPath <- FaceDetect.downloadModel(models)
    sfacePath <- Models.fetch(FaceRecognizer.modelSpec, models)
    detector <- FaceDetect.create(yunetPath.toString, Size(320, 320))
    rec <- FaceRecognizer.load(sfacePath.toString)
  yield
    try identifyAll(enrol(detector, rec, people), detector, rec, "group.jpg")
    finally
      detector.release()
      rec.close()

result match
  case Left(err)               => println(s"setup failed: $err")
  case Right(Left(err))        => println(s"the frame failed: $err")
  case Right(Right(sightings)) =>
    sightings.foreach {
      case Sighting(box, Some(name), s) => println(f"$name at $box (cosine
$s%.3f)")
      case Sighting(box, None, s)       => println(f"unknown face at $box (best
$s%.3f)")
    }
```

Four fallible steps compose in one `for`; the first `Left` short-circuits, and because the downloads come first, a failure there leaves nothing native to leak. The `try` / `finally` is the load-bearing part of the listing, and it is not there for OpenCV failures: `Image.reading` runs its whole block inside `Cv.attempt`, so a `CvError` out of `embed` returns as the inner `Left` rather than unwinding. It is there for what *does* propagate — a `require` in `FaceDetect.detect` rejecting a frame that is not 8-bit BGR, a use-after-move, anything your own code throws — because those are the paths on which a detector and a recogniser would otherwise be abandoned holding a parsed model each.

Table 25-2. Recognition symptoms and where the cause almost always is.

Symptom	Likely cause	Fix
Everyone matches everyone	threshold too low	keep `Gallery.CosineThreshold`, or raise it
Known people read as unknown	too few enrolments, or poor references	enrol several poses per person
Confident wrong matches	a crop was embedded instead of the frame	pass the frame the `Face` came from
An embedding now and then belongs to another face	one recogniser shared across threads	one per worker, or a lock
`load` returns a `Left`	wrong path, or not an SFace ONNX	check the `CvError.LoadFailed` message

What a score is not

Three things are worth saying once, plainly, before you ship any of this.

A similarity is not an identity claim. `FaceMatch("ada", 0.41)` says one vector was closer to another vector than a threshold you chose. It does not say the person in the frame is Ada, and any system that treats it as though it does — an unlock, an accusation, an automated report — has promoted a measurement into a fact without anyone deciding to.

Accuracy is not uniform across people. Published evaluations of face recognition systems, including NIST's, repeatedly find error rates that differ by demographic group, and a threshold chosen on one population does not transfer to another. A single global cutoff therefore distributes its false accepts and false rejects unevenly, and it will do so silently unless you measure per group. If you cannot measure it, you cannot claim it is not happening.

Face embeddings are biometric data. Under GDPR Article 9 they are a special category with a narrow set of lawful bases; Illinois' BIPA requires written consent before collection and imposes a private right of action; several other jurisdictions have equivalents, and the list grows. The practical consequences for the code in this chapter are small and concrete: treat the gallery as sensitive personal data at rest, keep embeddings and names out of logs and out of crash dumps, and build the deletion path — a person's enrolments removed on request, everywhere they were copied — before you build the enrolment path, because it is far harder to add afterwards. `Gallery` being an immutable value makes the storage question yours rather than the library's, which is the right place for it, and does not answer it.

Where this leaves you

You can now put a name on a face in a single frame, with a score you chose the meaning of and an `unknown` that is a real answer rather than an oversight. What you cannot yet do is keep that name attached to a person as they move. Run `identify` on every frame of a video and you pay for an SFace forward pass per face per frame, and you still watch the label flicker off the moment someone turns their head far enough that no enrolment clears the threshold.

Chapter 26 goes underneath both models in this pipeline before that problem is worth attacking. YuNet and SFace are ONNX graphs run by OpenCV's `dnn` module, and `Dnn` is that machinery with nothing wrapped around it: load a graph, build an input tensor from an image, run one forward pass, decode the output tensor yourself. Three things this chapter treated as given generalise there — a fetch-and-verify download, a caller-owned native model, and the pre-processing numbers that decide whether the output means anything — and they apply to any network you can export to ONNX.

The continuity problem is Chapter 30: tracking, from the `Kalman` smoother through the single-object `Tracker` and its `TrackerKind` choices to `ObjectTracker` and its stable ids. That is the machinery that lets you recognise a face once, expensively, and keep calling it by name for as long as it stays in the frame.

CHAPTER 26

Deep Networks and ONNX

Running a trained model on the CPU, and the four numbers that decide whether it was worth running.

Almost every technique in this book so far answered a question with geometry. A threshold asks whether a pixel is brighter than a number; a contour asks which pixels touch; a Hough transform asks how many edge points agree about a line. You can write down what each of them means, and when one is wrong you can point at the step that made it wrong.

Some questions have no such answer. A camera on a bird feeder sees a small brown shape in a hedge, and the useful question — goldfinch or siskin — is not a threshold, a contour, or a line. It is a learned function of a hundred thousand training photographs that nobody, including the people who trained it, can write down. The only way to get an answer is to run the function.

OpenCV can run it for you. The `dnn` module loads a graph that was trained elsewhere and executes one forward pass over it on plain CPU, with no PyTorch on the classpath and no GPU runtime to install. What it will not do is train anything: no autograd, no optimiser, no fine-tuning, no gradient of any kind — only weights that were fixed before your process started and arithmetic that reads them. If the model is wrong for your data, `dnn` is not the place you fix it.

That division of labour shapes this entire chapter. Everything deciding whether the answer is correct lives **outside** the API you are about to call: the size the model was trained at, the range its pixel values were scaled into, the mean subtracted from them, and the channel order they arrived in. Get one of those four wrong and nothing throws — the model loads, the pass runs, the output tensor has exactly the promised shape, and the numbers in it are confidently, quietly wrong. The API cannot fix that, but it can name and document the parameters so they are hard to transpose, which is what `scalacv.Dnn` is.

`Dnn` lives in `scalacv-vision`, the same module as the detectors of the last three chapters, so that dependency has to be on the classpath; `Image`, `Managed`, `Scalar` and the drawing verbs come from `scalacv-core` underneath it.

The chapter builds one thing: a bird feeder that names its visitors. It reads a frame, runs an ONNX classifier over it, turns the output tensor into a species and a confidence, draws them on the frame, and writes it out — with every native object owned by something that will free it.

Inference only, and only ONNX

OpenCV's importers can read Caffe, Darknet, TensorFlow, TFLite and Torch graphs as well as ONNX. `scalacv` exposes exactly one of them.

That is a deliberate narrowing. Each of those importers carries its own private list of unsupported layers and its own way of failing on them, and offering seven entry points would advertise a breadth of support no small library can honestly stand behind. ONNX is the format the other frameworks **export to** — one line in PyTorch, one tool for TensorFlow, one for scikit-learn — so a single importer covers the realistic cases, and the conversion happens once, on your machine, where you can watch it fail.

The work therefore starts before any Scala: get the model into a `.onnx` file, with a fixed input shape unless you specifically intend a dynamic one, and read its model card while you are there. The card is where the four numbers live.

Three functions and one owner

The entire deep-learning surface is one object with three methods. Each stage takes what the previous one produced, and each says in its return type who must free what it hands back.

Table 26-1. The whole `Dnn` surface. Every result is caller-owned; no argument is consumed.

Stage	Call	Gives back
Load	`Dnn.fromOnnx(path)`	`Either[CvError, Managed[Net]]`
Pre-process	`Dnn.blobFromImage(mat, …)`	`Managed[Mat]` — the input tensor
Infer	`Dnn.forward(net, blob, …)`	`Managed[Mat]` — the output tensor

Spelled out with the defaults the source declares, so that nothing below is a surprise:

Example 26-1. Three signatures. The defaults are OpenCV's, not any model's.

```
object Dnn:
  given Releasable[Net] = Releasable.nativeHandle

  def fromOnnx(path: String): Either[CvError, Managed[Net]]

  def blobFromImage(
      mat: Mat,
      scaleFactor: Double = 1.0,
      size: Option[Size] = None,
      mean: Scalar = Scalar(0, 0, 0),
      swapRB: Boolean = false,
      crop: Boolean = false
  ): Managed[Mat]

  def forward(net: Net, blob: Mat, outputName: Option[String] = None):
Managed[Mat]
```

Three calls, three `Managed` results, three things to scope. Nothing takes ownership of an argument: `blobFromImage` borrows the image and leaves it as it found it, and `forward` borrows both the network and the blob — the borrow-versus-consume split Chapter 4 established for `Image`, applied to types that are not `Image`.

Loading the model

`Dnn.fromOnnx` is the only way into the module, and every way it can fail is a `Left`.

Example 26-2. Load once, at startup. The Left names the file you asked for.

```
Dnn.fromOnnx("models/feeder-classifier.onnx") match
  case Left(e)    => sys.error(s"the model did not load: ${e.getMessage}")
  case Right(net) => net.use { n => /* the whole pipeline lives in here */ }
```

The failure modes are deliberately not teased apart. OpenCV reports a missing file, a file of arbitrary bytes and a structurally invalid graph as three **different** `CvException`s from three different lines of `onnx_importer.cpp`, and none of that is a stable interface worth pattern-matching on. A missing, unreadable or layerless model arrives as `CvError.LoadFailed`, whose `resource` field is the path you passed; a graph the importer throws on arrives as `CvError.NativeCall`, whose operation reads `reading an ONNX model from '…'`. Both name the file, and both mean the same thing to a caller: the model did not load, and the process is still alive.

Two checks bracket the native read. The path is examined **first**, with `java.io.File` — not because the native code would miss a missing file, but because that is the only way the message names your file instead of quoting a C++ source location. Afterwards comes an `empty()` guard: 4.13.0's importer throws rather than returning a layerless `Net`, but the sibling readers in the same header do not all behave that way, and an empty `Net` that escapes the load fails much later, inside `forward`, with nothing pointing back at the cause. If the guard fires, the handle is real even though the graph is not, so it is released before the `Left` is built.

> **WARNING**
> Loading is not cheap, and not something to do per frame: parsing a graph, allocating its weights and laying out its buffers costs orders of magnitude more than the forward pass. Load once at startup, keep the `Managed[Net]` alive for the life of the pipeline, and reuse it.

> **NATIVE MEMORY**
> `Net` is one of the 185 `org.opencv.*` types with no public `release()` — only `Mat`, `VideoCapture` and `VideoWriter` have one, out of the 188 that hold a native pointer. `scalacv` frees it anyway, through `Releasable.nativeHandle`: the address is read from the binding's own `nativeObj` field, the JVM finalizer is disarmed **before** the pointer is deleted, and the cached `MethodHandle` onto the private `delete(long)` does the deletion. If that bridge cannot be opened the library throws `CvError.NativesMissing` rather than degrading to a silent leak.

A leaked `Net` costs what its weights cost: parsed parameters as large as the `.onnx` file that produced them, held behind a Java object the collector sees as a few dozen bytes. The measured shape of that mistake for a handle-freed type is in the README — 4000 leaked `KalmanFilter`s reached 54 GB of RSS, against 86 MB when released. A model reloaded per frame gets there faster than anything else in this book.

If you obtain a `Net` some other way — `org.opencv.dnn.Dnn.readNet`, `Net.quantize` — `import Dnn.given` puts the same `Releasable[Net]` in scope, so `Managed(raw)` frees it on identical terms. One release path for networks, not two.

Getting the file onto the machine

A model is a large binary that has to arrive from somewhere, and `Models` is the library's answer. A `ModelSpec` pins a file name, mirror URLs tried in order, a SHA-256 the bytes must match, and optionally `sizeBytes`; `Models.fetch(spec, into)` downloads to a temp file **beside** the target and moves it into place only after verification, so an interrupted run never leaves a truncated model behind, and an already-verified target is returned without touching the network.

The size check is not redundant with the hash: it catches the mirror that answers with an HTML error page, or the Git LFS host serving a 131-byte pointer file, and it changes the **message** — "expected 232589 bytes, got 131" says what happened, where "SHA-256 mismatch" invites the reader to suspect tampering. URLs may be `file://`, so a model already on disk is another mirror and an air-gapped build is a spec whose only URL is local.

The blob, and the four numbers

A network does not take an image. It takes a **blob**: a four-dimensional `CV_32F` `Mat` in NCHW layout, with sizes `(1, C, H, W)` — batch, channels, height, width — whose values have been scaled into whatever range the model was trained on. `Dnn.blobFromImage` builds one.

Table 26-2. What each parameter does, and what it defaults to.

Parameter	Type	Default	Effect
`mat`	`Mat`	—	the source image, borrowed; must be non-empty
`scaleFactor`	`Double`	`1.0`	multiplier applied **after** the mean subtraction
`size`	`Option[Size]`	`None`	spatial size `(width, height)` to resize to
`mean`	`Scalar`	`Scalar(0, 0, 0)`	subtracted per channel, in the **blob's** channel order
`swapRB`	`Boolean`	`false`	swaps the first and third channels, BGR to RGB

`crop`	`Boolean`	`false`	`true` resizes the short side and centre-crops the rest away

Three of those interact in ways that produce a working program with wrong answers.

The arithmetic is `(pixel - mean) * scaleFactor`, in that order, and the mean is not scaled. A model trained on `[0, 1]` inputs wants `scaleFactor = 1.0 / 255` with a `mean` expressed in `[0, 255]`, not a mean already divided by 255. The tests pin the order with numbers whose two readings cannot agree: at `scaleFactor = 2.0` and `mean = Scalar(4, 0, 0)`, a channel of 10 becomes `(10 - 4) * 2 = 12`, never `10 * 2 - 4 = 16`.

`mean` is applied in the blob's channel order, which is after `swapRB`, not before. Measured against 4.13.0: OpenCV swaps first and subtracts second, so with `swapRB = true` on a BGR image the first component of `mean` meets red, not blue — exactly what published per-model triples assume. The two readings differ by the amount that makes a model quietly **worse** rather than visibly broken, which is far harder to notice.

`Size` is `(width, height)`, but the blob's trailing dimensions are `(height, width)`. `Size(300, 200)` yields sizes `(1, 3, 200, 300)`. Transposing that is the usual cause of a network that runs and returns nonsense rather than one that reports an error.

`size = None` keeps the source's own spatial size, correct only for a network with a dynamic input shape. A `size` that is given but not strictly positive is rejected with an `IllegalArgumentException`, as is an empty `mat`: OpenCV reads a zero extent as "do not resize", which would silently build a blob of the wrong shape instead of failing.

Here is the mistake almost everyone makes first — taking the defaults, which belong to OpenCV and to no model:

Example 26-3. Wrong. Values in [0, 255], channels in BGR, and a model trained on neither.

```
// The graph accepts this blob and returns a full tensor of confident nonsense.
Dnn.blobFromImage(frame.mat, size = Some(Size(640, 640)))
```

And the same call with the model card's numbers in it:

Example 26-4. Right. Every argument is copied from the model's documentation, not guessed.

```
Dnn.blobFromImage(
  frame.mat,
  scaleFactor = 1.0 / 255,          // the model was trained on [0, 1] inputs
  size = Some(Size(640, 640)),      // (width, height)
  mean = Scalar(0, 0, 0),           // this family subtracts nothing
  swapRB = true,                    // trained on RGB; OpenCV decodes BGR
  crop = false                      // stretch both axes rather than centre-crop
)
```

Naming every argument, including the ones that match the default, turns a review of the preprocessing into a diff against the model card rather than an exercise in remembering what `false` meant in position five.

Table 26-3. Normalisation recipes for families you will meet. Always confirm against the card.

Model family	`scaleFactor`	`mean`	`swapRB`	`size`
Caffe classics (ResNet, VGG, GoogLeNet)	`1.0`	`Scalar(104, 117, 123)`	`false`	224×224
TorchVision / ImageNet normalise	`1.0 / 255`	`Scalar(123.675, 116.28, 103.53)`	`true`	224×224
MobileNet-SSD (Caffe)	`1.0 / 127.5`	`Scalar(127.5, 127.5, 127.5)`	`false`	300×300
YOLO (Darknet or ONNX export)	`1.0 / 255`	`Scalar(0, 0, 0)`	`true`	416×416 or 640×640

The checklist

When a model that works in Python returns garbage here, the cause is on this list far more often than anywhere else. Walk it against the model card before debugging anything downstream:

1. **Input size.** Does `size` match the shape the graph declares, and is it written `(width, height)`?
2. **Range.** Does `scaleFactor` put pixels into the trained range — `1.0` for `[0, 255]`, `1.0 / 255` for `[0, 1]`, `1.0 / 127.5` for `[-1, 1]` with a matching mean?
3. **Mean.** Is it expressed in `[0, 255]` (pre-scaling), and is it written in the **blob's** channel order, meaning RGB whenever `swapRB` is `true`?
4. **Channel order.** Was the model trained on RGB? Almost all were, and OpenCV decodes BGR, so `swapRB = true` is the common case even though the default is `false`.
5. **Aspect ratio.** Did training stretch the image to a square, centre-crop it, or letterbox it? `crop = false` stretches, `crop = true` centre-crops, and letterboxing is neither — you would pad the image yourself before building the blob.
6. **Coordinate space.** If the model emits boxes, they are in the **blob's** pixel space. Whatever reshaping you chose above has to be inverted when you map them back onto the frame.

The forward pass

`Dnn.forward` sets the blob as the network's input and runs the graph — in one call, on purpose. The two operations are not independently useful: a `Net` whose input has been set but not forwarded is a half-applied mutation, and a `forward` with no preceding `setInput` reads whatever the last caller left behind. Fusing them makes the stateful pair atomic, and makes the window between them one no caller can accidentally widen.

`outputName` selects which blob to retrieve. `None` runs to the last layer, which is what a single-output classifier wants; a multi-output graph needs a name, and those are **blob** names — for an ONNX import, the graph's declared outputs — not the `onnx_node!…` layer names the importer generates. A name that matches nothing is a `CvError.NativeCall` mentioning `Net.forward`, not a crash, and an empty net or blob is an `IllegalArgumentException` raised before any native code runs. Retrieving several outputs from one graph gets its own listing further down.

The return value deserves a paragraph of its own, because OpenCV's behaviour here is a trap. `Net.forward` hands back a header onto the layer's **own** output buffer — the generated JNI wraps the returned `cv::Mat` with a constructor that shares pixels, and the `dnn` blob manager reuses that allocation for the next pass of the same shape. Measured on 4.13.0, two forwards from one `Net` returned the identical `dataAddr`, and the second rewrote the first result's pixels in place: anyone diffing this frame's heatmap against the previous frame's got zero, every time, with nothing thrown. `Dnn.forward` copies before returning and releases the borrowed header immediately, so holding two results from one network means what it looks like it means. The cost is one memcpy of the output blob, negligible beside the pass that produced it.

> **NATIVE MEMORY**
> `blob` is borrowed, not consumed, but it must stay alive until `forward` returns. Releasing it while a pass is in flight is a segmentation fault, not an exception — native code is reading the buffer you just freed. Keep the blob in a `Managed` whose scope strictly encloses the call, as every example here does. Releasing it twice afterwards is harmless: `release()` is idempotent, and a released handle throws `IllegalStateException` from `get` rather than handing JNI a dangling pointer.

One Net per thread

A `Net` is not a pure function. `setInput` mutates it and `forward` reads that mutation back, so one network cannot be driven from two threads at once. `Dnn.forward` narrows the window by fusing the pair, but it is not a lock and does not pretend to be.

The rule is one `Net` per thread, or serialised access that you own. For a pool of workers, build one network per worker at startup and never share: the cost is one copy of the weights per thread, which is a price to decide on knowingly rather than discover through corrupted output. Give each worker a throwaway `forward` at startup too — the first pass allocates the

graph's intermediate buffers, so an unwarmed network makes the first real request the slowest one you will ever serve.

Backends and targets

By default OpenCV runs the graph on its own CPU backend. To ask for anything else, the raw `Net` exposes `setPreferableBackend` and `setPreferableTarget`, taking `int` constants from `org.opencv.dnn.Dnn`. Call them once, after loading and before the first `forward`.

Example 26-5. Ask for OpenCL. Whether you get it is a property of the host, not of this code.

```
import org.opencv.dnn.Dnn as CvDnn

Dnn.fromOnnx("models/feeder-classifier.onnx").foreach { managed =>
  managed.use { net =>
    net.setPreferableBackend(CvDnn.DNN_BACKEND_OPENCV)
    net.setPreferableTarget(CvDnn.DNN_TARGET_OPENCL)
    // ... every forward from here on prefers the GPU, if there is one to
prefer ...
  }
}
```

Table 26-4. What the pairings mean, and whether the bundled natives can actually run them.

Goal	Backend	Target	Reachable?
Default CPU	`DNN_BACKEND_OPENCV`	`DNN_TARGET_CPU`	yes — the default
CPU with FP16 math	`DNN_BACKEND_OPENCV`	`DNN_TARGET_CPU_FP16`	ARM v8 only
GPU via an OpenCL driver	`DNN_BACKEND_OPENCV`	`DNN_TARGET_OPENCL`	yes, with an ICD installed
NVIDIA GPU via CUDA	`DNN_BACKEND_CUDA`	`DNN_TARGET_CUDA`	no

The honest answer about GPUs here has three parts. OpenCL works: the `libopencv_dnn` inside the ordinary bytedeco classifier is compiled with OpenCV's `ocl4dnn` backend, so nothing extra goes on the classpath — but the host needs an OpenCL driver, an ICD such as `libOpenCL.so.1`, which is not bundled because it is a driver for hardware nobody can predict, and layers `ocl4dnn` does not implement run on the CPU regardless.

CUDA does not work, and not because of anything in your code. The classifier this library tells you to add is a CPU-only build — ask `Core.getBuildInformation()` and you will find `cudaarithm` on its `Unavailable:` line on every platform. Real `-gpu` classifiers exist, but they keep their payload under `org/bytedeco/opencv/linux-x86_64-gpu/` while `OpenCv.load()` extracts from `org/bytedeco/opencv/linux-x86_64/`; that jar alone throws `CvError.NativesMissing`, and alongside the ordinary one it silently runs on the CPU. If you

need CUDA, run inference as a separate service and keep `scalacv` for the image work around it.

GPU acceleration of ordinary operations — `blur`, `resize`, `cvtColor` — is out of reach too, and that boundary is not this library's doing: it needs `cv::UMat`, and the official OpenCV **Java** bindings ship no `UMat` class at all.

> **WARNING**
> An accelerator you did not get is silent, not loud. A backend or target the build was not compiled with, or whose driver is missing, falls back to the CPU and returns a perfectly good answer: no exception, no `Left`, no log line. Setting a target is not evidence that it engaged. The only reliable check is a timing comparison — load once, warm up, time N forwards on one blob with `DNN_TARGET_CPU`, then the same N with the target you hope for, and compare the means.

What you do get without asking is the CPU acceleration that was on all along: runtime-dispatched SIMD up to AVX-512 where the hardware supports it, and OpenCV's thread pool across your cores.

Decoding: from a tensor to an answer

`forward` gives you a `Mat` whose shape is entirely the model's business, and the cheapest thing to do with a model you have not run before is print that shape. A transposed `size`, a graph exported with an axis order you did not expect, a rank of three where you assumed two — all of them show up here in one line, before any arithmetic has had a chance to make them plausible.

Example 26-6. The first thing to run against a model you did not export yourself.

```
import org.opencv.core.{CvType, Mat}

def describe(output: Mat): String =
  val dims  = output.dims()
  val sizes = (0 until dims).map(output.size).mkString("(", ", ", ")")
  s"$dims-D $sizes of ${CvType.typeToString(output.`type`())}"
```

Ask `dims()` and `size(i)`, never `rows()` and `cols()`: OpenCV reports both of the latter as `-1` for any `Mat` of more than two dimensions, so a YOLOv8-shaped `(1, 84, 8400)` output answers `-1` to every two-dimensional question, and answers it without complaint. The element type is almost always `CV_32F`; anything else means the graph was quantised and its outputs need dequantising before the numbers mean what the model card says they mean.

A classifier: one row of scores

An image classifier emits one score per class, in a `(1, N)` output. The prediction is the argmax, which `Core.minMaxLoc` finds directly:

Example 26-7. Top-1 from a classifier, with the label list the model shipped with.

```
import org.opencv.core.Core
import org.opencv.dnn.Net

def classify(net: Net, frame: Image, labels: Seq[String]): (String, Double) =
  Dnn
    .blobFromImage(frame.mat, scaleFactor = 1.0 / 255, size = Some(Size(224,
224)),
                   mean = Scalar(123.675, 116.28, 103.53), swapRB = true)
    .use { blob =>
      Dnn.forward(net, blob).use { output =>
        val mm = Core.minMaxLoc(output)
        (labels(mm.maxLoc.x.toInt), mm.maxVal)
      }
    }
```

`minMaxLoc` reads a two-dimensional single-channel array, which is exactly what a `(1, N)` score row is, and reports the winning column as `maxLoc.x`. The label list is your responsibility: it ships beside the model as a text file, one class per line, and its order is the order of the output columns. A label file whose order does not match the export gives you a model that runs and lies just as convincingly as a wrong `mean` does.

> **NOTE**
> `maxVal` is whatever the last layer emits. Many exported graphs stop **before** the softmax, so those numbers are unbounded logits rather than probabilities in `[0, 1]`. The argmax is identical either way; apply a softmax yourself only if you need a calibrated confidence to threshold on.

A detector: many rows, most of them duplicates

A detector's output is a grid of candidates rather than a row of scores, and it is far larger than the answer you want. A YOLOv5-shaped export gives `(1, N, 5 + K)` — for each of N candidate boxes, four numbers describing the box in the **blob's** pixel space, one objectness score, and K per-class scores. A YOLOv8-shaped export gives `(1, 4 + K, N)`: transposed, with the objectness column gone. Decoding one as though it were the other produces boxes rather than an error, which is why the `describe` call above is worth its two lines.

Whichever the layout, the same three steps follow, and none of them is deep-network work. Candidates below a confidence floor are dropped, which removes the overwhelming majority of the N rows. Duplicates are suppressed, because a detector fires on one bird from several neighbouring grid cells. And the survivors are mapped out of blob space back into frame space, inverting whatever `size` and `crop` did on the way in.

That last step is where the quiet bugs live: a broken coordinate map still draws boxes, still draws them near the objects, and still looks roughly right on the frame you happened to test with. Chapter 27, *Object Detection*, is entirely about the code between `forward` and the drawn box — letterbox padding and the two coordinate systems it creates, decoding while filtering rather than after, per-class non-maximum suppression through

`org.opencv.dnn.Dnn.NMSBoxes`, and picking the confidence and IoU thresholds from frames rather than from feel. This chapter stops at the tensor.

More than one output

A graph with several declared outputs needs `outputName`, and the valid names are the graph's own rather than any you invent. Ask the network for them:

Example 26-8. Reading the shape of every declared output of a multi-output graph.

```
import scala.jdk.CollectionConverters.*
import org.opencv.core.Mat
import org.opencv.dnn.Net

def outputShapes(net: Net, blob: Mat): Map[String, Seq[Int]] =
  net.getUnconnectedOutLayersNames.asScala.toSeq.map { name =>
    Dnn.forward(net, blob, outputName = Some(name)).use { out =>
      name -> (0 until out.dims()).map(out.size).toSeq
    }
  }.toMap
```

That runs the graph once per name, and it is worth being explicit about the cost rather than hiding it. OpenCV's plural overload, `net.forward(outputs, names)`, fills a `java.util.List[Mat]` in a single pass — but every element of that list is a borrowed header onto the network's own buffers, which is the exact hazard `Dnn.forward` copies to remove. If a multi-output graph sits on a hot path and the extra passes hurt, drop to that overload through the raw binding and copy each element yourself before anything forwards the network again.

Batching, and why it rarely pays here

`Dnn.blobFromImage` builds a blob whose batch dimension is always one — a real constraint, and rarely the one that matters.

Most exported graphs pin the batch dimension to 1 anyway; a dynamic batch axis is something you ask for at export time. Batching pays off by keeping a GPU's many cores fed, and in the general case there is no GPU — OpenCV's CPU backend already parallelises a single forward across your cores, so a batch of eight mostly serialises the same work through the same thread pool while multiplying peak native memory by eight. And a camera, a request handler, or any pipeline answering frames as they arrive has nothing to batch: the second frame does not exist when the first needs an answer.

If you have **measured** a win — a small model, many images in hand, a machine where OpenCL genuinely engaged — the door is not locked. OpenCV's plural sibling of the call this chapter wraps, `org.opencv.dnn.Dnn.blobFromImages(images, scaleFactor, size, mean, swapRB, crop)`, takes a `java.util.List[Mat]` and returns a blob whose leading dimension is the list's length. Wrap that `Mat` in a `Managed` and hand it to `Dnn.forward`, which cares only that the blob is non-empty and shaped the way the graph accepts. The output's leading dimension is then N as

well, and splitting it back into N answers is yours to write — which is the other half of why batching so rarely pays here.

The feeder, end to end

Everything above in one program: load once, then per frame preprocess, infer, decode, draw, write. Read it for who frees what as much as for what it computes.

Example 26-9. The whole pipeline. Every native object is owned by a scope that outlives its use.

```
import scalacv.*
import org.opencv.core.Core
import org.opencv.dnn.Net

OpenCv.load()

val labels: Seq[String] =
  scala.io.Source.fromFile("models/species.txt").getLines().toSeq

def identify(net: Net, frame: Image): (String, Double) =
  Dnn
    .blobFromImage(
      frame.mat,                        // borrowed: the frame is still alive
afterwards
      scaleFactor = 1.0 / 255,
      size = Some(Size(224, 224)),      // (width, height)
      mean = Scalar(123.675, 116.28, 103.53),
      swapRB = true,
      crop = false
    )
    .use { blob =>                      // blob outlives the forward, and only
just
      Dnn.forward(net, blob).use { out =>
        val mm = Core.minMaxLoc(out)
        (labels(mm.maxLoc.x.toInt), mm.maxVal)
      }
    }

val named: Either[CvError, String] =
  Dnn.fromOnnx("models/feeder-classifier.onnx").flatMap { model =>
    model.use { net =>                  // one Net, loaded once, freed at the end
of this block
      Image.reading("feeder.jpg") { frame =>
        val (species, score) = identify(net, frame)
        frame
          .drawText(f"$species $score%.2f", Point(12, 28), Scalar.Green)
          .write("feeder-annotated.png")
          .map(_ => species)
      }.flatMap(identity)
    }
  }
```

Four owned native objects exist at the deepest point of that program — the network, the frame, the blob and the output tensor — and each is released by the construct that acquired it, innermost first, on success and on exception alike. `frame.mat` borrows, so building the blob does not disturb the image; `drawText` consumes and returns, so the annotated image is the value the chain hands on rather than a mutation of `frame`; and `write` is a terminal, so the frame is already released when `Image.reading` closes it again — which is exactly why `close` is idempotent.

The two `flatMap`s are load-bearing rather than decorative. `Image.reading` wraps whatever its body returns in its own `Either`, so a body that already returns one produces `Either[CvError, Either[CvError, String]]`; `flatMap(identity)` flattens the read failure and the write failure into one channel, which is the shape a caller can actually match on.

Everything model-specific in that listing sits in two places: the arguments to `blobFromImage`, and the handful of lines that turn `out` into an answer. Swap the model and those are the only lines that change — a detector would replace the second with a decode of the kind the next chapter builds, and would leave the first alone but for its numbers.

Next

This chapter kept the model at arm's length: you brought the `.onnx`, read its card, chose the four numbers, ran one pass, and read a `Mat`. For a classifier that is the whole job — the answer is one argmax away from the tensor. For a detector it is barely half of it, and the missing half is not deep-learning work at all but coordinate arithmetic that fails without saying so.

Chapter 27, *Object Detection*, spends its whole length there, on these same three calls — `fromOnnx`, `blobFromImage`, `forward` — with everything interesting on either side of them: an input transform that does not lie about the frame's aspect ratio, a decode that filters eight thousand candidate rows as it reads them rather than afterwards, suppression done per class, and a map back into the frame's own coordinates that you can check rather than hope about.

CHAPTER 27

Object Detection

Boxes with labels, and the arithmetic that puts them back where they belong.

Everything in this book so far has answered a question about pixels with more pixels. A blur returns an image. A threshold returns an image. Even contours, which finally hand back plain data, hand back data of a length the picture itself decided. Detection is the first task where you ask a question whose answer is a list of assertions about the world — **there is a person here, at this confidence; there is a van there** — and where the list can be empty, or can have forty entries, and where most of the forty are wrong.

It is worth being precise about which of the three neighbouring tasks you actually have. Classification takes a whole image and returns one label: this photograph contains a cat. It has no opinion about where. Segmentation takes a whole image and returns a label per pixel: this pixel is cat, that one is not. It has an exact opinion about where, and no notion of how many cats. Detection sits between them and returns a variable-length list of axis-aligned rectangles, each with a class and a score. It is the only one of the three whose output length is a prediction, and that is the source of nearly all its difficulty: two of the rectangles may be the same cat, one may be a cushion, and nothing in the output tells you which.

The second thing to be precise about is where the work lives. A detector's forward pass is one call. Everything expensive to get right happens on either side of it — resizing the frame into the network's input space without lying about the aspect ratio, reading a raw output tensor of eight thousand candidate rows into boxes, discarding the duplicates, discarding the seventy-eight classes you do not care about, and mapping the survivors back into the coordinate system your frame is actually in. That last step is where the bugs live, and they are quiet bugs: a detector with a broken coordinate map still draws boxes, still draws them near the objects, and still looks roughly right on the frame you happened to test with.

This chapter builds one pipeline for a fixed camera watching a loading bay, counting people and vans. It is deliberately the boring case, because the boring case is the one where a coordinate map that is quietly wrong goes unnoticed for a month.

The words on the model card

You will not train the model. You will download an ONNX export from somewhere and read a page describing it, and that page assumes a vocabulary. Six terms carry almost all of the weight.

Table 27-1. The vocabulary a detector model card assumes you already have.

Term	What it means when you are the one calling the model
Anchor	One of the fixed positions, and sometimes fixed box shapes, at which the network is allowed to propose an object. A modern export proposes at every cell of several downsampled grids, so an anchor is essentially "a row of the output tensor". You never choose them; you only have to know there are thousands and that each one produces a candidate.
Stride	The downsampling factor of the grid an anchor sits on. A stride-32 grid over a 640-pixel input is 20 × 20 cells, each cell responsible for a 32-pixel patch. The largest stride finds large objects; the smallest stride is the one that decides whether small objects can be found at all.
Confidence	The score attached to a candidate. Depending on the export it is a class probability, or a class probability multiplied by an objectness term. It is calibrated against the training set and not against your camera, so treat it as an ordering, not as a probability.
IoU	Intersection over union: the area two boxes share divided by the area they jointly cover. Zero for disjoint boxes, one for identical ones. It is how duplicates are recognised, and how a prediction is judged to have matched a hand-drawn label.
NMS	Non-maximum suppression. Sort the candidates by confidence, keep the best, delete every remaining box whose IoU with it exceeds a threshold, repeat. It is the step that turns eight thousand rows into four boxes.
mAP	Mean average precision: the leaderboard number. See the sidebar — it is the term most likely to make you choose the wrong model.

What mAP does and does not tell you

`mAP@0.5:0.95` is an average of an average. For each class, precision is averaged over the whole recall range to give an average precision; that is computed at ten IoU match thresholds from 0.50 to 0.95 and averaged again; then the per-class results are averaged over all eighty classes.

Three consequences follow, and all three matter more than the number itself. It is dominated by classes you do not have — toothbrushes and giraffes count exactly as much as people. It rewards tight box regression, because the strict IoU thresholds punish a box that is correct but loose, and a loose box is usually fine for counting. And it is measured on a public dataset photographed by people, not on a fixed camera at four in the morning under sodium lighting.

A model two mAP points behind another can be the better model for your two classes and your camera. The only way to know is forty labelled frames of your own footage, which is also what the threshold-picking section below needs, so the work is not wasted.

What the library owns, and what it does not

scalacv ships finished detectors for the closed-world cases, and it deliberately does not ship one for the open-world case. Understanding why draws the line you will be working on either side of for the rest of this chapter.

`Dnn`, `Qr` and `Aruco` are all in `scalacv-vision`, so that dependency has to be on the classpath for anything in this chapter to compile.

`Detectors.scala` holds the fiducial and barcode detectors, and they follow one contract to the letter: the detector is built, used and freed inside a single call, and the results cross the boundary as ordinary immutable Scala data. `Qr.detectAndDecode(mat)` returns `Seq[QrCode]`, where a `QrCode` is a `text: String` and a `corners: Seq[Point]`. It uses OpenCV's **multi** variant unconditionally, because the single-code `detectAndDecode` returns only the first symbol it happens to find and gives you no way to learn that there were others. A symbol that was located but could not be decoded comes back with an empty `text` and usable `corners`, which is the right shape for re-cropping and retrying rather than a failure.

`Aruco.detect(mat, dictionary = ArucoDictionary.Dict4x4_50)` returns `Seq[ArucoMarker]`, each an `id: Int` and four `corners`, with `ArucoDictionary` naming every predefined dictionary OpenCV ships. `Aruco.generateMarker(dictionary, id, sizePixels)` is the one method in the file that hands back native memory — a `Managed[Mat]`, because a rendered marker is an image you own. Both detectors reject an empty `Mat` with an `IllegalArgumentException`, because an empty image is a programmer error, while an image with nothing in it returns an empty `Seq`, because that is a result. The high-level entry points are extension methods on `Image`: `img.qrCodes` and `img.arucoMarkers(dictionary)`. Chapter 28, *QR Codes, ArUco, and Augmented Reality*, takes those two apart properly; what matters here is the shape of the contract, because the rest of this chapter is about what happens when a detector cannot honour it.

> **NATIVE MEMORY**
> Neither `QRCodeDetector` nor `ArucoDetector` has a public `release()`; they are two of the 185 generated types that expose only a private `delete(long)`. That is exactly why these signatures return `Seq` and not `Managed[…]`. If the library handed you a detector, you would be holding a native object the OpenCV Java API gives you no supported way to free. Building it, using it and freeing it inside one call means there is nothing left to own by the time the result reaches you, and the `Point` case classes in that result survive the detector's death because they were copied across the boundary rather than viewed through it.

For eighty-class object detection there is no equivalent, and no `Detector` type to reach for. The reason is that the decode is not OpenCV's and not this library's — it is the model's. YOLOv5, YOLOv8, SSD and DETR exports disagree about the output tensor's rank, its axis order, whether there is a separate objectness column, and whether box coordinates are centre-plus-size or corner-to-corner in input pixels or in normalised units. A wrapper that guessed would be wrong silently, which is the worst of the available failures.

So generic detection is delivered as `Dnn` plus a decoding layer you write: `Dnn.fromOnnx(path)` for an `Either[CvError, Managed[Net]]`, `Dnn.blobFromImage(mat, scaleFactor, size, mean, swapRB, crop)` for the NCHW input, `Dnn.forward(net, blob, outputName)` for one pass, and then your own code plus `org.opencv.dnn.Dnn.NMSBoxes` for everything after. Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, explains why every one of those returns a `Managed`, and Chapter 26, *Deep Networks and ONNX*, covers the load and the blob argument by argument; this chapter is about the code between the second call and the last.

> **NATIVE MEMORY**
> A loaded `Net` holds the model's entire weight set in native memory, and `Net` is another of the 185 types with no public `release()`. Load it once, at start-up, outside every loop, and keep the `Managed[Net]` alive for the life of the pipeline — reloading per frame would dwarf the inference cost and, without a `Managed`, would leak the weights each time. A `Net` is also stateful: `setInput` mutates it and `forward` reads that mutation back. `Dnn.forward` fuses the two so the window between them cannot be widened by accident, but it is not a lock. One `Net` per thread.

Choosing a model, and being honest about the cost

Three numbers describe a detector's operating point: input resolution, parameter count, and the latency the two of them produce on the machine you actually have. Only the first two are published.

Input resolution is not a speed knob. It is the knob that decides which objects can exist. A letterbox scales by the **long** side, so a 1920 × 1080 frame going into a 640 input is scaled by 640/1920, a third — not by 640/1080, which is the mistake that makes the next section's bug attractive. A person 30 pixels tall in that frame is therefore 10 pixels tall in the network's input, and 5 pixels tall at 320, where they are smaller than one cell of the stride-8 grid and the network has no mechanism by which to propose them at all. Raising the input size is how you detect small things; it is not a general accuracy dial, and on objects that already fill a third of the frame it buys almost nothing.

The cost, on the other hand, tracks pixel count closely for a convolutional detector, and pixel count is quadratic in the side.

Table 27-2. What input side costs, relative to 320, and what it is for.

Input side	Pixels vs 320	What it buys	What it costs you
320	1.0×	The cheapest useful band; fine for objects filling a tenth of the frame or more.	Small and distant objects are absent, not merely missed.
416	1.7×	The usual compromise for a fixed camera at a fixed distance.	Little, if your objects are large; nothing at all if they were already detected at 320.

640	4.0×	Small objects, crowded scenes, anything reaching the network at 40 pixels tall or less.	Four times the convolution work and roughly four times the number of candidate rows to decode.
960 and up	9.0×	Aerial and wide-area footage where objects are tens of pixels across.	A band where CPU inference stops being a real-time option at all.

The right-hand column is where the honesty has to be. This project's own benchmark page carries no neural-network numbers at all, and says why: the heaviest thing most pipelines do is a forward pass, and its cost belongs to the model and to the OpenCV DNN backend rather than to scalacv. Its instruction is four words — time your own model. No published figure substitutes for it, and none is offered here.

What can be said without measuring is the shape of the bands. A nano-class detector at 320 or 416 is the band where a real-time CPU loop is plausible on ordinary laptop hardware; a medium model at 640 is the band where it is not, and where the design question becomes which of three compromises you want. You can drop frames, and accept that your effective frame rate is the detector's. You can detect every Nth frame and carry the boxes between detections with a tracker, which is Chapter 30′s subject and by far the most common answer. Or you can move off the CPU, remembering from Chapter 26 that a backend or target this build was not compiled with falls back to the CPU silently — no exception, no `Left`, no log line — so a `DNN_TARGET_CUDA` that appears to do nothing is almost never a bug in your code.

> **TIP**
> Time the pipeline in four pieces, not one: blob construction, forward, decode, and NMS. Warm the net with one throwaway `forward` before you start, because the first pass allocates the layer buffers and is not representative. Report the median rather than the mean — a stop-the-world pause in the tail will otherwise dominate your average and send you optimising the wrong stage. Decoding eight thousand candidate rows in Scala is a larger share of the total than most people guess, and it is the one stage you control completely.

Letterbox, and the two coordinate systems that are not the same

The network wants a square input of a fixed side. Your frame is 1920 × 1080. There are two ways to reconcile that and they fail differently.

`Dnn.blobFromImage` with `crop = false` resizes both axes independently, which changes the aspect ratio: a van becomes a squat van, and a model trained on undistorted crops has never seen one. With `crop = true` it scales the short side to fit and centre-crops the rest away, which preserves shapes and silently deletes everything near the left and right edges of a 16:9 frame — including, on a loading bay camera, the door.

The third option is a letterbox: scale by the smaller of the two ratios so the whole frame fits, then pad the remainder with a constant colour. Nothing is distorted and nothing is discarded. The price is that the network's coordinate space now contains padding that does not exist in your frame, and every box that comes back is expressed in that space.

Example 27-1. Letterboxing a frame, and recording exactly what was done to it.

```
import scalacv.*

/** How a frame was fitted into the network's square input, so boxes can be
mapped back. */
final case class Letterbox(scale: Double, padX: Int, padY: Int)

def letterbox(img: Image, side: Int, pad: Scalar = Scalar(114, 114, 114)):
(Image, Letterbox) =
  // Read the size BEFORE resizeTo, which consumes the image (Chapter 4).
  val scale = math.min(side.toDouble / img.width, side.toDouble / img.height)
  val w = math.max(1, (img.width * scale).round.toInt)
  val h = math.max(1, (img.height * scale).round.toInt)
  val padX = (side - w) / 2
  val padY = (side - h) / 2
  val boxed = img
    .resizeTo(Size(w.toDouble, h.toDouble), Interpolation.Area)
    .border(padY, side - h - padY, padX, side - w - padX, BorderType.Constant,
pad)
  (boxed, Letterbox(scale, padX, padY))
```

Two details in that listing are load-bearing. `Interpolation.Area` is the correct choice going down, because it averages every source pixel that falls inside a destination pixel where `Linear` samples a handful and aliases the rest away — and a small object surviving the downscale is precisely what detection depends on. And the far-side padding is computed by subtraction, `side - h - padY`, not as a second `(side - h) / 2`: the halves lose a row whenever the difference is odd, and a 640-pixel target holding a 361-pixel image would come out 639 rows tall and fail the network's shape check.

Now the mistake. Having letterboxed, the obvious way to map a box back is to scale each axis by the ratio of the frame's side to the network's:

```
// WRONG after a letterbox: the padding is scaled along with the object.
def toFrame(box: Rect, frameW: Int, frameH: Int, side: Int): Rect =
  val sx = frameW.toDouble / side
  val sy = frameH.toDouble / side
  Rect((box.x * sx).toInt, (box.y * sy).toInt, (box.width * sx).toInt,
(box.height * sy).toInt)
```

This is the correct map for a stretched resize and the wrong one for a letterbox, and the interesting part is how it fails. It is exactly right whenever the frame is already square, because then there is no padding to subtract and the two per-axis ratios coincide — which is how it passes the unit test somebody wrote against a 512 × 512 fixture. Everywhere else the error

grows with the aspect mismatch and with the distance from the centre of the frame. Feed it 1920 × 1080 letterboxed into 640: `sx` is 3.0, which happens to be exactly the inverse of the scale factor, so widths and horizontal positions come out perfect. `sy` is 1.6875 against the 3.0 it should be, so every box keeps its width, loses 44 per cent of its height, and is dragged towards the middle row — dead right at the centre of the frame, out by 236 pixels at the top and bottom edges. On a 4:3 camera the same bug is a gentle squash that looks like an ordinary regression error, survives review, survives a demo, and fails the day somebody asks whether the person was inside the marked zone.

The right map undoes the two operations in reverse order: subtract the padding, then divide by the single scale factor that was applied to both axes.

Example 27-2. The inverse of the letterbox, which is the only correct map back.

```
def toFrame(box: Rect, lb: Letterbox): Rect =
  Rect(
    ((box.x - lb.padX) / lb.scale).round.toInt,
    ((box.y - lb.padY) / lb.scale).round.toInt,
    (box.width / lb.scale).round.toInt,
    (box.height / lb.scale).round.toInt
  )
```

> **WARNING**
> A box mapped back to the frame can still fall outside it. Detectors regress boxes from anchors, so an object at the edge legitimately produces a negative `x` or a width that runs past the right-hand column. That is fine for drawing — OpenCV clips — and fatal for cropping, which rejects a rectangle that is not wholly inside the image. Clip before you crop, never before you draw.

Decode once, and filter while you decode

The output tensor's layout belongs to the export. The listing below decodes the common YOLOv8 shape: three dimensions, `(1, 4 + classes, anchors)`, box coordinates as centre-x, centre-y, width and height in **input-space pixels**, and one column per class with no separate objectness term. Read your own export's shape from `output.dims()` and `output.size(i)` before you trust any of this; a YOLOv5 export is `(1, anchors, 5 + classes)`, transposed and with an extra column, and decoding one as the other produces boxes rather than an error.

The detail worth copying is not the arithmetic but the `wanted` argument. A COCO model has eighty classes and this pipeline cares about two. Scanning all eighty score columns for the best one, then discarding seventy-eight of the results afterwards, does forty times the work in the innermost loop — and that loop runs once per anchor. At a 640 input the three grids have strides 8, 16 and 32, so the anchor count is (80 × 80) + (40 × 40) + (20 × 20) = 8,400, and the tensor is 84 rows deep: four box numbers plus eighty scores. The row itself still crosses the JNI boundary whole, in one `get`; what `wanted` removes is the scan on top of it, 672,000 comparisons a frame against 16,800, to keep at most a few dozen boxes.

Example 27-3. Decoding a YOLOv8-shaped output, reading only the classes you asked for.

```
import scalacv.*
import org.opencv.core.{Core, Mat}

/** One candidate, in the network's input space, before suppression. */
final case class Detection(box: Rect, classId: Int, score: Float)

def decode(output: Mat, classCount: Int, wanted: Set[Int], floor: Float):
Seq[Detection] =
  val attributes = 4 + classCount
  val anchors = output.size(2)
  Managed.scope: own =>
    // reshape gives a header onto the same buffer: (1, attributes, anchors) ->
attributes x anchors.
    val flat = own(output.reshape(1, attributes))
    val rows = own(Mat())
    Core.transpose(flat, rows) // anchors x attributes, one candidate per row
    val row = new Array[Float](attributes)
    (0 until anchors).flatMap: i =>
      val _ = rows.get(i, 0, row)
      var best = -1
      var bestScore = floor
      wanted.foreach: c =>
        val s = row(4 + c)
        if s > bestScore then
          best = c
          bestScore = s
      if best < 0 then Nil
      else
        val (cx, cy, w, h) = (row(0), row(1), row(2), row(3))
        val rect = Rect((cx - w / 2).toInt, (cy - h / 2).toInt, w.toInt, h.toInt)
        List(Detection(rect, best, bestScore))
```

> **NATIVE MEMORY**
>
> `output.reshape` does not copy pixels; it returns a second `Mat` header onto the same buffer, with the reference count raised. It is still a native object you own, which is why it goes through `own`. The `Mat` that `Core.transpose` fills **is** a fresh allocation, and `Managed.scope` releases both in reverse order — including when `get` throws part-way through the loop, which a `try` / `finally` written around a pair of bare `val`s would not.
>
> The `output` itself is safe to hold, which is not true of raw OpenCV: `Net.forward` hands back a header onto the layer's own buffer that the next pass overwrites in place. `Dnn.forward` copies before returning, so this frame's output and the previous frame's are genuinely two different things.

Suppression comes next, and it takes one decision that is easy to get wrong: NMS is a geometric operation with no notion of class, so running it across all candidates at once lets a high-scoring person delete the van they are standing in front of. Group by class first.

Example 27-4. Per-class non-maximum suppression through OpenCV's own implementation.

```
import scalacv.*
import org.opencv.core.{MatOfFloat, MatOfInt, MatOfRect2d, Rect2d}
import org.opencv.dnn.Dnn as CvDnn

def suppress(candidates: Seq[Detection], confidence: Float, iou: Float):
Seq[Detection] =
  if candidates.isEmpty then Seq.empty
  else
    Managed.scope: own =>
      val boxes = own(MatOfRect2d(candidates.map { d =>
        Rect2d(d.box.x.toDouble, d.box.y.toDouble, d.box.width.toDouble,
d.box.height.toDouble)
      }*))
      val scores = own(MatOfFloat(candidates.map(_.score)*))
      val keep = own(MatOfInt())
      CvDnn.NMSBoxes(boxes, scores, confidence, iou, keep)
      keep.toArray.toSeq.map(candidates) // plain data; nothing scoped escapes

def suppressPerClass(candidates: Seq[Detection], confidence: Float, iou: Float):
Seq[Detection] =
  candidates.groupBy(_.classId).values.flatMap(suppress(_, confidence,
iou)).toSeq
```

`MatOfRect2d`, `MatOfFloat` and `MatOfInt` are `Mat` subclasses, so the `Releasable[Mat]` given covers all three and `own` accepts them without ceremony — but they are native allocations like any other. Three per class per frame is ninety allocations a second at 15 fps with two classes, each of which has to be freed exactly once; that is what the scope is for.

Two thresholds, chosen from frames rather than from feel

The confidence threshold and the NMS IoU threshold are the only two numbers in the pipeline you get to choose, and they are frequently adjusted as though they were interchangeable. They are not.

Table 27-3. What each threshold actually moves.

	Raising it	Lowering it
Confidence	Fewer boxes survive. Precision rises, recall falls. Monotone, and reversible after the fact if you keep the raw candidates.	More boxes survive, including the ones the network was not sure about. Recall rises, precision falls.
NMS IoU	More overlapping boxes are kept, so a single object more often gets two or three boxes.	Boxes are merged more aggressively, so two genuinely adjacent objects — two people at a door, two vans in a row — collapse into one and the loss shows up nowhere in any score.

The asymmetry matters. Confidence is a filter on quality, and moving it trades precision against recall along a curve you can measure. The IoU threshold is a statement about the geometry of your scene: how much do the objects you care about genuinely overlap? A car park viewed from above has almost no true overlap and tolerates an aggressive 0.3; a queue of people viewed from head height overlaps constantly and needs 0.6 or more, at the cost of occasional duplicates. Reaching for the IoU threshold because recall is too low is a category error — it cannot recover a candidate that suppression never saw.

Picking the confidence threshold by watching a preview and stopping when it "looks about right" optimises for the twenty seconds of footage you happened to be looking at. The alternative costs an afternoon and is worth it. Take forty frames spread deliberately across the conditions that vary — dawn, midday glare, headlights, rain on the lens — and draw the true boxes for your two classes by hand. Run the pipeline once with `floor = 0.05f`, keeping every candidate with its score — and drop the `confidence` you hand `NMSBoxes` to the same 0.05, because it is a score threshold too and will otherwise quietly re-apply the number you were trying to measure. Then, offline, for each threshold from 0.05 to 0.95 in steps of 0.05, match predictions to labels greedily by descending score with a match counted at IoU 0.5 or better, and count true positives, false positives and misses. That gives you a precision and a recall per threshold, and the choice becomes a statement about the application rather than about the picture: a system that pages a human wants precision, because a pager that cries wolf is switched off; a system that counts vehicles per hour wants a stable operating point where the miss rate is predictable enough to correct for.

> **TIP**
> Pick the threshold per class. People and vans score differently — large, high-contrast, common objects score higher than small ones almost regardless of the model — and one number for both is a compromise nobody chose. A `Map[Int, Float]` from class id to threshold costs nothing and is applied at the same point in the loop.

If no threshold gives you an acceptable pair, stop turning it. Thresholds cannot manufacture detections that were never proposed, and a recall ceiling that will not move is telling you the input resolution is too low for the objects, or the model is wrong for the domain.

The whole loop

Everything assembled: load once, letterbox, forward, decode against a class filter, suppress per class, map back, and draw with a label plate readable against whatever is behind it — the plate from Chapter 14, *Drawing and Annotation*, sized from `Draw.textSize` so the descenders are not clipped.

Example 27-5. Detection over a video, with the model loaded exactly once.

```
import scalacv.*
import org.opencv.dnn.Net
```

```
val labels = Map(0 -> "person", 7 -> "truck")
val wanted = labels.keySet
val colours = Map(0 -> Scalar(60, 220, 60), 7 -> Scalar(60, 160, 250))
val Side = 640
val Confidence = 0.35f
val Iou = 0.5f

def labelled(img: Image, box: Rect, text: String, colour: Scalar): Image =
  val m = Draw.textSize(text, scale = 0.5)
  val plateW = m.size.width.toInt + 8
  val plateH = m.size.height.toInt + m.baseline + 6
  val top = math.max(0, box.y - plateH)
  img
    .drawRect(box, colour, Thickness.Stroke(2))
    .drawRect(Rect(box.x, top, plateW, plateH), colour, Thickness.Filled)
    .drawText(text, Point((box.x + 4).toDouble, top + m.size.height + 3),
Scalar.Black, scale = 0.5)

def annotate(net: Net, source: String, into: String): Either[CvError, Unit] =
  Camera.usingFile(source) { camera =>
    Recorder.using(into, camera.size, camera.fps) { recorder =>
      camera.foreach() { frame =>              // an owned Image, closed for you
        val (boxed, lb) = letterbox(frame.copy, Side)
        val found =
          try
            Dnn
              .blobFromImage(
                boxed.mat,                     // borrowed; boxed is still ours
to close
                scaleFactor = 1.0 / 255,
                size = Some(Size(Side.toDouble, Side.toDouble)),
                swapRB = true
              )
              .use { blob =>                   // the blob outlives the forward,
and only just
                Dnn.forward(net, blob).use { output =>
                  suppressPerClass(
                    decode(output, classCount = 80, wanted = wanted, floor =
Confidence),
                    Confidence,
                    Iou
                  )
                }
              }
          finally boxed.close()

        val annotated = found.foldLeft(frame) { (img, d) =>
          labelled(img, toFrame(d.box, lb), f"${labels(d.classId)}
${d.score}%.2f",
                   colours(d.classId))
        }
        try recorder.write(annotated).fold(throw _, _ => ())
        finally annotated.close()
      }
```

```
    }
  }.flatMap(identity)                          // two Eithers nest: camera open,
then recorder open

OpenCv.load()

val outcome: Either[CvError, Unit] =
  Dnn.fromOnnx("models/yolov8n-v3.onnx").flatMap { managedNet =>
    managedNet.use { net =>                    // loaded once, freed when this
block ends
      annotate(net, "loading-bay.mp4", "annotated.avi")
    }
  }
```

Four things in that program are worth naming. `frame.copy` exists because `letterbox` consumes what it is given and the original frame is still needed to draw on; branching a pipeline always costs one copy, and Chapter 4, *The Image Type*, explains why the type refuses to let you forget it. The `blob` is kept inside a `use` that outlives the `forward` call, because releasing it while a pass is in flight is a crash rather than an exception. `found.foldLeft(frame)` threads the image through the annotations precisely because each `draw` consumes its receiver — with no detections the fold returns the frame untouched, and `close` is idempotent, so the `finally` is correct either way. And the failures stay values: `fromOnnx`, `usingFile` and `Recorder.using` each return an `Either[CvError, …]`. The inner two nest into an `Either[CvError, Either[CvError, Unit]]` that `flatMap(identity)` collapses, and the model's own `Left` joins them through the outer `flatMap`. A model that will not load, a video that will not open and a codec this build cannot write are three distinct `Left`s that arrive at the same place, and none of them throws.

There is one honesty left in the label map. The loading bay has vans in it and COCO has no van class, so class 7, `truck`, is what a van is going to be called — sometimes, with the rest split between `car` and `bus` depending on the vehicle and the angle. The taxonomy belongs to whoever labelled the training set, and no threshold repairs a class boundary that was drawn somewhere else. Either fold the vehicle classes together into one count and say so, or accept that "van" is a category your model does not have and fine-tune one that does.

> **NOTE**
> `camera.foreach` hands you an owned `Image` copied out of the capture and closes it for you. The cheaper `Video.frames` yields one reused borrowed `Mat` instead, which is the right choice when you only read and reduce — but detection here annotates and re-encodes, so the copy is being paid for something.

From boxes to tracks

What this pipeline produces is a set of boxes per frame and nothing else. It has no memory. The person at the left of frame 400 and the person at the left of frame 401 are two unrelated assertions, and the pipeline cannot tell you that they are the same person, cannot count how

many distinct people passed through the bay in an hour, and cannot survive the single frame where the detector blinks and returns nothing.

That is not a defect in the detector; it is the boundary of what detection is. Chapter 28, *QR Codes, ArUco, and Augmented Reality*, sidesteps it by printing the object you want to find rather than training for it; Chapter 30, *Tracking*, closes it. `ObjectTracker.update` takes a bare `Seq[Rect]` — `found.map(d => toFrame(d.box, lb))` and nothing else, because it never looks at the image — matches those boxes against its live tracks by IoU, and hands back `ObjectTrack` values carrying a stable `id`, a `hits` count and an `age`. A track that goes unmatched is not deleted on the spot: it survives `maxAge` frames of misses before it is retired, so the frame where the detector blinks costs you one gap rather than a new identity, and `count` gives you the running total of distinct objects the bay has seen. That is the step where a box finally becomes an object with a history.

CHAPTER 28

QR Codes, ArUco, and Augmented Reality

The cheapest way to put a known object into a scene is to print one.

A camera looks down at a bench where plastic trays pass under an inspection head. You want three things from each frame: which tray is under the head, how far below the lens its surface sits, and whether it is lying square to the machine. A detector trained on trays gives you a tray-shaped rectangle and nothing else — not which of the four hundred trays in the building this one is, and not how far away it is, because it does not know how big a tray is: a small tray close to the lens and a large one further off project to the same rectangle. One photograph of an unknown object under-determines its distance, and no quantity of training data repairs that.

What repairs it is a measurement you supply. Four points on a plane whose coordinates you have written down, seen through a camera whose optics you have written down, determine the rigid transform between that plane and the camera — a solved problem with a name, perspective-*n*-point, and OpenCV ships the solver. The algebra was never the hard part. The hard part is having four points in the scene whose real positions you know.

A fiducial marker is the cheapest way to manufacture those four points: a printed square of measured side, carrying a pattern that stands out against clutter and encodes an integer. Finding it is not recognition and needs no model file — threshold the image, look for dark quadrilaterals on light ground, warp each candidate flat, read the bits, check them against a codebook. Tape one to a tray and the tray becomes an object whose identity, size and pose your program knows, for the price of a sheet of paper.

This chapter uses one running scene. Every tray carries two printed things: an ArUco tag, whose id is the tray number and whose corners carry the geometry, and a QR sticker, whose payload is the batch record. By the end you will decode the sticker, recover the tag's 3D pose, stand a wireframe cube on the tray, and record the result — and know which of those steps is lying when the cube wobbles.

Everything here that finds a marker lives in `scalacv-vision` — `Qr`, `Aruco`, `Ar` and the `Image` extension methods they carry — so that module has to be on the classpath beside `scalacv` itself, with the coordinates Chapter 2 lists. The camera model, `Intrinsics`, is the exception: it sits in the core module, which is why `Image.undistort` takes one without dragging the vision jar in. Use `import scalacv.*` for core and `import scalacv.vision.*` for marker APIs.

Two printed markers, two different jobs

The two families look alike on paper and are not interchangeable. A QR code is a data carrier: its geometry exists to make a payload readable, not to be measured. An ArUco marker holds an integer of at most ten bits and exists to be measured — fixed square shape, heavy border, and a codebook chosen so that a few misread bits resolve to the right id or to nothing at all.

Table 28-1. What each marker family is for, and what the library hands back.

Marker	Carries	Entry point	Result
QR code	arbitrary text, kilobytes of it	`image.qrCodes`, `Qr.detectAndDecode`	`Seq[QrCode]`
ArUco tag	one dictionary id, plus measurable geometry	`image.arucoMarkers`, `Aruco.detect`	`Seq[ArucoMarker]`
ArUco tag + camera model	id, rotation and translation	`image.arMarkers`	`Seq[MarkerPose]`

None of those calls hands back a native handle, and that is the ownership model rather than an omission.

> **NATIVE MEMORY**
> `QRCodeDetector` and `ArucoDetector` are two of the 185 `org.opencv.*` types that expose no public `release()` — only a private `delete(long)`. A program that constructs one per frame and lets the references go leaks native memory at a rate the heap never reveals: Chapter 1 measured 4000 leaked `KalmanFilter`s at 54 GB resident against 86 MB when released, and a detector is the same shape of object. scalacv frees them through `Releasable.nativeHandle`, the same bridge it uses for every handle-only type, inside the `Managed(...).use` block that created them. That is why a detector is never handed to you: an object with no public free is safest when its lifetime is a single call.

Reading the sticker

`image.qrCodes` is a borrowing query in the sense of Chapter 4: it reads the image and returns plain data, and the image is still yours afterwards.

Example 28-1. Decoding every QR code in a frame.

```
Image.reading("tray.jpg") { img =>
  img.qrCodes.map(c => s"${c.text} at ${c.corners.mkString(", ")}")
}
```

A `QrCode` is `final case class QrCode(text: String, corners: Seq[Point])`: the decoded payload, and the symbol's four corners in image coordinates, in OpenCV's order.

Underneath, `Qr.detectAndDecode(mat)` calls `detectAndDecodeMulti` unconditionally. OpenCV's single-symbol `detectAndDecode` is deliberately not exposed: it returns the first

symbol it happens to find and gives no way to learn there were others, so a tray with two stickers would become a tray with one.

The field that surprises people is `text`, because it can be empty. When OpenCV locates a symbol but cannot decode it — a blurred sticker, a thumb over one corner, a highlight across the finder patterns — the entry is still returned, with the corners filled in and the text empty. The library does not filter those out, and neither should you, at least not first:

```
img.qrCodes.filter(_.text.nonEmpty).map(_.text)   // discards the evidence you
needed
```

A located-but-undecoded code is not a non-event: it is a sticker present, framed and unreadable, which is actionable in a way that "no code found" is not. Partition instead, and draw the failures.

Example 28-2. Keep the codes that failed to decode --- they are the interesting ones.

```
Image.reading("tray.jpg") { img =>
  val (decoded, unreadable) = img.qrCodes.partition(_.text.nonEmpty)

  decoded.foreach(c => img.mat.drawPolyline(c.corners, true, Scalar.Green,
Thickness.Stroke(2)))
  unreadable.foreach(c => img.mat.drawPolyline(c.corners, true, Scalar.Red,
Thickness.Stroke(2)))

  img.drawText(s"${decoded.size} read, ${unreadable.size} unreadable", Point(10,
30))
     .write("tray-annotated.png")
}
```

`drawPolyline` is one of the mid-level drawing verbs from Chapter 14; it mutates the borrowed `Mat` in place, which is why the annotation happens before `drawText` consumes the image.

An image with no QR code is an empty `Seq`, not an error — there is nothing wrong with a blank bench. An empty `Mat` is an `IllegalArgumentException`, because handing a detector an image with no pixels is a bug rather than a fact about the world. That split is the error model of Chapter 6.

How large does the sticker have to be?

QR decoding has a resolution floor, expressed in pixels per module rather than pixels per code. A module is one of the small squares; a version-1 code is 21 modules on a side and each version adds four. Ask for more payload, or for a higher error-correction level at the same payload, and the code grows in modules, so the same sticker at the same distance puts fewer pixels on each one until the decoder stops. The trade runs the wrong way for intuition: raising error correction to survive scuffs makes the code harder to resolve at distance.

Motion blur destroys those module edges while leaving the finder patterns detectable — which is exactly how you end up with good corners and empty text. Glare does the same

locally. Both are reasons to keep an undecoded symbol: with the quad in hand you can crop that region out of a later frame.

The QR round-trip that killed the JVM

The library's own QR test encodes a payload with `QRCodeEncoder`, upscales it and decodes it straight back. For a while that test crashed the process: `SIGSEGV` inside `cv::Mat::release()`, called from `libopencv_objdetect`, with no Java frame to unwind.

Nothing was wrong with the detector. The old `OpenCv.load()` opened every OpenCV module library speculatively, and the bundled `libopencv_highgui.so` carries unversioned `NEEDED` entries, so loading it sent the linker down the system path and mapped six system `libopencv_*.so.5.0.0` libraries into the global namespace, where they interposed on the bundled 4.13.0 symbols. Decoding a QR code was the first operation in the suite to cross between the two ABIs, so it was the one that died. The loader is demand-driven now (Chapter 2).

The upscale is not incidental either: `QRCodeEncoder` emits one pixel per module, under the floor of the previous section, so the test resizes eightfold with `INTER_NEAREST` — anything smoothing would blur the module edges the decoder thresholds on — and only then adds the quiet zone.

The moral is not about QR codes: a native crash names the operation that tripped over the problem, not the code that caused it.

ArUco: identity you can measure

An ArUco dictionary is a codebook: a fixed bit-grid size, a fixed number of distinct markers, and a minimum Hamming distance between any two. `ArucoDictionary` is an enum over the predefined ones, and each name gives both numbers — `Dict5x5_250` is 250 markers on a 5 × 5 grid. There are 22 cases, AprilTag families and legacy sets included, all reachable through the same detector.

Table 28-2. The dictionary families, and what each costs you.

Family	Grid	Ids	Character
`Dict4x4_50` … `Dict4x4_1000`	4 × 4	50–1000	fewest bits: reads at the greatest distance, fewest ids
`Dict5x5_50` … `Dict5x5_1000`	5 × 5	50–1000	more ids, still forgiving
`Dict6x6_*`, `Dict7x7_*`	6 × 6, 7 × 7	50–1000	many ids, needs the tag to fill more pixels
`AprilTag16h5` … `AprilTag36h11`	AprilTag	587 for `36h11`	robotics toolchains; very high error tolerance
`ArucoOriginal`, `ArucoMip36h12`	legacy	1024 for `ArucoOriginal`	compatibility with existing printed tags

The rule that follows, and that people get backwards: pick the smallest dictionary with enough ids for the job, not the largest. Fewer markers means a larger Hamming distance between them, which means more bit errors survivable before a tag is rejected or, worse, misread. A bench with forty trays wants `Dict4x4_50`, not `Dict7x7_1000`, and the choice buys metres of working distance. Whatever you generated with, you must detect with.

Example 28-3. Every tag in the frame, with its id and its four corners.

```
Image.reading("bench.jpg") { img =>
  img.arucoMarkers(ArucoDictionary.Dict4x4_50).map(m => m.id -> m.corners)
}
```

`ArucoMarker(id: Int, corners: Seq[Point])` is the whole result type. The corners run clockwise from the top left of the marker's own frame, which is what makes the pose step possible: corner zero of a rotated tag is still the tag's top-left, not the image's.

A tag from the wrong dictionary comes back as nothing rather than as a wrong answer: ask for `Dict5x5_250` while the bench is printed with `Dict4x4_50` and the result is an empty `Seq`, because a 5 × 5 codebook demands more bits than a 4 × 4 pattern can satisfy. Silence is the correct failure, and it is also why a mismatched dictionary is such a frustrating bug — nothing is reported at all.

`Aruco.detect` builds a fresh `ArucoDetector` on every call, by decision rather than oversight: `ArucoDetector` copies the dictionary into its implementation at construction, so a shared instance would have to be immutable by convention or synchronised, and construction is cheap beside detection. Rejected candidates are discarded; they are a tuning aid, not a result.

> **NOTE**
> Everything OpenCV exposes for tuning detection — adaptive-threshold window sizes, the minimum candidate perimeter, corner refinement — lives on `org.opencv.objdetect.DetectorParameters`, which this library does not surface: `Aruco.detect` uses OpenCV's defaults. Drop to `org.opencv.objdetect` when you need those knobs; the low-level surface is never walled off.

Printing the tags

`Aruco.generateMarker` renders a marker so you have something to tape to a tray. It is the one call in this chapter that hands you ownership.

Example 28-4. Generate marker 7, give it a quiet zone, and write it out to print.

```
val bordered =
  Aruco
    .generateMarker(ArucoDictionary.Dict4x4_50, id = 7, sizePixels = 480)
    .use(_.border(60, 60, 60, 60, color = Scalar.White))
```

```
Image.wrap(bordered).write("tray-07.png")
```

The result is a `Managed[Mat]`: an 8-bit single-channel square of exactly `sizePixels` on a side, and yours to release — `use` does it here. What `border` returns may outlive that `use` block because `copyMakeBorder` writes a fresh Mat rather than a view of the source; a `submat` shares the parent's buffer, and letting one escape its owner is the aliasing mistake of Chapter 10. `Image.wrap` then takes over the bordered Mat, and `write` spends it.

A negative `id` or a non-positive `sizePixels` is an `IllegalArgumentException`. An id past the end of the dictionary, or a size too small for the bit grid, is a `CvError.NativeCall`, because OpenCV is what discovers it: `Dict4x4_50` holds markers 0 through 49, so 50 throws.

The `border` call is not decoration, and leaving it out is the most common reason a freshly generated marker "cannot be detected". `generateMarker` emits the marker's own black border and no quiet zone, and the detector looks for a dark quad on a light background, so a tag flush to the image edge is invisible to it however clean the bits are. The same holds for the printed article: leave white space around it, and do not mount one against a dark bracket.

From four corners to a pose

Detection gives you a quadrilateral in pixels. Pose estimation turns it into a position and an orientation in space, and it needs exactly two things from you that detection did not: the tag's real side length, and a model of the camera.

Example 28-5. Detect and solve in one step.

```
Image.reading("bench.jpg") { img =>
  val intrinsics = Intrinsics.approx(img.size, horizontalFovDegrees = 60)
  img.arMarkers(intrinsics, markerLength = 0.05, ArucoDictionary.Dict4x4_50)
     .map(mp => f"tray ${mp.id} at ${mp.distance}%.3f m")
}
```

`arMarkers(intrinsics, markerLength, dictionary)` detects every tag and solves each one's pose, returning `Seq[MarkerPose]`. A marker whose pose fails to solve is dropped, so the result can be shorter than `arucoMarkers` on the same frame. `MarkerPose(marker, pose)` exports `distance` from the pose and defines `id` as the marker's, so both read like fields; `mp.marker.corners` is still there when you want the pixels back.

`markerLength` is the tag's real side, in whatever unit you want the answer in; metres is the convention, and `distance`, the cube's edge and the height of a label anchor all inherit it. Measure the black square, not the paper.

When you already hold an `ArucoMarker` — filtered by id, or carried over from an earlier stage — solve it alone:

```
val pose: Option[Pose3D] = Ar.estimatePose(marker, markerLength = 0.05,
intrinsics)
```

`estimatePose` returns `Option[Pose3D]` because `solvePnP` can fail to converge — rare for four coplanar corners, not impossible. Four corners and a positive `markerLength` are required, and both are `IllegalArgumentException`s rather than a `None`, because they are mistakes rather than outcomes. It runs `solvePnP` with `SOLVEPNP_IPPE_SQUARE`, the square-planar solver, faster and steadier on a flat tag than the general iterative method.

> **NATIVE MEMORY**
> `estimatePose` needs six native `Mat`s for one call — object points, image points, the camera matrix, the distortion coefficients and two output vectors — and owns every one through `Managed.scope` (Chapter 5), so a throw from the fourth allocation frees the first three. `Pose3D` holds `Seq[Double]`, so the numbers are copied onto the heap before the scope closes and nothing native escapes. The same holds for `Ar.project`; neither hands you a handle.

What `rvec` actually is

`Pose3D(rvec: Seq[Double], tvec: Seq[Double])` looks like six numbers you can read off. `tvec` is: it is the translation from camera to marker, in your unit, and `pose.distance` is its length — the straight-line camera-to-tag distance, and often the only number you wanted.

`rvec` is not. It is a rotation in OpenCV's Rodrigues form: axis-angle packed into three numbers, the direction being the axis and the magnitude the angle in radians. Both fields are required to be exactly three elements long at construction, because `Pose3D` is public data you are expected to build by hand from another solver's output, and a four-element `rvec` would otherwise fail much later inside a native `put`, blaming a line that had nothing to do with the mistake.

Two consequences follow. First, an axis-angle triple is not a rotation matrix and not a set of Euler angles, and there is no shortcut between them. To compose two poses, transform a point into the camera frame, or hand an orientation to a robot controller, you convert — with `Calib3d.Rodrigues`, yourself:

Example 28-6. Rodrigues, when you need the rotation as a matrix.

```
import org.opencv.calib3d.Calib3d
import org.opencv.core.{CvType, Mat}

def rotationMatrix(pose: Pose3D): Seq[Seq[Double]] =
  Managed.scope: own =>
    val rvec = own(Mat(3, 1, CvType.CV_64F))
    rvec.put(0, 0, pose.rvec*)
    val rot = own(Mat())
    Calib3d.Rodrigues(rvec, rot)
    (0 until 3).map(i => (0 until 3).map(j => rot.get(i, j)(0)))
```

Second, axis-angle vectors do not add. Averaging a tag's `rvec` over five frames to smooth out jitter is arithmetic on a space that is not linear, and the error grows with the angles involved. Smooth the translation if you like; for rotation, convert to matrices or quaternions first, or filter the projected pixel points instead.

> **WARNING**
> A tag facing the camera head-on comes back with an `rvec` of magnitude near π, not near zero, and that is correct: OpenCV's marker frame is y-up while the image frame is y-down, so a fronto-parallel tag is a 180° flip about an axis lying in the marker plane. In-plane spin lives in the third component — `rvec(2)` near zero means the tag is not rotated in its own plane.

The camera model is the part you have to earn

`Intrinsics(fx, fy, cx, cy, distortion)` is the pinhole model: focal length in pixels on each axis, the principal point, and OpenCV's radial and tangential distortion coefficients. Focal lengths must be positive, and the distortion vector must have one of the counts OpenCV itself accepts — `Intrinsics.ValidDistortionSizes` is `Seq(0, 4, 5, 8, 12, 14)`, empty meaning an ideal lens. That check exists because a coefficient dropped from a five-element vector leaves a legal four-element one, so the native call runs happily and returns a quietly wrong pose. A rejected constructor costs a minute; a plausible wrong number costs a day.

`Intrinsics.approx(imageSize, horizontalFovDegrees = 60)` builds a model from the frame size and a guess at the field of view: centred principal point, square pixels, no distortion. A narrower field of view is a longer lens, so `fx` and `fy` grow as the angle shrinks.

> **CAUTION**
> `Intrinsics.approx` is enough to make an overlay sit on a tag and track it. It is not enough to measure with. The distance it reports is in the unit you gave `markerLength`, but its accuracy is bounded by the field-of-view guess, and being wrong by tens of percent is normal. A real lens also has distortion, which `approx` models as zero, so tags near the frame edge fare worse than tags near the centre. Chapter 31 turns the guess into a measured camera from a handful of chessboard photographs, and nothing else about your code changes: `Calibration` carries an `Intrinsics`, and every call here takes one.

Standing a cube on the tray

Two overlays are built in, and both do the whole detect-solve-project sequence internally.

Example 28-7. The AR hello-world: a wireframe cube on every tag.

```
Image.reading("bench.jpg") { img =>
  img.drawMarkerCube(Intrinsics.approx(img.size), markerLength = 0.05, color =
Scalar.Green)
     .write("bench-ar.png")
}
```

`drawMarkerAxes` draws a coordinate frame at each tag — X red, Y green, Z blue, Z pointing out of the tag toward the camera. It is the diagnostic overlay: axes that look glued to the paper mean the pose is right, and a Z axis that swings into the tag means it is not.

Table 28-3. The knobs on the two built-in overlays.

Overlay	Parameter	Default	Effect
`drawMarkerAxes`	`axisLength`	`None` → half the marker side	length of each drawn axis
`drawMarkerCube`	`color`	`Scalar.Green`	wireframe colour
`drawMarkerCube`	`size`	`None` → the marker side	cube edge length
both	`dictionary`	`ArucoDictionary.Dict4x4_50`	which codebook to detect

Both verbs consume the receiver and return the annotated image — move semantics, Chapter 4. That is safe inside `reading`, whose own `close()` on the way out is idempotent and finds the handle already spent; what it forbids is touching `img` again afterwards, which the compiler will not stop and the runtime will, with the use-after-release exception of Chapter 5. Take `copy` first if you need the original too. In a live loop you need neither: the frame flows straight through.

Each overlay runs its own detection and pose solve, so calling both on one frame detects the tags twice. When you want both, or anything of your own, call `arMarkers` once and project by hand.

Projecting your own geometry

`Ar.project(points, pose, intrinsics)` maps any `Seq[Point3]` in the marker's own frame through a pose and the camera to pixel `Point`s. That frame has its origin at the tag's centre, `x` right, `y` up and `z` out of the plane toward the camera, in the unit of `markerLength`. Empty in, empty out.

Example 28-8. A label floating 15 cm above each tray, anchored to its tag.

```
Image.reading("bench.jpg") { img =>
  val intr = Intrinsics.approx(img.size)
  img.arMarkers(intr, markerLength = 0.05).foreach { mp =>
    val pts = Ar.project(Seq(Point3(0, 0, 0), Point3(0, 0, 0.15)), mp.pose, intr)
    img.mat.drawLine(pts(0), pts(1), Scalar.Red, Thickness.Stroke(2))
    img.mat.drawText(s"tray ${mp.id}", pts(1), Scalar.Red)
  }
  img.write("bench-labelled.png")
}
```

Because `project` takes plain `Point3` data, anything you can describe as 3D points rides on the tag with no further machinery. `drawMarkerCube` is exactly this: eight corners and twelve edges, projected and stroked.

When the cube wobbles

The characteristic failure of marker AR is a cube that will not sit still. It jitters between two orientations, or it leans one way when the tag is left of centre and the other way when it is right of centre, while the detected corners themselves are visibly stable frame to frame.

That is almost never a detection problem, and chasing it in the detector wastes days. It is the camera model. `solvePnP` on a planar target has a two-fold ambiguity — two poses project the same four corners to nearly the same pixels — and how cleanly the solver separates them depends on the focal length you gave it. Feed it an `fx` wrong by tens of percent, routine for a field-of-view guess, and the two solutions sit close enough that a fraction of a pixel of noise flips the answer between them, which is why the cube snaps rather than drifts. The lens-edge lean has the same root: real distortion your model says is zero, worst at the corners of the frame.

The diagnosis needs no new machinery. Project the marker's own four corners back through the pose you recovered and compare them with the corners that were detected.

Example 28-9. A reprojection check: if this is large, do not trust the pose.

```
val h = markerLength / 2
val model = Seq(Point3(-h, h, 0), Point3(h, h, 0), Point3(h, -h, 0), Point3(-h, -
h, 0))
val err = marker.corners
  .zip(Ar.project(model, pose, intrinsics))
  .map((observed, projected) => observed.distanceTo(projected))
  .max
```

The library's own test asserts this stays under two pixels for a synthetic fronto-parallel tag. A large reprojection error with stable corners means the intrinsics; a large error with corners that dance means the image — exposure, motion blur, a tag too small in frame.

The bench loop

Everything above composes into the thing you actually ship: read frames, decode the sticker, solve each tag, draw the cube, write the annotated video.

Example 28-10. Detect, pose, overlay and record, from a live camera.

```
OpenCv.load()

val intrinsics = Intrinsics.approx(Size(1280, 720), horizontalFovDegrees = 60)
val markerLength = 0.05 // the printed black square, measured

Camera.using(0) { cam =>
  cam.recordTo("bench.avi") { frame =>
    frame.qrCodes.filter(_.text.nonEmpty).foreach(c => println(s"batch
${c.text}"))
```

```
    frame.arMarkers(intrinsics, markerLength).foreach { mp =>
      println(f"tray ${mp.id} at ${mp.distance}%.3f m")
    }

    frame.drawMarkerCube(intrinsics, markerLength, color = Scalar.Green)
  }
}
```

Three details there are load-bearing. `qrCodes` borrows, so the frame survives it and `drawMarkerCube` can consume it afterwards. `recordTo` returns `Either[CvError, Long]` inside `Camera.using`'s own `Either`, so the camera that would not open and the recorder that would not encode both reach you as values. And the file is `.avi`, because `recordTo` defaults to `Codec.Mjpg` — the one codec videoio can always write — and MJPG does not open in an `.mp4` container.

Where the numbers come from

Everything metric in this chapter — the reported distance, the cube's proportions, whether the overlay holds still — rests on `fx`, `fy`, `cx`, `cy` and the distortion coefficients, and `Intrinsics.approx` supplies all five by guessing. A wobbling cube is that guess's signature.

Chapter 31, *Camera Calibration*, replaces it with a measurement: a printed chessboard, a dozen photographs from varied angles, `Calibration.fromChessboard`, and a `reprojectionError` you can quote. The `Calibration` it returns carries an `intrinsics` field of exactly the type every call here already takes, so nothing else about the bench loop changes — and once it does, the cube sits at the right scale and the distances are measurements. If your cube wobbles, go there next.

Chapter 29, *Pose Estimation and Gestures*, comes first, and points the same solver at something you cannot print. `HeadPose.estimate` runs `solvePnP` against a canonical five-point 3D face, the known points supplied by a detected face's landmarks instead of by four corners of paper. It takes the same `Intrinsics`, and it performs for you the conversion this chapter did by hand: `Calib3d.Rodrigues` to a matrix, then `RQDecomp3x3`, so what comes back is a `HeadPose` of yaw, pitch and roll in degrees rather than an axis-angle triple.

CHAPTER 29

Pose Estimation and Gestures

Where the body is, and what it is doing.

A face detector answers a question with a rectangle. A pose estimator answers a much better question with a graph. Instead of "there is a person around here" you get "the left wrist is at (412, 208), the left elbow at (390, 265), and the model is 0.87 sure of the first and 0.41 sure of the second" — and from that you can ask things a bounding box never answers. Is the arm raised. Is the person sitting. Are the shoulders square to the camera. Is that hand making a fist.

None of that is answered by the network. A keypoint network emits a tensor: seventeen rows of three floats, or seventeen heatmap planes, depending on which model you exported. It does not tell you that row five is the left shoulder, and it certainly does not tell you that row five connects to row seven and row eleven. That knowledge — the naming and the connectivity — lives in the documentation of whichever model you downloaded, and every project that skips writing it down reinvents it, wrongly, three files later. Here it is a value: `PoseTopology`, a `Seq[String]` of names and a `Seq[(Int, Int)]` of index pairs, passed to the decode and passed again to the drawing.

This chapter sits on the far side of the boundary Chapter 23 drew. scalacv ships no pose weights and downloads none: the model file is yours, loaded with `Dnn.fromOnnx` exactly as Chapter 26 loaded every other network, and the library owns the typed result, the decode from a raw tensor into named pixel coordinates, the overlay, and a rule-based gesture layer on top. One estimator needs no model at all — head pose, solved from the five landmarks a `Face` already carries — and it is also the most native-memory-dense call in the book, since `solvePnP` wants six `Mat`s alive at once and hands none of them back.

One example runs the length of the chapter: a session monitor that reads a recorded video, tracks a hand, smooths the jitter out of the landmarks, and fires an event once when a gesture settles.

Everything new here — `Pose`, `PoseTopology`, `PoseEstimator`, `HeadPose`, `GestureRecognizer` and the two `Image` extensions — lives in `scalacv-vision`, next to the `Dnn` and `FaceDetect` of the last chapters, so `com.worxbend::scalacv-vision:0.4.2` has to be on the classpath. `Image`, `Camera`, `Point` and `Intrinsics` are core.

A skeleton is a graph with names

Three case classes carry a pose across the native boundary, all of them ordinary immutable Scala data. That matters more than it sounds. `Dnn.forward` hands back a `Managed[Mat]`, and

a result still shaped like a `Mat` is a result with a deadline — the end of the `use` block frees it. `PoseEstimator.decode` copies the numbers out into `Float`s and `Double`s, so what you hold afterwards outlives the frame, the blob, the tensor and the network, which is what lets the smoother later in this chapter keep the previous frame's pose in a field without owning anything native.

Example 29-1. The pose data model, as the source declares it.

```
final case class Keypoint(name: String, point: Point, score: Float)

final case class PoseTopology(names: Seq[String], edges: Seq[(Int, Int)]):
  def size: Int = names.size

final case class Pose(keypoints: Seq[Keypoint], topology: PoseTopology):
  def apply(name: String): Option[Keypoint]
  def confident(minScore: Float = 0.3f): Seq[Keypoint]
  def meanScore: Float
  def bones(minScore: Float = 0.3f): Seq[(Point, Point)]
```

A `Keypoint` is one landmark: its name, its position in image pixels — not normalised, and not in the model's input resolution, but in the coordinate space of the frame you handed in — and the model's confidence. A `PoseTopology` is the index scheme. A `Pose` is the pair, and it is where two `require`s live, both firing at construction rather than at use. `PoseTopology` rejects an edge indexing past the end of `names`. `Pose` rejects a keypoint count that disagrees with `topology.size`, quoting both numbers — without it the mismatch surfaces later as an `IndexOutOfBoundsException` from `bones`, whose edge indices are validated against the topology rather than the keypoints, or from `GestureRecognizer.recognize`, and both blame the wrong line.

Table 29-1. What you can ask a `Pose`, and what comes back.

Call	Result
`pose(name)`	`Option[Keypoint]` — the named landmark, or `None` if the model reported none under that name
`pose.confident(minScore)`	the keypoints at or above `minScore`; the default is `0.3f`
`pose.meanScore`	mean confidence over every keypoint, `0f` for an empty pose — a cheap "is anyone there"
`pose.bones(minScore)`	`Seq[(Point, Point)]` — the edges whose *both* endpoints clear `minScore`

`bones` is the one worth reading twice. It keeps a bone only when both endpoints are confident, so a half-detected limb draws nothing rather than drawing a line into the empty part of the frame where a low-confidence wrist was hallucinated — the difference between an overlay that looks like a skeleton and one that looks like a scribble.

The two topologies that ship

`PoseTopology.CocoBody17` is the seventeen-keypoint COCO body layout that both MoveNet and OpenPose(COCO) emit. Its names, in the model's output order, are `nose`, `left_eye`, `right_eye`, `left_ear`, `right_ear`, `left_shoulder`, `right_shoulder`, `left_elbow`, `right_elbow`, `left_wrist`, `right_wrist`, `left_hip`, `right_hip`, `left_knee`, `right_knee`, `left_ankle`, `right_ankle`. Its sixteen edges are grouped in the source the way you would draw them: four for the head, one across the shoulders, two per arm, three for the torso box, two per leg.

`PoseTopology.Hand21` is the twenty-one-landmark hand layout MediaPipe Hands uses, and it is generated rather than tabulated — a wrist followed by five fingers of four points each:

Example 29-2. `Hand21` is built, not listed.

```
val fingers = Seq("thumb", "index", "middle", "ring", "pinky")
val joints  = Seq("cmc", "mcp", "ip", "tip")
val names   = "wrist" +: fingers.flatMap(f => joints.map(j => s"${f}_$j"))
```

Landmark 0 is `wrist`, 1 through 4 the thumb ending at `thumb_tip`, and so on to `pinky_tip` at 20. The joint names are the thumb's — the other fingers are anatomically `mcp` / `pip` / `dip` / `tip`, and the source says so in a comment — because four points per finger is what the skeleton needs. The twenty edges follow the same pattern: a bone from the wrist to each finger's base, then three along the finger. If your model has a different keypoint set, build your own; `PoseTopology(names, edges)` is public and validates its edges immediately.

> **WARNING**
> A topology whose `names` are in the wrong order decodes perfectly and labels every point wrong. The tensor carries no names, so there is nothing scalacv can compare against — `pose("left_wrist")` will cheerfully return the right ankle. Write the topology from the model card, next to the code that loads the model.

Two output layouts, one decode

One `decode` serves both MoveNet and OpenPose because the difference between them is the encoding, not the keypoints. `KeypointLayout` names the two shapes that matter.

Table 29-2. The two tensor encodings `PoseEstimator.decode` understands.

`KeypointLayout`	Tensor	How one keypoint is read	Typical model
`Regression`	`[1, 1, K, 3]`	a row of `(y, x, score)`, normalised to `[0, 1]` and scaled up by `imageSize`	MoveNet
`Heatmap`	`[1, K, H, W]`	the arg-max cell of that keypoint's own `H × W` plane	OpenPose

Note the axis order in the regression case: the row is `y` then `x`, and `decode` puts them the right way round for you. That transposition is a classic afternoon lost to a skeleton that is a reflection of a skeleton.

Example 29-3. The decode signature. The topology defaults to the body layout.

```
def decode(
    output: Mat,
    imageSize: Size,
    layout: KeypointLayout,
    topology: PoseTopology = PoseTopology.CocoBody17
): Pose
```

`imageSize` is the size you want the keypoints expressed in — almost always the frame's own `img.size`, not the network's input size. You blobbed a 1280×720 frame down to 192×192 to run it, and you want the answer back in the frame's coordinates so the overlay lands on the person.

Both decoders validate the tensor before reshaping it, because OpenCV does not. A `Regression` output whose `total()` is not `K × 3`, or a `Heatmap` output that is not four-dimensional, raises a `CvError.NativeCall` naming the mismatch — "this is not the regression pose model this topology decodes" — instead of letting `reshape` throw a raw `CvException` about total sizes. Note the verb: `decode` returns a bare `Pose`, not an `Either`, so this `CvError` arrives *thrown*. It is not something the environment did to you, which is what Chapter 6 reserves a `Left` for, but a disagreement between two arguments you chose — the wrong `KeypointLayout`, or a topology whose `size` does not match the network. The heatmap path checks that second one on its own, with a `require` comparing the tensor's `K` against `topology.size` before it reads a plane.

Why you get one pose, not several

Both decoders return exactly one `Pose`. The regression path insists on `K × 3` values in total, which is one person's worth; the heatmap path takes the arg-max of each plane, and an arg-max has one answer even when the frame holds four people — so a crowd yields a skeleton assembled from several bodies.

That is a deliberate boundary. Multi-person OpenPose does not stop at heatmaps: it also emits part affinity fields, and turning those into per-person skeletons is a bipartite matching problem with its own tuning, not a decode. For multiple people, run a single-person model once per detected person box — detect, crop, estimate, map the keypoints back into frame coordinates — or write your own grouping over the output tensor `Dnn.forward` copies out for you.

Choosing between them

MoveNet-style regression is the layout to prefer when you have the choice. It is single-person by construction, its output is seventeen rows of three floats rather than seventeen full-resolution planes, and the decode is a `reshape` and a loop with no arg-max scan over `H × W` cells per keypoint. The Lightning export takes a 192×192 RGB input; the heavier Thunder export trades that input size for accuracy. Take the input size, the scale factor and the mean from the model card, not from this book: they are properties of the export, not of the layout.

Heatmaps are what you already have if you inherited an OpenPose pipeline, and they carry more than the decode uses — the peak's neighbourhood, the second peak, the affinity fields. For one skeleton the arg-max extracts the useful part and ignores the rest.

Arg-max is a quantised estimate

The heatmap decode maps a cell index straight into image coordinates: `px / w × imageWidth`. A keypoint's precision is therefore one heatmap cell, blown up by the ratio between the frame and the heatmap. Take an export with 46×46 planes — what a stride-8 network makes of a 368-pixel input — over a 1280-pixel-wide frame: every `x` lands on a 28-pixel grid, and a joint drifting across a cell boundary moves 28 pixels in one frame while the person moved one.

Refinements exist — fit a parabola to the peak and its neighbours, or read an offset head if the model has one — and neither is in `decode`, because both are model-specific. For a live overlay the practical answer is the temporal smoothing later in this chapter, which turns a 28-pixel jump into a few frames of glide.

From frame to skeleton

The full path is blob, forward, decode, and Chapter 26 covers the first two. When you need neither intermediate, `Image.estimatePose` collapses all three into one call — the pose counterpart to `image.faces(detector)`:

Example 29-4. The one-call form, with the defaults the extension declares.

```
def estimatePose(
    net: Net,
    inputSize: Size,
    layout: KeypointLayout,
    topology: PoseTopology = PoseTopology.CocoBody17,
    scaleFactor: Double = 1.0 / 255,
    mean: Scalar = Scalar(0, 0, 0),
    swapRB: Boolean = true
): Pose
```

Two of those defaults deliberately disagree with `Dnn.blobFromImage`, which defaults `scaleFactor` to `1.0` and `swapRB` to `false` because those are OpenCV's own. Here they are

aimed at a MoveNet-style export — RGB input, values scaled into `[0, 1]` — because that is the model this method exists to make easy. They remain model-specific knobs: pass what your card documents.

`estimatePose` borrows both arguments. The image stays alive and the `Net` is not released, which is what lets you call it once per frame with the network living outside the loop.

> **NATIVE MEMORY**
> A `Net` is one of the 185 generated OpenCV types with no public `release()`; scalacv frees it through the `delete(long)` bridge, so `Dnn.fromOnnx` hands back a `Managed[Net]` you must scope. Load the model *outside* the frame loop. A `fromOnnx` inside `foreach` leaks one network per frame, and an ONNX graph is not a rounding error: that is the difference between a process that runs for days and one killed at the end of the first clip.

Drawing it

`img.drawSkeleton(pose)` is a drawing transform in the sense of Chapter 14. It draws `pose.bones(minScore)` as two-pixel lines in `color` — green by default — and `pose.confident(minScore)` as filled three-pixel circles in `jointColor`, red by default. The topology does all the work: `bones` is the only thing that knows an elbow connects to a wrist, so a different topology changes the drawing without touching the call.

Being a transform, it *consumes* its receiver, and the move semantics from Chapter 4 apply without exception. That combines with `Camera.foreach` — which hands you an owned `Image` and closes it when your block returns — in a way that catches people out:

Example 29-5. Wrong. The annotated frame is never released.

```
cam.foreach() { frame =>
  val pose = frame.estimatePose(net, Size(192, 192), KeypointLayout.Regression)
  frame.drawSkeleton(pose)          // consumes `frame`, returns a new Image —
dropped
}
```

`drawSkeleton` takes the Mat out of `frame`'s handle without freeing it, mutates it, and rewraps it in a fresh `Image`. The `frame.close()` that `foreach` runs afterwards finds a spent handle and does nothing — correctly, since it now owns nothing — while the annotated `Image` you discarded still owns the pixels: one frame's worth of leak per frame, at video rates. End the chain in a terminal:

Example 29-6. Right. `write` is a terminal: it releases what it wrote.

```
var n = 0
cam.foreach() { frame =>
  val pose = frame.estimatePose(net, Size(192, 192), KeypointLayout.Regression)
  frame.drawSkeleton(pose, minScore = 0.3f).write(s"frames/$n.png")
```

```
  n += 1
}
```

> **NATIVE MEMORY**
> The rule generalises past `drawSkeleton`: any transform inside a `foreach` block moves ownership out of the `Image` the loop was going to close for you. Finish with a terminal (`write`, `bytes`, `close`), or hand the result to a `Recorder`. Nothing throws in the listing above, which is what makes the leak quiet; the neighbouring mistake — touching `frame` again after the transform — does throw, and under `-Dscalacv.trackOwnership=true` that `IllegalStateException` carries the stack of the call that spent the handle as its cause.

Head pose, and six matrices at once

Head pose is the estimator with no model file. It takes the five landmarks a `Face` from the YuNet detector already carries — right eye, left eye, nose tip, right mouth corner, left mouth corner, where "right" is the subject's right and therefore the image's left — and solves for the orientation of a canonical 3D head that would project to those points.

That reference is five `Point3`s in arbitrary units, nose tip at the origin, `x` to the image right, `y` down and `z` away from the camera: eyes at `(±45, -34, 27)`, nose tip at `(0, 0, 0)`, mouth corners at `(±30, 35, 22)`. It is a generic head, not yours, which is the first and largest reason the answer is indicative rather than metric.

The second input is the camera. `solvePnP` cannot turn pixels into angles without a focal length and a principal point, so `HeadPose.estimate` takes an `Intrinsics` — the pinhole model Chapter 31 measures with a chessboard. A second overload takes a `Size` and fabricates one: focal length equal to the image width, principal point centred, no distortion. It exists so you can run head pose the moment you have a face, and it is a guess.

Example 29-7. Head pose over a detected face, with a camera model.

```
FaceDetect.create("models/face_detection_yunet_2023mar.onnx", Size(320,
320)).flatMap { detector =>
  detector.use { yunet =>
    Image.reading("portrait.jpg") { img =>
      val intrinsics = Intrinsics.approx(img.size, horizontalFovDegrees = 60)
      img.faces(yunet).flatMap(face => HeadPose.estimate(face, intrinsics))
    }
  }
}
```

What the call actually owns

`solvePnP` wants object points, image points, a camera matrix, distortion coefficients and two output vectors — six `Mat`s alive at once — and `Rodrigues` and `RQDecomp3x3` want three more.

None is returned to the caller; every one has to be freed, including where a constructor throws part-way through.

Nested `Managed.use` blocks would bury the two interesting lines under nine levels of indentation, and the alternative people reach for instead — `val`s followed by `try`/`finally` — has a real hole, since every allocation before the `try` is unguarded. `Managed.scope` from Chapter 5 is both: `own` registers each object the moment it exists, so a throw anywhere releases everything acquired so far, in reverse order.

Example 29-8. The body of `HeadPose.estimate`, with the `org.opencv.calib3d.Calib3d` prefix shortened.

```
Managed.scope: own =>
  val objectPoints = own(MatOfPoint3f(model*))
  val imagePoints  = own(MatOfPoint2f(face.landmarks.map(_.toCv)*))
  val camera       = own(intrinsics.cameraMatrix)
  val distortion   = own(intrinsics.distCoeffs)
  val rvec         = own(Mat())
  val tvec         = own(Mat())
  Cv.attempt("solvePnP") {
    val ok = Calib3d.solvePnP(
      objectPoints, imagePoints, camera, distortion,
      rvec, tvec, false, Calib3d.SOLVEPNP_EPNP
    )
    if !ok then None
    else
      val rotation = own(Mat())
      Calib3d.Rodrigues(rvec, rotation)
      val euler = Calib3d.RQDecomp3x3(rotation, own(Mat()), own(Mat()))
      Some(HeadPose(yaw = euler(1), pitch = euler(0), roll = euler(2)))
  }.getOrElse(None)
```

Three details repay a pause. `own(Mat())` appears twice as an argument to `RQDecomp3x3`, because OpenCV needs somewhere to write the decomposition's factor matrices and this code wants neither — registering them inline says "allocate this, free it, never look at it". The block runs inside `Cv.attempt` because degenerate landmarks can make OpenCV throw rather than return `ok = false`, and the contract is `None` on failure, so `.getOrElse(None)` folds both failure shapes together. And nothing acquired in the scope escapes: what comes out is a `HeadPose`, three `Double`s.

> **NATIVE MEMORY**
> `Managed.scope` releases everything `own` registered, exactly once, at the end of the block — which is why `own` hands back the bare object rather than a `Managed`. A scoped handle has no second owner to defend against. The corollary is the rule you must not break: never return one of these objects from the block and never stash one in a field. Return plain data.

Yaw, pitch and roll, and the sign that flips

`RQDecomp3x3` returns Euler angles in degrees about the `x`, `y` and `z` axes, and `HeadPose` renames them for the head: `pitch` is the nod, `yaw` the turn, `roll` the tilt toward a shoulder. For "is this person looking at the screen" they are enough.

They are not enough for a number on a chart. The five landmarks sit on a shallow, nearly planar patch of the face, and near-planar point sets are where perspective-n-point is weakest: two solutions — a head turned slightly left and one turned slightly right — project to almost the same five pixels. A pixel or two of landmark noise moves the solver between them, so an uncalibrated yaw near zero can change sign between consecutive frames of a person sitting still. The model points are a generic head, `SOLVEPNP_EPNP` runs with no refinement pass, and the Euler decomposition has a branch that flips near the poles. Trust "looking left", "looking up", "head cocked". Do not trust "14.2 degrees" — for that you want a dedicated head-pose network through `Dnn` and a calibrated camera.

Gestures: geometry, not weights

A gesture recogniser sounds like a model and is not. `GestureRecognizer` has no weights, no state and no `Net`: it is a deterministic function from a twenty-one-landmark hand pose to a name, testable with a hand-built `Pose` and an assertion. It knows six answers, and the enum is closed:

Example 29-9. The whole vocabulary.

```
enum HandGesture:
  case Fist, OpenPalm, ThumbsUp, Pointing, Victory, Unknown
```

One geometric idea decides everything. For each finger, compare the distance from the wrist to the fingertip against the distance from the wrist to that finger's middle joint: tip farther means extended, tip nearer means curled. The comparison is orientation-independent — it holds whichever way the hand is rotated in frame, which is why it survives a hand held sideways where a "tip is above the joint" rule would not.

Table 29-3. The landmark pairs the extension rule compares, by `Hand21` index.

Finger	Tip	Joint	`Hand21` names
Thumb	4	2	`thumb_tip` vs `thumb_mcp`
Index	8	6	`index_tip` vs `index_mcp`
Middle	12	10	`middle_tip` vs `middle_mcp`
Ring	16	14	`ring_tip` vs `ring_mcp`
Pinky	20	18	`pinky_tip` vs `pinky_mcp`

The joint is always the landmark two before the tip. Under `Hand21`'s uniform naming that reads `..._mcp`; in MediaPipe's anatomy the non-thumb ones are the PIP joints. The index

is what the code uses, so the difference costs nothing but a moment of confusion when you print them.

`minScore`, defaulting to `0.3f`, gates the whole thing: a tip reported below that confidence counts as not extended, wherever it sits. That is the right failure direction: an occluded ring and pinky that the network placed by guesswork would put the count at four and turn a `Victory` into an `OpenPalm`, and gating on the tip's score is what stops them.

From the five booleans the mapping is one pattern match. No finger extended is a `Fist`; the thumb alone is `ThumbsUp`; the index alone `Pointing`; index and middle together `Victory`; four or more fingers out `OpenPalm`. Everything else is `Unknown` — not a failure, but the honest answer for a shape outside a six-word vocabulary, including every shape a hand passes through between two named ones.

> **WARNING**
> `recognize` requires a 21-landmark pose and says so with a `require`. Hand it a `CocoBody17` pose and it fails at the call, naming both sizes, rather than reading five body keypoints as fingers and returning a confident, meaningless `Pointing`.

A gesture is a trajectory; this layer sees a frame

The word "gesture" promises more than a per-frame rule can deliver. A held handshape — a thumbs-up, a fingerspelled letter — is a single hand configuration, which is exactly what `GestureRecognizer` reads. A wave, a swipe, or almost any sign in a signed language is movement: the meaning lives in how the landmarks travel over time, and no function of one frame can answer it.

scalacv's role there is upstream. You accumulate decoded poses in a sliding window, flatten each into its `(x, y)` pairs, and hand the window to a temporal classifier — an LSTM or a transformer, exported to ONNX and run through `Dnn` — that you train. The library gives per-frame landmark extraction and a small static vocabulary; the sequence model is yours, and nothing here ships one.

What you *can* build from the static layer is the thing most interfaces actually want: a gesture that has *settled*. That is temporal too, but it fits in ten lines.

Making it steady enough to act on

Three techniques turn a jittering per-frame reading into something you would let control a machine. None is in the library: all three depend on your frame rate and your tolerance.

Filter by confidence, twice

`pose.meanScore` is the frame-level gate: near zero means no hand in view, and running a gesture rule over a pose the network invented is worse than skipping the frame. `minScore` is the landmark-level gate inside `recognize`, `confident` and `bones`. Use both.

Smooth the landmarks, not the answer

Majority-voting over `HandGesture` values throws away the sub-pixel information that would have prevented the flapping in the first place. Smooth the geometry instead — an exponential moving average per keypoint is enough, and because `Pose` is immutable data it is a `map`:

Example 29-10. An EMA over poses. Low-confidence points hold their last position instead of jumping.

```
final class PoseSmoother(alpha: Double = 0.4, minScore: Float = 0.3f):
  private var previous: Option[Pose] = None

  def apply(pose: Pose): Pose =
    val smoothed = previous match
      case None => pose
      case Some(prev) =>
        val kps = pose.keypoints.zip(prev.keypoints).map: (now, was) =>
          val a = if now.score >= minScore then alpha else 0.0
          val x = was.point.x + a * (now.point.x - was.point.x)
          val y = was.point.y + a * (now.point.y - was.point.y)
          now.copy(point = Point(x, y))
        Pose(kps, pose.topology)
    previous = Some(smoothed)
    smoothed
```

A smaller `alpha` is steadier and laggier; `0.4` is a reasonable start at thirty frames per second. An unconfident keypoint gets `a = 0`, so it keeps its previous position rather than teleporting to wherever the network guessed — usually what you want for an overlay, and exactly what you do not want if the joint is genuinely moving fast. Every filter makes that trade.

The `zip` quietly assumes both poses share a topology. If they ever do not, it truncates to the shorter, and the `Pose` it then builds fails the constructor `require` from the first section of this chapter — loudly, at the frame where the topologies diverged, instead of producing a skeleton with the wrong names on it.

Normalise before you compare

Raw pixel coordinates encode how far the subject is from the camera. A shoulder-to-elbow distance of 90 pixels means one thing at two metres and another at four, so a threshold written against raw pixels is a threshold against distance. Before comparing shapes across frames, subjects or sessions, translate the keypoints relative to an anchor — the wrist for a hand, the hip midpoint for a body — and divide by a span that scales with the subject: for a hand, landmark 0 to landmark 9, the wrist to the middle finger's knuckle, which barely changes with the handshape.

Angles need this less than distances do, which is the argument for working in angles: the angle at an elbow, from three keypoints, is already scale-invariant. Compute it from smoothed points even so, because an angle amplifies the noise in both limbs that form it.

The session monitor

Everything above assembles into the running example: read a recorded session, decode a hand pose per frame, smooth it, and fire an event exactly once when a gesture becomes stable — not on every frame it is held, and not during the `Unknown` shapes a hand passes through on the way. The latch that turns a per-frame reading into an event counts consecutive agreeing frames and reports only the transition:

Example 29-11. Debounce: report a gesture once, when it has held for `hold` frames.

```
final class GestureLatch(hold: Int = 4):
  private var candidate: HandGesture = HandGesture.Unknown
  private var run = 0
  private var reported: HandGesture = HandGesture.Unknown

  /** Some(gesture) on the frame it becomes stable; None on every other frame. */
  def update(gesture: HandGesture): Option[HandGesture] =
    if gesture == candidate then run += 1
    else
      candidate = gesture
      run = 1
    if run >= hold && candidate != reported then
      reported = candidate
      Some(candidate)
    else None
```

At thirty frames per second, `hold = 4` is about 130 milliseconds of steadiness — long enough to reject transitional shapes, short enough that the interface still feels immediate. Tune it against your frame rate, not against a number in a book.

Example 29-12. The whole monitor: video in, gesture events out.

```
Dnn.fromOnnx("models/hand_landmark.onnx").flatMap { managedNet =>
  managedNet.use { net =>
    val smoother = PoseSmoother(alpha = 0.4)
    val latch    = GestureLatch(hold = 4)

    Camera.usingFile("session.mp4") { cam =>
      cam.foreach() { frame =>
        val raw = frame.estimatePose(
          net,
          inputSize = Size(224, 224),
          layout = KeypointLayout.Regression,
          topology = PoseTopology.Hand21
        )
        if raw.meanScore >= 0.3f then
          val pose = smoother(raw)
          latch.update(GestureRecognizer.recognize(pose)).foreach {
            case HandGesture.Victory => println("start")
            case HandGesture.Fist    => println("stop")
            case other               => println(s"ignored: $other")
```

```
          }
        }
      }
    }
  }
```

Trace the lifetimes once, because they are the point. `fromOnnx` yields a `Managed[Net]` and `use` releases the network when the video ends, on an exception, on anything. `Camera.usingFile` closes the capture the same way and returns an `Either[CvError, Unit]`, so a missing file is a value rather than a surprise. `foreach` hands over an owned `Image` per frame and closes it, and this block never consumes it — `estimatePose` only borrows `frame.mat` — so there is nothing to leak; the blob and the output tensor live and die inside that call. The only long-lived native object is the one `Net`.

The `Pose` values crossing between iterations are plain data, so `PoseSmoother` holds the previous frame's with no risk: there is nothing native left in it to dangle.

Where the numbers come from

Both of this chapter's estimators asked for something they could not produce. The gesture layer wanted landmarks, which came from a model Chapter 23 showed you how to fetch and verify and Chapter 26 how to load. Head pose wanted an `Intrinsics`, and every example here either guessed one from the image width or approximated one from a field-of-view estimate — serviceable for "the head is turned", not for a measurement. Chapter 31, on camera calibration, is where that guess becomes a chessboard, a set of views, and the focal lengths, principal point and distortion coefficients of your actual lens.

The debt the smoother and the latch left is nearer. Both are per-frame machines wearing a thin coat of memory: the smoother remembers one pose, the latch remembers one gesture, and neither has any notion of *which* hand it is watching. Put a second hand in the frame and a single-person decode hops between them, silently; the smoother averages two hands into one, and the latch reports the hop as an event. Chapter 30, *Tracking*, is the piece that fixes it — giving per-frame detections an identity that survives the next frame, so the state a smoother or a latch carries can belong to an object rather than to the camera.

CHAPTER 30

Tracking

Turning per-frame detections into objects with identity over time.

A detector has no memory. That is not a shortcoming to be apologised for — it is the contract. The face detector of Chapter 24 is handed one image and answers a question about that image; the motion detector of Chapter 21 compares a frame against a background model and reports the regions that differ. Neither is told what came before. Run either over a corridor camera and you get a list of boxes per frame, twenty-five times a second, with nothing joining them up.

So consider the smallest useful question a corridor camera can be asked: how many people walked through today. You have the boxes. You cannot count them, because the same person contributes a box to every frame they are visible in, and a detector that returns two boxes on frame 41 and two boxes on frame 42 has told you nothing about whether that is two people or four. The boxes on frame 42 are not even in a dependable order. `Image.contours` hands back regions in whatever order OpenCV walked the mask, and `MotionDetector`'s `Motion.regions` is sorted largest first, so the two swap places the moment one person walks nearer the camera than the other. There is no index you can trust and no field to compare.

Tracking supplies the missing thread. It takes the per-frame boxes and decides which box now is the same object as which box a moment ago, attaching an identity that persists: not "a person at (310, 88)" but "person #3, who was at (290, 86) last frame and has been in view for two seconds". Three things fall out of that. Identity makes counting possible, because a count is over distinct objects rather than over sightings. Continuity survives a detector that blinks: a person half-hidden behind a pillar stops producing detections, and a tracker that models motion carries the identity across the gap and picks it up on the far side. And a motion model smooths the jitter that makes a detector's box wobble by several pixels even on a stationary object.

scalacv puts all of this in the `scalacv.vision` package — `Tracker.scala`, `Kalman.scala` and `ObjectTracker.scala`, one type per file — so every type in this chapter needs `scalacv-vision` on the classpath beside `scalacv` — the same module as the detectors of Chapters 24 to 29, and as the `MotionDetector` of Chapter 21. It arrives in three layers that build on each other: a `Kalman` filter that models one moving point, a `Tracker` that follows one object by its appearance without any detector at all, and an `ObjectTracker` that runs a bank of Kalman filters to give many detections stable identities. Every one of them owns native memory that the Java bindings will not free, which is why this chapter's native-memory callouts run longer than most.

The Kalman filter, without the matrices

A Kalman filter is usually introduced as five matrix equations, which is accurate and unhelpful. The idea underneath is this: you have two independent statements about where an object is, and both are wrong. The first is a prediction from a model of how things move — it was here, it was going that way, so it should now be there. The second is a measurement — the detector says it is here. The filter's whole job is to blend them in proportion to how much you trust each, and to keep track of how uncertain the blend is so that the next blend can be weighted better.

`Kalman.point` builds the specific filter scalacv needs: constant velocity over a 2D point. Its state has four numbers, position and velocity, `(x, y, vx, vy)`, and its motion model says that each step adds the velocity to the position with a time step of one frame. The measurement is only the position, two numbers; velocity is never observed, it is inferred from how the position keeps changing. That inference is what makes the filter able to coast: after a few frames of a steadily moving object it has learned `vx` and `vy`, and a prediction with no measurement behind it still moves.

There are exactly two verbs, and they alternate.

Example 30-1. The two calls, and what each returns.

```
val k = Kalman.point(Point(0, 0))
try
  for i <- 1 to 5 do
    k.predict()                    // advance the model: where it *should* be
    k.correct(Point(i * 10.0, 0))  // fold in the reading: where it *is*
  val next = k.predict()           // one step past the last measurement
  println(f"next x = ${next.x}%.1f")   // past the last reading of 50
finally k.close()
```

`predict()` advances the state one step and returns the predicted `Point`; `correct(measurement)` folds a reading in and returns the corrected — that is, smoothed — `Point`. Skipping the `correct` is legal and is the entire point of having a model: a frame in which the detector found nothing is a frame in which you call `predict` and trust it. Do that twice and the box keeps gliding at the velocity the filter last believed in, which is what you want while an object is behind a pillar and not at all what you want fifty frames after it left the scene. That trade-off is what `maxAge` settles, below.

Two parameters set the filter's personality, and they pull against each other.

Table 30-1. The two noise knobs on `Kalman.point`.

Parameter	Default	Larger means
`processNoise`	`1e-2`	the model is allowed to drift more, so the filter chases the measurements: more responsive, more jitter

`measurementNoise`	`1e-1`	the measurements are trusted less, so the filter leans on the model: smoother, laggier

Both are variances written onto identity covariance matrices, so what matters is which is larger, not the absolute value. A box that twitches on a standing object wants a higher `measurementNoise`; a box that lags behind a fast walker wants a higher `processNoise`. Change one at a time on a recorded clip.

> **NATIVE MEMORY**
>
> `Kalman` wraps `org.opencv.video.KalmanFilter`, which is one of the 185 native-owning binding types with no public `release()`. It is freed through `Releasable.nativeHandle`, the reflective bridge onto the binding's private `delete(long)` described in Chapter 5, and it is freed only because you `close()` the `Kalman` — or let `Using.resource` do it.
>
> This is the type the project measured to make the point. Four thousand `KalmanFilter`s allocated and dropped without release came to **54 GB** of resident memory; the same four thousand released came to **86 MB**. Roughly six hundred and forty times, for an object the heap reports as a few dozen bytes — and the heap never grows enough to provoke a collection, so nothing reclaims them. Chapter 1's flat-heap, rising-RSS graph is this measurement drawn out over time.
>
> Four thousand is not a contrived number. `ObjectTracker` allocates one filter per track and a track is born for every detection that matches nothing, so a busy scene with a noisy detector births hundreds an hour.

Why the filter's own matrices can be released

`Kalman.point` configures the filter by fetching its internal matrices — `get_transitionMatrix`, `get_measurementMatrix`, `get_processNoiseCov`, `get_measurementNoiseCov`, `get_errorCovPost`, `get_statePost` — writing to each one, and releasing it through `Managed.use`. Read that quickly and it looks like a bug: the setup frees the filter's own transition matrix, then goes on to use the filter for the rest of its life.

It is safe because the generated binding does not hand back the member. It hands back a ref-count-sharing header copy, so `release()` drops the extra reference and the matrix the filter holds survives. That is an assumption about OpenCV's Java bindings, not a guarantee in scalacv's own code, so it is pinned by a test: five hundred `predict` / `correct` steps along a straight diagonal, asserting the filter is still tracking the line at the end. Had a release genuinely freed the transition matrix, a run that long would wander off it.

The construction is failure-safe for the same reason it has to be: `Kalman.point` builds the `Managed` first and wraps the setup in a `try` / `catch` that releases and rethrows, because a filter that throws halfway through configuration is otherwise stranded — unreachable and unreleasable, with no public `release()` for anyone else to call.

ObjectTracker: the tracking-by-detection loop

One filter smooths one point. Turning a stream of detections into a set of identities needs a filter per object plus a rule for deciding which detection belongs to which object, and that is `ObjectTracker` — the pattern usually called SORT, here in its plainest form.

Before the right version, the wrong one, because it is the mistake everybody makes once. Detections arrive as a `Seq[Rect]`, and a `Seq` has indices, and indices look like identities:

```
// Wrong. Contour order is however OpenCV walked the mask, not who is who.
val boxes = Using.resource(frame.copy.gray.threshold(128)): mask =>
  mask.contours().map(_.boundingRect)
boxes.zipWithIndex.foreach((box, i) => trailFor(i) += box)
```

Element 0 is whichever region the scan hit first. Two people crossing swap places in that ordering as soon as one of them starts a row earlier than the other, and a person leaving the scene shifts every index after them by one. The trails cross over and nothing warns you.

`ObjectTracker` replaces the index with an id it maintains itself. `update` takes this frame's detections and returns the tracks confirmed this frame, and it does six things in order:

1. **Predict.** Every live track is advanced one step by its own `Kalman`, its `box` is moved to the predicted centre keeping its last known size, and its `age` and `timeSinceUpdate` are incremented.
2. **Associate.** Every (track, detection) pair whose intersection-over-union reaches `iouThreshold` is a candidate; the candidates are sorted by IoU descending and taken greedily, each track and each detection used at most once. This runs against the **predicted** boxes from step 1, which is what lets association keep up with a fast-moving object.
3. **Correct.** Each matched track feeds its detection's centre to `correct`, adopts the detection's size and box, increments `hits`, and resets `timeSinceUpdate` to zero.
4. **Birth.** Every detection that matched nothing spawns a track with a fresh id, a new `Kalman` seeded at its centre, and `hits` of one. The running `count` goes up.
5. **Death.** Every track whose `timeSinceUpdate` now exceeds `maxAge` is retired and its filter closed.
6. **Report.** The tracks with at least `minHits` hits **and** a `timeSinceUpdate` of zero are returned as `ObjectTrack` values.

Ids come from a counter that only ever increases, so an id is never reused and `count` is the number of distinct objects the tracker has ever seen. It never looks at the image: `update` takes `Rect`s and returns `ObjectTrack`s, so it composes with any detector in the book — motion regions from Chapter 21, faces from Chapter 24, DNN boxes from Chapter 26, contour bounding boxes from Chapter 12.

Table 30-2. What an `ObjectTrack` carries.

Field	Meaning
`id`	the stable identity: unique, monotonically increasing, never recycled
`box`	the current bounding box
`hits`	how many frames this track has been matched to a detection
`age`	how many frames this track has existed

One consequence of step 6 is worth stating plainly, because the smoothing story invites the opposite assumption: only tracks with `timeSinceUpdate == 0` are reported, and those are exactly the tracks that took their detection's box in step 3, so the `box` you get back is always the raw detection. The Kalman prediction drives association and coasting; it does not soften what you draw. For a smoothed box on screen, smooth it downstream of `update` yourself.

Example 30-2. Two objects, three frames, two stable ids.

```
val tracker = ObjectTracker.create(iouThreshold = 0.3, maxAge = 5)
try
  val frames = Seq(
    Seq(Rect(10, 10, 20, 20), Rect(120, 120, 20, 20)),
    Seq(Rect(14, 10, 20, 20), Rect(124, 120, 20, 20)),
    Seq(Rect(19, 11, 20, 20), Rect(129, 121, 20, 20))
  )
  frames.foreach(dets => println(tracker.update(dets).map(_.id).sorted))
  println(s"distinct objects: ${tracker.count}")
finally tracker.close()
```

```
List(0, 1)
List(0, 1)
List(0, 1)
distinct objects: 2
```

Ids start at zero and are handed out in the order detections arrive on the frame that births them, so the leftmost box in this clip is `\#0` for as long as it is tracked. Nothing about that ordering is guaranteed on a real clip — what is guaranteed is that once an id is attached it stays attached until the track dies, and that the id is never handed to a second object.

Tuning, and where it goes wrong

`ObjectTracker.create` takes three numbers and they are the whole configuration. Two of them are checked: `iouThreshold` must lie in `[0, 1]` and `maxAge` must not be negative, both enforced by `require`, both programming errors rather than expected failures, so both throw rather than returning a `Left`.

Table 30-3. The three knobs, their defaults, and which way to turn them.

Parameter	Default	Turn it

`iouThreshold`	`0.3`	**down** when boxes jump between frames — a fast object at a low frame rate can leave its own predicted box entirely, and no overlap means no match; **up** when tracks latch onto the wrong neighbour in a crowd
`maxAge`	`5`	**up** to bridge occlusions, at the cost of a ghost track coasting across the frame after the object has gone; **down** for a clean scene where a lost object is genuinely gone
`minHits`	`1`	**up** when the detector produces one-frame false positives, at the cost of reporting every genuine track a frame or two late

`maxAge` has an off-by-one worth knowing, because the difference between five and six missed frames is the difference between recovering an identity and minting a new one. A track is retired when `timeSinceUpdate` is strictly greater than `maxAge`, and `timeSinceUpdate` is incremented before the test, so the default `maxAge = 5` survives five consecutive unmatched frames and dies on the sixth. `maxAge = 0` retires a track the first time it is missed.

`minHits` introduces a deliberate confirmation delay. A brand-new track predicts, associates and accumulates hits from its first frame, but it is not **reported** until it has been matched `minHits` times: with `minHits = 2` the first `update` that sees a new object returns nothing for it and the second returns it. `count`, by contrast, counts births rather than confirmations, so it rises on the first frame regardless — decide which of the two you mean before showing a figure to a user.

The honest failure is identity switching. Greedy IoU association has no notion of appearance and no notion of which pairing is best **overall**: it takes the highest-overlap pair, locks both sides, and repeats. When two people cross, their boxes overlap each other as much as they overlap their own predictions, and for a frame or two the highest-overlap pair is the wrong one. From then on the ids are swapped, permanently and silently — `count` stays correct while every per-id trail is nonsense. A Hungarian assignment over the full cost matrix would pick the better global pairing more often; it would not fix the case where the boxes genuinely coincide, and nothing that looks only at boxes can. If it matters **which** person went into which room, you need appearance: a `Tracker` per object, embeddings from Chapter 25, or a camera angle where people do not overlap.

> **WARNING**
> A retired track that reappears is a new object with a new id, and `count` goes up. A tracker pointed at a doorway where people pause out of the detector's sight will over-count, and no tuning of `maxAge` removes the problem — it only moves the threshold at which it starts.

Counting across a line

The commonest thing built on a tracker is a counter: how many crossed this line, and in which direction. It is also where a first attempt reliably produces a comically wrong number. The naive version tests proximity:

```
// Wrong: counts once per frame spent near the line, not once per crossing.
if math.abs(centreY - lineY) < 8 then crossings += 1
```

Somebody walking briskly gives you four. Somebody who stops on the line to check their phone gives you sixty. A smaller distance is not the fix — it makes it possible to step over the line between two frames and be missed entirely. The fix is to ask which **side** of the line each identity is on and count only transitions.

That still leaves an object hovering on the boundary flipping sides on detector jitter alone, so the side test gets a dead band: a strip `band` pixels either side of the line in which the counter forms no opinion at all. An object must leave the band on one side and arrive outside it on the other before anything is counted. This is hysteresis, the trick a thermostat uses to stop itself chattering, and it is the only interesting line in the class.

Example 30-3. A line-crossing counter with hysteresis, keyed by track id.

```
/** Counts tracks crossing the horizontal line at `y`, once each, per direction.
*/
final class LineCounter(y: Int, band: Int = 8):

  // The side each id was last confidently on: -1 above the band, +1 below it.
  private val side = scala.collection.mutable.Map.empty[Int, Int]
  private var down = 0
  private var up = 0

  def observe(tracks: Seq[ObjectTrack]): Unit =
    tracks.foreach: t =>
      val centreY = t.box.y + t.box.height / 2.0
      val now =
        if centreY < y - band then -1
        else if centreY > y + band then 1
        else 0 // inside the band: no opinion, and no update
      if now != 0 then
        side.get(t.id) match
          case Some(before) if before != now => if now > 0 then down += 1 else up
+= 1
          case _ => ()
        side(t.id) = now

  def counts: (Int, Int) = (down, up)
```

`observe` is called with whatever `update` returned, so it sees confirmed tracks only and inherits `minHits`' protection against one-frame phantoms. The band should be wider than the detector's jitter and narrower than the distance an object covers between frames: eight pixels is a fair starting point on a 640×480 corridor camera, and anything busier wants it measured rather than guessed — log `centreY` per id for a minute and look at the spread while an object is stationary.

> **NOTE**
> `side` grows by one entry per distinct id, and ids are never reused. That is heap, not native memory, so it will not take the process down the way a leaked filter will — but it is unbounded. Drop ids that have not appeared for a while, or rebuild the counter on whatever boundary your service already has.

Following one object without a detector

`ObjectTracker` needs detections. When you have none — a user has drawn a box around something the system has no model for — the tool is `Tracker`, which is **model-free**: it learns the object's appearance from the box you give it and finds that patch in the next frame. `Tracker.create(kind)` builds one, `init(image, box)` seeds it, and `update(image)` returns `Option[Rect]` per frame. `init` may be called again to re-seed on a fresh box, which is how you recover after a loss and how the "detect occasionally, track in between" pipeline is built.

Table 30-4. The three algorithms in `TrackerKind`.

`TrackerKind`	Character	Reports loss
`Csrt`	the accuracy pick: handles scale change and partial occlusion, slowest of the three	yes
`Kcf`	the speed pick: fast and steady, but its box never changes size	yes
`Mil`	robust to small appearance changes, with no failure detection at all	no

The "reports loss" column is the one that bites. CSRT and KCF return `None` when they have lost the object; MIL always returns a box, so a MIL tracker that has drifted onto the wallpaper reports the wallpaper with the same enthusiasm it reported the object. If you use MIL, add your own sanity check — a box that stops moving, or grows implausibly, or leaves the frame.

The idiom that makes a `Tracker` worth the trouble is **detect occasionally, track in between**: a detector runs only when there is nothing being followed, and the tracker carries the box the rest of the time. It costs one detector call per acquisition instead of one per frame, and it re-seeds itself the moment `update` returns `None`.

Example 30-4. Detect once, track until lost, then detect again.

```
import scala.util.Using
import scalacv.*

OpenCv.load()

Using.resource(Tracker.create(TrackerKind.Csrt)): tracker =>
  val motion = MotionDetector.backgroundSubtraction(minArea = 300)
  var seeded = false
  var n = 0
  try
    Camera.usingFile("clip.mp4"): cam =>
```

```
      cam.foreach(): frame =>
        n += 1
        val box =
          if seeded then tracker.update(frame)        // borrows the frame, never
consumes it
          else
            motion.detect(frame).largest.map: seed => // the biggest moving
region re-seeds
              tracker.init(frame, seed)
              seeded = true
              seed
        box match
          case Some(b) => frame.copy.drawRect(b).write(f"out/track$n%05d.png")
          case None => seeded = false                 // lost: back to the
detector next frame
  finally motion.close()
```

`Motion.largest` is `regions.headOption`, and `regions` is sorted largest first, so the seed is the biggest thing that moved — crude, but it is the whole point of the pattern that the seed only has to be right once. Both `tracker.update` and `motion.detect` **borrow** `frame`; the `.copy` before `drawRect` is what keeps the loop's own image intact for `foreach` to close.

`update` before `init` throws: the class holds a `started` flag and `require`s it, turning a sequencing mistake into a readable Scala exception rather than undefined behaviour on the far side of JNI. `Tracker.create` guards the other end — an OpenCV build without a given algorithm returns `null` from `TrackerCSRT.create()` and friends instead of failing, so `create` tests for it and throws a `CvError.NativeCall` naming the kind, rather than wrapping the `null` in a `Managed` and surfacing it as an opaque "already released" at the first `init`. It throws rather than returning a `Left` because a missing algorithm is a broken build, not a runtime condition to branch on.

> **NATIVE MEMORY**
> All three algorithms carry a learned appearance model — CSRT's is the largest — and `org.opencv.video.Tracker` has no public `release()` either. `Tracker` is `AutoCloseable` and releases through the same `Releasable.nativeHandle` bridge; `Using.resource` is the idiom. A tracker per object in a crowded scene, rebuilt whenever an object is re-acquired, is the shape that leaks fastest.
>
> `ObjectTracker` is `AutoCloseable` at one remove: its `close()` walks every live track and closes that track's filter. Retired tracks were already freed by step 5 of `update`, so a scene where objects come and go stays flat — but the tracks alive when you stop are freed only by `close()`. All three types are stateful and none is safe to share across threads, since `update` mutates the live-track buffer. Keep the detect-and-track loop on one thread, as Chapter 36 will insist for every detector in this part.

Detector, tracker, overlay

Put the pieces together and the corridor counter is one expression per frame: detect, track, count, draw. `drawTracks` is the one-call overlay — a box and a `#`-prefixed id label per track, in `Scalar.Green` unless you say otherwise — and like every drawing verb in Chapter 14 it consumes the image it is called on, so the frame is copied first: `Camera.foreach` owns each frame and closes it when the block returns. `LineCounter` is the class from the previous listing, unchanged.

Example 30-5. Motion boxes, stable ids, a crossing count, and trails.

```
import scala.collection.mutable
import scalacv.*

OpenCv.load()

val motion = MotionDetector.backgroundSubtraction(minArea = 300)
val tracker = ObjectTracker.create(iouThreshold = 0.2, maxAge = 10)
val counter = LineCounter(y = 240)
val trails = mutable.Map.empty[Int, mutable.ArrayBuffer[Point]]
var frameNo = 0

try
  Camera.usingFile("corridor.mp4"): cam =>
    cam.foreach(): frame =>
      val boxes = motion.detect(frame).regions        // detect: what moved,
where
      val tracks = tracker.update(boxes)              // track: stitch into
identities
      counter.observe(tracks)                         // count: transitions only

      val annotated = frame.copy                      // the loop owns `frame`
      tracks.foreach: t =>
        val centre = Point(t.box.x + t.box.width / 2.0, t.box.y + t.box.height /
2.0)
        val trail = trails.getOrElseUpdate(t.id, mutable.ArrayBuffer.empty)
        trail += centre
        if trail.size > 30 then trail.remove(0, 1)   // remove(idx, count)
returns Unit
        trail.sliding(2).foreach:
          case Seq(a, b) => annotated.mat.drawLine(a, b, Scalar.Red,
Thickness.Stroke(2))
          case _ => ()
      frameNo += 1
      annotated
        .drawTracks(tracks)
        .drawText(s"in ${counter.counts._1}  out ${counter.counts._2}", Point(10,
30))
        .write(f"out/frame$frameNo%05d.png")
finally
  motion.close()
  tracker.close()
```

```
println(s"${tracker.count} distinct objects passed")
```

Two ownership points, both from Chapter 4′s rules and both easy to get wrong here. `frame` is borrowed by `motion.detect` and by `.copy`, and released by `foreach` when the block returns, which is why the drawing happens on `annotated`. And `annotated.mat.drawLine` is a borrow: the mid-level `Mat` extension mutates in place and returns `Unit`, so the trail is drawn without spending the handle and `drawTracks` is still free to consume it afterwards. The `trails` map is plain heap data — `ObjectTrack`, `Rect` and `Point` are ordinary case classes of `Int`s and `Double`s, with nothing native behind them — but it grows one entry per id for the same reason `LineCounter`'s map does, and wants the same eviction. That is also why the final `println` may read `tracker.count` after `tracker.close()`: the running total is a field on the Scala object, and only the filters were freed.

Everything in that loop is boxes and numbers. The tracker never sees a pixel, which is what makes it swappable: replace `motion.detect(frame).regions` with `frame.faces(detector).map(_.box)` and the same loop counts faces, with the same ids, the same crossings and the same trails.

Where this goes next

Tracking answers "which one is which" in pixel coordinates, and every number in this chapter — the IoU gate, the band width, the trail — is measured in pixels. That is enough to count people through a doorway and not enough to say how far away one of them stood or how fast they were walking, because a pixel spans a different distance at every depth and the lens has been bending the geometry all along. Chapter 31, Camera Calibration, works out the intrinsics that turn pixels into rays: what a chessboard capture session measures, how to undistort a frame, and why every metric claim in the rest of this part depends on getting it right.

CHAPTER 31

Camera Calibration

What your lens actually is, measured rather than assumed.

The marker overlay from Chapter 28 tracked beautifully. It stuck to the tag through rotation, it survived motion blur, it looked — on screen — like a solved problem. Then somebody read the distance off it. The tag was 41 cm from the lens; the pose said 45. Moving the tag to the corner of the frame made it worse, and tilting it away from the camera made it worse again, in a way that was smooth and repeatable and therefore not noise. Nothing was broken. The number was the honest output of a camera model that had been guessed rather than measured.

That guess is `Intrinsics.approx`. It fabricates a camera from two things you already know — the image size and a rough field of view — and two things it assumes: that the optical centre is the centre of the sensor, and that the lens bends nothing. Both assumptions are wrong for every real camera, and the second one is spectacularly wrong for the wide-angle modules that go into webcams, dashcams and phones. A pose solver handed a wrong camera does not fail; it returns the pose that would be correct **if** the camera were the one it was told about. The error comes back looking exactly like a measurement.

The fix is a measurement of your own. Show the camera an object whose geometry you already know — a flat chessboard with squares of a known size — from enough angles that the only self-consistent explanation of what landed on the sensor is one particular focal length, one particular optical centre, and one particular set of distortion coefficients. That is calibration, and it is a physical procedure with a software step at the end, not the other way round. This chapter spends as much time on how to hold the board as on which method to call, because the method is three lines and the holding is where calibrations go wrong.

The running example is a bench rig: a fixed camera on an arm above a workbench, an ArUco tag of known edge length on every part that passes under it, and a requirement to report each part's height in millimetres to a tolerance the shop floor will accept. `Ar.estimatePose` from Chapter 28 takes exactly one input this chapter produces.

The camera as a function

A pinhole camera is a projection. A point sitting at (X, Y, Z) in front of the lens, measured in the camera's own frame with Z running out along the optical axis, lands at the pixel

```
u = fx · X/Z + cx
v = fy · Y/Z + cy
```

Four numbers, and they are the whole ideal camera: `fx`, `fy`, `cx`, `cy`. Written as the matrix everyone calls K, they sit like this — the exact layout `Intrinsics` writes into a 3×3 `CV_64F` Mat internally whenever a native call needs one:

```
    | fx   0  cx |
K = |  0  fy  cy |
    |  0   0   1 |
```

`fx` and `fy` are the focal length expressed in pixels, not millimetres — which trips up everyone arriving from photography, where a lens is 24 mm or 50 mm and that is a property of the glass alone. A focal length in pixels is a property of the glass **and** the sensor behind it: the optical focal length divided by the physical size of one pixel. Bolt the same 6 mm lens onto a sensor with 3 µm pixels and you get `fx = 2000`; put it on 6 µm pixels and you get `fx = 1000`. The pixel form is the useful one because it turns a metric ratio into a pixel coordinate without ever needing the sensor's dimensions. The two differ only when pixels are not square, so a recovered pair that disagrees by more than a percent or two is a sign the fit went somewhere strange.

`cx` and `cy` are the principal point: where the optical axis actually pierces the sensor. It is **near** the image centre and reliably not at it, because the lens assembly was placed by a machine with a tolerance, and that tolerance is applied per unit, not per model. The offset is small enough to be invisible in a picture and large enough to move a pose estimate, which is precisely why guessing it costs you.

Why a longer lens has a bigger `fx`

`Intrinsics.approx` computes the focal length from the horizontal field of view with

```
f = (width / 2) / tan(fov / 2)
```

so at 1280 px wide, a 60° field of view gives `fx ≈ 1109` and a 30° field of view gives `fx ≈ 2389`. Narrower angle, longer lens, larger `fx`: the number grows as the camera sees less. If your intuition says a wide lens should have the big number because it captures more, keep the formula instead — `fx` is how many pixels one unit of angle buys you, and a telephoto buys a great many.

What the lens does to a straight line

The pinhole model has no glass in it. Real glass bends rays by an amount that grows with how far off-axis they are, and the standard correction — the one OpenCV, and therefore `Intrinsics`, uses — is a polynomial in the radius r from the principal point:

- **Radial** terms `k1`, `k2`, `k3` multiply the point's distance from the centre by $1 + k_1 r^2 + k_2 r^4 + k_3 r^6$. A negative `k1` pulls the edges of the frame inward: straight lines near the border bow **outward** away from the centre, which is barrel distortion, and it is what almost every wide webcam has. A positive `k1` does the opposite — pincushion, straight lines sagging **towards** the centre — and shows up more on the long end of a zoom.
- **Tangential** terms `p1`, `p2` account for the lens not being exactly parallel to the sensor. They are small, they are asymmetric, and they are the reason your straightened image is still very slightly skewed if you leave them out.

The visible signature is always the same: a straight edge in the world arrives as a curve. Photograph a door frame at the edge of a wide shot and the jamb bows. That bow is a systematic error of several pixels at the frame border, and every geometric method in the vision layer treats a corner position as ground truth — so a few pixels of bend becomes a few percent of error in a distance.

The `Intrinsics` type

This chapter straddles two artifacts, and it is worth being exact about which. `Intrinsics` and `Image.undistort(intrinsics)` live in the core module — `Intrinsics.scala`, `Image.scala` and `Ops.scala`, all under `core/src/scalacv/` — because `undistort` is the one core operation that needs a camera model, and nothing above it should have to be on the classpath to describe a lens. `ChessboardPattern`, `Calibration`, `Calibration.findCorners`, `Calibration.fromChessboard` and the `undistort(calibration)` extension are all in `vision/src/scalacv/vision/Calibration.scala`, so **producing** a calibration needs `com.worxbend::scalacv-vision:0.4.2` on the classpath beside the core dependency of Chapter 2 — the same artifact Chapters 24 through 30 already asked for. Add `import scalacv.vision.*` alongside `import scalacv.*`; the optional module has its own package.

`Intrinsics` itself is a plain case class: four doubles and a sequence.

Example 31-1. The whole constructor, with an ideal lens and with a real one.

```
final case class Intrinsics(
    fx: Double,
    fy: Double,
    cx: Double,
    cy: Double,
    distortion: Seq[Double] = Seq.empty
)

// An ideal pinhole — no glass, no bend.
val ideal = Intrinsics(1109.0, 1109.0, 640.0, 360.0)

// A measured wide webcam: k1, k2, p1, p2, k3.
val bench = Intrinsics(
  fx = 1043.7, fy = 1041.9, cx = 651.2, cy = 358.4,
```

```
  distortion = Seq(-0.2831, 0.0967, 0.0004, -0.0011, -0.0142)
)
```

`distortion` defaults to empty, which means an ideal lens: the internal `distCoeffs` accessor hands OpenCV an empty `MatOfDouble`, every operation that consumes an `Intrinsics` reads that as "nothing to correct", and `Image.undistort` gives back pixels indistinguishable from a copy.

Two `require` checks run in the constructor. The first is obvious — `fx` and `fy` must be positive, because a camera with a non-positive focal length is not a camera. The second is the interesting one.

The distortion vector has legal lengths

OpenCV's distortion model is extensible: four coefficients or fourteen, each legal count switching on another block of the model. `Intrinsics` publishes the accepted counts as a value you can read:

```
val ValidDistortionSizes: Seq[Int] = Seq(0, 4, 5, 8, 12, 14)
```

Table 31-1. The coefficient counts `Intrinsics` accepts, and what each one adds.

Size	Coefficients	Model
0	–	an ideal lens; `undistort` becomes a copy
4	`k1, k2, p1, p2`	radial to r^4 plus tangential — the usual minimum
5	`+ k3`	radial to r^6; what `calibrateCamera` returns by default
8	`+ k4, k5, k6`	the rational model, for strongly curved wide lenses
12	`+ s1, s2, s3, s4`	thin-prism terms
14	`+ taux, tauy`	tilted-sensor terms

Here is the mistake the check exists for. You read that radial distortion is `k1`, `k2`, `k3`, you have three radial numbers from somewhere, and you write:

```
Intrinsics(1043.7, 1041.9, 651.2, 358.4, Seq(-0.2831, 0.0967, -0.0142))
```

Three coefficients is not a length OpenCV has a meaning for, and the constructor throws an `IllegalArgumentException` naming the legal counts and the layout `k1, k2, p1, p2[, k3[, k4, k5, k6[, s1, s2, s3, s4[, taux, tauy]]]]`. The correct vector is five long, with the tangential pair in the middle: `Seq(-0.2831, 0.0967, 0.0004, -0.0011, -0.0142)`.

Why check at all, rather than let the native call complain? Because it does not complain usefully. Hand `undistort` or `solvePnP` a badly-shaped vector whose length happens to be one OpenCV recognises, and the call runs to completion and gives back an image or a pose

that is quietly, plausibly wrong — the most expensive failure mode there is, a number that looks like an answer. Rejecting at construction moves the error to the line where the bad value was written rather than to a solver three layers away. It is the same argument as the focal-length check above it: refuse what cannot be a camera, at the point the camera is made.

> **WARNING**
> The check catches shapes, not values. A five-element vector with one coefficient mistyped, or with `k3` swapped into a tangential slot, is still five elements long, so the constructor accepts it and the results are wrong in exactly the way described above. The only real defence is to take the vector from a calibration rather than from a spreadsheet, which is what the rest of this chapter is about.

`Intrinsics.approx(imageSize, horizontalFovDegrees = 60.0)` remains what it always was: a centred principal point, square pixels, no distortion, and a focal length from the field-of-view formula in the sidebar. Keep it as the placeholder that makes a pipeline run before the camera has been measured, and treat every metric number that comes out of a pipeline running on it as indicative.

Describing the board

The target is a printed chessboard, and what the detector looks for is the **inner corners** — the points where four squares meet. The outer edge of the printed pattern is not a corner of that kind, so a board of 10×7 squares presents a 9×6 grid of inner corners, and 9×6 is what you pass.

Example 31-2. The bench rig's target: a 10 x 7 board with 25 mm squares.

```
val board = ChessboardPattern(columns = 9, rows = 6, squareSize = 0.025)

board.corners   // 54 — the number of inner corners the detector must find, all
of them
```

`columns` and `rows` must each be at least 2, and `squareSize` must be positive; both are enforced in the constructor. `squareSize` sets the unit of the world and only that: pass metres and the poses downstream come back in metres. It has no effect whatever on `fx`, `fy`, `cx` or `cy`, which are in pixels no matter what you feed the board. The default `1.0` means "board squares", which is fine when the intrinsics are all you want.

Prefer an asymmetric inner grid, 9×6 rather than 8×8. A square grid has a rotational ambiguity — nothing says which corner the detector called the origin from one view to the next — while an odd-by-even grid has exactly one consistent orientation.

Finding the board in a frame

`Calibration.findCorners` locates the whole inner grid in one image and refines every corner to sub-pixel precision with `cornerSubPix`. It returns `Option[Seq[Point]]`, and the `Option` is the point: the detector is all-or-nothing. If one corner is occluded by your thumb, or the board runs off the edge of the frame, you get `None` rather than a partial grid.

```
Calibration.findCorners(frame, board) match
  case Some(corners) => // exactly board.corners of them, in row-major order
  case None          => // the whole board is not in this frame
```

Two decisions inside `findCorners` set what the physical setup has to provide. The search runs `findChessboardCorners` with `CALIB_CB_ADAPTIVE_THRESH | CALIB_CB_NORMALIZE_IMAGE`: the first thresholds locally, so a board lit brighter on one side than the other still binarises into squares, and the second normalises the image gamma before the search, so the exposure does not have to be nailed. Then `cornerSubPix` refines each corner inside an 11×11 window, stopping at thirty iterations or a movement below `1e-3` px. That refinement is where sub-pixel accuracy comes from, and it is also why motion blur hurts so specifically: a smeared corner has no sharp saddle for the refinement to walk towards, so it settles somewhere confidently wrong rather than reporting nothing.

That `None` is an ordinary query result, not a failure, and the return type says so — the same choice the detectors in Chapter 27 and the trackers in Chapter 30 make. It is the solve step, later, where "too few boards" genuinely is a failure and an `Either` appears.

`findCorners` borrows the image in the sense Chapter 4 gave the word: the `Image` you pass is still alive and still yours to close. What comes back is `Seq[Point]`, immutable Scala data holding two doubles apiece, so the corners stay valid long after the frame that produced them is released. That is what makes the capture loop below possible.

Example 31-3. A capture tool that banks only the frames where the whole board is visible.

```
import scala.collection.mutable.ArrayBuffer

val keep = ArrayBuffer.empty[Image]

Camera.using(0) { cam =>
  cam.foreach() { frame =>
    Calibration.findCorners(frame, board) match
      case Some(_) if keep.size < 20 => keep += frame.copy
      case _                         => ()
  }
}
```

> **NATIVE MEMORY**
> `cam.foreach` closes the frame the moment the block returns — the contract from Chapter 20, and what keeps a camera loop from growing without bound. `keep += frame` would therefore

> bank twenty `Image` handles whose Mats have all been released, and the first thing `fromChessboard` touched would throw a use-after-release from `Managed`. `frame.copy` allocates a fresh Mat that outlives the loop. It is also twenty full-resolution frames of native memory that nothing will free for you: close every one when the calibration is done.

Shooting a set worth solving

This is the part that decides whether the numbers are any good, and no care in the code substitutes for it. Calibration recovers the camera from the **disagreements** between views: give it views that disagree in interesting ways and the solution is pinned down, give it twenty near-identical ones and there is nothing to work with however many of them there are.

The pathological case is a set of head-on shots. A board held flat and square to the sensor a metre away produces almost exactly the image of the same board at half the size from two metres — the geometry cannot separate "the board is far away" from "the lens is long", so `fx` and distance trade off against each other and the solver picks somewhere along that valley. Tilting breaks the tie, because a tilted plane's foreshortening depends on focal length in a way pure scale does not. This is why a beginner's calibration set, careful and neat and all square to the camera, is usually the worst one.

Table 31-2. The shot list for a calibration set.

What	Why
8–15 good views, more for a wide lens	three is the solver's floor, not a target; past a point you are adding views, not information
Tilt in both axes, not one	a head-on-only set is degenerate: focal length and distance trade off
Board in every part of the frame	distortion is estimated from off-axis corners, so the corners of the frame must be visited
Board large in frame, fully inside it	a small board localises noisily; a clipped board is not detected at all
Rigid, flat mount	a board that flexes is not the geometry you told the solver about
Even, diffuse light	glare on a glossy print erases corners; the detector then finds nothing at all
Short exposure, camera and board still	motion blur moves corners systematically, not randomly
Autofocus off, zoom fixed	the intrinsics belong to one focus and one zoom setting

Three of those earn extra words:

The board must be rigid. Glue the print to foam board, aluminium or glass. Paper on a clipboard bows by a couple of millimetres when you hold it, and the solver has no representation for a curved board — it absorbs the bow into the distortion coefficients, which is exactly where you did not want it.

Measure the squares you actually printed. Printers scale: a 25 mm square in the PDF arrives at 24.6 mm because a driver fitted the page to the paper. Put a caliper across ten squares and divide by ten. This affects nothing about `fx` or `cx`, but it scales every downstream distance by the same factor, silently.

Calibrate in the state the rig will run in. Focus, zoom and any knock to the mount change the intrinsics, and so does the capture resolution — which is why `Calibration` records the `imageSize` the numbers were measured at.

Solving

With the views in hand, the solve is one call.

Example 31-4. Recovering the bench rig's camera from a folder of captures.

```
val views: Seq[Image] = (1 to 22).flatMap(i => Image.read(s"calib/
$i.png").toOption)

val result = Calibration.fromChessboard(views, board, minViews = 12)

views.foreach(_.close())

result match
  case Right(calib) =>
    val i = calib.intrinsics
    println(f"fx=${i.fx}%.1f fy=${i.fy}%.1f  centre=(${i.cx}%.1f, ${i.cy}%.1f)")
    println(f"RMS reprojection error: ${calib.reprojectionError}%.3f px  @
${calib.imageSize}")
  case Left(err) =>
    System.err.println(err.getMessage)
```

`fromChessboard` runs the corner search over every view and **silently drops** the ones where the whole board is not visible, so handing it an entire capture folder — blurred frames, half-boards and all — is the intended use rather than sloppiness. `minViews` is your floor on how many must survive. It defaults to 3, the practical minimum for the solver to be well posed and far below what you want in production; set it near the number of good views you believe you captured, so a folder where two thirds of the frames failed the detector reports a failure instead of quietly calibrating from seven views.

Table 31-3. Choosing `minViews`.

`minViews`	When	What it buys
`3` (the default)	a quick check in controlled light	enough to watch the call work; the numbers wobble
8–15	a production calibration	where the intrinsics start being worth trusting

20 or more	wide or fisheye lenses, tight tolerance	angles enough to pin the distortion terms down

A `minViews` below one is a programmer error rather than a data one, and `require` throws for it — the split Chapter 6 draws, applied to one argument: bad arguments throw, bad **data** comes back as a `Left`.

The image size is taken from the first view, and every view should share it. The result is a `Calibration`: the recovered `intrinsics`, that `imageSize`, and the `reprojectionError`.

Reading the reprojection error

The reprojection error is the RMS distance, over every corner of every surviving view, between where a corner actually sat in the image and where the recovered model says it should have sat. It is reported alongside the intrinsics rather than buried, because a calibration you have not judged is a calibration you should not ship.

Table 31-4. How to read the RMS reprojection error.

RMS	Verdict	What to look at
< 0.5 px	excellent	–
0.5–1 px	good — ship it	–
1–2 px	marginal	some motion blur, or too few distinct angles
> 2 px	poor — recapture	a flexing board, bad focus, or a board that clipped the frame

The number has one important limitation: it measures **fit**, not **accuracy**. A model with enough free parameters can fit four near-identical views to a quarter of a pixel and still be wrong about the focal length by five percent, because those four views never asked the question that would have caught it. A low RMS from a rich, varied set is strong evidence; from a thin set it is almost none. For a second opinion, calibrate twice from disjoint halves of your captures and compare the two `fx` values — agreement to within a percent means something RMS alone does not.

The library's own test does the honest version of that check. It renders seven views of a 9×6 board by warping one flat rendering through homographies built from a known camera — `fx = fy = 800`, centre `(320, 240)`, a 640×480 frame — so the ground truth that produced the pixels is available to compare against. It then asserts three separate things: that the RMS comes back under one pixel, that each recovered focal length is within 15% of 800, and that the recovered principal point is within 40 px of `(320, 240)`. Those last two tolerances are far wider than a good real calibration achieves, and deliberately so — they are a regression guard against the solve going wrong, not a claim about precision. Note which number is tight and which are slack: the fit is excellent and the accuracy is merely bounded, on synthetic views that are **exactly** the model. That is the previous paragraph, in the test suite.

When it fails

`fromChessboard` returns `Either[CvError, Calibration]`, and there are two distinct ways to land on the left.

`CvError.CalibrationFailed` fires when the **input** was not enough: an empty `views` sequence gives `"no views were provided"`, and a run where fewer than `minViews` frames showed the whole grid gives a message naming the shortfall — views passed, views that contained the board, and the grid it was looking for. Both are facts about what the capture happened to catch rather than programmer errors, which is why they are values.

The second way is a throw from OpenCV's own `calibrateCamera`, and it does **not** arrive as `CalibrationFailed`. It is caught and wrapped as `CvError.NativeCall("calibrateCamera", cause)`, per the error model in Chapter 6. So this match is a trap:

```
Calibration.fromChessboard(views, board) match
  case Right(calib)                    => use(calib)
  case Left(CvError.CalibrationFailed(m)) => retryCapture(m)
  // MatchError when the native solver is the thing that failed
```

Match the sum type, not one member of it:

```
Calibration.fromChessboard(views, board) match
  case Right(calib)                     => use(calib)
  case Left(CvError.CalibrationFailed(m)) => retryCapture(m)
  case Left(err)                        => log.error(err.getMessage, err)
```

Note also what is **not** a failure here: a view that makes the corner detector itself throw is folded into "no board in this frame" and dropped, so one pathological frame in a folder of twenty cannot abort the whole calibration.

Straightening the frame

With a calibration, `undistort` maps the lens bend back out. There are two spellings — pass the `Intrinsics` directly, or pass the whole `Calibration` and let the extension method in the vision layer unwrap it for you.

```
Image.reading("bench-raw.png")(_.undistort(calib).write("bench-straight.png"))
```

Like every transform in Chapter 4, `undistort` **consumes** its receiver and returns a new `Image`. If you need the original as well — and for a calibration rig you often do, because the detector should run on one and the display on the other — take a `.copy` first.

> NATIVE MEMORY

`Intrinsics` is pure Scala data; the native call underneath is not. `cameraMatrix` and `distCoeffs` each allocate a fresh Mat on every access, so `undistorted` wraps both in a `Managed.scope` — freed whether the call returns or throws — and deliberately leaves the destination Mat **unowned**, because that one is the value being handed back. The leak suite pins it down: three hundred `undistort` calls in a row, each of them allocating its own camera matrix and distortion Mat out of the same `Intrinsics`, with an assertion that native growth stays bounded. It is the ownership rule from Chapter 5 in miniature: what the operation borrows goes in the scope, what it produces does not.

All the pixels, or only the valid ones

Undistortion is a resample: for each output pixel it asks where that ray came from in the input. Near the frame border under barrel distortion the answer is sometimes "from outside the sensor", and those output pixels have no source. What to do about them is a choice, spelled in OpenCV as the `alpha` parameter of `getOptimalNewCameraMatrix`: at one extreme you keep the whole rectified field of view and accept curved black wedges at the edges where nothing mapped in; at the other you crop to the largest rectangle in which every pixel is real, which costs field of view — most of it at the corners, where a wide lens bends hardest — and hands you a **different** camera matrix that anything measuring on the result must then use.

`Image.undistort` exposes no such knob. `Ops.undistorted` calls the four-argument `Calib3d.undistort(src, dst, cameraMatrix, distCoeffs)`, which takes the calibrated matrix as its own new-camera matrix, so the output keeps the input's size and the same `fx`, `fy`, `cx`, `cy`: a measurement made on the undistorted image goes straight back through the intrinsics you already hold, and the invalid corners come back black. That is the right default for both things undistortion is usually for — straightening a frame before measuring on it, and straightening one for a person to look at — and it is one fewer matrix to keep in step with the other. If you want the cropped variant, build the optimal new camera matrix yourself and pass it as `Calib3d.undistort`'s fifth argument; Appendix A shows how to reach the raw API without leaving the library's ownership model, which here means wrapping that matrix in a `Managed` so it is freed with everything else.

TIP

If the pipeline downstream is `Ar.estimatePose`, `HeadPose.estimate` or `Localizer.locate`, do **not** undistort first. Those solvers take the distortion coefficients themselves and account for the bend analytically — more accurate and far cheaper than resampling every pixel of every frame. Undistort when a person, or something that assumes straight lines, has to look at the frame; otherwise hand the `Intrinsics` along and leave the pixels alone.

Where the numbers live afterwards

An `Intrinsics` is not a property of your program. It is a property of one camera body, with one lens, at one zoom and one focus distance, sampled at one resolution. Two nominally identical webcams out of the same box do not share a principal point, because the tolerance

that placed each lens assembly was spent per unit. Store the calibration with the device, not with the build.

`Intrinsics` and `Calibration` are ordinary case classes of `Double` and `Seq[Double]`, so any codec you already use serialises them without ceremony; there is no bespoke format in the library and none is needed. What matters is what you store **alongside** them: the device identifier, the `imageSize`, the date, and the reprojection error you accepted. That last one earns its keep the day a rig starts producing suspect distances and the first question is whether the calibration was ever good.

Resolution is the trap. Calibrate at 1280×720, run the camera at 640×360, and every one of `fx`, `fy`, `cx`, `cy` is twice what it should be with nothing to tell you: the `Calibration` records the size it was measured at, but nothing enforces it at the point of use. Recalibrating at the running resolution is the honest fix. Scaling is the pragmatic one, and it is exact for a clean downsample:

Example 31-5. Rescaling intrinsics for a different capture resolution. Distortion coefficients carry over unchanged.

```
def scaledTo(i: Intrinsics, from: Size, to: Size): Intrinsics =
  val sx = to.width / from.width
  val sy = to.height / from.height
  i.copy(fx = i.fx * sx, fy = i.fy * sy, cx = i.cx * sx, cy = i.cy * sy)
```

The distortion coefficients need no scaling: they act on normalised coordinates — the image **after** `K` has been divided out — so they are dimensionless and survive a resize intact. Cropping is a different operation: it moves the principal point without changing the focal length, so a camera that switches between full-sensor and cropped modes has two calibrations, not one scaled one.

> **CAUTION**
> Do not carry a calibration across a firmware update, a lens change, a knock that moved the mount, or a switch of the camera's own digital zoom or "field of view" setting. All of those change the optics behind the numbers, and a stale calibration is worse than an honest `Intrinsics.approx`: the guess is visibly a guess, while the stale file looks measured.

What the bench rig gets

With `calib` in hand the rig's pipeline changes in exactly one place. Where Chapter 28 wrote `Ar.estimatePose(marker, markerLength = 0.05, Intrinsics.approx(img.size))` and got a pose that tracked, it now writes `Ar.estimatePose(marker, markerLength = 0.05, calib.intrinsics)` and gets a pose that measures. The same substitution turns `HeadPose.estimate` from Chapter 29 into real head distances.

Chapter 32, *Visual Navigation*, takes the calibrated camera and asks where it is: optical flow and ORB features, visual odometry frame to frame, absolute localisation against a map, and

stereo depth. Every one of them takes an `Intrinsics`, and every one is worth exactly as much as the calibration you hand it.

CHAPTER 32

Visual Navigation

Turning pixels into geometry: flow, features, depth, odometry and a map.

A camera on a moving thing — a drone, a rover, a phone held out at arm's length — is a sensor that measures nothing. It reports brightness. Every quantity a navigation stack wants (how far away that wall is, how fast I am going, whether I have been in this corridor before) must be inferred from how brightness moved between one frame and the next, and each inference is a different piece of geometry with its own failure mode.

The chain has a shape worth holding in your head before you meet the types. Distinctive points are found and followed. Their motion, run through a camera model, becomes the camera's motion. Two cameras a known distance apart turn the same points into distance. Chained motion becomes a trajectory, and the trajectory drifts, because every step's error is added to every step after it. A map fixes the drift, but only if you can tell you are looking at a piece of the map you already have — the recognition problem again, one level up.

That chain is the **front end** of a visual-SLAM system, and it is what this chapter covers, because it is what OpenCV provides and therefore what scalacv wraps. Everything on the front-end side is one typed call; everything past it is a decision to take a dependency.

The navigation sources under `vision/src/scalacv/vision/` carry it — `OpticalFlow`, `Features`, `VisualOdometry`, `Odometry`, `Depth`, `Navigator`, `Localizer`, `OccupancyGrid` and `LoopDetector` — so every type in this chapter needs `com.worxbend::scalacv-vision` on the classpath beside `scalacv`, the same module as the detectors of Chapters 24 to 29 and the trackers of Chapter 30. The worked example at the end also plots, so it adds `scalacv-graphs` for `Chart` and `Color`. Add `import scalacv.vision.*` and `import scalacv.graphs.*` alongside core's `import scalacv.*`.

Most of those nine own native memory the Java bindings will not free: an ORB detector, a brute-force matcher, a stereo matcher, a descriptor `Mat` per keyframe, a retained previous frame. Of the 188 `org.opencv.*` types that hold a native pointer, exactly three expose a public `release()` — `Mat`, `VideoCapture` and `VideoWriter` — and `ORB`, `BFMatcher` and `StereoSGBM` are not among them. That is why `Descriptors`, `Odometry` and `LoopDetector` are all `AutoCloseable`, and why the native-memory callouts below come thick and fast.

The line between front end and back end

Half the mistakes in this area are scope mistakes — reaching for a pose graph when a reflex would do, or expecting a drift-free trajectory out of dead-reckoning — so start with the territory.

Table 32-1. What the front end answers, and what it hands upward.

Task	In scalacv	Needs something beyond OpenCV
Tracking	`OpticalFlow` — follow points frame to frame	—
Recognition	`Features` — ORB keypoints and matching	—
Motion	`VisualOdometry.estimate` per pair; `Odometry` as the running loop	absolute scale, drift correction
Localization	`Localizer.locate` — absolute pose from a map	the map to localize against
Depth	`StereoDepth.disparity`, `Obstacles.fromDisparity`	—
Steering	`Navigator.steer` — reactive avoidance	a goal and a planner
Mapping	`OccupancyGrid`, `LoopDetector`	—
Full SLAM	all of the above, as the front end	pose-graph optimisation, bundle adjustment

Optical flow: following what you already found

The cheapest correspondence you can buy is the one you do not have to search for. If a point was at `(412, 208)` in the last frame and the camera has moved a little, the same point is **near** `(412, 208)` in this frame, and a local optimisation over a small window will find it in a few iterations. That is Lucas–Kanade, run over an image pyramid so that a large motion at full resolution is a small motion at a quarter of it.

`OpticalFlow` is two calls. `goodFeatures` picks the corners worth following — Shi–Tomasi, which scores a pixel by how badly a small window around it matches itself when nudged in any direction, so a corner scores well and a stretch of blank wall scores nothing. `track` follows a given set of points from one image into another.

Example 32-1. The two halves of sparse flow, and their defaults.

```
def goodFeatures(
    image: Image,
    maxPoints: Int = 200,
    quality: Double = 0.01,
    minDistance: Double = 7.0
): Seq[Point]

def track(previous: Image, current: Image, points: Seq[Point]): Seq[Track]
def track(previous: Image, current: Image): Seq[Track]

final case class Track(from: Point, to: Point, found: Boolean):
```

```
  def displacement: Point
  def distance: Double
```

`quality` is relative, not absolute: a corner is kept if its Shi–Tomasi score is at least that fraction of the best corner's, so `0.01` means "anything within two orders of magnitude of the strongest corner in this image". `minDistance` enforces a pixel spacing between kept corners, which is what stops the detector returning thirty points along one strong edge. These are the knobs you turn when the tracker gives back either too few points to estimate anything or two hundred clustered on one lamp post. `maxPoints` and `quality` must be positive; both are `require`d.

The two-argument `track` seeds `goodFeatures(previous)` for you. Use it for a first cut, and the three-argument form the moment you want to reuse last frame's survivors instead of re-detecting — which is what a real loop does.

The mistake: trusting a lost point

`track` returns one `Track` per input point, in the order the points went in, and a lost point still occupies its slot — its `to` being whatever the optimiser held when it gave up. So this computes a number, and the number is wrong:

```
val tracks = OpticalFlow.track(previous, current)
val meanX = tracks.map(_.displacement.x).sum / tracks.size   // wrong: includes
lost points
```

The fix is one filter, and it is why `found` is a field rather than expressed by omitting the track:

```
val kept = OpticalFlow.track(previous, current).filter(_.found)
```

Positional alignment with the input is worth more than a compacted result, because it is what lets you zip a forward pass against a backward one.

The forward--backward check

`found` is the tracker's own opinion, and the tracker is an optimiser: it converges happily onto the wrong pixel when a point crosses an occlusion boundary or lands on a repeating texture. The standard defence costs one more `track` call — follow the survivors back and keep only the points that land where they started.

Example 32-2. A forward--backward consistency filter, built from the public API.

```
def stableTracks(previous: Image, current: Image, maxDrift: Double = 1.0):
Seq[Track] =
  val seeds     = OpticalFlow.goodFeatures(previous)
  val forward   = OpticalFlow.track(previous, current, seeds).filter(_.found)
  val backward  = OpticalFlow.track(current, previous, forward.map(_.to))
```

```
  forward.zip(backward).collect:
    case (f, b) if b.found && b.to.distanceTo(f.from) <= maxDrift => f
```

The `zip` is only sound because of the ordering guarantee: `backward(i)` is the return trip of `forward(i)`. A point that survives both passes and comes home within a pixel was matched consistently in both directions, which an occlusion-induced slide almost never is. It costs a second Lucas–Kanade pass over the survivors, and it is worth it, because a handful of consistently wrong correspondences will bend an essential-matrix estimate in a way RANSAC does not always catch.

> **NOTE**
> Sparse flow is what scalacv wraps. A **dense** field — a displacement for every pixel, as `calcOpticalFlowFarneback` produces — is a two-channel float `Mat` rather than a `Seq` of typed values, and costs far more per frame. Nothing hides it: as Appendix A spells out, the full typed `org.opencv.*` surface is one step away, and `Managed` will own the result mat as it owns everything else.

Features: recognising a place you have left

Flow assumes the camera barely moved. Recognition assumes nothing — the two views may be a minute and a corridor apart, rotated, at different scales. That needs a descriptor: a compact signature of the neighbourhood around a keypoint, robust enough to be matched against a signature computed elsewhere, elsewhen. `Features` wraps ORB and returns its output as an owned value.

Example 32-3. Detect, then match. Both results are plain data; the container is not.

```
final case class FeatureMatch(queryIndex: Int, trainIndex: Int, distance: Float)

final class Descriptors extends AutoCloseable:
  val points: Seq[Point]
  def size: Int
  def isEmpty: Boolean
  def close(): Unit

object Features:
  def detect(image: Image, maxFeatures: Int = 500): Descriptors
  def matches(a: Descriptors, b: Descriptors, maxDistance: Float = 64f):
Seq[FeatureMatch]
```

`maxFeatures` must be positive, and `detect` **borrows** the image — it takes its own greyscale copy internally and never consumes or closes what you passed — while **returning** an owned `Descriptors`. There is no public constructor: `Features.detect` is the only way to make one, which is what keeps the `Mat` inside it from ever arriving unowned. That is Chapter 4's

borrow/consume split applied to a second resource type, and it means a two-image snippet has four native handles alive, not two.

> **NATIVE MEMORY**
> `Descriptors` holds the descriptor `Mat` — 32 bytes per keypoint, so 16 KB for a default 500-feature frame, which sounds harmless until a detector loop runs for an hour. Close it, or take it into a `Using` block. The ORB detector and the `BFMatcher` are shorter-lived but no less native: neither has a public `release()`, so `Features` declares a `Releasable` for each via `Releasable.nativeHandle` and frees them through the binding's private `delete(long)` (Chapter 5). `ORB.create` in a per-frame loop with nothing freeing it is a leak measured in gigabytes.

Matching, and what is thrown away

`matches` runs a brute-force matcher with `NORM_HAMMING` and cross-check enabled, keeps the pairs within `maxDistance`, and sorts them best-first. In each `FeatureMatch`, `queryIndex` indexes `a.points` and `trainIndex` indexes `b.points`, so the two are not interchangeable and swapping the arguments swaps the fields. One metric and two filters, each worth understanding.

The **Hamming distance** is a popcount of an XOR. ORB's descriptor is a bit string, so comparing two of them is a couple of machine instructions rather than a 128-dimensional float subtraction — most of why ORB is affordable at frame rate.

Cross-check is the outlier filter, and it ships on: a pair survives only if `a`'s best match in `b` is also `b`'s best match in `a`. It rejects the classic failure, twelve keypoints in this frame all claiming the same distinctive keypoint in the old one, for the price of matching both directions. It is a stricter, cheaper cousin of Lowe's ratio test, and because it is built into the matcher rather than exposed as a knob, no caller gets a one-directional result by accident.

`maxDistance` is the absolute cutoff, `64f` by default — a quarter of ORB's 256 descriptor bits differing. Raise it for recall on hard viewpoint changes, and take the junk with it; lower it and `LoopDetector` stops firing on genuine revisits. It is the number that repays tuning against your own footage. The result is sorted, so `matches.take(30)` is the thirty most confident correspondences and needs no further ceremony.

Why ORB, and not something with a better reputation

ORB is FAST corners with an orientation, described by a rotation-steered BRIEF bit string, published in 2011 as a freely usable alternative to the then-patented SIFT and SURF. The licensing weather has since changed; the engineering argument never depended on it.

A 256-bit binary descriptor is 32 bytes. A SIFT descriptor is 128 floats: 512 bytes, sixteen times the memory per keypoint, matched by Euclidean distance rather than a popcount. On a keyframe store that grows for the length of a drive, that ratio is the difference between a bounded process and a slow leak. You give up some robustness to scale and severe viewpoint

change; for loop closure over a trajectory that revisits places from a similar direction — what a ground vehicle does — it is a good trade, and the one scalacv makes for you.

Table 32-2. Flow or features? They answer different questions.

	`OpticalFlow`	`Features`
Assumes	small motion between consecutive frames	nothing — any two views
Cost	one pyramidal LK pass over N points	detect, describe, then N×M descriptor comparisons
Output	`Seq[Track]`, aligned with the seeds	`Descriptors` (owned) and `Seq[FeatureMatch]`
Use for	frame-to-frame tracking, odometry	recognition: relocalization, loop closure
Breaks on	large jumps, occlusion, motion blur	textureless scenes, repeated architecture

Motion between two frames

Given matched correspondences and a calibrated camera, the geometry is determined. Two views of the same rigid scene constrain each other through the essential matrix, and decomposing it recovers the rotation and translation that separate the two camera positions. `VisualOdometry.estimate` is that, in one call.

Example 32-4. From correspondences to camera motion.

```
final case class CameraMotion(rotation: Seq[Seq[Double]], translation:
Seq[Double], inliers: Int)

def estimate(from: Seq[Point], to: Seq[Point], intrinsics: Intrinsics):
Option[CameraMotion]
```

`from` and `to` must be the same length and in matched order — a `require`, because a length mismatch is a bug in your pairing code, not a runtime condition. Fewer than five correspondences returns `None`; five is the theoretical minimum for the essential matrix, and the internal call uses `Calib3d.RANSAC` at 0.999 confidence with a one-pixel threshold, so it fits a consensus rather than solving exactly. `inliers` is how many correspondences that consensus kept, and it is the honest quality signal in the result: a `CameraMotion` with eleven inliers out of two hundred is arithmetic, not measurement.

`rotation` is 3 rows by 3 columns and `translation` has length 3, both copied out of their native mats before the call returns. Nothing native survives it except the `Image`s you already owned: a `Managed.scope` registers every intermediate mat as it is built, so a throw from `findEssentialMat` — which does throw on genuinely degenerate input — frees the ones already made. Note the consequence: `estimate` returns `Option`, but it is not total. A native

failure comes back as a thrown `CvError.NativeCall`, not as `None`. `None` means **too few points, or no pose recoverable**; an exception means OpenCV refused the input.

> **WARNING**
> `translation` is a **unit direction**. It has to be: a single camera cannot distinguish a small nearby motion from a large distant one, because both produce the same image displacement. Multiplying it by metres requires scale from somewhere else — wheel odometry, an IMU, a stereo baseline, or a known object size. Any code that adds these vectors up and calls the sum a position is measuring shape, not distance, and should say so in its own type or its own comment.

The odometry loop

`Odometry` is the running form: `goodFeatures`, `track`, `filter(_.found)` and `estimate` composed, with the previous frame retained across calls.

Example 32-5. The whole surface of the pipeline.

```
final class Odometry extends AutoCloseable:
  def update(frame: Image): Option[CameraMotion]
  def framesProcessed: Int
  def close(): Unit

object Odometry:
  def monocular(intrinsics: Intrinsics): Odometry
```

`update` returns `None` on the first frame — that frame becomes the reference — and whenever fewer than eight tracked points survive the `found` filter. Note the eight: the estimator itself accepts five, and the pipeline is deliberately stricter than the thing it calls, because a five-point fit from a nearly-lost tracker is a number that looks like an answer.

> **NATIVE MEMORY**
> `Odometry` keeps a `copy` of the previous frame — a full `Mat`, roughly 900 KB for a 640×480 colour frame, held for the pipeline's whole lifetime. That single retained handle is why the class is `AutoCloseable`, and why it is not thread-safe: two `update` calls race on one native buffer. The internals are worth reading for the pattern: the new baseline and its features are built **before** either field is mutated, and the old frame is closed only once its replacement exists.

Depth from a stereo pair

Two cameras a fixed distance apart see the same point at different horizontal positions, and that shift is inversely proportional to distance. Write the shift as `d` pixels, the focal length as `f` pixels and the baseline — the separation of the two cameras — as `B` metres, and depth is `Z = f * B / d`. Everything about stereo follows from that fraction: doubling the baseline

doubles the precision at a given range, and a shift of zero means infinitely far away, which is why the far field is the noisiest part of any disparity map.

`StereoDepth.disparity` computes the shift for every pixel by semi-global block matching: a `blockSize` window from the left image is matched against candidate positions in the right over a search range of `numDisparities` pixels, and — the "semi-global" part — solutions in which neighbouring pixels disagree are penalised, aggregated along several scanline directions at once. Plain block matching decides each pixel alone; it is faster and far noisier on low-texture surfaces. SGBM's smoothness term is what makes a blank wall come out as a wall rather than confetti.

Example 32-6. A disparity map, with the two knobs that matter.

```
def disparity(left: Image, right: Image, numDisparities: Int = 64, blockSize: Int
= 9): Image
```

Three preconditions run before anything native does: `numDisparities` positive and a multiple of 16 (OpenCV's requirement, not scalacv's invention), `blockSize` odd and at least 3, and the pair agreeing in size. `numDisparities` is the range of depths you are willing to look for, so raising it finds closer objects at a linear cost in time; `blockSize` trades detail for stability.

> **NOTE**
> The result is an 8-bit single-channel `Image`, normalised so that **brighter is nearer**. That normalisation is what makes it viewable and what makes `nearness` a clean `0 – 1` quantity, and it is also what discards the metric depth: the raw `CV_16S` fixed-point disparity that SGBM produces is consumed inside the call. If you need metres rather than an ordering, compute `Z = f * B / d` from the raw disparity via the low-level surface — and remember you need a real `B`, which is a calibration output (Chapter 31), not a tape measurement.

The pair must already be **rectified**: row-aligned, so a point in the left image lies on the same scanline in the right one. That is a one-time stereo-calibration step (`stereoRectify`), done off the hot path, and it is deliberately not wrapped here.

> **NATIVE MEMORY**
> `StereoDepth.disparity` creates a `StereoSGBM` per call, and it has no public `release()` either — so it too is freed through `Releasable.nativeHandle`, inside a `use` block that runs on the exception path as well. Two greyscale conversions and a raw `CV_16S` mat are allocated and released inside the call. What comes back is one owned `Image`, yours to close.

From a depth map to obstacles, and from obstacles to a decision

A disparity map is still an image. Two small types turn it into a decision.

Example 32-7. Obstacles, and the steering reflex on top of them.

```
final case class Obstacle(region: Rect, nearness: Double)

def fromDisparity(disparity: Image, minNearness: Double = 0.5, minArea: Int =
200): Seq[Obstacle]

enum Steering:
  case Straight, Left, Right, Stop

final case class Guidance(
    steering: Steering,
    clearanceAhead: Double,
    leftNearness: Double,
    centreNearness: Double,
    rightNearness: Double
)

def steer(disparity: Image, dangerNearness: Double = 0.55, blockedNearness:
Double = 0.8): Guidance
```

`Obstacles.fromDisparity` is the pipeline from Chapters 9, 11 and 12 pointed at a depth map instead of a colour one: threshold at `minNearness × 255`, close the mask with a radius-2 morphological close to knit up speckle, find contours, take bounding rectangles, drop anything under `minArea` pixels, and attach each region's mean nearness from a submat. The list comes back nearest first, which is the order you want for an avoidance decision.

`Navigator.steer` is deliberately dumber and therefore faster: split the map into vertical thirds, take each one's mean nearness, and choose. Centre clearer than `dangerNearness`, go `Straight`. Both sides past `blockedNearness`, `Stop`. Otherwise turn toward the clearer side. `clearanceAhead` is `1.0 - centreNearness`, and the three raw per-third numbers come back with it so you can log why a decision was taken rather than reconstruct it later.

This is a Braitenberg reflex, not a planner: no memory, no goal, no map. It keeps you off the wall while something slower thinks. Four `require`s guard the way in: both thresholds in `[0, 1]`, the map non-empty, and — the interesting one — at least three pixels wide. Below three columns the thirds collapse, and the right band gets a zero or negative width, which is a raw `CvException` out of `submat` rather than a steering answer. `Obstacles.fromDisparity` guards the same way: `minNearness` in `[0, 1]` and `minArea` non-negative.

Absolute pose: where am I, really

Odometry drifts because it only ever knows about the previous frame. `Localizer` answers in map coordinates instead, and its error does not accumulate.

Example 32-8. Pose from 3D-to-2D correspondences.

```
final case class CameraPose(rotation: Seq[Seq[Double]], translation:
Seq[Double]):
  def position: Seq[Double]
```

```
def locate(
    worldPoints: Seq[(Double, Double, Double)],
    imagePoints: Seq[Point],
    intrinsics: Intrinsics
): Option[CameraPose]
```

The inputs are pairs: a known 3D point in the map, and where it appears in this frame. In practice they come from matching this frame's `Features` against descriptors stored with the map's points — the same recognition machinery loop closure uses, pointed at a different question. `solvePnP` recovers six degrees of freedom, and `rotation` and `translation` are the transform into the camera frame, `x_cam = R·x_world + t`. Note which part of `Intrinsics` each call reads: `locate` passes both the camera matrix **and** the `distortion` coefficients, so a wide lens is corrected for; `estimate` uses the camera matrix alone. Undistort your points before an essential-matrix estimate on a lens that needs it. `position` is the inverse of that: the camera's own location in world coordinates, `-Rᵀ·t`, computed for you so you do not transpose a matrix by hand at two in the morning.

How many correspondences you need is not one number. Four are enough when the world points are coplanar — all on one wall or one floor — because that admits a homography-based initialiser; six are needed when they are not, because the non-planar initialiser is a direct linear transform with no solution below six. OpenCV decides which case applies, by an SVD on the point covariance, so scalacv does not try to predict it — it contains the consequence instead. On four or five non-coplanar points the default solver aborts in its own assertion and throws a raw `org.opencv.core.CvException`, which is not a `CvError` and which a caller catching scalacv's error type would miss entirely. The call is wrapped in `Cv.attempt` and `locate` returns `None`. It does not throw. Too few pairs, a degenerate configuration, a solver that refuses: all `None`. The mismatched-length case is the one `require`, because that one is your bug.

Mapping: the grid and the loop

Two pieces move the front end toward something that deserves the word **map**.

The occupancy grid

`OccupancyGrid` is a top-down grid of cells, each holding a log-odds belief that the cell is occupied. Log-odds because evidence then combines by addition: an obstacle at a cell adds a constant, seeing **through** a cell subtracts one, and repeated observations accumulate without probabilities underflowing.

Example 32-9. Accumulating free and occupied space from range readings.

```
val grid = OccupancyGrid(cols = 200, rows = 200, resolution = 0.05)   // 10m ×
10m at 5cm
```

```
grid.observe(fromX = 0.0, fromY = 0.0, obstacleX = 2.0, obstacleY = 0.0)

grid.isOccupied(2.0, 0.0)      // true
grid.probability(1.0, 0.0)     // < 0.5 — seen through, so believed free
grid.probability(4.0, 0.0)     // 0.5 — never observed
grid.toImage.write("map.png")
```

`observe` does the work: it walks a Bresenham line from the sensor to the obstacle, marks every cell along the way free, and marks the endpoint occupied. `hit` and `miss` are the single-cell forms. The asymmetry in the constants is intentional — an obstacle reading adds 0.85 to a cell and seeing through one subtracts only 0.4, because a return is stronger evidence of an obstacle than a non-return is of free space — and every cell is clamped at ±4 log-odds, so a hundred identical readings cannot saturate a cell beyond what a few contrary ones can undo. Without that clamp a map can never unlearn a parked car that has since driven away.

`resolution` is world units per cell, `0.05` by default, with the grid centred on the origin. `cellOf` exposes the quantisation if you need to reason in cells. `probability` returns `0.5` for unobserved **and** out-of-bounds cells — the right answer for both, and worth remembering when a map seems to end abruptly; `isOccupied` takes a `threshold` parameter, `0.5` by default, so you can ask for a stricter belief before treating a cell as solid. `toImage` renders it greyscale: occupied white, free black, unknown mid-grey. The constructor requires positive dimensions, a positive `resolution`, and `cols × rows` inside `Int.MaxValue` — that last one so an over-large grid fails with a named message rather than a bare `NegativeArraySizeException` from an overflowed array size.

> **NATIVE MEMORY**
> `OccupancyGrid` is the one type here that owns no native memory: a `Double` array and nothing else, so a 200×200 grid is 320 KB of ordinary heap the collector handles correctly. The exception is `toImage`, which hands you an owned `Image` — and even that is built in the careful order, filling a JVM byte array **before** allocating the `Mat`, so an `OutOfMemoryError` on a large grid cannot strand a native buffer in the window between construction and ownership.

> **WARNING**
> Display-normalized disparity is not metric depth. Its brightness changes with unrelated scene extrema; the 0.4.1 whole-band steering could miss narrow obstacles. The current local-mean policy avoids whole-band dilution but is still a heuristic. Treat the image-based navigator as a demo heuristic, never a safety controller. Use measurements retaining disparity units and validity with an explicit obstacle policy. Similarly, a native pose solver's success is not sufficient evidence of observable motion or a supported pose.

Loop closure

`LoopDetector` keeps one keyframe's ORB descriptors per stored place and matches each new frame against all of them. When a new frame matches an **old** keyframe strongly, you have

been here before — which is the constraint a back end needs to redistribute accumulated drift around the loop.

Example 32-10. The detector's surface, and a run over a video.

```
final case class LoopClosure(keyframe: Int, matches: Int, score: Double)

final class LoopDetector extends AutoCloseable:
  def detect(image: Image): Option[LoopClosure]
  def addKeyframe(image: Image): Int
  def process(image: Image): Option[LoopClosure]
  def keyframeCount: Int
  def close(): Unit

object LoopDetector:
  def apply(
      maxFeatures: Int = 500,
      minMatches: Int = 20,
      recentExclusion: Int = 5,
      maxKeyframes: Int = Int.MaxValue
  ): LoopDetector

val loops = LoopDetector(minMatches = 25, maxKeyframes = 200)
try
  Camera.usingFile("drive.mp4"): camera =>
    camera.foreach(): frame =>
      loops.process(frame).foreach: closure =>
        println(f"revisited keyframe ${closure.keyframe}: ${closure.matches}
matches, " +
          f"score ${closure.score}%.2f")
finally loops.close()
```

`process` is `detect` followed by `addKeyframe`; `detect` alone checks without storing, which is what you want when frames arrive faster than you want keyframes. Both borrow the frame — neither consumes nor closes it — so `camera.foreach` is still the one closing each `Image`.

`recentExclusion` skips the most recent keyframes, defaulting to five, because the last few places you were always look like where you are now and would report a "loop closure" every frame. `minMatches` is the acceptance bar: the best-matching old keyframe is returned only if it clears that count, twenty by default. Raise it and you miss real revisits under changed lighting; lower it and repeated architecture starts firing. `score` is the honest companion — matched features as a fraction of the **current** frame's — so twenty-five matches out of thirty features and twenty-five out of five hundred, which are very different pieces of evidence, are distinguishable in the result rather than collapsed into one integer.

Matching is brute force against every live keyframe, which is fine for hundreds. A city-scale system swaps in a bag-of-words index; the contract would not change, and that index is not in scalacv.

NATIVE MEMORY

The most dangerous object in the chapter, because its leak is proportional to how long the run goes well. Every keyframe owns a native descriptor `Mat`, and `maxKeyframes` defaults to `Int.MaxValue` — unbounded. An hour of driving at one keyframe a second is 3600 descriptor mats nothing will free. Set the cap for any run whose length you do not control: past it, the oldest keyframes are evicted and their descriptors freed. Eviction leaves a tombstone rather than renumbering, so a `LoopClosure.keyframe` index handed out earlier stays meaningful — it refers to a slot that is now skipped. `keyframeCount` reports the live count, not the total ever added, and `close()` frees whatever is still live.

The front end, assembled

The pieces compose in one direction. Here is the whole flow, before the worked example puts part of it to work.

Table 32-3. What feeds what, from pixels to a map.

Stage	Consumes	Produces
`OpticalFlow.goodFeatures`	one `Image`	`Seq[Point]` — the seeds
`OpticalFlow.track`	two `Image`s, the seeds	`Seq[Track]` — correspondences
`Features.detect`	one `Image`	`Descriptors` (owned)
`Features.matches`	two `Descriptors`	`Seq[FeatureMatch]`
`VisualOdometry.estimate`	correspondences, `Intrinsics`	`Option[CameraMotion]` — relative, up to scale
`Odometry.update`	a frame per call	the same, as a running loop
`StereoDepth.disparity`	a rectified pair	an `Image`, brighter = nearer
`Obstacles.fromDisparity`	that map	`Seq[Obstacle]`, nearest first
`Navigator.steer`	that map	`Guidance` — a `Steering` and its reasons
`Localizer.locate`	3D↔2D pairs, `Intrinsics`	`Option[CameraPose]` — absolute, metric
`LoopDetector.process`	a frame per keyframe	`Option[LoopClosure]`
`OccupancyGrid.observe`	a pose and an obstacle	accumulated belief; `toImage` to look at it

Read the middle column: every stage consumes either an `Image` you own or plain Scala data, and only three hand back something that owns native memory. That is what makes the front end assemblable — you can hold a `CameraMotion` or a `Seq[Track]` for the length of a run without holding anything that has a deadline.

A trajectory from a video

Read a file, run monocular odometry over it, integrate the per-step motions into a path, and plot the path with the `Chart` API from Chapter 17. The result is shape rather than distance, for the reason the warning above gave, and the code says so where it happens.

Example 32-11. Monocular VO over a video, plotted as a trajectory.

```
import scalacv.*

OpenCv.load()

// Accumulated camera pose in the first frame's coordinate system.
final case class Where(rotation: Seq[Seq[Double]], position: Seq[Double])

def mul(a: Seq[Seq[Double]], b: Seq[Seq[Double]]): Seq[Seq[Double]] =
  (0 until 3).map(i => (0 until 3).map(j => (0 until 3).map(k => a(i)(k) * b(k)
(j)).sum))

def rotate(r: Seq[Seq[Double]], v: Seq[Double]): Seq[Double] =
  (0 until 3).map(i => (0 until 3).map(k => r(i)(k) * v(k)).sum)

val eye   = (0 until 3).map(i => (0 until 3).map(j => if i == j then 1.0 else
0.0))
val start = Where(eye, Seq(0.0, 0.0, 0.0))

val path = scala.collection.mutable.ArrayBuffer(start)

// Real numbers come from Chapter 31; Intrinsics.approx(frameSize) is the
stopgap.
val odometry = Odometry.monocular(Intrinsics(fx = 500, fy = 500, cx = 320, cy =
240))
try
  Camera.usingFile("drive.mp4"): camera =>
    camera.foreach(): frame =>
      odometry.update(frame).foreach: step =>
        // Every step is given unit length. That IS the scale ambiguity, made
explicit.
        val here = path.last
        val moved = rotate(here.rotation, step.translation)
        path += Where(
          mul(here.rotation, step.rotation),
          here.position.zip(moved).map((a, b) => a + b)
        )
finally odometry.close()

// Ground plane for a forward-facing camera: x to the right, z forward.
val plan = path.map(w => (w.position(0), w.position(2))).toSeq

Chart
  .scatter(plan, width = 480, height = 480, color = Color.Orange, radius = 2.0)
  .render(480, 480, Color.gray(20))
  .write("trajectory.png")
```

```
println(s"${odometry.framesProcessed} frames, ${path.size} pose estimates")
```

Trace the lifetimes once, because this loop is the shape every video pipeline in the book takes. `Camera.usingFile` closes the capture when the block ends and returns an `Either[CvError, Unit]`, so a missing file is a value rather than a surprise (Chapters 6 and 20). `foreach` hands over an owned `Image` per frame and closes it; this block never consumes it, because `update` only borrows `frame` — it takes its own `copy` for the next iteration's baseline, and `odometry.close()` in the `finally` releases exactly that copy. `Chart.scatter` returns a `Picture`, an immutable value with no native content, and `render` is where a single `Mat` is finally allocated, one call before `write` releases it.

Be blunt about what the plot shows. The path is the right **shape** over short intervals and wrong over long ones, in two independent ways: every step was assigned unit length, so a slow metre and a fast metre look identical, and every step's rotation error is multiplied into every later pose by that `mul`, so the whole trajectory bends. Drive a closed loop and the plotted path will not close. That is the drift `LoopDetector` detects and a back-end optimiser removes.

Getting scale back

Four practical sources. A **stereo pair** gives it directly: `B` is metres, so `Z = f * B / d` is metres, and each step scales by the depth of the points it came from. **Wheel odometry** gives distance travelled per step — exactly the missing scalar. An **IMU**, integrated twice, gives it badly alone and well when fused with vision. A **known object size** — a door, a calibration target, an ArUco marker of measured edge length (Chapter 28) — gives it whenever that object is in frame.

The multiplication happens in your code, not in `CameraMotion`: the type deliberately carries no scale field, because a field that is sometimes metres and sometimes not is worse than none.

What is honestly here

The front end reaches further than people expect: it tracks, recognises, estimates relative motion and absolute pose, turns stereo into obstacles and obstacles into steering, detects revisited places, and accumulates an occupancy map — each a typed call over data you own.

What is not here is the global optimisation: taking every keyframe, odometry constraint and loop closure and solving for the one trajectory and map that best explain all of them at once. That is pose-graph optimisation and bundle adjustment — nonlinear least squares over a large sparse system, numerics rather than computer vision, and the business of g2o, GTSAM or Ceres. Nothing in scalacv will correct a drifting trajectory for you. `LoopDetector` tells you **when** a correction is possible and **which** two poses it constrains; that is the interface to the

thing you would add. Absent for related reasons: dense flow fields, a persistent map format, bag-of-words place recognition at city scale, and stereo rectification.

Next

Every metric answer in this chapter went through an `Intrinsics`, and most examples took those numbers on faith. Chapter 31 is where they come from a chessboard rather than a guess, and it is worth rereading now that you have seen what a wrong focal length does to a pose.

Chapter 33 changes the subject from geometry to symbols. OCR asks what none of this machinery answers — not **where** is the thing, but **what does it say** — and the answer hinges almost entirely on what you do to the pixels before any recognition engine sees them: deskewing, binarising, and the pluggable engine boundary that keeps scalacv out of the business of shipping a language model.

CHAPTER 33

OCR Preprocessing

The library reads no characters. It decides whether the engine can.

Every other chapter in this part ends with an answer: a box, a pose, a marker's rotation. This one ends with a picture. scalacv does not recognise text and has no intention of learning how — recognition is a mature, well-served problem, and the engines that solve it are large native projects with their own licences, their own language data and their own release cadence. Tesseract ships a multi-megabyte `traineddata` file per language and its own natives underneath. A cloud OCR is an HTTP call with a bill attached. Neither belongs inside a thin wrapper over OpenCV.

What the wrapper can do is the part that actually decides the outcome. Take a page of printed text, scan it flat at a sensible resolution, and any modern engine reads it essentially perfectly. Photograph the same page on a table, at a slight angle, with a window on one side, and the same engine returns a mess: dropped words, `rn` where the page said `m`, a column of prices read as a column of dates. Nothing changed about the characters. Everything changed about the image.

That asymmetry is why this chapter exists. The work in front of the engine — greyscale, denoise, binarise, straighten, crop, resample — is ordinary image processing, all of it covered in Part II, and it is where your accuracy is won or lost. scalacv owns that half, defines the contract for the other half, and is explicit about the seam between them.

The running example is a supermarket receipt photographed on a desk: thermal paper, slightly curled, lit from a lamp on the left so the right-hand edge falls into shadow, held at about five degrees off level because nobody lines up a phone with a table edge. It is the worst realistic input and the most common one.

One line of build setup first. `Ocr`, `OcrEngine`, `OcrResult`, `OcrWord` and the `forOcr` extension live in `scalacv-vision`, beside the detectors of the last ten chapters, so `mvn"com.worxbend::scalacv-vision:0.4.2"` has to be on the classpath next to the core dependency of Chapter 2. Everything `forOcr` composes — `gray`, `medianBlur`, `adaptiveThreshold`, `deskew` — and everything the pipelines below add to it — `scale`, `crop`, `invert`, `morphology`, `contours` — is core, in the plain `scalacv` artifact underneath. Use `import scalacv.*` for core and `import scalacv.vision.*` for OCR; nothing in this chapter needs `scalacv-graphs`.

The seam

Three plain data types and one trait carry the whole design. `OcrEngine` has a single method, and that method borrows.

Example 33-1. The entire recognition contract.

```
trait OcrEngine:
  def recognize(image: Image): OcrResult
```

`Ocr.read` sits in front of it and does the preparation:

Example 33-2. Preparation, then delegation.

```
def read(image: Image, engine: OcrEngine, preprocess: Boolean = true): OcrResult
=
  if preprocess then
    val prepared = image.copy.forOcr()
    try engine.recognize(prepared)
    finally prepared.close()
  else engine.recognize(image)
```

Read that body as an ownership statement rather than as code. `image.copy` clones the pixels, so the image you passed in is untouched and still yours. `forOcr()` consumes the clone — every step in it is a transform, and transforms consume (Chapter 4) — leaving exactly one live handle. The `finally` frees it whether the engine returns, throws, or hangs up on a socket. Your original is never closed by this call, and never modified by it.

> **NATIVE MEMORY**
> `Ocr.read` borrows `image` and frees only the copy it made. The image you handed it is still alive when the call returns and is still yours to close. The one-line summary of the whole library applies unchanged here: whoever created the handle releases it.

Table 33-1. What comes back, and what each piece is for.

Type or member	What it holds
`OcrResult(text, words)`	The recognised `text`, plus per-word detail when the engine bothers to supply it. `words` defaults to `Seq.empty`, so an engine that only produces a blob of text is a legal engine.
`OcrResult.isEmpty`	`true` when `text.trim` is blank — the difference between "read nothing" and "read something dubious".
`OcrResult.confident(minConfidence: Float = 0.5f)`	The words at or above a confidence. How you quarantine junk without throwing the page away.
`OcrWord(text, confidence, box)`	One word: its text, the engine's score, and the bounding `Rect` it occupied.

All four are immutable Scala values with no native handle behind them, so they outlive the image they describe: an `OcrResult` can be returned from inside `Image.reading`, put on a queue, and inspected an hour later with nothing kept alive off-heap.

The `box` on each `OcrWord` is the one field with a trap in it. It is in the coordinate system of the image the engine actually saw — which, if you cropped, upscaled or deskewed first, is not the photograph you started from. Nothing in the library maps it back. If you intend to draw the boxes over the original, keep the crop rectangle and the scale factor and undo them yourself.

Under `Ocr.read`'s default the damage is smaller than it looks, and it is worth knowing exactly how small. `forOcr` composes `gray`, `medianBlur`, `adaptiveThreshold` and `deskew`, and not one of those four changes the frame size — `deskew` in particular rotates into a destination the same size as its source. So a box the engine reports under `Ocr.read(image, engine)` is already in your image's pixel dimensions, and the only thing between it and your photograph is a rotation about the centre by the skew `deskew` measured. Under a hand-built pipeline that crops or scales, all bets are off, and the mapping is yours to carry.

Results survive the images, so they are the natural thing to inspect and filter after the fact.

Example 33-3. Keeping what the engine was sure about, and looking at what it was not.

```
def readPage(photo: Image, engine: OcrEngine): Option[String] =
  val result = Ocr.read(photo, engine)      // borrows photo; photo is still
yours
  if result.isEmpty then None
  else if result.words.isEmpty then Some(result.text)   // an engine that only
returns a blob
  else
    result.words
      .filter(_.confidence < 0.6f)
      .foreach(w => println(s"review '${w.text}' at ${w.box}, scored
${w.confidence}"))
    Some(result.confident(0.6f).map(_.text).mkString(" "))
```

`confident` defaults to `0.5f`, which is a reasonable floor for a page you are going to read yourself and much too low for one that feeds a database. The three-branch shape above is what makes the function honest about engines: `isEmpty` separates a blank page from a hard one, and the `words` check keeps an engine that reports no per-word detail — a legal engine, since `words` defaults to `Seq.empty` — from silently returning an empty string.

What `forOcr` does, one step at a time

`forOcr` is a composition of four operations you already know, and its whole body fits in four lines:

Example 33-4. The OpenCV half of OCR, in full.

```
def forOcr(denoise: Int = 1, blockSize: Int = 15, c: Double = 10): Image =
  val gray = if img.channels >= 3 then img.gray else img
  val cleaned = if denoise > 0 then gray.medianBlur(denoise) else gray
  cleaned.adaptiveThreshold(blockSize = blockSize, c = c).deskew()
```

It is an extension method on `Image`, not a member of it — OCR preparation is a domain verb, and domain verbs live beside their domain. `import scalacv.vision.*` brings in this extension.

Each step earns its place, and each has a failure it is there to prevent.

Table 33-2. The four steps, and what goes wrong without each.

Step	Method	What it prevents
Greyscale	`gray`	Colour carries no information a recogniser uses, and costs three times the memory and bandwidth to carry it. The step is skipped when the image already has one channel, so `forOcr` on an already-grey scan does not waste a conversion.
Denoise	`medianBlur(denoise)`	A median blur is the specific cure for salt-and-pepper speckle: it replaces a pixel with a neighbourhood median, so an isolated black dot vanishes instead of being smeared into a grey smudge the way a Gaussian would. Without it, thresholding promotes every sensor speck to ink.
Binarise	`adaptiveThreshold(blockSize, c)`	A per-neighbourhood cutoff, which is what makes a lamp on one side survivable. A single global threshold has to choose between losing the shadowed edge and flooding the lit one.
Deskew	`deskew()`	Tilt. See the next section — it is the step people skip and the one that costs the most.

The binarisation is the step readers most often get wrong before they find `forOcr`, and the wrong version is worth seeing. On the receipt, with the lamp on the left:

```
photo.gray.threshold(128)   // one cutoff for the whole frame
```

The lit half of the paper sits well above 128 and comes out clean. The shadowed right-hand quarter sits below it, so the paper itself is classified as ink and the last two characters of every price disappear into a solid black block. The engine reads a truncated total with no error and no warning. The adaptive version compares each pixel against a weighted average of its own `blockSize` × `blockSize` neighbourhood, minus `c`, so a uniformly darker region is still mostly paper:

```
photo.gray.adaptiveThreshold(blockSize = 31, c = 12)
```

The weighting is `AdaptiveMethod.Gaussian` by default, which is what `forOcr` gets — a Gaussian-weighted neighbourhood, softer at the edges than the flat `AdaptiveMethod.Mean` and slightly more tolerant of a stroke sitting near the edge of the window. `Image.adaptiveThreshold` also takes `method` and `mode` (`mode = Threshold.Mode.BinaryInv`

gives white ink on a black page, which is what `contours` and the morphology below want); `forOcr` exposes neither, so a pipeline that needs them is a hand-built one.

Table 33-3. The three knobs on `forOcr`, and which way to turn them.

Parameter	Default	What it does, and when to move it
`denoise`	`1`	Median-blur radius, where `1` means a 3×3 kernel; `0` skips the step entirely. Raise it for a noisy phone photo at high ISO. Set it to `0` for a rendered PDF page or a clean flatbed scan, where there is no speckle to remove and the blur only softens the strokes.
`blockSize`	`15`	The adaptive neighbourhood, which must be odd and at least 3 — anything else is rejected with an `IllegalArgumentException` before OpenCV sees it. It wants to be comfortably larger than a stroke and smaller than a lighting gradient. Raise it for large text, lower it for small dense text.
`c`	`10`	The bias subtracted from the weighted neighbourhood average. Raise it to keep less ink when letters blob together; lower it when thin strokes vanish. It is the knob to reach for first, because it is the cheapest to test.

Deskew, and what two degrees costs

Tilt is the failure mode that surprises people, because a page two degrees off level still looks straight. To a recogniser it is not. Layout analysis begins by finding rows of text, and it finds them by projecting ink onto the vertical axis and looking for the gaps. Over a 1500-pixel-wide receipt, two degrees of tilt drags the right-hand end of a line `1500 * tan(2 deg)` below its left-hand end — about fifty pixels, several times the gap between lines. The projection that should have shown five sharp peaks separated by white shows one continuous smear, the line finder gives up or guesses, and characters from two rows get assembled into words that were never printed.

`deskew` is the correction, and its implementation is short enough to describe exactly. It binarises the image with an inverted Otsu threshold, so the ink becomes the white foreground and the page becomes black. It calls `Core.findNonZero` to collect every ink pixel, converts that cloud to `CV_32F` points, fits a minimum-area rectangle to it, and takes the rectangle's angle. That angle is folded into the range from just above −45 to 45 degrees, and the image is rotated by it about its centre with `warpAffine`, filling the exposed corners white.

Two guards make it safe to call unconditionally. A page with no ink — `findNonZero` returning zero rows — is cloned through untouched. A measured skew below 0.1 degrees, or above `maxAngle` (default `45.0`), is also left alone: a page dominated by a large graphic can fool the estimate badly, and rotating by a wrong guess is worse than not rotating at all. If you know your scans are never more than a few degrees out, say so:

```
scan.deskew(maxAngle = 8.0)   // anything larger is a misread, not a tilt
```

`maxAngle` is itself checked: it must lie in `(0, 90]`, and a zero or negative value is an `IllegalArgumentException` rather than a silently disabled correction.

Two properties of the rotation matter when you compose it with anything else.

The frame size is preserved. `deskew` rotates into a destination the same size as the source, so the corners of a strongly tilted page swing outside the frame and are clipped. This is deliberate — a deskewed page that changes dimensions breaks every downstream coordinate — but it means a ten-degree correction on a page whose text reaches the margins will shave the corners. `rotate`, the general-purpose transform from Chapter 10, does the opposite: it widens the destination to the rotated bounding box so nothing is lost. If your text lives near the edges, pad before you deskew.

> **WARNING**
> Pad with white, not with the default. `pad(size)` and `border(...)` fill with `Scalar.Black` unless you say otherwise, and `deskew` binarises with an **inverted** Otsu threshold — so a black border is not neutral background to it, it is a thick rectangular blob of ink around the page. The minimum-area rectangle then fits that border rather than the text, and the measured skew is whatever the border happens to be, usually zero. Write `page.pad(60, color = Scalar.White)` and the estimate is undisturbed.

The rotation resamples with bilinear interpolation. That is the right choice for a photograph and a slightly awkward one for a binary image: `forOcr` binarises and **then** deskews, so the two-valued image it produces comes back with soft grey edges on every stroke. Engines cope with this comfortably — most re-binarise internally anyway — but if you need a strictly two-valued result, swap the order and threshold last.

Example 33-5. Deskew first, binarise second, when the output must stay two-valued.

```
photo.gray
  .medianBlur(1)
  .deskew(maxAngle = 15.0)               // resamples greys, which is fine: they
are greys
  .adaptiveThreshold(blockSize = 31, c = 12)
```

Two ways to measure a tilt

`deskew` fits a minimum-area rectangle to the ink. It is fast, needs no parameters, and works on anything with a dominant text direction — but it measures the **shape of the ink cloud**, so a page whose text block happens to be taller than it is wide, or one with a large logo in a corner, can pull the estimate off.

The other classical estimator uses the line structure directly, with the Hough transform from Chapter 13. Detect edges, take the long near-horizontal segments, and use their median angle:

```
val edges = photo.copy.gray.canny(80, 160)
val angles = edges.mat
  .houghLinesP(threshold = 80, minLineLength = photo.width / 4.0, maxLineGap
= 20)
  .map(s => math.toDegrees(math.atan2((s.y2 - s.y1).toDouble, (s.x2 -
s.x1).toDouble)))
  .filter(a => math.abs(a) < 20)
edges.close()
val tilt = if angles.isEmpty then 0.0 else angles.sorted.apply(angles.size /
2)
```

A median is the right statistic here: one spurious segment along the edge of the desk cannot move it. Feed `tilt` to `rotate`, and check the sign against one sample page before you trust it — OpenCV's positive angle is counter-clockwise.

This is more robust on documents with ruled lines, tables or long underscores, and more expensive and more parameterised on everything else. `deskew` is the default because it has no knobs to get wrong. Reach for Hough when the default visibly misjudges your particular pages.

Cropping to the text

An engine given a photograph of a receipt on a desk spends its layout analysis on the desk. Crop first, and both the accuracy and the runtime improve.

The text block is findable with the contour machinery from Chapter 12. Binarise, then use a morphological closing to weld neighbouring characters into solid blobs — a closing dilates then erodes, so it fills the gaps between letters without growing the outline overall — then take the union of the bounding boxes that are large enough to be text.

Example 33-6. Finding the ink, and nothing but the ink.

```
def textBlock(prepared: Image, minArea: Long = 40): Option[Rect] =
  // adaptiveThreshold leaves ink black on white; contours want the ink to be the
foreground.
  val blobs = prepared.copy.invert.morphology(MorphOp.Close, radius = 4)
  try
    val boxes = blobs.contours().map(_.boundingRect).filter(_.area > minArea)
    if boxes.isEmpty then None
    else
      val x0 = boxes.map(_.x).min
      val y0 = boxes.map(_.y).min
      val x1 = boxes.map(b => b.x + b.width).max
      val y1 = boxes.map(b => b.y + b.height).max
      Some(Rect(x0, y0, x1 - x0, y1 - y0))
  finally blobs.close()
```

`prepared.copy` is there because `invert` and `morphology` are both transforms and both consume; without it, asking where the text is would destroy the image you were about to recognise. `contours` is a query, so it borrows `blobs` and leaves it alive for the `finally`.

Give the crop a margin. Engines want white space around the glyphs, and a box that clips the descenders of the last line reads worse than one that includes a strip of desk. `crop` refuses a rectangle that does not fit inside the image — an `IllegalArgumentException` naming the rectangle and the dimensions — so clamp rather than hope:

Example 33-7. Padding a region of interest without walking off the image.

```
def padded(r: Rect, by: Int, w: Int, h: Int): Rect =
  val x = math.max(0, r.x - by)
  val y = math.max(0, r.y - by)
  Rect(x, y, math.min(w - x, r.width + 2 * by), math.min(h - y, r.height + 2 *
by))
```

Resolution, and the order of operations

The engines are conventionally trained on pages scanned at roughly 300 dpi, which puts ordinary body text around twenty pixels tall — a rule of thumb to plan against, not a number scalacv measures. A receipt photographed from a phone at arm's length may give you eight. Somewhere around ten, accuracy starts falling away in a manner no amount of thresholding recovers, because the information is not in the file: a stroke thinner than a pixel was never sampled.

Upscaling does not add information either, but it does give the engine's own filters something to work with, and it reliably helps. `scale` with cubic interpolation is the tool:

```
photo.gray.scale(2.0, Interpolation.Cubic)
```

Do it on the greyscale image, before binarising. Interpolating a two-valued image produces grey edges that then have to be thresholded a second time, and each round trip thickens the strokes a little more; interpolating continuous tone and thresholding once does not. This is the second reason to assemble a pipeline by hand rather than call `forOcr` — the composition it hard-codes is a good default, not a law.

> **TIP**
> Measure before you scale. Run `textBlock` on a prepared page, divide the block's height by the number of lines you expect, and you have an estimate of the line height in pixels. Scale to bring that to twenty, and no further — upscaling past the point where it helps only makes the engine slower.

Tables, columns, and text that is not level

Three layouts break the single-block assumption, and each has a preprocessing answer.

Columns. Two columns of text produce, after a closing, two clusters of bounding boxes with a vertical gap between them: group the boxes `textBlock` collects by their `x`, find the widest empty vertical band, and recognise each side as its own image. Engines attempt column detection themselves, but on the assumption of a clean scan; handing them one column at a time removes the guess.

Tables. Ruled lines are ink, and they confuse a character recogniser, which sees a horizontal rule as a very long underscore or a row of hyphens. The classical removal uses a deliberately anisotropic structuring element — long and one pixel high — to isolate the horizontal rules, which are then subtracted. scalacv's morphology deliberately will not build one. Both `Image.morphology` and the mid-level `Ops.morphology` derive their structuring element from a single `radius`, always `2 * radius + 1` on both axes, with `shape` choosing between `MorphShape.Rect`, `Ellipse` and `Cross` and `Ops.morphology` adding an `iterations` count on top. There is no parameter that can make it 40 wide and 1 tall, because a symmetric element is what nine uses in ten want and an axis-dependent one is easy to get backwards. For the tenth, borrow the `Mat` and call OpenCV directly.

Example 33-8. The escape hatch, for a kernel the high-level API will not build.

```
import org.opencv.core.{Mat, Size as CvSize}
import org.opencv.imgproc.Imgproc

/** The long horizontal rules of `ink` --- a binary image with the ink white.
`ink` is borrowed. */
def horizontalRules(ink: Image): Image =
  // A 40x1 element opens to runs at least forty pixels long and one tall: rules,
not glyphs.
  Managed.use(Imgproc.getStructuringElement(MorphShape.Rect.cvValue, CvSize(40,
1))): kernel =>
    val dst = Mat()
    Cv.orThrow("openRules")(Imgproc.morphologyEx(ink.mat, dst,
MorphOp.Open.cvValue, kernel))
    Image.wrap(Managed(dst))
```

Three ownership moves are packed into those five lines. `Managed.use` adopts the kernel OpenCV allocated and releases it when the block returns, whatever the block does. `ink.mat` borrows — it hands out the underlying `Mat` without spending the `Image`, so the caller's handle is still alive afterwards. And `Image.wrap(Managed(dst))` takes ownership of the raw `Mat` the call filled in, which is the only way a `Mat` you allocated yourself becomes an `Image` the rest of the library will free for you. `Cv.orThrow` is there because `morphologyEx` is a native call like any other: it turns a raw `CvException` into a `CvError.NativeCall` carrying the operation name — the same currency of failure as every other error in the library (Chapter 6) — and throws that instead.

Subtracting the rules is the other half, and it is a plain absolute difference:

Example 33-9. Rules found, rules removed.

```
def withoutRules(prepared: Image): Image =
  val ink = prepared.copy.invert                          // ink white, page black
  try
    val rules = horizontalRules(ink)                      // borrows ink
    try Image.wrap(ink.mat.absdiff(rules.mat)).invert     // borrows both; back to
ink-on-white
    finally rules.close()
  finally ink.close()
```

`prepared.copy` again, because `invert` consumes. `absdiff` is an extension on `Mat` returning a `Managed[Mat]`, so both operands are borrowed and both are still yours to close — hence the nested `finally`. The cost of this trick is that it erases the parts of a glyph that sat on the rule: a descender crossing a table line comes back with a bite out of it. That is almost always a better trade than leaving the line in, but it is a trade.

Text that is not level by a quarter turn. `deskew` folds its measured angle into the range from −45 to 45 degrees, which is exactly right for a tilt and exactly wrong for a page scanned sideways: a 90-degree rotation folds to zero, the guard sees a skew below 0.1 degrees, and the page is returned untouched and still sideways. Quarter turns are yours to detect and yours to fix, with the lossless `rotate(Rotation.Clockwise)` from Chapter 10. The cheapest detector is the aspect ratio of the text block: a portrait document whose block is wider than it is tall has been laid on its side.

Bringing your own engine

An adapter over Tesseract is about a dozen lines. scalacv has already produced the clean, upright, binarised image; the adapter's job is to move pixels across a boundary and shape the answer.

Example 33-10. A Tesseract binding behind the library's interface. The library ships no such class.

```
import net.sourceforge.tess4j.Tesseract
import javax.imageio.ImageIO
import java.io.ByteArrayInputStream

final class TesseractEngine(dataPath: String, language: String = "eng") extends
scalacv.OcrEngine:
  private val tess = new Tesseract()
  tess.setDatapath(dataPath)   // the folder holding eng.traineddata
  tess.setLanguage(language)

  def recognize(image: scalacv.Image): scalacv.OcrResult =
    val png = image.copy.bytes(".png").fold(e => throw e, identity)
    val buffered = ImageIO.read(new ByteArrayInputStream(png))
    scalacv.OcrResult(text = tess.doOCR(buffered).trim)
```

NATIVE MEMORY

`image.copy` in that adapter is load-bearing, not defensive habit. `bytes` is a terminal: it encodes and then releases. Written as `image.bytes(".png")`, the adapter would free an image it only borrowed. Under `Ocr.read`'s default that damage is contained — the engine would be destroying the internal copy, which was about to be freed anyway. Under `preprocess = false` it is your image, and your next use of it throws `IllegalStateException` from a line that has nothing to do with OCR. The rule for an `OcrEngine` implementation is one sentence long: the image is borrowed, so copy before you consume.

The engine object has native memory of its own, held inside the recognition library and entirely outside `Managed` and `Releasable` (Chapter 5). Nothing in scalacv can free it and nothing will warn you. Construct one, keep it for the life of the process, and give it whatever shutdown its own API asks for — building one per page is both a leak and, because the language data is reloaded each time, the slowest possible way to call it.

The same shape fits anything: a cloud OCR behind an HTTP client, a text-recognition ONNX model through the `Dnn` layer of Chapter 26. Implement `recognize`, return an `OcrResult`, and if your engine reports per-word boxes and scores, populate `words` — callers get `confident` and box overlays for nothing.

The receipt, end to end

Putting it together, with each stage doing one job on the photograph described at the top of the chapter.

Example 33-11. Photograph to clean crop, with the engine at the end.

```
def prepareReceipt(photo: Image): Image =
  val flat = photo.gray.medianBlur(1).scale(2.0, Interpolation.Cubic)
  val straight = flat.deskew(maxAngle = 15.0)
  val binary = straight.adaptiveThreshold(blockSize = 31, c = 12)
  textBlock(binary) match
    case Some(box) => binary.crop(padded(box, 12, binary.width, binary.height))
    case None      => binary   // no ink found; hand over what we have

Image.reading("receipt.jpg"): photo =>
  val prepared = prepareReceipt(photo.copy)
  try Ocr.read(prepared, engine, preprocess = false).text
  finally prepared.close()
```

Two details in that block are about ownership rather than pixels. `photo.copy` keeps the image `reading` is managing intact, so you can still draw on it or write it out for an audit trail after recognition. And `preprocess = false` tells `Ocr.read` not to run `forOcr` over work you have already done — a second adaptive threshold over a binary image is not merely wasted, it thickens every stroke.

Table 33-4. What each stage fixed, on one photographed receipt.

Stage	Before	After
`gray`	Three channels of a colour cast from a warm desk lamp.	One channel. Nothing lost that a recogniser reads, two thirds of the bytes gone.
`medianBlur(1)`	Isolated dark pixels from a high-ISO phone sensor, scattered across the paper.	Speckle gone; strokes intact, because a median does not average across an edge.
`scale(2.0, InterpolationCubic)`	Body text about nine pixels tall — under the floor where recognition degrades badly.	About eighteen pixels, interpolated on continuous tone before any thresholding.
`deskew(15.0)`	Five degrees off level; over the receipt's width, line ends displaced far enough to merge rows in a horizontal projection.	Level to a fraction of a degree. Rows separate cleanly. The corners are clipped, which is why the crop comes last.
`adaptiveThreshold(31, 12)`	A lit left half and a shadowed right edge, with no single cutoff that keeps both.	Two-valued throughout; the shadowed prices survive because their neighbourhood is dark too.
`crop(padded(...))`	A receipt occupying perhaps a third of a frame otherwise full of desk grain, which layout analysis would have to reason about.	Just the paper, with a twelve-pixel white margin the engine expects.

The stage that most often turns an unusable result into a usable one is the deskew, and it is the stage with no visible symptom — the photograph looks fine before it and fine after it. When an engine returns plausible-looking nonsense, straighten the page first and measure again before you touch anything else.

Where this goes next

Everything here has assumed a still image that someone deliberately captured. Chapter 34 turns the camera around: screen analysis, where the input is a rendering rather than a photograph and template matching becomes exact, and the video-conferencing transforms, where the frames arrive at thirty a second and the preparation budget is measured in milliseconds rather than in what the page deserves.

CHAPTER 34

Screen Analysis and Video Conferencing

Finding a known patch on a screen, and separating a person from the room behind them.

Every other chapter in this part has treated a frame as a photograph of the world. These two do not. A screenshot is a picture of a rendering: pixels a compositor produced from a display list, deterministic in principle and not quite reproducible in practice — a font hinted differently on the next machine, a scroll position a pixel off, a clock that ticks. A webcam frame of somebody on a call is the reverse. Those pixels are natural, but you already know what is in them: one person, roughly centred, and the only question is where they stop and the room begins.

Neither is the question "what is this?". Screen automation asks two narrower ones — is this control on screen and where, and what changed since the last capture — and answers both with arithmetic over pixels, no model anywhere. Conferencing asks which pixels are the person, and that one does want a model; but softening the silhouette and blending two layers is arithmetic again.

So `Screen` is a complete application in fewer than seventy lines of code, and `BackgroundEffect` a complete compositor that hands the hard half straight back to you: bring a mask, from wherever — a network you exported, a green screen, a filled contour from Chapter 12. Both then work perfectly on the frame you tested them on and degrade on the next machine, at the next display scale, under the next lighting. The first half of this chapter builds a nightly watchdog over an internal dashboard, the second a conference feed recorded with a virtual background.

One line of setup first. `Screen`, `BackgroundEffect` and `Segmenter` all live in `scalacv-vision`, the same module as the detectors of the last ten chapters, so `mvn"com.worxbend::scalacv-vision:0.4.2"` has to sit on the classpath beside the core dependency of Chapter 2. Everything else this chapter touches — `Image`, `Rect`, `Camera`, `Codec` — is core. Use both `import scalacv.*` and `import scalacv.vision.*`; the optional artifact has its own package.

Part one: what is on the screen

Every method on `Screen` is a query in the sense Chapter 4 gave the word: it borrows the images you hand it and returns plain immutable data that outlives everything it was computed from.

Table 34-1. The three entry points on Screen, and what each rejects.

Call	Result	Throws when
`Screen.locate(image, template, minScore)`	`Option[TemplateMatch]`	the template is larger than the image
`Screen.findAll(image, template, minScore, maxMatches)`	`Seq[TemplateMatch]`, best first	the template is larger than the image; `maxMatches` is below `1`
`Screen.diff(before, after, threshold, minArea)`	`Seq[Rect]`, largest first	the two captures differ in size

The result type is two fields, neither of them native:

Example 34-1. Everything a match tells you.

```
final case class TemplateMatch(location: Rect, score: Double)
```

`location` is a `Rect` at the peak, carrying the template's own width and height; `score` is how well it matched. `minScore` defaults to `0.8` on both `locate` and `findAll`, `maxMatches` to `20`.

> NATIVE MEMORY

> `Screen` never takes ownership. Every image you pass it is alive when the call returns and still yours to `close()` — the reverse of the `Image` transforms in Chapter 4, and the rule that decides whether a polling loop runs for a week or four minutes.

Template matching is correlation, and only correlation

Template matching slides the template over every position it fits, scores the overlap, and builds a surface of scores — one per candidate top-left corner. Finding the template is finding the peak of that surface. There is no feature extraction, no descriptor, no invariance to anything.

OpenCV offers six scoring functions for the overlap. Two are squared-difference measures, where a perfect match scores **zero** and a worse one higher; two more are unnormalised correlations whose magnitude rides on how bright the template happens to be. `Screen` fixes `Imgproc.TM_CCOEFF_NORMED` and does not expose the choice.

That method subtracts the mean from both the template and the window under it, then divides by the product of their standard deviations, and the two consequences are what make one `minScore` constant usable across a whole test suite. The score always lands in `[-1, 1]` — `1.0` pixel-perfect, `0` no linear relationship, negatives inverted — so `0.8` means the same thing for a 20×20 icon and a 400×90 banner. And with the mean and deviation divided out, a uniform brightness or contrast shift barely moves it: the same button on a dimmer theme still matches.

The price sits in the denominator. A template of one flat colour has zero standard deviation, so the correlation is undefined — OpenCV does not throw, it returns a surface you cannot interpret. People meet this first, because a block of brand colour looks like the most distinctive thing on the page:

Example 34-2. Wrong: a template with no variance to correlate against.

```
// A solid 40x24 swatch of the brand colour. Nothing to correlate with.
val flat = Image.blank(40, 24, Scalar(0, 90, 200))
try Screen.locate(screenshot, flat) // meaningless, whatever it returns
finally flat.close()
```

The fix is to include the edges: crop a border of surrounding pixels so the template carries the button's outline, shadow and label — the parts that vary.

Example 34-3. Right: the control plus a margin, cropped from a copy so the capture survives.

```
// `crop` is a transform and consumes its receiver, so crop a copy — the
// screenshot has to stay alive to be searched.
val refreshButton = screenshot.copy.crop(Rect(844, 96, 88, 40))

Screen.locate(screenshot, refreshButton, minScore = 0.85) match
```

```
  case Some(m) => click(m.location.topLeft)
  case None    => fail("the refresh control never rendered")
```

`Rect` gives you `topLeft` and `bottomRight` as `Point`s, so a hit hands the automation layer somewhere to aim.

Every occurrence, and the hole each one leaves behind

`locate` is `findAll(image, template, minScore, maxMatches = 1).headOption` — literally, in the source. When the same element can appear more than once, call `findAll` directly.

Its loop dictates what you can ask of it. Each iteration takes the global maximum of the score surface with `Core.minMaxLoc`; if that clears `minScore` it is recorded, and the peak's footprint is painted out of the surface — a filled rectangle reaching half a template-width either side and half a template-height above and below — so the next iteration finds a different location rather than the same peak's shoulder. The loop ends at `maxMatches` hits, or at the first maximum below `minScore`.

So `findAll` cannot report two occurrences whose corners sit closer than about half a template apart. For a grid of controls that is what you want; for two overlapping windows showing the same icon a few pixels apart, it finds one and suppresses the other, whatever `minScore` you pick.

The score surface is not an image

The `Mat` that `matchTemplate` fills is `CV_32F`, one channel, and smaller than the input: one score per position the template fits, so its width is the image's minus the template's, plus one. It is a scalar field, not a picture, and `Screen` treats it as one: the suppression step calls `Imgproc.rectangle` directly rather than the library's own `drawRect`, and fills with a raw `-1.0` rather than a `Scalar` colour — neither of which a `Rect` and a `Scalar` would model honestly. `-1.0` is the floor of `TM_CCOEFF_NORMED`'s range, so a suppressed footprint can never outscore a live peak in a later `minMaxLoc`.

Scale and rotation: the limit you must design around

> **WARNING**
> Template matching is not scale invariant and it is not rotation invariant. A template captured at one display scale will not match the same control captured at another, and a control rotated five degrees will not match at all. No `minScore` value fixes it. Every pixel of the template is correlated against a fixed offset in the window, so a rescaled control misaligns its own interior against itself, and the peak sinks towards the surrounding noise rather than degrading gently.

In screen work this arrives through DPI. A template cut on a 1× display and searched for in a 2× capture returns `None` with total confidence — as it does for a browser at 110%

zoom, a remote-desktop session scaling the framebuffer, or a CI runner whose virtual display differs from the laptop the fixtures came from. Remove the variable where you can: capture templates at the scale you search at, and record that scale beside the fixture. Where you cannot, sweep the scale yourself, since `Screen` offers no multi-scale entry point:

Example 34-4. A scale sweep, written by hand.

```
/** The best hit for `template` across a range of rescalings, with the scale that
won. */
def locateMultiScale(
    screen: Image,
    template: Image,
    scales: Seq[Double] = Seq(0.75, 0.9, 1.0, 1.1, 1.25, 1.5)
): Option[(Double, TemplateMatch)] =
  val hits = scales.flatMap { s =>
    // `scale` consumes its receiver, so rescale a copy and release it each pass.
    val scaled = template.copy.scale(s)
    try
      // `locate` throws on a template bigger than the image; skip rather than
throw.
      if scaled.width > screen.width || scaled.height > screen.height then None
      else Screen.locate(screen, scaled, minScore = 0.7).map(s -> _)
    finally scaled.close()
  }
  hits.maxByOption(_._2.score)
```

The `copy` keeps the caller's template alive across six passes, and the size guard keeps an upscaled template from tripping `locate`'s `require`. Scores are comparable across scales because the method is normalised — but a smaller template has fewer pixels to disagree over, so on a tie prefer the larger.

Anti-aliasing, subpixel text, and where minScore should sit

What is left to erode a score is rendering, not geometry, and text is the worst offender. Subpixel rendering writes different values into the red, green and blue channels of the same edge pixel, so an identical string rendered on two machines differs per-channel along every stroke while looking the same to a person. `matchTemplate` sums correlation over channels, so that fringing lands straight in your score. Greyscaling both images averages it away:

```
val hit = Screen.locate(shot.copy.gray, tmpl.copy.gray, minScore = 0.85)
```

> **WARNING**
> Convert both or neither: OpenCV requires the image and the template to share a type, so greyscaling one raises a `CvError.NativeCall` rather than a poor score. It also discards the only signal separating two controls that differ purely in colour.

Table 34-2. Choosing minScore, and what each band assumes about the capture.

minScore	Behaviour	Use when
`0.95` +	near-exact only	the template is a byte-for-byte crop of the same asset
`0.8` (default)	a confident hit	the normal case — same widget, same theme, same scale
`0.6` – `0.75`	tolerant	mild rescaling, JPEG compression or anti-aliasing differ
below `0.6`	loose	expect false positives; usually a sign the template is wrong

What changed since the last capture

The second question needs no template. `Screen.diff(before, after, threshold = 25, minArea = 100)` compares two same-size captures and returns the changed regions as plain `Rect`s, largest first, in five unsurprising steps: absolute difference, greyscale, threshold, a `dilate(radius = 2)` fusing neighbouring changed pixels into one blob, then the bounding rectangle of every contour whose area reaches `minArea`.

Raise `threshold` when compression noise and subpixel jitter produce specks; lower it to catch a status colour shifting one shade. One giant merged region instead of several means the dilation bridged nearby changes, and its radius is not a knob — `crop` into the regions you care about and diff those separately. Sizes must agree exactly: a resized window against yesterday's capture is an `IllegalArgumentException`, because a diff of misaligned frames is not a wrong answer but a meaningless one.

`diff` is stateless: two captures in, rectangles out, nothing remembered between calls. That is what a poll wants, because the two captures it compares are minutes apart and the baseline is whichever one you kept. It is **not** what a live feed wants — for thirty frames a second against a background that drifts with the light, reach for the stateful `MotionDetector` of Chapter 21, which retains the previous frame or an adaptive MOG2 model for you.

Masking out the parts that always change

A dashboard has a clock, a "last updated 4s ago" caption and a sparkline on a timer, and a diff a minute apart reports all three. `diff` takes no mask parameter, and filtering afterwards is cheaper, since it neither copies nor consumes the captures. `Rect` carries no intersection helper, so the overlap test is four comparisons you write inline:

Example 34-5. Volatile regions declared once, and excluded from every poll.

```
val volatile = Seq(
  Rect(1712, 12, 200, 32),  // the clock
  Rect(24, 640, 360, 180)   // the auto-refreshing sparkline
)

def overlaps(a: Rect, b: Rect): Boolean =
  a.x < b.x + b.width && b.x < a.x + a.width &&
    a.y < b.y + b.height && b.y < a.y + a.height

def realChanges(before: Image, after: Image): Seq[Rect] =
  Screen
```

```
    .diff(before, after, threshold = 30, minArea = 200)
    .filterNot(r => volatile.exists(overlaps(r, _)))
```

Painting those regions flat on copies of both captures instead — `drawRect(clock, Scalar.Black, Thickness.Filled)` — costs two clones per poll, and earns them when a volatile blob keeps merging with a real change beside it.

Detecting a stuck render

A live dashboard is **supposed** to change, so "nothing changed" is itself the alarm — but an unmasked diff cannot tell a frozen page from a healthy quiet one. Split it across two diffs, one on the clock alone and one on everything else:

Example 34-6. The clock proves the page is alive; the rest proves it is doing something.

```
val clock = Rect(1712, 12, 200, 32)

def health(before: Image, after: Image): String =
  // `crop` consumes, so crop copies — the full captures are still needed below.
  val b = before.copy.crop(clock)
  val a = after.copy.crop(clock)
  try
    val ticking = Screen.diff(b, a, minArea = 4).nonEmpty
    (ticking, realChanges(before, after).nonEmpty) match
      case (false, _)    => "STUCK: the render is frozen"
      case (true, false) => "alive, no data change"
      case (true, true)  => "alive, data moved"
  finally
    b.close()
    a.close()
```

The poll loop, and why it is the memory-critical part

A watchdog is a loop that holds two captures: the baseline and the one just taken. `Screen` borrows both and closes neither, so the loop owns them — and the moment it forgets one, the leak is proportional to uptime rather than to anything you will see in a heap dump.

Example 34-7. A watchdog that holds exactly one capture between iterations.

```
/** Polls until `stop` says otherwise, keeping the previous capture as the
baseline. */
def watch(grab: () => Image, everyMs: Long, stop: () => Boolean): Unit =
  var last = grab()
  try
    while !stop() do
      Thread.sleep(everyMs)
      val now = grab()
      try println(health(last, now))
      finally
        // The new capture becomes the baseline; the old one is released here,
```

```
        // on the exception path as well as the normal one.
        last.close()
        last = now
  finally last.close()
```

> **NATIVE MEMORY**
> A 1920×1080 BGR capture is `1920 × 1080 × 3` bytes — 5.9 MiB of native memory behind a Java object the collector sizes in bytes. Drop one per poll at one poll a minute and the process is 8.3 GiB heavier a day later, with a heap that never grew. That is Chapter 1's arithmetic applied to a loop that is meant to run for weeks.

Part two: the person and everything else

The conferencing half has a smaller API and a harder problem. Two extension methods on `Image` do the compositing:

Example 34-8. The two effects, exactly as the source declares them.

```
extension (img: Image)
  def blurBackground(mask: Image, strength: Int = 15, feather: Int = 7): Image
  def replaceBackground(mask: Image, background: Image, feather: Int = 7): Image
```

Both are transforms: they consume `img` and return a new owned `CV_8UC3` `Image`, while `mask` and `background` are borrowed and come back alive.

The mask is a `CV_8UC1` image, white over the person and black over the background, the same size as the frame. That convention is the whole contract: `BackgroundEffect` holds no model and no notion of a human, so a filled contour, a white-filled detection rectangle, a green-screen key and a network's output are equally valid and composited identically.

Inside the blend, because it explains both knobs

`BackgroundEffect.alphaBlend` is private, but its steps are what `feather` and `strength` control. It Gaussian-blurs the mask with a kernel of side `2 * feather + 1` — the only place a hard binary mask acquires a soft edge — converts that to a `CV_32F` alpha in `[0, 1]`, expands it to three channels, builds the complement, and computes `fg × alpha + bg × (1 - alpha)` in float before returning to 8 bits.

`blurBackground` produces its background layer by Gaussian-blurring the frame with a kernel of side `2 * strength + 1`: the default `strength = 15` is a 31×31 blur, `strength = 21` a 43×43 studio softening. `replaceBackground` resizes your backdrop to the frame instead, which does not preserve aspect ratio. `strength` must be at least `1`, `feather` may be `0` for a hard cut but not negative, and the mask must match the frame's rows and columns.

> **NATIVE MEMORY**

A size-mismatched mask trips `alphaBlend`'s `require`, and because these are transforms the frame is consumed on that path too: the compositing deliberately runs **inside** the `try` whose `finally` closes the receiver, so a throw releases the frame rather than stranding it. The mask is never released for you.

From tensor to mask: Segmenter

`Segmenter.decodeMask(output, imageSize, threshold)` turns a segmentation network's output into the mask the effects want. It accepts a three- or four-dimensional tensor — `[C, H, W]` or `[1, C, H, W]`, which is the difference between an export that keeps its batch axis and one that does not — and reads the channel count off whichever layout it got. A one-channel output is a single foreground probability plane; a two-channel output is background and foreground, and the **last** channel is taken as the person. It then thresholds each probability at `threshold` (default `0.5f`), writes `255` or `0`, and resizes up to `imageSize` with `Interpolation.Nearest`.

So **the mask is binary**: the confidence gradient at the silhouette is discarded and the nearest-neighbour upscale leaves stair-steps. All of the softness in the composite comes from `feather`, which is why its defaults are as generous as they are.

`segment` collapses the whole path into one call:

Example 34-9. Blob, forward and decode, with the receiver left alive on purpose.

```
def segment(
    net: Net,
    inputSize: Size,
    threshold: Float = 0.5f,
    scaleFactor: Double = 1.0 / 255,
    mean: Scalar = Scalar(0, 0, 0),
    swapRB: Boolean = true
): Image
```

Unlike almost everything else on `Image`, `segment` only reads its receiver — the one reason `val mask = frame.segment(net, size)` can be followed by a line that consumes `frame`. The blob parameters mirror `Dnn.blobFromImage` from Chapter 26 and default to what a MediaPipe-selfie or MODNet-style export wants: RGB input in `[0, 1]`. The weights are yours; MediaPipe's own selfie model ships as TFLite, which OpenCV's DNN module does not read, so convert it to ONNX first.

NATIVE MEMORY

A `Net` is one of the 185 `org.opencv.*` native types with no public `release()`. `Dnn.fromOnnx` returns `Either[CvError, Managed[Net]]` so scalacv can free it anyway; load it once, outside the loop, and let `managedNet.use` scope it around the whole capture. Loading a network per frame abandons tens of megabytes of weights per second behind an object the collector sees as a few dozen bytes — the failure Chapter 1 opened with.

A mask with no model in sight

If you control the room you do not need a network. Key the backdrop colour in HSV and invert, and the white lands on the person — exactly the convention the effects expect. It is the colour segmentation of Chapter 11 with one extra step:

```
val key = frame.copy.toHsv
  .inRange(Scalar(35, 80, 80), Scalar(85, 255, 255))
  .invert
  .erode(radius = 2)
  .dilate(radius = 2)
```

The `erode` / `dilate` pair is an opening: it removes the speckle every raw colour key has, before the feather would turn it into fuzz. A green-screen key is crisper than a network's mask, so it wants a **smaller** feather — `3` to `5` rather than `7`.

The four things that look wrong

Everyone who ships this feature meets the same four artefacts.

Edge halo — a rim of old background survives around the person, or the outline is eaten into. The mask is systematically wrong at the boundary, and `feather` spreads that error over more pixels so none reads as a hard mistake. `9` to `13` suits a network mask; beyond that the background bleeds onto the shoulders.

Hair and glasses — a strand covering a third of a pixel is classified as fully one thing, and no parameter fixes that. What hides it is contrast: with `replaceBackground`, pick a backdrop whose luminance is close to the real room's, so the pixels the mask gets wrong differ less. That is the largest quality lever in the feature and it costs nothing.

A background the colour of the person — a selfie network degrades badly when the wall behind is the colour of a shirt or of skin. Lowering `threshold` below `0.5f` grows the person and recovers dropouts at the cost of grabbing wall; raising it shrinks. No single value suits both a bright room and a dim one, which argues for putting it in front of whoever is on the call.

Temporal flicker — each frame is segmented independently, so boundary pixels flip and the edge crawls. It is the one people notice first and the only one with an algorithmic fix: blend this frame's mask with the last. Because `alphaBlend` reads the mask as an alpha channel rather than a boolean, an averaged mask composites correctly — and its intermediate greys become soft edge pixels.

Example 34-10. Temporal smoothing, built from Image.blend and nothing else.

```
/** Carries the previous mask between frames so the alpha edge moves instead of
snapping. */
```

```
final class MaskSmoother(weight: Double = 0.6) extends AutoCloseable:
  private var previous: Option[Image] = None

  /** Consumes `fresh`, returns an owned smoothed mask the caller closes. */
  def smooth(fresh: Image): Image =
    // `blend` consumes the receiver and borrows the argument, so `fresh` is
spent here.
    val blended = previous.fold(fresh)(prev => fresh.blend(prev, weight))
    previous.foreach(_.close())
    previous = Some(blended.copy)
    blended

  def close(): Unit =
    previous.foreach(_.close())
    previous = None
```

One clone of a single-channel mask per frame is the price. `blend` requires both operands to match in size and type, so reset the smoother if the source changes resolution mid-stream.

> **TIP**
> A stronger blur hides a weaker mask: `blurBackground(mask, strength = 25)` destroys enough detail that wrong pixels along the edge have nothing recognisable to be wrong **about**. Prefer blur to replacement where you can.

The latency budget

Thirty frames a second is 33 milliseconds each, and the effect is not the only thing spending them: an encoder, a network stack and a conferencing client compete for the same cores.

Table 34-3. Where a frame's time goes, and the knob for each stage.

Stage	Scales with	Knob
Capture	frame size	`Camera` copies one frame per iteration; `Video.frames` borrows instead
`blobFromImage`	`inputSize`	the resize is to `inputSize`, so it is cheap whatever the frame size
`forward`	`inputSize` and the model	`inputSize` — the dominant term, and the first thing to shrink
`decodeMask`	model output, then frame size	a threshold at model resolution plus one nearest-neighbour upscale
Feather	frame size × `feather`	one Gaussian over a single-channel mask
Background blur	frame size × `strength`	`strength`; skipped entirely by `replaceBackground`
`alphaBlend`	frame size	fixed — nine full-frame passes in `CV_32FC3`
Encode	frame size and codec	`Codec`, and the resolution you record at

Every compositing row is a handful of linear passes over the frame; the `forward` row is a convolutional network. For scale, the project's benchmark page records a three-stage full-frame chain — greyscale, blur, Canny — at 145.3 µs for 640×480 and 574.2 µs for 1920×1080 on one developer machine. Those are different operations from these, and the absolutes belong to that machine rather than yours, but they place the order of magnitude: a few full-frame passes are hundreds of microseconds, and a selfie network is milliseconds.

So, the knobs in order. Shrink `inputSize` first: 160×160 is 39% of the pixels of 256×256, and most selfie exports tolerate it because the mask is upscaled and feathered anyway. Second, segment on alternate frames and reuse the previous mask — half the dominant term and half the flicker rate, for a frame of lag. Only then drop the frame resolution, which scales every remaining row at once.

A camera to a recorder, correctly scoped

The running example finishes as a complete loop. Five things in it own native memory. Four — the network, the backdrop, the camera and the recorder — are scoped by a construct rather than a `close()` you must remember. The fifth is the smoother's retained mask, which outlives no block the library knows about, so it gets the one explicit `try` / `finally` in the listing.

Example 34-11. Camera to recorder, with a virtual background and no leaks.

```
OpenCv.load()

Dnn.fromOnnx("models/selfie_segmentation.onnx").flatMap { managedNet =>
  managedNet.use { net =>
    Image.reading("backdrops/office.png") { backdrop =>
      Camera.using(0) { cam =>
        val smoother = MaskSmoother(weight = 0.6)
        try
          cam.recordTo("call.avi", fps = 30, codec = Codec.Mjpg) { frame =>
            // `segment` reads the frame and leaves it alive, so the effect on
            // the next line can consume it.
            val mask = smoother.smooth(frame.segment(net, Size(256, 256)))
            try frame.replaceBackground(mask, backdrop, feather = 11)
            finally mask.close()
          }
        finally smoother.close()
      }
    }
  }
}
```

`recordTo`'s transform is handed an owned frame and must return an `Image`; the recorder writes that result and closes it, so nothing in the lambda is yours to release except the mask. `replaceBackground` consumes `frame` and borrows `mask` and `backdrop`, so the backdrop — read once outside the loop — survives every frame. The constructs nest longest-lived outermost.

Two details of `recordTo` matter. It sizes the recorder from the **first transformed frame**, not from the camera's reported geometry, because a camera that has not yet delivered a frame

reports 0×0 — so the transform may resize, as long as it resizes every frame identically. And its default codec is `Codec.Mjpg`, the one codec videoio can always write — and it does not open in an `.mp4` or an `.mkv`, so a path ending `.mp4` fails even though the codec is present. Record to `.avi`.

> **NATIVE MEMORY**
> `Camera.using` closes the capture on every path out of the block, thrown exceptions included, and `recordTo` closes the recorder in its own `finally`. `Camera.open` does neither, and `Either.foreach` is not a scoping combinator: `Camera.open(0).foreach { cam => ... }` runs the body and leaves the device open, so the camera light stays on until the process exits.

What these two hand to Part VI

Both halves of this chapter have their correctness settled long before their performance. A template match either finds the control or it does not, and if it does not, no profiler helps: the fix is a different template or a different capture scale. A background effect either has a good enough mask or it does not, and the knobs only redistribute an error the model already made.

But both are loops around native memory. A watchdog holds two captures and runs for weeks; a conference loop holds a network, a backdrop, a camera, a recorder and two `Image`s per frame, thirty times a second, beside a video call already using the machine. This part has been about getting the answer right; from here the question is what happens when it must be produced continuously, on a machine that is also doing something else.

Chapter 35, *Performance*, opens Part VI by making that measurable rather than assumed: where the time goes, where the memory goes, and how to find out instead of guessing.

PART VI

Into Production

The part that decides whether the service survives its first week: where the time and the memory go, what may be shared between threads, what to measure, what to do when you cannot keep up, and how to package natives without a four-hundred-megabyte surprise.

CHAPTER 35

Performance

Where the time and the memory actually go, and how to find out rather than guess.

A vision pipeline has two budgets, and they are not the same budget. The first is wall clock: at 30 frames per second a frame arrives every 33.3 milliseconds, and everything you intend to do to that frame — decode, convert, filter, detect, annotate, encode — has to fit inside that window or you fall behind in the way Chapter 22, *Streaming and Backpressure*, priced out. The second is resident memory: a 1080p BGR frame is about 6 MB of off-heap pixels, so a loop that keeps one frame too many alive per iteration is losing 180 MB per second of runtime. Neither budget bounds the other. A pipeline can hit 60 fps and still be killed by the kernel at the four-minute mark, and a pipeline whose footprint is flat at 200 MB can miss its deadline on every frame.

They need separate instruments for the same reason. Wall clock is measured with a clock, and the JVM gives you a good one. Resident memory is not measured by anything the JVM offers, because the bytes are not in the heap: the forty on-heap bytes of a `Mat` header are the only part of a six-megabyte buffer that a heap profiler can see. Chapter 1, *Why This Library Exists*, gave the number — 2000 unreleased `Mat(1000, 1000, CV_8UC3)` instances left 5,865 MB resident, against 144 MB for the same 2000 with `release()` called, a factor of 41 on both JDK 21 and JDK 25 — and that ratio is what a heap graph is blind to. You will need `/proc/self/statm`, or the RSS-based `Pointer.physicalBytes()`, and you will need to watch it at the same time as you watch the clock.

The good news is that the two budgets usually fail for the same handful of reasons, and the reasons are unglamorous. Almost nothing in a scalacv pipeline is slow because an inner loop is badly written; OpenCV's inner loops are vectorised C++ that you are not going to improve from Scala. What is slow is doing the work at four times the resolution it needed, converting colour spaces twice, allocating a detector inside the loop, and copying a full frame across a module boundary that had a borrowing form available. Every measured win in this repository removed a copy. Not one of them made a kernel cleverer.

So this chapter is mostly about measurement, and about a short list of structural changes that are worth making before you measure anything at all. It ends with an ordered recipe: what to look at first when a pipeline is too slow, and what to look at first when it is too fat.

Two budgets, two instruments

Keep them separate on the dashboard as well as in your head. A single "performance" number that blends them will tell you nothing on the day one of them breaks.

Table 35-1. The two budgets and how each one is actually read.

Budget	Unit	Read it with	What a violation looks like
Wall clock	ms per frame	`System.nanoTime()` around the loop body; `Bench.measure` for one operation	growing lag with no error — reads still succeed, the frames keep getting older
Resident memory	bytes of process RSS	`/proc/self/statm` field 2 × page size, else `Pointer.physicalBytes()`	a steady climb under steady load, then the OOM killer

The instrument that does *not* work for the memory budget deserves naming, because it is the one people reach for first. `Pointer.totalBytes()` and the `-Dorg.bytedeco.javacpp.maxBytes` ceiling account for buffers JavaCPP allocated. scalacv wraps the official `org.opencv.core.Mat` JNI API, whose pixel buffers come from OpenCV's own `cv::fastMalloc` — outside that accounting entirely. A `Mat` leak runs RSS to the moon while `totalBytes()` reads flat. The ceiling that *does* see it is `-Dorg.bytedeco.javacpp.maxPhysicalBytes`, which is RSS-based:

```
java -Dorg.bytedeco.javacpp.maxPhysicalBytes=512M -jar your-app.jar
```

> **NATIVE MEMORY**
> Do not gate a leak test on `Pointer.totalBytes()`. It cannot see a scalacv `Mat`, so the test passes for exactly as long as it takes production to fall over. Snapshot RSS before and after N iterations, settle with `System.gc()` followed by `Pointer.deallocateReferences()`, and assert *bounded* growth rather than zero — allocator arenas and the JIT code cache never fully return. Chapter 40, *Testing*, has the harness this repository actually uses.

Measure before you change anything

The repository ships a `benchmarks` module for this. It is deliberately not JMH: JMH's compile-time annotation processor is awkward to wire into Mill 1.1.7 alongside the JNI natives, and the wins this project chases — whole-`Mat` allocations and full-image pixel copies removed — are large relative to timer noise. What it does instead is spelled out in `Bench.scala`, and it is worth knowing before you read a number it printed.

`Bench.measure(name, warmup, iterations)(body)` runs `body` `warmup` times untimed — the default is 2000 — then `iterations` times with `System.nanoTime()` bracketing each individual call, the default being 5000. It reports the mean, the sample standard deviation, and a 95% confidence half-width on the mean using a normal approximation, `ci95 = 1.96 × σ / √n`. Every result is folded through `Bench.blackhole`, which XORs a hash of the value into a `@volatile var sink`, because otherwise the JIT is entitled to notice that nothing reads the output and delete the work you were trying to time. `Bench.report` prints `Bench.environment`

— JVM name and version, OS and architecture, `availableProcessors`, and the names of the active collectors — above the rows, so a pasted table carries its own provenance.

Each benchmark is a plain `main`. Nothing needs an image file: `BenchImages` draws its fixtures from a fixed seed, so a run on any machine sees the same pixels.

Example 35-1. Every benchmark in the repository, and what each one isolates.

```
./mill benchmarks.runMain scalacv.bench.GrayBlurCloneBench   # a wasted clone in
Motion.prepare
./mill benchmarks.runMain scalacv.bench.ArenaReuseBench      # per-frame
destination allocation
./mill benchmarks.runMain scalacv.bench.ToMatBench           # BufferedImage ->
Mat fast path
./mill benchmarks.runMain scalacv.bench.ToBufferedImageBench # Mat ->
BufferedImage fast path
./mill benchmarks.runMain scalacv.bench.GraphicsAlphaBench   # translucent-shape
blend overhead
./mill benchmarks.runMain scalacv.bench.PictureBoundsBench   # scene-graph layout
cost vs shape count
./mill benchmarks.runMain scalacv.bench.ConfigProbeBench     # how a heavy op
scales with setNumThreads
```

The module is never published and never a CI gate. A timing regression will not turn the build red; a person running the harness is what catches it. The half of the project rule that *is* automated is the other half: no optimisation ships without a measured delta *and* a bit-identical output hash from `BenchImages.hash`, so that a change which is faster because it quietly computes something different fails before it lands.

The reading rule comes from the harness's own scaladoc and is deliberately coarse: two results are distinguishable when their confidence intervals do not overlap. That is the whole test. It is worth holding onto the caveats too — the harness does not fork a JVM per variant, so a benchmark that times two variants in one run lets the first pollute the second's JIT profile; and `1.96 × σ / √n` assumes independent, identically distributed samples, which GC pauses and OS scheduling make optimistic. Both are arguments for distrusting a 1% delta, not a 40% one.

What the benchmarks actually measured

Four optimisations shipped with recorded deltas. Every one of them deleted a copy.

Table 35-2. The four shipped wins, at the sizes they were measured at.

What changed	Case	Before	After	Delta
Translucent shapes blend a bounding box, not the canvas	640×480	194.8 µs	22.5 µs	−88%
	1920×1080	1287.5 µs	76.4 µs	−94%

	3840×2160	15297 µs	489.7 µs	−97%
`BufferedImage` → `Mat` skips the redraw for a BGR raster	640×480	131.8 µs	18.5 µs	−86%
	1920×1080	982.5 µs	116.1 µs	−88%
	3840×2160	7103.8 µs	1653.8 µs	−77%
`Mat` → `BufferedImage` writes into AWT's array	1920×1080, ch=3	1893.7 µs	976.3 µs	−48%
	3840×2160, ch=3	12540.3 µs	5446.1 µs	−57%
Motion detection stops cloning a frame it only blurs	640×480	46.3 µs	28.5 µs	−38%
	1920×1080	218.9 µs	126.8 µs	−42%

> **WARNING**
> Every absolute figure above came from one developer machine running OpenJDK 25. The commits do not record the CPU model, and it would not help you if they did: core count, OpenCV build and memory bandwidth all move these numbers. *The deltas reproduce; the absolutes do not.* Treat a µs figure as a ratio between two rows, never as a budget for your service. The full record, including the half-widths, is `docs/mdoc/benchmark-results.md` .

Read the third row group as a warning rather than a victory. Displaying a live 1080p feed now costs about half what it used to, and it is still roughly a millisecond per frame of pure copying. On-screen display stays a real per-frame cost even after the fast path.

The optimisation that was measured and not built

Received wisdom says the largest single win available in an OpenCV pipeline is reusing destination `Mat` s across frames — a scratch arena, so a 30 fps loop allocates once instead of once per stage per frame. scalacv's ownership model makes that awkward to offer, since every mid-level op allocates a fresh destination and hands you something you own. So before designing the feature, `ArenaReuseBench` sized it: the `gray → blur → canny` chain run two ways over the same frame, once through the pure API and once with three preallocated destinations reused across frames, with bit-identical output verified by hash.

At 640×480 reuse won 4%. At 1920×1080 it *lost* 0.4%. At 3840×2160 it won 0.8%. The sign of the delta flips as the frame grows and the magnitude never leaves the noise band, which is a stronger argument than any single p-value: OpenCV's allocator plus G1 make per-frame `Mat` allocation a sub-1% cost next to the compute. The arena would have added a large API surface and reversed the no-in-place ownership contract the whole library rests on, in exchange for nothing. It was not built.

If you are chasing per-frame cost, stop copying frames before you start reusing buffers. The measured wins there are an order of magnitude larger and need no new API.

Where the time goes

Resolution, which is quadratic

Cost is proportional to pixel count, and pixel count is quadratic in linear size. Halving the width of a frame quarters the work. This is the single largest lever in the chapter and it is available in almost every pipeline, because almost every pipeline is doing detection at a resolution chosen by the camera rather than by the algorithm.

The `ArenaReuseBench` sweep of the plain `gray → blur → canny` chain shows the scaling, and shows that it is not clean:

Table 35-3. The same three-stage chain across three frame sizes. The last column is derived.

Frame	Megapixels	Fresh allocation	Per megapixel
640×480	0.31	145.3 µs	473 µs
1920×1080	2.07	574.2 µs	277 µs
3840×2160	8.29	7556 µs	911 µs

1080p has 6.75 times the pixels of VGA and takes only 3.95 times as long — fixed per-call costs dominate at small sizes, so the small frame looks inefficient per pixel. 4K has 4 times the pixels of 1080p and takes 13.2 times as long, which is the opposite failure: an 8-megapixel buffer no longer fits in cache, and every stage pays for memory bandwidth instead of arithmetic. The lesson is not "cost is quadratic" so much as "cost gets worse than quadratic once you leave cache", and the practical consequence is that the step down from 4K to 1080p buys more than the arithmetic predicts.

Colour conversion, kernels, and copies

After resolution, the recurring costs are ordinary:

- **Colour conversion** touches every pixel and produces a new buffer. Converting to grey and back, or converting inside a loop what could have been converted once outside it, is a full-frame pass you did not need. Chapter 8, *Typed Constants and Colour Spaces*, is the reference for which `ColorConversion` you actually want.
- **Kernel size** multiplies. A separable Gaussian is linear in the kernel's side length, but `bilateralFilter(9, 75, 75)` — the operation `ConfigProbeBench` uses precisely because it is heavy — is not separable, and its cost grows with the square of the diameter. Doubling a neighbourhood is rarely a small change.

- **Per-frame native objects.** Not the destination `Mat`s — the sidebar above established those are free at realistic sizes — but detectors, nets, matchers and models, which are expensive to build and, in most cases, cannot be freed at all by OpenCV's own API.
- **Copies at module boundaries.** Video decode, AWT display, recorder input. Each of these has a borrowing form and a copying form in scalacv, named so you choose deliberately.

Resize early

The wrong version is the one that reads most naturally, which is why it is so common. Detect on the frame you were handed, then shrink for display:

Example 35-2. Detection at the camera's resolution, because that is the frame that arrived.

```
// WRONG at 4K: every detection pass does 8.3 megapixels of work because the
// camera chose the resolution and nobody overruled it.
var n = 0
camera.foreach() { frame =>
  n += 1
  frame.markFaces(frame.faces(detector)).write(f"out/frame-$n%05d.png")
}
```

Shrink first, detect on the small frame, and map the results back:

Example 35-3. Detect small, annotate large. One extra buffer, a quarter of the detection work.

```
val factor = 0.5

def scaledBox(r: Rect, k: Double): Rect =
  Rect((r.x * k).round.toInt, (r.y * k).round.toInt,
       (r.width * k).round.toInt, (r.height * k).round.toInt)

var n = 0
camera.foreach() { frame =>
  n += 1
  // `.copy` branches: `small` gets its own buffer, `frame` stays alive to be
drawn on.
  val small = frame.copy.scale(factor)
  val faces =
    try small.faces(detector)
    finally small.close()

  // Those boxes are in `small`'s coordinates. Map them back before they touch
`frame`.
  frame.drawRects(faces.map(f => scaledBox(f.box, 1.0 / factor)))
       .write(f"out/frame-$n%05d.png")
}
```

Two things about that second listing. `scale(factor)` takes a single factor applied to both axes and consumes its receiver, which is why the `.copy` is there: the branch is the deliberate

cost that Chapter 4, *The Image Type*, described, and it is one allocation per frame against three quarters of a detection pass saved. And the coordinate mapping is not optional. Every box, landmark and contour that comes back is in the coordinate system of the image you ran the detector on. Chapter 27, *Object Detection*, made this the central caveat of downscaled inference, and it is the bug you will actually ship: boxes that are consistently half the size they should be and clustered in the top-left quadrant are always this mistake, never a bad model.

> **TIP**
> Do not assume the detector is already shrinking the frame for you. `FaceDetect.detect` sets the detector's input size to the incoming image's size on every call — that is what lets one `FaceDetectorYN` handle frames of differing sizes at all — so a 4K frame really is detected at 4K, whatever `inputSize` you passed to `FaceDetect.create(modelPath, inputSize, scoreThreshold, nmsThreshold)`. Downscaling first therefore cuts the model's own work, not only the pipeline around it. The floor is the smallest face you must find: the detector cannot report what the shrunken frame no longer resolves.

Hoist the natives out of the loop

A `CascadeClassifier` load parses an XML model. A `Net` load parses weights. Both are measured in tens of milliseconds and both are trivially loop-invariant, so the mistake is easy to see once stated — and easy to write, because the constructor sits right next to the code that uses it.

Example 35-4. A cascade loaded, and leaked, once per frame.

```
// WRONG: reparses the model XML thirty times a second, and drops a native
// handle each time it does.
var n = 0
camera.foreach() { frame =>
  n += 1
  Cascades.load(CascadeName.FrontalFaceAlt).foreach { cascade =>
    frame.drawRects(frame.detectHaar(cascade.get)).write(f"out/frame-$n%05d.png")
  }
}
```

> **NATIVE MEMORY**
> The leak is the worse half. Of the 188 `org.opencv.*` types that own native memory, exactly three expose a public `release()`, and `CascadeClassifier` is not one of them — scalacv frees it anyway, through the `Managed[CascadeClassifier]` that `Cascades.load` returns, but only if something releases that handle. Dropped on the floor thirty times a second, it is thirty native objects a second that nothing will ever reclaim. Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, has the numbers: 4000 leaked `KalmanFilter` instances measured 54 GB against 86 MB released.

Load once, outside; scope the handle around the whole loop; let the scope release it on every exit path.

Example 35-5. One load, one release, whatever happens inside.

```
Managed.scope { own =>
  // `adopt`, not `apply`: Cascades.load already hands back a Managed, and the
  // scope takes that handle over rather than wrapping it a second time.
  val cascade = own.adopt(Cascades.load(CascadeName.FrontalFaceAlt).toTry.get)
  var n = 0
  camera.foreach() { frame =>
    n += 1
    frame.drawRects(frame.detectHaar(cascade)).write(f"out/frame-$n%05d.png")
  }
}
```

The same reasoning covers the copying at module boundaries. `Video.frames` decodes into a single reused `Mat`, refilled in place, so per-frame allocation is zero however long the video runs; `Video.framesCopied` clones per frame, lazily as you pull, for when you must keep a frame past its turn. `Recorder.write` is overloaded on `Mat` as well as `Image` for exactly this reason — a borrowed frame goes straight to the encoder with no intervening `Image` clone.

Table 35-4. The frame sources, and what each one costs you per frame.

API	What you get	Cost	Reach for it when
`Camera.foreach`	owned `Image`, closed for you	one clone	you transform or annotate the frame
`Camera.snapshot` / `take` / `taking`	owned `Image`(s)	one clone each	you want a handful of frames as images
`Video.frames`	**borrowed** reused `Mat`	**zero**	you only read the frame, or run mid-level ops over it
`Video.framesCopied`	owned `Managed[Mat]` per pull	one clone per pull	you must keep specific frames

The trade on the borrowed form is the borrowing contract from Chapter 19, *Video I/O*: do not retain the frame past its turn, and do not feed it to an `Iterator` combinator that retains. `toList`, `sliding` and `buffered` all hand you N references to one buffer holding the last frame.

What the JVM contributes, and what it cannot fix

The first hundred frames lie

The JIT compiles a method after it has been interpreted enough times to be worth compiling, and it recompiles when its speculation turns out wrong. A frame loop's first few hundred iterations are therefore measuring the interpreter and the C1 tier, not the code you shipped. `Bench.measure` defaults to 2000 untimed warmup iterations for precisely this reason, and your own instrumentation needs the same discipline: discard the opening frames before you compute a mean, or you will chase a regression that was warm-up all along. Native code inside OpenCV does not warm up — it was compiled by a C++ compiler years ago — so the

warm-up you observe belongs entirely to the Scala around the JNI calls, which makes it small in absolute terms and easy to mistake for something else.

`-Xmx` does not bound your footprint

`-Xmx2g` bounds the Java heap. It says nothing whatever about the pixel buffers, which are the part of your process that is measured in gigabytes. A container limit of 2 GB against `-Xmx2g` and a `Mat` leak gives you a process killed by the kernel with no `OutOfMemoryError`, no heap dump, and a heap graph that is flat and healthy right up to the last sample. Size the container against RSS, and set `-Dorg.bytedeco.javacpp.maxPhysicalBytes` below that so you get a Java exception rather than a SIGKILL — Chapter 39, *Degradation and Error Budgets*, is about what to do with that exception when it arrives.

> **TIP**
> One scalacv-specific flag has a per-frame cost of its own. `-Dscalacv.trackOwnership=true` makes every spent `Managed` handle allocate a `Throwable`, so that a later use-after-move can name the transform that consumed the handle rather than only the line that touched it. That is exactly what you want for the half hour you spend chasing an `IllegalStateException`, and exactly what you do not want in a frame loop, where every transform and every terminal spends a handle. It is read once at class load and off by default; leave it off in production.

GC tuning does not help native pressure

This one is worth stating flatly because it is where tuning effort most often goes to die. Heap pressure is the only thing that triggers a collection, and it is uncorrelated with native pressure. A frame loop can exhaust native memory while the heap stays small and the collector never runs at all. Switching collectors, raising the young generation, lowering a pause target — none of them change when a `Mat` is freed, because none of them can see one. The 41× from Chapter 1 is the whole argument: reclamation was never impossible, nothing made it happen in time. Releasing the handle is what makes it happen. The collector is not a participant.

Native threading fights your thread pool

OpenCV runs its own internal thread pool, and OpenBLAS runs another. For a single sequential pipeline that is exactly what you want: the heavy kernels — bilateral filter, DNN forward pass, resize — are parallelised for you, for free. The moment you fan pipelines out across your own executor, the two layers oversubscribe: every kernel spawns workers that contend with your own parallelism for the same physical cores, and the total throughput goes down. Cap the inner pools so your outer parallelism owns the cores:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 java -jar your-app.jar
```

or, from code, `org.opencv.core.Core.setNumThreads(1)` before you spread work.

How much this is worth is a property of your machine, not of the library, which is why no number for it appears in `docs/mdoc/benchmark-results.md`. `ConfigProbeBench` is the

one benchmark you are expected to run rather than read: it prints `Core.useOptimized`, `Core.getNumThreads`, `Core.getNumberOfCPUs` and the parallel, IPP and OpenCL lines of the build information, then times a `bilateralFilter(9, 75, 75)` on a 1280×720 scene at 1, 2, 4 and your CPU count of threads, restoring `Core.setNumThreads(-1)` afterwards. Run it before you decide anything about threading. Chapter 36, *Concurrency and Thread Safety*, covers the ownership rules that decide what you are even allowed to fan out.

A profiling recipe

In order, because the order is most of the value:

1. **Confirm which budget is broken.** Time the loop body and watch RSS, in the same run. Slow and flat is a compute problem; fast and climbing is a lifetime problem; both is two problems.
2. **Time the stages, not the pipeline.** Wrap each transform in `System.nanoTime()` and print a breakdown per hundred frames after the first few hundred. In practice one stage is 80% of the budget and the answer is visible in the first table you print.
3. **Check the resolution before anything else.** If the dominant stage is running at the camera's native size and does not need to be, stop here and fix that first; nothing else on this list pays as well.
4. **Count the copies.** Look for `.copy` inside the loop, `Seq[Image]` of intermediates, a `Camera` frame source where `Video.frames` would do, and `Recorder.write(image)` where a borrowed `Mat` was already to hand.
5. **Sample, with wall-clock and not CPU sampling.** JFR or async-profiler will show you where the Java side sits, and both have the same blind spot: a JNI call appears as a single frame. You will learn what `cvtColor` costs; you will not see inside it, and there is nothing inside it for you to change anyway. Wall-clock sampling at least attributes the blocked time honestly.
6. **Only then think about threads.** Run `ConfigProbeBench`, read your own scaling curve, and decide whether to cap the inner pools or leave them alone.

The step most often skipped is the second one, and skipping it is how an afternoon disappears into tuning a stage that was 3% of the frame. Measure the breakdown, fix the one that is 80%, and re-measure — the profile will have moved.

Next

Everything in this chapter assumed one thread doing one pipeline at a time, and the last two sections started to strain against that assumption. Chapter 36, *Concurrency and Thread Safety*, lifts it properly: which native handles may be shared, which must be owned by exactly one thread, why a `Managed` release is safe under a race when using the same handle is not, and what "share results, never handles" costs you when the result is a loaded model.

CHAPTER 36

Concurrency and Thread Safety

What you may share between threads, what you must not, and why the difference is a dead process rather than a wrong answer.

Every concurrency bug you have debugged in Scala had one merciful property: the program stayed alive. A lost update, a torn read, a `HashMap` corrupted by two writers — all of them produce wrong answers, and a wrong answer can be printed, logged, asserted on, and eventually understood. The JVM's memory model is a contract about visibility and ordering that holds no matter how badly you break it, and the worst outcome it permits is nonsense.

None of that applies below the JNI boundary. An `org.opencv.core.Mat` is about forty bytes of Java object holding a pointer to a `cv::Mat` that owns megabytes of C++ heap. The memory model governs the forty bytes. It has nothing to say about the megabytes, because they are not Java memory, were not allocated by the JVM, and are not visible to anything the JVM knows how to synchronise. Two threads writing the same pixel buffer is not a race in the Java sense at all. It is a race in C++, and the way C++ reports one is `SIGSEGV`: no stack trace, no exception, no failing test, just a process that stops existing mid-sentence.

That asymmetry is why this chapter is blunt rather than nuanced. There is no "probably fine", no "it only corrupts a few frames", no flag that turns the failure into something catchable. You are either inside the rules or you are gambling with the lifetime of the process, and because the failure is a crash rather than a fault, the gamble usually pays off in testing and stops paying off under load.

The good news is that the rule fits in a sentence, and the shape of parallel image work fits the rule almost by accident. Detection is embarrassingly parallel over frames or files; each unit of work wants its own buffers anyway; and everything a detector hands back is plain Scala data with no pointer behind it. What follows is the rule, the reason `Managed` makes one part of it safer than you might expect, and the two patterns — one detector per worker, one owner per handle — that cover almost every real deployment.

The rule in one line

One owner per handle. A `Mat`, a `Managed`, an `Image`, a detector, a `VideoCapture`: keep it on one thread, or guard it with a lock you wrote yourself. scalacv does not use JavaCPP's `PointerScope`, so there is no thread-local scope to reason about and no way for ownership to be implicit. Ownership is always the `Managed` you are holding, wherever that handle travels.

The reason the rule is livable rather than crippling is the other half of the design: **detector results are plain, immutable Scala data, copied out of native memory before they**

reach you. A `Seq[Face]`, a `Seq[Contour]`, an `ObjectTrack`, a `FaceMatch`, an `Int` — once computed, these belong to no thread and to every thread. So the natural pattern, do the native work on a worker and hand the **result** back, needs no synchronisation whatsoever. Chapter 24, *Face Detection*, described `FaceDetect.detect` decoding OpenCV's result `Mat` into `Face` values and releasing it before returning; that decision, which looked like ergonomics at the time, is what makes fan-out safe at all.

Here is the classification in full. Read the middle column as a verdict, not a preference.

Table 36-1. What scalacv lets you share, and what it does not.

Thing	Concurrent use	Why
`Contour`, `Rect`, `Scalar`, `Face`, `Pose`, `FaceMatch`, `ObjectTrack`, `QrCode`, `Gallery`, and every `Seq[…]` a detector returns	free	Plain Scala values copied out of native memory. No pointer behind them.
`Managed.release()` / `close()`	safe	A `getAndSet` on an `AtomicReference`. Frees exactly once even under a race.
`OpenCv.load()`	safe	Double-checked on a `@volatile` flag inside a `synchronized` block. Idempotent.
A shared `Mat`, read by several threads and written by none	safe	Concurrent reads of a buffer nobody mutates are fine.
A shared `Mat` with any writer at all	**unsafe**	OpenCV's refcount ops are atomic; pixel writes are not.
`Image`	one owner	Move semantics — a transform spends the handle, and the next use throws wherever it happens.
`FaceDetectorYN`, `Net`, `FaceRecognizer`, `Tracker`, `Kalman`	one per worker	Stateful native objects. A second thread corrupts the in-flight call.
`MotionDetector`, `ObjectTracker`, `Odometry`, `LoopDetector`	one per worker	Mutable state in Scala **and** in C++; documented single-thread.
`VideoCapture`, `Camera`, `Recorder`	one owner	A decode or encode in progress is not reentrant.

Two rows do most of the work. The top row is why the "detect on a worker, combine on the caller" pattern below needs no locks. The `Managed.release()` row is subtler than it looks, and it is worth being precise about what it buys you.

What the crash actually looks like

The reason this chapter refuses to hedge is that the failure mode leaves you almost nothing to work with. `FaceDetect`'s own test suite produced this, before the release path was made to disarm the binding's unconditional `finalize()`:

```
SIGSEGV (0xb)  C  [libopencv_java.so+0x163155]
Java_org_opencv_objdetect_FaceDetectorYN_delete
Current thread: JavaThread "Finalizer"
```

Note the thread. The mistake was made on a worker; the crash surfaced on the collector's finalizer thread, minutes later, in a frame that names no code of yours. A JVM crash log is what you get instead of a stack trace, the offending call site does not appear in it, and the reason the collector was running at that moment was that a DNN had allocated enough to trigger it. This is what "undefined behaviour" costs in practice: not a wrong answer you can chase, but a post-mortem in a language you were not writing.

What `Managed` actually promises

`Managed[A]` holds its object in a `java.util.concurrent.atomic.AtomicReference[A | Null]`, and every path that ends the handle's life — `release()`, `close()`, and the internal `take()` that `Image`'s move semantics run on — goes through `ref.getAndSet(null)` and branches on what came back. `null` means someone else got there first, and the caller does nothing at all. A non-null value means this caller won, and it is this caller that runs the `Releasable`.

That gives you one guarantee, exactly, and it is worth stating in full because it is the only concurrency guarantee in the library:

> *If two threads race to release the same `Managed`, the underlying object is freed exactly once, by exactly one of them, and the loser is a silent no-op.*

A double `delete` in C++ is undefined behaviour that merely **often** happens to survive, which makes it the worst possible class of bug: it passes CI, it passes staging, and it takes down the production process on the Tuesday you deploy something unrelated. The compare-and-set removes that possibility entirely. It is why a `LatestFrame` slot can hand ownership between a capture thread and a worker (Chapter 22, *Streaming and Backpressure*) with no lock, and why a `finally` block in a worker and a shutdown hook on the main thread can both close the same handle without either of them needing to know about the other.

Now the part people over-read. The atomic reference protects the **handle**, not the **object**. It says nothing about two threads calling `detector.get` and then using the same `FaceDetectorYN` concurrently: both `get` calls succeed, both hand back the same live pointer, and what happens next is between them and OpenCV. It says nothing about one thread writing a `Mat` while another reads it. It does not make `Managed` a lock, a mutex, or a

synchronisation point of any kind. It makes exactly one operation — the last one — safe under a race, and leaves every other operation exactly as unsafe as the native object underneath it.

> **NATIVE MEMORY**
> Release being idempotent under a race is not a licence to be vague about ownership. A handle that **no** thread releases still leaks: the compare-and-set makes a second release harmless, not a first release optional. In a fan-out, name the thread that owns each handle before you write the code, and prefer `Managed.use` or `Managed.scope` so that the release is in a `finally` you did not have to remember to write.

A spent `Image` is spent everywhere

`Image`'s move semantics were introduced in Chapter 4, *The Image Type*, as a way of keeping exactly one live `Mat` in a pipeline. They have a second property that only becomes visible once threads are involved: the failure they produce is the **same** failure whichever thread makes the mistake, and it is an exception rather than a crash. Which thread loses is a coin toss; what the loser gets is not.

A transform spends its receiver by taking the `Mat` out of the handle. Any later access — from the same thread, from a worker, from a thread that started before the transform even ran — finds `null` in the atomic reference and throws `IllegalStateException` from Scala, before anything crosses JNI. The `getAndSet` is what decides the race: exactly one caller walks away with the `Mat`, and every other caller gets the exception rather than a second reference to a buffer that is about to be rewritten underneath it.

So the mistake that would be a use-after-free in C++ is a caught exception here, and the classic version of it looks like this:

Example 36-1. Wrong. One image, two workers --- and only one of them can win.

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext.Implicits.global

import scalacv.*

// One worker's whole job: consume the Image it was handed, answer with an Int.
def edgeCount(image: Image): Int =
  val edges = image.gray.canny(80, 160)
  try edges.contours().size
  finally edges.close()

val img = Image.read("street.jpg").fold(throw _, identity)

// Both futures are handed the SAME Image. `gray` spends it, so whichever
// future reaches it second throws IllegalStateException -- and which one
// that is changes from run to run.
val a = Future(edgeCount(img))
val b = Future(edgeCount(img))
```

Be careful about what the guarantee covers, because the obvious repair is also wrong. `get` is a read, not a lock. A thread that reads the handle a microsecond before another thread's transform takes it holds a `Mat` reference that was perfectly valid when it was read, is being written somewhere else now, and carries no way of finding that out. So writing `Future(edgeCount(img.copy))` does not fix the example, it only narrows the window: `copy` clones through `handle.get`, which still races the `gray` that is spending the same handle, and the outcome is either the exception you were trying to avoid or a clone taken from a buffer mid-write.

The branch has to happen on one thread, before the fan-out, which is also the version that reads correctly:

Example 36-2. Right. The split happens before either worker starts; only plain data comes back.

```
val branch = img.copy            // one clone, on this thread, before anything
forks

val a = Future(edgeCount(branch))   // owns `branch` end to end
val b = Future(edgeCount(img))      // owns `img` end to end
```

Two handles now, one per worker, each spent and closed by the worker that owns it. Note what is **not** in the fixed version: no lock, no `synchronized`, no `AtomicReference` of your own. Two `Int`s come back and they compose freely, because an `Int` is not a handle. That is the shape to aim for everywhere — native work strictly inside a worker, plain data across the boundary, and every handle given its owner before the first thread starts.

> **TIP**
> When the `IllegalStateException` arrives from a worker, the stack points at the **reuse**, which is rarely the interesting line and is now on the wrong thread as well. Start the JVM with `-Dscalacv.trackOwnership=true` and the exception carries, as its cause, the stack of the transform or terminal that actually spent the handle. It is off by default because it allocates a `Throwable` every time a handle is spent; the check that reads it lives only on the already-failing path.

Detectors, nets and matchers: one per worker

The stateful native objects are the ones that tempt you, because they are the expensive ones. A loaded ONNX `Net` costs a file read and a graph build; an SFace `FaceRecognizer` is a 37 MB model. Everything in your instincts says: build it once, share it, save the memory. Do not.

The reason is not caution, it is mechanism. `Net.setInput` mutates the network and `Net.forward` reads that mutation back — `Dnn.forward` performs both in one call precisely so that a caller cannot accidentally widen the window between them, but that is not a lock and was never claimed to be one. `FaceDetect.detect` calls `setInputSize` on the detector for every single frame, unconditionally, because YuNet

enforces `CV_CheckEQ(input_image.size(), input_size)` and any other caller holding the same detector can change that size out from under you. And the results are stateful too: `FaceRecognizerSF.feature` writes into an internal buffer that the next call overwrites, which is why `FaceRecognizer.embed` copies the row out before returning, and why `Dnn.forward` copies its output `Mat` before handing it back.

Two threads inside any of those sequences do not get slightly wrong answers. They get a corrupted in-flight native call.

The pattern is one detector per worker, constructed once, released when the worker retires. A `ThreadLocal` expresses it directly for a pool you did not build yourself:

Example 36-3. One net per thread, built on first use --- and never shared.

```
import org.opencv.dnn.Net

import scalacv.*

val perThreadNet = ThreadLocal.withInitial[Either[CvError, Managed[Net]]] { () =>
  Dnn.fromOnnx("model.onnx")
}
```

`Dnn.fromOnnx` hands back an `Either[CvError, Managed[Net]]`, so each thread gets its own handle and its own load failure, and the `Either` is still there to be dealt with at the call site rather than thrown from a static initialiser you cannot see. What the type does **not** give you is a retirement hook: a `ThreadLocal` has no destructor, so a thread that dies still holding a `Managed[Net]` leaks the network, and there is nowhere in the declaration above to put the `close()`. That makes this the right shape for a fixed pool whose threads outlive the work, and the wrong shape for threads that come and go.

For a pool you **do** build, `Managed.scope` is the better tool, because a worker usually needs more than one native object and wants all of them released on every exit path — a normal shutdown, an exception in the detector, a model that fails to load half way through construction. `scope` registers each object the moment it is created and releases them in reverse order however the block ends, which is the guarantee a `val`-then-`try`/`finally` cannot give you: anything allocated before the `try` is unguarded.

Example 36-4. A worker's entire native footprint, scoped to the worker's lifetime.

```
import java.util.concurrent.{BlockingQueue, TimeUnit}
import java.util.concurrent.atomic.AtomicBoolean

import scalacv.*

final case class Result(path: String, faces: Seq[Face])

def worker(jobs: BlockingQueue[String], results: java.util.Queue[Result],
           modelPath: String, running: AtomicBoolean): Unit =
```

```
  Managed.scope: own =>
    // `adopt` takes over a handle that arrived already wrapped, so the detector
is
    // still released exactly once -- here, when this worker's scope ends.
    val detector = own.adopt(FaceDetect.create(modelPath, Size(320,
320)).fold(throw _, identity))

    while running.get do
      jobs.poll(50, TimeUnit.MILLISECONDS) match
        case null => ()                          // idle tick; re-check
`running`
        case path =>
          // Native work begins and ends inside this line. `reading` closes the
image
          // on success, on failure and on exception; only `Seq[Face]` escapes.
          val faces = Image.reading(path)(_.faces(detector)).getOrElse(Seq.empty)
          results.add(Result(path, faces))
```

The `poll` with a timeout rather than a blocking `take` is the shutdown mechanism, and it is the part worth copying. A worker parked forever in `take()` cannot notice that `running` went false; a worker that wakes every 50 ms notices within 50 ms, leaves the `while` loop normally, and falls out of the bottom of the `Managed.scope`, which is what releases the detector. Interrupting a thread parked in a **native** call does nothing at all — the interrupt flag is set and the C++ code carries on — so a timeout you chose beats an interrupt you cannot deliver.

Example 36-5. The pool: N workers, N detectors, one bounded queue, one clean shutdown.

```
import java.util.concurrent.{ArrayBlockingQueue, ConcurrentLinkedQueue}
import java.util.concurrent.atomic.AtomicBoolean

import scala.jdk.CollectionConverters.*

def analyseAll(paths: Seq[String], modelPath: String, workers: Int): Seq[Result]
=
  val jobs    = ArrayBlockingQueue[String](workers * 4)   // bounded: submission
blocks
  val results = ConcurrentLinkedQueue[Result]()
  val running = AtomicBoolean(true)

  val threads = (1 to workers).map: n =>
    val t = Thread(() => worker(jobs, results, modelPath, running), s"scalacv-
worker-$n")
    t.start()
    t

  try paths.foreach(jobs.put)                              // backpressure lives
here
  finally
    while !jobs.isEmpty do Thread.sleep(5)                 // let the queue drain
    running.set(false)                                     // then ask the workers
```

```
to stop
    threads.foreach(_.join())                          // and wait for every
scope to close

  results.iterator.asScala.toVector                    // plain data, safe to
read now
```

The queue is bounded on purpose. An unbounded queue of file paths is harmless, but the same skeleton with frames in it is not: every queued frame is a native allocation of several megabytes that no collector will reclaim, so an unbounded queue in front of a slow worker is a native memory leak with a schedule. Chapter 22 priced exactly that, and its latest-frame-wins slot is the version to use when the producer is a camera you cannot slow down.

Shutdown order is the other thing to copy verbatim: drain, **then** stop, **then** join, and only read `results` after the last `join()` has returned. The queue publishes each `Result` safely on its own, which is what a `ConcurrentLinkedQueue` is for, so the `join` is not there for visibility. It is there for two other things: it is the only evidence that no worker is still adding, and it is the only evidence that every `Managed.scope` has run its releases. Return before the join and you have returned while detectors are still open.

> **WARNING**
> Do not shut the pool down by setting the flag and walking away without the `join()`. A worker abandoned between `FaceDetect.create` and the end of its scope still owns a native detector, and nothing will ever take it back: a thread's death frees no native memory, and neither does the collector. In a service that builds a pool per job the leak compounds, and the compounding has been measured — 4000 leaked `KalmanFilter` instances cost **54 GB** of resident memory against **86 MB** for the same run with releases.

OpenCV is already using your cores

Every discussion of "how many workers should I run" in this library has a second party to it that you did not invite. OpenCV runs its own internal thread pool, and OpenBLAS runs another. A bilateral filter, a resize, a DNN forward pass — all of them already fan out across cores without being asked. So a pool of `availableProcessors` workers, each running a kernel that itself spawns `availableProcessors` threads, is not eight-way parallelism on eight cores. It is sixty-four runnable threads contending for eight cores, plus the context-switching and cache thrash that implies, and it is routinely **slower** than one thread doing the work sequentially.

Think of it as a single budget. Total useful threads should be roughly the core count, and you get to decide which layer spends it:

- **One pipeline at a time** — a single video being processed, a batch job with no outer parallelism. Leave OpenCV's threading alone. It parallelises the heavy kernels for you, and it is usually the faster default.

- **You fan out yourself** — a worker pool, a request handler, a per-file batch. Cap the inner pools so your outer parallelism owns the cores: `org.opencv.core.Core.setNumThreads(1)` before you spread the work, or from the environment:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 java -jar your-app.jar
```

`Core.setNumThreads(-1)` puts OpenCV's default back, which matters if you cap it for one phase of a job and want the sequential phase to go fast again.

Which of the two wins on your hardware is not something this book can tell you, and it is not something the library's benchmark page will tell you either — deliberately. `ConfigProbeBench` prints your actual runtime configuration (`useOptimized`, `getNumThreads`, `getNumberOfCPUs`, and the parallel, IPP and OpenCL lines of `Core.getBuildInformation`) and then measures a bilateral filter on a 1280×720 scene with `setNumThreads` stepped through 1, 2, 4 and your CPU count:

```
./mill benchmarks.runMain scalacv.bench.ConfigProbeBench
```

Its numbers are the one set the project refuses to publish, because thread scaling is a property of the machine rather than of the library, and quoting one laptop's answer would be worse than quoting none. Run it on the box you deploy to. Chapter 35, *Performance*, has the rest of the measurement discipline.

Virtual threads change the scheduler, not the boundary

The JDK's virtual threads are the obvious modern answer to "I have a blocking workload and want a lot of it", and video decoding plus native inference certainly looks like a blocking workload. Be precise about what they change here.

What they change: the cost of **waiting**. A worker parked on a bounded queue, a handler waiting for a request, a fan-out over a thousand files that is mostly I/O — those get cheaper, because a parked virtual thread costs a heap object rather than a megabyte of stack. A `StructuredTaskScope` fanning out over files and joining is a nicer way to write the pool above, and the ownership rules in it do not change at all.

What they do not change: anything about the native boundary. A virtual thread executing a native frame is **pinned** to its carrier platform thread for the duration of that call — the scheduler cannot unmount a stack that has C++ frames on it. So a virtual thread inside `FaceDetectorYN.detect` occupies a real OS thread for as long as the detection runs, exactly as a platform thread would. Ten thousand virtual threads all calling into OpenCV at once do not give you ten thousand concurrent detections. They give you as many detections as the scheduler has carriers, which is the core count by default, and ten thousand minus that many threads waiting for one — having consumed a scheduler's worth of bookkeeping to arrive at what a fixed pool of carrier-count size does directly.

Virtual threads are excellent for the **structure** around the native work and irrelevant to the native work itself. Size the layer that actually touches OpenCV to your cores, whichever kind of thread it runs on, and let virtual threads carry the waiting.

> **NOTE**
> A ZIO fiber has the same relationship to the rule, for the same reason. "One owner per handle" is a statement about the handle, not about the concurrency primitive: a native object belongs to exactly one fiber at a time, and `Scope` is how you make that ownership survive interruption — which `try` / `finally` cannot promise once an interrupt is in play. Chapter 37, *ZIO Integration*, has the details.

The mistakes, collected

Every one of these compiles.

Table 36-2. Five ways to break the rule, and what each of them costs.

You wrote…	What happens	Fix
Shared one `Net` or detector across futures	In-flight native call corrupted, then a `SIGSEGV`	One per worker, or a `ThreadLocal`
Collected borrowed `Video.frames` `Mat`s into a `Seq` and processed them off-thread	N references to one reused buffer, racing the decoder	`Video.framesCopied`
Two workers `crop` the same `Image`	One of them throws use-after-move — which one varies	Branch with `.copy` on the calling thread, before either starts
Two threads write the same `Mat`	Undefined behaviour	Give each its own buffer
Locked around an immutable detector result	Nothing — it is harmless and pointless	Delete the lock

The last row is there on purpose. It is the cheapest mistake in the table and the most common one in review, and it is a sign that the boundary has been drawn in the wrong place: if you feel the need to lock a `Seq[Face]`, the thing you are actually worried about is upstream, where a handle crossed a thread it should not have.

Where this leaves you

Three sentences carry the chapter. Share results, never handles. One owner per handle, one detector per worker, constructed once and released by the scope that made it. Count your threads across every layer, not only the one you wrote.

Everything here has been expressed with threads, queues and `AtomicBoolean` flags, which is the right vocabulary when the surrounding application is written that way — and it has one hole in it that this chapter has not closed. A `try` / `finally` releases on success and on exception. It does not release on **interruption**, and once an effect system is cancelling fibers,

interruption is a routine event rather than an emergency. Chapter 37, *ZIO Integration*, shows how `Scope` closes that hole, what the borrowing contracts look like once frames arrive as a `ZStream`, and the one place where the synchronous end-of-stream semantics do not carry over.

CHAPTER 37

ZIO Integration

Native lifetimes under a runtime that can cancel a fiber between two frames.

A `try` / `finally` is a promise about one thread's control flow, and it keeps that promise perfectly as long as the thread is the only thing deciding what runs next. Chapter 5 built the whole lifetime model on it: `Managed.use` acquires, runs, and releases in a single block; `Managed.scope` registers each object as it is created so a throw part-way through unwinds everything already acquired; `Image.reading` closes the image on success, on failure, and on exception. Nothing there can leak, because nothing there can be stopped from outside.

An effect runtime can stop you from outside. A ZIO fiber is interruptible at every suspension point, and a `for` comprehension has one between every step. Acquire a `CascadeClassifier` in one step, use it three steps later, release it in the fourth, and you have written a window: an interrupt delivered in the middle skips the fourth step entirely, and the `finally` you were relying on never runs because the block it belongs to never runs. For an ordinary JVM object that is a leak nobody notices. Here it is a parsed cascade sitting in native memory behind a Java object that holds nothing but a pointer, and the collector weighs the pointer. That is the arithmetic Chapter 1 opened with: 2000 unreleased 1000×1000 Mats reach 5 865 MB of RSS against 144 MB when released, because a multi-megabyte native buffer looks like about 40 bytes of Java header to the collector, and no amount of heap pressure will make it care.

The pleasant surprise is that ZIO already has the right abstraction, under a different name. ZIO's `Scope` is `Managed.scope`: a region that owns a set of resources and releases them in reverse acquisition order when it closes. `ZIO.acquireRelease` is `Managed.use` with the acquisition made uninterruptible and the finalizer guaranteed to run on every exit path an effect has — success, typed failure, defect, and interruption. Two libraries arrived at the same shape from opposite directions, one because native memory demands it and one because cancellation demands it, and the `scalacv-zio` module is nine definitions of glue that let you use the second to satisfy the first.

Nothing in this module changes the memory model. It changes who drives it. A `Mat` acquired through a ZIO scope is freed exactly once, by the scope, through the same `Releasable` instance the synchronous API uses. Using it after the scope closes is the same use-after-release error Chapter 5 catalogued, and it is caught in the same two ways: reached through a `Managed` or an `Image`, it is an `IllegalStateException` raised on the Scala side before anything crosses JNI; reached as a raw `Mat` you handed straight to `acquireRelease`, there is no wrapper to guard it and you get an emptied Mat whose `dataAddr()` reads `0`.

The module

`scalacv-zio` is published as its own artifact. Its Mill module lists `moduleDeps = Seq(core)` and two library dependencies, `dev.zio::zio:2.1.26` and `dev.zio::zio-streams:2.1.26` — nothing from `scalacv-vision`, nothing from `scalacv-graphs`. Add it beside the natives you already picked in Chapter 2:

Example 37-1. The dependency. `zio` and `zio-streams` come in transitively; the natives are still yours to choose.

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"com.worxbend::scalacv-zio:0.4.2",
  // The detector examples later in this chapter need Cascades, which lives here.
  mvn"com.worxbend::scalacv-vision:0.4.2",
  mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
  mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
)
```

Everything the module provides is a top-level definition in the package `scalacv.zio`, so `import scalacv.zio.*` brings all of it into scope. The module's own sources import ZIO as `_root_.zio.*` because the package it lives in is itself called `zio`; your code does not have to, unless you are writing inside a package of your own with the same collision.

Table 37-1. Everything in `scalacv.zio`. Nine definitions, and that is the whole module.

Definition	Type	What it does
`loadNatives`	`Task[Unit]`	`OpenCv.load()` on the blocking pool. Idempotent.
`acquireRelease(make)`	`ZIO[Scope, Throwable, A]`	Ties any `Releasable` object to the scope.
`mat.scoped`	`ZIO[Scope, Throwable, Mat]`	Extension: adopts an existing `Mat` into the scope.
`fromCv(result)`	`IO[CvError, A]`	Lifts an `Either[CvError, A]` into the typed error channel.
`readImage(path, flags)`	`IO[CvError, Image]`	Decodes an image; caller-owned.
`imageScoped(path, flags)`	`ZIO[Scope, CvError, Image]`	Decodes and closes with the scope.
`captureScoped(source, options)`	`ZIO[Scope, CvError, VideoCapture]`	Opens a source, releases with the scope.
`frameStream(capture)`	`ZStream[Any, Throwable, BorrowedMat]`	Frames as one **borrowed**, reused buffer.
`framesCopied(capture)`	`ZStream[Any, Throwable, Managed[Mat]]`	Frames as **owned** clones.

That is a deliberately small surface, and the last section of this chapter is about lifting the rest of the library yourself rather than waiting for a wrapper that is not coming. Two defaults are worth memorising now: `readImage` and `imageScoped` take `flags: ImreadFlags = ImreadFlags.Color`, and `captureScoped` takes `options: CaptureOptions = CaptureOptions.Default`.

`loadNatives` is `ZIO.attemptBlocking(OpenCv.load())` and nothing else. The blocking pool matters on the first call, which extracts about 196 MB of natives into `~/.javacpp` and `dlopen`s them; every call after that is a no-op, so requiring it at the top of each entry point costs nothing. Put it first in every program in this chapter — every other definition here touches native code.

Scope, which is `Managed.scope` with a cancellation story

`acquireRelease` takes anything with a `Releasable` instance — by name, so the construction happens on acquisition and not at the call site — and returns it in the `Scope` environment:

```
def acquireRelease[A](make: => A)(using r: Releasable[A]): ZIO[Scope, Throwable,
A]
```

The release path is not a special ZIO-only route. It calls `r.release(a)`, so a `Mat` goes through `release()` and one of the other 185 native types goes through the `delete(long)` bridge with its finalizer disarmed first, exactly as Chapter 5 described. Acquisition runs on the blocking pool, because constructing a native object frequently means opening a model file.

Example 37-2. Two objects, one scope, released in reverse order when the scope closes.

```
import _root_.zio.*
import org.opencv.core.{CvType, Mat}
import scalacv.*
import scalacv.zio.*

val program: ZIO[Any, Throwable, Int] =
  ZIO.scoped {
    for
      _     <- loadNatives
      frame <- acquireRelease(Mat(1080, 1920, CvType.CV_8UC3))
      work  <- ZIO.attemptBlocking(Mat(1080, 1920, CvType.CV_8UC1))
      grey  <- work.scoped
    yield frame.rows + grey.rows
  }
```

`work.scoped` is the extension for the case where something else already allocated the `Mat` and you want the scope to own it from here on — a mid-level `Ops` call that handed back a raw `Mat`, or a clone you made deliberately. It is `acquireRelease` with the construction already done, which means the object exists for a moment before the scope has it; keep that moment to a single `flatMap`.

> **NATIVE MEMORY**
> The scope frees these objects the instant it closes, and `ZIO.scoped` closes when its body's effect completes — not when the value you yielded is finished with. Yielding a `Mat`, an `Image`, or anything aliasing their native buffers out of `ZIO.scoped` is a use-after-release with a delay fuse. Reduce to owned data **inside** the scope — a number, an `Array[Byte]` from `Images.encode`, a fresh `Image` that owns its own Mat — and yield that. An escaped `Image` at least fails loudly: it reaches its Mat through a `Managed`, whose spent-handle guard throws an `IllegalStateException` at the first `get` rather than letting a SIGSEGV out, and `-Dscalacv.trackOwnership=true` attaches the consuming site as the cause. An escaped raw `Mat` has no wrapper and no guard: it comes back empty, and reads off it are garbage or a crash. Both are better caught in a test than in production.

The interruption guarantee is the reason to do any of this. Write the same program with a `try` / `finally` spanning two steps of a comprehension and it is correct until the day something calls `.timeout`, `.race`, or `.interrupt` on the fiber running it — at which point the release step is skipped in silence, the process RSS starts climbing, and the heap graph in your dashboard shows nothing at all.

The error channel: `CvError` typed, programmer errors as defects

Chapter 6 drew a line through the library's failure modes, and the ZIO module keeps that line where it is instead of redrawing it. Data-dependent failure — a file that is not there, bytes that do not decode, a model that will not load — is an `Either[CvError, A]`. A precondition violation is an `IllegalArgumentException`, because it is a bug in your code and pattern-matching on it would be pretending otherwise.

`fromCv` is the bridge for the first kind, and it is deliberately nothing but `ZIO.fromEither`:

```
def fromCv[A](result: => Either[CvError, A]): IO[CvError, A]
```

The point is the type. Here is the version people write first, and it throws away the distinction the core spent its whole design budget establishing:

Example 37-3. Wrong. The `CvError` is intact but the type says `Throwable`, so nothing downstream can match on it without a cast.

```
val img: Task[Image] =
  ZIO.attempt(Image.read("photo.jpg").fold(throw _, identity))
```

Example 37-4. Right. The failure keeps its type, and the effect declares that it can only fail this way.

```
val img: IO[CvError, Image] = fromCv(Image.read("photo.jpg"))
```

`readImage` is that composition with the blocking pool attached: `ZIO.blocking(fromCv(Image.read(path, flags)))`, so a decode that hits a slow disk cannot occupy a compute thread while it waits. Its result is caller-owned — you close it, or you use `imageScoped` and let a scope do it.

Table 37-2. Where each kind of failure lands, and why that is the right place for it.

Failure	ZIO channel	Reasoning
`CvError.DecodeFailed`, `LoadFailed`, `EncodeFailed`	typed error	Data-dependent and expected; you have a recovery.
`CvError.NativeCall`	typed error	OpenCV threw where `Cv.attempt` was watching.
`IllegalArgumentException` from a `require`	defect	A programmer error. Recovering from it hides the bug.
`IllegalStateException` from a spent `Managed`	defect	Use-after-release. There is no correct recovery.
A `CvError` thrown past an `Either`	defect, via `attempt`	Still a `RuntimeException`; catchable if you must.

Defects are the correct destination for the middle two rows, and it is worth being explicit about why. `.catchAll` will not see them — that is the whole point. A fiber that hands a negative `minNeighbors` to a detector, or uses an `Image` a transform already consumed, has a bug that a retry cannot fix; ZIO's defect channel takes it to the fiber's supervisor and your logs rather than into a recovery branch that pretends it was a transient failure. `CvError` itself extends `RuntimeException`, so when a `CvError` does escape as a defect — from a code path that throws rather than returning — `.catchAllDefect` can still match it by type. You should rarely want to.

Frames as a stream, with Chapter 19's borrowing contract intact

`frameStream` hands you frames as a `ZStream[Any, Throwable, BorrowedMat]`, and it inherits the contract of the synchronous `Video.frames` rather than ZStream's usual value semantics. Each emitted `BorrowedMat` is a liveness-checked view over one buffer decoded into in place, and it is valid only until the next pull or stream exit. That is not an oversight; it is the property that lets the stream stay flat in memory across an arbitrarily long video, and the module's test suite asserts it directly: streaming eight frames yields eight elements whose `dataAddr()` values collapse to a single distinct address.

The consequence is that every combinator which **retains** elements is wrong here, and the spent views throw on access.

Table 37-3. Combinators that mislead on `frameStream`. Map to an owned value first, then combine.

Combinator	What you actually get
`runCollect`	N spent views; accessing their pixels throws.
`broadcast`	Fan-out consumers racing on a single buffer.
`buffer`, `bufferSliding`	A queue of spent views.
`zipWithNext`	The previous view is spent by the next pull.

Example 37-5. Wrong. Collecting borrowed views succeeds, but their pixels are no longer accessible.

```
ZIO.scoped {
  for
    _   <- loadNatives
    cap <- captureScoped("clip.mp4")
    all <- frameStream(cap).runCollect        // Chunk[BorrowedMat]; views are
spent at stream exit
  yield all.size
}
```

Example 37-6. Right. The reduction to an owned value happens inside the stream, so what leaves it is data.

```
ZIO.scoped {
  for
    _      <- loadNatives
    cap    <- captureScoped("clip.mp4")
    // A Double per frame, not a frame per frame.
    levels <- frameStream(cap).map(f => f.get(0, 0)(0)).runCollect
  yield levels
}
```

Note which function opened the capture. `captureScoped` wraps `Video.open`, which checks `isOpened` and performs the retry that `CaptureOptions` documents, so a missing file or a busy camera arrives as a typed `CvError` at the point of opening. `acquireRelease(VideoCapture(source))` does not: OpenCV reports "I could not open that" by leaving `isOpened` false rather than by throwing, so the bare constructor hands you a live object whose every `read` returns false — which is indistinguishable from a video with no frames in it.

Why the open check is an effect and not a `require`

`frameStream` guards against being handed an unopened capture, and it does so by prepending a `ZStream.execute` that fails with `CvError.LoadFailed("capture", …)` rather than by writing a `require` at the top of its body.

The difference is where the failure happens. `frameStream` is a value-returning constructor: its body runs when the pipeline is **assembled**, which may be on a different fiber, at a different

time, and outside any error channel at all. A `require` there throws during assembly — past the `ZStream`'s error type, in whatever fiber happened to build the description. As an effect inside the stream, the check runs on the first pull, on the fiber that runs the stream, and surfaces as a typed failure the consumer can match. The module's spec pins that behaviour down: a capture opened on `/does/not/exist.avi` must make the stream **fail**, and the failure must be a `CvError.LoadFailed`, not a stream that completes with zero frames.

`framesCopied` emits caller-owned clones, not automatically released stream elements. Filtering, dropping, abandoned queues and interruption before adoption can leak them. For effectful work, the unreleased `processFrames(capture)(mat => ZIO.attemptBlocking(process(mat)))` API brackets clone acquisition and the entire callback before emitting reduced results. Keep raw Mats, native views and lazy work inside that callback, and join child work before it returns. The capture remains scoped separately. This path is sequential; parallelize reduced data, not owned frames.

> **NATIVE MEMORY**
> `Managed.use` is synchronous. Returning `ZIO.succeed(mat.rows())` from its callback closes the Mat before that effect runs. Wrap synchronous processing as `ZIO.attemptBlocking(m.use(process))`; use `m.useZIO` for effectful work after adoption, or `useManaged(acquire)(process)` to bracket acquisition too. Neither downstream form protects an earlier caller-owned stream handoff. Prefer `processFrames`, or reduce a sequential `frameStream` borrow before the next pull. Blocking finalizers are offloaded and uninterruptible, but JNI reads still cannot be forcibly cancelled by fiber interruption.

Backpressure you can actually have

Chapter 22 made the case that a camera is not a cooperating producer: there is no protocol by which your detector tells a CMOS sensor to wait, so of drop, buffer, and slow-the-producer, only the first two are available against a live source. ZIO gives you both of the available ones as operators, which is the single strongest practical argument for running your frame pipeline as a `ZStream`.

The ordering rule from the previous section decides the shape. Reduce each frame to an owned value **first**; only then reach for a buffering operator, because the operator's whole job is to retain elements and a borrowed `Mat` cannot survive being retained.

Example 37-7. Keep the newest, drop the oldest, bound the memory. The `map` must come before the buffer.

```
import scala.concurrent.duration.*

// summarise: Mat => Detection, a small owned value. record: Detection =>
Task[Unit].
ZIO.scoped {
```

```
  for
    _   <- loadNatives
    // A network source is the case the timeouts exist for, and FFMPEG honours
them.
    cap <- captureScoped(
             "rtsp://camera.local/stream",
             CaptureOptions.withTimeout(5.seconds, CaptureBackend.FFmpeg)
           )
    // Each frame becomes a small owned result inside the stream …
    _   <- frameStream(cap)
             .mapZIO(f => ZIO.succeed(summarise(f)))
             // … and only owned data reaches the sliding buffer, which drops the
             // OLDEST pending element to make room for the newest arrival.
             .bufferSliding(1)
             .mapZIO(record)
             .runDrain
  yield ()
}
```

`bufferSliding` is the drop-latency policy: a full buffer discards its oldest entry, so a slow consumer always sees recent work. `bufferDropping` is the opposite policy — a full buffer discards the **incoming** element — and it is the wrong one for frames, because the element it throws away is the newest one, which is the one you wanted. If you need the two ends on separate fibers, the same choice appears as `Queue.sliding(capacity)` against `Queue.dropping(capacity)`, with `ZStream.fromQueue` on the consuming side. `frameStream(cap, attemptsPerFrame = 3)` uses the same bounded read-retry policy as `Video.frames`; both default to one attempt for file end-of-stream.

> **WARNING**
> Neither buffer bounds how long a **read** can block. `frameStream` wraps its read in `attemptBlockingInterrupt`, which delivers a JVM `Thread.interrupt()` — and a thread parked inside OpenCV's native `read` never observes one. Interrupting the stream, or closing the scope around it, takes effect only once the in-flight read returns by itself; until then the buffer `Mat`, the exception-mode restore, and every enclosing `Scope` stay pending. Running on the blocking pool means a wedged source pins a blocking thread instead of a compute one. It does not mean the read can be cancelled. Bound it at the source instead. Pass `captureScoped` a second argument built by `CaptureOptions.withTimeout`, which sets the open timeout and the per-read timeout to the same `FiniteDuration` and takes the backend as its second parameter. Both are best-effort, because only a backend that honours `CAP_PROP_READ_TIMEOUT_MSEC` acts on them — `FFmpeg` and `GStreamer` do; `V4L2`, `AVFoundation` and the built-in MJPEG reader ignore it, and a local file needs neither, which is why both timeouts are `None` in `CaptureOptions.Default`.

A detector per fiber is not free

Fibers are cheap enough that people stop counting them, and that habit is expensive here. A `CascadeClassifier` is not a lightweight value. `Cascades.load` goes through `Cascades.resolve`, which extracts the XML from the bytedeco classifier jar to disk — javacpp caches that file,

so only the first call pays the extraction — and then parses the whole cascade, which every call pays for, and holds the parsed model in native memory for as long as the classifier lives. Spawning one per frame, or one per fiber in a pool of a few hundred, multiplies that by a number nobody chose. It is also the class of object with no public `release()` — Chapter 5′s 185 — so leaking them is the expensive kind of leak: 4000 unreleased `KalmanFilter`s measured 54 GB against 86 MB released.

Detectors are not thread-safe (Chapter 36), so sharing one across fibers is not an option either. The shape that works is a pool sized to your parallelism, with each detector scoped:

Example 37-8. One detector per pool slot, each freed when the outer scope closes.

```
import _root_.zio.*
import org.opencv.core.Mat
import org.opencv.objdetect.CascadeClassifier
import scalacv.*        // Cascades and Mat.detect live in scalacv-vision
import scalacv.zio.*

// Cascades.load returns Either[CvError, Managed[CascadeClassifier]], so the
Managed
// is what the scope releases — release() on it, not on the raw classifier.
val detectorScoped: ZIO[Scope, CvError, CascadeClassifier] =
  ZIO
   .acquireRelease(ZIO.blocking(fromCv(Cascades.load(CascadeName.FrontalFaceAlt))))
(
      m => ZIO.succeed(m.release())
    )
    .map(_.get)

// frames: Seq[Mat] you own — clones, or images you decoded. Not borrowed frames
// from frameStream, which are one buffer and cannot be looked at in parallel.
def pooled(frames: Seq[Mat]): ZIO[Scope, CvError, Unit] =
  for
    pool <- ZPool.make(detectorScoped, 4)
    _    <- ZIO.foreachParDiscard(frames) { frame =>
              ZIO.scoped(pool.get.map(detector => frame.detect(detector,
minNeighbors = 5)))
            }
  yield ()
```

The pool's finalizers run when the scope that built it closes, which is the same guarantee every other resource here gets. Size it to the number of frames you can actually process in parallel — which Chapter 35 argues is usually smaller than your core count, because OpenCV is already using threads underneath you.

A complete application

Everything above, as one program: open a source, clone each borrowed frame into a scoped owner, run a detector, annotate the owned copy, and write it. Recording an unmodified

borrowed frame can use `rec.write(frame.mat)` without copying; this example copies because it mutates the pixels.

Example 37-9. Open, detect, annotate, record. Every native object belongs to a scope.

```
import _root_.zio.*
import org.opencv.core.Mat
import org.opencv.objdetect.CascadeClassifier
import scalacv.*
import scalacv.zio.*

object AnnotatedRecording extends ZIOAppDefault:

  // The detector from the previous listing, scoped: acquired on the blocking
pool because
  // it extracts and parses an XML, released through the Managed that
Cascades.load returns.
  private val detectorScoped: ZIO[Scope, CvError, CascadeClassifier] =
    ZIO
    .acquireRelease(ZIO.blocking(fromCv(Cascades.load(CascadeName.FrontalFaceAlt))))
(
        m => ZIO.succeed(m.release())
      )
      .map(_.get)

  private def recorderScoped(path: String, info: CaptureInfo): ZIO[Scope,
CvError, Recorder] =
    val fps = if info.fps > 0 then info.fps else 30.0
    ZIO.acquireRelease(ZIO.blocking(fromCv(Recorder.open(path, info.size, fps))))
(
      r => ZIO.succeed(r.close())
    )

  private def annotate(frame: Mat, detector: CascadeClassifier): Unit =
    frame
      .detect(detector, minNeighbors = 5, minSize = Some(Size(60, 60)))
      .foreach(box => frame.drawRect(box, Scalar.Green, Thickness.Stroke(2)))

  def run: ZIO[Any, Throwable, Unit] =
    ZIO.scoped {
      for
        _        <- loadNatives
        cap      <- captureScoped("clip.mp4")
        info     <- ZIO.succeed(Video.info(cap))
        detector <- detectorScoped
        rec      <- recorderScoped("annotated.avi", info)
        written  <- frameStream(cap)
                      .mapZIO { frame =>
                        // Own the mutable copy and release it before the next
pull.
                        ZIO.attemptBlocking {
                          Managed.use(frame.clone()) { owned =>
                            annotate(owned, detector)
                            rec.write(owned)
```

```
                    }
                  }.flatMap(result => fromCv(result))
                }
                .runCount
      _      <- Console.printLine(s"wrote $written frames")
    yield ()
  }
```

Trace the exits. A missing `clip.mp4` fails at `captureScoped` with a typed `CvError.LoadFailed`, and nothing after it runs. An unavailable codec fails at `recorderScoped` the same way, after the capture is open — and the capture is released on the way out, because the scope holds it. A `Ctrl-C` during recording interrupts the fiber: the recorder is finalised (so the AVI's index is written and the file plays), the detector goes through the `delete(long)` bridge, the capture is released, and the stream's own decode buffer is freed — in reverse order, without a single `finally` in the program.

> **WARNING**
> `Video.info` reads `CAP_PROP_*` properties, and every one of them is advisory. A camera that has not yet delivered a frame commonly reports `0×0`, which would trip `Recorder.open`'s positive-size precondition and throw an `IllegalArgumentException` — a defect, not a typed failure. Against a live camera, size the recorder from the first decoded frame instead, which is what `Camera.recordTo` does for you.

Lifting the rest yourself

Nine definitions do not wrap a library this size, and they are not meant to. The module covers the things whose ZIO shape is not obvious: the blocking-pool placement, the typed error channel, the open-check on a capture, and a frame stream that respects the borrowing contract. Everything else is mechanical, and writing it inline is clearer than hunting for a wrapper.

There are three patterns and no fourth. A pure `Either[CvError, A]` becomes `fromCv(…)`. An object you own for a while becomes `ZIO.acquireRelease(acquire)(release)` — with `ZIO.succeed(m.release())` for a `Managed`, `ZIO.succeed(c.close())` for a `Camera`, `Recorder` or `Image`, and plain `acquireRelease(…)` when a `Releasable` instance exists for the raw type. A synchronous loop that must stay synchronous — `Camera.foreach`, `Video.frames` with an `attemptsPerFrame` bound — becomes `ZIO.attemptBlockingInterrupt(…)` wrapping the whole traversal, reducing each frame to owned data inside.

Example 37-10. A camera by index, which `captureScoped` does not cover: `Camera.open` takes an `Int`, so lift that.

```
// Camera.open(index, options) discards 5 warm-up frames by default, so the first
frame
// this hands back is one the exposure loop has already converged on.
```

```
def cameraScoped(
    index: Int,
    options: CaptureOptions = CaptureOptions.Default
): ZIO[Scope, CvError, Camera] =
  ZIO.acquireRelease(ZIO.blocking(fromCv(Camera.open(index, options))))(c =>
    ZIO.succeed(c.close())
  )
```

TIP

The one rule that is not mechanical: use `ZIO.attemptBlocking` or `ZIO.attemptBlockingInterrupt`, never a bare `ZIO.attempt`, around anything that touches native code or the filesystem. A hung `VideoCapture.read` on the CPU-sized default executor can starve every other fiber in the process. The module holds itself to this so strictly that one of its own tests reads its own source file, strips the block comments — the scaladoc examples do use `ZIO.attempt` — and asserts that the string `ZIO.attempt(` does not appear in what is left.

Where this goes next

The scoping discipline in this chapter makes a fiber's native footprint deterministic, which is the precondition for measuring it. Chapter 38, *Observability*, takes that up: what a frame pipeline should emit, how to watch RSS rather than heap when the interesting allocations are off-heap, and how to instrument the stages of a `ZStream` so a rising frame age shows up as a metric long before it shows up as a wrong answer.

CHAPTER 38

Observability

The four numbers a vision service is judged on, none of which a JVM dashboard shows you.

A vision service that has gone wrong rarely looks wrong. The HTTP handler still returns 200. The thread pool is not saturated. Heap usage is a flat line at 380 MB, garbage-collection pause time is under a millisecond, and the error rate is zero because nothing has thrown. Every panel on the standard JVM dashboard is green, and the service has been returning empty detection lists for twenty minutes because someone bumped the exposure on the camera.

That is the shape of the problem. The instrumentation the JVM ecosystem gives you for free was designed for a system whose work is heap-allocated objects and whose failure is an exception. A frame pipeline's work is native pixel buffers, and its failure modes are **silence**, **lateness** and **drift** — none of which raises anything. A leaked `Mat` produces no `OutOfMemoryError`, because the memory is not on the heap; the kernel arrives instead, with a `SIGKILL` and exit code 137. A consumer running four seconds behind the sensor produces no error either; it produces correct answers about a world that no longer exists. And a detector whose input conditions have shifted produces no error at all — it produces fewer boxes, with slightly lower scores, forever.

So the dashboard has to be built out of different material. Four numbers matter, and a JVM agent gives you none of them: process resident set size, the ratio of frames processed to frames the source produced, per-stage latency at the tail, and the distribution of detection scores. All four are a few lines of code away, and not one of them is written for you. scalacv ships no logging facade, no metrics registry and no tracing hooks — `scalacv`, `scalacv-vision` and `scalacv-graphs` each declare exactly one dependency, the OpenCV Java API, and `scalacv-zio` adds ZIO and nothing else. That is a trade with two halves. Nothing conflicts with your stack: no facade to bind, no `NOP` warning at startup, no fight between the SLF4J version your service chose and the one a library forced on it. And nothing is instrumented for you. This chapter is the other half of that bargain — which numbers to add, where the seams are, and which of them may wake someone up.

The signals that are worth a panel

Start from what can go wrong, not from what is easy to measure. Each row below exists because a real failure is invisible without it.

Table 38-1. The vision-service dashboard. None of these is provided by the library; all of them are yours to add.

Signal	Type	What a bad value means
process RSS	gauge	rising under steady load: a `Mat`, `Image` or detector is never closed
frames in	counter	rate below the source's fps: you are not seeing every frame the device sent
frames processed	counter	drifting below frames in: your drop discipline is running, or the loop is stalling
frames dropped	counter	any sustained rate: the consumer is slower than the producer
per-stage seconds	histogram	p99 above the frame budget: one stage owns the whole backlog
detections per frame	gauge	a step to zero with no errors: the model, the lens or the light changed
detection score	histogram	mass sliding toward the threshold: drift, before it becomes a step to zero
`CvError` by case	counter	a rise in one label localises the fault to one resource
thrown, not returned	counter	anything above zero is a bug, not bad data

Count **frames in** yourself, inside the loop body. The temptation is to take a denominator from the device instead — `Camera.info` returns a `CaptureInfo` with an `fps` and a `frameCount` — but every field of it is a `CAP_PROP_*` query the backend is free to guess at. A live camera usually answers `frameCount == 0`, because the question is meaningless for a stream that has not ended, and `fps` can be `0` before the first frame arrives. Those fields are a hint for a file transcode, where `Camera.recordTo` hands you the frames it actually wrote as a `Long` to divide by, and they are nothing at all for the live source you are most likely watching.

Two of those rows are the ones people leave off, and both deserve an argument.

Detections per frame is the only signal that catches a silent model failure. A `Net` that loads, runs and returns nothing is indistinguishable from a working detector pointed at an empty room — unless you know what the room usually looks like. A warehouse camera that has averaged 3.2 detections per frame for six months and now averages 0.0 is telling you something no error rate can.

Detection score is the leading indicator for the same failure, and it is already in your data: `Face` carries `score: Float` in `[0, 1]`, `Keypoint` carries `score`, `TemplateMatch` carries a `Double`, `OcrWord` carries `confidence`. A detector whose scores are drifting from a mode at 0.94 toward a `scoreThreshold` of 0.9 is a week away from returning nothing, and its detection count looks healthy for most of that week. Bucket the scores and you see it coming.

The `CvError` row is cheap because the vocabulary is fixed: a sealed hierarchy of exactly six cases, so an exhaustive match over it is also an exhaustive list of your metric labels. Never

label with `e.getMessage` — it contains file paths and pixel dimensions, and a metrics backend creates one time series per distinct label value.

Example 38-1. Six labels, and a timer that is honest about the failures that never reach the Either.

```
/** Your metrics client: a Micrometer `Timer.record`, a Prometheus histogram
`observe`, a Kamon timer. */
def record(operation: String, nanos: Long, outcome: String): Unit = ???

def errorLabel(e: CvError): String = e match
  case _: CvError.NativesMissing    => "natives_missing"
  case _: CvError.DecodeFailed      => "decode_failed"
  case _: CvError.LoadFailed        => "load_failed"
  case _: CvError.EncodeFailed      => "encode_failed"
  case _: CvError.CalibrationFailed => "calibration_failed"
  case _: CvError.NativeCall        => "native_call"

/** Times `call` and records it on every path — Right, Left, and thrown. */
def observing[A](operation: String)(call: => Either[CvError, A]): Either[CvError,
A] =
  val start = System.nanoTime()
  var outcome = "threw"
  try
    val result = call
    outcome = result.fold(errorLabel, _ => "ok")
    result
  finally record(operation, System.nanoTime() - start, outcome)
```

The `"threw"` outcome is the part people delete, and the part that earns its keep. scalacv returns a `Left` for failures that depend on the data — a file that is not there, bytes that do not decode — and **throws** for programmer errors: an `IllegalArgumentException` from a violated precondition, an `IllegalStateException` from an `Image` used after a transform consumed it. Those never reach the `Either`, so a helper that records only `fold` outcomes reports a perfect success rate on a code path crashing every request. The `try` / `finally` is what makes the timer tell the truth.

Take the operation name from the library rather than inventing one. Every fallible native call goes through `Cv.attempt(operation)`, whose first argument ends up inside `CvError.NativeCall(operation, cause)` and therefore inside the message an on-call engineer reads. When a timer called `Imgproc.GaussianBlur` spikes and a log line says `OpenCV failed during Imgproc.GaussianBlur`, the two join without a translation table.

Watching native memory, which is the whole ball game

Everything above is ordinary service instrumentation applied to an unusual workload. This section is the part that is genuinely different.

OpenCV's pixel buffers are allocated by C++, through `cv::fastMalloc`, outside the Java heap. `-Xmx` does not bound them, heap-used gauges do not see them, and garbage-collection metrics do not react to them. A leaked 1080p BGR frame is about six megabytes of resident

memory hiding behind roughly forty bytes of on-heap object, so a leak of a hundred frames per minute moves your heap graph by four kilobytes and your machine by six hundred megabytes.

> **NATIVE MEMORY**
> The measurement from Chapter 1 is the calibration for every dashboard in this chapter: 2000 `Mat(1000, 1000, CV_8UC3)` with their references dropped and no explicit `System.gc()` finished at 5,865 MB of RSS; the same loop with `release()` on each finished at 144 MB. Forty-one times the memory for one omitted call, and **no JVM-visible signal changed between the two runs**. Whatever else your dashboard shows, it must show RSS.

Here is the exact picture to recognise at three in the morning. Put the heap-used gauge and the RSS gauge on the same panel, with the same time axis, and read the pair rather than either one.

Table 38-2. Reading the heap graph and the RSS graph together. Only the second row is a leak.

Heap	RSS	Diagnosis
flat, sawtooth	flat plateau	healthy. RSS never returns to its start — arenas and the code cache keep what they took
flat, sawtooth	climbing, linear	a native leak. The slope is proportional to your frame rate, and it does not stop
flat, sawtooth	one step, then flat	a bigger input, not a leak — somebody uploaded a 40-megapixel photograph
climbing	climbing	an ordinary Java leak. Take a heap dump; this chapter is not about you

The second row is the signature, and its distinguishing feature is that the climb is **straight**. A Java leak bends, because the collector fights it. A `Mat` leak has no opponent: nothing in the runtime has a motive to reclaim the memory or a deadline by which to try, so the line rises at exactly the rate you allocate until the cgroup limit is reached and the kernel ends the argument.

The gauge that sees it is process resident set size. On Linux the cheapest source is `/proc/self/statm`, a one-line pseudo-file whose second field is the resident page count; elsewhere, JavaCPP's `Pointer.physicalBytes()` is also RSS-based and works everywhere. This is exactly the implementation the repository's own leak harness uses, in `leaks/test/src/scalacv/LeakAssertions.scala`.

Example 38-2. The only counter that can see an `org.opencv.core.Mat`.

```
import java.nio.file.{Files, Path}
import org.bytedeco.javacpp.Pointer

object NativeMemory:
```

```
  /** 4 KiB on x86_64 and arm64; the env var is the escape hatch for a large-page
kernel. */
  private val pageBytes: Long =
    sys.env.get("SCALACV_PAGE_BYTES").flatMap(_.toLongOption).getOrElse(4096L)

  private val statm: Path = Path.of("/proc/self/statm")

  def rssBytes(): Long =
    if Files.isReadable(statm) then
      String(Files.readAllBytes(statm)).trim.split("\\s+")(1).toLong * pageBytes
    else Pointer.physicalBytes() // macOS, Windows, or a container without /proc
```

Export it as a gauge on a slow schedule — once every 10 to 60 seconds is ample for a trend. Two warnings come with it. RSS is a **whole-process** number, so a cache, another library's off-heap buffers or a second workload in the same JVM moves it too: fine on a dashboard, where total footprint is what you care about, and fatal in an assertion, which is why the repository's leak suite runs in a JVM of its own. And pair the gauge with `-Dorg.bytedeco.javacpp.maxPhysicalBytes`, set comfortably **below** the container's memory limit, so that crossing the ceiling gives you a `java.lang.OutOfMemoryError` with a stack trace instead of a silent kernel kill. Chapter 39 explains why that error is not a `CvError` and passes straight through every `Either` in the library.

Two counters that will lie to you about this

The first is `Pointer.totalBytes()`, JavaCPP's own accounting, and the budget `-Dorg.bytedeco.javacpp.maxBytes` that is checked against it. It tracks memory JavaCPP allocated through its own `Pointer` allocators. scalacv wraps the official `org.opencv.core.Mat` Java API, whose buffers never pass through one. This project's memory audit measured the consequence precisely: a deliberate 1.4 GB `Mat` leak moved `Pointer.totalBytes()` by **zero bytes**. A dashboard built on it reads flat while the process doubles.

The second is the JVM's own Native Memory Tracking, which you reach with `-XX:NativeMemoryTracking=summary` and `jcmd <pid> VM.native_memory summary`. It is an excellent tool and it is equally blind here, for the same structural reason: NMT accounts for memory the JVM allocates — heap, metaspace, thread stacks, code cache, GC structures — and OpenCV's `fastMalloc` is none of those. A leaking pipeline shows an NMT summary that is entirely unremarkable.

What NMT is good for is the **subtraction**. Take its total committed away from RSS and the remainder is memory the JVM did not allocate — in a scalacv service, overwhelmingly pixels. A growing remainder puts the leak in your frame handling and nowhere in Java.

The one diagnostic flag the library actually has

Grep the sources for system properties and you find exactly one: `-Dscalacv.trackOwnership=true`, read once at class load in `Managed`. It emits no metric and it is not a production setting; it solves one specific and maddening problem.

The move semantics of `Image` make the commonest mistake a reuse of a handle some transform already consumed, and the resulting `IllegalStateException` fires at the **reuse**, which is almost never the interesting line. Turn the flag on and the exception carries, as its cause, the stack of the transform or terminal that actually spent the handle. It is off by default because turning it on allocates and fills in a `Throwable` on every spend — every `gray`, every `blur`, every `write` — which is the one thing you do not want at frame rate. Off, the spend path is a read of a `private val` that the JIT folds away and the field stays `null`, so a correct program pays nothing for the flag existing. Run it in the reproduction rather than the fleet — one pod with the flag set and the failing request steered to it, or a staging tier that mirrors production traffic.

Logging: almost nothing, once per frame

The instinct that has served you well in a request/response service is a disaster here, and it is worth writing out the wrong version because it is what everybody writes first.

Example 38-3. Wrong. At 30 fps this is 2,592,000 lines per camera per day.

```
camera.foreach() { frame =>
  val faces = frame.faces(detector)
  logger.info(s"frame processed: ${faces.size} faces,
scores=${faces.map(_.score)}")
  // …
}
```

Thirty structured lines a second per camera, each carrying a serialised sequence of floats; eight cameras on a box is 240 a second. The JSON encoding shows up in your frame budget, the log shipper's back-pressure shows up in your frame **drops**, and the ingestion bill arrives at the end of the month with more force than either. Logging at frame rate is its own outage, and the outage it causes looks exactly like the performance problem you were trying to diagnose.

The right version keeps the same information and pays for it once per second. Aggregate in the loop — it is one thread, so plain `var`s are correct and free — and emit on a boundary.

Example 38-4. Right. One line per second, with the shape of the whole second in it.

```
/** Single-threaded on purpose: it lives inside one frame loop, and the loop is
one thread. */
final class SecondSummary(source: String):
```

```
  private var windowStartMs = System.currentTimeMillis()
  private var frames = 0L
  private var detections = 0L
  private var minScore = Float.MaxValue
  private var slowFrames = 0L

  def observe(faces: Seq[Face], elapsedMs: Long, budgetMs: Long): Unit =
    frames += 1
    detections += faces.size
    faces.foreach(f => if f.score < minScore then minScore = f.score)
    if elapsedMs > budgetMs then slowFrames += 1
    val now = System.currentTimeMillis()
    if now - windowStartMs >= 1000 then
      // `minScore` is still MaxValue when the whole second saw no detection at
all.
      val worst = if minScore == Float.MaxValue then 0f else minScore
      logger.info(
        s"source=$source frames=$frames detections=$detections " +
          s"minScore=$worst slow=$slowFrames"
      )
      windowStartMs = now; frames = 0; detections = 0; slowFrames = 0
      minScore = Float.MaxValue
```

Two things do belong at frame granularity, and only two: a `Left` you did not expect, and a thrown exception. Both are rare by construction, and if either becomes common the volume is itself the signal. Everything else — counts, durations, scores — goes into metrics, where the aggregation happens in the backend rather than in your log pipeline.

> **WARNING**
> Do not alert on OpenCV's own stderr output. Several of those lines appear on paths scalacv **expects** to fail and handles correctly: the speculative open that a `CaptureOptions.withTimeout` triggers, which attaches the timeout parameters, lets several backends reject them outright, and then retries without them; the codec probe in `Recorder.open` that becomes a clean `Left(CvError.LoadFailed(...))`; and the `imread` warning accompanying every `Left(CvError.DecodeFailed(...))` from a bad upload. They are written by native code straight to file descriptor 2, so `System.setErr` does not even capture them — redirect fd 2 at the process level and keep the stream as context to read **after** an alert, never as the trigger.

Sampling: one frame in N, and the only debugger you get

Every production vision bug eventually reduces to one sentence: **I need to see what the camera saw**. You cannot attach a debugger to a frame that arrived nine hours ago, and no quantity of numbers will tell you that the lens has a smear on it. The technique that works is a small, bounded, **deterministic** sample of annotated frames — sampled on the frame counter, not on a random number. One frame in every 900 at 30 fps is one every thirty seconds, the volume is exactly predictable, and a problem reported at 14:32 is bracketed by two frames you can name.

Example 38-5. A sampler that scopes its image and encodes one JPEG every thirty seconds.

```
final class FrameSampler(everyN: Long, sink: (Long, Array[Byte]) => Unit):
  require(everyN > 0, s"everyN must be positive, was $everyN")
  private var seen = 0L

  /** Borrows `frame`; the caller keeps ownership and may go on using it. */
  def offer(frame: Image, faces: Seq[Face]): Unit =
    seen += 1
    if seen % everyN == 0 then
      val id   = seen
      val best = faces.map(_.score).maxOption.getOrElse(0f)
      // `.copy` first: drawRect and drawText are transforms, and would consume
the caller's handle.
      val boxed = faces.foldLeft(frame.copy) { (img, f) =>
        img.drawRect(f.box, Scalar.Green, Thickness.Stroke(2))
      }
      // `.bytes` is a terminal: it encodes, then releases the copy on every
path.
      boxed
        .drawText(s"frame $id, best score $best", Point(10, 30))
        .bytes(".jpg")
        .foreach(jpeg => sink(id, jpeg))
```

NATIVE MEMORY

The `.copy` is not defensive style, it is a correctness requirement. `drawRect` and `drawText` are transforms: they consume the handle they are given. Hand them the caller's `frame` and the caller's next line throws `IllegalStateException` — one frame in every 900, the worst possible frequency for a bug. The `.bytes` terminal then releases the copy on every path, so the sampler costs one extra frame on the sampled iteration and nothing on the other 899.

CAUTION

A sampled frame contains identifiable faces, licence plates, screens with data on them, and whatever was on the desk behind the subject. It is personal data the moment it lands in your bucket, and a debugging sample is the classic route by which personal data reaches a store nobody wrote a retention policy for. Before turning this on: set a lifecycle rule that deletes the objects (thirty days is generous), give the bucket its own access control rather than reusing the one your application logs go to, and ask whether the sample can be blurred first. Chapter 34 covers the background-blur path; the same operation applied to detected face boxes yields a frame that shows geometry, timing and lighting while showing nobody's face.

Tracing one frame from sensor to answer

Distributed tracing assumes a request identifier handed to you at an ingress. A frame loop has no ingress, so mint one: a monotonic counter beats a UUID here, because it is also the frame's position in the stream. Carry three fields — sequence number, capture timestamp in nanoseconds, source name — into every log line, every sampled object key and every span.

The pay-off is the join: a downstream consumer reporting a wrong answer reports the frame id, and that id names the sampled JPEG, names the log line carrying the per-stage timings, and gives you a timestamp to line up against the camera's own recording.

Example 38-6. Per-stage timing that keeps the frame id, so a bad output names its own input.

```
final case class StageTiming(stage: String, micros: Long)

final case class FrameTrace(id: Long, source: String, capturedNanos: Long,
                            stages: Vector[StageTiming]):
  def totalMicros: Long = stages.map(_.micros).sum

/** Times one stage and appends it to the trace. A `Left` is a result too, so it
keeps its timing;
  * a thrown stage is recorded by the same `finally` that made `observing`
honest. */
def stage[A](trace: FrameTrace, name: String)(body: => A): (FrameTrace, A) =
  val start   = System.nanoTime()
  var outcome = "threw"
  try
    val result  = body
    val elapsed = System.nanoTime() - start
    outcome = "ok"
    (trace.copy(stages = trace.stages :+ StageTiming(name, elapsed / 1000)),
result)
  finally record(name, System.nanoTime() - start, outcome)
```

Keep the stage names few and stable — `decode`, `preprocess`, `detect`, `annotate`, `encode` is a complete taxonomy for most pipelines — because they are metric labels as well as trace names, and a name derived from the data multiplies your time series without telling you anything. And record the capture timestamp when the frame arrives in your loop, not when you finish with it: the difference between the two, accumulated, **is** the lag Chapter 22 showed nothing in the capture API will report. A frame whose id is 40,000 and whose capture timestamp is four seconds old is the backlog, made observable.

Alert on the derivative, not on the level

The last decision is which of these numbers may wake a human, and the answer is fewer than you would like. Two rules cover most of it.

Do not alert on CPU. A vision service is supposed to be CPU-bound; OpenCV parallelises its heavy kernels across cores by design, and a box at 85 per cent CPU with every frame inside budget is doing its job well. CPU says nothing about whether the answers are right or timely, and a CPU alert trains everyone to ignore the pager.

Alert on slopes and ratios. Two earn a page: the dropped-frame rate — the divergence between frames in and frames processed, which measures the service failing at its actual job — and the slope of RSS over an hour of steady load, which is the only warning before the kernel kills the process.

Table 38-3. A starter alert set. Everything else is a dashboard panel, not a page.

Alert	Condition	Why this one
frames dropped	processed / in below 0.98 for 5 min	the service is not doing its job, and nothing throws
RSS slope	monotonic rise over 60 min of steady load	the only warning before exit code 137
not ready	`OpenCv.isLoaded` false past the grace period	a packaging fault; no restart fixes it
thrown outcomes	any rate above zero	these are bugs, not bad data — there is no acceptable rate
detections per frame	below the 7-day floor for 15 min	the silent failure no error rate can see
stage p99	above the frame budget for 10 min	the backlog, localised to one stage

The readiness alert is easy to get wrong. `OpenCv.load()` **throws** `CvError.NativesMissing` rather than returning it, and the message carries a copy-pasteable dependency line naming the platform you are actually on. That is a packaging failure, so report not-ready rather than not-alive: a liveness probe that kills and reschedules the container achieves nothing but a faster crash loop. Gate readiness on `OpenCv.isLoaded`, warm any DNN model in the same place before readiness flips true, and let the orchestrator route around the instance.

The RSS-slope alert is the one to tune carefully, since it is the one that pages legitimately at four in the morning. Do not alert on RSS crossing a threshold: a service that genuinely runs at 3.2 GB crosses any threshold you pick once and then never tells you anything again. Alert on the **sign of the trend** over a window long enough to average out a burst of large uploads. An hour of monotonic increase under a flat request rate is a leak, and an hour is still early enough to redeploy before the container dies.

Where this goes next

Nine signals, one flag, a sampler and six alerts, none of which the library provides and all of which are a few dozen lines: that is the shape of the dependency-free bargain.

What this chapter deliberately did not answer is what to **do** when one of those alerts fires. A red signal is only useful if the service has somewhere to degrade to — a codec it can fall back to, a capture it can reopen, a bundled cascade for when the model mirror is down, a request it can shed when native memory runs out before anything catches it. Chapter 39, *Degradation and Error Budgets*, is the ladder each of these signals climbs down, and the arithmetic for deciding how much of each failure is normal enough to leave alone.

CHAPTER 39

Degradation and Error Budgets

Deciding, in advance, what your pipeline does when it cannot keep up.

A web service that runs out of capacity returns 503s. That is unpleasant to watch on a dashboard, but it is honest: the failure is visible, it is counted, and everyone downstream learns about it in the same second it happens. A vision service does not have that decency. It has three failure stages, and only the last one looks like a failure.

First it gets slower. Chapter 22 priced that exactly: a 50 ms loop body against a 33.3 ms frame budget falls behind by 16.7 ms per frame, forever, and after a minute you are analysing a frame that is twenty seconds old. Every read succeeded. Every frame decoded. Nothing threw. Then it gets **wrong**, because a face detector twenty seconds behind the world is not a slow detector — it is a detector reporting, with full confidence and a plausible score, on a room that emptied while it was thinking. Then, eventually, it dies: native memory climbs past whatever ceiling exists, and the process goes away either with a `java.lang.OutOfMemoryError` nothing in this library catches or with a `SIGKILL` from the kernel that leaves exit code 137 and no stack trace at all.

The interval between "slower" and "dead" is where the decisions get made, and it is a bad time to be making them. The alternative is a **degradation ladder**: an ordered list of things the service will give up, written down before the incident, each rung with a trigger you can measure and a cost you have agreed to pay. Climbing down one is a choice; being pushed off the top is an outage. The repository has a page on this — `docs/mdoc/degradation-and-error-budgets.md` — and this chapter is that page expanded, because none of it is improvisable at three in the morning.

The second half of the discipline is an **error budget**: a number, agreed in advance, saying how much of a particular failure is normal. Without one, every dropped frame is either an emergency or invisible, depending on who is looking, and the two states alternate unpredictably. With one, "we dropped 0.4% of frames this hour against a 1% budget" is a sentence that ends a conversation.

Sort failures by what will catch them

Chapter 6 sorted scalacv's failures by what they **mean** — a value to handle, or a bug to fix. Operationally you need a different sort: which of them your code will ever see.

Table 39-1. The three tiers, by the thing that catches them. Tier 3 is the one that takes services down.

Tier	Arrives as	Caught by	Your move

1	`Either[CvError, A]`	an ordinary `match` at your boundary	branch, degrade, count it
2	thrown `IllegalArgumentException` or `IllegalStateException`	a `catch` you should not write	fix the call site
3	thrown `java.lang.Error`	nothing in scalacv	catch it yourself, at the outermost frame

Tier 1 is a closed set. `CvError` is sealed with exactly six cases, so an exhaustive match over it is also an exhaustive list of the labels your metrics can carry. Use one stable string per case for the log line, the metric label and the budget row alike, so nobody has to translate between three vocabularies at two in the morning.

Example 39-1. Six cases, six labels. `NativeCall` keeps its operation name; the others do not need one.

```
/** The label to attach to a metric or a log line for each failure the library
can return. */
def budgetLabel(e: CvError): String = e match
  case CvError.DecodeFailed(_, _)    => "decode_failed"      // bytes that are not
an image
  case CvError.EncodeFailed(_, _)    => "encode_failed"      // an unwritable
path, an unknown extension
  case CvError.LoadFailed(_, _)      => "load_failed"        // a model, cascade,
codec or capture source
  case CvError.CalibrationFailed(_) => "calibration_failed" // too few views, or
no convergence
  case CvError.NativesMissing(_, _) => "natives_missing"    // a build problem,
not a runtime one
  case CvError.NativeCall(op, _)     => s"native_call:$op"   // OpenCV rejected a
call; `op` names which
```

`NativeCall` is the one case whose label deserves a dimension: `native_call:cvtColor(BGR2GRAY)` tells you where to look, and a flat `native_call` rate tells you only that something is unhappy. Split it by operation before you alert on it. Failures spread evenly across operations are load; failures piled onto one operation are a shape or type bug that no amount of shedding will fix.

Tier 3 is the one that surprises people. `Cv.attempt` has exactly three `catch` clauses: `org.opencv.core.CvException`, a `CvError` passing through unchanged, and a guarded `case e: Exception if e.getClass == classOf[Exception]` matching a **bare** `java.lang.Exception` and nothing else — what OpenCV's `throwJavaException` degrades to for failures it cannot classify, `std::bad_alloc` included. There is no `case e: Throwable` and no `case e: Error`, so `OutOfMemoryError` and `UnsatisfiedLinkError` travel straight through every `flatMap` in your pipeline and out of the thread. The worker thread dies, the request never completes, and with an executor that swallows uncaught throwables nothing is logged. The pod stays ready and returns timeouts.

> **WARNING**
> The memory ceiling is tier 3. `-Dorg.bytedeco.javacpp.maxPhysicalBytes` is worth setting, but when it fires it throws `java.lang.OutOfMemoryError`, which extends `VirtualMachineError` extends `Error`: no `Either` in scalacv contains it. Set it comfortably **below** the container's own limit so JavaCPP reaches the limit before the kernel does — one path gives you a message and a stack trace, the other gives you exit code 137.

The ladder, rung by rung

A rung is only useful if it names three things: what it gives up, what triggers the climb down, and which number you are watching. Vague rungs ("reduce load") do not survive contact with an incident. Here is the ladder for a face-detection service on a live camera, cheapest sacrifice first.

Table 39-2. A degradation ladder. Each rung costs something specific and is triggered by a number you already emit.

Rung	What it gives up	Trigger	Signal that drives it
Half resolution	the smallest faces — a quarter of the pixels remain	lag ratio above 1.1 for one second	`LagMeter` ratio (Chapter 22)
Frame skip	temporal resolution: one frame in three	lag ratio still above 1.1	the same ratio
Motion gate	nothing, on a still scene; everything, on a busy one	lag ratio still above 1.1	the same ratio, plus `Motion.ratio`
Cheap detector	landmarks, accuracy, recall on angled faces	detector error rate, or the model unavailable	`Left(NativeCall)` rate, breaker state
Reduced output	image quality on the response	egress or encode latency	`bytes(".jpg")` timer
Shed load	new streams; existing ones continue	RSS above 85% of the ceiling	the RSS gauge (Chapter 38)
Refuse work	everything; the instance drains	a tier-3 catch fired	the health flag

The order is not arbitrary. Every rung above the cheap detector is **lossy but not misleading**: a half-resolution frame finds fewer faces, and the ones it finds are real. From the cheap detector down, the output changes character rather than volume — a much more serious thing to do quietly.

Half resolution is first because detection cost scales with pixels: halving each side quarters the pixel count, which for a detector dominated by the image it scans often approaches a 4× speed-up. The cost is exact — the smallest objects now occupy a quarter of the pixels they did, and below some size the detector stops seeing them. Write down what that size is for your camera before you ship the rung.

Frame skip buys throughput linearly and costs latency-to-detection linearly. It is second because a skipped frame is a blind interval, invisible in a way a missed small face is not. Note what it breaks: Chapter 21's frame-difference detector, optical flow and any tracker assuming small motion between frames all read a skipped frame as a large jump, the input they are least robust to. If you skip, make elapsed time a parameter of the algorithm rather than an assumption inside it.

The motion gate has the best ratio and the worst worst case. A `MotionDetector.frameDifference()` costs a greyscale conversion, a blur, an absolute difference, a threshold, a dilation and a contour pass; the project's `GrayBlurCloneBench` puts the blur at the front of that chain at 126.8 µs on an already-grey 1080p frame, and the rest of the chain is the same order of magnitude — which against a convolutional detector is nothing. On a corridor camera at night it removes essentially all the work. But it degrades to zero benefit exactly when you need it most: a busy scene sets `moving` on every frame, and you have added the gate's cost to a pipeline already over budget. Gate on motion because most frames are boring, not because you hope the busy ones will be.

A ladder you can read

Rungs written in a design document are aspirations. Rungs written as a type are a thing you can test, log and alert on. The enum below is the entire policy; the controller after it is the entire mechanism. Only the output-quality rung is missing, because it belongs at the response boundary rather than in the frame loop.

Example 39-2. The rungs as a type. The ordinal is the ladder, so climbing is arithmetic.

```
/** The rungs, best first. Climbing down always costs something; the comment says
what. */
enum Rung:
  case Full     // every frame, full resolution, the YuNet detector
  case Half     // every frame at half resolution — a quarter of the pixels
  case Sampled  // half resolution, one frame in three
  case Gated    // motion decides which frames reach the detector at all
  case Cheap    // the bundled Haar cascade: no landmarks, more false positives
  case Shedding // existing streams only; a new one gets a 503
  case Refusing // nothing is accepted; the instance is draining

/** The three signals the ladder reads. None of them comes from scalacv:
`lagRatio` is the
  * lag meter of Chapter 22, `nativeErrorRate` your own counter over
`Left(CvError.NativeCall)`,
  * `rssFraction` the resident-set gauge of Chapter 38 over your configured
ceiling.
  */
final case class Reading(lagRatio: Double, nativeErrorRate: Double, rssFraction:
Double)
```

The controller needs hysteresis in both directions, with different constants for each. Climbing down on a single bad reading turns a garbage-collection pause into a policy change; climbing back up quickly turns the ladder into an oscillator. Down after a second of consistent trouble, up after half a minute of consistent calm, is a reasonable starting pair for a 30 fps loop.

Example 39-3. The whole controller: one rung at a time, slow up, quick down.

```
final class Ladder(downAfter: Int = 30, upAfter: Int = 900):
  private val rungs = Rung.values
  private var index = 0
  private var over  = 0
  private var under = 0

  def current: Rung = rungs(index)

  /** Feed one reading per frame. Returns the rung to run the next frame at. */
  def observe(r: Reading): Rung =
    val struggling  = r.lagRatio > 1.1 || r.nativeErrorRate > 0.01 ||
r.rssFraction > 0.85
    val comfortable = r.lagRatio < 0.9 && r.nativeErrorRate == 0.0 &&
r.rssFraction < 0.6
    if struggling then
      over += 1
      under = 0
    else if comfortable then
      under += 1
      over = 0
    else
      over = 0
      under = 0
    if over >= downAfter && index < rungs.length - 1 then
      index += 1
      over = 0
    else if under >= upAfter && index > 0 then
      index -= 1
      under = 0
    current
```

Applying a rung to a frame is where Chapter 5's move semantics matter, and where the wrong version is easy to write. The frame `Camera.foreach` hands you is closed when the body returns; a query such as `faces` or `detect` borrows it, while `scale` **consumes** it and hands back a new `Image` that is yours. So this is wrong:

```
case Rung.Half =>
  frame.scale(0.5).faces(yunet).map(_.box)  // leaks the scaled frame, every
frame
```

`scale` returned an `Image` nobody closed. `foreach` will close the original — which `scale` already spent, so that release is a no-op — and the half-size copy leaks at the frame rate of your camera. Give it a name and a `finally`.

Example 39-4. One rung, one frame. Queries borrow, `scale` consumes, and the copy it makes is yours.

```
import org.opencv.objdetect.{CascadeClassifier, FaceDetectorYN}
import scalacv.*

def boxes(
    rung: Rung,
    frame: Image,
    n: Long,
    gate: MotionDetector,
    yunet: Managed[FaceDetectorYN],
    haar: CascadeClassifier
): Seq[Rect] =
  def scaleBack(r: Rect, f: Int) = Rect(r.x * f, r.y * f, r.width * f, r.height *
f)
  rung match
    case Rung.Full =>
      frame.faces(yunet).map(_.box) // `faces` borrows; there is nothing to close
    case Rung.Half =>
      val small = frame.scale(0.5) // consumes `frame`; `foreach`'s close becomes
a no-op
      try small.faces(yunet).map(face => scaleBack(face.box, 2))
      finally small.close() // `small` is ours, on every path
    case Rung.Sampled =>
      if n % 3 != 0 then Seq.empty else boxes(Rung.Half, frame, n, gate, yunet,
haar)
    case Rung.Gated =>
      // `detect` borrows the frame and returns plain data, so the frame is still
ours to scale.
      if !gate.detect(frame).moving then Seq.empty
      else boxes(Rung.Half, frame, n, gate, yunet, haar)
    case Rung.Cheap =>
      frame.detectHaar(haar) // a box and nothing else: no landmarks, no score
    case Rung.Shedding | Rung.Refusing =>
      Seq.empty
```

`detectHaar` takes the raw `CascadeClassifier` rather than the `Managed[CascadeClassifier]` that `Cascades.load` returns, because a defaulted overload for the `Managed` is not expressible alongside the `scaleFactor`, `minNeighbors` and `minSize` defaults. Keep the classifier inside its `use` block for the life of the loop — `Cascades.load(name).map(_.use(c => ...))` — so the spent-handle guard is still there.

> **NATIVE MEMORY**
> A rung change must not change who owns what. The version above closes `small` in a `finally` on every branch that creates one, and never closes `frame`, which belongs to `foreach`. A ladder that gets this wrong leaks only under load — only at the moment it was added to help — and the leak is a full frame per iteration: about 6 MB at 1080p, 180 MB per second at 30 fps.

Budgets: the numbers that make a rung acceptable

A rung is a promise to lose something; an error budget is the agreement saying how much of that loss is fine. Without the second, nobody can approve the first.

The distinction that matters most is between a frame you **dropped** and a frame you **got wrong**. A dropped frame is a decision: you know how many, you chose the policy, and the budget line writes itself. A wrong frame is an unbounded liability — it costs the same latency, produces a confident-looking answer, is indistinguishable from a right one in your logs, and the only thing that reveals it is a labelled ground truth you probably do not have in production. Budget the two at wildly different levels. One percent of frames dropped is a Tuesday; one percent silently served by a fallback detector for three weeks is an accuracy regression nobody declared.

Table 39-3. Budget lines, and the signal each one is computed from. Every signal is yours to emit: scalacv has no metrics of its own.

Budget line	Computed from	A start	When it burns
Frames dropped	`Sampled` and `Gated` skips over frames offered	under 1%	the pipeline is slower than the source
Frames unprocessed	`recordTo`'s `Right(n)` over `CaptureInfo.frameCount` (advisory)	under 1% per clip	shrink the transform or the resolution
Degraded detections	share served by the fallback rung	under 1%	accuracy is quietly worse than you think
Capture reopens	reopens per camera-hour	under 1	a flaky link or a contended device
Native-call errors	`Left(NativeCall)` rate, split by `operation`	near 0	one operation dominating is a bug, not load
Decode failures	`Left(DecodeFailed)` share of uploads	under 2%	a client bug — a 4xx, not a page
Model-fetch failures	`Left(LoadFailed)` per boot	0	a mirror outage or a checksum mismatch
OOM sheds	503s from the tier-3 `catch`	0	page immediately: an instance quarantined itself
RSS headroom	RSS over `maxPhysicalBytes`	under 70%	a steady climb under steady load is a leak

Two rows carry warnings. `CaptureInfo.frameCount` is a `CAP_PROP_*` query answered by a backend allowed to guess — a live camera commonly reports `0` — so it is a denominator to watch over time, never a loop bound. And keep OpenCV's stderr out of the budget: some of that output is the library working correctly. Alert on your own `CvError` rate instead.

The failure with no counter

Every budget line above is a division you can compute from numbers your service already emits. “Frames processed wrongly” is not, and pretending otherwise is the most common way a degradation design fails review.

Two things work. The first is a **held-out sample**: a few hundred labelled frames from your real cameras, replayed through the production configuration on every deploy, with recall and false-positive rate recorded per rung. That gives you the honest cost of each degradation, which is exactly the number you need to argue about where the cheap detector belongs in the ladder.

The second is a **shadow comparison**: on a small share of frames — one in a hundred is plenty — run both the current rung and the one above it, and record the disagreement rate. It costs a fixed fraction of your budget, needs no labels, turns “the fallback has been serving for three weeks” from a discovery into a graph, and catches what a held-out sample cannot: a scene that has drifted away from your labelled data.

Timeouts, and the thing you cannot have

Every other chapter about resilience would put a timeout on the expensive call here. You cannot, and it is worth being exact about why rather than discovering it under load.

A JNI call in flight cannot be interrupted. `Thread.interrupt()` sets a flag that a thread parked in OpenCV’s C++ never looks at, and there is no cancel and no timeout parameter on `read`. Chapter 37’s ZIO integration is the only path in the library that even attempts interruption — `frameStream` wraps its read in `ZIO.attemptBlockingInterrupt` — and its own documentation says plainly that this parks a blocked read on the blocking pool instead of the compute executor and does not buy cancellation.

A timeout on a native call is therefore a decision about the **caller**, never about the callee. You can stop waiting; the work continues, on a thread you no longer control, holding native buffers you can no longer see. Three things remain that you can actually do.

> NATIVE MEMORY
> Abandoning a native call does not abandon its memory. A future you gave up on is still executing, still holds every `Mat` its pipeline allocated, and — if it was written with `Managed` — releases them only when it finishes. Two abandoned 4K pipelines are hundreds of megabytes no gauge attributes to anything. Bound how many calls you are willing to abandon, or do not abandon them.

Bound the input so the call cannot take long. This is the substitute that works, because it makes duration a property of the data rather than a race. Cost scales with pixels, so a pixel gate at the boundary is a latency gate — and it is the cheapest check in the service, since a decoded `Image` answers `width` and `height` without touching a pixel.

Example 39-5. A pixel budget at the boundary. The only reliable bound on a native call's duration.

```
/** Twelve megapixels — about 34 MiB of BGR pixels, and a decode-plus-detect that
fits a budget. */
val MaxPixels = 4000L * 3000L

/** Admits an image or rejects it, closing it either way if it is not admitted.
*/
def admit(image: Image): Either[CvError, Image] =
  val pixels = image.width.toLong * image.height // queries borrow: nothing is
consumed here
  if pixels <= MaxPixels then Right(image)
  else
    image.close()
    Left(CvError.DecodeFailed("<upload>", s"$pixels pixels exceeds the $MaxPixels
this service accepts"))
```

Bound the source, where a backend will honour it. `CaptureOptions.withTimeout(5.seconds)` sets `CAP_PROP_OPEN_TIMEOUT_MSEC` and `CAP_PROP_READ_TIMEOUT_MSEC`, which FFmpeg and GStreamer honour for network sources and which V4L2, AVFoundation and the built-in MJPEG reader ignore entirely, with nothing in the API to say which you got. Set it for `rtsp://` and `http://` only.

Supervise at the process level. A watchdog that has seen no progress for N seconds, a readiness probe that goes false, and an orchestrator that drains and restarts the instance is the real timeout for work that cannot be cancelled. It is coarse, slow, and the only item here that always works.

Circuit-breaking a detector

A model that has started failing is worse than no model: each call costs its full latency and returns nothing. The point of a breaker is to stop paying that toll once the evidence is in, and to pay it again occasionally in case the situation has changed.

Example 39-6. A breaker with a cool-off. Every failure at or past the threshold restarts the clock.

```
/** Opens after `threshold` consecutive failures; lets one call through every
`coolOffMillis`
  * afterwards, so a recovered dependency is noticed without a flood of retries.
  */
final class Breaker(threshold: Int = 5, coolOffMillis: Long = 30_000L):
  private var consecutive = 0
  private var openedAt    = 0L

  def closed(nowMillis: Long): Boolean =
    consecutive < threshold || nowMillis - openedAt >= coolOffMillis

  def record(failed: Boolean, nowMillis: Long): Unit =
    if !failed then consecutive = 0
    else
```

```
      consecutive += 1
      if consecutive >= threshold then openedAt = nowMillis

/** The primary detector while the breaker is closed, the bundled cascade while
it is open.
  * The first element of the pair is the tag that must travel with the result.
  */
def facesOrFallback(
    image: Image,
    yunet: Managed[FaceDetectorYN],
    haar: CascadeClassifier,
    breaker: Breaker,
    nowMillis: Long
): (String, Seq[Rect]) =
  if !breaker.closed(nowMillis) then ("degraded", image.detectHaar(haar))
  else
    val attempt = Cv.attempt("FaceDetectorYN.detect")
(image.faces(yunet).map(_.box))
    breaker.record(attempt.isLeft, nowMillis)
    attempt.fold(_ => ("degraded", image.detectHaar(haar)), found => ("primary",
found))
```

The fallback deserves the honesty the model ladder in Chapter 23 demands. YuNet is a 232 kB convolutional network that is both more accurate and faster than the Haar cascades, and it returns five facial landmarks per face — eyes, nose tip, mouth corners — that the cascade cannot give you at all. `FrontalFaceAlt` is the best of the shipped cascades and still produces visibly more false positives, finds fewer faces at an angle or in poor light, and returns a bounding box and nothing else. A degraded detection is not interchangeable with a primary one: the `"degraded"` tag must travel with the result to whatever consumes it, and any landmark-dependent feature — alignment, the recognition of Chapter 25, an AR overlay — must be **disabled** rather than fed a guess.

> **CAUTION**
> The cascade fallback does not exist on Windows. The `windows-x86_64` bytedeco jar ships an empty `share/` directory and no cascade XML, so `Cascades.load` returns `Left(CvError.LoadFailed)` there and says so. On Windows the bottom rung is not "no network needed", it is "no detector". Ship `haarcascade_frontalface_alt.xml` with your application and load it with `Cascades.loadFrom(path)`, which takes a filesystem path and is otherwise identical.

Bulkheads

A bulkhead stops one bad stream from sinking the service. Here the resource that needs partitioning is not threads but native memory — the one resource with no per-request accounting anywhere in the JVM.

Three partitions are worth building. Give each stream a fixed cap on live `Image` values, enforced by refusing to start the next frame rather than by queueing it, so a camera that renegotiates to 4K mid-stream cannot claim four times its share. Give each stream its own

`Ladder` — deliberately a per-stream object with per-stream counters — so the rung a struggling RTSP feed has fallen to does not degrade the six healthy cameras beside it. And give each stream its own capture thread, which Chapter 36′s ownership rule requires anyway: a `VideoCapture` has exactly one owner because a decode in progress is not reentrant, and closing it from another thread mid-decode is undefined behaviour in C++ — a crash rather than an exception.

What you cannot partition is the resident set. Chapter 38′s RSS gauge is a whole-process number, so an instance serving twelve streams reports 85% of its ceiling without saying which stream took the memory. The answer is arithmetic rather than instrumentation: multiply the per-stream frame cap by the frame size and the stream count, and if that product is anywhere near the ceiling, run fewer streams per instance. Process-per-stream is a legitimate bulkhead, and where native memory is invisible to the collector it is often the honest one.

When the process dies anyway

Some failures are not degradable. A `SIGSEGV` takes the process with it: no stack trace, no `catch`, nothing in the log but a truncated line. A supervisor and a fast restart are therefore part of the design, not an admission of defeat — and the design is yours: state outside the process, so a restart costs a warm-up rather than data; a `~/.javacpp` cache already populated, so the restart does not pay the roughly 196 MB extraction again; models warmed before readiness flips true.

This is why `Managed` exists at all. It releases exactly once and throws `IllegalStateException` on use-after-release — in Scala, before anything reaches JNI, where the same mistake is a segmentation fault with no stack trace. Of the 188 `org.opencv.*` types that own native memory, exactly three expose a public `release()`; scalacv frees the rest anyway. Chapter 5′s ownership model is a machine for turning the crash you cannot supervise into an exception you can, and the supervisor covers what it cannot reach: a bug in OpenCV, a corrupt frame from a driver, a native library mismatch.

> **TIP**
> Distinguish a restart that helps from one that cannot. `CvError.NativesMissing` is a packaging problem no restart fixes: report it as **not ready** rather than **not alive**, or a liveness probe turns it into a crash loop. A tier-3 shed is the opposite — that instance has a leak or a workload it cannot serve, and should drain rather than limp.

The last rung runs after everything else has failed: catch `OutOfMemoryError` at the outermost frame you own, shed the request with a 503, and clear the health flag so the instance stops taking new work instead of failing each request in turn.

Example 39-7. The bottom of the ladder: shed one request, quarantine the instance.

```
import java.util.concurrent.atomic.AtomicBoolean
import scalacv.*

/** Flipped to false once this instance has shed. The readiness probe reads it.
*/
val healthy = AtomicBoolean(true)

def handleUpload(upload: Array[Byte]): (Int, String) =
  try
    Image.decode(upload).flatMap(admit).flatMap(_.gray.canny(80,
160).bytes(".jpg")) match
      case Right(jpeg)                   => (200, s"${jpeg.length} bytes")
      case Left(e: CvError.DecodeFailed) => (400, budgetLabel(e)) // the caller's
problem
      case Left(e)                       => (500, budgetLabel(e))
  catch
    case e: OutOfMemoryError =>
      // Tier 3. Nothing below this line caught it, and nothing was going to.
      healthy.set(false)
      (503, s"shed: ${e.getMessage}")
```

Catch `OutOfMemoryError`, not `Throwable`, which would also swallow tier 2 — the programmer errors you want to see — and turn a bug into a quiet 503. Do not retry inside the handler: the process is at its ceiling, and a retry allocates again and fails again, faster. Gate readiness on `OpenCv.isLoaded && healthy.get()`, so the shed is visible to the thing that can act on it.

What comes next

The ladder in this chapter is a set of claims: that half resolution costs you faces below a particular size, that the motion gate pays for itself on your cameras, that the cheap detector is acceptable for a bounded share of frames, that the breaker opens when you think it does. Every one is testable, and an untested ladder is a second failure mode wearing the costume of a mitigation — the code that runs least often, on the worst day, is the code you have exercised least.

Chapter 40, *Testing*, is where those claims become assertions: the tolerance metrics that compare a degraded output against a reference without demanding bit-equality, the property-based suites that hold the ownership rules `boxes` depends on, and the resident-set assertion that turns "no leak under load" from an intention into a build gate.

CHAPTER 40

Testing

Deterministic assertions over approximate results, on a runner with no display and no photographs.

The test you want to write for a blur is "the output looks blurred", and there is no assertion for that. So you write something else, and the something else is usually wrong in one of three ways that ordinary test suites never have to think about.

The first is that images are big. The instinct is to commit a photograph and assert on what a pipeline does to it, and now the repository carries a binary blob forever, in every clone, under a licence nobody read. Worse, the blob hides what the test depends on: when the assertion breaks, nothing in the diff says whether it was the code or the picture, because the picture is not in the diff at all. This library ships no image asset — the rule in `CONTRIBUTING.md` is "no image assets", and `Lena.png` was deleted to honour it.

The second is that results are approximate. OpenCV dispatches on the widest SIMD instruction set the CPU offers, so the same `gaussianBlur` runs different code on AVX-512 and on Apple silicon and the last bit of a pixel does not always agree. A test that demands bit-equality passes on the machine that recorded the expectation and fails, by one imperceptible unit, everywhere else — and that is worse than no test, because it teaches a team that red is normal.

The third is native memory. A leaking test suite fails no assertion; it goes green, then greener, and then the runner is killed by the kernel's OOM reaper on an unrelated job twenty minutes later with no stack trace. Every ordinary assertion is blind to it, because the leak is off-heap and the heap is fine. If you want a leak to fail a build, you have to go and measure the leak.

The repository answers all three with generated fixtures, tolerance-based assertions, and native RSS canaries. Unit suites belong to `core.test`, `vision.test`, or `graphs.test`; cross-module integration, ownership/property tests, and the API/POM gates stay in `core.test`. The leak workloads run separately in `leaks.test`, and no image assets are committed.

Run the module that owns the suite

The complete local test command names every module. Run it from the repository root:

Example 40-1. A sequential full run, including native RSS canaries.

```
./mill --no-server -j 1 core.test + vision.test + graphs.test + zio.test +
examples.test + leaks.test
```

The `+` separators are not decoration: `test` is a Mill command, so space-separated targets can become test-name filters rather than separate module runs. `-j 1` keeps the full native/RSS run sequential; `--no-server` avoids sharing a long-lived Mill worker with another build.

Example 40-2. Select a suite through its owning module, not its package name.

```
./mill --no-server -j 1 core.test.testOnly scalacv.FrameSourceSafetyTest
./mill --no-server -j 1 vision.test.testOnly scalacv.vision.RigidPoseInteropTest
./mill --no-server -j 1 graphs.test.testOnly scalacv.RendererContractTest
```

A suite in package `scalacv` may live in the vision or graphics module. Package names were preserved when those suites moved; running only `core.test` no longer runs their unit tests.

Draw the fixture, do not ship it

A drawn fixture answers three questions a committed PNG cannot: what is in it (the code says so), why the test cares (the assertion targets a shape you placed), and whether it changed (a source diff, not a binary blob). `Image.blank` plus the drawing primitives from Chapter 14 is the whole toolkit.

Example 40-3. A deterministic scene: every shape is at a coordinate the assertion can name.

```
private def scene(): Image =
  Image
    .blank(160, 120, Scalar(20, 40, 60))
    .drawRect(Rect(20, 20, 60, 40), Scalar.Green, Thickness.Filled)
    .drawCircle(Point(120, 80), 20, Scalar.Red, Thickness.Filled)
```

`Image.blank(width, height, color = Scalar.Black, channels = 3)` gives you a canvas of exactly the shape you asked for, and each `draw*` call consumes its receiver and hands back the next one, so the chain holds one live `Mat` from start to finish. The circle is filled with `Scalar.Red`, which is `(0, 0, 255)` in BGR order, so `img.mat.get(80, 120)` must come back with 255 in element 2. That assertion depends on what you drew and not on how OpenCV rounds, which is why it is stable on every runner in the matrix.

When a fixture wants to look busy — texture, speckle, something for a lossy codec to lose — seed the randomness and it stays reproducible. `ToleranceMetricTest` builds its scene from `scala.util.Random(11)` and twelve circles; `PropertyTest` takes the seed as a generated parameter, so a shrunk counterexample is replayable. Random-looking is not the same as random, and only the first belongs in a test.

The examples module keeps its fixtures in `examples/src/scalacv/Fixtures.scala`, at the lower `Managed[Mat]` level because the examples work there: `Fixtures.shapes(size)` for hard geometric edges, `Fixtures.qrCode(payload, scale)`, which runs `QRCodeEncoder` and scales past the detector's resolution floor, and `Fixtures.arucoMarker(id, sizePx)`, which generates a tag and then pads it, because `Aruco.generateMarker` emits no quiet zone and the detector hunts

for a dark quad on a light ground. Without the margin the marker is undetectable and the fixture tests nothing. That padding is the case for drawn fixtures in miniature: a line of code with a comment saying why it exists, where in a committed PNG it would have been an invisible property of a file that the next regeneration silently cropped away.

> **NATIVE MEMORY**
> A fixture is an allocation like any other, and the shape that goes wrong is a fixture used only by queries: `val img = scene(); assertEquals(img.width, 160)` spends nothing, so nothing is freed — once per test, several hundred times across a suite. Either finish with a terminal, or `try … finally img.close()`, which is what `ToleranceMetricTest` does around every comparison.

Compare with a tolerance, never byte for byte

Here is the assertion readers write first, and it is wrong:

Example 40-4. Bit-exact equality against a recorded hash. Green on your laptop, red on arm64.

```
test("blur is stable"):
  val out = scene().gray.blur(2)
  assertEquals(PixelHash.of(out), 0x3f7c1a09bb42e5d1L)   // recorded on one
machine
  out.close()
```

That constant is a fact about a CPU, an OpenCV build and a vectoriser, and this project runs its suite on `ubuntu-latest` x86-64 across three JDKs *and* on `macos-26` arm64. Compare within a threshold instead; two comparators cover nearly everything, and `ToleranceMetricTest` defines both as one-liners:

Example 40-5. The two tolerance metrics. Both borrow their arguments rather than consuming them.

```
/** Peak signal-to-noise ratio in dB — higher is closer. */
private def psnr(a: Image, b: Image): Double = Core.PSNR(a.mat, b.mat)

/** Largest absolute per-element difference across all channels — 0 is pixel-
identical. */
private def maxAbsDiff(a: Image, b: Image): Double = Core.norm(a.mat, b.mat,
Core.NORM_INF)
```

Both take `a.mat`, the borrowing accessor from Chapter 4, so a comparison leaves both images alive for the next assertion and for the `finally` that closes them.

Table 40-1. Choosing a comparator. The third row is a local convenience, not a CI gate.

Comparator	Measures	Assert	Use it for

`maxAbsDiff`	largest single-element error	`== 0.0`	PNG round-trips, flips, quarter-turns, crops — anything lossless
`psnr`	overall similarity, log scale	`> 30.0`	JPEG and WebP round-trips, blurs, filters — anything lossy or SIMD-sensitive
pixel hash	did anything at all change	equality	a fast same-platform regression key; never across an OS matrix

The thresholds are not folklore; `ToleranceMetricTest` pins each with a test that fails if the metric stops behaving. A JPEG round-trip is asserted both to differ from its source and to clear 30 dB, so it fails if the codec becomes lossless *and* if it becomes bad. And `gamma(1.0)` — an identity in arithmetic that nonetheless routes through an integer lookup table — is asserted to differ by at most 1, which is the case for tolerance in one line: an exact operation a bit-exact check would reject.

When you do want exactness, fold the pixels the same way everywhere. The test module has one hash, `PixelHash.of`, an FNV-1a over the raw bytes read *one row at a time* — an OpenCV `Mat` can be a view into a larger buffer, whose rows are not adjacent in memory, so a whole-buffer read would fold in padding that is not part of the image.

Golden images, and the trap in them

A golden image earns its place when correctness is genuinely visual — a chart with axis labels, a scene graph composited with alpha, an annotated frame where you care that the text did not move three pixels. If you keep them: small, because they live in your history forever; PNG, because lossless storage adds nothing to the comparison; and generated by a committed script, because "run this and review the diff" is a reviewable action where "someone dragged a file in" is not.

The trap is that a golden image records bytes and you care about a picture. An OpenCV point release, a libpng bump, a changed JPEG quality default or a new SIMD path on the runner all move bytes without moving anything a human could see. The suite goes red for something that is not a defect, someone regenerates to make it green, and the regeneration absorbs a real regression sitting in the same diff. Two things prevent that: compare goldens with `psnr` rather than equality, and pin the OpenCV version — which this build does in one place, `Deps.opencv` in `build.mill`, at `4.13.0-1.5.13`.

The golden this repository does keep

There is no golden image here, but there are three golden *files*: `core/api.golden`, `vision/api.golden` and `graphs/api.golden`, together 3,522 lines. Each is the compiled public

surface of one published module, rendered by `PublicApiTest` as sorted text read back out of the class files — JVM-public, not source-public, because `private[scalacv]` compiles to `ACC_PUBLIC` and really is callable from Java.

It cannot suffer the trap above, because every list is sorted and every compiler-invented name is excluded: `$anonfun` lambda bodies, `$lzy` backing fields, the `OFFSET$_m_N` offsets whose numbering shifts when an unrelated lazy val appears, the per-`enum` `$$anon$N` classes, and the static forwarders Scala emits so Java can call an `object`. What is left changes only when the API changes. Regenerate with `SCALACV_API_REGENERATE=1` — an environment variable, because Mill 1.1.7 offers no way to add a `-D` to a test fork from the command line — and `git diff` is the review.

Properties, where the law is the point

Example-based tests check the cases you thought of; the regressions that happen live at the edges you did not, an odd width or a fourth channel. `PropertyTest` extends `munit.ScalaCheckSuite` (`org.scalameta::munit-scalacheck:1.3.1`, added to `core.test` on top of the `munit:1.3.6` every test module already carries) and states laws instead.

Example 40-6. An identity law over generated sizes and channel counts.

```
property("rotate 90° clockwise then 90° counter-clockwise is the identity"):
  forAll(genDims, genChannels, Gen.choose(0, 1_000_000)): (dims, ch, seed) =>
    val (w, h) = dims
    val src = build(w, h, ch, seed)
    val back =
src.copy.rotate(Rotation.Clockwise).rotate(Rotation.CounterClockwise)
    try samePixels(src, back)
    finally
      src.close()
      back.close()
```

Three details are worth copying. `src.copy` is there because `rotate` consumes its receiver and the law still needs `src` to compare against; without it the comparison throws `IllegalStateException` on a spent handle instead of failing an assertion. The `finally` matters more here than in an example test, because ScalaCheck runs the body once per generated case, so a missing `close()` is not one leak but twenty-five. And `samePixels` compares width, height, channel count *and* `PixelHash.of` — a law checking only dimensions would pass for an operation that returned a black rectangle. The generators stay deliberately small (`Gen.choose(4, 40)` per dimension, `Gen.oneOf(1, 3, 4)` for channels, `withMinSuccessfulTests(25)` rather than the default 100), because every case crosses JNI and allocates native memory.

Table 40-2. The laws PropertyTest pins, and what each one would catch.

Property	Regression it catches
`imencode` / `imdecode` PNG round-trip is lossless	an encoder default that silently quantises
double horizontal flip is the identity	an off-by-one on odd widths
rotate clockwise then counter-clockwise is the identity	a transposed-but-not-flipped rotation
`BgrToRgb` then `RgbToBgr` is the identity	a channel swap applied twice, or not at all
resize up then down preserves dimensions	rounding that loses a row at odd sizes
`canny` always yields one channel	an output type that changes with the input

Testing lifetimes

This section has no equivalent in a normal Scala project, and it splits in two: proving a particular handle was released, and proving memory does not accumulate at scale. The first is a deterministic assertion; the second cannot be.

The deterministic half rides on the guard from Chapter 5. Because `Managed` turns a use-after-move into an `IllegalStateException` on the Scala side — before anything reaches JNI, where the same mistake is a SIGSEGV with no Java stack trace — the ownership contract is directly assertable. `ImageTest` does that four times:

Example 40-7. The move semantics, asserted rather than documented.

```
test("a transform spends the image it was called on"):
  val img = sample()
  val g = img.gray
  intercept[IllegalStateException](img.width)   // the source is spent
  assertEquals(g.channels, 1)
  g.close()

test("a query borrows: the image is still usable afterwards"):
  val img = sample()
  assertEquals(img.qrCodes, Seq.empty)          // there is no QR code in the
scene
  assertEquals(img.width, Width)                // still alive after the query
  img.close()
```

The subtler case in that suite is `blur(0)`. A radius of zero is the identity, and the tempting implementation returns the receiver untouched — silently breaking the contract for that one argument. The test asserts `img` is spent *even when* `r == 0`, and that the result still reports the original dimensions: the handle moved, the pixels did not.

When such a test fails, the exception fires at the reuse, which is rarely the interesting line. Start the JVM with `-Dscalacv.trackOwnership=true` and `Managed` attaches, as the cause, a `Throwable` captured where the handle was spent. It is off by default because it allocates that `Throwable` on every consumption. Mill cannot push a `-D` into a test fork from the command line, so turn it on by adding it to that module's `forkArgs`.

A test with no assertion in it

`DoubleFreeTest` allocates and releases 300 `CascadeClassifier` s, 300 `QRCodeDetector` s, 300 `ArucoDetector` s and 300 `Net` s, calling `System.gc()` every fiftieth iteration, and then asserts nothing at all.

It is a crash canary. The 185 `org.opencv.*` types with no public `release()` each carry an unconditional `protected void finalize() { delete(this.nativeObj); }` . Free one through `Releasable.nativeHandle` , then drop it, and `delete` runs twice on the same address: once from you, once from the finalizer thread. `Managed` 's compare-and-set is no defence — it makes *your* release idempotent and knows nothing about another thread's finalizer. The result is not a failed assertion but a corrupted heap that takes the JVM down somewhere else entirely, which is how the bug was found in the first place, in an unrelated suite. Reaching the end of the loop alive *is* the pass; if the guard regresses, the worker dies without writing a result and the run goes red anyway.

One test there does assert, and it makes the rest diagnosable: after `Managed(c).release()` , `c.getNativeObjAddr` must be `0L` . Zeroing the field is exactly what makes the inherited finalizer delete a null pointer instead of a live one.

The leaks module

A leaked `Mat` fails no assertion, so you have to gate memory directly — and measure the right number, which here is not the obvious one. The memory audit put a deliberate 1.4 GB `org.opencv.core.Mat` leak in front of JavaCPP's `Pointer.totalBytes()` and it moved by zero bytes, because those buffers come from OpenCV's own JNI via `cv::fastMalloc` rather than through JavaCPP's tracked allocators. The `-Dorg.bytedeco.javacpp.maxBytes` budget derived from that counter is equally blind. The only signal that sees these buffers is process resident set size, which is the same conclusion Chapter 35 reaches from the other direction.

`leaks/test/src/scalacv/LeakAssertions.scala` reads it from `/proc/self/statm` — field 2, resident pages, times a page size that defaults to 4096 and is overridable through `SCALACV_PAGE_BYTES` — and falls back to `Pointer.physicalBytes()` , itself RSS-based, where that file does not exist.

Example 40-8. The whole leak gate, in one function.

```
def assertBounded(name: String, n: Int = 300, warmup: Int = 40, toleranceMB: Long
= 48)(
    workload: () => Unit
): Unit =
  var i = 0
  while i < warmup do
    workload(); i += 1
  settle()
  val before = rssBytes()
  i = 0
```

```
  while i < n do
    workload(); i += 1
  settle()
  val grewMB = (rssBytes() - before) / (1024L * 1024L)
  assert(grewMB <= toleranceMB, s"$name: RSS grew ${grewMB}MB over $n iterations
…")
```

Every part of that is load-bearing. The `warmup` runs amortise one-time costs — native library extraction, JIT compilation, the allocator's arenas reaching their working size — into the baseline, so the measurement sees growth and not start-up. `settle()` is `System.gc()`, a 150 ms sleep, `Pointer.deallocateReferences()`, another `System.gc()` and another sleep, run identically before both readings. And the bound is a ceiling rather than zero, because RSS never returns exactly to baseline: arenas do not shrink and the code cache only grows. A per-iteration leak of even a modest `Mat` clears 48 MB long before 300 iterations; a leak-free workload stays flat.

Table 40-3. What the four leak tests drive, and what a regression would cost per call.

Workload	What a regression leaks
`blank → gray → blur(2) → canny(80, 160) → bytes(".png")` at 640×480	one intermediate per stage; the pipeline should hold exactly one live `Mat`
`blurBackground` with a deliberately size-mismatched mask, 200 iterations	a 320×240×3 receiver, about 230 KB per failed call — roughly 46 MB over the run
`undistort` with a fresh `Intrinsics` per call, 300 iterations	the camera matrix and distortion coefficients, allocated per call
`ZIO.scoped(acquireRelease(Mat(...)))`, 200 iterations	a 640×480×3 native buffer per scope if the finalizer stops releasing it

The second row is the shape leaks actually take: the happy path was never the problem, the `catch` was. An error branch that forgets to release is invisible to every functional test in the suite — the exception is thrown, the assertion about the exception passes, and 230 KB goes missing each time.

The suite runs with `./mill leaks.test`. RSS is process-global, so its single suite owns a JVM and its workloads run sequentially. The functional MUnit suites also receive one fork per suite, with the build rejecting empty discovery. Separate forks prevent a native crash from erasing another suite's results; they do not prevent concurrent processes from contending for host memory, which is why the full local command uses `-j 1`.

CI runs a coarser version of the same idea in its `leak-check` job. After `leaks.test` it compiles a plain-Java driver against `examples.runClasspath`, runs `Fixtures.shapes(240) → gray → blur(1) → canny(60, 160) → bytes(".png")` 2000 times in one long-lived JVM under `-Xmx256m`, reads `VmRSS` from `/proc/self/status` after a 200-iteration warm-up and again at the end, and fails if the delta clears 204,800 KB. The heap cap is the

point of the pairing: an on-heap leak OOMs and fails there, while a native `Mat` leak escapes the heap and only the RSS delta sees it. At about 170 KB per leaked 240×240×3 frame a regression grows roughly 300 MB across the measured run — far past a 200 MB ceiling that steady-state jitter never approaches.

> **NATIVE MEMORY**
> If you copy `assertBounded` into your own project, copy the isolation with it. Two mistakes make the gate useless without making it fail: running it alongside other suites, and running it with parallelism on inside its own module. Both add allocations the workload did not make, and both push you toward "raise the tolerance until it stops flapping" — a bound no leak can reach.

Isolation: one load, one JVM, one scope

`OpenCv.load()` is idempotent but not free: the first call extracts classifier libraries into the JavaCPP cache and loads them into the process. Native suites put it in `beforeAll`, once per suite in that suite's forked JVM. Pure geometry and renderer contracts do not need to load natives.

Test modules fork by default in Mill, and `ScalacvTests` sets `forkArgs` to `Deps.headlessJvmArgs(jvmMajor())`. That adds `-Djava.awt.headless=true`, which is an assertion rather than a preference: a GUI-toolkit regression then fails loudly in CI instead of silently opening a window on whichever developer machine has a display. It also adds `--enable-native-access=ALL-UNNAMED` and `--sun-misc-unsafe-memory-access=allow`, but only on JDK 24 and later, because both flags are younger than the JDK 17 floor and passing either to a 17 or 21 JVM is not a warning but `Unrecognized option` and a process that does not start.

Within a test, scope resources the way you would in production. `Managed.scope` releases in reverse acquisition order on both the normal and the throwing path, and `ManagedTest` asserts both, including the case that motivated it: a later allocation throws, and everything acquired before it must still lose its buffer. For files, `Files.createTempFile` with a `finally` that deletes — there is no fixture directory, because no fixture file ships.

What this project's CI actually runs

There is no `apt-get` line anywhere in `.github/workflows/` and no GUI package. `OpenCv.load()` resolves what it needs out of the classifier jars, so a stock GitHub runner is a sufficient environment — the claim the smoke job exists to keep honest.

Table 40-4. The six CI jobs and the failure each one exists to catch.

Job	What it gates
`build` (JDK 17, 21, 25 on `ubuntu-latest`)	compile, `core.test + vision.test + graphs.test + zio.test + examples.test`, `docs.mdocCheck`, and the headless natives smoke

`leak-check`	`leaks.test`, then a 2000-iteration external stress driver gated on RSS growth
`style`	`scalafmt checkFormatAll` and `scalafix --check` on every published module plus `examples`
`natives` (`macos-26`)	the same smoke and `core.test + vision.test + graphs.test + zio.test + examples.test` on Apple-silicon arm64
`publish-shape`	that exactly `core graphs vision zio` are publishable, and `PublishedPomTest`
`consumer-smoke`	a from-nothing resolve of the four published coordinates on a clean cache

Two of those catch failures that are invisible from inside the build.

The matrix carries a step titled "Assert the build really ran on JDK 17" (and 21, and 25). Mill 1.1.7 ignores `JAVA_HOME` and `MILL_JVM_VERSION` when it runs modules, so `actions/setup-java` cannot steer it and is deliberately absent from the matrix — `leak-check` is the one job that does use it, and only to put a plain `javac` on `PATH` for the stress driver, never to choose Mill's JDK. The per-rung JDK comes from `MILL_JVM_ID`, which `build.mill` reads. If that override stopped being honoured, all three rungs would quietly run the same JDK and all three would still go green. So the job runs `./mill examples.runMain scalacv.jvmReport` and greps for `java.version=17`. Without that step the matrix is decorative.

The `consumer-smoke` job publishes all four modules to the local ivy repository, points `COURSIER_CACHE` at a fresh temporary directory, and `cs fetch`es the four published coordinates plus the two documented natives ones. It then compiles and runs `ci/consumer-smoke/ConsumerSmoke.java` — Java on purpose, since it needs no Scala compiler and calling `scalacv.OpenCv.load()` through the generated static forwarder proves the published bytecode is callable from plain JVM code. It loads the natives, allocates an 8×8 `Mat` across JNI (`Core.VERSION` is a constant and would prove nothing), and references `scalacv.QrCode.class` and `scalacv.Color.class` so a failure to resolve the vision or graphs artifact surfaces here rather than in a user's project. This is the gate that catches "green build, zero natives shipped".

> **WARNING**
> The smoke, leak and consumer jobs all gate on a success line — `OK`, `LEAK-OK`, `CONSUMER-OK` — rather than on the exit code. The reason will bite you too: OpenCV and OpenBLAS teardown at JVM shutdown occasionally exits non-zero after a completely successful run. A genuine load failure prints no success line, so the gate still fails loudly.

Snippets that cannot rot

Prose can lie about an API; a compiled snippet cannot. The `docs` Mill module runs mdoc as a plain subprocess over `docs/mdoc/`, compiling every fenced ```` ```scala mdoc ```` block against the *runtime* classpath of `core`, `vision`, `graphs`, `zio` and `examples` — natives included, because a snippet that calls OpenCV must link the JNI and not merely the Java API. `./mill docs.mdoc` splices each block's evaluated output back into the markdown; `./mill docs.mdocCheck` passes `--check` instead, so a snippet that no longer compiles fails the build rather than being quietly rewritten. That check runs on every pull request in the `build` job, which is what turns the documentation from a courtesy into a gate.

The modifiers earn their keep. Of the 993 `scala mdoc` blocks across `docs/mdoc/`, 311 carry no modifier and print their evaluated result into the page. The remaining 682 are shaped: 403 `mdoc:silent` (compile and run, show nothing), 195 `mdoc:compile-only` (type-check without running — the right choice for anything that would open a camera), 57 `mdoc:invisible` (setup the reader does not need to see) and 27 `mdoc:crash`, which assert that a snippet *does* throw. That last one is how the use-after-move example on the testing page stays true: if `Managed` stopped throwing on a spent handle, the documentation build would fail because the promised crash did not happen.

A book applies that discipline by hand: these listings are checked against the source, not compiled by a build. Where you want certainty, the equivalent page under `docs/mdoc/` is the copy a compiler has seen.

Skip what the runner cannot provide

A test that needs hardware the environment may not have should step aside, not go red. munit's `assume` marks a test skipped rather than failed, and the suite uses it in three shapes:

Example 40-9. Opt-in hardware, opt-in models, and a platform that ships no data.

```
// A webcam, if one is attached and you asked for it: SCALACV_CAMERA=1 ./mill
core.test
assume(sys.env.contains("SCALACV_CAMERA"), "set SCALACV_CAMERA=1 to run camera
tests")

// A model file the suite will not download: SCALACV_SFACE_MODEL=/path/to/
face_recognition_sface.onnx
assume(model.isDefined, "set SCALACV_SFACE_MODEL to the SFace ONNX to run this
test")

// The windows-x86_64 classifier jar ships an empty share/, so no Haar cascade
can be resolved.
assume(cascadesShipped, "this platform ships no Haar cascades")
```

The camera gate protects developers as much as CI: an unguarded `Video.open(0)` on a laptop opens whatever webcam is attached, which is a surprising thing for the headless CI test command to do. Note the second `assume` inside that test, on the `Left` branch of

`Video.open(0)` — opting in does not guarantee a device is there, so a missing camera skips rather than fails even then.

`CascadesTest` shows the shape worth imitating: alongside seven tests gated on `cascadesShipped` it has one gated on `assume(!cascadesShipped)`, asserting that `Cascades.resolve` fails cleanly where the data is absent. Every branch is covered somewhere; nothing is merely skipped everywhere.

Where this leaves you

One question sits underneath all of it, and `CONTRIBUTING.md` states it as a rule: an assertion that would still pass if the implementation returned a constant, or did nothing, is a false assurance and worse than no test. Ask it of the fixture (would this scene tell a blur from a copy?), of the tolerance (would 30 dB pass an image of noise?), of the leak bound (would 48 MB catch a 230 KB per-call leak?) and of the skip (is this branch exercised anywhere at all?).

What you have then is a suite that is green because the code is right, not because the runner resembles the machine that recorded the answer. Chapter 41, *Deploying to Production*, takes that build the rest of the way: the JVM flags a container needs, where `~/.javacpp` goes when the home directory is read-only, and what to size a pod's memory limit against when the interesting allocations never appear on the heap.

CHAPTER 41

Deploying to Production

Getting it onto a server without a 400 MB surprise.

The service worked on your laptop. It reads an upload, detects faces, draws boxes, returns a PNG, and it does all of that fast enough that nobody thought to ask how fast. Then you containerise it, and three numbers you had never considered become the only numbers anyone wants to discuss.

The first is the image size. Add OpenCV the convenient way and one dependency line puts about 408 MB of jars into the image, most of it native code for operating systems you will never deploy to. Your registry bill notices. So does every autoscaling event, because the node has to pull those bytes before the pod can start.

The second is cold start. The first request after a deployment is slower than every request that follows it by orders of magnitude, and nothing in your code changed between the two. What happened is that the first `OpenCv.load()` in a fresh container unpacked about 196 MB of shared libraries out of the jars and onto disk, and the request that triggered it paid for all of it — while your load balancer, which had already been told the pod was ready, kept sending traffic into the wait.

The third is the one that wakes people up. The pod is configured with `-Xmx512m` and a 1 GB memory limit. The heap graph is flat. The pod restarts anyway, with exit code 137 and no stack trace, no exception, no log line. Chapter 1 explained why: a `Mat` is about forty bytes on the heap and megabytes off it, so the number your heap graph draws and the number the kernel enforces are measuring different things. In a container that divergence stops being an interesting fact about the JVM and becomes an outage.

None of the three is a defect. All three are the direct, predictable consequence of shipping native code, and each has a fix that costs one line in a build file, one layer in a Dockerfile, or one field in a pod spec. This chapter is those lines.

Package for one platform, not for all of them

The natives ship in per-platform classifier jars, and Chapter 2 explained why scalacv cannot pick one for you: a build tool has no way to put a `<classifier>` into a published POM, so the coordinate that would carry the natives is the one coordinate scalacv is unable to publish. That left you with a choice, and for local development the lazy answer is fine. For a production build it is not.

Example 41-1. The convenient dependency, and what it costs in a registry.

```
// Works on every machine you will ever own. Ships about 408 MB to every machine
// you will ever run.
mvn"org.bytedeco:opencv-platform:4.13.0-1.5.13"
```

`opencv-platform` bundles every platform bytedeco builds for — Linux x86-64 and ARM64, macOS Intel and Apple Silicon, Windows x86-64, and several you have never deployed to — with the matching OpenBLAS jar for each. It is about 408 MB, against roughly 51 MB for the `linux-x86_64` pair a Linux container actually uses. The other 350-odd megabytes are dead weight in every layer, every pull, and every registry garbage-collection pass you will ever run.

Example 41-2. The production dependency: one platform, pinned.

```
def mvnDeps = Seq(
  mvn"com.worxbend::scalacv:0.4.2",
  mvn"org.bytedeco:opencv:4.13.0-1.5.13;classifier=linux-x86_64",
  mvn"org.bytedeco:openblas:0.3.31-1.5.13;classifier=linux-x86_64"
)
```

Both lines, always. `libopencv_core` links `libopenblas`, and the OpenBLAS payload ships only in OpenBLAS's own classifier jar — omit it and `OpenCv.load()` fails with a `CvError.NativesMissing` carrying both dependency lines, the classifier filled in from `Loader.getPlatform` and the versions read out of `Build`, which the build generates from the same constants it resolves the jars with. The message cannot tell you to add a release that is no longer the one the library was compiled against.

Table 41-1. What each packaging choice actually puts in the image.

Choice	Jar bytes	When it is the right answer
`opencv` + `openblas`, one classifier	36–80 MB (`linux-x86_64`: ~31 + ~20 MB)	every production deployment, and CI
`opencv-platform`	~408 MB	a developer laptop, a sample repository, a workshop — convenience only

If you build multi-arch images, build one artifact per architecture with its own matching classifier and let the manifest list choose between them. Do not build one fat artifact carrying both: you would be paying `opencv-platform`'s tax by hand, with extra steps. And note that bytedeco publishes no OpenCV natives at all for `windows-arm64` — scalacv's own build fails fast and says so rather than producing an artifact that cannot load.

The cache that is not in your image

Adding the classifier jars gets the natives onto the classpath, not onto disk as loadable libraries. They are payloads inside the jars. The first `OpenCv.load()` in a fresh environment extracts the platform payload out of them and into a cache directory — on Linux, about 196 MB under `~/.javacpp` — and loads the libraries from there by absolute path. Every later run reuses the extracted copy.

Two properties make that manageable rather than merely expensive: it is **content-addressed**, so re-running never re-extracts, and two processes can share one directory safely. Together they mean the unpack can happen anywhere, at any time before the first request — including in a build layer that has no relationship to the container that eventually uses it.

The directory is relocatable, which matters in a container because the default is a home directory that a non-root user may not own and a read-only root filesystem will refuse outright.

Example 41-3. Two ways to say the same thing; the second is the one containers usually need.

```
java -Dorg.bytedeco.javacpp.cachedir=/opt/javacpp -jar app.jar

# When the entrypoint is not yours to edit, the JVM reads this instead:
export JAVA_TOOL_OPTIONS="-Dorg.bytedeco.javacpp.cachedir=/opt/javacpp"
```

That property name is exact: `org.bytedeco.javacpp.cachedir`, all lower case, no camel hump. It is javacpp's, not scalacv's, so nothing in this library validates it — a typo does not fail, it silently leaves the cache in the default location, and you find out when a read-only root filesystem rejects the write at boot.

Table 41-2. Where to put the extracted natives, and what each choice buys.

Strategy	How	Trade
Bake into the image	run a warm-up main in a build layer, `COPY` the result into the runtime stage	image grows by ~196 MB; cold start is instant and self-contained — the right default for autoscaling
Mount a shared volume	point `cachedir` at a mount; first pod to boot populates it	thin image; cold start waits on the volume being warm, and the volume is now a dependency that can be empty
Leave it in the container's home	do nothing	no configuration; every cold start pays the 196 MB unpack, and it breaks under `readOnlyRootFilesystem`

The third row is not a strategy, it is what happens when nobody chooses. It is also the source of the slow first request from the opening of this chapter.

A Dockerfile, line by line

Here is the whole thing, and then the reasoning for each part of it. It is multi-stage, it warms the cache in a layer nobody serves traffic from, it runs as a non-root user, and it survives a read-only root filesystem.

Example 41-4. A production image: build, warm, run.

```
# ---- 1. build the fat jar -------------------------------------------------
FROM eclipse-temurin:17-jdk AS build
WORKDIR /src
COPY . .
RUN ./mill app.assembly && cp out/app/assembly.dest/out.jar /src/app.jar

# ---- 2. warm the javacpp cache in a throwaway stage -----------------------
FROM eclipse-temurin:17-jre AS warm
COPY --from=build /src/app.jar /app/app.jar
RUN java -Dorg.bytedeco.javacpp.cachedir=/opt/javacpp \
         -cp /app/app.jar com.example.Warmup

# ---- 3. the runtime image -------------------------------------------------
FROM eclipse-temurin:17-jre
RUN useradd --system --uid 10001 --create-home --home-dir /home/app app
COPY --from=build /src/app.jar /app/app.jar
COPY --from=warm --chown=10001:10001 /opt/javacpp /opt/javacpp
ENV OPENBLAS_NUM_THREADS=1 \
    OMP_NUM_THREADS=1 \
    JAVA_TOOL_OPTIONS="-Dorg.bytedeco.javacpp.cachedir=/opt/javacpp \
      -Dorg.bytedeco.javacpp.maxPhysicalBytes=1600M \
      -XX:MaxRAMPercentage=30 \
      -Djava.io.tmpdir=/tmp"
USER 10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

The build stage carries a JDK; the runtime stages carry a JRE. Nothing at run time compiles Scala, and the JRE base image is smaller by a margin worth having — one of the few size wins on this page that costs nothing at all.

Stage 2 exists only to run one main. `com.example.Warmup` is any class whose `main` calls `OpenCv.load()` — the whole point is the side effect on `/opt/javacpp`. Giving it its own stage keeps the runtime image free of the build's leftovers; `COPY --from=warm` brings across only the extracted libraries.

The warm-up must use the same `cachedir` the runtime will. The extraction is content-addressed under whatever root it is given; warm `/opt/javacpp` and then run with the property unset and you have baked 196 MB into the image and will unpack it again anyway, into the home directory, on the first request.

`--chown` on the `COPY`. The runtime user is `10001`, and javacpp will at least stat and open those files. Copying them as root and then dropping privileges works for reading, but a cache

directory the process cannot write is one it cannot repair if the copy is ever incomplete. Own it.

`-Djava.io.tmpdir=/tmp` with a writable `/tmp`. Under `readOnlyRootFilesystem` you must mount an `emptyDir` (or `--tmpfs /tmp` under plain Docker) for the JVM's own temporary and perf files. Note where the **other** writes go: `Models.fetch` does not use `java.io.tmpdir` at all — it creates its temp file beside the target and moves it into place after verifying, so it is the model directory that has to be writable, not `/tmp`. The javacpp cache is fine read-only after the warm-up: it must be writable on first use, and the first use already happened in stage 2.

The thread-pool environment variables. Chapter 36 worked through this: OpenCV and OpenBLAS each maintain an internal pool, and in a service already handling several requests at once those inner pools oversubscribe the cores and cost you latency rather than buying throughput. Pinning them to one thread each and letting your request executor own the parallelism is the right default **for a concurrent server**. For a batch job running one large pipeline at a time, do the opposite and leave them on.

Alpine, and the honest answer

The instinct that produced the multi-stage build — make the image smaller — points next at Alpine, and here it stops working.

The Linux payload in the bytedeco classifier jars is compiled against glibc. Alpine's libc is musl, which is not ABI-compatible with it, and javacpp's platform detection does not distinguish the two: `Loader.getPlatform` reports `linux-x86_64` on Alpine exactly as it does on Debian, so scalacv extracts the glibc payload and hands it to a linker that cannot satisfy it. The `UnsatisfiedLinkError` becomes a `CvError.NativesMissing` telling you to add dependencies you demonstrably already have — misleading, but honestly so: "the jar is missing" and "the jar is here and unusable" look identical from inside the loader.

There are compatibility shims that make glibc binaries run under musl. Installing one costs most of the size you came to Alpine for and buys a configuration nobody upstream tests. The unglamorous answer is the correct one: use a glibc base — `eclipse-temurin:17-jre`, or a `debian:*-slim` with a JRE — and spend the size budget on the 196 MB cache and the single classifier pair instead. Both are real wins, and neither risks a link failure at boot.

Budgeting memory when the heap is not the footprint

A container memory limit is enforced against the cgroup's total resident set. It counts the heap, and it counts every byte of pixel data OpenCV allocated through its own `cv::fastMalloc`, and it has no interest in which is which. `-Xmx` bounds one of those terms. Sizing a pod from `-Xmx` alone is how you get the flat heap graph and the restart.

The failure mode is worth recognising by sight, because it looks like nothing. There is no `OutOfMemoryError`, because the JVM never ran out of heap, and no exception in your logs, because the process did not fail — it was killed, by the kernel, between two instructions. The

pod status says `OOMKilled`, the exit code is 137, and the heap dashboard is a flat line right up to the restart. Chapter 1 has the underlying measurement: 2000 unreleased 1000×1000 three-channel `Mat`s reached 5,865 MB of RSS where the released version stayed at 144 MB, and the heap was never under pressure in either case.

Table 41-3. A container memory budget. Every term is measured, not guessed.

Term	Where the number comes from	Example
JVM heap	`-Xmx`, or `-XX:MaxRAMPercentage` of the limit	600 MB
Native pixels at peak	bytes per in-flight pipeline × maximum concurrency. A 1920×1080 BGR frame is 6.2 MB; a pipeline holding six of them at once is 37 MB	8 × 37 = 296 MB
Loaded native code and allocator arenas	process RSS at idle, after `OpenCv.load()`, minus the heap	measure it
JVM overhead	metaspace, code cache, thread stacks, GC structures — `-XX:NativeMemoryTracking=summary`	~250 MB
Headroom	so the ceiling below fires before the kernel does	20%

The third row is the one people want a constant for, and there is not an honest one to give. The 196 MB in the cache is what the extraction writes to **disk**; what counts against RSS is the portion of those libraries the process actually maps in, which depends on which OpenCV modules your pipeline touches. Read it once from your own image, at idle, immediately after `OpenCv.load()` returns — Chapter 38 showed the cheap way, the second field of `/proc/self/statm` multiplied by the page size, which is the same source scalacv's own leak harness uses.

With a total, set the container limit above it and set the JVM's own physical ceiling **below** it, so that a leak produces a diagnostic instead of a kill.

Example 41-5. The two ceilings, and which counter each one watches.

```
java -Xmx600m -Dorg.bytedeco.javacpp.maxPhysicalBytes=1600M -jar app.jar
# ...in a pod whose memory limit is 2Gi.
```

> **NATIVE MEMORY**
> `-Dorg.bytedeco.javacpp.maxBytes` is the wrong knob and it looks like the right one. It is checked against `Pointer.totalBytes()`, javacpp's own allocation counter, which is blind to the `org.opencv.core.Mat` buffers scalacv actually manages — a `Mat` leak sails straight past it and the process dies with the ceiling reporting healthy numbers. `maxPhysicalBytes` is RSS-based and does see them. Set that one.

> When it fires it throws `java.lang.OutOfMemoryError`, not a `CvError`, so no `Either` in this library catches it and no `Cv.attempt` turns it into a value. Chapter 39 covers what that means for a request handler. Treat it as a backstop that produces a good diagnostic at the moment of failure, not as a substitute for releasing images.

Models: in the image, on a volume, or nowhere at all

Chapter 23 covered `Models.fetch` and `ModelSpec` in full; what changes at deployment time is only where the file comes from. Three options, and the first two are the same mechanism.

Baking the model in makes the container self-contained: `COPY models/ /opt/models/`, and the pod needs no network to become ready. It costs image size — SFace is about 37 MB, YuNet 232,589 bytes — and makes a model update an image rebuild, which for a pinned, checksummed artefact is arguably the correct coupling. Mounting them instead keeps the image thin and lets several services share one copy, at the cost of a volume that has to exist and be populated before the first pod schedules.

The third case is the one that decides your `ModelSpec`. In an air-gapped deployment there is no outbound route at all, and a fetch that reaches for a URL does not fail fast — it waits out a 15-second connect timeout per mirror, on the boot path, and up to sixty seconds more for a mirror that connects and then stalls. `Models.fetch` handles `file://` by falling out of the HTTP client onto a plain stream, so a model already on disk is another source rather than a special case, and mirrors are tried strictly in order.

Example 41-6. A spec that works air-gapped and still works when the network is there.

```
val yunet = ModelSpec(
  fileName = "face_detection_yunet_2023mar.onnx",
  urls = Seq(
    "file:///opt/models/face_detection_yunet_2023mar.onnx",  // the baked copy
first…
    "https://media.githubusercontent.com/media/opencv/opencv_zoo/main/" +
      "models/face_detection_yunet/face_detection_yunet_2023mar.onnx"  // …then
the mirror
  ),
  sha256 = "8f2383e4dd3cfbb4553ea8718107fc0423210dc964f9f4280604804ed2552fa4",
  sizeBytes = Some(232589L)    // checked first: one stat, and a better message
than a hash mismatch
)
```

The SHA-256 is verified on download **and** on every cache hit, which is what makes a shared model volume safe to point several pods at: a truncated or half-written file is rejected and refetched rather than loaded as a model. The pinned `sizeBytes` is checked before the digest, which is what turns the classic deployment failure — a proxy answering the model request with an HTML error page, or a Git LFS host serving a 131-byte pointer file, both with HTTP 200 — into "expected 232589 bytes, got 131" instead of a checksum mismatch that reads like tampering.

Cascade XML needs none of this: it travels inside the bytedeco classifier jar you already depend on, under `share/opencv4/haarcascades/`, and `Cascades.load` extracts the one you name on demand. The exception is `windows-x86_64`, whose jar ships an empty `share/` directory — there, and only there, you provision the XML yourself and load it with `Cascades.loadFrom`. It is a property of the upstream jar, not of your build, so no dependency you add will fix it.

Fail at boot, not on the first request

`OpenCv.load()` is idempotent and thread-safe, so calling it costs nothing after the first time, which makes it tempting to call it lazily at the point of first use. Do not. A native problem that surfaces on the first request surfaces as a 500, after the load balancer has been told the pod is healthy, during a rollout that is by then halfway done.

Call it at boot, before the server binds its port, and let a failure fail the process. The error you get is designed for exactly this moment: `CvError.NativesMissing` carries a copy-pasteable pair of dependency lines naming the platform the container is actually on, which is a far better thing to find in a crash-loop log than an `UnsatisfiedLinkError` stack.

That error has a second form, and it only ever shows up in a deployment. If the natives are on the classpath but another classloader in the same JVM has already loaded them, the message says so and tells you to move scalacv to a classloader both sides share — a servlet container's shared library directory rather than each application's `WEB-INF/lib`. Loading a second copy is not an available answer: two copies of `libopencv_java` each keep their own globals, and a `Mat` allocated by one is meaningless to the other. One fat jar in one container never meets this; three war files in an application server meet it on the second deployment.

Now the readiness endpoint. The version everybody writes first is this:

Example 41-7. A readiness probe that proves the JVM is running and nothing else.

```
// WRONG: true the instant the HTTP server binds, natives or no natives.
def ready: Boolean = true
```

It answers a question nobody asked. The process being up says nothing about whether the natives extracted, the cache directory was writable, the model arrived, or a pixel can cross JNI at all. Make the probe do the smallest real piece of work instead.

Example 41-8. A warm-up that runs once at boot, and a probe that exercises a real pixel.

```
import scalacv.*
import scala.util.Using

object Warmup:

  @volatile private var warm = false

  /** Called once, before the server binds. A failure here should kill the
```

```
process. */
  def run(): Either[CvError, Unit] =
    OpenCv.load()                          // throws CvError.NativesMissing if the
natives are absent
    probe().map(_ => warm = true)

  /** One pixel, all the way through: allocate, convert, encode, release. */
  def probe(): Either[CvError, Array[Byte]] =
    Cv.attempt("readiness") {
      Using.resource(Image.blank(1, 1, Scalar.White))(_.gray.bytes(".png"))
    }.flatMap(identity)

  /** What the HTTP handler answers with: the boot probe's verdict, not a fresh
guess. */
  def ready: Boolean = OpenCv.isLoaded && warm
```

`Image.blank(1, 1, Scalar.White)` allocates a real three-channel `Mat`, `gray` runs a real `cvtColor` across JNI, and `bytes(".png")` runs a real encoder and releases the image as its terminal step. Four native round trips on a single pixel, and the `Using.resource` closes the source image even though `gray` already consumed it — `close` delegates to `Managed.release`, which is idempotent, so the belt-and-braces version is safe rather than a double free. `Cv.attempt` turns anything OpenCV throws into a `Left` naming the operation, so a probe that fails reports which stage failed instead of unwinding a `CvException` through your HTTP handler.

If your service also runs a DNN, add one warm-up `Dnn.forward` on a dummy blob here too — it returns a `Managed[Mat]`, so release it. The first inference on a `Net` pays a one-time layer-allocation cost that has no business landing on a user's request.

> **TIP**
> Split the two probes. A Kubernetes `startupProbe` with a generous `failureThreshold` covers the extraction and model-fetch window without forcing you to loosen the `readinessProbe` interval that protects you for the rest of the pod's life. With a pre-warmed cache the startup window is short enough that this is cheap insurance rather than a workaround.

Kubernetes: requests, limits, and the pool you did not configure

Two fields on a pod spec deserve more thought than they usually get for this workload.

Memory: set the request equal to the limit. The scheduler places pods by request; the kernel kills by limit. For a workload whose memory is mostly off-heap the gap between them is not a safety margin, it is a lottery — the pod runs fine on a quiet node and is killed on a busy one, with nothing in your code or traffic to explain the difference. Equal request and limit gets the pod the `Guaranteed` QoS class and makes the failure reproducible, which is the first requirement for fixing it.

CPU: think hard before setting a limit at all. A CPU limit is enforced by CFS quota, which works by stopping every runnable thread in the cgroup until the next period. That interacts badly with a library running its own thread pool: OpenCV sizes that pool from the CPU count it observes, which is the machine's, not your quota. Give a pod a 500-millicore limit on a 32-core node and one bilateral filter can fan out to dozens of threads allowed 50 ms of CPU per 100 ms period between them — they run, get throttled as a group, and the operation's tail latency goes up while the CPU **usage** graph sits reassuringly under the limit.

Example 41-9. Pinning the pools so the quota and the parallelism agree.

```
resources:
  requests: { memory: 2Gi, cpu: "1" }
  limits:   { memory: 2Gi }          # no CPU limit; the request reserves the
share
env:
  - { name: OPENBLAS_NUM_THREADS, value: "1" }
  - { name: OMP_NUM_THREADS,      value: "1" }
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
```

From code the equivalent lever is `org.opencv.core.Core.setNumThreads`, called before you spread work across your own executor. Which way to set it is a measurement, not a rule: Chapter 35's `ConfigProbeBench` steps a heavy operation through 1, 2, 4 and your CPU count precisely because the answer is a property of your hardware and your concurrency, and publishing one machine's result would be actively misleading.

Rolling upgrades, and models that do not collide

A rolling upgrade runs two versions of your service at once, briefly, on the same nodes and against the same volumes. Three things about this deployment survive that, and it is worth knowing why.

The javacpp cache survives because extraction is content-addressed and concurrent readers are safe. Two images built months apart, sharing one mounted cache directory, each find or write their own payload without disturbing the other. If the natives version changed between them, the new one extracts alongside the old rather than over it.

Models survive because a `ModelSpec` pins a **file name** as well as a checksum, and the names carry their provenance: `face_detection_yunet_2023mar.onnx` is a different file from a 2024 revision, not a newer version of the same one. Upgrading a model is therefore adding a file, never overwriting one, and a shared model volume can hold both while the two versions of the service coexist. Resist the temptation to normalise the name to `yunet.onnx` — the moment you do, two versions of the service point one path at two different files, each one's cache check fails against the other's pinned hash, and for the length of the rollout the two pods re-download over each other. `Models.fetch` at least makes that loud rather than corrupting:

it downloads to a temp file beside the target and moves it into place only after the size and digest verify, so no reader ever sees a truncated model.

The old pods survive because your readiness probe does real work. A new image with a broken classifier line, a missing model, or a cache directory it cannot write fails its probe, the rollout stalls with the old replicas still serving, and you get a `CrashLoopBackOff` naming the actual problem instead of a 50% error rate that takes ten minutes to attribute.

Table 41-4. The deployment checklist, and where each item was earned.

Check	Chapter
One classifier pair, not `opencv-platform`	2, 41
`cachedir` relocated, and warmed in a build layer	41
`maxPhysicalBytes` set below the container memory limit	35, 39, 41
Inner thread pools pinned when the server is the one running concurrently	36
Models provisioned via `file://` or baked in; checksums pinned	23
`OpenCv.load()` at boot; readiness gated on a real pixel operation	41
Every request scopes its images (`Image.reading` , `Managed.scope` , `Camera.using`)	5
Decode and codec failures handled as `Either` , with fallbacks	6, 20
Process RSS on a dashboard, as the leak alarm	38

When it still goes wrong

Everything above is the configuration that prevents the failures this library's shape makes likely. It does not prevent the failures that come from the machine, the base image, the classloader or the codec build — an `UnsatisfiedLinkError` that names no library, a `Left` from a recorder complaining about a codec that exists on your laptop and not on the node, an OpenCV warning printed to stderr for a call that returned successfully. Chapter 42, *Troubleshooting*, is the field guide to reading those messages: what each one actually means, which of them are benign, and the shortest path from the text in your log to the line in your build file that caused it.

CHAPTER 42

Troubleshooting

What each failure actually means, indexed by the words you would use to describe it.

A vision program fails in two registers. In the JVM register a mistake produces a stack trace with your own file names in it. In the native register it produces a signal, a file called `hs_err_pid31474.log`, and no indication of which line of Scala was running. Most of this book has been about staying in the first register; this chapter is for the days you end up in the second, and for the days the message in front of you is telling the truth about something you have not read carefully.

The failures below are not evenly distributed. The overwhelming majority are one of three: the platform classifier is absent or wrong, so nothing runs; a handle was reused after a transform consumed it, so `IllegalStateException` fires on a line that looks innocent; or native memory is climbing while the heap graph stays flat, so nothing alerts until the kernel kills the process.

These messages are written to be read rather than grepped. Read all of one before searching for it.

Table 42-1. The index. Find the symptom, read the section.

What you saw	Section
`UnsatisfiedLinkError`, or `NativesMissing` at load	The natives are not there
The kernel killed the process; heap was flat	RSS climbs, heap does not
`IllegalStateException: … already been released or consumed`	A spent handle
`SIGSEGV`, an `hs_err_pid*.log`, no Java frames	A hard crash
`InaccessibleObjectException`, a `--add-opens` in the text	The module system said no
`Left(DecodeFailed)` on a file you can see	The image will not read
A video file of zero bytes, or one nothing will play	The recording is empty
Everything is orange where it should be blue	The colours are swapped
A detector that returns an empty `Seq`, always	The detector finds nothing
The same input, two machines, two answers	Results differ between machines
The first frames take ten times as long as the rest	The first frames are slow
A 24-megapixel photograph takes seconds	Everything is slow on a big image
The camera opened; every frame is black or absent	The camera opens but delivers nothing

`CalibrationFailed`, or an RMS error above 2 px	Calibration will not converge
It compiles, it resolves, and it will not run	The build resolves but nothing runs

Start-up and the build

The natives are not there

What it means

The `scalacv` jar contains no native code. `OpenCv.load()` extracts the OpenCV libraries out of the per-platform classifier jars and loads them by absolute path; if those jars are absent, the extraction finds nothing and the JNI shim will not link. The failure surfaces as `CvError.NativesMissing` rather than a bare `UnsatisfiedLinkError`: the loader catches the link error and replaces it.

The message is the fix, computed at the moment of failure from `Loader.getPlatform`, so the classifier it names is the one you are actually on rather than the one somebody wrote into a document once:

Example 42-1. The diagnostic. The versions come from the build, the classifier from `Loader.getPlatform`.

```
OpenCV natives are not on the classpath (no opencv_java in java.library.path).

scalacv depends on the classifier-less OpenCV Java API only, because a build tool
cannot
express a per-platform classifier in a published POM. Add the natives for your
platform:

  "org.bytedeco" % "opencv"   % "4.13.0-1.5.13" classifier "linux-x86_64"
  "org.bytedeco" % "openblas" % "0.3.31-1.5.13" classifier "linux-x86_64"

Both lines are needed: libopencv_core links libopenblas. If you would rather not
pick a
platform, "org.bytedeco" % "opencv-platform" % "4.13.0-1.5.13" bundles every one,
at a
cost of about 408 MB.
```

What to do

Add both lines, with the classifier for the machine that *runs* the code rather than the one that builds it. `libopencv_core` links `libopenblas`, so omitting the second line leaves the first unresolvable — which looks identical to having added nothing.

Table 42-2. The five classifiers. Pick the one for the target, or take the 408 MB fat artifact.

Target	Classifier
Linux x86-64	`linux-x86_64`
Linux ARM64	`linux-arm64`
macOS Intel	`macosx-x86_64`

macOS Apple silicon	`macosx-arm64`
Windows x86-64	`windows-x86_64`

One failure wears this exception's clothes without being it. A JVM maps a given native library file into exactly one classloader, so a second classloader loading scalacv gets `UnsatisfiedLinkError: … already loaded in another classloader` with the jars demonstrably present — and that message gets its own text, because telling you to add dependencies you already have is a dead end. Loading a second copy is not the alternative: two copies of `libopencv_java` each keep their own globals, and a `Mat` allocated by one is meaningless to the other. Load scalacv from a classloader both sides share — in a servlet container, the container's shared library directory rather than each application's `WEB-INF/lib`; in OSGi, one bundle that exports `scalacv` rather than a copy per bundle.

The build resolves but nothing runs

What it means

`scalacv` compiles fine with no native line at all: the classifier-less OpenCV Java API contains the classes but none of the symbols behind them. A build that resolves, a file that compiles, and a program that dies on the first `OpenCv.load()` is therefore the expected state until you add the platform jars yourself — the two-line dependency problem from Chapter 2, and the most common first-day failure there is.

Its compile-time cousin, `value faces is not a member of scalacv.Image`, is a different problem with a similar feel: the symbol lives in `scalacv-vision` or `scalacv-graphs`, and an import can bring into scope only what the classpath contains.

What to do

Add the natives, or add the module. Keep the wildcard `import scalacv.*` either way — a single-symbol import brings in the type and none of the extension methods defined beside it.

Memory and lifetimes

RSS climbs, heap does not

What it means

The process was killed by the OOM killer, or by a container limit, while every heap metric you have was healthy. That is the shape of a native leak, and it is the reason this library exists. A `Mat` is about forty bytes of Java header in front of a multi-megabyte off-heap buffer. Heap pressure is the only thing that triggers a collection, and heap pressure is uncorrelated with native pressure, so nothing fails — memory grows, and then the kernel intervenes.

> NATIVE MEMORY

> The measurement on this project's test machine: 2000 × `Mat(1000, 1000, CV_8UC3)`, references dropped, no explicit `System.gc()`. Unreleased, RSS reached 5 865 MB. Released, 144 MB. Same 41× on JDK 21 and JDK 25. Reclamation is not impossible; nothing makes it happen in time.

What to do

Put a ceiling on *physical* bytes and the leak fails fast, in a test rather than in production:

Example 42-2. A leak that took an hour to notice now takes a minute.

```
java -Dorg.bytedeco.javacpp.maxPhysicalBytes=512M -jar your-app.jar
```

`maxBytes` will not catch it: javacpp's own accounting is blind to buffers OpenCV allocated inside `cv::Mat`. Once it fails fast, bisect by moving suspects onto a scoped entry point.

Table 42-3. The four substitutions that close nine leaks out of ten.

Instead of	Use	It closes
`Image.read(p)`, then forgetting	`Image.reading(p)(use)`	the image, even if the body consumed it
`Camera.open(i)` in a loop	`Camera.using(i)(use)`	the camera and its capture
`Video.frames` with a retained `Mat`	`Video.framesCopied`	each owned copy, per iteration
a raw `Mat`	`Managed.use(mat)(f)`	the Mat on the way out

One case survives all four: the *arguments* to a two-image operation are borrowed, not consumed. The mask you hand `applyMask`, `inpaint`, `blurBackground` or `seamlessCloneInto`, the second image you hand `blend`, and the background you hand `seamlessCloneInto` all come back yours to `close()`. Only the receiver is spent. A slow leak in an otherwise disciplined codebase is usually a borrowed mask.

A spent handle

What it means

`IllegalStateException: this Mat has already been released or consumed`. You used a value that a transform already spent. The message names `Mat` rather than `Image` because an `Image` wraps a `Managed[Mat]`, and the handle reports the class of the thing it actually owns; Chapter 5 spells that out. `Image` has move semantics — queries borrow, transforms and terminals consume — so the receiver of `gray`, `blur`, `canny`, any `draw*`, `write`, `bytes` or `close` is dead the moment the call returns.

Example 42-3. The mistake, then the fix. `img` is spent by `.gray`, so the second line has nothing to work on.

```
val img = Image.blank(8, 8)
val a = img.gray
```

```
val b = img.blur(2)          // throws: `img` was consumed by `.gray`

val original = Image.blank(8, 8)
val edges = original.copy.gray.canny(50, 150)  // `.copy` borrows; `original`
survives
val small = original.resize(4, 4)              // now `original` is consumed
```

What to do

Take a `.copy` before the first use when you need the image twice. The harder part is finding the consuming call, since the exception fires at the reuse. Start the JVM with `-Dscalacv.trackOwnership=true` and it carries, as its `cause`, the stack of the transform or terminal that spent the handle:

Example 42-4. Ownership tracking: off by default, and free until something already went wrong.

```
java -Dscalacv.trackOwnership=true -jar your-app.jar
```

It is off by default because it allocates a `Throwable` every time a handle is spent; the read happens only on the already-failing path, so a correct program pays nothing for the check.

A hard crash

What it means

The JVM died with `SIGSEGV`, wrote an `hs_err_pid*.log`, and produced no Java stack trace, because there was no Java frame to produce one from. Three causes account for nearly all of these.

The first is a double free. Every one of the 185 `org.opencv.*` types without a public `release()` carries `protected void finalize() throws Throwable { delete(this.nativeObj); }`, unconditionally — so a raw handle you freed yourself is freed a second time by the finalizer thread whenever the collector next runs. That is heap corruption, and it surfaces somewhere else entirely, later:

Example 42-5. The frame that names the culprit. Read the `C` line and the thread name together.

```
SIGSEGV (0xb)  C  [libopencv_java.so+0x163155]
Java_org_opencv_objdetect_FaceDetectorYN_delete
Current thread: JavaThread "Finalizer"
```

> **NATIVE MEMORY**
> `Managed` prevents this by zeroing `nativeObj` before deleting, so the finalizer's `delete(0)` becomes `delete nullptr`, a no-op in C++. A crash of this shape means a raw handle escaped `Managed` somewhere.

The second is a handle shared across threads. Native handles are one-owner-per-thread, exactly as in raw OpenCV: detector *results* are immutable data and safe to share, the detectors are not, and a `FaceDetectorYN` is mutated by every `detect` call.

The third is ABI mixing, which does not look like your bug at all. The bundled `libopencv_highgui.so` carries unversioned dependency names, so on a machine with a system OpenCV installed a bulk `dlopen` maps `libopencv_*.so.5.0.0` into the global namespace, where it interposes on the 4.13.0 symbols; the next call crossing between the two ABIs dies inside `cv::Mat::release()`. `OpenCv.load()` resolves dependencies on demand rather than speculatively, for exactly that reason.

What to do

Read the `hs_err` frame list from the top. The first `C` frame names the native symbol, and `Java_org_opencv_*_delete` in that position means a lifetime bug rather than a computation bug. `Current thread` narrows it further: `"Finalizer"` means a double free. Then check whether anything calls `Loader.load` directly instead of `OpenCv.load()`, and whether any raw `org.opencv.*` handle is constructed outside a `Managed` or a `Managed.scope`. Keep the file: it is usually the only evidence a native crash leaves.

The module system said no

What it means

An `InaccessibleObjectException` reported as `CvError.NativesMissing`, in the middle of a pipeline that has been working for twenty minutes. Note when it fires: as a handle is *freed*, typically at the end of a `Managed.use` block, not at `OpenCv.load()`. OpenCV was loaded from a named module rather than the classpath, so reflection cannot reach the private `delete(long)` that frees the 185 detector types, or cannot write the `nativeObj` the disarm step needs.

The library refuses to degrade here: falling back to the collector is an unbounded leak that looks like success, and deleting without disarming is the double free of the previous section.

What to do

Add the flag the message prints. It is computed from the offending class's own module and package at the moment of failure, so it is right for the type that failed.

Example 42-6. The flag for a `CascadeClassifier` or a `QRCodeDetector`. A `Net` asks for `org.opencv.dnn` instead.

```
--add-opens org.bytedeco.opencv/org.opencv.objdetect=ALL-UNNAMED
```

The pair is what people get wrong by hand: the module is `org.bytedeco.opencv`, the package is `org.opencv.objdetect`, and the two do not share a prefix. It is never `java.base/java.lang` — the field being opened is OpenCV's own. On the classpath there is no module to open, so the message says so and asks for a bug report rather than printing a flag with a `null` in it. Where you have the choice, running OpenCV on the classpath removes the problem instead of papering over it.

Messages that carry their own remedy

Three diagnostics here are generated rather than written: the missing-natives text, which fills in `Loader.getPlatform` and the artifact versions from the build; the `--add-opens` line, which reads the class's own `Module` and package; and the Windows cascade failure, which names the platform's jar and the empty `share/` directory inside it. All three exist because a static sentence and a documentation link would make the reader do work the process has already done.

The consequence: when a scalacv message runs past one line, the rest is not boilerplate.

Wrong output

The image will not read

What it means

`Image.read` returned `Left(DecodeFailed)` for a file you can see in your editor. `imread` does not throw on a missing file, a directory, or bytes it cannot decode — it returns an empty `Mat`, which would run a whole pipeline on nothing — so scalacv makes the check explicit and turns it into a `Left`.

What to do

Read `details`. `Images.read` fetches the bytes with the JVM's own file I/O and leaves only the decoding to OpenCV, so the message distinguishes `there is no file at this path`, `this path is a directory, not a file`, `the file is empty`, `this is not a usable filesystem path` and `the bytes are not an image in a format OpenCV can decode`. The first two are almost always a relative path resolved against a working directory that is not the one your editor shows you — print `java.io.File(path).getAbsolutePath` and the argument ends. `the file is empty` is a truncated download or a half-written file. The last is a codec this build does not carry. And OpenCV understands neither URLs nor `classpath:` resources: fetch the bytes yourself and use `Image.decode`.

`DecodeFailed` is specifically about image bytes; a model, cascade or video source that will not load reports `LoadFailed`, a distinction Chapter 6 draws in full.

The recording is empty

What it means

`Recorder.open` returned a `Left`, or opened and produced a file nothing will play. `VideoWriter` reports a missing codec by leaving `isOpened` false rather than by throwing, so scalacv turns that into `CvError.LoadFailed` whose message names the fallback.

The codec and the container move together, and that is the trap. `Codec.Mjpg` opens only in an `.avi`; `Codec.Mp4v` and `Codec.Avc1` only in an `.mp4`. Worse, the `linux-x86_64` and

`windows-x86_64` payloads this project builds against ship no FFmpeg plugin at all, so `Mp4v` and `Avc1` do not open there — which is why `Codec.Mjpg` is the default for `Recorder.open`, `Recorder.using`, `Camera.recordTo` and `Animation.record`.

What to do

Example 42-7. Prefer the compact codec, fall back to the one that always opens. Note the extension moving with it.

```
def openRecorder(base: String, size: Size, fps: Double): Either[CvError,
Recorder] =
  Recorder.open(s"$base.mp4", size, fps, Codec.Mp4v)
    .orElse(Recorder.open(s"$base.avi", size, fps, Codec.Mjpg))
```

If even `Codec.Mjpg` in an `.avi` fails, the path is unwritable — check the parent directory exists. If the file exists but is tiny, the recorder was never closed: `close()` finalises the container, and `Recorder.using` does it on every path. A frame whose size does not match the recorder's is not silent corruption — `write` requires the match and throws `IllegalArgumentException` naming both sizes, as it does for a frame that is not 8-bit.

The colours are swapped

What it means

Skies are orange, skin has a cyan cast, a red annotation came out blue. OpenCV stores three-channel images as *BGR*, not RGB, for historical reasons that are now nobody's fault. Anything that hands pixels to a library expecting RGB — a web encoder, a tensor, another imaging toolkit — swaps two channels unless something converts.

What to do

Convert with `ColorConversion.BgrToRgb`, or read straight into RGB with `ImreadFlags(ImreadColor.ColorRgb)`, which skips the conversion — the object's own shorthands are `ImreadFlags.Color`, `.Grayscale` and `.Unchanged`, and RGB is not among them. For AWT, `toBufferedImage` and `Image.fromBufferedImage` handle the ordering already; a hand-rolled raster copy is where this creeps in. Inside scalacv, use `Scalar.Red`, `Scalar.Green` and `Scalar.Blue` and channel order stops being something to remember. Chapter 7 has the round-trip.

The detector finds nothing

What it means

An empty `Seq`, every frame, with no error anywhere. A detector that finds nothing is indistinguishable from a scene containing nothing, so nothing complains.

What to do

Work down four things in order. First, is the model loaded? `Cascades.loadFrom` verifies the classifier is non-empty precisely so a typo comes back as a `Left` rather than as a detector that finds nothing forever — and on Windows the bytedeco jar ships an empty

`share/` directory with no cascade XML, so `Cascades.load` by name *cannot* succeed there. Second, the input: Haar detection wants a single-channel, histogram-equalised image — `img.gray.equalizeHist` — and a colour Mat works but is slower. An object smaller than the window the cascade was trained on is not found at any pyramid level, which is why a subject that occupies a few dozen pixels of a 4000 px photograph needs the photograph cropped rather than the parameters loosened. Third, the parameters: `detectHaar` defaults to `scaleFactor = 1.1`, `minNeighbors = 3` and `minSize = None`, and raising `minNeighbors` to suppress false positives is how people accidentally suppress everything. `FaceDetect.create` defaults to `scoreThreshold = 0.9f` and `nmsThreshold = 0.3f`; 0.9 is strict, and 0.6 is a fair first experiment. Fourth, the colour space: a mask built in BGR when the threshold was designed for HSV returns empty for the reason above.

Results differ between machines

What it means

Three independent causes, and it is worth deciding which one you have before chasing any of them.

The OpenCV build differs. The classifier jars do not all carry the same videoio backends — no FFmpeg plugin in the `linux-x86_64` and `windows-x86_64` payloads — and forcing a `CaptureBackend` that is not compiled in turns a working `open` into a failing one. The Windows jar ships no Haar cascades. Those are properties of the artifact, not of your code.

The model differs. `ModelSpec.unverified` disables cache invalidation as well as integrity: whatever file is on disk under that name is served forever and the URL is never contacted again, so two machines that downloaded at different times can run different weights with no signal that they do.

And floating point varies. OpenCV dispatches on the SIMD the CPU reports and parallelises kernels across a thread pool sized from the core count, so a bilateral filter or a forward pass can differ in the last bits between machines. That is not a defect to fix.

What to do

Pin a SHA-256 in every `ModelSpec` and let a mismatch re-download. Leave `CaptureBackend.Any` unless you have a concrete reason. And compare images with a tolerance — `org.opencv.core.Core.PSNR`, or a bounded maximum absolute difference — rather than byte equality, the assertion that makes a portable pipeline look broken. Chapter 40 has the shape of those tests.

Performance and hardware

The first frames are slow

What it means

Two warm-ups, stacked. The first `OpenCv.load()` in a fresh environment extracts about 196 MB of native libraries into `~/.javacpp`; every later run reuses that cache, so it is a cold-start

cost rather than a per-run one. On top of it the JIT needs many iterations before the pixel loops reach steady state, which is why this project's benchmark harness runs 2000 untimed iterations by default, in `Bench.measure`, before it times anything.

What to do

Relocate the cache with `-Dorg.bytedeco.javacpp.cachedir=/var/cache/javacpp`, or pre-warm it at container build time by running any main that calls `OpenCv.load()` — the extraction is content-addressed and idempotent, so a baked layer makes every cold start instant. For the JIT, push a few frames through the real pipeline before reporting any timing, and never quote a first-frame number as a benchmark.

Everything is slow on a big image

What it means

Work scales with pixel count, and pixel count with the square of the linear dimension: a 24-megapixel photograph is roughly twelve times the work of a 1080p frame for the same operation. A 1080p BGR frame is about 6 MB, so a copying loop at 30 fps moves 180 MB/s before doing anything useful.

What to do

Resize before you detect, not after: detect at a smaller scale and scale the boxes back up. Setting `minSize` on a Haar detector is the cheapest speed-up available, because it truncates the pyramid. Then check you are not fighting yourself — OpenCV parallelises its own kernels, so a program that fans pipelines across its own executor should cap `org.opencv.core.Core.setNumThreads` before the two pools oversubscribe the machine. Chapter 35 is the whole story, with the harness that gates every claim in it.

The camera opens but delivers nothing

What it means

A webcam is not ready the instant `open` returns. Auto-exposure, auto-white-balance and auto-gain are closed loops on the device that need real frames to converge, so `open` then `snapshot` produces a black or badly underexposed image and *reports it as a success*. There is no property to poll for "converged".

A stalled network stream looks similar and is not the same. `read` blocks in native code with no timeout of its own, and the ones `CaptureOptions.withTimeout` sets are advisory: FFmpeg and GStreamer honour them, V4L2, AVFoundation and the built-in MJPEG reader ignore them, and nothing reports which you got.

What to do

For a dim room or a slow device, raise the warm-up: `CaptureOptions(warmupFrames = Some(20))`. The default is `None`, meaning five frames for a camera index and zero for a file — a file has no exposure loop, and discarding frames there would silently skip content. For dropped frames on a live camera, `attemptsPerFrame` is the bound that rides them out; it defaults to 3 on `Camera.foreach` and 1 on `Video.frames`, where

the first `false` is end-of-file. It is a bound and not a retry-forever, so a dead camera ends the loop instead of hanging the thread.

Calibration will not converge

What it means

`Left(CvError.CalibrationFailed)` names the count. With `minViews = 10` over a folder of forty captures it reads `need at least 10 views showing the whole 9×6 inner grid, but only 3 of 40 did`. `fromChessboard` silently skips views where the board is not fully visible, so an entire capture folder is a fine argument; only when fewer than `minViews` survive does it fail. A large `reprojectionError` is the same problem one step further on — the solver converged, but not on much.

What to do

The failure is almost always the capture, not the solver. The board must be *wholly* in frame, rigid — a sheet of paper that flexes ruins the run — and seen from a range of angles, which is what makes the problem well-posed. Ten to fifteen good views is where the numbers become trustworthy; `minViews` defaults to 3, a floor for seeing it work rather than a production setting. Judge the result by its RMS: under 0.5 px excellent, 0.5–1 px fine to ship, 1–2 px marginal, above 2 px recapture. And the intrinsics are in pixels at `imageSize` — calibrate at 1920×1080, run at 960×540, and every number is out by a factor of two. Chapter 31 has the capture procedure.

Filing a bug that can be fixed

Most of what makes a vision bug reproducible is environment, and none of it is visible from the report unless you write it down. The bug template asks for four lines of it: the scalacv version, `java -version`, the platform and bytedeco classifier you depend on, and the OS. Those four decide which OpenCV build was running, which is frequently the whole answer.

Then the reproduction, and generated inputs beat attached ones. `Image.blank`, a drawn rectangle and a synthetic gradient reproduce on the maintainer's machine with no download — the project ships no image assets for the same reason. If the failure needs a specific photograph, say what is distinctive about it (16-bit, CMYK, an exotic `.tiff` variant), because that property is usually the bug. If the JVM crashed, attach the `hs_err_pid*.log`; `CONTRIBUTING.md` is blunt about why. And paste the whole scalacv message.

A feature request has its own template, with one question in it that is not decoration: does the OpenCV type you want wrapped own native memory, and does it expose a public `release()`? Three of the 188 do, and the answer decides whether the wrapper is a one-line `given` or a `Releasable.nativeHandle` bridge with a disarm step — which is most of the work.

Where the reference material lives

This chapter closes Part VI, and with it the running narrative; what follows is reference. Appendix A, *Dropping to OpenCV Java*, is the one to read next if any section above ended with "borrow the raw `Mat` and call it yourself". It sets out that borrowing contract in full: which side of `image.mat` owns the pixels, how to adopt a raw `Mat` back with `Image.wrap`, and which `org.opencv.*` calls need a `Managed.scope` to hold their out-parameters.

PART VII

Appendices

The escape hatch to raw OpenCV, and the lookup tables — operations, typed constants, sample data, and the vocabulary.

APPENDIX A

Dropping to OpenCV Java

The map of what is underneath, and how to go down to it and come back without leaking.

Sooner or later you will want a function this library does not have. `Image` carries 65 public methods and the mid-level `Mat` extensions in `Ops.scala` — with the photo effects in `Effects.scala` and the deskew pipeline in `Deskew.scala` — add 45 more, which between them cover the operations people reach for repeatedly — and that is a small fraction of what `imgproc`, `core`, `photo`, `calib3d`, `features2d`, `video`, `dnn` and `objdetect` expose. The gap is not a defect waiting to be closed. A wrapper that tried to cover all of OpenCV would be a worse wrapper, so the interesting question was never "does scalacv have `distanceTransform`?" It is: what does it cost you to call the one it does not have, and what do you have to know to not corrupt memory doing it?

The answer this library gives is that it wraps the official `org.opencv.*` Java API rather than reimplementing it. There is no shadow binding, no private JNI layer, no second `Mat`. `Image` holds a `Managed[Mat]`; the `Managed[Mat]` holds an `org.opencv.core.Mat` from the same jar the OpenCV documentation describes. Anything in the OpenCV 4.13 Java reference is therefore reachable from scalacv code without ceremony, without a build change, and without leaving the pipeline you are already in.

What it does *not* give you is the memory model for free. The moment a raw call hands you a bare `Mat`, you are back in the world Chapter 5, *Lifetimes: Managed, Releasable, and Scope*, exists to get you out of: an object whose forty on-heap bytes conceal megabytes the collector cannot see, with no owner and no scope. This appendix is one instruction repeated in five settings — wrap the thing before the next line runs — plus what you need to read a C++ signature and land on the right Java overload.

Three altitudes

There are three levels, and picking the right one *per step* rather than per file is the skill.

Table A-1. What each level gives you, and what it costs.

Level	What you hold	Who frees it	The cost
High — `Image`	An owned image, chained by verbs	The chain: every transform releases its receiver, every terminal releases the result	Move semantics.

			You cannot use one `Image` twice without a `.copy`, and the knobs the verbs do not expose are not reachable from here.
Mid — `Managed[Mat]` and the `Mat` extension ops	One owned Mat at a time	`use`, `pipe`, `Managed.scope`, or you	You thread the handles. Nothing stops you from dropping one on the floor except `pipe` being shorter

			to type than the alternative.
Low — raw `org.opencv.*`	Whatever the Java binding hands back	You, entirely	Untyped constants, output parameters, a `finalize()` that will not save you, and no guard between a spent pointer and a SIGSEGV.

The levels are not sealed tiers you commit to at the top of a file. One pipeline can start on `Image`, borrow the Mat for a single `Imgproc` call the library does not wrap, and rise back to `Image` for the write. That round trip is the normal shape, and the rest of this appendix is those moves, written out.

It is worth saying plainly what dropping a level does *not* buy: speed. The obvious reason to go low is to reuse destination buffers across frames instead of allocating one per stage, and that optimisation was measured on this project's `gray → blur → canny` chain before it was designed. Reusing three preallocated destinations against fresh allocation came out at −4% at 640×480, −0.8% at 3840×2160, and **+0.4%** — reuse *slower* — at 1920×1080. A result whose

sign flips across the sweep is not a result. Go low because a call is missing, not because you are hoping for microseconds; Chapter 35, *Performance*, has the wins that are real.

Going down: `mat` borrows, `managed` hands over

An `Image` owns exactly one `Managed[Mat]`, and there are three doorways between it and everything below. Which one you want depends on a single question: are you *borrowing* the Mat for the length of an expression, or *taking* it?

Table A-2. The three doorways, and who is on the hook afterwards.

Doorway	Direction	Ownership, and what you must do
`img.mat`	Down, a borrow	Stays with the `Image`. Read it, pass it to a detector, run an `Imgproc` function against it — and do **not** release it, because the `Image` will.
`img.managed`	Down, a handover	Moves to the returned `Managed[Mat]`, and the `Image` is spent. Release the `Managed`, or `use` it.
`Image.wrap(managed)`	Up, a handover	Takes a `Managed[Mat]`, not a bare `Mat`: wrap the raw pointer with `Managed(mat)` first. Ownership moves into the new `Image`, so do not also release the `Managed` yourself.

`img.mat` is the one you want for almost every raw call. It is `handle.get` — a borrow with the spent-handle check in front of it, so using it after the `Image` has been consumed throws `IllegalStateException` in Scala rather than segfaulting in native code.

Example A-1. A raw `Core` call on a borrowed Mat.

```
val img = Image.blank(64, 64)

// org.opencv.core.Core.mean is not wrapped. Call it on the borrowed Mat.
val average = org.opencv.core.Core.mean(img.mat)

img.close() // this frees the Mat; `average` is already a plain cv.Scalar, safe
to keep
```

The mistake to see once is the one where you decide to be tidy about the Mat you were lent:

Example A-2. Wrong: releasing a Mat you only borrowed.

```
// WRONG — `mat` is a loan, not a gift.
val img = Image.blank(64, 64)
val average = org.opencv.core.Core.mean(img.mat)
img.mat.release()  // frees the buffer the Image still believes it owns
img.close()        // frees it again
```

> **NATIVE MEMORY**
> `img.mat` hands back the same pointer the `Image` holds, so releasing it yourself means the `Image`'s own `close()` frees that pointer a second time. `Managed` cannot protect you here: its compare-and-set guarantees that *it* releases once, and you went around it. A double `delete` is undefined behaviour that merely *often* survives — worse than a crash, because your test suite passes. If the Mat must outlive the `Image`, take `img.managed`, or `img.copy.managed` for an independent copy.

A subtler version of the same error catches people who have internalised move semantics but write the borrow on one line:

Example A-3. Wrong: a transform in the middle of a borrow.

```
import org.opencv.core.Mat
import org.opencv.imgproc.Imgproc

// WRONG — `.gray` is a transform: it consumes `img` and returns a NEW Image
// that nothing here has a name for, and so nothing closes.
val corners = Mat()
Imgproc.cornerHarris(img.gray.mat, corners, 2, 3, 0.04)
img.close() // a no-op: `img` was already spent by `.gray`
```

Give the transform's result a name and close *that*. `.mat` is a query and `.gray` is not, so `img.gray.mat` spends one `Image` and abandons another; Chapter 4, *The Image Type*, has the full account of which methods borrow and which consume.

The mid level, and the contract it writes down

`Ops.scala` declares an ownership contract in its header comment, worth quoting because every mid-level call in the library depends on it holding:

> *Every operation in this file is pure with respect to its receiver: it allocates a fresh destination Mat, writes the result there, and hands that back as a `Managed[Mat]` that the caller now owns and must release. The receiver is never written to, never released, and never aliased into the result — the two Mats have different `dataAddr()`s, so releasing one cannot invalidate the other.*

Three consequences follow. An op can be applied to a *borrowed* Mat with no transfer ceremony — a frame `Video.frames` reuses across iterations, a detector's input, `img.mat` — because `cvtColor` will not touch it. There are no in-place variants, so no call site has to be read twice to work out whether it mutated something; the scaladoc commits to the fact that if one is ever added it will say so in its name and return `Unit`. And an op that takes a *second* Mat borrows that too: `addWeighted(alpha, other, beta)`, `absdiff(other)`, `masked(mask)`, `inpaint(mask, radius)` and `seamlessCloneInto(background, mask, center)` all leave their extra operand as they found it.

The corollary is the one leak the mid level makes easy to write. Every stage returns a Mat you own, so a two-stage pipeline produces two, and the intermediate has to go somewhere:

Example A-4. Wrong: `use` on a middle stage strands the result.

```
// WRONG — `use` frees the blur output, then hands back the canny result's bare
// Mat, stripped of the Managed that owned it. That Managed is now unreachable,
// so nothing will ever free the canny output.
val edges: Mat = src.gaussianBlur(Size(5, 5), 1.5).use(_.canny(50, 150).get)
```

`pipe` exists for exactly that shape. It is defined on `Managed[Mat]`, feeds the wrapped Mat to the next stage, and releases it in a `finally` once that stage has produced its own output — so the intermediate can neither be leaked nor used after the chain has moved on, even if the stage throws.

Dropping the `.get` — `use(_.canny(50, 150))` — is in fact correct: the canny `Managed` travels out of the block intact and the type of `edges` becomes `Managed[Mat]` rather than `Mat`. That is the uncomfortable part. The safe version and the leaking version differ by four characters and the compiler accepts both, so the shape worth writing is the one that has a name.

Example A-5. Right: `pipe` between Mat stages, `use` at the terminal.

```
import org.opencv.core.Core

val edgePixels: Either[CvError, Int] =
  Image.reading("photo.jpg") { img =>
    img.mat.cvtColor(ColorConversion.BgrToGray)  // mid-level extension ->
Managed[Mat]
       .pipe(_.gaussianBlur(Size(5, 5)))
       .pipe(_.canny(80, 160))
       .use(Core.countNonZero(_))                // terminal: an Int, so `use`,
not `pipe`
  }
```

The division of labour is mechanical: `pipe` when the next stage returns another `Managed[Mat]`, `Managed.use` when it returns anything else — `Core.countNonZero` for a count, `findContours` for a `Seq[Contour]`, `Images.encode` for a byte array. Watch the return type on that last one: `encode` hands back its own `Either[CvError, Array[Byte]]`, so putting it inside `Image.reading` leaves you holding two nested `Either`s to flatten, which is usually a sign the encode belongs outside the block. `Mats.chain` is the n-stage form, a fold of `pipe`. It *borrows* its source and never releases it, which is what lets you hand it a frame you do not own.

Example A-6. `Mats.chain` on a borrowed frame.

```
Mats.chain(frame)(
  _.cvtColor(ColorConversion.BgrToGray),
  _.gaussianBlur(Size(5, 5), 1.5),
```

```
  _.canny(50, 150)
)
```

Not everything at this level returns a `Managed[Mat]`. The `Mat` extensions in `Draw.scala` return `Unit` and mutate the receiver, as the naming rule promises; the ones in `Contours.scala` and `Hough.scala` return plain immutable Scala data and free every native object they allocated on the way. Appendix B, *Operations Reference*, lists which is which.

> **NOTE**
> `Mats.chain` is the only public member of `Mats`. `produce`, `grayscale`, `column` and the rest are `private[scalacv]`: they are where the ownership contract is enforced, in one place, and exposing them would expose the ability to break it.

Coming back up

A raw call hands you a bare `Mat`, and from that instant it is a native allocation with no owner. Give it one before anything else happens: `Managed(mat)` at the mid level, `Image.wrap(Managed(mat))` at the high level, which hands the chaining API back with it.

Example A-7. Adopting a raw result, at both levels.

```
import org.opencv.core.{CvType, Mat}
import org.opencv.imgproc.Imgproc

val raw = Mat()
Imgproc.Laplacian(src.mat, raw, CvType.CV_16S) // a borrowed Mat in, a Mat you
own out

val owned: Managed[Mat] = Managed(raw)         // freed exactly once, from here
on
val encoded = Image.wrap(owned).bytes(".png")  // ...or lift it and keep chaining
```

Ownership transfers *into* the `Image`, so do not also release the `Managed`; `bytes` is a terminal and releases it for you. `img.managed` is the same handover run backwards.

> **CAUTION**
> Never leave a raw `Mat` unwrapped past the next line. If an exception fires between the allocation and the wrap — and the native call is the likeliest thing to throw — the buffer leaks, and nothing will reclaim it in a timeframe that matters. The library holds itself to this: `Mats.produce` releases its half-built destination before rethrowing, so a failed operation leaks nothing.

The full recipe for an unwrapped call

Calling something OpenCV has but scalacv does not is five steps:

1. Borrow the input Mat with `img.mat`, or produce it with mid-level ops.
2. Open a `Managed.scope` for the temporaries. Anything the call needs and nothing hands back — intermediate images, kernels, output vectors — is registered with `own` the moment it is constructed, so a throw anywhere releases everything acquired so far, in reverse order.
3. Allocate the destination and make the raw call inside `Cv.orThrow("name")`, so a `CvException` arrives as a `CvError.NativeCall` naming the operation instead of as an opaque C++ message. Chapter 6, *The Error Model*, has the rest.
4. Return **plain data** from the scope, or a value allocated outside it. Nothing the scope owns may escape — the same rule `use` carries, applied to a group.
5. Wrap anything going onward in `Managed` or `Image.wrap`.

Here is all five on a call the rest of this book never uses: `Imgproc.distanceTransform`, which replaces every non-zero pixel of a binary image with its distance to the nearest zero pixel — the standard way to find the thickest part of a blob. Its input must be a single-channel 8-bit mask.

Transliterated straight from the C++ tutorial, it has three independent defects:

Example A-8. Wrong: three defects, none of which the compiler sees.

```
import org.opencv.core.Mat
import org.opencv.imgproc.Imgproc

// WRONG.
val src = Image.read("shapes.png").toOption.get
val grey = Mat()
Imgproc.cvtColor(src.mat, grey, Imgproc.COLOR_BGR2GRAY)
val bin = Mat()
Imgproc.threshold(grey, bin, 128, 255, Imgproc.THRESH_BINARY)
val dist = Mat()
Imgproc.distanceTransform(bin, dist, Imgproc.DIST_L2, 3)
Image.wrap(Managed(dist)).write("dist.png")
src.mat.release()
```

`grey` and `bin` are never freed — two full-frame buffers per call, unbounded across a loop. `src.mat.release()` frees a Mat the `Image` still owns, and `src` is never closed. And `dist` comes back `CV_32F`, holding distances in pixels rather than an image: a blob twenty pixels thick peaks at ten, not at 255, so half the job — turning the measurement into something displayable — has not happened. `Image.toBufferedImage` rejects a Mat in that state and `colorMap` aborts in native code on it. The missing step is `normalize`, and the reason it is easy to forget is that no signature here mentions it.

Example A-9. Right: scoped temporaries, a named native call, plain data out.

```
import org.opencv.core.{Core, Mat}
import org.opencv.imgproc.Imgproc
```

```
val thickest: Either[CvError, Double] =
  Image.reading("shapes.png") { src =>
    Managed.scope: own =>
      val grey = own.adopt(src.mat.cvtColor(ColorConversion.BgrToGray))
      val bin = own.adopt(grey.threshold(128)._1)
      val dist = own(Mat())
      Cv.orThrow("distanceTransform"):
        Imgproc.distanceTransform(bin, dist, Imgproc.DIST_L2,
Imgproc.DIST_MASK_5)
      Core.minMaxLoc(dist).maxVal // a Double — plain data, safe to leave the
scope
  }
```

Two details carry the weight. `own(...)` is for a bare object you constructed; `own.adopt(...)` is for one that arrives already wrapped, and it takes over that handle rather than building a second one around the same pointer. And the value leaving the block is a `Double`, read out of native memory before the scope closes over it.

To keep the distance map itself, the escaping value has to be allocated outside the scope's ownership — which a mid-level op does for free, because it hands back a `Managed` the scope was never told about:

Example A-10. An owned result escaping a scope, legitimately.

```
// Same imports and the same `src` as the listing above.
val map: Managed[Mat] =
  Managed.scope: own =>
    val grey = own.adopt(src.mat.cvtColor(ColorConversion.BgrToGray))
    val bin = own.adopt(grey.threshold(128)._1)
    val dist = own(Mat())
    Cv.orThrow("distanceTransform"):
      Imgproc.distanceTransform(bin, dist, Imgproc.DIST_L2, Imgproc.DIST_MASK_5)
    // Allocated by a mid-level op, so NOT one of the scope's objects: it may
escape.
    dist.normalize().pipe(_.colorMap(Colormap.Jet))
```

`normalize` defaults its `depth` to `OutputDepth.Unsigned8` rather than to OpenCV's own `dtype = -1`, and that default is doing real work here: with `-1` a `CV_32F` input rescaled to `[0, 255]` comes back `CV_32F` holding the values 0 to 255, which is the same trap the previous listing fell into wearing a different hat. `OutputDepth.SameAsSource` is there for the case that genuinely wants it — rescaling a float image into `[0, 1]` for a model input, where 8 bits would collapse the range onto 256 levels. Departing from the C++ default like this is the kind of judgement you give up by going low; the reasoning behind each such departure is in Appendix C, *Enums and Types Reference*.

> **WARNING**

> `Point`, `Size`, `Rect` and `Scalar` are Scala case classes copied across the boundary, and their `toCv` and `from` conversions are `private[scalacv]` — your code cannot call them. When a raw signature wants an `org.opencv.core.Rect`, construct one directly, `org.opencv.core.Rect(r.x, r.y, r.width, r.height)`, and convert the result back by hand. The copying is deliberate: OpenCV's geometry types are mutable Java objects with public fields, so a `Seq[cv.Rect]` from a detector is a set of live handles whose contents can change underneath you.

Reading the C++ documentation, landing in Java

Every reference page you will search for is a C++ page. Three translation rules get you from there to the Java binding, mechanically enough to apply without checking.

Namespaces become classes, and capitalisation is preserved exactly

`cv::GaussianBlur` is `Imgproc.GaussianBlur` — capital G and all. `cv::cvtColor` is `Imgproc.cvtColor`, lower case, because that is how C++ spells it. The binding generator does not normalise, so `Imgproc.Canny`, `Imgproc.Sobel`, `Imgproc.Laplacian` and `Core.LUT` sit next to `Imgproc.resize`, `Core.mean` and `Core.normalize`, and `Core.bitwise_not` keeps its underscore. The module a function lives in decides its class: `imgproc` → `Imgproc`, `core` → `Core`, `photo` → `Photo`, `calib3d` → `Calib3d`, `dnn` → `Dnn`, `objdetect` → `Objdetect`. If a call is not where you expect it, it is almost always because you guessed the module rather than the name.

Outputs are parameters, not return values

A C++ signature like `void GaussianBlur(InputArray src, OutputArray dst, Size ksize, double sigmaX)` becomes a Java method returning `void` and taking a `Mat dst` you allocated. `InputArray` means borrowed; `OutputArray` means "I will `create()` this to the size and type I need, then fill it"; `InputOutputArray` means it reads *and* writes the same Mat, which is the one shape to be careful of, because it is a mutation the type system does not show you. A function that has genuinely more than one output — `distanceTransformWithLabels`, `solvePnP` — takes one `Mat` per output, which is precisely the case `Managed.scope` was written for.

Constants are bare `int`s on the module class

`Imgproc.COLOR_BGR2GRAY`, `Imgproc.THRESH_BINARY`, `Core.NORM_MINMAX`, `Photo.INPAINT_TELEA`, `Imgproc.DIST_L2`. They are `public static final int`, they live on the same class as the functions that consume them, and — this is the whole reason Chapter 8, *Typed Constants and Colour Spaces*, exists — nothing stops you passing a border type where an interpolation was wanted. Both are ints; both compile. When you come back up, come back to the enums.

> **The overload that does not exist**

C++ default arguments do not survive the binding as defaults. `Imgproc.threshold` is a single five-argument overload with none at all — which is why the library's own `threshold` extension spells out all five rather than layering Scala defaults over Java ones that are not there.

Where C++ *does* have defaults, the generator emits one overload per trailing prefix, and the one you want may be two arities up from what autocomplete offers first. `Imgproc.arrowedLine` is the sharp case: only its eight-argument overload reaches `tipLength`, and the seventh parameter you must pass to get there is `shift`, which almost nothing wants — the library's `drawArrow` passes `0` for it and says so in a comment, because a bare `0` in the middle of a call is otherwise unreadable. Count the C++ parameters, subtract the defaults you are happy with, and pick that arity.

Mat basics, for when you must

A `Mat` is a header plus a reference-counted pixel buffer. Two numbers describe its shape and one describes its element type, and the spellings are not the ones you expect.

Type codes

A type code is `CV_<bits><U|S|F>C<channels>`: `CV_8UC3` is eight-bit unsigned, three channels, the default BGR image. The *depth* is the type without the channel count, which is why `CvType.depth` and `CvType.channels` are separate calls over the code `Mat`'s `type()` method returns — and why the library checks depth rather than type when it wants "any 8-bit image". (`type` is a Scala keyword, so that call is spelled with backticks around the method name.) `CvType.typeToString` turns a code into something readable, and it is what the library's own error messages print rather than a bare integer.

Table A-3. The type codes that appear in this library, and where.

Code	Element	Where you meet it
`CV_8UC3`	8-bit unsigned ×3	The default image. `Image.blank(w, h)` and every colour read.
`CV_8UC1`	8-bit unsigned ×1	Greyscale, and every binary mask: the output of `canny`, `inRange` and `threshold`, and what `findContours` wants (it also accepts `CV_32SC1`) and `houghLines` insists on.
`CV_8UC4`	8-bit unsigned ×4	BGRA. `Image.blank(w, h, channels = 4)`; flattened to BGR on the way to AWT.
`CV_16S`	16-bit signed	`OutputDepth.Signed16` — a `sobel` that keeps its negative lobe instead of clipping it to zero.
`CV_32F`	32-bit float ×1	Distance and disparity maps, unnormalised filter responses, a `MatOfPoint2f` converted for `minAreaRect`.
`CV_64F`	64-bit float ×1	Small solver matrices — a camera matrix, a rotation, a translation.

Rows and columns, width and height

`Mat` is row-major and constructed row-count first, `Mat(rows, cols, type)`; `Size` is constructed extent-first, `Size(width, height)`. A 640×480 image is therefore

`Mat(480, 640, CV_8UC3)` and `Size(640, 480)`, and the two sit next to each other in real code. `Image` papers over this — `width` is `cols`, `height` is `rows`, and `Image.blank(width, height, …)` builds `Mat(height, width, …)` — but the moment you allocate a Mat yourself the order is yours to get right, and getting it wrong on a non-square image produces something OpenCV processes happily and you do not recognise.

Submatrices alias their parent

`m.submat(rect)` does not copy. Neither do `rowRange`, `colRange`, `row` or `col`. They return a new `Mat` header pointing into the parent's buffer, which is the single most useful and most dangerous fact about the type: writing through a view writes through to the parent, and reading a view after the parent's buffer is gone reads freed memory.

> **NATIVE MEMORY**
> A view and its parent share one allocation. A view that outlives its parent is a read of freed memory, with no guard anywhere to catch it, because the view's handle looks perfectly alive. If a region has to survive the image it came from, `clone()` it. That is exactly what `Image.crop` does: it takes the `submat`, clones it, and releases the view before the parent, so the caller gets an independent copy and no header is stranded — and the `Managed.use` around the view makes that hold even when `clone` throws.

Aliasing has a second consequence. A submat is usually not *continuous* — its rows are strided across the parent's wider ones — while a bulk `Mat.get(0, 0, bytes)` reads a contiguous run. `toBufferedImage` checks `isContinuous` and clones only when it is false; do the same in your own bulk reads.

Where the Java bindings are wrong, and what the library does about it

Some of what this library does is not convenience. It is repair.

The largest item is the one Chapter 5 is built around: of the 188 `org.opencv.*` types that own native memory, exactly three — `Mat`, `VideoCapture` and `VideoWriter` — expose a public `release()`. `CascadeClassifier`, `Net`, `QRCodeDetector`, `FaceDetectorYN`, `KalmanFilter` and 180 others expose only a `private static native void delete(long)` and an unconditional `finalize()`. `Releasable.nativeHandle` reads the binding's own `nativeObj` field reflectively, **disarms the finalizer first** by zeroing it, and only then calls `delete(long)` through a cached `MethodHandle`. The order is not fussiness: a finalizer that later runs against a pointer you already freed is a double free, and this project found that one as a `SIGSEGV` inside `Java_org_opencv_objdetect_FaceDetectorYN_delete` on the JVM's own `Finalizer` thread. Measured, 4000 leaked `KalmanFilter`s reached 54 GB against 86 MB released.

The second item is a leak the library documents rather than fixes. The generated binding for `polylines`, `fillPoly` and `drawContours` runs its input through `Converters.vector_vector_Point_to_Mat`, which allocates one `Mat` per polygon plus one for

the outer vector and releases none of them. The library's own `withPolygons` helper — file-private in `Draw.scala`, and the single place those polygons are built — frees the `MatOfPoint`s *it* allocated, in a `finally`, and its scaladoc states flatly that this does not make the call leak-free: the residue is upstream in the official Java API and cannot be reached from here without reimplementing the converter. It is bounded per call and unbounded across a video loop — Chapter 14, *Drawing and Annotation*, has the mitigation.

The rest are smaller divergences worth knowing before you trust a C++ page:

Table A-4. Places the Java binding does not behave like the C++ documentation.

Call	The divergence, and what the library does
`findContours`	The C++ version modifies its input; the Java binding copies internally, so it does not. The library still treats a thresholded image as consumed, because the habit is cheaper than the exception.
`imread` / `imwrite`	`imread` reports a missing file, a directory, an empty file and undecodable bytes identically, as a `Mat` whose `empty()` is true; `imwrite` collapses the same range onto a bare `false`. `Images.read` and `Images.write` call neither: the JVM reads and writes the bytes and OpenCV is left only the codec work. That sidesteps the `GetStringUTFChars` narrowing that breaks every non-ASCII path on Windows, and as a side effect lets `CvError` name which failure happened.
`FaceDetectorYN.detect`	Returns an `int` status flag — `1` for "the network ran" — not a face count. Read as a count it reports one face per successful call. No faces gives a 0×0 Mat, not an N×15 Mat with zero rows. Chapter 24, *Face Detection*, has all four traps.
`Imgproc.Sobel` with `ddepth = -1`	On an 8-bit source every negative derivative clips to zero and half of each edge silently disappears. `OutputDepth` exists so the default cannot be chosen by accident.
Geometry types	`cv.Rect`, `cv.Point` and friends are mutable objects with public fields, handed back live from detectors. The library copies them into immutable case classes at the boundary.

> **TIP**
> When a detector you manage yourself refuses to free, the thrown `CvError.NativesMissing` names the exact `--add-opens` flag, computed from the offending class's own module and package rather than guessed — typically `--add-opens org.bytedeco.opencv/org.opencv.objdetect=ALL-UNNAMED`. The module is `org.bytedeco.opencv` and the package is `org.opencv.objdetect`, which do not share a prefix; that is the pair people get wrong, and the reason the message computes it rather than asking you to. It is never `java.base/java.lang` — the field being opened is OpenCV's own `nativeObj`, not a JDK internal. Chapter 42, *Troubleshooting*, has the fix.

When to stay high

The rule is short: if `Image` has it, use `Image`. Not because the lower levels are dangerous in some vague way, but because each level down moves one guarantee from the library to you — release exactly once, then the typed constant, then the ownership of every temporary — and those obligations are worth collecting one at a time, at the one step that needs them, rather than for a whole file.

So drop exactly one level, at exactly the step where the level above does not expose the knob you need, and come straight back up. `mat` borrows, `Managed` adopts, `Image.wrap` lifts. Reaching for `org.opencv.*` is a normal thing to do rather than an admission that the wrapper failed.

Appendix B, *Operations Reference*, is the other half of this map: every wrapped operation, at which level it lives, what it consumes and what it hands back — so that before you write a raw call you can check whether the call is already there.

APPENDIX B

Operations Reference

Every operation the library exposes, grouped by the job it does.

There are two moments when you want a reference like this one. The first is when you know what you want to do to an image and cannot remember what the library calls it — you want the word for "stretch the values so a float disparity map is visible", and you want it in five seconds, not after scrolling a Scaladoc index. The second is the more dangerous one: you are fairly sure a method exists, you write the call, and it compiles because Scala found something else with a similar name on a different tier. `boxBlur` and `blur` are different filters. `Image.adaptiveThreshold` and `Mat.adaptiveThreshold` take their arguments in a different order on purpose. `estimatePose` means one thing for a human body and something else entirely for an ArUco marker.

So the tables below answer three questions per row, and nothing else. Does this operation exist, spelled this way? What does it take, and what comes back? Which chapter explains why you would want it? The parameter lists are abbreviated to the shape you need at a call site: defaults are shown where getting one wrong changes the result, enum defaults are written by their case name alone (`shape = Rect`, not `shape = MorphShape.Rect`, which is what the source says), and the trailing `border` / `lineType` arguments that almost nobody passes are often elided with an ellipsis. Appendix C has the full enumerations; the generated Scaladoc has the exact declarations.

Two tiers appear throughout, and the Signature column always says which. A row whose signature begins `img.` is a method on `Image`, the owned, move-semantic type from Chapter 4. A row beginning `mat.` is an extension method on `org.opencv.core.Mat`, brought into scope by `import scalacv.*` and returning a `Managed[Mat]` you own. The `Image` tier is a curated subset: it exposes the operations a pipeline usually wants, with the arguments a pipeline usually tunes. Everything the `Image` tier omits — `sobel`, `laplacian`, `boxBlur`, `absdiff`, every Hough transform, most of the drawing primitives — is on the `Mat` tier only, reachable through the borrowed `img.mat` in one step.

The grouping is by the job, not by the source file, because that is how you look something up. An operation that appears on both tiers gets one row with both signatures separated by a middle dot, since the difference between them is almost always a parameter or two rather than a difference in what happens to the pixels. Where the two tiers genuinely disagree — `adaptiveThreshold`'s argument order, `threshold`'s return type — they get a row each, adjacent, so the disagreement is visible rather than buried in a footnote.

What each group needs on the classpath

Every operation in this appendix ships in the core artifact, `com.worxbend::scalacv:0.4.2`, plus the platform natives from Chapter 2. There is exactly one exception, marked in its row: `img.draw(picture)` in the Drawing table comes from `com.worxbend::scalacv-graphs`. Nothing here needs `scalacv-vision`; the detector verbs that live there — `faces`, `qrCodes`, `arucoMarkers`, `estimatePose`, `detectHaar`, `markFaces`, `drawSkeleton`, `drawTracks` — are documented in Chapters 24 through 34 rather than repeated here, because their arguments are models and detectors rather than pixels.

Core uses `import scalacv.*`; the graphics extension also needs `import scalacv.graphs.*`. An import cannot conjure a module you have not declared as a dependency: if a call does not resolve, check the build file before you check the spelling.

How to read a row

The Signature column is abbreviated consistently, and knowing how saves you a trip to the Scaladoc. Parameter names are exact — they are what you will type in a named argument, and this library expects you to use named arguments wherever two adjacent parameters share a type. Defaults are shown only where the default is a decision: `depth = Unsigned8` is there because leaving it alone changes the pixel type of the result, while the `border = Reflect101` that trails half the filters is elided as an ellipsis because almost no caller sets it and the ones who do already know it exists. Enum defaults lose their type prefix, so a row reads `shape = Rect` where the source says `shape = MorphShape.Rect`. And return types appear only when they are not the obvious one: an `Image` method with no stated return gives you an `Image`, a `Mat` extension with none gives you a `Managed[Mat]`.

The What-it-does column carries the preconditions, because a precondition is the thing most likely to turn a correct-looking call into an exception at run time. They come in two kinds and the wording tells them apart. A rule stated about an *argument* — an odd kernel, a positive scale, a radius of at least 1, a range on a normalised parameter — is a `require` in the source: it throws `IllegalArgumentException` on the JVM side, before anything reaches JNI, and the message names the value you passed. A rule stated about the *pixels* — "`CV_8UC1` only", "needs 8-bit 3-channel input" — is almost never checked here, because the library would have to duplicate OpenCV's own type algebra to do it; hand one of those the wrong Mat and you get a `CvError.NativeCall` quoting a C++ assertion instead. Two exceptions are worth knowing: an empty Mat is rejected up front by `findContours` and by every drawing primitive, and the Hough transforms check `CV_8UC1` themselves, because handing them anything but a Canny result is common enough that the native message was worth intercepting. Chapter 6 draws the line; the tables assume it.

The See column names one chapter, the one that explains the operation rather than merely uses it. A few rows name two, separated by a comma, where the operation is introduced in one place and earns its living in another — `absdiff` is a `Core` call explained by motion detection, `undistort` is a geometry call explained by calibration. A letter means an appendix.

The four kinds, in one line each

Chapter 5 established the vocabulary; the tables assume it. A *query* borrows — the image survives the call and you get plain data back. A *transform* consumes the receiver and returns a fresh `Image`; the old handle throws `IllegalStateException` if you touch it again. A *draw* also consumes, but mutates the pixels in place rather than allocating a destination, which is why annotating a 4K frame costs no allocation per overlay. A *terminal* consumes and releases.

Everything in the tables below is a transform unless the row says otherwise. Where an operation takes a second image — a mask, a background, a blend partner — that second image is *borrowed*, never consumed, and closing it stays your job.

Constructing and destroying an image

The constructors are all on the `Image` companion; the destructors are `close`, the two terminals in the I/O table, and `managed`, which is a destructor only in the sense that it ends this `Image`'s ownership without freeing anything. Of the six ways in, `reading` is the synchronous scoped form: it closes on success, on failure and on exception, and it runs the body inside `Cv.attempt`, so a transform that throws deep in a chain comes back as a `Left` instead of escaping past a signature that promised an `Either`. Prefer it unless you have a reason to thread the ownership yourself.

Table B-1. Making an Image, and ending one.

Operation	Signature (abbreviated)	What it does	See
`blank`	`Image.blank(width, height, color = Black, channels = 3)`	A filled canvas. `channels` must be 1, 3 or 4; a non-positive extent throws `IllegalArgumentException`.	4
`read`	`Image.read(path, flags = Color): Either[CvError, Image]`	Reads a file. `Left` distinguishes an unusable path, a missing file, a directory, an empty file, an unreadable one and undecodable bytes.	7
`decode`	`Image.decode(bytes, flags = Color): Either[CvError, Image]`	Decodes bytes already in memory — an HTTP body, a BLOB, a fixture.	7
`reading`	`Image.reading(path, flags = Color)(use): Either[CvError, A]`	Reads, runs `use`, closes on every path. The body runs inside `Cv.attempt`, so a transform's throw becomes a `Left`.	5

`wrap`	`Image.wrap(handle: Managed[Mat]): Image`	Adopts an existing handle. Ownership transfers; do not also release it yourself.	4
`fromBufferedImage`	`Image.fromBufferedImage(bi): Image`	From AWT. Always 3-channel BGR.	7
`copy`	`img.copy: Image`	Query. An independent deep copy — the way to branch a chain.	4
`managed`	`img.managed: Managed[Mat]`	Spends this `Image` and hands the handle on. Frees nothing.	5
`close`	`img.close(): Unit`	Releases now. Idempotent, and `AutoCloseable`, so `Using` works.	5

NATIVE MEMORY

`img.mat` borrows and `img.managed` transfers, and the difference is the whole of this appendix's safety story. A Mat obtained from `mat` is still owned by its `Image` — release it and the `Image`'s next call reads freed memory. A `Managed[Mat]` obtained from `managed` is yours: `use` it, `release()` it, or hand it back to `Image.wrap`. Letting one escape without any of those three is the one leak the type system will not catch for you.

Queries

Queries borrow. None of these spends the image, so you can read a dimension in the middle of a chain without taking a `copy` first, and none of them is affected by move semantics at all. The two that cross the boundary out of native memory — `toBufferedImage` and `contours` — copy rather than alias, which is why their results stay valid after the image they came from has been released.

Table B-2. Reading an image without consuming it.

Operation	Signature (abbreviated)	What it does	See
`width` `height`	`img.width: Int`, `img.height: Int`	Pixel dimensions, read from the Mat's `cols` / `rows`.	4
`size`	`img.size: Size`	The same pair as a `Size`, whose extents are `Double`.	4
`channels`	`img.channels: Int`	1 grey, 3 BGR, 4 BGRA.	4
`isEmpty`	`img.isEmpty: Boolean`	True for a Mat with no pixel data.	4
`mat`	`img.mat: Mat`	Borrows the raw handle for anything the `Image` tier does not wrap.	4, A
`toBufferedImage`	`img.toBufferedImage: BufferedImage`	A copy, for AWT and for notebook display. 8-bit only; 4 channels are flattened to BGR.	7

`contours`	`img.contours(retrieval = External, approximation = Simple): Seq[Contour]`	Outlines of a binary image, copied out of native memory as plain data.	12
`toString`	`img.toString: String`	`Image(640x480, 3ch)`, or `Image(<closed>)` once released — it never dereferences a freed handle.	4

Colour and channels

Colour conversions change the channel count to whatever the conversion implies, so `channels` after `gray` is 1 and after `convert(ColorConversion.BgrToBgra)` is 4. Everything in this group is a transform, and several of the tone operations are built from the ones above them: `gamma` and `posterize` are both a 256-entry lookup table, `saturate` is a blend against a grey copy, `sepia` and `temperature` are 3×3 colour matrices. That matters when you are counting allocations rather than lines of code — `saturate` costs two intermediate Mats, not one.

Table B-3. Colour space, tone and per-channel work.

Operation	Signature (abbreviated)	What it does	See
`gray`	`img.gray`	BGR to single-channel grey — `convert(BgrToGray)` under a shorter name.	8
`convert`	`img.convert(conversion)` · `mat.cvtColor(conversion)`	Any of the ten `ColorConversion` cases.	8
`toHsv`	`img.toHsv`	BGR to HSV, the space to threshold colour in.	11
`invert`	`img.invert` · `mat.bitwiseNot()`	Bitwise NOT: `255 - v` per channel for 8-bit.	8
`adjust`	`img.adjust(brightness = 0, contrast = 1.0)`	Linear brightness and contrast in one pass, via `convertScaleAbs`.	15
`convertScaleAbs`	`mat.convertScaleAbs(alpha = 1, beta = 0)`	Scale, take the absolute value, saturating-cast to 8-bit. The companion to a `Signed16` Sobel.	9
`normalize`	`img.normalize(min = 0, max = 255, depth = Unsigned8)`	Min-max stretch. The default depth is what makes a float or 16-bit result displayable; pass `SameAsSource` to keep precision. The `Mat` twin names the bounds `alpha` / `beta`.	15
`gamma`	`img.gamma(g)`	Gamma correction through a 256-entry LUT. `g < 1` darkens, `g > 1` lifts. Must be positive.	15

`saturate`	`img.saturate(factor)`	`0` is grey (still 3-channel), `1` unchanged, above that vivid. Negative throws.	15
`temperature`	`img.temperature(shift)`	Warm above zero, cool below. `shift` must lie in `[-1, 1]`.	15
`channel`	`img.channel(index)` · `mat.extractChannel(index)`	One channel as its own single-channel image. An out-of-range index throws.	8
`colorMap`	`img.colorMap(map)`	False-colours a single-channel image through one of the ten `Colormap` cases. Needs 8-bit input.	15
`equalizeHist`	`img.equalizeHist` · `mat.equalizeHist()`	Histogram equalisation. `CV_8UC1` only, so usually after `gray`.	9

Filtering and smoothing

The radius-based names on the `Image` tier all derive an odd kernel as `2 * radius + 1`, so `blur(2)` is a 5×5, and `medianBlur(1)` a 3×3. The `Mat` tier takes the kernel instead and validates it: `gaussianBlur` and `boxBlur` want a `Size` whose extents are odd and positive, with `Size(0, 0)` allowed for `gaussianBlur` alone, which then derives the kernel from the sigma; `medianBlur` wants the odd side length as a bare `Int`.

Table B-4. Smoothing, and the two filters that keep edges.

Operation	Signature (abbreviated)	What it does	See
`blur`	`img.blur(radius)`	Radius-based Gaussian. `radius = 0` is the identity but still spends the receiver; negative throws.	9
`gaussianBlur`	`img.gaussianBlur(kernel, sigmaX = 0, sigmaY = 0)` · `mat.gaussianBlur(kernel, sigmaX = 0, sigmaY = 0, border = Reflect101)`	Full control. A zero sigma means "derive it from the kernel"; a zero kernel means "derive it from the sigma"; both zero throws.	9
`boxBlur`	`mat.boxBlur(kernel, anchor = Point(-1, -1), border = Reflect101)`	Normalised box filter — a different family from `blur`, which is why it has a different name.	9
`medianBlur`	`img.medianBlur(radius)` · `mat.medianBlur(ksize)`	Kills salt-and-pepper noise without smearing edges. `radius` starts at 1; `ksize` must be odd and at least 3, so there is no do-nothing value.	9
`bilateralFilter`	`img.bilateralFilter(diameter = 9, sigmaColor = 75, sigmaSpace = 75)`	Edge-preserving smooth. Markedly slower than a Gauss-	9

		ian. A `diameter` of 0 or less lets OpenCV derive it.	
`edgePreserving`	`img.edgePreserving(strength = 60, detail = 0.4f)`	Flattens texture while holding edges — the basis of the painterly filters.	15
`sharpen`	`img.sharpen(amount = 1.0)`	Unsharp mask: adds back `amount` × (image − its blur). Overdo it and edges halo.	9

Edges and gradients

`canny` is the one everyone reaches for, and its two thresholds are both `Double` and silently swappable — name them at the call site. The derivative operators live on the `Mat` tier only, because their useful output is not 8-bit and the `Image` tier has no way to say so.

Table B-5. Derivatives and edge maps.

Operation	Signature (abbreviated)	What it does	See
`canny`	`img.canny(threshold1, threshold2, apertureSize = 3, l2Gradient = false)`	Hysteresis edge detection; the result is always `CV_8UC1`. `threshold1` is the weak level. `apertureSize` must be 3, 5 or 7.	9
`sobel`	`mat.sobel(dx, dy, kernelSize = 3, depth = SameAsSource, scale = 1, delta = 0, …)`	First or higher derivative. `kernelSize = -1` selects the 3×3 Scharr kernel. Leave `depth` alone on an 8-bit image and the negative lobe is clipped away.	9
`laplacian`	`mat.laplacian(kernelSize = 1, depth = SameAsSource, scale = 1, delta = 0, …)`	Second derivative. `kernelSize = 1` is the 3×3 aperture OpenCV special-cases.	9
`absdiff`	`mat.absdiff(other): Managed[Mat]`	Per-element `|self - other|`. `other` is borrowed. The basis of frame-difference motion detection.	21

Morphology

A morphological operation needs a structuring element, and both tiers build one for you from a radius: the element is `2 * radius + 1` on a side, in the `MorphShape` you name. The `Mat` tier adds `iterations`; both throw on a radius below 1.

Table B-6. Erosion, dilation and the compound operators.

Operation	Signature (abbreviated)	What it does	See

`erode`	`img.erode(radius = 1, shape = Rect)` `mat.erode(radius = 1, shape = Rect, iterations = 1)`	Shrinks bright regions; clears small bright specks.	9
`dilate`	`img.dilate(radius = 1, shape = Rect)` `mat.dilate(radius = 1, shape = Rect, iterations = 1)`	Grows bright regions; fills small dark gaps.	9
`morphology`	`img.morphology(op, radius = 1, shape = Rect)` `mat.morphology(op, radius = 1, shape = Rect, iterations = 1)`	The compound operators: `Open`, `Close`, `Gradient`, `TopHat`, `BlackHat`. There is no bare `close` method, because `close()` already releases.	9

Thresholding and masks

The `Image` tier's `threshold` discards the level OpenCV computed, which is fine for a fixed value and wrong for `Threshold.otsu()` — that number is usually the reason you called it. The `Mat` tier returns it as a `ThresholdResult`.

Table B-7. Binarising, masking and compositing.

Operation	Signature (abbreviated)	What it does	See
`threshold`	`img.threshold(value, maxValue = 255, kind = Binary)`	Fixed or automatic binarisation. Drops the computed level.	11
`threshold`	`mat.threshold(value, maxValue = 255, kind = Binary): (Managed[Mat], ThresholdResult)`	The same, keeping the level — which for `Threshold.otsu()` and `Threshold.triangle()` is the answer.	11
`adaptiveThreshold`	`img.adaptiveThreshold(blockSize = 11, c = 2.0, method = Gaussian, inverse = false)`	Per-neighbourhood threshold, for uneven lighting. `CV_8UC1` only; `blockSize` odd and at least 3.	11
`adaptiveThreshold`	`mat.adaptiveThreshold(maxValue = 255, method = Gaussian, blockSize = 11, c = 2.0, inverse = false)`	The same operation in OpenCV's own argument order. Use named arguments and the divergence stops mattering.	11
`inRange`	`img.inRange(lo, hi)` `mat.inRange(lo, hi)`	A `CV_8UC1` mask, 0 or 255, of the pixels whose every channel lies in range. Usually run on HSV.	11
`applyMask`	`img.applyMask(mask)` `mat.masked(mask)`	Keeps pixels where the mask is non-zero, blacks out the rest. The mask is borrowed.	11
`blend`	`img.blend(other, weight = 0.5)`	`this * weight + other * (1 - weight)`. The weight is *this* image's share;	15

		outside `[0, 1]` it throws. `other` is borrowed and must match in size and type.	
`addWeighted`	`mat.addWeighted(alpha, other, beta, gamma = 0)`	The unconstrained form: `self * alpha + other * beta + gamma`.	9

Geometry: resize, crop, rotate, warp

`resize` on the `Image` tier takes no interpolation argument and always uses `Interpolation.Linear`; `resizeTo` and `scale` take one. Sizes are `Double` and truncate toward zero on the way into native code, which is why a computed target that lands between 0 and 1 is rejected here rather than aborting inside OpenCV.

Table B-8. Moving pixels around the plane.

Operation	Signature (abbreviated)	What it does	See
`resize`	`img.resize(width, height)`	Absolute pixel size. Always linear interpolation.	10
`resizeTo`	`img.resizeTo(size, interpolation = Linear)` · `mat.resize(size, interpolation = Linear)`	Absolute resize with the interpolation of your choice. A target that truncates to under 1 pixel throws.	10
`scale`	`img.scale(factor, interpolation = Linear)` · `mat.scaled(fx, fy, interpolation = Linear)`	Factor-based. The `Mat` form takes independent axes and checks the result against the receiver's own extent, because OpenCV rounds half to even.	10
`crop`	`img.crop(rect)`	An independent copy, not an aliasing view. The rectangle must lie inside the image.	10
`flip`	`img.flip(how)` · `mat.flip(flip)`	Mirror: `Horizontal`, `Vertical`, `Both`.	10
`rotate`	`img.rotate(rotation)` · `mat.rotate(rotation)`	Lossless quarter turn — exact pixels, no interpolation.	10
`rotate`	`img.rotate(degrees, scale = 1.0)` · `mat.rotated(degrees, scale = 1.0, interpolation = Linear, …)`	Arbitrary angle, canvas expanded so no corner clips. A positive `degrees` turns the same way as `Rotation.CounterClockwise`, but it resamples, so prefer the quarter-turn form when the angle is a right angle.	10
`pad`	`img.pad(size, borderType = Constant, color = Black)`	A uniform border on all four sides.	10
`border`	`img.border(top, bottom, left, right, borderType = Constant, color = Black)` · `mat.border(…)`	Independent widths per side. Negative widths throw.	10

`undistort`	`img.undistort(intrinsics)` `mat.undistorted(intrinsics)`	·	Maps out the lens bend using calibrated `Intrinsics`. A plain copy when the distortion vector is empty.	31
`deskew`	`img.deskew(maxAngle = 45.0)` `mat.deskew(maxAngle = 45.0)`	·	Finds the dominant text tilt and rotates upright, filling the corners white. A skew beyond `maxAngle` is treated as a misread and left alone.	33

Photographic and stylisation

Five of these wrap `org.opencv.photo`, and the three stylisation ones — `stylize`, `sketch`, `enhance` — want 8-bit 3-channel input; hand one a greyscale image and it fails in native code rather than at the call site. `filter` is the composition point: a `Filter` is a named `Image => Image`, so your own are as first-class as the seventeen built-ins.

Table B-9. Looks, repairs and composites.

Operation	Signature (abbreviated)		What it does	See
`stylize`	`img.stylize(strength = 60, detail = 0.45f)`		A smooth painterly cartoon, via edge-aware smoothing.	15
`sketch`	`img.sketch(strength = 60, detail = 0.07f, shade = 0.02f)` `mat.pencilSketch(…)`	·	Colour pencil-sketch rendering.	15
`enhance`	`img.enhance(strength = 10, detail = 0.15f)` `mat.detailEnhance(…)`	·	Boosts local contrast and texture.	15
`sepia`	`img.sepia`		Sepia tone, through a 3×3 colour matrix.	15
`emboss`	`img.emboss`		Directional convolution — a raised-relief look.	15
`posterize`	`img.posterize(levels)`		Quantises to `levels` tones per channel. `levels` must be in `[2, 256]`.	15
`inpaint`	`img.inpaint(mask, radius = 3.0)`		Fills the region under the mask from its surroundings — a scratch, an object, a watermark. The mask is borrowed.	15
`seamlessCloneInto`	`img.seamlessCloneInto(background, mask, center)`		Poisson clone of this image into `background`. Both extra images are borrowed; the result is background-sized.	15
`filter`	`img.filter(f: Filter)`		Applies a named filter. `Filter.all` holds the seventeen built-ins, in or-	15

		der from `Filter.grayscale` to `Filter.dramatic`.	
`Filter`	`Filter(name)(run)` · `f.andThen(next)`	Names any `Image => Image` as a filter, and composes two.	15

Drawing

Drawing is the one family that mutates rather than allocating: OpenCV has no out-of-place rasteriser, and cloning a frame per overlay is exactly what a video loop cannot afford. The `Image` tier wraps five primitives and consumes the receiver each time; the `Mat` tier has the full set, returns `Unit`, and leaves the `Image` that lent you the Mat alive. That is how you draw a line on an `Image`: there is no `img.drawLine`, so reach through `img.mat`.

Table B-10. Rasterising onto an image you already own.

Operation	Signature (abbreviated)	What it does	See
`drawRect`	`img.drawRect(rect, color = White, thickness = Default)` · `mat.drawRect(…, lineType = Connected8)`	An axis-aligned rectangle. `Thickness.Filled` makes it solid.	14
`drawRects`	`img.drawRects(rects, color = Green, thickness = Default)`	Many rectangles in one pass — detector boxes, ROIs.	14
`drawCircle`	`img.drawCircle(center, radius, color = White, thickness = Default)` · `mat.drawCircle(…)`	A circle. A negative radius throws.	14
`drawText`	`img.drawText(text, at, color = White, scale = 1.0)` · `mat.drawText(text, at, color = White, font = Simplex, scale = 1.0, thickness = Default, …)`	Hershey vector text, anchored on the *baseline's* left end, not the top-left. Non-ASCII draws as `?`.	14
`drawContours`	`img.drawContours(contours, color = White, thickness = Default)` · `mat.drawContours(…)`	Renders what `findContours` returned. `Thickness.Filled` turns them back into a mask.	12, 14
`drawLine`	`mat.drawLine(from, to, color = White, thickness = Default, lineType = Connected8)`	A straight line. Coordinates outside the image are clipped, not rejected.	14
`drawArrow`	`mat.drawArrow(from, to, …, tipLength = 0.1)`	A line with a head at `to`, sized as a fraction of the line.	14
`drawPolyline`	`mat.drawPolyline(points, closed = true, …)`	A connected run of segments. An empty `points` draws nothing rather than failing.	14
`fillPolygon`	`mat.fillPolygon(points, color = White, lineType = Connected8)`	Fills the implicitly closed outline, even-odd rule.	14

`drawSegments`	`mat.drawSegments(segments, …)`	The renderer for a `houghLinesP` result.	13, 14
`textSize`	`Draw.textSize(text, font = Simplex, scale = 1.0, thickness = Default): TextMetrics`	Query. Measures without drawing. A background box needs `size.height + baseline` to enclose descenders.	14
`draw`	`img.draw(picture: Picture)` — needs `scalacv-graphs`	Renders a whole scene graph in one call.	16

NATIVE MEMORY

`drawPolyline`, `fillPolygon` and `drawContours` reach OpenCV through the generated Java binding's `Converters.vector_vector_Point_to_Mat`, which allocates one `Mat` per polygon plus one for the outer vector and releases none of them. scalacv frees the `MatOfPoint`s it built itself, in a `finally`, but that upstream residue cannot be fixed from here. It is bounded per call and unbounded across a video loop — so if you annotate every frame with contours, draw them from as few calls as you can, and watch RSS rather than heap.

Contours and shape measurement

`findContours` copies every outline out of native memory and frees the `MatOfPoint`s before it returns, so a `Contour` is ordinary immutable Scala data that outlives the image it came from. Its measurements are `lazy val`s: reading three of them off one contour materialises the point conversion once.

Table B-11. Finding outlines and measuring them.

Operation	Signature (abbreviated)	What it does	See
`findContours`	`mat.findContours(retrieval = External, approximation = Simple): Seq[Contour]`	Outlines of a binary image. Single-channel 8-bit input; the hierarchy is not exposed.	12
`contours`	`img.contours(retrieval = External, approximation = Simple): Seq[Contour]`	Query. The same call from the `Image` tier, without spending the image.	12
`points`	`contour.points: Seq[Point]`	The outline itself. `Simple` collapses straight runs, so a rectangle is four points.	12
`boundingRect`	`contour.boundingRect: Rect`	OpenCV's upright box, *inclusive* of the extreme pixels.	12
`area`	`contour.area: Double`	Shoelace area, measured between boundary pixel centres — a filled 100×50 rectangle reports 4851, not 5000. Always non-negative.	12
`perimeter`	`contour.perimeter: Double`	Closed arc length.	12

`centroid`	`contour.centroid: Option[Point]`	Centre of mass from image moments. `None` when the area is zero.	12
`convexHull`	`contour.convexHull: Contour`	The tightest convex outline, whose vertices are points of this contour.	12
`approx`	`contour.approx(epsilon, closed = true): Contour`	Ramer–Douglas–Peucker simplification; `epsilon` is usually a small fraction of the perimeter.	12
`area`	`rect.area: Long`	Rectangle area. `Long`, because `width * height` overflows an `Int` past a 46340-pixel side. `rect.topLeft` and `rect.bottomRight` give the corners as `Point`s.	8, 12
`distanceTo`	`point.distanceTo(other): Double`	Euclidean distance, via `math.hypot`.	8

Hough

All three transforms demand a non-empty `CV_8UC1` image — the output of `canny`, typically — and say so with an `IllegalArgumentException` rather than aborting in native code. `threshold` is a vote count, not a fraction, so it scales with image size: a threshold tuned on a 400-pixel thumbnail is wrong on the 4000-pixel original. There is no circle transform here; Chapter 13 shows `Imgproc.HoughCircles` through the low-level API, and says why the wrapper stops at lines.

Table B-12. The line transforms, and the types they return.

Operation	Signature (abbreviated)	What it does	See
`houghLines`	`mat.houghLines(threshold, rho = 1.0, theta = Pi/180, srn = 0.0, stn = 0.0, minTheta = 0.0, maxTheta = Pi): Seq[PolarLine]`	Infinite lines in Hesse normal form. `theta` is the angle of the *normal*.	13
`houghLinesP`	`mat.houghLinesP(threshold, rho = 1.0, theta = Pi/180, minLineLength = 0.0, maxLineGap = 0.0): Seq[Segment]`	Finite segments with integer endpoints — the underlying Mat is `CV_32SC4`.	13
`houghLinesWithAccumulator`	`mat.houghLinesWithAccumulator(threshold, rho = 1.0, theta = Pi/180, …): Seq[PolarLineWithVotes]`	As `houghLines`, keeping each line's vote count so you can rank the results.	13
`start` `end`	`segment.start: Point`, `segment.end: Point`	The endpoints as `Point`s, for drawing and measurement.	13
`length`	`segment.length: Double`	Euclidean length — the near-universal "keep the long ones" filter.	13

`line`	`polarLineWithVotes.line: PolarLine`	Drops the votes once you have ranked by them.	13

I/O and encoding

Neither `read` nor `write` calls `imread` / `imwrite`. The bytes move through `java.nio.file` and OpenCV is left only the codec work, because the JNI layer narrows a Java `String` path to modified UTF-8 and any non-ASCII character then resolves to a different, nonexistent name on Windows. The price is that the encoded file passes through a JVM byte array, so peak heap grows by its size and a file above 2 GB is out of reach.

Table B-13. The boundary between OpenCV and everything else.

Operation	Signature (abbreviated)	What it does	See
`write`	`img.write(path): Either[CvError, Unit]`	Terminal. Encodes by extension, writes, then releases — on success and on failure.	7
`bytes`	`img.bytes(format = ".png"): Either[CvError, Array[Byte]]`	Terminal. Encodes in memory, then releases.	7
`Images.read`	`Images.read(path, flags = Color): Either[CvError, Managed[Mat]]`	The `Mat`-tier read. `Left` says which of six causes it was.	7
`Images.write`	`Images.write(path, mat): Either[CvError, Unit]`	Borrows the Mat. Encoding completes before the destination is touched, so a failure cannot leave a half-written file.	7
`Images.encode`	`Images.encode(mat, ext = ".png"): Either[CvError, Array[Byte]]`	In-memory encode. A missing leading period is added; an unregistered extension is a `Left`, not a throw.	7
`Images.decode`	`Images.decode(bytes, flags = Color): Either[CvError, Managed[Mat]]`	In-memory decode. An empty array is rejected before it reaches OpenCV.	7
`ImreadFlags`	`ImreadFlags(color, scale = Full, ignoreOrientation = false)`	A total model, not a bitmask: the pair maps onto exactly one `IMREAD_*` constant. `ImreadFlags.Color`, `.Grayscale` and `.Unchanged` are the shorthands.	7, 8
`DecodeFailed`	`CvError.DecodeFailed(path, details)`	The one error every read path produces.	6
`EncodeFailed`	`CvError.EncodeFailed(path, details)`	The one error every write and encode path produces.	6

Interop

The library never walls off the layer underneath it. These are the moves between tiers, plus the handful of combinators that make an owned `Managed[Mat]` pleasant to chain. Two of them are worth committing to memory, because together they cover most of what a hand-written OpenCV sequence needs: `pipe` consumes the `Managed` it is called on, releasing it once the next stage has produced its own output, and `Mats.chain` is that same guarantee written as a list of stages rather than a nest of lambdas. `chain`'s `src` is the exception: it is a borrowed `Mat`, never released, because it belongs to whoever made it.

Table B-14. Moving between the tiers, and owning what you find there.

Operation	Signature (abbreviated)	What it does	See
`OpenCv.load`	`OpenCv.load(): Unit`	Loads the natives. Idempotent, thread-safe, headless. `OpenCv.isLoaded` reports the state.	2
`Managed`	`Managed(a)(using Releasable[A]): Managed[A]`	Takes ownership of any native handle for which a `Releasable` exists.	5
`use`	`managed.use(f)` · `Managed.use(a)(f)`	Runs `f` and releases afterwards, exception or not.	5
`get`	`managed.get: A`	Borrows. Throws `IllegalStateException` after release, in Scala, rather than segfaulting in JNI.	5
`release`	`managed.release(): Unit`	Frees exactly once. `managed.isReleased` reports the state.	5
`scope`	`Managed.scope(body: Scope => B): B`	Ties several handles to one block. `own(a)` registers a new one; `own.adopt(handle)` takes over an existing `Managed`.	5
`pipe`	`managed.pipe(f: Mat => Managed[Mat]): Managed[Mat]`	Feeds the Mat to the next stage and releases it once that stage has produced its own output.	5
`chain`	`Mats.chain(src)(stages*): Managed[Mat]`	The n-stage form of `pipe`. `src` is borrowed and never released.	A
`attempt`	`Cv.attempt(operation)(a): Either[CvError, A]`	Folds a native throw into a value. This is how you give a transform an `Either`.	6
`orThrow`	`Cv.orThrow(operation)(a): A`	The throwing twin, tagging the failure with the operation's name.	6
`nativeHandle`	`Releasable.nativeHandle[A]`	A `Releasable` for any `org.opencv.*` binding that keeps its address in `nativeObj` — which is all 185 of the	5, A

		types with no public `release()` — and cannot be handed the wrong accessor.	
`handle`	`Releasable.handle[A](getNativeAddr)`	The explicit-accessor form, for a type whose address lives somewhere else.	5, A

Where the verb is not where you looked

Six lookups fail often enough to be worth naming.

There is no `img.drawLine`: the `Image` tier wraps five drawing primitives and the line is not one of them, so draw it through `img.mat`, which leaves the image alive rather than consuming it. There is no circle transform at all — the wrapper commits to the line transforms, and Chapter 13 shows `Imgproc.HoughCircles` through the low-level API instead.

`img.blur` is a Gaussian and `mat.boxBlur` is a box filter; they are deliberately not the same name, because dropping from one tier to the other must not switch filter families behind your back. `img.threshold` throws away the level OpenCV computed, which makes `Threshold.otsu()` pointless on that tier — use `mat.threshold`, which returns it.

Recognising text is `Ocr.read(image, engine, preprocess = true)`, an object method that borrows the image; `recognize` is the one method *you* implement, on `OcrEngine`. And `estimatePose` is two unrelated operations: `img.estimatePose(net, inputSize, layout, …)` gives you human body keypoints, while `Ar.estimatePose(marker, markerLength, intrinsics)` gives a detected marker a 3-D pose. Both live in `scalacv-vision`.

Where to go from a row

A row in this appendix tells you a name exists and roughly what it costs; it deliberately does not tell you which `Colormap` to pick, what `ContourRetrieval.Tree` promises about nesting, or why `ImreadFlags` refuses to be OR-ed together. Those are questions about the enumerations rather than about the operations, and the two references are meant to be read together: find the verb here, find the constant you have to hand it there.

Appendix C, *Enums and Types Reference*, answers them case by case, with the OpenCV constant printed underneath each one so you can check a translation against the C++ documentation without leaving the page — including the three types that look like enumerations and are not.

APPENDIX C

Enums and Types Reference

Every typed constant and value type in the library, with the OpenCV constant underneath it.

Chapter 8 argued the case for typing OpenCV's integer constants and showed the mechanism: a Scala 3 `enum` with one parameter, `cvValue`, carrying the number the native call actually receives. This appendix is the other half — the complete list, so that a question of the form "what are the cases?" or "which OpenCV constant is that?" has one place to be answered without an IDE open.

Two things are worth knowing before the tables start. The first is that nothing here is a wrapper around a pointer. Every type in this appendix is either a Scala 3 `enum` whose runtime representation is a singleton object, or a `final case class` holding primitives and other case classes. None of them owns native memory, none of them needs releasing, and all of them stay valid after the `Mat` they were computed from has been freed. That is a deliberate property and it is what makes the detector results in the second half of this appendix safe to store, serialise and return from a scope.

The second is that the OpenCV constant is never hidden. `cvValue` is a public `val` on every enum that wraps one, so `Interpolation.Area.cvValue` is `Imgproc.INTER_AREA` and you can log it, compare it, or pass it to a raw call. Three enums carry a different payload, because a single `int` is not what they name: `ImreadScale` carries `denom` (`1`, `2`, `4`, `8`), `Codec` a `fourcc: Int`, and `CascadeName` a `fileName: String`. Five carry nothing at all — `ImreadColor`, `TrackerKind`, `KeypointLayout`, `HandGesture` and `Steering` — because each stands for a decision made in Scala rather than for a constant OpenCV defines. Every enum also has `.values`, an `Array` of its cases, which is the fastest way to answer a "what are the options" question from a REPL rather than from a book.

The core enums

Thirteen true enumerations live in `core/src/scalacv/Enums.scala`, together with the two structured value types — `Threshold` and `ImreadFlags` — that are deliberately not enums, the four small enums that make up their axes, and `ThresholdResult`, which the mid-level `Mat.threshold` returns beside its Mat.

Colour conversion

`ColorConversion` steers `Image.convert` and the mid-level `Mat.cvtColor`. The `gray` and `toHsv` shortcuts cover the two commonest choices; chapter 8 covers when each colour space earns its keep, and chapter 11 uses HSV in anger for segmentation.

Table C-1. ColorConversion — the colour spaces scalacv converts between.

Case	OpenCV constant	Meaning
`BgrToGray`	`COLOR_BGR2GRAY`	Three channels to one luminance channel.
`GrayToBgr`	`COLOR_GRAY2BGR`	One channel to three identical ones — how you draw colour on a grey image.
`BgrToRgb`	`COLOR_BGR2RGB`	Channel order swap, for handing pixels to a library that expects RGB.
`RgbToBgr`	`COLOR_RGB2BGR`	The same swap the other way; the operation is its own inverse.
`BgrToHsv`	`COLOR_BGR2HSV`	Hue, saturation, value — the space colour thresholding belongs in.
`HsvToBgr`	`COLOR_HSV2BGR`	Back to displayable pixels.
`BgrToLab`	`COLOR_BGR2Lab`	CIE L*a*b*, where Euclidean distance approximates perceived difference.
`LabToBgr`	`COLOR_Lab2BGR`	Back from L*a*b*.
`BgrToBgra`	`COLOR_BGR2BGRA`	Adds an opaque alpha channel.
`BgraToBgr`	`COLOR_BGRA2BGR`	Drops the alpha channel.

Resizing, borders and mirroring

`Interpolation` is the resampling kernel for `resize`, `scale` and the warps of chapter 10. `Area` is the one worth remembering: it is the only case that averages rather than samples, which is why it is the right choice when shrinking an image and a poor one when enlarging it.

Table C-2. Interpolation — how a resampled pixel is computed.

Case	OpenCV constant	Meaning
`Nearest`	`INTER_NEAREST`	Take the closest source pixel. Blocky, exact, and the only correct choice for a label mask.
`Linear`	`INTER_LINEAR`	Bilinear over the 2×2 neighbourhood. OpenCV's default and scalacv's.
`Cubic`	`INTER_CUBIC`	Bicubic over 4×4. Sharper enlargement, slower.
`Area`	`INTER_AREA`	Pixel-area resampling — averages the source region, so down-scaling does not alias.
`Lanczos4`	`INTER_LANCZOS4`	Windowed sinc over 8×8. The most expensive, and the sharpest.

`BorderType` says what pixels exist beyond the edge of an image, for every operation that reads a neighbourhood. Chapters 9 and 10 both take it, and they do not share a default: the filters default to `Reflect101`, while `pad`, `border` and `rotated` default to `Constant` filled with `Scalar.Black`.

Table C-3. BorderType — how pixels past the edge are invented.

Case	OpenCV constant	Meaning
`Constant`	`BORDER_CONSTANT`	One fixed colour outside. The padding default.
`Replicate`	`BORDER_REPLICATE`	The edge pixel repeated outward: `aaaaaa|abcdefgh`.
`Reflect`	`BORDER_REFLECT`	Mirrored, including the edge pixel: `fedcba|abcdefgh`.
`Reflect101`	`BORDER_REFLECT_101`	Mirrored, excluding it: `gfedcb|abcdefgh`. The filter default.
`Wrap`	`BORDER_WRAP`	The opposite edge, as if the image tiled. **Padding and `rotated` only.**

> **WARNING**
> `Wrap` is the one case in this appendix that is not accepted everywhere its type is. OpenCV's `imgproc` filter engine asserts `columnBorderType != BORDER_WRAP` and aborts the process. Worse, it is inconsistent about when: measured against OpenCV 4.13, `GaussianBlur` with `BORDER_WRAP` silently ignores the mode on `CV_8U` input and aborts on `CV_32F`, so the same call works against the 8-bit frames it was developed on and kills the process the first time it meets a float image. `BorderType.requireFilterSupport(op, border)` is the guard `gaussianBlur`, `boxBlur`, `sobel` and `laplacian` each run, turning that into an `IllegalArgumentException` before any native call. `BORDER_ISOLATED` and `BORDER_TRANSPARENT` are not in the enum at all — the first is meaningless without the region-of-interest calls scalacv does not expose, and the second can only produce a throw or uninitialised pixels.

`Flip` and `Rotation` are chapter 10's lossless pair. `Flip` is named for the visible effect rather than for OpenCV's axis-centric flip code, which is the usual source of a mirrored-the-wrong-way bug.

Table C-4. Flip and Rotation — the operations that move pixels without resampling them.

Case	OpenCV constant	Meaning
`Flip.Horizontal`	flip code `1`	Mirror left–right, about the vertical axis.
`Flip.Vertical`	flip code `0`	Mirror top–bottom, about the horizontal axis.
`Flip.Both`	flip code `-1`	Both at once — a 180° point reflection.
`Rotation.Clockwise`	`ROTATE_90_CLOCKWISE`	A quarter turn clockwise.
`Rotation.CounterClockwise`	`ROTATE_90_COUNTERCLOCKWISE`	A quarter turn anti-clockwise.
`Rotation.Half`	`ROTATE_180`	A half turn. Exact pixels, no interpolation.

`Rotation` covers the quarter turns only. The separate `rotate(degrees, scale)` overload resamples through `warpAffine`, expanding the canvas so no corner is clipped, and it measures its angle *counter-clockwise* — so `rotate(90.0)` turns the image the same way `rotate(Rotation.CounterClockwise)` does.

> **WARNING**
> Two rotation conventions live in this library. `Image.rotate(degrees)` is counter-clockwise, matching `getRotationMatrix2D` and the `Rotation` enum's `CounterClockwise` case. `Picture.rotate(degrees, about)` in `scalacv-graphs` is clockwise: the scene graph works in screen coordinates, where `y` points down the image rather than up, and in that frame the same positive angle turns the other way. Check which layer you are holding before reaching for a minus sign.

Morphology

`MorphShape` builds the structuring element for `erode`, `dilate` and `morphology` — all three take a `radius` and a `shape`, defaulting to `MorphShape.Rect`. `MorphOp` names the compound operation that `morphology` performs; erosion and dilation have their own methods and so are not cases here. Chapter 9 is the chapter.

Table C-5. MorphShape and MorphOp — the kernel, and what to do with it.

Case	OpenCV constant	Meaning
`MorphShape.Rect`	`MORPH_RECT`	A filled rectangle. The cheapest, the library default, and it squares off round shapes.
`MorphShape.Ellipse`	`MORPH_ELLIPSE`	A filled ellipse. Worth passing by hand for organic shapes.
`MorphShape.Cross`	`MORPH_CROSS`	A plus sign. Touches only the 4-connected neighbours.
`MorphOp.Open`	`MORPH_OPEN`	Erode then dilate — removes small bright specks.
`MorphOp.Close`	`MORPH_CLOSE`	Dilate then erode — fills small dark holes.
`MorphOp.Gradient`	`MORPH_GRADIENT`	Dilation minus erosion — an outline of the shapes.
`MorphOp.TopHat`	`MORPH_TOPHAT`	Source minus its opening — bright detail smaller than the kernel.
`MorphOp.BlackHat`	`MORPH_BLACKHAT`	Closing minus the source — dark detail smaller than the kernel.

Thresholding

`Threshold` is not an enum, and chapter 11 explains why: `Imgproc.threshold` takes a mode OR-ed with at most one automatic-threshold modifier, so the useful combinations would be unrepresentable as a flat enumeration. It is a case class of a `Threshold.Mode` and an `Option[Threshold.Auto]`, with `cvValue` doing the OR and `computesThreshold` reporting whether the `ThresholdResult` the mid-level call returns will carry a number worth reading.

Table C-6. Threshold.Mode and Threshold.Auto — the two axes of a threshold.

Case	OpenCV constant	Meaning
`Mode.Binary`	`THRESH_BINARY`	Above the cutoff becomes `maxValue`, below becomes `0`.

`Mode.BinaryInv`	`THRESH_BINARY_INV`	The same, inverted.
`Mode.Truncate`	`THRESH_TRUNC`	Above the cutoff is clamped to it; below is untouched.
`Mode.ToZero`	`THRESH_TOZERO`	Below the cutoff becomes `0`; above is untouched.
`Mode.ToZeroInv`	`THRESH_TOZERO_INV`	Above the cutoff becomes `0`; below is untouched.
`Auto.Otsu`	`THRESH_OTSU`	OpenCV picks the cutoff by minimising intra-class variance.
`Auto.Triangle`	`THRESH_TRIANGLE`	OpenCV picks it by the triangle method — better for one dominant peak.

Three shorthands sit in the companion. `Threshold.Binary` is the plain fixed-cutoff value and the default of the `kind` parameter both `threshold` methods take — note that it is a `Threshold`, not a `Mode`, and shadows `Threshold.Mode.Binary` in prose if not in code. `Threshold.otsu(mode)` and `Threshold.triangle(mode)` both default `mode` to `Mode.Binary`. `AdaptiveMethod` is the separate enum that `adaptiveThreshold` takes: `Mean` (`ADAPTIVE_THRESH_MEAN_C`, the unweighted neighbourhood average) and `Gaussian` (`ADAPTIVE_THRESH_GAUSSIAN_C`, weighted by distance, and the default).

Contours

Both of chapter 12's enums have a case whose name shadows something in `scala.Predef`. `ContourRetrieval.List` and `ContourApproximation.None` are the source's spellings and are correct; inside `Enums.scala` itself the compiler needs `scala.None` written out, which is why `Threshold`'s default argument reads `auto: Option[Threshold.Auto] = scala.None`.

Table C-7. ContourRetrieval and ContourApproximation — which outlines, and how compressed.

Case	OpenCV constant	Meaning
`ContourRetrieval.External`	`RETR_EXTERNAL`	Outermost contours only. The default, and usually what you want.
`ContourRetrieval.List`	`RETR_LIST`	Every contour, flat, with no hierarchy.
`ContourRetrieval.CComp`	`RETR_CCOMP`	Two levels: outer boundaries and the holes inside them.
`ContourRetrieval.Tree`	`RETR_TREE`	The full nesting hierarchy.
`ContourApproximation.None`	`CHAIN_APPROX_NONE`	Every boundary pixel, uncompressed.
`ContourApproximation.Simple`	`CHAIN_APPROX_SIMPLE`	Straight runs collapsed to their endpoints. The default.
`ContourApproximation.Tc89L1`	`CHAIN_APPROX_TC89_L1`	Teh–Chin chain approximation, L1 variant.
`ContourApproximation.Tc89Kcos`	`CHAIN_APPROX_TC89_KCOS`	Teh–Chin, k-cosine variant.

Drawing

`Font` and `LineType` reach the mid-level `Mat.drawText` and the `Picture` layer of chapter 16; the chainable `Image` verbs of chapter 14 fix the font at `Simplex` and the line type at `Connected8` rather than carry two more parameters through every signature.

Table C-8. Font and LineType — the Hershey vector fonts, and rasterisation.

Case	OpenCV constant	Meaning
`Font.Simplex`	`FONT_HERSHEY_SIMPLEX`	Sans-serif, single stroke. The default.
`Font.Plain`	`FONT_HERSHEY_PLAIN`	Smaller sans-serif.
`Font.Duplex`	`FONT_HERSHEY_DUPLEX`	Sans-serif, double stroke — heavier than `Simplex`.
`Font.Complex`	`FONT_HERSHEY_COMPLEX`	Serif, single stroke.
`Font.Triplex`	`FONT_HERSHEY_TRIPLEX`	Serif, triple stroke.
`Font.Script`	`FONT_HERSHEY_SCRIPT_SIMPLEX`	Handwriting-style.
`LineType.Connected4`	`LINE_4`	4-connected Bresenham.
`LineType.Connected8`	`LINE_8`	8-connected Bresenham. The default.
`LineType.AntiAliased`	`LINE_AA`	Gaussian-filtered edges. Smoother, and slower.

These are the only fonts there are: OpenCV cannot render a system font, and a non-ASCII character comes out as `?`.

False colour

`Colormap` turns a single-channel image — a depth map, a motion field, any scalar field — into a colour heatmap. One argument, two entry points: `Image.colorMap(map)` in the chain, and the mid-level `Mat.colorMap(map)` on a borrowed handle. Chapter 15 uses it, and `Filter.heatmap` is the prepackaged `gray` then `colorMap(Colormap.Inferno)` pair; a disparity map in chapter 32 is far easier to read through one.

Table C-9. Colormap — the false-colour ramps.

Case	OpenCV constant	Meaning
`Autumn`	`COLORMAP_AUTUMN`	Red through yellow.
`Bone`	`COLORMAP_BONE`	Grey with a blue cast — the medical-imaging ramp.
`Jet`	`COLORMAP_JET`	The classic rainbow. Not perceptually uniform; it invents bands that are not in the data.
`Ocean`	`COLORMAP_OCEAN`	Green through blue to white.
`Hot`	`COLORMAP_HOT`	Black through red and yellow to white.
`Magma`	`COLORMAP_MAGMA`	Perceptually uniform, black to pale yellow.
`Inferno`	`COLORMAP_INFERNO`	Perceptually uniform, higher-contrast than `Magma`.

`Plasma`	`COLORMAP_PLASMA`	Perceptually uniform, blue through magenta to yellow.
`Viridis`	`COLORMAP_VIRIDIS`	Perceptually uniform, and legible in greyscale print.
`Turbo`	`COLORMAP_TURBO`	A rainbow rebuilt to be perceptually uniform — `Jet`'s range without its lies.

Reading an image

`ImreadFlags` is the second structured value type, and chapter 7 explains the trap it exists to close: OpenCV's `IMREAD_*` constants look like OR-able bits and are not. Each `IMREAD_REDUCED_*` value already carries its own colour bit, and `IMREAD_UNCHANGED` is `-1`, whose bits swamp every other flag. So the `(colour, scale)` pair maps *totally* onto exactly one named constant rather than composing, and only `ignoreOrientation` (bit 128) is genuinely independent.

Table C-10. ImreadColor and ImreadScale — the two axes of ImreadFlags.

Case	OpenCV constant	Meaning
`ImreadColor.Grayscale`	`IMREAD_GRAYSCALE`	One channel, whatever the file holds.
`ImreadColor.Color`	`IMREAD_COLOR`	Three channels, BGR. The default.
`ImreadColor.ColorRgb`	`IMREAD_COLOR_RGB`	Three channels, RGB order — no `cvtColor` afterwards.
`ImreadColor.Unchanged`	`IMREAD_UNCHANGED`	Keep alpha and bit depth exactly as stored. Cannot carry any other flag.
`ImreadColor.AnyDepth`	`IMREAD_ANYDEPTH`	Keep 16- or 32-bit depth rather than truncating to 8.
`ImreadScale.Full`	(no reduction)	Decode at full resolution. `denom` is `1`.
`ImreadScale.Half`	`IMREAD_REDUCED_*_2`	Half each side. `denom` is `2`.
`ImreadScale.Quarter`	`IMREAD_REDUCED_*_4`	A quarter each side. `denom` is `4`.
`ImreadScale.Eighth`	`IMREAD_REDUCED_*_8`	An eighth each side. `denom` is `8`.

Reduced-size decode exists only for `Grayscale` and `Color`, so `ImreadFlags` carries two `require`s that reject the combinations OpenCV has no constant for — a reduction on `ColorRgb`, `Unchanged` or `AnyDepth`, and any extra bit on `Unchanged`. Three constants cover the common cases: `ImreadFlags.Color`, `ImreadFlags.Grayscale` and `ImreadFlags.Unchanged`.

Enums that live beside their operations

Not every typed constant is in `Enums.scala`. Where an enum is used by exactly one operation or one module, it sits with that code instead, which keeps the scaladoc and the signature on the same screen.

Table C-11. The typed constants defined outside Enums.scala.

Type	Defined in	Cases, and where the book covers them
`OutputDepth`	`core` — `Ops.scala`	`SameAsSource` (`-1`), `Unsigned8` (`CV_8U`), `Signed16` (`CV_16S`), `Float32` (`CV_32F`), `Float64` (`CV_64F`). Chapter 9.
`Thickness`	`core` — `Draw.scala`	A sealed trait, not an enum: `Thickness.Stroke(pixels)` and `Thickness.Filled` (`Imgproc.FILLED`), plus the `Thickness.Default` one-pixel stroke. Chapter 14.
`Codec`	`core` — `Codec.scala`	`Mp4v`, `Avc1`, `Mjpg`, `Xvid`, each carrying a `fourcc: Int`. Chapter 19.
`CaptureBackend`	`core` — `Video.scala`	`Any`, `FFmpeg`, `GStreamer`, `V4L2`, `AVFoundation`, `MediaFoundation`, `DirectShow`, `ImageSequence`, `BuiltinMjpeg`. Chapter 20.
`CascadeName`	`vision` — `Cascades.scala`	Thirteen bundled Haar cascades, each carrying a `fileName`. Chapter 24.
`ArucoDictionary`	`vision` — `Detectors.scala`	Twenty-two marker families. Chapter 28.
`TrackerKind`	`vision` — `Tracker.scala`	`Csrt`, `Kcf`, `Mil`. No `cvValue` — each case builds a different OpenCV class. Chapter 30.
`KeypointLayout`	`vision` — `PoseEstimator.scala`	`Regression`, `Heatmap` — how a pose network encodes its output tensor. Chapter 29.
`HandGesture`	`vision` — `Gesture.scala`	`Fist`, `OpenPalm`, `ThumbsUp`, `Pointing`, `Victory`, `Unknown`. Chapter 29.
`Steering`	`vision` — `Navigator.scala`	`Straight`, `Left`, `Right`, `Stop`. Chapter 32.

`Thickness` is the one entry above that is a sealed trait rather than an enum, and the reason is worth a line. OpenCV encodes "filled" as a thickness of `-1`, a sentinel ordinary arithmetic will produce by accident, and it is meaningful only for closed shapes — `cv::line` asserts `0 < thickness` and aborts on it. Splitting the two cases into distinct types lets `drawRect` and `drawCircle` accept a `Thickness` while lines and text accept `Thickness.Stroke` only, so the mistake stops compiling instead of crashing.

`OutputDepth` earns its type for the opposite reason: `SameAsSource` is the trap. `Sobel` on an 8-bit unsigned image with `ddepth = -1` clips every negative derivative to zero, so half of each edge silently disappears; `Signed16` followed by `convertScaleAbs` is the fix, and chapter 9 works through it.

`CascadeName`'s cases are `FrontalFaceAlt`, `FrontalFaceAlt2`, `FrontalFaceDefault`, `ProfileFace`, `Eye`, `EyeTreeEyeglasses`, `LeftEye2Splits`, `RightEye2Splits`, `Smile`, `FullBody`, `UpperBody`, `LowerBody` and `RussianPlateNumber`. `ArucoDictionary`'s are `Dict4x4_50` through `Dict7x7_1000` — four grid sizes crossed with 50, 100, 250 and 1000 markers — plus `ArucoOriginal`, `AprilTag16h5`, `AprilTag25h9`, `AprilTag36h10`, `AprilTag36h11` and `ArucoMip36h12`. Prefer the

smallest dictionary with enough ids for the job: fewer markers means a larger Hamming distance between them and a more robust detection.

The geometry value types

`core/src/scalacv/Geometry.scala` holds five case classes. They exist because `org.opencv.core.Point`, `Size`, `Rect` and `Scalar` are mutable Java objects with public fields, and a `Seq` of them handed back from a detector is a set of live handles whose contents can change underneath you. Copying four numbers across the boundary is cheap, and it turns the result into ordinary immutable Scala data.

Table C-12. The geometry value types, and their org.opencv.core counterparts.

Type	OpenCV equivalent	Constructor and members
`Point`	`org.opencv.core.Point`	`Point(x: Double, y: Double)`. Origin top-left, `x` right, `y` down. `distanceTo(other)` via `math.hypot`.
`Point3`	`org.opencv.core.Point3`	`Point3(x: Double, y: Double, z: Double)`. A model coordinate for pose work; `z` points out of the marker plane toward the camera.
`Size`	`org.opencv.core.Size`	`Size(width: Double, height: Double)`. Neither side may be negative; zero is allowed.
`Rect`	`org.opencv.core.Rect`	`Rect(x: Int, y: Int, width: Int, height: Int)`. `area: Long`, `topLeft: Point`, `bottomRight: Point`.
`Scalar`	`org.opencv.core.Scalar`	`Scalar(v0, v1 = 0, v2 = 0, v3 = 0)`, all `Double`. Up to four channel components in the Mat's own channel order.

Three details in that table pay for themselves. `Rect.area` is a `Long` because `width * height` overflows a signed `Int` past roughly a 46 340-pixel side, which a full-frame region of interest on a large image reaches. `Rect.bottomRight` is `(x + width, y + height)` — one past the last enclosed pixel, the half-open convention OpenCV itself uses. And `Scalar` is in the Mat's channel order, which for OpenCV means **BGR, not RGB**: `Scalar.Red` is `Scalar(0, 0, 255)`. The companion also offers `Black`, `White`, `Green` and `Blue`, in the same order.

> **NATIVE MEMORY**
> Nothing in this section holds a native pointer, and that is the point of it. A `Seq[Rect]` returned by a detector, a `Contour`'s `Seq[Point]`, a `Face`'s landmarks — all of them remain valid after the `Image` or `Managed[Mat]` they were computed from has been released, and none of them needs a `Managed` wrapper, a scope, or a `close`. The rule from chapter 5 still holds for everything that does own memory; these types are outside it by construction. The one thing to keep straight is that holding a `Rect` does not keep its pixels alive: crop before you leave the scope, not after.

Why toCv and from are private

Each of these types has a `toCv` producing the `org.opencv.core` object, and each companion has a `from` reading one back. Both are `private[scalacv]`, and that is deliberate rather than an oversight.

The conversion allocates. `cv.Point` and `cv.Scalar` are plain Java objects with no native handle, so those are harmless — but a public bridge invites the shape where a caller converts, hands the result to a raw OpenCV call, and keeps it. Keeping the bridge internal means every `org.opencv` object scalacv creates is one scalacv also frees, and the public surface stays one of pure values.

When you genuinely need the OpenCV form — because you are calling a method the library does not wrap — construct it yourself from the fields: `org.opencv.core.Rect(r.x, r.y, r.width, r.height)`. It is one line rather than one method call, and it puts the ownership where you can see it. Appendix A covers that descent in full.

The `scalacv-graphs` module adds three value types of its own on the same model: `Color(red, green, blue, alpha = 255)` with `lighten`, `darken`, `fadeOut`, `withAlpha` and `blend`; `Dash(on, off)` with the `Dash.dashed`, `Dash.dotted` and `Dash.dense` presets; and `Bounds(minX, minY, maxX, maxY)` with `width`, `height`, `centerX`, `centerY` and `union`. Chapter 16 covers all three.

Result types

Every detector, estimator and analyser in the library returns plain immutable data rather than a handle. This is the full list, with the chapter that puts each to work.

Table C-13. The result types, and where the book covers them.

Type	What it carries	Chapter
`ThresholdResult`	`value: Double` — the cutoff Otsu or Triangle chose. Only the mid-level `Mat.threshold` returns it; `Image.threshold` drops it.	11
`Contour`	`points: Seq[Point]`, plus lazy `boundingRect`, `area`, `perimeter`, `centroid`, `convexHull`.	12
`PolarLine`	`rho: Float`, `theta: Float` — an infinite line in Hough space.	13
`PolarLineWithVotes`	The same, plus `votes: Int`.	13
`Segment`	`x1`, `y1`, `x2`, `y2`, all `Int` — a finite line segment. Plus `start`, `end`, `length`.	13
`TextMetrics`	`size: Size`, `baseline: Int` — what a string will occupy once drawn.	14
`CaptureInfo`	`width`, `height`, `fps`, `frameCount`, `backendName`, plus `size`. Every field advisory.	19
`CaptureOptions`	How a capture should be opened. An input, but the same shape.	19

`Motion`	`moving: Boolean`, `ratio: Double`, `regions: Seq[Rect]`, plus `regionCount` and `largest`.	21
`ModelSpec`	`fileName`, `urls`, `sha256: Option[String]`, `sizeBytes: Option[Long]`. The constructor is private; `ModelSpec(...)` pins a hash, `ModelSpec.unverified` opts out.	23
`Face`	`box: Rect`, five `landmarks: Seq[Point]`, `score: Float`, plus the named accessors.	24
`FaceEmbedding`	`values: Vector[Float]` — a face as a comparable vector.	25
`FaceMatch`	`name: String`, `similarity: Double`.	25
`QrCode`	`text: String` (empty when located but undecodable), `corners: Seq[Point]`.	28
`ArucoMarker`	`id: Int`, `corners: Seq[Point]`, clockwise from the marker's top left.	28
`Pose3D`	`rvec: Seq[Double]`, `tvec: Seq[Double]` — a marker's rotation and translation.	28
`MarkerPose`	`marker: ArucoMarker`, `pose: Pose3D`.	28
`Keypoint`	`name: String`, `point: Point`, `score: Float`.	29
`PoseTopology`	`names: Seq[String]`, `edges: Seq[(Int, Int)]` — the skeleton's wiring.	29
`Pose`	`keypoints: Seq[Keypoint]`, `topology: PoseTopology`.	29
`HeadPose`	`yaw`, `pitch`, `roll`, all `Double`.	29
`ObjectTrack`	`id: Int`, `box: Rect`, `hits: Int`, `age: Int`.	30
`ChessboardPattern`	`columns`, `rows`, `squareSize` (default `1.0`), plus `corners`, the inner-corner count `columns × rows`. An input, not a result.	31
`Intrinsics`	`fx`, `fy`, `cx`, `cy`, `distortion: Seq[Double]` — the camera model.	31
`Calibration`	`intrinsics`, `imageSize`, `reprojectionError`.	31
`Track`	`from: Point`, `to: Point`, `found: Boolean` — one optical-flow track. Plus `displacement` and `distance`.	32
`FeatureMatch`	`queryIndex`, `trainIndex`, `distance: Float`.	32
`Obstacle`	`region: Rect`, `nearness: Double`.	32
`Guidance`	`steering: Steering`, `clearanceAhead`, and `leftNearness`, `centreNearness`, `rightNearness`.	32
`CameraMotion`	`rotation: Seq[Seq[Double]]`, `translation: Seq[Double]`, `inliers: Int`.	32
`CameraPose`	`rotation`, `translation` — an absolute pose rather than a delta.	32
`LoopClosure`	`keyframe: Int`, `matches: Int`, `score: Double`.	32
`OcrWord`	`text: String`, `confidence: Float`, `box: Rect`.	33
`OcrResult`	`text: String`, `words: Seq[OcrWord]`.	33
`TemplateMatch`	`location: Rect`, `score: Double`.	34

`CvError`, the sealed error hierarchy of chapter 6, belongs to the same family: it is plain data, returned as the left of an `Either`, and it survives the scope that produced it.

Eight types deliberately sit outside that table, because they are handles rather than results and the distinction is the whole point of listing the rest. `Descriptors` (chapter 32) holds a `Managed[Mat]` of ORB descriptors; `Tracker` and `Kalman` (chapter 30) each own one native object; `ObjectTracker` (chapter 30) owns a `Kalman` per live track; `MotionDetector` (chapter 21) retains either the previous frame or an MOG2 background model; `FaceRecognizer` (chapter 25) wraps a `Managed[FaceRecognizerSF]`; `Odometry` (chapter 32) retains the previous frame and its tracked points; and `LoopDetector` (chapter 32) owns a `Descriptors` per keyframe. All eight are `AutoCloseable` and caller-owned: close them, or hand them to `Using.resource`. Everything they hand back — a `Seq[Point]`, a `FeatureMatch`, an `ObjectTrack`, a `Motion`, a `CameraMotion`, a `LoopClosure` — is in the table above and needs nothing.

When the case you need is missing

The enums here are curated, not exhaustive. OpenCV has more than a hundred colour conversions and `ColorConversion` types ten of them; `Videoio` defines many more capture backends than the nine `CaptureBackend` names, most of them for hardware no supported platform exposes; two border modes are left out on purpose. Sooner or later you will want one that is not a case.

The escape hatch is the same one Appendix A describes at length, and it is not a defeat: the raw `org.opencv.*` surface is never walled off, so pass the integer directly and own the result. `mat` borrows the `Image`'s handle without consuming it, `Managed.scope` takes ownership of whatever you allocate inside it, and `Cv.attempt` turns the native throw into a `Left[CvError]`.

The wrong version allocates the destination and then hopes:

Example C-1. The destination leaks on every failing path.

```
val dst = org.opencv.core.Mat()
Imgproc.cvtColor(img.mat, dst, Imgproc.COLOR_BGR2YUV)  // throws? dst is lost
Images.encode(dst, ".png")                             // returns Left? dst is
lost
```

The right version puts the destination under a scope before anything can go wrong, and returns plain data rather than the handle:

Example C-2. Reaching a conversion the enum does not name.

```
import org.opencv.imgproc.Imgproc
import scalacv.*

// COLOR_BGR2YUV has no ColorConversion case. Pass the constant; let the scope
own the Mat.
def yuvBytes(img: Image): Either[CvError, Array[Byte]] =
```

```
Managed.scope: own =>
  val dst = own(org.opencv.core.Mat())
  Cv.attempt("cvtColor")(Imgproc.cvtColor(img.mat, dst, Imgproc.COLOR_BGR2YUV))
    .flatMap(_ => Images.encode(dst, ".png"))
```

Two rules make that safe rather than merely possible. Name the OpenCV constant, never the number: `Imgproc.COLOR_BGR2YUV` survives a version bump and a hard-coded integer does not. And let the `Array[Byte]` escape the scope, never the `Mat` — the rule chapter 5 sets for `Managed.scope` applies here unchanged. Dropping to a raw constant changes the argument you pass, not who owns the result.

Then open a pull request. A case is a two-line change plus a test, `CONTRIBUTING.md` has the build commands (explicit headless modules, `+`-separated test and scalafix commands, and the formatter before you push), and an enum that grows by the cases people actually reached for is a better enum than one designed in advance. A constant that is genuinely dangerous will be argued about rather than merged — `BORDER_TRANSPARENT` and `BORDER_ISOLATED` are both absent on purpose, and the reasoning is in the scaladoc so the argument does not have to be had twice.

Next

Appendix D turns from types to material: the notebooks, sample images and video clips the examples throughout this book were written against, where they live, and how to fetch them.

APPENDIX D

Notebooks and Sample Data

Where to type the code, and what to point it at when you have no photographs.

Two practical questions sit underneath every chapter in this book and belong to none of them. The first is where the code goes while you are still deciding what it should be — a build, a scratch file, a notebook cell — and the second is what you feed it, given that this repository contains no photographs and no video clips at all.

Neither question is about computer vision, which is why they have been deferred to an appendix. Both of them will nevertheless cost you an afternoon if you get them wrong, and the second one has a sharper edge than it looks: a great many people learn a vision library by pulling an image off a search engine, and end up with a licensing problem attached to a repository they then push.

scalacv's own answer is visible in its build. The documentation site type-checks and runs every snippet it prints (`mill docs.mdocCheck`, on every pull request), the example module asserts its own output in CI, and the test suites draw their inputs the same way — none of it reads a bitmap from disk, because there is none to read: no `.png`, `.jpg`, `.avi` or `.mp4` ships in any module. `examples/src/scalacv/Fixtures.scala` says so in its opening comment: "there is no image asset in this repository (no bitmap fixture ships, for licensing reasons). So every example generates its input here rather than reading a file." That is not a workaround the project tolerates; it is the better technique, and the second half of this appendix is a catalogue of it.

Almond: scalacv in a Jupyter cell

Almond is the Scala Jupyter kernel. It resolves dependencies at run time through coursier, so there is no build file, and it renders a `java.awt.image.BufferedImage` inline — which is the entire reason a notebook is worth the setup for image work. You get pixels under the cell that made them.

The dependency story is the one Chapter 2 tells: the API jar carries no native code, so the natives arrive as a separate coordinate. What a notebook changes is how that coordinate is spelled — and whether it is worth carrying a classifier at all.

Table D-1. Three ways to put the natives on a notebook classpath.

Spelling	The co-or-

		di-nate
Mill's `mvn"…"`	—	`org.bytedeco:x86_64` the at-tribute fol-lows a semi-colon
coursier's dependency string (scala-cli's `//> using dep`, and anything else that hands the coordinate to coursier verbatim)	—	`org.bytedeco:x86_64` the at-tribute fol-lows a comma
No classifier at all	—	`org.bytedeco:platform:4.13` every plat-form in one co-or-di-nate, open-blas in-cluded

The separator is the tool's, not scalacv's, and getting it wrong is a resolution error rather than a subtle one. A notebook is where that difference costs the most time for the least benefit, so the first cell below takes the third row. `opencv-platform` resolves anywhere, needs no classifier table and no second line for openblas, and costs about 408 MB against 36–80 MB for a matched pair. Trade it for the pair — `linux-x86_64`, `linux-arm64`, `macosx-arm64`, `macosx-x86_64` or `windows-x86_64`, as Chapter 2 lists them — once you know the platform and want that download back.

Example D-1. The first cell of any scalacv notebook.

```
import $ivy.`com.worxbend::scalacv:0.4.2`
import $ivy.`org.bytedeco:opencv-platform:4.13.0-1.5.13`

import scalacv.*
OpenCv.load()
```

Add `com.worxbend::scalacv-vision:0.4.2` for the detectors, DNN, pose, tracking, OCR and calibration — which is what the ArUco, chessboard and OCR generators later in this appendix need — or `com.worxbend::scalacv-graphs:0.4.2` for `Picture`, charts and GIF. Both depend only on the core, so adding one does not drag in the other.

`OpenCv.load()` is idempotent and safe to call from several threads, so putting it in the first cell and re-running that cell later costs nothing. It also needs no GUI toolkit: it brings javacpp up through a preset that links none, so a container without GTK loads OpenCV rather than failing in `opencv_highgui` and taking `objdetect` down with it. That is what makes a headless notebook server a supported place to run this, not a workaround.

Flags to start the kernel with

scalacv's own build forks every OpenCV-running JVM with the same short list, and a kernel benefits from all of it.

- `-Dorg.bytedeco.javacpp.cachedir=…` — the first `load()` on Linux extracts about 196 MB into `~/.javacpp`. Point it somewhere writable when the notebook server's home is read-only or ephemeral, or the extraction repeats on every kernel restart.
- `-Djava.awt.headless=true` — an assertion rather than a preference. A stray call that wants a window then fails loudly instead of blocking on a display that is not there. `toBufferedImage` is unaffected: a `BufferedImage` is heap memory and needs no display to exist.
- `--enable-native-access=ALL-UNNAMED` on JDK 24 and later, which silences the JEP 472 warning `System.load` emits.
- `--sun-misc-unsafe-memory-access=allow` on JDK 24 and later, which silences the four lines scala3-library 3.3.8's `LazyVals` provokes — cosmetic, but otherwise a preface to every cell that prints.

The last two are exactly what `Deps.headlessJvmArgs` in `build.mill` adds for the examples, the benchmarks, the leak harness and the tests, guarded by the same JDK-major check.

Why the display method is toBufferedImage

Almond has a renderer for `BufferedImage` and none for `Mat`. That asymmetry is the whole reason `Image.toBufferedImage` exists in the shape Chapter 7 described: it copies, it borrows the image rather than consuming it, and it accepts 8-bit images with 1, 3 or 4 channels. Return it from the last expression in a cell and the cell shows a picture.

Example D-2. The habit that makes every cell show its work.

```
def show(img: Image): java.awt.image.BufferedImage = img.toBufferedImage
```

Define that once, near the top of the notebook, and any cell whose last expression is `show(...)` renders a picture instead of printing a type name.

Two restrictions bite in practice. `toBufferedImage` requires depth `CV_8U`, so a disparity map, a distance transform or a raw gradient response throws an `IllegalArgumentException` rather than silently truncating — bring it down first with `normalize()` for plain intensity, or `.gray.colorMap(Colormap.Viridis)` for legible false colour. And transforms move the image, so `show(img.blur(5))` spends `img` and the rest of the cell has nothing left to work on. Display a `.copy` when you want to see an intermediate step and keep going.

The leak a notebook makes easy

A notebook is a long-lived JVM with a top-level scope you keep adding to. That combination is exactly the one the `Image` lifetime rules were designed against, and the failure mode is quiet.

Example D-3. A cell that leaks, and leaks again every time you run it.

```
val frame = Image.read("photo.jpg").toOption.get
val edges = frame.copy.gray.canny(80, 160)
edges.toBufferedImage
```

`.toOption.get` is the first sin and the visible one — Chapter 6 is about the `Left` it throws away. The second is invisible, and worse. `frame` and `edges` are top-level notebook bindings, so they live as long as the kernel does; the roughly forty bytes of Java header the collector sees for each `Mat` never come under enough pressure to trigger a collection, and the megabytes behind them are never freed. Re-run the cell to adjust a threshold and the previous `frame` and `edges` are rebound, not released — so a twenty-iteration tuning session has twenty generations of pixels resident. Chapter 1 measured what that costs at scale: 2000 unreleased `Mat(1000, 1000, CV_8UC3)` held 5 865 MB where releasing them held 144 MB.

> **NATIVE MEMORY**
> Notebook cells are the worst case for native memory, because every leak is permanent for the life of the kernel and re-running a cell multiplies it. Never bind an `Image` or a `Managed` at a cell's top level. Scope it, and let the last expression be a `BufferedImage`, an array, or a number — values the kernel can hold safely because they are on the heap.

The fix is the same one Chapter 5 gave for any other long-lived process, and it fits on one line more than the leaking version.

Example D-4. The same cell, owning nothing when it ends.

```
Image.reading("photo.jpg") { img =>
  val edges = img.gray.canny(80, 160)
  try show(edges)
  finally edges.close()
}
```

`Image.reading` releases the source when the block returns — on success, on failure, and on exception. The `try` / `finally` is there because `gray` and `canny` each move ownership into a **new** `Image`, and `reading` only knows about the one it opened; `edges` is a handle the cell created and so a handle the cell has to close. What escapes is a `BufferedImage`, which is heap memory the collector understands. Where a cell needs several native objects at once, `Managed.scope` does the same job for all of them:

Example D-5. Several handles, one scope, and only plain data escapes.

```
Managed.scope { own =>
  val marker = own.adopt(Aruco.generateMarker(ArucoDictionary.Dict4x4_50, id = 7,
sizePixels = 200))
  // marker is a borrowed org.opencv.core.Mat; the scope releases it when the
block returns.
  val bgr    = own.adopt(marker.cvtColor(ColorConversion.GrayToBgr))
  val sheet  = own.adopt(bgr.border(60, 60, 60, 60, BorderType.Constant,
Scalar.White))
  Images.encode(sheet, ".png") // Either[CvError, Array[Byte]] — heap bytes, safe
to bind
}
```

The rule `Managed.scope` carries is that nothing acquired inside may escape: the value the block returns must be plain data, or an object with a separate owner. In a notebook that rule is a gift, because "plain data" is exactly what a cell can safely keep — a byte array, a `Seq[Contour]`, a count, a `BufferedImage`.

Restarting the kernel is not a memory strategy

It is tempting to treat "restart the kernel" as the release. It works, in the sense that the process exits and the operating system reclaims everything — and it is precisely how people discover the problem, because the restart is what makes the machine usable again. The reason not to rely on it is that the discipline you practise in a notebook is the discipline you carry into the service you write afterwards, where there is no restart between frames. Scope every cell, and the frame loop in Chapter 20 will already look normal to you.

scala-cli, when a notebook is more machinery than you need

For a scratch script — one input, one output, no cells — scala-cli is lighter. The dependency declarations live in `using` directives at the top of the file, so the file is the build.

Example D-6. A complete scratch script, run with scala-cli run edges.scala.

```
//> using scala 3.3.8
//> using dep com.worxbend::scalacv:0.4.2
//> using dep org.bytedeco:opencv:4.13.0-1.5.13,classifier=linux-x86_64
//> using dep org.bytedeco:openblas:0.3.31-1.5.13,classifier=linux-x86_64

import scalacv.*

@main def edges(): Unit =
  OpenCv.load()
  val scene = Image
    .blank(320, 200, Scalar(30, 30, 30))
    .drawCircle(Point(150, 70), 30, Scalar.White, Thickness.Filled)
    .drawRect(Rect(40, 120, 80, 50), Scalar.White, Thickness.Filled)
  scene.gray.canny(80, 160).write("edges.png") match
    case Right(_)  => println("wrote edges.png")
    case Left(err) => println(s"failed: ${err.getMessage}")
```

That script reads no file, which is the point of the second half of this appendix.

Sample data you draw rather than download

Every tutorial input in this book can be generated, with three exceptions named at the end. A drawn input has exactly the properties you meant it to have, so when a step misbehaves you know whether the bug is in your code or in your picture — and you can change the picture to find out. Move two discs until they touch and watch the contour count fall from six to five: the merged-blob problem, reproduced deliberately, at the exact separation where it starts.

Write every generator as a `def`, never a `val`. Transforms consume the image they are called on, so a shared `val` scene is good for exactly one pipeline before it is spent. A `def` costs one allocation and removes a whole class of use-after-move error. Chapter 40 uses the same rule in the test suite for the same reason.

A scene with countable objects

`Image.blank` gives a canvas; the drawing verbs paint onto it and hand the image straight back, so a scene is one chain. Colours are BGR — `Scalar.Red` is `Scalar(0, 0, 255)` — and `Thickness.Filled` is what makes a shape solid enough to threshold.

Example D-7. A dark tray with five bright discs and one speck of noise.

```
def coinTray(): Image =
  Image
    .blank(320, 200, Scalar(30, 30, 30))
    .drawCircle(Point(60, 60), 25, Scalar.White, Thickness.Filled)
    .drawCircle(Point(150, 70), 30, Scalar.White, Thickness.Filled)
    .drawCircle(Point(240, 60), 20, Scalar.White, Thickness.Filled)
    .drawCircle(Point(90, 150), 22, Scalar.White, Thickness.Filled)
```

```
    .drawCircle(Point(200, 150), 28, Scalar.White, Thickness.Filled)
    .drawCircle(Point(280, 180), 2, Scalar.White, Thickness.Filled)
```

`coinTray().gray.threshold(128).contours()` returns six contours. Five of them measure between 1200 and 2740 units of `Contour.area`; the speck measures 8.0, and filtering it out on area is the exercise in Chapter 12.

A scene composed rather than chained

The drawing verbs mutate an image and consume it, which is why a generator has to be a `def`. The `Picture` scene graph from `scalacv-graphs` moves the reuse one level up: a `Picture` is an immutable description holding no native memory, so it may safely be the `val` that an `Image` may not, and each `render` builds its own fresh image from it.

Example D-8. One disc, described once and rendered three times into one tray.

```
/** A description, not pixels — no native memory, so a top-level binding is safe.
*/
val disc = Picture.circle(Point(0, 0), 24).fillColor(Color.Orange).noStroke

def tray(): Image =
  Picture
    .all(Seq(Point(60, 60), Point(150, 70), Point(240, 60)).map(disc.at))
    .render(320, 200, Color.DarkGray)
```

That tray is 320 × 200, three channels, and thresholds to three contours of area 1820 apiece — identical discs, because they are one description placed three times rather than three calls that might drift apart. `Picture.at` translates the shape's origin onto a point, `Picture.all` stacks a `Seq` of them, and `render(width, height, background)` is the only step that allocates. Chapter 16 is the scene graph in full; here the point is narrower — the "def, never `val`" rule is about pixels, not about the plan for them.

A chessboard for calibration

`ChessboardPattern(columns, rows)` counts **inner** corners, so a 9 × 6 pattern is a board of 10 × 7 squares. Draw one more square than corners in each direction and leave a white margin, or the corner finder has no background to work against.

Example D-9. A flat chessboard with a quiet zone.

```
def chessboard(columns: Int = 9, rows: Int = 6, square: Int = 60, margin: Int =
60): Image =
  val squaresX = columns + 1
  val squaresY = rows + 1
  val dark =
    for
      r <- 0 until squaresY
      c <- 0 until squaresX
```

```
      if (r + c) % 2 == 0
    yield Rect(margin + c * square, margin + r * square, square, square)
  val canvas =
    Image.blank(squaresX * square + 2 * margin, squaresY * square + 2 * margin,
Scalar.White)
  dark.foldLeft(canvas)((img, cell) => img.drawRect(cell, Scalar.Black,
Thickness.Filled))
```

With those defaults the canvas is 720 × 540, and `Calibration.findCorners(board, ChessboardPattern(columns = 9, rows = 6))` returns `Some` of 54 points — `columns × rows`, which `ChessboardPattern.corners` will tell you. The detector is all-or-nothing, so a `Some` means the whole board was found, never a subset. The `foldLeft` is there because `drawRect` consumes the image and returns a new one — that is the general shape of "draw N things" in scalacv.

> **WARNING**
> A drawn board calibrates nothing. `Calibration.fromChessboard` recovers focal length and distortion from how perspective changes between views, and every view of a flat drawn board is the same view. Use the drawn board to prove that corner detection and your pattern dimensions are right; use a printed board and a real camera for numbers you can trust. Chapter 31 has the procedure.

An ArUco marker sheet

Fiducial markers are generated rather than photographed, so there is no compromise here at all — you can make a pixel-perfect one and decode it straight back. `Aruco.generateMarker` returns the tag with its own black border and **no** quiet zone, and the detector hunts for a dark quadrilateral on a light background, so the padding is load-bearing.

Example D-10. One marker, padded so it can be found.

```
def markerSheet(id: Int, sizePixels: Int = 200): Image =
  Image
    .wrap(Aruco.generateMarker(ArucoDictionary.Dict4x4_50, id, sizePixels))
    .pad(60, color = Scalar.White)
```

`markerSheet(7)` is 320 × 320 once padded, and `sheet.arucoMarkers()` on it returns exactly one `ArucoMarker`, with `id == 7` and four corners. The extension defaults to `Dict4x4_50`, the dictionary the marker was generated from; ask for a different one and you get an empty `Seq` rather than a wrong answer. For a multi-marker sheet, generate several and paste them into a blank canvas; Chapter 28 covers what the detector does with them.

A document for OCR

The test suite's synthetic document is horizontal black bars on white — ink-shaped enough for the deskew step to find a dominant angle, with no font rendering involved.

Example D-11. A page of text-shaped bars, then a skewed scan of it.

```
def document(): Image =
  Image
    .blank(200, 140, Scalar.White)
    .drawRects(
      Seq(Rect(30, 20, 140, 8), Rect(30, 45, 140, 8), Rect(30, 70, 140, 8),
Rect(30, 95, 140, 8)),
      Scalar.Black,
      Thickness.Filled
    )

/** The same page as a tilted scan. Rotate on a WHITE background, as a real
scanner
  * would leave it: the default black fill reads as ink and drags the deskew
angle.
  */
def skewedDocument(degrees: Double = 12.0): Image =
  val doc = document()
  try Image.wrap(doc.mat.rotated(degrees, color = Scalar.White))
  finally doc.close()
```

`rotated` is the mid-level verb on the borrowed `Mat`, not a transform on the `Image`, which is why `doc` is closed by hand; it expands the canvas so no corner is clipped, so a 200 × 140 page comes back 225 × 179 at twelve degrees. Run `skewedDocument().forOcr()` and you get a single-channel binarised image — `channels == 1` — with the bars back near horizontal. What it will not give you is text: `Ocr.read` needs an `OcrEngine` you supply, and no engine will read a bar. Bars test the geometry; real glyphs test the engine. Chapter 33 draws that line.

A moving object for tracking and motion detection

A `Recorder` writes `Image`s, and the images can be drawn ones, so a video file is about ten lines. Use `Codec.Mjpg` with an `.avi` extension: Motion-JPEG is served by OpenCV's built-in writer, so it needs no FFmpeg, no GStreamer and no system codec, and it is `Recorder`'s default for that reason. The container is part of the deal — MJPG will not open inside an `.mp4`.

Example D-12. Fifty frames of honest motion.

```
Recorder.using("clip.avi", Size(320, 240), fps = 25.0, codec = Codec.Mjpg) { rec
=>
  (0 until 50).foreach { i =>
    val f = Image
      .blank(320, 240, Scalar(20, 20, 20))
      .drawCircle(Point(20 + i * 5, 120), 18, Scalar.Green, Thickness.Filled)
    try rec.write(f).fold(e => throw e, identity)
    finally f.close()
  }
}
```

`Recorder.using` closes the writer on the success path and on an exception — a `VideoWriter` that is never released leaves a truncated, unplayable file — and hands back an `Either[CvError, A]`, so the cell's value tells you whether the codec opened at all. `rec.write` **borrows** the frame, so the `try` / `finally` is yours to write; every frame is allocated and freed inside the loop, which is why a 50-frame clip and a 50 000-frame clip use the same memory.

Read it back with `Camera.usingFile("clip.avi")`, which closes the capture for you and returns your block's value in an `Either`. Count the frames with `cam.foreach(attemptsPerFrame = 1)`, which hands each frame over as an owned `Image` and closes it for you: `attemptsPerFrame` is a parameter of the frame-pulling methods, not of `usingFile`, and its default of 3 exists so a flaky live camera can drop a frame without ending the stream. On a finite file it only costs two extra blocking reads at the end.

A disc that moves five pixels a frame is exactly what a frame-difference detector is looking for (Chapter 21), and its bounding box is a clean seed for `Tracker.init` (Chapter 30).

A gradient and a noise field

Filters are easier to judge against a signal you can predict than against a scene. Neither a ramp nor a noise field is a drawing, so both are built by filling a byte array and handing it to a `Mat` — the same pattern `OccupancyGrid.toImage` uses, with the pixels allocated **before** the `Mat` so nothing can throw in the window where the buffer has no owner.

Example D-13. A horizontal ramp and a uniform noise field, both 8-bit single-channel.

```
import org.opencv.core.{CvType, Mat}

private def fromBytes(width: Int, height: Int, bytes: Array[Byte]): Image =
  val mat = Mat(height, width, CvType.CV_8UC1)
  val handle = Managed(mat)
  try
    mat.put(0, 0, bytes): Unit
    Image.wrap(handle)
  catch
    case e: Throwable =>
      handle.release()
      throw e

/** 0 on the left edge, 255 on the right. */
def ramp(width: Int = 256, height: Int = 64): Image =
  require(width > 1, "a ramp needs at least two columns")
  val bytes = new Array[Byte](width * height)
  for y <- 0 until height; x <- 0 until width do
    bytes(y * width + x) = (x * 255 / (width - 1)).toByte
  fromBytes(width, height, bytes)

/** Uniform 0–255 noise, reproducible from `seed`. */
def noise(width: Int = 256, height: Int = 256, seed: Long = 42L): Image =
  val bytes = new Array[Byte](width * height)
  java.util.Random(seed).nextBytes(bytes)
  fromBytes(width, height, bytes)
```

The ramp makes a `posterize` step count, a gamma curve and a `colorMap` legible at a glance: the output is a graph of the operation, not a picture of a scene. The noise field is the denoiser's test bench, and the seed is the point of it — `noise()` gives the same pixels on every run, so `medianBlur(1)` and `bilateralFilter()` are compared on identical input rather than on two fields that are merely similar. `Managed(mat)` picks up the `given Releasable[Mat]` from the companion, and `Image.wrap` takes ownership of that handle; Appendix A covers the wider `org.opencv.*` surface both of them drop into.

The inputs you have to supply yourself

Three things cannot be drawn, and pretending otherwise wastes a day.

- **A photographed face.** Haar cascades and YuNet both key on the light-and-dark structure of a photographed face — brows darker than forehead, the highlight on the nose bridge, skin texture. A cartoon has none of it. The detector will load and run on a drawn face and return some number, and that number tells you nothing either way. Chapter 24 assumes a real portrait.
- **A lens.** Distortion coefficients, vignetting and rolling shutter come from optics, not from rasterisation. Chapter 31's undistortion is a no-op on a drawn frame.
- **Real degradation.** Sensor noise, motion blur, JPEG artefacts and uneven lighting are what production thresholds are actually tuned against. Drawn inputs prove your logic; real captures tell you what your numbers need to be.

Real datasets, and the licence attached to each of them

When you do need photographs, take the licence as seriously as the pixels, and keep the files **outside** your repository — the same reasoning that keeps them out of scalacv's.

Table D-2. Where to get images you are allowed to use.

Source	What to check before you commit to it
Wikimedia Commons	Every file page states its licence. Filter for public domain or CC0; CC BY-SA files oblige you to share alike, which reaches your derived outputs.
Unsplash	The Unsplash licence permits free use, commercial included, with no attribution required. It does not permit redistributing the photos as a competing stock library.
Your own camera	No licence question at all, and the only source that matches your actual optics and lighting. This is what a calibration set has to be.

> **CAUTION**
> An academic vision dataset is not automatically usable. Several of the best-known face and pedestrian sets are licensed for non-commercial research only, some require a signed agreement, and a few have been withdrawn over consent. Read the terms before a dataset reaches a build that ships, and record which one you used — a model trained on restricted data inherits the restriction.

What good output looks like

Every value below was produced by running the generators exactly as printed above, against this repository's `core` and `vision`.

Table D-3. Expected results from the generators in this appendix.

Input	Check	Expected
`coinTray()`	`gray.threshold(128).contours()`	`6` contours; five of area 1200–2740, one of 8.0
`tray()`	`gray.threshold(128).contours()`	`3` contours, area 1820 apiece
`chessboard()`	`Calibration.findCorners(…, ChessboardPattern(9, 6))`	`Some(54)` on a 720 × 540 canvas
`markerSheet(7)`	`arucoMarkers()` on a 320 × 320 sheet	one marker, `id == 7`, 4 corners
`document()`	`forOcr().channels`	`1`, still 200 × 140
`clip.avi`	frames counted inside `Camera.usingFile`	`Right(50)`, each 320 × 240
`ramp()`	first and last column of row 0	`0` and `255`, one channel

If a number here does not match on your machine, check the input before you touch the algorithm: print the generator's size and channel count first, and only then suspect the operation.

Where this leaves you

You now have a place to type code that will not leak between cells, and a way to manufacture every input this book has asked for except the three that genuinely require a camera. That is the last of the practical apparatus. Appendix E, the Glossary, closes the book with the vocabulary — owned, borrowed, move semantics, spent, query/transform/terminal, classifier — that these forty-two chapters and five appendices have been using in a specific and deliberate sense.

APPENDIX E

Glossary

Every word this book leans on, defined for this library rather than for the field.

Three vocabularies collide in a scalacv program. OpenCV's terms carry twenty years of C++ assumptions — that a `Mat` is a value you copy, that a detector is an object you construct and forget. The JVM's terms come from a runtime that does not know native memory exists. This library's terms bridge the two, and they are the ones worth looking up, because they mean something narrower here than anywhere else: *release*, *close* and *spend* are not synonyms, though any other wrapper would tell you they are, and *owned* versus *borrowed* is the difference between a process that runs for a week and one that is OOM-killed in six minutes. OpenCV is not innocent either — *blob* names two unrelated things inside the same library.

Every entry is correct for the code in this repository, not merely true of the field. Where a term names something scalacv deliberately does not wrap — `HoughCircles`, `findHomography`, a perspective warp — the entry says so: knowing what is absent saves more time than knowing what is present, and Appendix A shows how to reach it anyway.

Table E-1. The five ownership words, and what each one obliges you to do.

Word	Who frees it	What you must do
owned	you	reach a terminal, `close()`, or use `Managed.use`
borrowed	somebody else	never close it, never outlive the owner
copied-out	nobody — it is plain data	keep it as long as you like
spent	the transform that consumed it	never touch it again; a second use throws
scoped	the enclosing `Managed.scope`	let nothing acquired inside escape

Three verbs that are not synonyms

Release is what `Managed.release()` does: a `getAndSet(null)` that admits exactly one caller, so a second call is a no-op rather than a double free. *Close* is the `AutoCloseable` method `Image`, `Descriptors`, `Kalman` and `ObjectTracker` expose so `Using` can drive them; it delegates to a release. *Spend* is what an `Image` transform does to its receiver — the handle is emptied and the old value throws `IllegalStateException` from then on. A spent image has been closed; a closed image was not necessarily spent by anything.

A

affine transform — A 2×3 mapping under which parallel lines stay parallel — rotation, scale, shear, translation. `Image.rotate(degrees)` is the wrapped case; the general `warpAffine` with a matrix from `getRotationMatrix2D` goes through the escape hatch. Chapter 10.

alpha channel — A fourth channel carrying transparency, 0 clear to 255 opaque. `ColorConversion.BgrToBgra` adds one, `BgraToBgr` drops it. Most operations expect three channels and are not polite about four. Chapter 8.

ArUco — A square fiducial marker — a black border around a bit grid encoding an integer id. `ArucoDictionary` names grid size and family size (`Dict4x4_50` upward); fewer markers means a larger Hamming distance between them and more robust decoding. Chapter 28.

aspect ratio — Width over height of a box, and the cheapest shape filter there is: a licence plate is roughly 4, a face roughly 1. `Dnn.blobFromImage` changes it when you resize both axes, which is why letterboxing exists. Chapter 12.

B

backpressure — What a consumer exerts on a producer it cannot keep up with. A camera does not care that your detector is slow, so the pipeline must choose explicitly between buffering, dropping and blocking — and buffered frames are buffered native memory. Chapter 22.

baseline — The line text glyphs sit on, and the anchor `drawText` positions from. `TextMetrics(size, baseline)` reports how far descenders fall below it, so a background box must be `size.height + baseline` tall; forgetting it clips every `g` and `y`. Chapter 14.

BGR — OpenCV's channel order — blue, green, red — and not RGB. Every `Scalar`, every pixel you read by hand, every buffer you hand to another library. Chapter 8.

bilateral filter — An edge-preserving smoother that weights neighbours by colour difference as well as distance. `bilateralFilter(diameter = 9, sigmaColor = 75, sigmaSpace = 75)`, and markedly slower than a Gaussian. Chapter 9.

binary image — A single-channel image whose pixels are 0 or 255, produced by a threshold or a colour mask. Contours, morphology and `findNonZero` all want one. Chapter 11.

blob (image) — A connected region of foreground pixels — the white patch a threshold leaves and the thing `contours()` outlines. This is the sense the book uses everywhere but one chapter. Chapter 12.

blob (DNN) — The unrelated other sense: the 4-dimensional NCHW tensor a network takes as input, from `Dnn.blobFromImage` and caller-owned. `Size(width, height)` is the argument order while the blob's trailing dimensions are height then width. Chapter 26.

border mode — How a filter or warp invents pixels past the edge. `BorderType` names `Constant`, `Replicate`, `Reflect`, `Reflect101` and `Wrap`; `Wrap` is valid only for `pad` / `border` (`copyMakeBorder`) and a rotation by degrees (`warpAffine`), so `BorderType.requireFilterSupport` rejects it elsewhere rather than letting a filter leave the destination uninitialised. Chapter 8.

borrowed — A handle somebody else owns and will free. `Image.mat` borrows; a `mask` argument is borrowed; the image an `OcrEngine` receives is borrowed. Closing one is how you cause a double free in code that reads as tidy resource management. Chapter 5.

C

Canny — The classic edge detector. `canny(threshold1, threshold2, apertureSize = 3, l2Gradient = false)` returns a single-channel edge map, wants greyscale input, and is the usual stage before a Hough transform. Chapter 9.

cascade classifier — A classical detector defined by an XML file of Haar features, extracted from the bytedeco classifier jar by `Cascades.resolve` and named by `CascadeName` rather than a path. `CascadeClassifier` does not throw for a file it cannot read: it returns an empty classifier that detects nothing forever, so `Cascades.load` checks `empty()` and returns a `Left`. Chapter 24.

centroid — A shape's centre of mass, from its image moments. `Contour.centroid` is an `Option[Point]` and answers `None` when `m00` is zero — an empty or collinear contour — rather than dividing by it. Chapter 12.

channel — One of the numbers a pixel carries: one is greyscale, three colour, four colour plus alpha. Chapter 4.

classifier — Anything answering "which of these is it?" for a region. Distinct from a detector, which must also answer "where?". Chapter 27.

colour space — The meaning assigned to a pixel's channels. `ColorConversion` names the conversions scalacv wraps — grey, RGB, HSV, Lab, BGRA and back — and choosing the right one is usually cheaper than choosing a cleverer algorithm. Chapter 8.

confidence — A model's own estimate of how right it is, always a number to threshold rather than trust. `Face.score` lies in `[0, 1]` and only faces at or above `scoreThreshold` (default `0.9f`) are reported; `OcrResult.confident(minConfidence = 0.5f)` filters words. Chapter 24.

contour — One connected outline as a sequence of points. `Contour` is plain immutable data: `findContours` copies the points out and frees the native `MatOfPoint` list before returning — that list is the most reliable leak in the OpenCV Java API — so a contour stays valid after its source Mat is gone. Chapter 12.

convolution — Sliding a kernel over an image and replacing each pixel with a weighted sum of its neighbourhood. Blur, sharpen and Sobel are this operation with different weights. Chapter 9.

`CvError` — The sealed hierarchy for everything scalacv can fail with. `DecodeFailed`, `LoadFailed`, `EncodeFailed`, `CalibrationFailed` and `NativeCall` come back as a `Left` where failure is data-dependent and expected; `NativesMissing` is always thrown, because absent native libraries are a build problem no data can cause. It extends `RuntimeException` rather than being a pure ADT, because `CvException` escapes from ordinary calls and no wrapper can make the core total. Chapter 6.

`Cv.attempt` — The wrapper turning a native throw into `Left(CvError.NativeCall(operation, cause))`, naming the operation and preserving OpenCV's message verbatim rather than parsing it for codes. `Cv.orThrow` is the same call where failure is a bug. Chapter 6.

D

descriptor — A fixed-length encoding of the appearance around a keypoint, so the same corner is recognisable in another frame. ORB's are binary, compared by Hamming distance. `Descriptors` owns a native Mat and is caller-owned. Chapter 32.

dilate — Grow the bright regions of a binary image by a structuring element, closing small gaps. `dilate(radius = 1, shape = MorphShape.Rect)`. Chapter 9.

disparity — How far a pixel shifts between the left and right images of a rectified stereo pair — inverse to distance. `StereoDepth.disparity(left, right, numDisparities = 64, blockSize = 9)` returns it 8-bit and normalised so brighter is nearer. Chapter 32.

distortion coefficients — The radial and tangential terms `k1, k2, p1, p2` (optionally extended) describing how a real lens bends straight lines. `Intrinsics.distortion` accepts only the counts OpenCV does — 0, 4, 5, 8, 12 or 14 — because any other length is still a legal `Seq` and yields a silently wrong undistort. Chapter 31.

dnn — OpenCV's deep-learning module, which runs pre-trained networks but does not train them. `Dnn.blobFromImage`, `Dnn.forward`, and a `Net` whose native memory the Java binding will not free. Chapter 26.

E

embedding — A face's identity as a fixed-length vector — 128 dimensions from SFace. `FaceEmbedding.cosineSimilarity` compares by angle (higher is more alike), `l2Distance` by magnitude, where about 1.13 and below is the same person. Chapter 25.

erode — Shrink the bright regions of a binary image by a structuring element, removing speckle. `erode(radius = 1, shape = MorphShape.Rect)`. Chapter 9.

error budget — A number agreed in advance for how much failure is acceptable — 1% of frames dropped in an hour — so a degraded pipeline can be judged rather than argued about. It is what turns "it feels slow" into a comparison. Chapter 39.

extrinsics — Where the camera is, rather than what it is: a rotation and a translation relating the camera to a world or object frame — exactly what `Pose3D(rvec, tvec)` carries. Intrinsics are measured once; extrinsics change every frame. Chapter 28.

F

feature — A distinctive, repeatably-findable spot such as a corner. `OpticalFlow.goodFeatures` picks ones worth tracking, `Features.detect` ones worth describing. Chapter 32.

fiducial marker — A pattern printed specifically to be found and identified — an ArUco tag, a QR code. Its point is to make pose recovery a solved problem rather than a research problem. Chapter 28.

FourCC — A four-character codec name packed into an `Int`. `Codec` spells the four scalacv exposes — `Mp4v`, `Avc1`, `Mjpg`, `Xvid` — and `Mjpg` in an `.avi` is the default because it is the one combination videoio's built-in writer serves on every build. Chapter 19.

G

Gaussian blur — Smoothing by a bell-shaped kernel. `Image.blur(radius)` takes a radius in pixels; the mid-level `gaussianBlur(Size(w, h))` takes the kernel directly. Chapter 9.

GIF — The animated format `Animation.gif` writes, through `Imgcodecs.imwriteanimation` — 256 colours per frame, dithered by OpenCV to fit. Full colour or a long clip wants `Animation.record` and a video codec instead. Chapter 18.

greyscale — A one-channel image, each pixel a single intensity. Corner detection, optical flow, stereo matching, ORB and every threshold want it, which is why `gray` opens so many pipelines. Chapter 4.

ground truth — The answer you already know, used to judge the answer you computed. This library's calibration and face tests synthesise inputs from a known camera so assertions compare against truth rather than against a previous run. Chapter 40.

H

Haar feature — The difference between summed intensities of adjacent rectangles — the primitive a cascade classifier is built from, and cheap only because of the integral image. Chapter 24.

Hamming distance — The number of differing bits between two binary descriptors; smaller is better. `Features.matches` uses a brute-force Hamming matcher with cross-check and keeps matches within `maxDistance = 64f`. Chapter 32.

headless — Running with no display or GUI toolkit — the normal state of a server. `OpenCv.load()` brings javacpp up through a GUI-free preset and resolves the JNI shim's dependencies on demand, so nothing needs installing with a package manager. Chapter 2.

homography — The 3×3 plane-to-plane transform, eight degrees of freedom and so four point correspondences. scalacv wraps neither `findHomography` nor `warpPerspective`; marker pose goes through `solvePnP` instead. Chapter 10.

Hough transform — Detecting shapes by having every edge pixel vote in a parameter space. `houghLines` returns infinite lines as `PolarLine`s, `houghLinesP` finite `Segment`s with real endpoints. `HoughCircles` is deliberately unwrapped — measure circularity on contours instead. Chapter 13.

HSV — Hue, saturation, value. A hue threshold survives lighting changes that destroy an RGB one, which is why colour segmentation converts first. Chapter 11.

hysteresis — Canny's two-threshold rule — a pixel above `threshold2` starts an edge, one above `threshold1` continues an edge already started. One threshold gives you either broken edges or noise. Chapter 9.

I

integral image — A table where each entry is the sum of every pixel above and left of it, so any rectangle's sum costs four lookups whatever its size. It is what makes a Haar cascade fast. Chapter 24.

interpolation — How a resize or warp invents values between source samples. `Interpolation` names `Nearest`, `Linear` (the default), `Cubic`, `Area` and `Lanczos4`; use `Area` when shrinking, and `Nearest` for anything whose values are labels. Chapter 10.

intrinsics — The camera's internal geometry: focal lengths `fx`/`fy` in pixels, principal point `cx`/`cy`, lens distortion. `Calibration` produces an `Intrinsics`; `Intrinsics.approx` guesses one from image size and field of view — enough to watch an overlay track, not to measure with. Chapter 31.

IoU — Intersection over union, two boxes' overlap in `[0, 1]`. It is the threshold in non-maximum suppression (`nmsThreshold = 0.3f` for YuNet) and the association metric in `ObjectTracker` (`iouThreshold`). Chapter 30.

J

JavaCPP — The bytedeco machinery shipping OpenCV's natives in per-platform classifier jars and extracting them on first use into `~/.javacpp` — about 196 MB on Linux. Redirect it with `-Dorg.bytedeco.javacpp.cachedir=…`. Chapter 2.

JNI — The Java Native Interface, the boundary every OpenCV call crosses, and where a use-after-free stops being an exception and becomes a SIGSEGV with no stack trace. That is why `Managed` checks on the Scala side of it. Chapter 5.

K

Kalman filter — A predict-and-correct estimator that smooths a noisy measurement and carries on through gaps. `Kalman.point` builds a constant-velocity filter over a 2D point; it owns native memory, is caller-owned, and must not be shared across threads. Chapter 30.

kernel — The small grid of weights a convolution slides over the image, or the shape a morphological operation uses. Blur radii and `MorphShape` are both kernel choices. Chapter 9.

keypoint — The location half of a feature. `Descriptors.points` gives them as a `Seq[Point]`; the descriptor half stays in native memory until you close it. Chapter 32.

L

landmark — A named point on a detected object. A YuNet `Face` has exactly five, always in the order right eye, left eye, nose tip, right mouth corner, left mouth corner — "right" meaning the subject's, so it appears on the image's left. A hand model has 21. Chapter 29.

leak — A native object nothing will ever free. It is not a JVM leak — the forty-byte on-heap object is collectable, and usually collected — which is why it shows in resident set size and never in a heap dump. Chapters 5 and 35.

M

`Managed[A]` — The library's one ownership primitive: a handle releasing its native object exactly once, which throws `IllegalStateException` in Scala — before anything reaches JNI — if used after release. Release is a compare-and-set, so sixty-four threads racing produce exactly one free. Chapter 5.

`Managed.scope` — The form for an operation needing several native objects at once. Each is registered as it is created, so a throw anywhere releases everything acquired so far, in reverse order. `own(mat)` hands the object straight back; `own.adopt(handle)` takes over an existing `Managed`. Nothing acquired inside may escape. Chapter 5.

mask — A single-channel image restricting where an operation applies — non-zero means "here". Masks are borrowed, never consumed. Chapter 11.

`Mat` — OpenCV's native matrix, the off-heap pixel grid behind about forty bytes of on-heap object. One of only three `org.opencv.*` types with a public `release()`, which drops the multi-megabyte buffer at once. Chapter 4.

moment — A weighted sum over a shape's pixels; the low-order ones give area (`m00`) and centre of mass (`m10/m00`, `m01/m00`). `Imgproc.moments` returns plain data with no native handle, so there is nothing to free. Chapter 12.

morphology — Shape operations on binary images built from erode and dilate. `MorphOp` names `Open` (removes speckle), `Close` (fills holes), `Gradient`, `TopHat` and `BlackHat`. Chapter 9.

move semantics — The rule that an `Image` transform consumes its receiver and returns a new image, so a chain of any length holds one live Mat rather than a pile of intermediates. To branch, take a `copy` first. Chapter 4.

N

native memory — Memory allocated by C++, off the JVM heap, where every pixel buffer lives. The collector runs on heap pressure, which is uncorrelated with native pressure: measured here, 2000 unreleased 1000×1000 three-channel Mats reach 5 865 MB of RSS against 144 MB when released. Chapter 5.

NMS — Non-maximum suppression — keeping the strongest of a cluster of overlapping detections and dropping the rest by IoU. A detector parameter (`nmsThreshold`) for YuNet, and a hand-rolled loop in `Screen.findAll`, which paints each hit's footprint out of the score map. Chapter 24.

O

ONNX — An open, portable model format, and the usual way a trained network reaches scalacv. Weights are never shipped in the jars: `Models` fetches and verifies them at run time, so the licence obligation stays with whoever chose to redistribute them. Chapter 23.

optical flow — How pixels or tracked features move between consecutive frames. `OpticalFlow.goodFeatures` seeds points and `OpticalFlow.track` follows them — the primitive under both motion analysis and odometry. Chapter 32.

ORB — A fast, rotation-aware keypoint detector with binary descriptors and no patent encumbrance. `Features.detect(image, maxFeatures = 500)`. Chapter 32.

Otsu's method — Choosing a threshold automatically by maximising the separation between two intensity populations. `Threshold.otsu(mode)` asks for it; the mid-level `threshold` returns the chosen value beside the image as a `ThresholdResult`, which `Image.threshold` drops. Take the pair when a bimodal assumption might stop holding — that number is the one worth logging. Chapter 11.

owned — A handle you are responsible for: every `Managed` you construct, every `Image` you hold, every detector `create` returns, the `Managed[Mat]` `Dnn.forward` hands back. One that escapes without reaching a terminal is a leak. Chapter 5.

P

perspective transform — A warp with vanishing points, where parallel lines converge — four correspondences rather than three. `Image` does not wrap it; `getPerspectiveTransform` and `warpPerspective` go through the raw-Mat escape hatch, inside a `Managed.scope`. Chapter 10 and Appendix A.

`Picture` — The immutable scene graph the graphics layer draws: shapes, text and groups composed as plain data, rendered in one pass by `picture.render(width, height)` or

`renderOn(image)`. Nothing native exists until that render, so a scene is built, stored and compared like any other value. Chapter 16.

pinhole model — The camera model everything 3D here assumes — rays through a single centre, described by `Intrinsics`, with lens distortion bolted on as a correction. Chapter 31.

pose — A position and orientation together. `Pose3D` carries `rvec` in Rodrigues axis-angle form and `tvec` as translation, both three elements in the marker's units, with `distance` the length of `tvec`. The word also names a human skeleton's keypoints, which is a different thing. Chapters 28 and 29.

principal point — Where the optical axis meets the sensor — `cx` / `cy` in `Intrinsics`, near but not exactly at the image centre. `Intrinsics.approx` assumes exactly central. Chapter 31.

Q

query, transform, terminal — The three kinds of `Image` method. A query (`width`, `faces`, `contours`) borrows and leaves the image alive; a transform (`gray`, `blur`, `crop`, a `draw*`) spends the receiver and returns a new image; a terminal (`write`, `bytes`, `close`) consumes and releases. Knowing which you called is the whole high-level memory model. Chapter 4.

R

RANSAC — Random sample consensus — fit a model to a minimal random subset, count the points that agree, repeat, keep the best. It is how `findEssentialMat` survives the wrong matches every real matcher produces. Chapter 32.

`Releasable[A]` — The type class answering *how* a native object is freed, where `Managed` answers *when*. `Mat`, `VideoCapture` and `VideoWriter` have a public `release()`; the other 185 `org.opencv.*` types that own native memory expose only a private `delete(long)`, reached through `Releasable.nativeHandle`, which throws loudly rather than degrading into a silent leak. Chapter 5.

reprojection error — A calibration's headline quality number: the root-mean-square distance in pixels between where the solver predicts each chessboard corner should land and where it was found. `Calibration.reprojectionError`. Chapter 31.

resident set size — The physical memory the kernel says the process is using, and the only reliable signal for a native leak — `Pointer.totalBytes()` cannot see OpenCV's own allocations and the heap dump looks innocent. Chapter 35.

RGB — The channel order everything except OpenCV uses. `ColorConversion.BgrToRgb` is the conversion you owe any library you hand pixels to. Chapter 8.

ROI — Region of interest, a rectangular sub-area to work on. `Image.crop(rect)` rejects a rectangle running outside the frame, which is why `Face.clippedBox` exists. Chapter 10.

S

SGBM — Semi-global block matching, the stereo correspondence algorithm `StereoDepth.disparity` drives. `numDisparities` must be a positive multiple of 16 and `blockSize` an odd number of at least 3. Chapter 32.

SLAM — Simultaneous localisation and mapping — working out where the camera is while building the map it is located in. scalacv provides the front end (features, flow, odometry, loop detection); the global optimisation lives in a back end beyond OpenCV. Chapter 32.

SORT — The tracking-by-detection pattern `ObjectTracker` implements in reduced form: predict every track with its Kalman filter, associate detections greedily by IoU, correct the matches, spawn tracks for the unmatched, retire tracks unseen for `maxAge` frames. It never looks at the image, only at the boxes. Chapter 30.

spent — The state of an `Image` after a transform or terminal consumed it. Using it again throws `IllegalStateException` rather than reading freed memory, and `-Dscalacv.trackOwnership=true` makes the exception name the call that spent it. Chapter 4.

structuring element — The shape a morphological operation uses — `MorphShape.Rect`, `Ellipse` or `Cross`, at a given radius. `Ellipse` is the honest choice for round objects; `Rect` is faster and is the default. Chapter 9.

T

template matching — Sliding a known sub-image over a larger one and scoring correlation everywhere. `Screen.locate(image, template, minScore = 0.8)` returns the best hit; the score is a normalised correlation in `[-1, 1]`. No model, no training, and no tolerance for a change of scale. Chapter 34.

threshold — Turning a greyscale image binary by a cutoff. `Threshold` pairs a `Mode` with an optional `Auto` (`Otsu` or `Triangle`), because Otsu is a modifier on a mode rather than a mode of its own; `adaptiveThreshold` computes a cutoff per neighbourhood, which is what uneven lighting needs. Chapter 11.

tracking-by-detection — Running a detector every frame and stitching the boxes into identities afterwards, rather than tracking appearance. Detector-agnostic, recovers from occlusion as soon as the detector does, and costs a detection per frame. Chapter 30.

V

visual odometry — Estimating the camera's own motion from what it sees. `VisualOdometry` recovers one step from point correspondences via the essential matrix and `recoverPose`; `Odometry` chains the steps, which is dead reckoning and therefore drifts. Chapter 32.

Y

YuNet — The small convolutional face detector OpenCV supports — 232,589 bytes of weights, documented as both far faster than a Haar cascade and markedly more accurate. `FaceDetect.create(modelPath, inputSize, scoreThreshold = 0.9f, nmsThreshold = 0.3f)` returns `Either[CvError, Managed[FaceDetectorYN]]`, and the weights are a runtime download. Chapter 24.

Z

ZIO `Scope` — What the `zio` module uses in place of `Managed.use`. `acquireRelease`, `imageScoped` and `captureScoped` tie a native object to the enclosing `Scope`, so the runtime frees it on success, on failure and on interruption alike; `frameStream` hands out a frame valid only until the next pull. Chapter 37.

Where to look next

A word that is not here is most likely an operation or a constant rather than a concept, and those have their own indexes: Appendix B, *Operations Reference*, lists every verb with its signature, and Appendix C, *Enums and Types Reference*, every typed constant with the `org.opencv.*` integer behind it. When the word came out of an error message rather than a chapter, Chapter 42, *Troubleshooting*, is organised by the message you actually saw.